PREPNPLACED

100 SQL Interview Questions

for product-based company interviews

Easy to Expert | PostgreSQL 16 | Real outputs

Every query run on PrepNPlaced Commerce Dataset v2: 46,438 orders, 168,567 order lines.

25	35	25	15
Easy	Medium	Hard	Expert
Q1 to Q25	Q26 to Q60	Q61 to Q85	Q86 to Q100

Every question walks you through

- Business scenario
- Tables and relationships
- Final SQL solution
- Real output from the dataset
- Common mistakes
- Optimisation or alternative
- Problem and expected output
- Query logic, step by step
- Line-by-line explanation
- Why it works
- Interview follow-up

Durgesh Yadav
Founder, PrepNPlaced | prepnplaced.com

October 2026
266 SQL blocks, all executed

Contents

Tap a title in your PDF reader's bookmarks panel to jump straight to it.

Deliverable A: Dataset assessment 4

Deliverable B: Master question index 6

Deliverable C: Complete question bank 11

Easy: Questions 1 to 25 11

1 Brand directory 11
2 Ten highest-priced products 12
3 Customers in metro cities 14
4 Customer emails that cannot be delivered 15
5 Orders placed in Q1 2025 16
6 Most expensive products that are never disco... 17
7 Discount amount and percentage per product 19
8 Price band labels 20
9 Check how Active and Allowed are encoded 21
10 Headline order KPIs 23
11 How much of the catalogue has ever sold? 24
12 Price and discount statistics per category ID 25
13 States with at least five customers 27
14 Orders per month in 2025 28
15 Product count for every category 29
16 December 2025 orders with customer names 30
17 Orders with payment type and shipper 32
18 Customers with no order in the last 90 days 33
19 Units sold per product 34
20 Days to ship and days to deliver 35
21 Top five customers by number of orders 37
22 Orders with more than eight line items 38
23 Chocolate or cookie products under 250, excl... 39
24 Suppliers outside India or with no email 40
25 Customer sign-ups by year and quarter 42

Medium: Questions 26 to 60 44

26 List-price value by category 44
27 Average order value by customer state 45
28 Payment mix: share of orders and value 47
29 Order value and units by shipper without dou... 49
30 Repeat versus one-time customers 51
31 Customers who spend more than the averag... 52
32 Products priced above their category average 54
33 Products that have never been ordered 56
34 Each supplier's share of units supplied 58
35 Late-delivery rate by shipper 59
36 Payment method mix by month as columns 61
37 Prove no order was paid with a method that i... 63
38 Products from inactive categories that still so... 65
39 Categories with no products, and categories... 67
40 Days from sign-up to first order 68
41 Most expensive product in each category 70
42 RANK vs DENSE_RANK vs ROW_NUMBER on ord... 72
43 Month-to-date order value for the latest thre... 74
44 Month-over-month growth in order value 76
45 Typical gap between a customer's orders 77
46 Same-brand products where one costs at le... 79
47 Brands with at least three products, and how... 81
48 Do discounted products sell more? 83
49 Top three products by units in each category 85
50 Customers who buy across at least ten categ... 86
51 Orders fulfilled by one supplier vs several 88
52 Where shipping time goes: dispatch vs transi... 90
53 Thirty-day sign-up to purchase conversion b... 91
54 Weighted vs simple average discount by sub... 93
55 Category list-price value: latest year vs previ... 95
56 Customers who ordered last year but not this... 97
57 One contact directory for suppliers and shipp... 99
58 Each customer's latest order 100
59 Basket size by month 102
60 Sub-category share of its category 104

Hard: Questions 61 to 85 106

61 Monthly cohort retention 106
62 Which products make up 80% of list-price val... 109
63 RFM segmentation 111
64 Three-month moving average of order value,... 113
65 Longest run of consecutive months with an or... 116
66 How fast do customers come back for a seco... 118
67 Median vs mean order value by payment type 121
68 Second-highest spender in each state, count... 123
69 Customers whose spend grew every active q... 125
70 Sub-categories most often bought together 128
71 Deduplicate customer accounts by email 130
72 Same product on several lines of one order: s... 132
73 Category share of list-price value: latest vs pr... 134
74 Customers whose latest order is below their o... 136
75 Rolling 28-day active customers, week by we... 139
76 Order value from new vs returning customers... 141
77 Single-source products and the value at risk 144
78 Expensive within their sub-category but sellin... 146
79 At which order does a customer cross 10,000 i... 148
80 Sub-category repurchase rate within 90 days 151
81 Year-to-date vs same period last year 153
82 Each customer's favourite category 156
83 Underperforming sub-categories in the last s... 158
84 Delivery time by state and shipper with subto... 160
85 Win-back customers returning after 180+ days 162

Expert: Questions 86 to 100 166

86 Reconcile order headers with order lines 166
87 Referential integrity audit across all seven rel... 169
88 Date validity audit 171
89 Incremental load with a watermark and look... 175
90 Column-by-column completeness profile of... 178

Contents (continued)

Tap a title in your PDF reader's bookmarks panel to jump straight to it.

91 Cumulative value per customer by cohort (LT… 180
92 How duplicate accounts distort the repeat-c… 184
93 Possible duplicate order submissions 186
94 Supplier concentration per category (HHI) 189
95 Debug a revenue-by-category query that giv… 193
96 Make a slow order report sargable and index it 196
97 What do customers buy on their second orde… 199
98 Category association rules: support, confiden… 203
99 Flag unusual weeks in order volume 206
100 Monthly KPI scorecard with month-over-mon… 210

Deliverable D: Topic coverage matrix 214

Deliverable E: Final quality audit 217

Deliverable A: Dataset assessment

Every question in this book was executed on PostgreSQL 16 against the PrepNPlaced Commerce Dataset v2, loaded with `load_dataset_v2.sql`. Each Step 6 table is the actual output of the query above it.

Where the data comes from

- **Products are real.** The 22,088 products are the real catalogue from your Customer Dataset: real product names, brands, sub-categories, sale prices and printed prices of an Indian online grocery and household store.
- **Everything else is generated, on purpose.** Real customer orders are not public anywhere, so customers, orders, order lines, suppliers, shippers and product ratings were generated to behave like a growing Indian e-commerce business. Supplier and courier names are fictional. Ratings are synthetic and say nothing about the real brands.
- **The problems are planted.** The data carries the faults real SQL work runs into, from NULLs and orphan keys to duplicate accounts and mixed date formats. Instructors get the full list, with exact counts, in `instructor/planted_issues.md`. Learners should find them with the data-quality questions first.

Tables and size

Table	Rows	What a row is	Columns
Suppliers	18	One supplier	SupplierID, CompanyName, Address, City, State, PostalCode, Country, Phone, Email
Products	22,088	One catalogue product	ProductID, Product, CategoryID, Sub_Category, Brand, Sale_Price, Market_Price, Type, Rating
Category	12	One category	CategoryID, CategoryName, Active
Shippers	10	One courier	ShipperID, CompanyName, Phone
Payments	10	One payment method	PaymentID, PaymentType, Allowed
Customers	5,072	One registered account	CustomerID, FirstName, LastName, City, State, Country, PostalCode, Phone, Email, DateEntered
Orders	46,438	One order header	OrderID, CustomerID, PaymentID, OrderDate, ShipperID, Total_Order_Amount, ShipDate, DeliveryDate
OrderDetails	168,567	One product line in an order	OrderDetailID, OrderID, ProductID, Quantity, SupplierID

The columns match the schema diagram exactly, including `Products.Rating` and `Suppliers.Address`. No foreign keys are enforced, as in most raw loads, so orphan rows can exist and some do.

What the business looks like

- **Period:** orders from 1 Jan 2023 to 28 Dec 2025. The extract was taken on 28 December, so December 2025 is a partial month and the last orders are still in transit.
- **Growth:** about 740 orders in the first quarter of 2023 and more than 10,000 in the last quarter of 2025, with festive peaks every October and November and sale-week bumps in January and August.
- **Orders:** average order value about ₹1,320, median ₹806, about 3.6 lines per order. Most lines are one or two units; a few are bulk buys.
- **Customers:** 5,072 accounts, mostly in Indian cities (Bengaluru, Mumbai, Delhi, Hyderabad and Chennai lead), some in the UAE, USA, UK and Singapore. 647 registered but never ordered. Ordering habits range from one-time buyers to a few hundred customers with more than 30 orders.
- **Payments:** UPI grows from about 42% of orders in 2023 to about 52% in 2025. Cash on Delivery shrinks. Pay Later was switched off on 1 July 2025.
- **Delivery:** median 4 days from order to delivery, 9 days at the 90th percentile. Couriers differ a lot, from same-day hyperlocal to slow long-haul freight.
- **Catalogue use:** 16,524 products have sold at least once; 5,564 never have. Popular products have hundreds of buyers, so product-level analysis works.
- **Order totals:** for 89% of orders `Total_Order_Amount` equals `SUM(Quantity * Sale_Price)` exactly. Most of the rest sit on a payment-linked discount tier (5% or 10% on cards, 12% or 15% on wallets). The schema has no discount column; Q86 shows how to find the rule from the data.

Data-quality problems you will meet

Area	What you will find	Question that deals with it
Flags	`Active` and `Allowed` stored as Yes, yes, 'Yes ', Y, 1, No, 0, N	Q9, Q37, then every flag test
Text	City variants (Bengaluru / Bangalore / BENGALURU, Gurugram / Gurgaon, Delhi / New Delhi), trailing spaces, mixed phone formats, country spelled India / INDIA / 'india '	Q3, Q24
Missing values	NULL payment, shipper, amount, quantity, sale price, rating, brand and sign-up date	Q90, Q10
Broken keys	Orders from deleted customers, an unknown courier, lines for missing orders, products and suppliers, products in a category that was never added	Q87, Q26

Dates	Shipped before ordered, delivered before shipped, delivered with no ship date, orders before sign-up, orders never shipped	Q88, Q20, Q52
Duplicates	Re-registered customer accounts, reloaded order lines, the same product on two lines, double-submitted orders	Q71, Q72, Q92, Q93
Money	Inflated, zero and missing order totals; headers with no lines	Q86
Policy	Orders paid with Pay Later after it was switched off	Q37

Loading notes

- `Orders.csv` stores dates as `DD-MM-YYYY` text. PostgreSQL's default date style reads `12-01-2025` as 1 December 2025 without complaint, so the load script parses dates with `TO_DATE(..., 'DD-MM-YYYY')`.
- `Customers.DateEntered` arrives in three formats (`YYYY/MM/DD`, `DD-MM-YYYY`, `YYYY-MM-DD`) or blank. The load script picks the format per row with a pattern test, which is itself a useful interview pattern.
- Nothing else is cleaned at load time. The faults stay in the tables for the questions to find.

SQL dialect

PostgreSQL 16. PostgreSQL-specific syntax used in the bank, with the usual MySQL 8 equivalent:

PostgreSQL	MySQL 8
`DATE_TRUNC('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`
`d::date`, `x::numeric`	`CAST(d AS DATE)`, `CAST(x AS DECIMAL(12,2))`
`date_b - date_a` (integer days)	`DATEDIFF(date_b, date_a)`
`COUNT(*) FILTER (WHERE ...)`	`SUM(CASE WHEN ... THEN 1 ELSE 0 END)`
`ILIKE`	`LOWER(x) LIKE ...`
`STRING_AGG(x, ', ' ORDER BY x)`	`GROUP_CONCAT(x ORDER BY x SEPARATOR ', ')`
`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)`	No direct equivalent; use `ROW_NUMBER()` and `COUNT(*)` (Q78)
`DISTINCT ON (...)`	`ROW_NUMBER()` in a subquery
`generate_series(...)`	Recursive CTE (Q64)
`CROSS JOIN LATERAL (VALUES ...)`	`UNION ALL` of one `SELECT` per row
`FULL OUTER JOIN`	`LEFT JOIN ... UNION ALL` right-only rows

Assumptions used throughout

- **Late delivery:** there is no promised date in the schema, so late means more than 7 days from order to delivery, a one-week promise. Every question that uses it says so.
- **Placed orders:** there is no order status column. Every `Orders` row counts as a placed order; orders never shipped are reported, not removed.
- **Money:** `Total_Order_Amount` is net order value. `Quantity * Sale_Price`, called `list_price_value`, is gross value at today's catalogue price, used only where lines are needed (product, category, supplier splits).
- **Anchoring:** "latest" periods are anchored to `MAX(OrderDate)`, never `CURRENT_DATE`, so every result reproduces on this extract.
- **Style:** aliases `o` Orders, `od` OrderDetails, `p` Products, `cat` Category, `c` Customers, `s` Suppliers, `sh` Shippers, `pay` Payments; half-open date ranges; flags tested with `UPPER(TRIM(CAST(col AS VARCHAR))) IN ('1','TRUE','T','Y','YES')`; order headers never summed after joining to their lines (Q29).

Limitations

- Transactions, ratings, suppliers and couriers are generated, so results describe this teaching dataset, not any real company.
- No hierarchy exists in the schema, so recursive CTEs appear only as a calendar alternative (Q64).
- Currency is Indian rupees, matching the catalogue prices.

Deliverable B: Master question index

One row per question. Difficulty split: 25 Easy, 35 Medium, 25 Hard, 15 Expert.

No.	Title	Difficulty	Primary topic	Combined topics	Tables used	Business domain
1	Brand directory	Easy	SELECT DISTINCT	IS NOT NULL, ORDER BY	Products	Catalogue
2	Ten highest-priced products	Easy	ORDER BY with LIMIT	NULLS LAST, FETCH FIRST ... WITH TIES	Products	Merchandising
3	Customers in metro cities	Easy	IN list filtering	TRIM and LOWER on dirty text, ORDER BY	Customers	Marketing
4	Customer emails that cannot be delivered	Easy	LIKE pattern matching	NOT LIKE, CASE to label the problem	Customers	Marketing ops
5	Orders placed in Q1 2025	Easy	Date range filtering	Half-open ranges vs BETWEEN, DATE literals	Orders	Finance
6	Most expensive products that are never discounted	Easy	NULL handling (IS NULL, NULLIF, COALESCE)	CAST to text for display, ORDER BY with LIMIT	Products	Promotions
7	Discount amount and percentage per product	Easy	Arithmetic expressions	NULLIF to avoid division by zero, ROUND, NULLS LAST	Products	Pricing
8	Price band labels	Easy	CASE WHEN	Condition order, NULL branch	Products	Merchandising
9	Check how Active and Allowed are encoded	Easy	GROUP BY profiling	UNION ALL, CAST to text	Category, Payments	Data platform
10	Headline order KPIs	Easy	Aggregate functions	COUNT DISTINCT, AVG and NULLs	Orders	Leadership reporting
11	How much of the catalogue has ever sold?	Easy	COUNT(*) vs COUNT(column)	LEFT JOIN, COUNT DISTINCT, Percentage	Products, OrderDetails	Merchandising
12	Price and discount statistics per category ID	Easy	GROUP BY with several aggregates	ROUND, Conditional COUNT with FILTER, AVG and NULLs	Products	Pricing
13	States with at least five customers	Easy	HAVING	GROUP BY, NULL group	Customers	Regional sales
14	Orders per month in 2025	Easy	DATE_TRUNC grouping	Date range filter	Orders	Operations
15	Product count for every category	Easy	LEFT JOIN with GROUP BY	COUNT(column) after a LEFT JOIN	Category, Products	Catalogue
16	December 2025 orders with customer names	Easy	INNER JOIN	CONCAT_WS, Date range filter	Orders, Customers	Customer support
17	Orders with payment type and shipper	Easy	Multi-table LEFT JOIN	COALESCE labels, Date anchor from the data	Orders, Payments, Shippers	Operations
18	Customers with no order in the last 90 days	Easy	LEFT JOIN anti-join	Filter in the ON clause vs WHERE, Date anchored to the data	Customers, Orders	Retention
19	Units sold per product	Easy	JOIN with SUM at product grain	Order lines vs units, COUNT DISTINCT	OrderDetails, Products	Merchandising
20	Days to ship and days to deliver	Easy	Date arithmetic	NULL propagation, Casting to date	Orders	Logistics
21	Top five customers by number of orders	Easy	GROUP BY with ORDER BY and LIMIT	FETCH FIRST ... WITH TIES, LEFT JOIN for names	Orders, Customers	CRM
22	Orders with more than eight line items	Easy	HAVING on counts	COUNT vs COUNT DISTINCT, SUM	OrderDetails	Fulfilment
23	Chocolate or cookie products under 250, excluding sugar-free	Easy	AND / OR / NOT precedence	ILIKE, Parentheses	Products	Merchandising

24	Suppliers outside India or with no email	Easy	NOT and OR with NULLs	IS NULL, CASE issue label	Suppliers	Procurement compliance
25	Customer sign-ups by year and quarter	Easy	Date-part grouping	ORDER BY, NULL dates	Customers	Growth
26	List-price value by category	Medium	Three-table JOIN with GROUP BY	LEFT JOIN for unmapped categories, COUNT DISTINCT	OrderDetails, Products, Category	Category management
27	Average order value by customer state	Medium	JOIN with GROUP BY and HAVING	AOV on non-null totals, LEFT JOIN, COALESCE labels	Orders, Customers	Regional marketing
28	Payment mix: share of orders and value	Medium	Percentage of total with a scalar subquery	LEFT JOIN, COALESCE labels, GROUP BY	Orders, Payments	Payments
29	Order value and units by shipper without double counting	Medium	One-to-many joins (fan-out)	Pre-aggregation in a CTE, LEFT JOIN, COALESCE	Orders, OrderDetails, Shippers	Logistics finance
30	Repeat versus one-time customers	Medium	Two-level aggregation with a CTE	CASE buckets, Percentage of total	Orders	Growth
31	Customers who spend more than the average customer	Medium	Scalar subquery over a CTE	Aggregation to customer grain, LEFT JOIN for names, Percentage difference	Orders, Customers	CRM
32	Products priced above their category average	Medium	Correlated subquery	Window AVG with PARTITION BY as an alternative	Products	Pricing
33	Products that have never been ordered	Medium	NOT EXISTS anti-join	NOT IN NULL trap, LEFT JOIN anti-join alternative	Products, OrderDetails	Merchandising
34	Each supplier's share of units supplied	Medium	Window function over an aggregate	FULL OUTER JOIN to keep both unmatched sides, Percentage contribution	OrderDetails, Suppliers	Supply chain
35	Late-delivery rate by shipper	Medium	Conditional aggregation	CASE status in a CTE, Exclusion rules for invalid dates, LEFT JOIN	Orders, Shippers	Logistics
36	Payment method mix by month as columns	Medium	Pivot with FILTER	DATE_TRUNC grouping, LEFT JOIN, Catch-all column	Orders, Payments	Payments
37	Prove no order was paid with a method that is not allowed	Medium	LEFT JOIN with a normalised flag check	Conditional status with CASE, Pre-aggregation, Business-rule validation	Payments, Orders	Risk / compliance
38	Products from inactive categories that still sold in the latest year	Medium	Multi-join aggregation with a business-rule filter	Anchored latest year, Flag normalisation, CTE	Products, Category, OrderDetails, Orders	Catalogue governance
39	Categories with no products, and categories with products but no sales	Medium	LEFT JOIN chain for unmatched records at two levels	COUNT DISTINCT, CASE status, COALESCE for zero	Category, Products, OrderDetails	Catalogue
40	Days from sign-up to first order	Medium	MIN per group with a date difference	CTE then JOIN, Negative-gap data flag	Customers, Orders	Growth / onboarding
41	Most expensive product in each category	Medium	ROW_NUMBER vs RANK for top-1 per group	Ties, DISTINCT ON, LEFT JOIN for unmapped categories	Products, Category	Merchandising
42	RANK vs DENSE_RANK vs ROW_NUMBER on order counts	Medium	Ranking function comparison	Ties, Aggregation before ranking	Orders	CRM
43	Month-to-date order value for the latest three months	Medium	Running total with PARTITION BY month	Daily aggregation first, ROWS vs RANGE frames, Anchored date window	Orders	Finance
44	Month-over-month growth in order value	Medium	LAG window function	Monthly aggregation in a CTE, NULLIF for safe division	Orders	Leadership reporting

45	Typical gap between a customer's orders	Medium	LAG partitioned by customer	Date difference, Aggregating window output in a CTE, HAVING	Orders	CRM / lifecycle
46	Same-brand products where one costs at least twice the other	Medium	Self join	Non-equi join condition to orient pairs, Ratio with NULLIF	Products	Pricing
47	Brands with at least three products, and how much they sold	Medium	COUNT DISTINCT with HAVING across a join	LEFT JOIN to keep unsold products, List-price value	Products, OrderDetails	Brand management
48	Do discounted products sell more?	Medium	CASE banding with pre-aggregation	LEFT JOIN to keep zero sellers, CTE, AVG over a product grain	Products, OrderDetails	Pricing / promotions
49	Top three products by units in each category	Medium	DENSE_RANK top-N per group	Aggregation in a CTE, Filtering on a window result, LEFT JOIN for unmapped category	OrderDetails, Products, Category	Category management
50	Customers who buy across at least ten categories	Medium	COUNT DISTINCT with HAVING over joins	Ordered list aggregation (STRING_AGG / ARRAY_AGG), Multi-table join	Orders, OrderDetails, Products	CRM / cross-sell
51	Orders fulfilled by one supplier vs several	Medium	Two-level aggregation with bucketing	COUNT DISTINCT per order, CASE buckets, Percentage of total with a window over an aggregate	OrderDetails	Fulfilment
52	Where shipping time goes: dispatch vs transit by shipper	Medium	Averages with validity filters (FILTER)	Date differences, LEFT JOIN with label, CTE	Orders, Shippers	Logistics
53	Thirty-day sign-up to purchase conversion by country	Medium	LEFT JOIN with a per-customer aggregate	MIN per group in a CTE, Conditional COUNT with FILTER, Percentage, Date difference	Customers, Orders	Growth
54	Weighted vs simple average discount by sub-category	Medium	Weighted average (SUM / SUM)	Pre-aggregation with LEFT JOIN, Validity filter on Market_Price, Multiple CTEs	Products, OrderDetails	Pricing
55	Category list-price value: latest year vs previous year	Medium	Conditional aggregation by year	Anchored year boundaries, Multi-table join, Growth percentage	Orders, OrderDetails, Products, Category	Category management
56	Customers who ordered last year but not this year	Medium	EXCEPT set operation	Anchored year boundaries, NOT EXISTS alternative, Join back for details	Orders, Customers	Retention
57	One contact directory for suppliers and shippers	Medium	UNION ALL vs UNION	Literal type column, Column alignment and NULL placeholders	Suppliers, Shippers	Partner ops
58	Each customer's latest order	Medium	ROW_NUMBER for the latest record per group	Deterministic tie-break, DISTINCT ON alternative, LEFT JOIN for names	Orders, Customers	CRM
59	Basket size by month	Medium	Two-level aggregation (per order, then per month)	Pre-aggregation CTE, Orders without lines, DATE_TRUNC	Orders, OrderDetails	Merchandising
60	Sub-category share of its category	Medium	Window SUM with PARTITION BY over aggregates	GROUP BY at sub-category grain, LEFT JOIN for unmapped category, Percentage	OrderDetails, Products, Category	Category management
61	Monthly cohort retention	Hard	Cohort analysis	Multiple CTEs, Conditional aggregation with FILTER, Anchored partial-period handling	Orders	Growth
62	Which products make up 80% of list-price value (Pareto)	Hard	Pareto analysis	Running total window with an explicit ROWS frame, Contribution percentage, CTE aggregation	OrderDetails, Products	Merchandising
63	RFM segmentation	Hard	Customer segmentation	NTILE scoring, Multiple CTEs, CASE segment rules, Anchored recency	Orders, Customers	CRM
64	Three-month moving average of order value, with no missing months	Hard	Moving averages	generate_series month spine, LEFT JOIN to fill gaps, Explicit ROWS frame, Multiple CTEs	Orders	Finance
65	Longest run of	Hard	Gaps and islands	DISTINCT customer-months, ROW_NUMBER difference,	Orders	Retention

	consecutive months with an order			Top N with ties		
66	How fast do customers come back for a second order?	Hard	Repeat purchase behaviour	LEAD and ROW_NUMBER, PERCENTILE_CONT, Multiple CTEs	Orders	Retention
67	Median vs mean order value by payment type	Hard	Median and percentiles	PERCENTILE_CONT WITHIN GROUP, LEFT JOIN with Unknown label, Skew ratio	Orders, Payments	Payments
68	Second-highest spender in each state, counting delivered orders only	Hard	Conditional ranking	DENSE_RANK over an aggregate, SUM with FILTER, LEFT JOIN with Unknown and Unmapped labels	Orders, Customers	CRM
69	Customers whose spend grew every active quarter of the latest year	Hard	Comparing current and previous periods	LAG over quarterly aggregates, BOOL_AND with FILTER, FIRST_VALUE and LAST_VALUE with a full frame, Anchored year	Orders	CRM
70	Sub-categories most often bought together	Hard	Self join for pair counting	DISTINCT before pairing, Inequality join to drop mirrored pairs, RANK for top N with ties, Choosing a grain dense enough to count	OrderDetails, Products	Merchandising
71	Deduplicate customer accounts by email	Hard	Deduplication with window functions	Normalised match key, Deterministic tie-break, Self join to the survivor row	Customers	Data platform
72	Same product on several lines of one order: split or duplicate?	Hard	Window functions over a grouped key (ROW_NUMBER, COUNT, SUM OVER)	GROUP BY HAVING to find repeated keys, COUNT DISTINCT for classification, CASE verdicts, Deduplication design	OrderDetails	Data platform
73	Category share of list-price value: latest vs previous year	Hard	Contribution percentages	Multiple CTEs, Window SUM partitioned by year, Percentage-point change, Anchored years	Orders, OrderDetails, Products, Category	Category management
74	Customers whose latest order is below their own average	Hard	Window frames (ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)	ROW_NUMBER for the latest row, Named WINDOW clause, Anchored date filter, Percentage with NULLIF	Orders	CRM
75	Rolling 28-day active customers, week by week	Hard	Rolling distinct counts with a range join	generate_series week spine, COUNT(DISTINCT) with FILTER, Anchored dates, LEFT JOIN	Orders	Growth
76	Order value from new vs returning customers by month	Hard	Window MIN to tag each customer's first order month	Conditional aggregation with FILTER, Multiple CTEs, Anchored 12-month window, Contribution percentage	Orders	Growth
77	Single-source products and the value at risk	Hard	COUNT(DISTINCT) with a single-value filter	Window SUM for share of total, Multiple CTEs, LEFT JOIN for orphan keys, List-price value	OrderDetails, Products, Suppliers	Supply chain risk
78	Expensive within their sub-category but selling below its median	Hard	PERCENT_RANK and PERCENTILE_CONT per group	Multiple CTEs, Window vs ordered-set aggregates, LEFT JOIN to keep zero sellers, Minimum group size	Products, OrderDetails	Product / pricing
79	At which order does a customer cross 10,000 in spend?	Hard	Running SUM per customer with an explicit ROWS frame	First crossing with DISTINCT ON, Named WINDOW clause, Date difference from first order, NULL amounts	Orders	CRM / LTV
80	Sub-category repurchase rate within 90 days	Hard	Correlated EXISTS with a later-date condition	Multiple CTEs, DISTINCT at the customer-sub-category-day grain, Observation window with anchored cut-off, HAVING minimum sample, Semi-join vs per-row subplan	Orders, OrderDetails, Products	Merchandising / retention
81	Year-to-date vs same period last year	Hard	Anchored date ranges with conditional aggregation	Multiple CTEs, Interval arithmetic and 29 February, FILTER for two periods in one pass, Growth percentage	Orders	Finance
82	Each customer's favourite category	Hard	ROW_NUMBER over aggregates with a deterministic tie-break	Multiple CTEs, Window SUM for share of customer units, LEFT JOIN for unmapped products, Multi-table join	Orders, OrderDetails, Products, Category	CRM / personalisation
83	Underperforming	Hard	PERCENTILE_CONT	Anchored date window, LEFT JOIN to keep zero sellers,	Orders, OrderDetails,	Category

	sub-categories in the last six months		per group as a relative threshold	Normalising by group size, Multiple CTEs, Flag normalisation	Products, Category	management
84	Delivery time by state and shipper with subtotals	Hard	GROUP BY ROLLUP with GROUPING()	Small-cell suppression with CASE, Date validity filters, LEFT JOIN with labelled unknowns, Date arithmetic	Orders, Customers, Shippers	Logistics
85	Win-back customers returning after 180+ days	Hard	LAG gap per customer	Filter after the window, Anchored 12-month reporting window, Date difference, Deterministic tie-break	Orders	Retention
86	Reconcile order headers with order lines	Expert	Reconciliation with FULL OUTER JOIN of pre-aggregates	Multiple CTEs, Rule-based status with CASE, Fixed status list with LEFT JOIN so zeros show, Discovering a hidden business rule from the data, STRING_AGG DISTINCT as evidence	Orders, OrderDetails, Products, Payments	Finance / data platform
87	Referential integrity audit across all seven relationships	Expert	Referential integrity validation	UNION ALL of anti-joins, NOT EXISTS, NULL FK vs broken FK, Conditional aggregation with FILTER, Relationship spine with VALUES	Products, Category, Orders, Customers, Payments, Shippers, OrderDetails, Suppliers	Data platform
88	Date validity audit	Expert	Invalid date detection	Rule table with CROSS JOIN LATERAL (VALUES ...), UNION ALL across tables, Anchored date logic, LEFT JOIN	Orders, Customers	Data quality
89	Incremental load with a watermark and lookback	Expert	Incremental extraction by date	Watermark CTE, INSERT ... ON CONFLICT upsert, Data-modifying CTE with RETURNING, IS DISTINCT FROM change detection	Orders	Data engineering
90	Column-by-column completeness profile of Orders	Expert	NULL and missing-value analysis	Unpivot with CROSS JOIN LATERAL (VALUES ...), Conditional aggregation with FILTER, COUNT DISTINCT, Anchored recent window	Orders	Data quality
91	Cumulative value per customer by cohort (LTV curve)	Expert	Cohort analysis	Cohort spine with generate_series, Running SUM window, Pivot with FILTER, Maturity masking with an anchored date, Multiple CTEs	Orders	Finance / growth
92	How duplicate accounts distort the repeat-customer rate	Expert	Deduplication impact on metrics	Survivor mapping with a window MIN, Normalised match key, Metric recomputed before and after, UNION ALL, Conditional aggregation	Customers, Orders	Data quality / growth
93	Possible duplicate order submissions	Expert	Duplicate detection	Line signature with ordered STRING_AGG, Self join on orders, NOT EXISTS to pick the earliest original, Two-level aggregation, IS NOT DISTINCT FROM	Orders, OrderDetails	Data platform / risk
94	Supplier concentration per category (HHI)	Expert	Share-squared sums (HHI)	Window SUM for shares, ROW_NUMBER for the top supplier, CASE bands, LEFT JOIN for unknown keys	OrderDetails, Products, Category, Suppliers	Supply chain risk
95	Debug a revenue-by-category query that gives wrong numbers	Expert	Debugging an incorrect query	One-to-many fan-out, WHERE clause on a LEFT JOIN, Anchored calendar-year filter, FULL JOIN to keep unmatched rows	Orders, OrderDetails, Products, Category	Analytics engineering
96	Make a slow order report sargable and index it	Expert	Query performance and indexing	Sargable date ranges, Correlated subquery to join rewrite, Covering indexes, EXPLAIN	Orders, OrderDetails	Performance
97	What do customers buy on their second order?	Expert	Sequence analysis with ROW_NUMBER	Dominant category per order, Conditional aggregation pivot, Window SUM over aggregates for row percentages, LEFT JOIN to keep customers without a second order	Orders, OrderDetails, Products, Category	Growth / merchandising
98	Category association rules: support, confidence, lift	Expert	Market basket analysis	Distinct order-category baskets, Self join on OrderID, Multiple CTEs, Ratio metrics with a minimum-support filter	OrderDetails, Products, Category	Merchandising
99	Flag unusual weeks in order volume	Expert	Anomaly detection with trailing window statistics	generate_series week spine, ROWS frame that excludes the current row, STDDEV_SAMP and AVG as window functions, Z-score with NULLIF	Orders	Monitoring
100	Monthly KPI scorecard with month-over-month change	Expert	Multi-stage KPI calculation (capstone)	Multiple CTEs, Partial-period handling with like-for-like windows, Conditional aggregation with FILTER, Unpivot with CROSS JOIN LATERAL (VALUES ...), Fan-out control, Percentage-point vs percent change	Orders, OrderDetails	Leadership reporting

Deliverable C: Complete question bank

Easy

Questions 1 to 25

Question 1: Brand directory

Difficulty: Easy

Primary Topic: SELECT DISTINCT

Combined Topics: IS NOT NULL, ORDER BY

Business Scenario: The catalogue team is building the "Shop by brand" page in the app. They need one clean list of brand names to load into the brand filter. Products sold without a brand, such as loose staples, should not create an empty entry in that filter.

Problem Statement: Return each brand name in `Products` once, in alphabetical order. Leave out products whose `Brand` is NULL, empty or only spaces.

Expected Output:

- `Brand` : one row per distinct brand name

Tables Required: `Products`

SQL Concepts Tested: SELECT DISTINCT, filtering NULLs with IS NOT NULL, ORDER BY.

Step 1 — Understand the Problem: `Products` has one row per product, and many products share a brand. We want the list of brands, not the list of products, so repeated brand names have to collapse into one row each.

Step 2 — Identify the Tables and Relationships: Only `Products`. The brand name is stored directly on the product row, so no join is needed.

Step 3 — Build the Query Logic:

1. Drop rows where `Brand` is NULL or blank.
2. Keep one copy of each remaining brand with `DISTINCT`.
3. Sort A to Z.

Step 4 — Final SQL Solution:

```sql
SELECT DISTINCT p.Brand
FROM Products AS p
WHERE NULLIF(TRIM(p.Brand), '') IS NOT NULL
ORDER BY p.Brand;
```

Step 5 — Line-by-Line Explanation:

- `SELECT DISTINCT p.Brand` removes duplicate rows from the result. Ten products from one brand become one row.
- `WHERE NULLIF(TRIM(p.Brand), '') IS NOT NULL` removes unbranded products. TRIM turns a brand made only of spaces into `''`, and NULLIF turns that into NULL. `IS NOT NULL` alone would keep the product stored with a single-space brand, which is exactly the empty entry the team wants gone. Without any filter, DISTINCT treats all NULLs as one value and you get a single NULL row at the end of the list.
- `ORDER BY p.Brand` sorts alphabetically. Without an ORDER BY, the database is free to return rows in any order.

Step 6 — Sample Output:

brand
&Me
&Stirred
109Ã°F
137 Degree
18 Herbs
1mg
1st Bites
24 Mantra

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 2244 rows.

Step 7 — Why This Solution Works: DISTINCT compares whole result rows. The result has one column, so a row is just the brand name and each brand survives once. WHERE runs before DISTINCT, so neither the NULL nor the blank row reaches the output.

Step 8 — Common Mistakes:

- Adding a second column. DISTINCT applies to the full row, so this returns one row per brand and product pair, which is the whole catalogue again:

```
SELECT DISTINCT p.Brand, p.Product
FROM Products AS p
WHERE p.Brand IS NOT NULL
ORDER BY p.Brand, p.Product;
```

- Expecting DISTINCT to merge spelling variants. `'Sunrise'`, `'sunrise'` and `'Sunrise '` (trailing space) are three different strings, so all three survive. A blank string `''` is not NULL either, so it passes `IS NOT NULL`.

Step 9 — Interview Follow-Up: The app team now wants the number of products shown next to each brand. How does the query change, and what happens to the products with no brand?

Step 10 — Optimization or Alternative: `GROUP BY p.Brand` returns the same list and is the form to use once you need a count per brand. Before publishing the filter, check whether any brand hides behind more than one spelling:

```
SELECT LOWER(TRIM(p.Brand))    AS brand_key,
       COUNT(DISTINCT p.Brand) AS spellings
FROM Products AS p
WHERE p.Brand IS NOT NULL
GROUP BY LOWER(TRIM(p.Brand))
HAVING COUNT(DISTINCT p.Brand) > 1
ORDER BY brand_key;
```

An empty result means every brand has exactly one spelling.

Question 2: Ten highest-priced products

Difficulty: Easy

Primary Topic: ORDER BY with LIMIT

Combined Topics: NULLS LAST, FETCH FIRST ... WITH TIES

Business Scenario: The merchandising team fills a "Premium picks" carousel on the home page. It has exactly ten slots and shows the ten products with the highest current `Sale_Price`. The job runs every night, and the carousel must not reshuffle when nothing has changed, so a tie for the last slot needs a fixed rule.

Problem Statement: Return the ten products with the highest `Sale_Price`, highest first. A product with no price must never take a slot. If products tie on price at the cut-off, the lower `ProductID` wins.

Expected Output:

- `ProductID`
- `Product`
- `Brand`
- `Sale_Price`

Tables Required: `Products`

SQL Concepts Tested: filtering NULLs with IS NOT NULL, ORDER BY ... DESC, where NULLs sort (NULLS LAST), LIMIT, deterministic tie-breaking, FETCH FIRST ... WITH TIES.

Step 1 — Understand the Problem: "Top ten" is a sort followed by a cut. Two details decide whether the answer is right: products with no price must be kept out, and the product that wins a tie for the tenth-highest price must always be the same one. The first is a rule about which rows qualify, so it belongs in WHERE, not in the sort.

Step 2 — Identify the Tables and Relationships: Only `Products`, one row per product. Each row is a candidate for a slot.

Step 3 — Build the Query Logic:

1. Drop products with no `Sale_Price`.
2. Sort the rest by `Sale_Price` descending.
3. Add `ProductID` as a second sort key so ties always resolve the same way.
4. Keep the first ten rows.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       p.Brand,
       p.Sale_Price
FROM Products AS p
WHERE p.Sale_Price IS NOT NULL
ORDER BY p.Sale_Price DESC,
         p.ProductID
LIMIT 10;
```

Step 5 — Line-by-Line Explanation:

- `WHERE p.Sale_Price IS NOT NULL` applies the rule "no price, no slot". Unpriced products are removed before the sort, so they can never appear, however few priced products there are.
- `ORDER BY p.Sale_Price DESC` puts the most expensive product first. Because the filter has already removed NULLs, it does not matter where this database would sort them.
- `p.ProductID` breaks ties. Without it, two products at the same price can swap places between runs.
- `LIMIT 10` keeps the first ten rows of the sorted result.

Step 6 — Sample Output:

productid	product	brand	sale_price
11012	Bravura Clipper	Wahl	12500.00
10851	Pet Food - N&D Team Breeder Puppy Top Farmina	Farmina	10090.00
21909	Epilator SE9-9961 Legs-Body-Face	Braun	8184.00
18645	Gas Stove-4 Burner Royale Plus Schott Glass, Black (40278)	Prestige	7999.00
18649	4 Burner Gas Stove - Marvel Plus Glass Tables, GTM04 40355	Prestige	7270.00
18617	Gas Stove-3 Burner Royale Plus Schott Glass, Black (40177)	Prestige	6999.00
7187	Whey Protein Powder	BTN Sports	6700.00
4888	Good Girl Eau De Parfum For Women	Carolina Herrera	6660.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 10 rows.

Step 7 — Why This Solution Works: The filter enforces the business rule directly, before anything is sorted. LIMIT is applied after ORDER BY, so it always cuts a sorted list. `ProductID` is the primary key, which makes the two sort keys together unique. The order is fully determined, and the same ten products come back on every run.

Step 8 — Common Mistakes:

- Leaving out the filter. PostgreSQL treats NULL as larger than any value, so in a DESC sort the unpriced products come first and take the top slots. MySQL and SQL Server put NULLs last in a DESC sort, so the same query looks fine there and breaks here.
- Using `NULLS LAST` instead of the filter. It moves unpriced products to the bottom, and on this catalogue the result is the same ten rows, because only 4 products have no price. But it only keeps them out while at least ten products have a price; with fewer, the unpriced ones fill the empty slots. The rule says "never", and only WHERE guarantees never:

```sql
SELECT p.ProductID, p.Product, p.Sale_Price
FROM Products AS p
ORDER BY p.Sale_Price DESC NULLS LAST, p.ProductID
LIMIT 10;
```

- Sorting on price alone and adding `LIMIT 10`. When two products tie at tenth place, the database picks one arbitrarily, and the pick can change from run to run:

```sql
SELECT p.ProductID, p.Product, p.Sale_Price
FROM Products AS p
ORDER BY p.Sale_Price DESC
LIMIT 10;
```

Step 9 — Interview Follow-Up: How would you return the three highest-priced products within each `CategoryID`?

Step 10 — Optimization or Alternative: If the team would rather include every product tied at tenth place, use `FETCH FIRST 10 ROWS WITH TIES`. Ties are judged on the ORDER BY keys, so `ProductID` must not be in the sort, and the result can have more than ten rows:

```sql
SELECT p.ProductID, p.Product, p.Brand, p.Sale_Price
FROM Products AS p
WHERE p.Sale_Price IS NOT NULL
ORDER BY p.Sale_Price DESC
FETCH FIRST 10 ROWS WITH TIES;
```

MySQL has no WITH TIES; there you would filter on `RANK() OVER (ORDER BY Sale_Price DESC) <= 10` in a subquery.

Question 3: Customers in metro cities

Difficulty: Easy

Primary Topic: IN list filtering

Combined Topics: TRIM and LOWER on dirty text, ORDER BY

Business Scenario: Marketing is launching same-day delivery in five metros: Bengaluru, Mumbai, Delhi, Hyderabad and Chennai. They need the customers in those cities for the launch campaign. City was typed in by hand at sign-up, so some values carry stray spaces or odd capitalisation.

Problem Statement: Return customers whose city is one of the five metros, matching regardless of case and surrounding spaces. Show the city with the spaces removed. Sort by city, then `CustomerID`.

Expected Output:

- `CustomerID`
- `FirstName`
- `LastName`
- `City`: the city with leading and trailing spaces removed
- `State`

Tables Required: `Customers`

SQL Concepts Tested: IN lists, TRIM and LOWER for text comparison, ORDER BY.

Step 1 — Understand the Problem: An IN list is shorthand for several `=` checks joined by OR. Text equality is exact: `'Mumbai '` with a trailing space does not equal `'Mumbai'`, and neither does `'MUMBAI'`. The column has to be cleaned before it is compared.

Step 2 — Identify the Tables and Relationships: Only `Customers`, one row per customer account. City and State are on the same row.

Step 3 — Build the Query Logic:

1. Clean the city: TRIM removes outer spaces, LOWER fixes case.
2. Compare the cleaned value with a lower-case IN list.
3. Return the trimmed city so the campaign tool gets clean values.

Step 4 — Final SQL Solution:

```sql
SELECT c.CustomerID,
       c.FirstName,
       c.LastName,
       TRIM(c.City) AS City,
       c.State
FROM Customers AS c
WHERE LOWER(TRIM(c.City)) IN ('bengaluru', 'bangalore', 'mumbai', 'bombay',
                              'delhi', 'new delhi', 'hyderabad', 'chennai')
ORDER BY TRIM(c.City), c.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `LOWER(TRIM(c.City))` turns `' Mumbai'`, `'Mumbai '` and `'MUMBAI'` into `'mumbai'`.
- `IN (...)` is true when the cleaned city equals any value in the list, which is written in lower case to match.
- `TRIM(c.City) AS City` returns the city without stray spaces and keeps the stored capitalisation for display.
- A NULL city makes the IN test NULL, which is not true, so customers with no city are left out.
- `ORDER BY TRIM(c.City), c.CustomerID` keeps each stored spelling together and gives a fixed order inside it. Spellings of one metro still sort apart: `BENGALURU`, `Bangalore` and `Bengaluru` form separate blocks, and in this database's C collation `bangalore` sorts after `New Delhi`.

Step 6 — Sample Output:

customerid	firstname	lastname	city	state
100021	Kritika	Agarwal	BENGALURU	Karnataka
100249	Reyansh	Banerjee	BENGALURU	Karnataka
100348	Anika	Yadav	BENGALURU	Karnataka
100973	Rahul	Sheikh	BENGALURU	Karnataka
101103	Prakash	Kamath	BENGALURU	Karnataka
101202	Aditya	Kapoor	BENGALURU	Karnataka
101835	Isha	Sharma	BENGALURU	Karnataka
101848	Megha	Agarwal	BENGALURU	Karnataka

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 2453 rows.

Step 7 — Why This Solution Works: Both sides of the comparison are in the same form, trimmed and lower case. A value that differs only by spaces or case now matches. A value spelled with different letters still does not, which is the second mistake below.

Step 8 — Common Mistakes:

- Comparing the raw column. Every row with a stray space or different case is missed, and nothing warns you:

```
SELECT c.CustomerID, c.City
FROM Customers AS c
WHERE c.City IN ('Bengaluru', 'Mumbai', 'Delhi', 'Hyderabad', 'Chennai')
ORDER BY c.City, c.CustomerID;
```

- Assuming cleaning also fixes old names. `'Bangalore'`, `'Bombay'` and `'New Delhi'` are different strings. Run `SELECT DISTINCT TRIM(c.City)` first and add any variants you find to the list.
- Listing each city once. This data stores Bengaluru as `Bengaluru`, `Bangalore`, `bangalore` and `BENGALURU`, Delhi as `Delhi` and `New Delhi`, and a few Mumbai rows as `Bombay`. LOWER and TRIM fold the case and spacing variants, but the old names are different words and must be listed. Run `SELECT LOWER(TRIM(City)), COUNT(*) FROM Customers GROUP BY 1 ORDER BY 1` before you write the list.

Step 9 — Interview Follow-Up: Marketing also wants the number of customers in each metro. How do you make sure `'Mumbai'` and `'Mumbai '` land in the same group?

Step 10 — Optimization or Alternative: A function around a column stops PostgreSQL from using a plain index on `City`. If this filter runs often, clean the column at load time or create an expression index on `LOWER(TRIM(City))`.

Question 4: Customer emails that cannot be delivered

Difficulty: Easy

Primary Topic: LIKE pattern matching

Combined Topics: NOT LIKE, CASE to label the problem

Business Scenario: The marketing operations team is about to send a re-engagement campaign. Every hard bounce hurts the sender reputation, so they want to catch addresses that cannot work before the send. A valid address here has no spaces and the shape `something@something.something`.

Problem Statement: Return every customer whose `Email` is missing, contains a space, or does not match the shape `_@_._` (at least one character, an @, at least one character, a dot, at least one character). Label the problem. Sort by `CustomerID`.

Expected Output:

- `CustomerID`, `FirstName`, `LastName`
- `Email`: as stored
- `issue`: `Missing`, `Contains a space` or `Not in name@domain.tld form`

Tables Required: `Customers`

SQL Concepts Tested: LIKE wildcards (`%` any run of characters, `_` exactly one), NOT LIKE, combining conditions with OR, NULL behaviour of LIKE.

Step 1 — Understand the Problem: Each output row is one customer with a bad address. Three rules, any one is enough, so they are joined with OR. LIKE patterns describe shapes: `% %` means "a space somewhere", and `%_@_%._%` means "at least one character, an @, at least one character, a dot, at least one character".

Step 2 — Identify the Tables and Relationships: Only `Customers`, one row per customer, with `Email` as text.

Step 3 — Build the Query Logic:

1. Flag NULL or blank emails.
2. Flag emails containing a space with `LIKE '% %'`.
3. Flag emails that do NOT match `'%_@_%._%'`.
4. Label the first rule that matches with CASE, and sort.

Step 4 — Final SQL Solution:

```
SELECT c.CustomerID,
       c.FirstName,
       c.LastName,
       c.Email,
       CASE WHEN c.Email IS NULL OR TRIM(c.Email) = '' THEN 'Missing'
            WHEN c.Email LIKE '% %'                    THEN 'Contains a space'
            ELSE 'Not in name@domain.tld form'
       END AS issue
FROM Customers AS c
WHERE c.Email IS NULL
   OR TRIM(c.Email) = ''
   OR c.Email LIKE '% %'
   OR c.Email NOT LIKE '%_@_%._%'
ORDER BY c.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `c.Email IS NULL OR TRIM(c.Email) = ''` catches both kinds of "no address". LIKE on a NULL returns NULL, so a NULL email would slip past the other two tests.
- `LIKE '% %'`: `%` matches any run of characters, including none, so this finds a space anywhere.
- `NOT LIKE '%_@_%._%'`: `_` matches exactly one character, so the pattern demands something before the @, something after it, a dot later on, and something after the dot.
- The WHERE clause uses OR, so a row qualifies if any rule fails. The CASE then names the first problem found, in the same order.

Step 6 — Sample Output:

customerid	firstname	lastname	email	issue
100324	Farhan	Fernandes	NULL	Missing
100360	Mahesh	Menon	NULL	Missing
100397	Jyoti	Chowdary	NULL	Missing
100501	Jyoti	Banerjee	NULL	Missing
100663	Siddharth	Gill	NULL	Missing
100731	Sunita	Shetty	NULL	Missing
100739	Aarav	Chowdary	NULL	Missing
100972	Harsh	Verma	NULL	Missing

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 64 rows.

Step 7 — Why This Solution Works: Each rule is a cheap shape test the database can run on every row, and together they catch the errors that bounce: missing addresses, spaces copied from a full name, and strings with no domain. The CASE gives support a reason to fix, not just a list. Shape checks cannot prove an address exists; they only remove the ones that certainly do not work.

Step 8 — Common Mistakes:

- Forgetting the NULL case. `NOT LIKE` on NULL is NULL, not TRUE, so missing emails silently pass a filter built only from LIKE.
- Using `LIKE '%@%'` as the validity test. It accepts `@`, `a@` and `a@b`, none of which can receive mail.
- Expecting LIKE to ignore case. In PostgreSQL LIKE is case-sensitive; use `ILIKE` (PostgreSQL only) or `LOWER(c.Email) LIKE ...` when case matters, for example when matching a domain such as `%@gmail.com`.

Step 9 — Interview Follow-Up: Marketing also wants addresses whose domain is not `gmail.com` flagged for a separate review. How would you extract the domain, and how would you make the match case-insensitive?

Step 10 — Optimization or Alternative: PostgreSQL regular expressions express the shape in one test: `c.Email !~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'` flags anything that is not one @ with no spaces and a dot in the domain. MySQL 8 has `REGEXP` with the same idea. A leading `%` in a LIKE pattern cannot use a normal B-tree index, so on a large table this check runs as a full scan, which is fine for a nightly data-quality job.

Question 5: Orders placed in Q1 2025

Difficulty: Easy

Primary Topic: Date range filtering

Combined Topics: Half-open ranges vs BETWEEN, DATE literals

Business Scenario: Finance is closing January to March 2025 and needs the orders placed in that quarter to tie against the payment gateway report. Cancellations and refunds are not recorded in this dataset, so every row in `Orders` counts as a placed order.

Problem Statement: Return every order with an `OrderDate` from 1 January 2025 to 31 March 2025 inclusive. Sort by `OrderDate`, then `OrderID`.

Expected Output:

- `OrderID`
- `CustomerID`
- `OrderDate`
- `Total_Order_Amount`

Tables Required: `Orders`

SQL Concepts Tested: date range filters, half-open ranges, BETWEEN, ISO DATE literals.

Step 1 — Understand the Problem: Q1 2025 covers every moment from the start of 1 January to the end of 31 March. If `OrderDate` ever carries a time, "the end of 31 March" is awkward to write. The clean description is: on or after 1 January, and before 1 April.

Step 2 — Identify the Tables and Relationships: Only `Orders`, one row per order.

Step 3 — Build the Query Logic:

1. Lower bound `>= DATE '2025-01-01'`, inclusive.

2. Upper bound `< DATE '2025-04-01'`, exclusive. Inclusive start plus exclusive end is called a half-open range.
3. Sort for a stable list.

Step 4 — Final SQL Solution:

```sql
SELECT o.OrderID,
       o.CustomerID,
       o.OrderDate,
       o.Total_Order_Amount
FROM Orders AS o
WHERE o.OrderDate >= DATE '2025-01-01'
  AND o.OrderDate <  DATE '2025-04-01'
ORDER BY o.OrderDate, o.OrderID;
```

Step 5 — Line-by-Line Explanation:

- `DATE '2025-01-01'` is an ISO date literal. It cannot be misread the way `'01/03/2025'` can (1 March or 3 January, depending on settings).
- `>=` keeps orders placed on 1 January.
- `< DATE '2025-04-01'` keeps everything on 31 March, at any time of day, and nothing from April.
- Both conditions compare the raw column, so an index on `OrderDate` can be used.

Step 6 — Sample Output:

orderid	customerid	orderdate	total_order_amount
5017767	100024	2025-01-01	413.00
5017768	100072	2025-01-01	4018.00
5017769	100143	2025-01-01	204.00
5017770	100279	2025-01-01	418.00
5017771	100362	2025-01-01	1770.00
5017772	100428	2025-01-01	1010.00
5017773	100630	2025-01-01	1122.00
5017774	100791	2025-01-01	179.10

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 5179 rows.

Step 7 — Why This Solution Works: The half-open range gives the same answer whether `OrderDate` is a DATE or a TIMESTAMP. It also chains cleanly: Q2 is `>= '2025-04-01' AND < '2025-07-01'`, so quarters never overlap and never leave a gap.

Step 8 — Common Mistakes:

- `BETWEEN DATE '2025-01-01' AND DATE '2025-03-31'`. Correct on a DATE column, but on a TIMESTAMP the upper bound means midnight at the start of 31 March, and the rest of that day is lost. Two sample timestamps show it:

```sql
SELECT t.ts,
       t.ts BETWEEN DATE '2025-01-01' AND DATE '2025-03-31'      AS kept_by_between,
       t.ts >= DATE '2025-01-01' AND t.ts < DATE '2025-04-01' AS kept_by_half_open
FROM (VALUES (TIMESTAMP '2025-03-31 00:00:00'),
             (TIMESTAMP '2025-03-31 18:45:00')) AS t(ts);
```

- `EXTRACT(QUARTER FROM o.OrderDate) = 1` without a year returns the first quarter of every year. Even with a year added, the function on the column stops an index from being used.

Step 9 — Interview Follow-Up: Finance wants the same list for the most recent complete quarter in the data instead of a hard-coded one. How would you derive the two bounds?

Step 10 — Optimization or Alternative: For a reusable report, compute the bounds instead of typing them: `DATE_TRUNC('quarter', some_date)` gives the first day of that quarter, and adding `INTERVAL '3 months'` gives the exclusive end. The filter stays half-open either way.

Question 6: Most expensive products that are never discounted

Difficulty: Easy

Primary Topic: NULL handling (IS NULL, NULLIF, COALESCE)

Combined Topics: CAST to text for display, ORDER BY with LIMIT

Business Scenario: The promotions team is planning a festive sale and wants to start with high-value products that currently sell at the printed price, with no discount at all. They want a short list of the ten most expensive such products, with a discount column that reads "Full price" instead of a number, so the sheet is easy to scan.

Problem Statement: Compute each product's discount amount as `Market_Price - Sale_Price`, but store "no discount" as NULL rather than 0. Return the ten most expensive products whose discount is NULL, with a `discount_display` column that shows the text `Full price` for them. A product with a missing `Sale_Price` or `Market_Price` has an unknown discount, not a full price, and must never take a slot. Sort by `Sale_Price` descending, then `ProductID`.

Expected Output:

- `ProductID`, `Product`, `Brand`
- `Sale_Price`, `Market_Price`
- `discount_display`: the discount as text, or `Full price` when there is none

Tables Required: `Products`

SQL Concepts Tested: creating NULL on purpose with NULLIF, testing with IS NULL, why `= NULL` never matches, COALESCE with a type cast, ORDER BY with LIMIT.

Step 1 — Understand the Problem: NULL means "no value", and it is often the honest way to store "not applicable". A discount of 0 and "no discount" look the same as numbers, but turning the zero into NULL lets the rest of the query test for it directly and keeps it out of any later averages. The output is product grain: one row per product, ten rows at most.

Step 2 — Identify the Tables and Relationships: Only `Products` is needed. `Sale_Price` is the price on the application and `Market_Price` is the printed price, so their difference is the discount.

Step 3 — Build the Query Logic:

1. Compute `NULLIF(p.Market_Price - p.Sale_Price, 0)`: the discount, or NULL when it is zero.
2. Keep rows where both prices are present and that expression `IS NULL`.
3. Display it with `COALESCE(... ::text, 'Full price')`.
4. Sort by price, highest first, and keep ten rows.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       p.Brand,
       p.Sale_Price,
       p.Market_Price,
       COALESCE(NULLIF(p.Market_Price - p.Sale_Price, 0)::text, 'Full price') AS discount_display
FROM Products AS p
WHERE p.Sale_Price IS NOT NULL
  AND p.Market_Price IS NOT NULL
  AND NULLIF(p.Market_Price - p.Sale_Price, 0) IS NULL
ORDER BY p.Sale_Price DESC, p.ProductID
LIMIT 10;
```

Step 5 — Line-by-Line Explanation:

- `NULLIF(a, b)` returns NULL when `a = b`, otherwise `a`. Here a zero discount becomes NULL.
- `p.Sale_Price IS NOT NULL AND p.Market_Price IS NOT NULL` comes first. A missing price also makes the subtraction NULL, so without it unpriced products would be labelled `Full price`, and because PostgreSQL sorts NULLs first in a DESC sort they would take the top slots.
- `NULLIF(...) IS NULL` then keeps the zero-discount rows. `IS NULL` is the only correct test for a missing value.
- `::text` converts the number to text. COALESCE needs all its arguments to share a type, and `'Full price'` is text.
- `COALESCE(x, 'Full price')` returns the first non-NULL argument, so NULL discounts read `Full price`.
- `ORDER BY p.Sale_Price DESC, p.ProductID` sorts by price and breaks ties by ID, so the ten rows are always the same ten.

Step 6 — Sample Output:

productid	product	brand	sale_price	market_price	discount_display
11012	Bravura Clipper	Wahl	12500.00	12500.00	Full price
10851	Pet Food - N&D Team Breeder Puppy Top Farmina	Farmina	10090.00	10090.00	Full price
7187	Whey Protein Powder	BTN Sports	6700.00	6700.00	Full price
9606	Olive Oil - Extra Virgin	Colavita	6500.00	6500.00	Full price
4259	Man In Black Eau De Parfum	Bvlgari	6100.00	6100.00	Full price
10989	U-Clip Clipper	Wahl	6000.00	6000.00	Full price
18787	Casserole Set - Die-Cast, Induction Base, Granite	Wonderchef	6000.00	6000.00	Full price
6632	Whey Protein Powder - Vanilla	FITWHEY	5900.00	5900.00	Full price

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 10 rows.

Step 7 — Why This Solution Works: The query turns "no discount" into a real NULL once and then uses the right tools on it: `IS NULL` to filter and COALESCE to label. The same NULLIF trick also protects divisions (`x / NULLIF(y, 0)`), which later questions rely on. A row with a missing price would also produce a NULL discount, but that NULL means "unknown", not "no discount". The WHERE clause removes those rows instead of calling them full price.

Step 8 — Common Mistakes:

- Writing `WHERE NULLIF(p.Market_Price - p.Sale_Price, 0) = NULL`. Any comparison with NULL returns unknown, never true, so the query returns nothing:

```sql
SELECT COUNT(*) AS rows_matched_with_equals_null
FROM Products AS p
WHERE NULLIF(p.Market_Price - p.Sale_Price, 0) = NULL;
```

- Writing `COALESCE(NULLIF(...), 'Full price')` without the cast. PostgreSQL tries to read `'Full price'` as a number and fails with an invalid input error.
- Filtering with `p.Sale_Price = p.Market_Price` and calling it done. That finds the rows, but the moment someone asks for the average discount of discounted items only, you need the NULL version anyway.

Step 9 — Interview Follow-Up: How would you return the average discount per brand counting only discounted products, and why does the NULL version of the discount make that a one-line change?

Step 10 — Optimization or Alternative: `p.Market_Price = p.Sale_Price` in the WHERE clause is simpler and returns the same ten rows, because `=` is never true when either price is NULL. No ordinary index can search a comparison between two columns, but a partial index `ON Products (Sale_Price DESC, ProductID) WHERE Market_Price = Sale_Price` serves both the filter and the sort; keep the NULLIF form in the SELECT list for display. MySQL has `IFNULL` and `NULLIF` too, and casts with `CAST(x AS CHAR)`.

Question 7: Discount amount and percentage per product

Difficulty: Easy

Primary Topic: Arithmetic expressions

Combined Topics: NULLIF to avoid division by zero, ROUND, NULLS LAST

Business Scenario: The pricing team is checking how deep discounts run across the catalogue. `Market_Price` is the MRP printed on the pack and `Sale_Price` is what the store charges today. A missing or zero `Market_Price` means the MRP was never captured, so that product has no measurable discount.

Problem Statement: For every product, return the discount in rupees and as a percentage of `Market_Price`, rounded to two decimals. Products with a missing or zero `Market_Price` get NULL for both. Sort by discount percentage, highest first, with NULLs at the end.

Expected Output:

- `ProductID`
- `Product`
- `Market_Price`
- `Sale_Price`
- `discount_amount`: `Market_Price - Sale_Price`
- `discount_pct`: discount as a percentage of `Market_Price`, 2 decimals

Tables Required: `Products`

SQL Concepts Tested: column arithmetic, NULLIF, safe division, ROUND, NULLS LAST.

Step 1 — Understand the Problem: The discount is a calculation on two columns of the same row. The risk is the division: a zero `Market_Price` makes PostgreSQL stop the whole query with "division by zero", and a zero MRP is bad data anyway.

Step 2 — Identify the Tables and Relationships: Only `Products`. Both prices sit on the product row, so each row is computed on its own.

Step 3 — Build the Query Logic:

1. Turn a zero `Market_Price` into NULL with `NULLIF(p.Market_Price, 0)`.
2. Subtract for the amount. Divide by the NULLIF result and multiply by 100 for the percentage.
3. Round to two decimals and sort with NULLs last.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       p.Market_Price,
       p.Sale_Price,
       NULLIF(p.Market_Price, 0) - p.Sale_Price                                    AS discount_amount,
       ROUND(100.0 * (p.Market_Price - p.Sale_Price) / NULLIF(p.Market_Price, 0), 2) AS discount_pct
FROM Products AS p
ORDER BY discount_pct DESC NULLS LAST,
         p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `NULLIF(p.Market_Price, 0)` returns NULL when the price is 0, and the price otherwise.
- `NULLIF(...) - p.Sale_Price` is NULL for an unusable MRP, because arithmetic with NULL returns NULL. Without NULLIF, a zero MRP shows a large negative discount that looks real.
- Dividing by NULL returns NULL instead of an error. `100.0` keeps the maths decimal even on integer columns, where PostgreSQL turns `7 / 2` into `3`.
- `DESC NULLS LAST` puts the deepest discounts first and unusable rows last.

Step 6 — Sample Output:

productid	product	market_price	sale_price	discount_amount	discount_pct
7703	Fruit & Vegetables Hand Juicer	1995.00	349.00	1646.00	82.51
924	Small Silicone Spatula With Plastic Handle - Assorted Colours	399.00	75.00	324.00	81.20
517	Decorative Party Light Big Star String LED Light 2 M - Multicolour	1099.00	209.00	890.00	80.98
6274	NHS 860 Temperature Control Professional Hair Straightener	2395.00	467.00	1928.00	80.50
7927	Concealer Brush 930	1750.00	350.00	1400.00	80.00
511	Decorative Party Light Golden Bell String LED Light 7 M - Multicolour	1999.00	415.00	1584.00	79.24
520	Decorative Party Light Golden Bell String LED Light 7 M - Warm White	1999.00	415.00	1584.00	79.24
425	USB String Fairy Lights 3M 30 LED For Decoration - Multicolour	399.00	85.00	314.00	78.70

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 22088 rows.

Step 7 — Why This Solution Works: Every row gets either a real discount or an honest NULL. Division by zero cannot happen, because the zero is gone before the division runs.

Step 8 — Common Mistakes:

- Dividing by `p.Market_Price` directly. One zero MRP anywhere in the table makes the whole query fail, not only that row.
- Using `COALESCE(p.Market_Price, 1)` or similar to avoid the error. The query runs, but it invents a discount for a product whose MRP is unknown.
- Reading a negative discount as a markup strategy. A `Sale_Price` above `Market_Price` means a stale MRP or a keying error. List those rows for the catalogue team:

```
SELECT p.ProductID, p.Product, p.Market_Price, p.Sale_Price
FROM Products AS p
WHERE p.Sale_Price > p.Market_Price
ORDER BY p.ProductID;
```

Step 9 — Interview Follow-Up: How would you report the average discount percentage per `Sub_Category`, and should products with an invalid MRP count towards that average?

Step 10 — Optimization or Alternative: `CASE WHEN p.Market_Price > 0 THEN ... END` does the same job and also rejects a negative MRP. MySQL returns NULL on division by zero by default, so the problem can hide there; NULLIF states the intent on both.

Question 8: Price band labels

Difficulty: Easy

Primary Topic: CASE WHEN

Combined Topics: Condition order, NULL branch

Business Scenario: Merchandising is adding a "Shop by budget" filter to the app. Each product must carry exactly one band based on its current `Sale_Price`: Budget below 200, Mid from 200 up to 699, Premium from 700. A product with no price should show `Unknown` so it can be fixed, not disappear from the filter.

Problem Statement: Label every product with a price band. Mid means 200 or more and less than 700, so a price of 699.50 is Mid. Sort by `ProductID`.

Expected Output:

- `ProductID`
- `Product`
- `Sale_Price`
- `price_band`: `Budget`, `Mid`, `Premium` or `Unknown`

Tables Required: `Products`

SQL Concepts Tested: CASE WHEN, order of conditions, NULL handling, boundaries on decimal values.

Step 1 — Understand the Problem: "200 to 699" is how people say it, but prices have paise. A band written as `BETWEEN 200 AND 699` leaves 699.01 to 699.99 in no band at all. Write each band as "at least the lower bound and below the next band's lower bound".

Step 2 — Identify the Tables and Relationships: Only `Products`. Each product is labelled from its own `Sale_Price`.

Step 3 — Build the Query Logic:

1. Handle NULL first, so it gets its own label.
2. Test the thresholds from low to high. CASE stops at the first true condition, so each later branch only sees prices that failed the earlier tests.
3. Let ELSE catch everything left, which can only be 700 and above.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       p.Sale_Price,
       CASE
           WHEN p.Sale_Price IS NULL THEN 'Unknown'
           WHEN p.Sale_Price < 200   THEN 'Budget'
           WHEN p.Sale_Price < 700   THEN 'Mid'
           ELSE 'Premium'
       END AS price_band
FROM Products AS p
ORDER BY p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `WHEN p.Sale_Price IS NULL` comes first. Every comparison with NULL is unknown, so without this branch a NULL price would skip both `<` tests and fall into ELSE as Premium.
- `WHEN p.Sale_Price < 200` catches all prices below 200, including 199.99.
- `WHEN p.Sale_Price < 700` only sees prices of 200 or more, because smaller ones were already labelled. So it means "200 up to, not including, 700".
- `ELSE 'Premium'` takes what is left: 700 and above.

Step 6 — Sample Output:

productid	product	sale_price	price_band
1	Original Disinfectant Toilet Cleaner Liquid	489.00	Mid
2	Surface Disinfectant Spray	257.00	Mid
3	Harpic Disinfectant Toilet Cleaner Original200ml+Lizol Floor Cleaner Citrus200ml	74.00	Budget
4	Harpic Toilet Cleaner Liquid - Original 1 L + Lizol Floor Cleaner, Citrus - 2L	462.00	Mid
5	Disinfectant Bathroom Cleaner Liquid - Lemon	299.00	Mid
6	Super Saver Pack Toilet Cleaner Original, 500ml + Bathroom Cleaner Lemon, 500ml	162.00	Budget
7	Original 1 ltr + Bathroom Cleaner Lemon 500 ml	247.00	Mid
8	Disinfectant Toilet Cleaner Liquid - 1 L + Bathroom Cleaner Liquid - 500 ml	247.00	Mid

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 22088 rows.

Step 7 — Why This Solution Works: The branches split the number line at 200 and 700 with no gaps and no overlaps, and NULL has its own branch, so every row gets exactly one label. You can prove it on the edge values:

```sql
SELECT v.price,
       CASE
           WHEN v.price IS NULL THEN 'Unknown'
           WHEN v.price < 200   THEN 'Budget'
           WHEN v.price < 700   THEN 'Mid'
           ELSE 'Premium'
       END AS price_band
FROM (VALUES (199.99), (200.00), (699.50), (700.00), (NULL)) AS v(price)
ORDER BY v.price NULLS LAST;
```

Step 8 — Common Mistakes:

- Putting the wider condition first. Every price below 700 stops at the first branch, so no product is ever labelled Budget:

```sql
SELECT p.ProductID, p.Sale_Price,
       CASE
           WHEN p.Sale_Price < 700 THEN 'Mid'
           WHEN p.Sale_Price < 200 THEN 'Budget'
           ELSE 'Premium'
       END AS price_band
FROM Products AS p
ORDER BY p.ProductID;
```

- Using `BETWEEN 200 AND 699` for Mid, which leaves 699.50 unmatched.
- Leaving out ELSE. Unmatched rows get NULL with no warning.

Step 9 — Interview Follow-Up: Merchandising wants the number of products in each band, with the bands listed in the order Budget, Mid, Premium, Unknown rather than alphabetically. How do you control that order?

Step 10 — Optimization or Alternative: If the thresholds change often, keep them in a small band table (band name, lower bound, upper bound) and join on `Sale_Price >= lower AND Sale_Price < upper`. Merchandising can then change a band without anyone editing SQL.

Question 9: Check how Active and Allowed are encoded

Difficulty: Easy

Primary Topic: GROUP BY profiling

Combined Topics: UNION ALL, CAST to text

Business Scenario: The data platform team is publishing shared views that filter on "active categories" and "allowed payment methods". `Category.Active` and `Payments.Allowed` came in through CSV loads from two systems, and nobody documented the encoding. The team wants to see every stored value first.

Problem Statement: In one result, list each distinct raw value of `Category.Active` and `Payments.Allowed` with its row count. NULL counts as a value.

Expected Output:

- `source_table`: `Category` or `Payments`
- `flag_column`: `Active` or `Allowed`
- `raw_value`: the stored value as text, NULL when missing
- `row_count`: rows holding that value

Tables Required: `Category`, `Payments`

SQL Concepts Tested: GROUP BY for profiling, UNION ALL, CAST to a common type, NULL as a group.

Step 1 — Understand the Problem: A yes/no flag can be BOOLEAN, 1/0, or text such as `Y`, `Yes` or `TRUE`. CSV loads turn booleans into text, and hand typing adds case and space variants. A filter written for one encoding silently returns wrong rows on another.

Step 2 — Identify the Tables and Relationships: `Category` (one row per category) and `Payments` (one row per payment method). They are unrelated, so we stack two profiles.

Step 3 — Build the Query Logic:

1. Per table, group by the flag cast to VARCHAR and count rows.
2. Add literal labels for table and column.
3. Stack with UNION ALL and sort.

Step 4 — Final SQL Solution:

```
SELECT 'Category'                  AS source_table,
       'Active'                    AS flag_column,
       CAST(cat.Active AS VARCHAR) AS raw_value,
       COUNT(*)                    AS row_count
FROM Category AS cat
GROUP BY CAST(cat.Active AS VARCHAR)
UNION ALL
SELECT 'Payments',
       'Allowed',
       CAST(pay.Allowed AS VARCHAR),
       COUNT(*)
FROM Payments AS pay
GROUP BY CAST(pay.Allowed AS VARCHAR)
ORDER BY source_table, raw_value NULLS LAST;
```

Step 5 — Line-by-Line Explanation:

- `CAST(cat.Active AS VARCHAR)` turns a BOOLEAN into `true`/`false`, an integer into `1`/`0`, and leaves text alone. UNION ALL needs one type per column position, so a BOOLEAN flag and a text flag cannot be stacked without it.
- `GROUP BY` the same expression gives one row per distinct value. All NULLs form one group, so missing flags show up.
- `UNION ALL` stacks the two branches; the first one names the columns.

Step 6 — Sample Output:

source_table	flag_column	raw_value	row_count
Category	Active	0	1
Category	Active	1	1
Category	Active	No	2
Category	Active	Y	1
Category	Active	Yes	5
Category	Active	'Yes '	1
Category	Active	yes	1
Payments	Allowed	N	1

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 10 rows.

Step 7 — Why This Solution Works: It shows the real encoding, not a guess. Every later question in this bank turns a flag into true/false with one rule:

```
UPPER(TRIM(CAST(cat.Active AS VARCHAR))) IN ('1','TRUE','T','Y','YES')
```

CAST makes it work whatever the column type is. TRIM removes stray spaces. UPPER folds `yes`, `Yes` and `YES` together. The IN list covers the usual spellings of "true". Everything else, such as `No`, `0`, `false` or an unexpected word, counts as false. One catch: a NULL flag makes the rule return NULL, and `NOT (rule)` is NULL as well, so that row lands in neither list. Use `COALESCE(rule, FALSE)` when every row must fall

on one side.

Step 8 — Common Mistakes:

- Comparing a text flag with a number or boolean. PostgreSQL refuses:

```
WHERE cat.Active = 1
ERROR:  operator does not exist: character varying = integer
```

- `WHERE cat.Active = 'Yes'` works today and returns nothing after a reload switches to `Y`.
- Eyeballing the list. `Yes` and `Yes` look identical; add `LENGTH(raw_value)` to spot the space.

Step 9 — Interview Follow-Up: A new load brings `Enabled` into `Payments.Allowed`. What does the rule do with it, and how would you make the pipeline alert?

Step 10 — Optimization or Alternative: Check what the rule does to each stored value:

```sql
SELECT CAST(pay.Allowed AS VARCHAR) AS raw_value,
       COALESCE(UPPER(TRIM(CAST(pay.Allowed AS VARCHAR))) IN ('1','TRUE','T','Y','YES'), FALSE) AS treated_as_allowed,
       COUNT(*) AS row_count
FROM Payments AS pay
GROUP BY CAST(pay.Allowed AS VARCHAR)
ORDER BY raw_value NULLS LAST;
```

On MySQL, use `CAST(x AS CHAR)`.

Question 10: Headline order KPIs

Difficulty: Easy

Primary Topic: Aggregate functions

Combined Topics: COUNT DISTINCT, AVG and NULLs

Business Scenario: Leadership opens every monthly review with one strip of numbers: how many orders, how many customers ordered, total order value, average order value (AOV), and the date range the data covers. Cancellations and refunds are not recorded here, so every row in `Orders` counts as a placed order.

Problem Statement: Return one row with the headline numbers for the whole `Orders` table. AOV is the average `Total_Order_Amount` per order, rounded to two decimals.

Expected Output:

- `total_orders`: number of orders
- `ordering_customers`: number of distinct customers who placed an order
- `total_order_value`: sum of `Total_Order_Amount`
- `avg_order_value`: average `Total_Order_Amount`, 2 decimals
- `first_order_date`: earliest `OrderDate`
- `last_order_date`: latest `OrderDate`

Tables Required: `Orders`

SQL Concepts Tested: COUNT, COUNT DISTINCT, SUM, AVG, MIN, MAX, how aggregates treat NULL.

Step 1 — Understand the Problem: Each number is one aggregate over the whole table. With no GROUP BY, the whole table is one group and the result is one row. The one thing to think about is what each function does with NULL.

Step 2 — Identify the Tables and Relationships: Only `Orders`, one row per order. All the columns needed are on the order row, so there is no join and no risk of counting an order twice.

Step 3 — Build the Query Logic:

1. `COUNT(*)` for orders.
2. `COUNT(DISTINCT CustomerID)` for customers.
3. `SUM` and `AVG` on `Total_Order_Amount`.
4. `MIN` and `MAX` on `OrderDate`.

Step 4 — Final SQL Solution:

```sql
SELECT COUNT(*)                          AS total_orders,
       COUNT(DISTINCT o.CustomerID)      AS ordering_customers,
       SUM(o.Total_Order_Amount)         AS total_order_value,
       ROUND(AVG(o.Total_Order_Amount), 2) AS avg_order_value,
       MIN(o.OrderDate)                  AS first_order_date,
       MAX(o.OrderDate)                  AS last_order_date
FROM Orders AS o;
```

Step 5 — Line-by-Line Explanation:

- `COUNT(*)` counts rows, whatever is in them.
- `COUNT(DISTINCT o.CustomerID)` counts different customer IDs and ignores NULL. A customer with ten orders counts once.
- `SUM(o.Total_Order_Amount)` adds the amounts and skips NULLs.

- `AVG(o.Total_Order_Amount)` is the sum of non-NULL amounts divided by the number of non-NULL amounts, not by `COUNT(*)`.
- `MIN` and `MAX` on a date give the earliest and latest date.

Step 6 — Sample Output:

total_orders	ordering_customers	total_order_value	avg_order_value	first_order_date	last_order_date
46438	4427	61273574.03	1319.61	2023-01-01	2025-12-28

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 1 row returned.

Step 7 — Why This Solution Works: Every aggregate runs on the order grain, so nothing is duplicated. AVG treats an order with no amount as "unknown" and leaves it out, which is the right reading when the amount was simply not captured. Report how many orders were left out so nobody is surprised.

Step 8 — Common Mistakes:

- Calculating AOV as `SUM / COUNT(*)`. NULL amounts add nothing to the top but still count at the bottom, so this AOV is lower than AVG whenever amounts are missing. Put both side by side:

```
SELECT ROUND(AVG(o.Total_Order_Amount), 2)                    AS aov_known_amounts,
       ROUND(SUM(o.Total_Order_Amount) / NULLIF(COUNT(*), 0), 2) AS aov_all_rows,
       COUNT(*) - COUNT(o.Total_Order_Amount)                 AS orders_without_amount
FROM Orders AS o;
```

- Forgetting that a zero amount is not NULL. AVG includes zeros and they drag AOV down. If 0 means "not keyed", use `AVG(NULLIF(o.Total_Order_Amount, 0))`.
- Reading `ordering_customers` as verified customers. It counts IDs on orders, including any that have no row in `Customers`.

Step 9 — Interview Follow-Up: Leadership wants the same strip for each calendar year side by side. What do you add to the query?

Step 10 — Optimization or Alternative: This is one pass over the table. On a very large table, `COUNT(DISTINCT ...)` is the expensive part. A dashboard that needs speed over exactness can use an approximate distinct count, which PostgreSQL offers through extensions.

Question 11: How much of the catalogue has ever sold?

Difficulty: Easy

Primary Topic: COUNT(*) vs COUNT(column)

Combined Topics: LEFT JOIN, COUNT DISTINCT, Percentage

Business Scenario: The merchandising head suspects the catalogue is far bigger than what customers actually buy. Before a clean-up, she wants four numbers in one row: how many products are listed, how many sale lines exist, how many distinct products have sold at least once, and what share of the catalogue that is.

Problem Statement: Starting from every product, attach its order lines. In one row, return the rows produced by that join, the number of order lines, the number of listed products, the number of products that sold at least once, and the percentage of listed products that sold.

Expected Output:

- `rows_after_join`: rows produced by `Products LEFT JOIN OrderDetails`
- `sale_lines`: order lines matched to a product
- `listed_products`: distinct products in the catalogue
- `products_sold`: distinct products with at least one line
- `pct_sold`: `products_sold` as a percentage of `listed_products`, 2 decimals

Tables Required: `Products`, `OrderDetails`

SQL Concepts Tested: what `COUNT(*)`, `COUNT(column)` and `COUNT(DISTINCT column)` each count, how a LEFT JOIN creates NULLs for unmatched rows, a percentage from two counts.

Step 1 — Understand the Problem: The four numbers come from one LEFT JOIN, read in different ways. A product with five sale lines produces five rows. A product that never sold produces one row with NULLs in every `OrderDetails` column. The trick is knowing which COUNT sees which rows.

Step 2 — Identify the Tables and Relationships: `Products` is the full catalogue, one row per product. `OrderDetails.ProductID` → `Products.ProductID`, one product to many lines. The join starts from `Products` so unsold products are kept.

Step 3 — Build the Query Logic:

1. LEFT JOIN `OrderDetails` to `Products` on `ProductID`.
2. `COUNT(*)` counts every joined row, NULL-padded ones included.
3. `COUNT(od.OrderDetailID)` counts only rows where a line matched.
4. `COUNT(DISTINCT p.ProductID)` and `COUNT(DISTINCT od.ProductID)` count products, listed and sold.
5. Divide sold by listed.

Step 4 — Final SQL Solution:

```sql
SELECT COUNT(*)                                    AS rows_after_join,
       COUNT(od.OrderDetailID)                     AS sale_lines,
       COUNT(DISTINCT p.ProductID)                 AS listed_products,
       COUNT(DISTINCT od.ProductID)                AS products_sold,
       ROUND(100.0 * COUNT(DISTINCT od.ProductID)
             / NULLIF(COUNT(DISTINCT p.ProductID), 0), 2) AS pct_sold
FROM Products AS p
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `COUNT(*)` counts rows, whatever they contain. Every sale line is a row, and every unsold product adds one more NULL-padded row.
- `COUNT(od.OrderDetailID)` counts non-NULL values only. The padded rows have NULL there, so this is exactly the number of sale lines.
- `COUNT(DISTINCT p.ProductID)` counts each product once, however many lines it has.
- `COUNT(DISTINCT od.ProductID)` also skips NULLs, so unsold products are not counted as sold.
- `rows_after_join - sale_lines` is therefore the number of unsold products, a quick consistency check.
- `100.0 *` forces decimal division; `NULLIF` protects an empty catalogue.

Step 6 — Sample Output:

rows_after_join	sale_lines	listed_products	products_sold	pct_sold
174116	168552	22088	16524	74.81

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 1 row returned.

Step 7 — Why This Solution Works: All five numbers come from one pass over the same join, so they are consistent by construction: `listed_products - products_sold` equals `rows_after_join - sale_lines`. The difference between `COUNT(*)` and `COUNT(column)` is exactly the rows the LEFT JOIN padded with NULLs, which here are the products nobody bought.

Step 8 — Common Mistakes:

- Using `COUNT(*)` for sale lines after a LEFT JOIN. Every unsold product adds a phantom line.
- Using an INNER JOIN. `listed_products` silently becomes "products that sold", and the percentage becomes 100.
- Counting `COUNT(od.ProductID)` and calling it products sold. Without DISTINCT it counts lines, so a product sold 40 times counts 40 times:

```sql
SELECT COUNT(od.ProductID)          AS looks_like_products_but_is_lines,
       COUNT(DISTINCT od.ProductID) AS products_sold
FROM Products AS p
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID;
```

Step 9 — Interview Follow-Up: Show the same four numbers per category. Which column do you group by, and does `rows_after_join` still mean anything at that level?

Step 10 — Optimization or Alternative: Two scalar subqueries avoid building the joined rows when you only need the product counts: `(SELECT COUNT(*) FROM Products)` and `(SELECT COUNT(DISTINCT ProductID) FROM OrderDetails)`. That version also counts order lines whose `ProductID` has no catalogue row, which the LEFT JOIN from `Products` would miss, so compare the two when checking data quality.

Question 12: Price and discount statistics per category ID

Difficulty: Easy

Primary Topic: GROUP BY with several aggregates

Combined Topics: ROUND, Conditional COUNT with FILTER, AVG and NULLs

Business Scenario: The pricing team is preparing a quarterly review of category price ranges. For each category code they want the size of the range, the price spread, and how much of it is sold below the printed price (`Market_Price`), so they can spot categories that lean too hard on discounts.

Problem Statement: For each `CategoryID` in `Products`, return the number of products, the lowest, highest and average `Sale_Price`, how many products sell at full price (`Sale_Price = Market_Price`), and the average discount percentage across the category. Round averages to two decimals. Sort by `CategoryID`.

Expected Output:

- `CategoryID`
- `products`: number of products
- `min_price`, `max_price`: lowest and highest `Sale_Price`
- `avg_price`: average `Sale_Price`, 2 decimals
- `full_price_products`: products whose `Sale_Price` equals `Market_Price`
- `avg_discount_pct`: average of `(Market_Price - Sale_Price) / Market_Price * 100`, 2 decimals, ignoring products with no usable `Market_Price`

Tables Required: `Products`

SQL Concepts Tested: GROUP BY, several aggregates in one query, ROUND, COUNT with FILTER, how AVG treats NULL.

Step 1 — Understand the Problem: Each output row is one category, so the product rows are grouped by `CategoryID` and each aggregate summarises its group. Two columns need care. "Full price" is a count of rows meeting a condition, not a count of all rows. The discount percentage divides by `Market_Price`, so a zero or missing printed price must not crash the query or pollute the average.

Step 2 — Identify the Tables and Relationships: Only `Products`, one row per product, each with a `CategoryID`. The team asked for IDs, so no join is needed.

Step 3 — Build the Query Logic:

1. Group by `p.CategoryID`.
2. Compute COUNT, MIN, MAX and AVG of `Sale_Price`.
3. Count full-price products with `COUNT(*) FILTER (WHERE ...)`.
4. Compute each product's discount percentage with `NULLIF` guarding the division, then average it.
5. Round the averages and sort.

Step 4 — Final SQL Solution:

```
SELECT p.CategoryID,
       COUNT(*)                                                AS products,
       MIN(p.Sale_Price)                                       AS min_price,
       MAX(p.Sale_Price)                                       AS max_price,
       ROUND(AVG(p.Sale_Price), 2)                             AS avg_price,
       COUNT(*) FILTER (WHERE p.Sale_Price = p.Market_Price)   AS full_price_products,
       ROUND(AVG(100.0 * (p.Market_Price - p.Sale_Price)
                 / NULLIF(p.Market_Price, 0)), 2)              AS avg_discount_pct
FROM Products AS p
GROUP BY p.CategoryID
ORDER BY p.CategoryID NULLS LAST;
```

Step 5 — Line-by-Line Explanation:

- `GROUP BY p.CategoryID` makes one group per code. Products with a NULL `CategoryID` would form one extra group, sorted last.
- `COUNT(*)` counts all products in the group.
- `COUNT(*) FILTER (WHERE ...)` counts only the rows that pass the condition. It is PostgreSQL syntax; the portable form is `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`.
- `NULLIF(p.Market_Price, 0)` turns a zero printed price into NULL, so that product's discount becomes NULL instead of a division-by-zero error.
- `AVG(...)` skips NULLs, so products with no usable printed price are left out of the average rather than counted as zero.
- `ROUND(..., 2)` keeps the output readable. If your prices were loaded as `float` rather than `numeric`, write `ROUND(AVG(...)::numeric, 2)`, because PostgreSQL has no two-argument ROUND for float.

Step 6 — Sample Output:

categoryid	products	min_price	max_price	avg_price	full_price_products	avg_discount_pct
5001	2264	5.00	3600.00	224.07	1013	10.86
5002	2877	17.00	12500.00	520.61	602	22.79
5003	2040	10.00	4099.00	168.06	901	10.42
5004	3967	5.00	6500.00	302.83	2418	7.26
5005	504	15.00	3299.00	555.33	340	5.50
5006	2326	5.00	1950.00	125.86	1482	6.65
5007	725	5.00	1108.00	134.55	397	6.97
5008	6142	3.00	8184.00	439.84	2624	12.13

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Every column in SELECT is either the grouping column or an aggregate, so each group returns exactly one row. The full-price count and the average discount answer the team's question from two sides: how many products are never discounted, and how deep the discounts go on average across the category.

Step 8 — Common Mistakes:

- Selecting a column that is neither grouped nor aggregated. PostgreSQL stops with an error, because a category has many product names and it cannot pick one:

```
SELECT p.CategoryID, p.Product, COUNT(*) FROM Products AS p GROUP BY p.CategoryID;
ERROR:  column "p.product" must appear in the GROUP BY clause or be used in an aggregate function
```

- Dividing without `NULLIF`. One product with `Market_Price = 0` makes the whole query fail.
- Wrapping the discount in `COALESCE(..., 0)` before averaging. Every product with no usable printed price becomes a "0% discount" and drags the average down.
- Counting full-price products with `COUNT(p.Sale_Price = p.Market_Price)`. COUNT counts non-NULL values, and FALSE is not NULL, so this counts every product.

Step 9 — Interview Follow-Up: The team wants category names instead of IDs, and products whose `CategoryID` has no match in `Category` must still appear. Which join do you use and how do you label them?

Step 10 — Optimization or Alternative: The mean discount hides how it is spread. `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ...)` gives the median discount per category, which is closer to what a typical shopper sees. That is PostgreSQL syntax; MySQL has no direct equivalent.

Question 13: States with at least five customers

Difficulty: Easy

Primary Topic: HAVING

Combined Topics: GROUP BY, NULL group

Business Scenario: Regional sales is deciding where to place field representatives. A rep is only worth assigning to a state with at least five registered customers. They want the qualifying states with the biggest first. Customers with no state on file should be visible as their own line so the team can see how many cannot be placed.

Problem Statement: Count customers per state and keep only states with five or more. Label a missing state as `Unknown`. Sort by customer count, highest first, then by state.

Expected Output:

- `State`: state name, or `Unknown`
- `customer_count`: customers registered in that state

Tables Required: `Customers`

SQL Concepts Tested: GROUP BY, HAVING, WHERE vs HAVING, NULL as a group.

Step 1 — Understand the Problem: "At least five customers" is a condition on a count, and a count only exists after rows are grouped. That is what HAVING is for: it filters groups. WHERE filters single rows before any grouping happens.

Step 2 — Identify the Tables and Relationships: Only `Customers`, one row per customer account. State sits on that row.

Step 3 — Build the Query Logic:

1. Clean the state with TRIM and label NULL as `Unknown`.
2. Group by the cleaned value and count.
3. Keep groups with `COUNT(*) >= 5` in HAVING.
4. Sort by count, then state.

Step 4 — Final SQL Solution:

```sql
SELECT COALESCE(TRIM(c.State), 'Unknown') AS State,
       COUNT(*)                           AS customer_count
FROM Customers AS c
GROUP BY COALESCE(TRIM(c.State), 'Unknown')
HAVING COUNT(*) >= 5
ORDER BY customer_count DESC, State;
```

Step 5 — Line-by-Line Explanation:

- `COALESCE(TRIM(c.State), 'Unknown')` merges `'Kerala'` with `'Kerala '` and turns NULL into a readable label.
- `GROUP BY` repeats the same expression. Grouping by the alias `State` would be risky here: when an alias matches a real column name, PostgreSQL's GROUP BY uses the raw column.
- `COUNT(*)` counts customers in each group.
- `HAVING COUNT(*) >= 5` runs after grouping and drops the small states.
- `ORDER BY customer_count DESC, State` puts the largest first and breaks ties by name.

Step 6 — Sample Output:

state	customer_count
Maharashtra	952
Karnataka	647
Tamil Nadu	500
Delhi	489
Telangana	393
Gujarat	284
Uttar Pradesh	280
West Bengal	258

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 26 rows.

Step 7 — Why This Solution Works: The query runs in this order: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. The count exists from GROUP BY onwards, so HAVING can test it. All NULL states fall into one group, and COALESCE gives that group a name. The `Unknown` line only appears when it has five or more customers itself.

Step 8 — Common Mistakes:

- Putting the count condition in WHERE. WHERE runs before grouping, so there is no count yet:

```
WHERE COUNT(*) >= 5
ERROR:  aggregate functions are not allowed in WHERE
```

- Using HAVING for a row condition such as `HAVING c.Country = 'India'`. Here it fails, because Country is not grouped. Row filters belong in WHERE, which removes rows before grouping.
- Reading `customer_count` as people. It counts accounts, so one person who registered twice counts twice.

Step 9 — Interview Follow-Up: Sales now wants only states where at least five customers have placed an order. What changes, and which table do you need to add?

Step 10 — Optimization or Alternative: Run the query once without HAVING. The full list shows how close the excluded states are to the cut-off and how large the `Unknown` group is:

```sql
SELECT COALESCE(TRIM(c.State), 'Unknown') AS State,
       COUNT(*)                           AS customer_count
FROM Customers AS c
GROUP BY COALESCE(TRIM(c.State), 'Unknown')
ORDER BY customer_count DESC, State;
```

Question 14: Orders per month in 2025

Difficulty: Easy

Primary Topic: DATE_TRUNC grouping

Combined Topics: Date range filter

Business Scenario: Operations plans warehouse staffing month by month and wants to see how 2025 actually went: how many orders each month and the order value they carried. Cancellations and refunds are not recorded in this dataset, so every row in `Orders` counts as a placed order.

Problem Statement: For each month of 2025, return the first day of the month, the number of orders and the total `Total_Order_Amount`. Sort by month.

Expected Output:

- `order_month`: first day of the month, as a date
- `orders`: orders placed that month
- `order_value`: sum of `Total_Order_Amount`

Tables Required: `Orders`

SQL Concepts Tested: DATE_TRUNC, grouping by a derived value, half-open date filter, SUM and NULL.

Step 1 — Understand the Problem: The output grain is one row per calendar month. Each order has to be mapped to its month, and orders in the same month grouped together. A month label must carry the year, or January 2024 and January 2025 end up in one bucket.

Step 2 — Identify the Tables and Relationships: Only `Orders`, one row per order, with `OrderDate` and `Total_Order_Amount`.

Step 3 — Build the Query Logic:

1. Keep 2025 with a half-open range on `OrderDate`.
2. Map each date to the first day of its month with `DATE_TRUNC('month', ...)`.
3. Group by that value, count orders and sum the amount.

Step 4 — Final SQL Solution:

```sql
SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
       COUNT(*)                               AS orders,
       SUM(o.Total_Order_Amount)              AS order_value
FROM Orders AS o
WHERE o.OrderDate >= DATE '2025-01-01'
  AND o.OrderDate <  DATE '2026-01-01'
GROUP BY DATE_TRUNC('month', o.OrderDate)::date
ORDER BY order_month;
```

Step 5 — Line-by-Line Explanation:

- `DATE_TRUNC('month', o.OrderDate)` turns 2025-03-17 into 2025-03-01. It keeps the year, so each month is distinct.
- `::date` casts the result back to a plain date. DATE_TRUNC returns a timestamp, which would print with a midnight time.
- The WHERE range keeps all of 2025 and works the same for DATE or TIMESTAMP columns.
- `COUNT(*)` counts orders. `SUM(o.Total_Order_Amount)` skips NULL amounts, so a month with a missing amount is slightly understated.
- `ORDER BY order_month` sorts by real date, not by text.

Step 6 — Sample Output:

order_month	orders	order_value
2025-01-01	1724	2453879.00
2025-02-01	1521	2054908.37
2025-03-01	1934	2534473.10
2025-04-01	1791	2387828.67
2025-05-01	1956	2512126.33
2025-06-01	2058	2640332.13
2025-07-01	2248	2893332.84
2025-08-01	2501	3307866.17

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Every order in a month truncates to the same date, so GROUP BY puts them together. The value is a real date, so it sorts correctly and charting tools read it as a time axis.

Step 8 — Common Mistakes:

- Grouping by `EXTRACT(MONTH ...)` alone. Without the year filter, every January in the table lands in one row:

```
SELECT EXTRACT(MONTH FROM o.OrderDate) AS month_number,
       COUNT(*)                        AS orders_all_years_mixed
FROM Orders AS o
GROUP BY EXTRACT(MONTH FROM o.OrderDate)
ORDER BY month_number;
```

- Building a text label such as `TO_CHAR(o.OrderDate, 'Mon')` and sorting by it. Apr comes before Jan alphabetically.
- Filtering with `EXTRACT(YEAR FROM o.OrderDate) = 2025`. The result is right, but the function on the column stops an index from being used.

Step 9 — Interview Follow-Up: If a month had no orders, it would be missing from this output instead of showing 0. How would you make sure all twelve months appear?

Step 10 — Optimization or Alternative: On MySQL, which has no DATE_TRUNC, use `DATE_FORMAT(o.OrderDate, '%Y-%m-01')`. To show every month even when it is empty, generate the twelve months with `generate_series` and LEFT JOIN this result to it.

Question 15: Product count for every category

Difficulty: Easy

Primary Topic: LEFT JOIN with GROUP BY

Combined Topics: COUNT(column) after a LEFT JOIN

Business Scenario: The catalogue team is cleaning up the app's category menu. An empty category leads shoppers to a blank page, so they want every category with its product count, empty ones included and listed first.

Problem Statement: For every row in `Category`, return the number of products assigned to it. A category with no products must appear with 0. Sort by product count ascending, then `CategoryID`.

Expected Output:

- `CategoryID`
- `CategoryName`
- `product_count`: products in that category, 0 when none

Tables Required: `Category`, `Products`

SQL Concepts Tested: LEFT JOIN, table grain, GROUP BY, COUNT(*) vs COUNT(column).

Step 1 — Understand the Problem: "Every category" means `Category` drives the result. An INNER JOIN keeps only categories with a matching product, so empty ones would vanish. The count also needs care, because of what a LEFT JOIN produces for a category with no match.

Step 2 — Identify the Tables and Relationships: `Category` has one row per category. `Products` has one row per product, each with a `CategoryID` (`Products.CategoryID` → `Category.CategoryID`, one category to many products). After the join, the grain is one row per product, plus one row with NULL product columns for each category that has no products.

Step 3 — Build the Query Logic:

1. Start from `Category` and LEFT JOIN `Products` on `CategoryID`.
2. Group back to one row per category.
3. Count `p.ProductID`, not `*`.

Step 4 — Final SQL Solution:

```sql
SELECT cat.CategoryID,
       cat.CategoryName,
       COUNT(p.ProductID) AS product_count
FROM Category AS cat
LEFT JOIN Products AS p ON p.CategoryID = cat.CategoryID
GROUP BY cat.CategoryID, cat.CategoryName
ORDER BY product_count, cat.CategoryID;
```

Step 5 — Line-by-Line Explanation:

- `LEFT JOIN Products` adds one row per matching product. A category with no products keeps one row, with NULL in every `p` column.
- `GROUP BY cat.CategoryID, cat.CategoryName` collapses the product rows back to one row per category.
- `COUNT(p.ProductID)` counts non-NULL product IDs. The padding row of an empty category has a NULL `ProductID`, so it counts as 0.
- `ORDER BY product_count, cat.CategoryID` puts empty categories at the top.

Step 6 — Sample Output:

categoryid	categoryname	product_count
5013	Electronics Accessories	0
5011	Fruits & Vegetables	160
5010	Eggs, Meat & Fish	334
5005	Baby Care	504
5009	Beverages	709
5007	Bakery, Cakes & Dairy	725
5003	Foodgrains, Oil & Masala	2040
5001	Cleaning & Household	2264

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: The LEFT JOIN keeps every category. Counting a right-hand column counts real matches only, so an empty category shows 0. `ProductID` is the primary key, never NULL on a real product, so every real product is counted.

Step 8 — Common Mistakes:

- Using `COUNT(*)`. It counts the NULL padding row, so an empty category shows 1 product, the exact case this report exists to find:

```sql
SELECT cat.CategoryID,
       cat.CategoryName,
       COUNT(*) AS product_count_wrong
FROM Category AS cat
LEFT JOIN Products AS p ON p.CategoryID = cat.CategoryID
GROUP BY cat.CategoryID, cat.CategoryName
ORDER BY product_count_wrong, cat.CategoryID;
```

- Using an INNER JOIN. Empty categories disappear, and the report says the menu is clean when it is not.
- Expecting this to catch every problem. A product whose `CategoryID` has no row in `Category` is counted nowhere here; start from `Products` to find those.

Step 9 — Interview Follow-Up: The team also wants to know how many products in each category have actually been ordered. Which table do you add, and how do you stop the extra join from inflating `product_count`?

Step 10 — Optimization or Alternative: A correlated subquery gives the same result and cannot be inflated by later joins: `(SELECT COUNT(*) FROM Products AS p WHERE p.CategoryID = cat.CategoryID)` in the SELECT list. Here `COUNT(*)` is safe, because the subquery only sees real product rows.

Question 16: December 2025 orders with customer names

Difficulty: Easy

Primary Topic: INNER JOIN

Combined Topics: CONCAT_WS, Date range filter

Business Scenario: Customer support is calling customers for feedback on orders placed in December 2025. Each agent's call sheet needs the order, the customer's full name and city, and the order amount, so the agent can greet the customer by name and confirm the right order.

Problem Statement: List every December 2025 order that belongs to a known customer, with the customer's full name and city. Sort by `OrderDate`, then `OrderID`.

Expected Output:

- `OrderID`
- `OrderDate`

- `customer_name`: first and last name joined with a space
- `City`: the customer's city, spaces trimmed
- `Total_Order_Amount`

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: INNER JOIN, join keys, CONCAT_WS with NULL parts, half-open date range.

Step 1 — Understand the Problem: The order facts live in `Orders` and the name and city live in `Customers`. A join puts them on one row. An order whose customer cannot be found is no use on a call sheet, so an INNER JOIN, which keeps only matching rows, fits this request.

Step 2 — Identify the Tables and Relationships: `Orders` has one row per order. `Customers` has one row per customer. `Orders.CustomerID` → `Customers.CustomerID` is many orders to one customer. Each order matches at most one customer, so the join cannot duplicate orders: the result is still one row per order.

Step 3 — Build the Query Logic:

1. Filter `Orders` to December 2025 with a half-open range.
2. INNER JOIN `Customers` on `CustomerID`.
3. Build the full name with CONCAT_WS and trim the city.

Step 4 — Final SQL Solution:

```sql
SELECT o.OrderID,
       o.OrderDate,
       CONCAT_WS(' ', c.FirstName, c.LastName) AS customer_name,
       TRIM(c.City)                            AS City,
       o.Total_Order_Amount
FROM Orders AS o
JOIN Customers AS c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= DATE '2025-12-01'
  AND o.OrderDate <  DATE '2026-01-01'
ORDER BY o.OrderDate, o.OrderID;
```

Step 5 — Line-by-Line Explanation:

- `JOIN Customers AS c ON c.CustomerID = o.CustomerID` pairs each order with its customer. `JOIN` on its own means INNER JOIN.
- `CONCAT_WS(' ', c.FirstName, c.LastName)` joins the parts with a space and skips NULL parts, so a customer with no last name still gets a name.
- `TRIM(c.City)` removes stray spaces typed at sign-up.
- Every column carries its table alias, which avoids "ambiguous column" errors.

Step 6 — Sample Output:

orderid	orderdate	customer_name	city	total_order_amount
5043464	2025-12-01	Komal Rao	Mysuru	80.00
5043465	2025-12-01	Pooja Singh	New Delhi	785.00
5043466	2025-12-01	Sanjay Gill	Noida	1420.00
5043467	2025-12-01	Kavya Chatterjee	Lucknow	1338.00
5043468	2025-12-01	Navya Rao	Chennai	1593.00
5043469	2025-12-01	Imran Rawat	Chennai	2733.00
5043470	2025-12-01	Shalini Kaur	New Delhi	329.00
5043471	2025-12-01	Ankit Chatterjee	Delhi	2982.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 2930 rows.

Step 7 — Why This Solution Works: The join key is unique on the `Customers` side, so the row count equals the number of December orders that have a known customer. Orders with a NULL `CustomerID`, or an ID missing from `Customers`, are dropped by the INNER JOIN. Count them so support knows how many calls are missing:

```sql
SELECT COUNT(*) AS december_orders_without_customer
FROM Orders AS o
LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= DATE '2025-12-01'
  AND o.OrderDate <  DATE '2026-01-01'
  AND c.CustomerID IS NULL;
```

Step 8 — Common Mistakes:

- Building the name with `c.FirstName || ' ' || c.LastName`. If either part is NULL, the whole name becomes NULL and the agent sees a blank.
- Filtering with `EXTRACT(MONTH FROM o.OrderDate) = 12`. That returns every December in the table, not only 2025.
- Assuming an INNER JOIN keeps all orders. It silently removes unmatched ones, and nothing in the output tells you.

Step 9 — Interview Follow-Up: Support wants every December order on the sheet, with `Unknown customer` where no match exists. What do you change?

Step 10 — Optimization or Alternative: Indexes on `Orders(OrderDate)` and on `Customers(CustomerID)` let PostgreSQL find the month's orders and look up each customer cheaply. If `CustomerID` is the primary key, the second index already exists.

Question 17: Orders with payment type and shipper

Difficulty: Easy

Primary Topic: Multi-table LEFT JOIN

Combined Topics: COALESCE labels, Date anchor from the data

Business Scenario: Every morning the operations desk reviews the most recent week of orders: how each was paid and which courier took it. An order with a missing or unrecognised payment or shipper code is exactly what they need to chase, so it must appear on the list, not vanish from it.

Problem Statement: For orders placed in the last 7 days covered by the data, return the payment type and shipper name. Show `Unknown` where the code is NULL or matches nothing. Sort newest first, then by `OrderID` descending.

Expected Output:

- `OrderID`
- `OrderDate`
- `payment_type`: `PaymentType`, or `Unknown`
- `shipper_name`: shipper `CompanyName`, or `Unknown`

Tables Required: `Orders`, `Payments`, `Shippers`

SQL Concepts Tested: chained LEFT JOINs, COALESCE labels, a scalar subquery as a date anchor.

Step 1 — Understand the Problem: `Orders` stores codes; the names live in two lookup tables. Every order in the window must survive both lookups, even with a blank or wrong code. "Last 7 days" means the last 7 days in the data, because an extract is often older than today.

Step 2 — Identify the Tables and Relationships: `Orders` (one row per order) → `Payments` on `PaymentID` and → `Shippers` on `ShipperID`. Both lookups are many orders to one row, so neither join can duplicate an order.

Step 3 — Build the Query Logic:

1. Find the latest `OrderDate` in the table and step back 6 days. With the latest day itself, that is 7 days.
2. LEFT JOIN each lookup table so unmatched orders stay.
3. Replace missing names with `Unknown`.

Step 4 — Final SQL Solution:

```sql
SELECT o.OrderID,
       o.OrderDate,
       COALESCE(pay.PaymentType, 'Unknown') AS payment_type,
       COALESCE(sh.CompanyName, 'Unknown')  AS shipper_name
FROM Orders AS o
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
LEFT JOIN Shippers AS sh  ON sh.ShipperID  = o.ShipperID
WHERE o.OrderDate >= (SELECT MAX(latest.OrderDate)::date - 6 FROM Orders AS latest)
ORDER BY o.OrderDate DESC, o.OrderID DESC;
```

Step 5 — Line-by-Line Explanation:

- `(SELECT MAX(latest.OrderDate)::date - 6 ...)` is a scalar subquery returning one value, the first day of the window. DATE minus an integer is a DATE. This anchor moves with the data; `CURRENT_DATE` would return nothing on an old extract.
- `LEFT JOIN Payments` keeps every order. If `PaymentID` is NULL or unknown, the `pay` columns are NULL. `LEFT JOIN Shippers` does the same for couriers.
- `COALESCE(..., 'Unknown')` turns those NULL names into a label the desk can act on.

Step 6 — Sample Output:

orderid	orderdate	payment_type	shipper_name
5046393	2025-12-28	UPI	BlueLine Express
5046392	2025-12-28	Credit Card	BlueLine Express
5046391	2025-12-28	Credit Card	XpressHaul
5046390	2025-12-28	Credit Card	BlueLine Express
5046389	2025-12-28	Debit Card	MetroDash Hyperlocal
5046388	2025-12-28	UPI	MetroDash Hyperlocal
5046387	2025-12-28	UPI	Bharat Parcel Service
5046386	2025-12-28	Credit Card	MetroDash Hyperlocal

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 718 rows.

Step 7 — Why This Solution Works: The driving table is `Orders`, and both joins are LEFT, so the row count equals the number of orders in the window, whatever state their codes are in. The lookups are unique on their keys, so no order appears twice.

Step 8 — Common Mistakes:

- Using INNER JOINs. Any order with a NULL or unmatched code disappears, which hides the very rows the desk wants. Compare the counts over the whole table:

```
SELECT (SELECT COUNT(*) FROM Orders) AS all_orders,
       COUNT(*)                      AS orders_left_after_inner_joins
FROM Orders AS o
JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
JOIN Shippers AS sh  ON sh.ShipperID  = o.ShipperID;
```

- Applying COALESCE to the code instead of the name. `COALESCE(o.ShipperID, 0)` does nothing for an ID that matches no shipper.
- Anchoring on `CURRENT_DATE`. On historical data the window is empty.

Step 9 — Interview Follow-Up: The desk wants `Missing code` and `Unrecognised code` as separate labels. How do you tell the two apart in SQL?

Step 10 — Optimization or Alternative: Compute the anchor once in a CTE (`WITH bounds AS (SELECT MAX(OrderDate)::date - 6 AS start_date FROM Orders)`) when several filters reuse it.

Question 18: Customers with no order in the last 90 days

Difficulty: Easy

Primary Topic: LEFT JOIN anti-join

Combined Topics: Filter in the ON clause vs WHERE, Date anchored to the data

Business Scenario: The retention team sends a "we miss you" offer to customers who have gone quiet. The list is every registered customer with no order in the last 90 days of data, whether they went quiet or never ordered at all; the team splits the two groups later. "Last 90 days" ends on the latest `OrderDate` in the table, not on today's date, because the extract may be weeks old.

Problem Statement: Return every customer who has no order dated within the 90 days ending on the latest `OrderDate` in `Orders`. Show their name, email and sign-up date. Sort by `DateEntered`, then `CustomerID`.

Expected Output:

- `CustomerID`
- `customer_name`: first and last name
- `Email`
- `DateEntered`

Tables Required: `Customers`, `Orders`

SQL Concepts Tested: LEFT JOIN plus IS NULL to find missing matches, where the extra join condition must go, anchoring a date window with a scalar subquery.

Step 1 — Understand the Problem: This is an anti-join: keep customers for whom a matching row does NOT exist. The match is not "any order" but "an order in the window". The window runs from 89 days before the latest order date to that date, which is 90 calendar days counting both ends.

Step 2 — Identify the Tables and Relationships: `Customers` is the full list. `Orders.CustomerID` → `Customers.CustomerID`, one customer to many orders. The anchor date comes from `Orders` itself.

Step 3 — Build the Query Logic:

1. LEFT JOIN `Orders` on the customer AND on the date window, both inside `ON`.
2. Keep rows where the joined `o.OrderID` IS NULL: no order in the window.
3. Sort.

Step 4 — Final SQL Solution:

```
SELECT c.CustomerID,
       CONCAT_WS(' ', c.FirstName, c.LastName) AS customer_name,
       c.Email,
       c.DateEntered
FROM Customers AS c
LEFT JOIN Orders AS o
       ON o.CustomerID = c.CustomerID
      AND o.OrderDate >= (SELECT MAX(OrderDate) FROM Orders) - 89
WHERE o.OrderID IS NULL
ORDER BY c.DateEntered, c.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `LEFT JOIN Orders ... ON` keeps every customer. A customer with no qualifying order gets one row with NULLs in every `o` column.
- `AND o.OrderDate >= (...) - 89` sits in the ON clause, so it decides which orders count as a match. Older orders simply do not match.
- `(SELECT MAX(OrderDate) FROM Orders)` is a scalar subquery: it returns one value, the anchor. Subtracting an integer from a DATE moves

it back that many days.

- `WHERE o.OrderID IS NULL` keeps only customers with no match. `OrderID` is the primary key, so it is NULL only when no row was joined.

Step 6 — Sample Output:

customerid	customer_name	email	dateentered
104417	Kabir Pandey	kabir.pandey@gmail.com	2022-10-04
100379	Arnav Pandey	arnav.pandey@rediffmail.com	2022-10-06
101748	Aditi Nair	aditi.nair@gmail.com	2022-10-06
104927	Deepak Kaur	deepak.kaur@gmail.com	2022-10-06
100819	Vikram Negi	vikram.negi@tcs.com	2022-10-09
101046	Anika Jain	anika.jain4677@gmail.com	2022-10-09
102608	Payal Ansari	payal.ansari@gmail.com	2022-10-11
103633	Kritika Kamath	kritika.kamath7133@gmail.com	2022-10-11

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 2414 rows.

Step 7 — Why This Solution Works: The LEFT JOIN produces at least one row per customer, and the date condition in ON means recent orders are the only ones that can fill the `o` columns. A customer whose orders are all older than the window keeps a NULL row and passes the IS NULL test. A customer with two recent orders produces two joined rows, but neither has a NULL `OrderID`, so both are filtered out and the customer is never duplicated in the result.

Step 8 — Common Mistakes:

- Moving the date condition to WHERE. Old orders now match in the join, so the customer gets non-NULL rows and drops out, even though they have not ordered recently. The anti-join quietly becomes "customers with no orders at all":

```sql
SELECT COUNT(*) AS customers_found_wrong
FROM Customers AS c
LEFT JOIN Orders AS o ON o.CustomerID = c.CustomerID
WHERE o.OrderID IS NULL
  AND (o.OrderDate IS NULL OR o.OrderDate >= (SELECT MAX(OrderDate) FROM Orders) - 89);
```

- Using `CURRENT_DATE` as the anchor. On a historical extract every customer looks inactive.
- Writing `o.OrderDate = NULL` instead of testing the key with IS NULL. Comparisons with NULL are never true.

Step 9 — Interview Follow-Up: Rewrite the same logic with NOT EXISTS. Which version would you choose for a large table, and does it change the result?

Step 10 — Optimization or Alternative: `WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID AND o.OrderDate >= ...)` states the intent directly and PostgreSQL plans it as an anti-join too. An index on `Orders(CustomerID, OrderDate)` serves both forms.

Question 19: Units sold per product

Difficulty: Easy

Primary Topic: JOIN with SUM at product grain

Combined Topics: Order lines vs units, COUNT DISTINCT

Business Scenario: Merchandising decides which products get the end-of-aisle display in the app. They look at three numbers per product: units sold, the number of order lines, and the number of distinct orders. A product bought in bulk by a few customers and one bought one at a time by many need different promotions. Cancellations and returns are not recorded here, so every order line counts as sold.

Problem Statement: For each product that appears in `OrderDetails`, return units sold, order lines and distinct orders. Sort by units sold, highest first, then `ProductID`.

Expected Output:

- `ProductID`
- `Product`
- `units_sold`: sum of `Quantity`
- `order_lines`: rows in `OrderDetails` for the product
- `orders`: distinct `OrderID`s containing the product

Tables Required: `OrderDetails`, `Products`

SQL Concepts Tested: table grain, INNER JOIN, SUM vs COUNT vs COUNT DISTINCT, GROUP BY.

Step 1 — Understand the Problem: Start with grain. One `OrderDetails` row is one product line in one order, with a `Quantity`. "Units" adds up quantities. "Order lines" counts rows. "Orders" counts distinct order IDs. They are three different numbers, and mixing them up is the usual error.

Step 2 — Identify the Tables and Relationships: `OrderDetails.ProductID` → `Products.ProductID`, many lines to one product. Each line matches at most one product, so joining lines to products does not duplicate lines. The result stays at line grain until we group it.

Step 3 — Build the Query Logic:

1. Join each line to its product to get the name.
2. Group by product.
3. `SUM(Quantity)` for units, `COUNT(*)` for lines, `COUNT(DISTINCT OrderID)` for orders.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       SUM(od.Quantity)           AS units_sold,
       COUNT(*)                   AS order_lines,
       COUNT(DISTINCT od.OrderID) AS orders
FROM OrderDetails AS od
JOIN Products AS p ON p.ProductID = od.ProductID
GROUP BY p.ProductID, p.Product
ORDER BY units_sold DESC NULLS LAST, p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `JOIN Products` adds the name. Lines whose `ProductID` has no product row are dropped.
- `GROUP BY p.ProductID, p.Product` makes one row per product. Grouping by the ID keeps two products with the same name apart.
- `SUM(od.Quantity)` adds units and skips NULL quantities.
- `COUNT(*)` counts lines. `COUNT(DISTINCT od.OrderID)` counts each order once, even if the product appears on two lines of it.

Step 6 — Sample Output:

productid	product	units_sold	order_lines	orders
5733	Naturally Long Shikakai Shampoo	2493	1302	1302
15197	Nachoz - Cheese	1767	957	957
13924	Vitamin C, E & Hyaluronic Acid De Pigmentation Cream	1665	927	927
12002	Sugar - Castor	1626	875	875
19725	Ultimate Paneer	1525	841	841
3460	Candy - Amla Salted	1389	750	750
20091	Bellar Dal/Avarebele - Split	1385	721	721
21577	Buffalo Ghee/Tuppa - Special Grade with Milk Fat	1378	754	754

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 16524 rows.

Step 7 — Why This Solution Works: Every aggregate runs on line-grain rows that were never duplicated, so the totals match what `OrderDetails` holds. Normally `order_lines` equals `orders`, because one order lists a product once. When `order_lines` is higher, the same product sits on two lines of one order, which usually means a duplicate load. This check lists them:

```sql
SELECT od.ProductID,
       COUNT(*)                   AS order_lines,
       COUNT(DISTINCT od.OrderID) AS orders
FROM OrderDetails AS od
GROUP BY od.ProductID
HAVING COUNT(*) > COUNT(DISTINCT od.OrderID)
ORDER BY od.ProductID;
```

Step 8 — Common Mistakes:

- Using `COUNT(*)` as "units". A line with quantity 6 counts as 1.
- Calling `COUNT(*)` "orders". Duplicate lines inflate it; `COUNT(DISTINCT od.OrderID)` does not.
- Grouping by `p.Product` only. Two different products with the same name merge into one row.

Step 9 — Interview Follow-Up: Merchandising also wants products that sold nothing, shown with 0 units. What changes in the FROM clause and in the aggregates?

Step 10 — Optimization or Alternative: To see how many lines the INNER JOIN dropped, LEFT JOIN `Products` and count lines where `p.ProductID IS NULL`. On a large table, aggregating `OrderDetails` by `ProductID` first and then joining the small result to `Products` does less join work.

Question 20: Days to ship and days to deliver

Difficulty: Easy

Primary Topic: Date arithmetic

Combined Topics: NULL propagation, Casting to date

Business Scenario: The logistics team is reviewing November 2025 order by order. For each order they want two numbers: how many days the warehouse took to hand it to the courier, and how many days the customer waited from placing the order to receiving it.

Problem Statement: For every order placed in November 2025, return the three dates plus `days_to_ship` (order to ship) and `days_to_deliver` (order to delivery) in whole days. Missing dates must give NULL, not a made-up number. Sort by `OrderDate`, then `OrderID`.

Expected Output:

- `OrderID`
- `OrderDate`, `ShipDate`, `DeliveryDate`
- `days_to_ship`: `ShipDate` minus `OrderDate`, in days
- `days_to_deliver`: `DeliveryDate` minus `OrderDate`, in days

Tables Required: `Orders`

SQL Concepts Tested: date subtraction, casting to DATE, NULL in arithmetic, half-open date range.

Step 1 — Understand the Problem: Each number is a difference between two dates on the same order row. Two details matter: what type the subtraction returns, and what happens when one of the dates is missing.

Step 2 — Identify the Tables and Relationships: Only `Orders`, one row per order. All three dates are on the order row.

Step 3 — Build the Query Logic:

1. Filter to November 2025 with a half-open range.
2. Cast each date to DATE and subtract, which gives an integer number of days.
3. Leave NULL alone; it marks an order without that event.

Step 4 — Final SQL Solution:

```sql
SELECT o.OrderID,
       o.OrderDate,
       o.ShipDate,
       o.DeliveryDate,
       o.ShipDate::date     - o.OrderDate::date AS days_to_ship,
       o.DeliveryDate::date - o.OrderDate::date AS days_to_deliver
FROM Orders AS o
WHERE o.OrderDate >= DATE '2025-11-01'
  AND o.OrderDate <  DATE '2025-12-01'
ORDER BY o.OrderDate, o.OrderID;
```

Step 5 — Line-by-Line Explanation:

- `o.ShipDate::date - o.OrderDate::date` subtracts two DATEs. In PostgreSQL that returns an integer: the number of days between them.
- The `::date` casts change nothing when the columns are already DATE. If they are TIMESTAMPs, the casts drop the time of day so the result is still whole calendar days.
- If either date is NULL, the result is NULL. Arithmetic with NULL always gives NULL.

Step 6 — Sample Output:

orderid	orderdate	shipdate	deliverydate	days_to_ship	days_to_deliver
5039735	2025-11-01	2025-11-01	2025-11-04	0	3
5039736	2025-11-01	2025-11-05	2025-11-09	4	8
5039737	2025-11-01	2025-11-01	2025-11-03	0	2
5039738	2025-11-01	2025-11-02	2025-11-02	1	1
5039739	2025-11-01	2025-11-04	2025-11-08	3	7
5039740	2025-11-01	2025-11-01	2025-11-06	0	5
5039741	2025-11-01	2025-11-05	2025-11-07	4	6
5039742	2025-11-01	2025-11-04	2025-11-07	3	6

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 3732 rows.

Step 7 — Why This Solution Works: Casting both sides to DATE makes the result an integer for either column type, so it can be averaged, compared with a threshold or charted directly. NULL results stay honest: an order not yet delivered has no delivery time. A negative value means the dates are in an impossible order (shipped before it was ordered) and should be raised as a data issue, not averaged.

Step 8 — Common Mistakes:

- Subtracting TIMESTAMPs without casting. The result is an INTERVAL such as `3 days 16:00:00`, which does not compare cleanly with a number of days:

```sql
SELECT t.ordered_at,
       t.delivered_at,
       t.delivered_at - t.ordered_at             AS raw_difference,
       t.delivered_at::date - t.ordered_at::date AS calendar_days
FROM (VALUES (TIMESTAMP '2025-03-01 18:00', TIMESTAMP '2025-03-05 10:00')) AS t(ordered_at, delivered_at);
```

- Filling a missing delivery date with `COALESCE(o.DeliveryDate, CURRENT_DATE)`. It invents a delivery that never happened.

- Writing `DATEDIFF(...)`. PostgreSQL has no DATEDIFF. MySQL writes `DATEDIFF(DeliveryDate, OrderDate)` and SQL Server writes `DATEDIFF(day, OrderDate, DeliveryDate)`.

Step 9 — Interview Follow-Up: Logistics wants the share of November orders delivered within 7 days of ordering. How do you treat orders with no delivery date or a negative gap?

Step 10 — Optimization or Alternative: `days_to_deliver - days_to_ship` gives the time the parcel spent with the courier, which separates warehouse delays from courier delays. For hours rather than days, use `EXTRACT(EPOCH FROM delivered_at - ordered_at) / 3600` on timestamp columns.

Question 21: Top five customers by number of orders

Difficulty: Easy

Primary Topic: GROUP BY with ORDER BY and LIMIT

Combined Topics: FETCH FIRST ... WITH TIES, LEFT JOIN for names

Business Scenario: The CRM team is sending thank-you hampers to its most frequent buyers, meaning the most orders placed, regardless of value. The budget covers five, but if customers tie for fifth place, the team would rather send an extra hamper than leave one out.

Problem Statement: Return the customers with the five highest order counts, including everyone tied at fifth place. Show `Unknown customer` when the `CustomerID` on the orders has no match in `Customers`. Sort by orders placed, highest first, then `CustomerID`.

Expected Output:

- `CustomerID`
- `customer_name`: first and last name, or `Unknown customer`
- `orders_placed`: number of orders

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: GROUP BY, ORDER BY with LIMIT, FETCH FIRST ... WITH TIES, LEFT JOIN, table grain.

Step 1 — Understand the Problem: Count orders per customer, sort, and cut at five. `LIMIT 5` cuts at exactly five rows, even in the middle of a tie, but the team's rule keeps everyone tied at fifth place.

Step 2 — Identify the Tables and Relationships: `Orders` (one row per order) → `Customers` (one row per customer) on `CustomerID`, many orders to one customer. Attaching the name repeats nothing, so `COUNT(*)` still counts orders.

Step 3 — Build the Query Logic:

1. LEFT JOIN `Customers` so orders with an unknown customer stay.
2. Group by customer and count orders.
3. Keep five rows plus ties with `FETCH FIRST 5 ROWS WITH TIES`.
4. Re-sort the kept rows with `CustomerID` as a tiebreaker.

Step 4 — Final SQL Solution:

```sql
SELECT top5.CustomerID,
       top5.customer_name,
       top5.orders_placed
FROM (
    SELECT o.CustomerID,
           CASE WHEN c.CustomerID IS NULL THEN 'Unknown customer'
                ELSE CONCAT_WS(' ', c.FirstName, c.LastName)
           END      AS customer_name,
           COUNT(*) AS orders_placed
    FROM Orders AS o
    LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
    GROUP BY o.CustomerID, c.CustomerID, c.FirstName, c.LastName
    ORDER BY orders_placed DESC
    FETCH FIRST 5 ROWS WITH TIES
) AS top5
ORDER BY top5.orders_placed DESC, top5.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `LEFT JOIN Customers` keeps orders whose `CustomerID` has no customer row; `c.CustomerID IS NULL` then picks the `Unknown customer` label.
- `GROUP BY o.CustomerID, ...` makes one row per customer. The name columns are listed because SELECT uses them.
- `FETCH FIRST 5 ROWS WITH TIES` keeps five rows, plus any further rows with the same count as the fifth.
- The outer `ORDER BY ... top5.CustomerID` fixes the order of tied customers. It cannot go inside, because WITH TIES compares every ORDER BY key, and a unique ID would remove all ties.

Step 6 — Sample Output:

customerid	customer_name	orders_placed
104801	Yash Ansari	192
102192	Rekha Sinha	188

102422	Tarun Kapoor	187
100092	Jyoti Sheikh	182
101237	Pooja Pillai	151

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 5 rows returned.

Step 7 — Why This Solution Works: The join does not repeat orders, so each count is exact. WITH TIES decides who qualifies and the outer sort decides the listing order, so the result is fair and stable between runs.

Step 8 — Common Mistakes:

- Using `LIMIT 5` with a tiebreaker. The list is stable, but a customer tied at fifth place is dropped without anyone noticing:

```sql
SELECT o.CustomerID, COUNT(*) AS orders_placed
FROM Orders AS o
GROUP BY o.CustomerID
ORDER BY orders_placed DESC, o.CustomerID
LIMIT 5;
```

- Using an INNER JOIN. A busy `CustomerID` with no customer row disappears, and the sixth customer moves into a slot they did not earn.

Step 9 — Interview Follow-Up: The team now wants the top five by total order value within each state. Which feature lets you rank inside each state?

Step 10 — Optimization or Alternative: `RANK() OVER (ORDER BY COUNT(*) DESC)` in a subquery, filtered to `rank <= 5`, gives the same tie-aware result and is the usual form on MySQL, which has no WITH TIES.

Question 22: Orders with more than eight line items

Difficulty: Easy

Primary Topic: HAVING on counts

Combined Topics: COUNT vs COUNT DISTINCT, SUM

Business Scenario: The fulfilment centre sends orders with more than eight line items to a separate packing station with bigger cartons. The station supervisor wants to see those orders along with how many different products and how many units each one holds, so they can plan staffing for the station.

Problem Statement: For every order with more than eight rows in `OrderDetails`, return the number of lines, the number of distinct products and the total units. Sort by line items, highest first, then `OrderID`.

Expected Output:

- `OrderID`
- `line_items`: rows in `OrderDetails` for the order
- `distinct_products`: distinct `ProductID`s in the order
- `total_units`: sum of `Quantity`

Tables Required: `OrderDetails`

SQL Concepts Tested: GROUP BY, HAVING, COUNT(*) vs COUNT(DISTINCT), SUM, line grain.

Step 1 — Understand the Problem: One `OrderDetails` row is one product line inside one order. Grouping lines by `OrderID` rebuilds each order's basket. The cut-off is on a count per order, so it goes in HAVING.

Step 2 — Identify the Tables and Relationships: Only `OrderDetails`, at line grain; `OrderID` ties lines to their order (one order to many lines). `Orders` is not needed.

Step 3 — Build the Query Logic:

1. Group lines by `OrderID`.
2. Count lines, count distinct products, sum quantities.
3. Keep groups where the line count is above eight.

Step 4 — Final SQL Solution:

```sql
SELECT od.OrderID,
       COUNT(*)                    AS line_items,
       COUNT(DISTINCT od.ProductID) AS distinct_products,
       SUM(od.Quantity)            AS total_units
FROM OrderDetails AS od
GROUP BY od.OrderID
HAVING COUNT(*) > 8
ORDER BY line_items DESC, od.OrderID;
```

Step 5 — Line-by-Line Explanation:

- `COUNT(*)` counts lines in each order.
- `COUNT(DISTINCT od.ProductID)` counts different products, so a product loaded twice counts once.
- `SUM(od.Quantity)` adds units and skips NULL quantities.

- `HAVING COUNT(*) > 8` repeats the aggregate instead of the alias. PostgreSQL resolves HAVING before SELECT names exist, so `HAVING line_items > 8` fails with `column "line_items" does not exist`. MySQL accepts the alias.

Step 6 — Sample Output:

orderid	line_items	distinct_products	total_units
5002509	11	10	18
5005649	11	10	23
5017442	11	10	17
5022522	11	10	41
5033342	11	11	12
5000009	10	10	21
5000094	10	10	14
5000098	10	10	26

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 1799 rows.

Step 7 — Why This Solution Works: All three numbers are computed from the same group of lines, so they describe the same basket. When `distinct_products` is lower than `line_items`, the same product appears on two lines of one order. That is usually a duplicate load, and it means the order may have been routed to the big-carton station by mistake. If the business rule is really "more than eight different products", filter on the distinct count instead:

```sql
SELECT od.OrderID,
       COUNT(*)                    AS line_items,
       COUNT(DISTINCT od.ProductID) AS distinct_products
FROM OrderDetails AS od
GROUP BY od.OrderID
HAVING COUNT(DISTINCT od.ProductID) > 8
ORDER BY line_items DESC, od.OrderID;
```

Step 8 — Common Mistakes:

- Treating units and lines as the same thing. An order with one line of 12 units is a single line item and should not go to the big-carton station.
- Joining to `Orders` "for completeness" and then summing `Total_Order_Amount`. Each order would be counted once per line.
- Putting `COUNT(*) > 8` in WHERE. WHERE filters single lines before they are grouped, so it cannot see a count.

Step 9 — Interview Follow-Up: How would you list the duplicated lines themselves, meaning the same order, product, quantity and supplier appearing more than once?

Step 10 — Optimization or Alternative: An index on `OrderDetails(OrderID)` lets PostgreSQL read lines already grouped by order. If the station also needs order dates, aggregate here first and join the result to `Orders`, which keeps the join one-to-one.

Question 23: Chocolate or cookie products under 250, excluding sugar-free

Difficulty: Easy

Primary Topic: AND / OR / NOT precedence

Combined Topics: ILIKE, Parentheses

Business Scenario: Merchandising is building a "Sweet treats under 250" banner for Diwali. It should show products from the chocolate and cookie sub-categories priced below 250. Sugar-free products are going into a separate wellness banner, so they must be left out of this one.

Problem Statement: Return products whose `Sub_Category` contains "chocolate" or "cookie" (any case), whose `Sale_Price` is below 250, and whose name does not mark them as sugar-free. Sort by `Sale_Price`, then `ProductID`.

Expected Output:

- `ProductID`
- `Product`
- `Sub_Category`
- `Sale_Price`

Tables Required: `Products`

SQL Concepts Tested: operator precedence of NOT, AND and OR, parentheses, ILIKE.

Step 1 — Understand the Problem: The request has three conditions: (chocolate OR cookie), AND price below 250, AND NOT sugar-free. The OR must be settled first and the result combined with the other two. SQL evaluates NOT first, then AND, then OR, so the grouping has to be written out with parentheses.

Step 2 — Identify the Tables and Relationships: Only `Products`. Sub-category, price and name are all on the product row.

Step 3 — Build the Query Logic:

1. Put the two sub-category tests inside parentheses.
2. AND the price test.
3. AND a NOT ILIKE test on the name for sugar-free.

Step 4 — Final SQL Solution:

```
SELECT p.ProductID,
       p.Product,
       p.Sub_Category,
       p.Sale_Price
FROM Products AS p
WHERE (p.Sub_Category ILIKE '%chocolate%' OR p.Sub_Category ILIKE '%cookie%')
  AND p.Sale_Price < 250
  AND p.Product NOT ILIKE '%sugar%free%'
ORDER BY p.Sale_Price, p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `ILIKE` is PostgreSQL's case-insensitive LIKE, so `'%chocolate%'` matches `Chocolates` and `Dark Chocolate`. On MySQL, plain LIKE is usually case-insensitive already.
- The parentheses make the OR one unit: the product is in either sub-category.
- `p.Sale_Price < 250` then applies to every product that passed, chocolate or cookie.
- `NOT ILIKE '%sugar%free%'` drops `Sugar-Free`, `Sugar Free` and `sugarfree`. The `%` between the words allows any separator.

Step 6 — Sample Output:

productid	product	sub_category	sale_price
2159	50-50 Timepass Salted Biscuits	Biscuits & Cookies	5.00
2228	Tiger Elaichi Cream Biscuits	Biscuits & Cookies	5.00
2240	50-50 Timepass Biscuits	Biscuits & Cookies	5.00
2244	Tiger Chocolate Cream Biscuits	Biscuits & Cookies	5.00
2274	Biscuits - Magix Kreams Choc	Biscuits & Cookies	5.00
3353	Sugar Coated Chocolate	Chocolates & Candies	5.00
3391	Cadbury Perk - Chocolate Bar	Chocolates & Candies	5.00
3410	Polo - The Mint With The Hole	Chocolates & Candies	5.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 813 rows.

Step 7 — Why This Solution Works: With the OR in parentheses, the WHERE clause reads exactly like the business rule: (A OR B) AND C AND NOT D. Every condition applies to every candidate row, so no expensive item and no sugar-free item can slip through.

Step 8 — Common Mistakes:

- Leaving out the parentheses. AND binds tighter than OR, so SQL reads this as `chocolate OR (cookie AND under 250 AND not sugar-free)`. Every chocolate product comes back, whatever its price and whether or not it is sugar-free:

```
SELECT p.ProductID, p.Product, p.Sub_Category, p.Sale_Price
FROM Products AS p
WHERE p.Sub_Category ILIKE '%chocolate%' OR p.Sub_Category ILIKE '%cookie%'
  AND p.Sale_Price < 250
  AND p.Product NOT ILIKE '%sugar%free%'
ORDER BY p.Sale_Price, p.ProductID;
```

- Writing `NOT p.Sub_Category ILIKE '%chocolate%' OR ...` and expecting NOT to cover the whole OR. NOT binds tighter than AND and OR, so it negates only the next test. Use `NOT (A OR B)`.
- Matching sugar-free with `= 'Sugar-Free'`. It misses every product whose name has other words around it.

Step 9 — Interview Follow-Up: Products with a NULL name or sub-category: are they included or excluded by this WHERE clause, and why?

Step 10 — Optimization or Alternative: `p.Sub_Category ILIKE ANY (ARRAY['%chocolate%', '%cookie%'])` replaces the OR pair and grows more neatly as the banner adds categories. If merchandising maintains an explicit list of sub-category names, prefer `IN (...)` on exact, cleaned values over pattern matching.

Question 24: Suppliers outside India or with no email

Difficulty: Easy

Primary Topic: NOT and OR with NULLs

Combined Topics: IS NULL, CASE issue label

Business Scenario: Procurement compliance runs a supplier review each quarter. Suppliers outside India need import paperwork on file, and suppliers with no email cannot receive purchase orders from the new e-procurement system. A supplier with no country recorded is also a problem, because nobody can confirm it is domestic.

Problem Statement: Return every supplier whose country is not India, whose country is missing, or whose email is missing or blank. Add an `issue` column naming every problem found. Sort by `SupplierID`.

Expected Output:

- `SupplierID`
- `CompanyName`
- `Country`
- `Email`
- `issue`: `Outside India`, `Country missing`, `No email`, or two of these joined by `; `

Tables Required: `Suppliers`

SQL Concepts Tested: OR conditions, three-valued logic with NULL, IS NULL, CASE, CONCAT_WS.

Step 1 — Understand the Problem: In SQL a condition can be true, false or unknown. `Country <> 'India'` is unknown when Country is NULL, and WHERE keeps only true rows. So a supplier with no country fails both `= 'India'` and `<> 'India'`, and drops out of both lists. Missing values must be tested explicitly with IS NULL.

Step 2 — Identify the Tables and Relationships: Only `Suppliers`, one row per supplier.

Step 3 — Build the Query Logic:

1. Flag the country: missing, or not India after trimming and upper-casing.
2. Flag the email: NULL, or only spaces.
3. Keep rows where any flag is true.
4. Build `issue` from one CASE per check, joined with CONCAT_WS.

Step 4 — Final SQL Solution:

```sql
SELECT s.SupplierID,
       s.CompanyName,
       s.Country,
       s.Email,
       CONCAT_WS('; ',
           CASE WHEN s.Country IS NULL                THEN 'Country missing'
                WHEN UPPER(TRIM(s.Country)) <> 'INDIA' THEN 'Outside India'
           END,
           CASE WHEN s.Email IS NULL OR TRIM(s.Email) = '' THEN 'No email'
           END) AS issue
FROM Suppliers AS s
WHERE s.Country IS NULL
   OR UPPER(TRIM(s.Country)) <> 'INDIA'
   OR s.Email IS NULL
   OR TRIM(s.Email) = ''
ORDER BY s.SupplierID;
```

Step 5 — Line-by-Line Explanation:

- `s.Country IS NULL` catches suppliers with no country. Without it they are lost.
- `UPPER(TRIM(s.Country)) <> 'INDIA'` treats `'india'` and `'India '` as India, so formatting alone does not flag a supplier.
- `s.Email IS NULL OR TRIM(s.Email) = ''` treats a blank string the same as a missing email.
- Each CASE returns a label or NULL. `CONCAT_WS('; ', ...)` joins the labels and skips the NULLs, so a supplier with two problems shows both.

Step 6 — Sample Output:

supplierid	companyname	country	email	issue
4	Ganga Distributors LLP	India	NULL	No email
12	Himalayan Naturals Co	Nepal	trade@himalayannaturals.np	Outside India
13	Gulf Gourmet Trading LLC	United Arab Emirates	import@gulfgourmet.ae	Outside India
14	Lion City Imports Pte Ltd	Singapore	orders@lioncityimports.sg	Outside India

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 4 rows returned.

Step 7 — Why This Solution Works: Every failure is tested directly, so no supplier is lost to an unknown result. WHERE and `issue` use the same checks, so every returned row has a label. This test shows the NULL trap on its own:

```sql
SELECT v.country,
       v.country <> 'India'                    AS plain_test,
       v.country IS NULL OR v.country <> 'India' AS safe_test
FROM (VALUES ('India'), ('Nepal'), (NULL)) AS v(country)
ORDER BY v.country NULLS LAST;
```

Step 8 — Common Mistakes:

- Relying on `<>` alone. A supplier with no country but a valid email would be missing from this result (none exists in this extract). On this

data it also shows the opposite fault: without UPPER and TRIM, `INDIA` (supplier 7) and `india` (supplier 9) are flagged as outside India, so it returns 6 rows instead of 4:

```
SELECT s.SupplierID, s.CompanyName, s.Country, s.Email
FROM Suppliers AS s
WHERE s.Country <> 'India'
   OR s.Email IS NULL
ORDER BY s.SupplierID;
```

- Writing `NOT (s.Country = 'India')`. It is the same test with the same gap: NOT of unknown is still unknown.
- Using `s.Country NOT IN ('India')` for the same purpose. NOT IN also returns unknown for a NULL country.

Step 9 — Interview Follow-Up: Compliance also wants suppliers whose email domain does not match their country, for example a `.np` address on an Indian supplier. How would you extract the domain ending to compare?

Step 10 — Optimization or Alternative: `COALESCE(UPPER(TRIM(s.Country)), '') <> 'INDIA'` folds the NULL test into one expression. It is shorter, but the explicit `IS NULL` is clearer and lets the label say why the row was flagged.

Question 25: Customer sign-ups by year and quarter

Difficulty: Easy

Primary Topic: Date-part grouping

Combined Topics: ORDER BY, NULL dates

Business Scenario: The growth team reports new-customer acquisition to the board every quarter. A sign-up is the day an account was created (`DateEntered`), whether or not the person has ordered. They want new accounts per year and quarter in time order, with undated accounts on their own line.

Problem Statement: Count customers by the year and quarter of `DateEntered`. Accounts with no `DateEntered` form one group with NULL year and quarter, listed last. Sort by year, then quarter.

Expected Output:

- `signup_year`: year of `DateEntered`, as an integer
- `signup_quarter`: quarter 1 to 4, as an integer
- `new_customers`: accounts created in that quarter

Tables Required: `Customers`

SQL Concepts Tested: EXTRACT, grouping by two derived columns, casting, NULL groups, ORDER BY with NULLS LAST.

Step 1 — Understand the Problem: The output grain is one row per year and quarter. Each customer row is mapped to its year and its quarter, and rows with the same pair are counted together. The quarter on its own is not enough: Q1 2023 and Q1 2024 must stay apart.

Step 2 — Identify the Tables and Relationships: Only `Customers`, one row per account. `DateEntered` is the sign-up date.

Step 3 — Build the Query Logic:

1. Extract the year and the quarter from `DateEntered`.
2. Group by both and count.
3. Sort by year, then quarter, with the missing-date group at the end.

Step 4 — Final SQL Solution:

```
SELECT EXTRACT(YEAR FROM c.DateEntered)::int    AS signup_year,
       EXTRACT(QUARTER FROM c.DateEntered)::int AS signup_quarter,
       COUNT(*)                                 AS new_customers
FROM Customers AS c
GROUP BY EXTRACT(YEAR FROM c.DateEntered)::int,
         EXTRACT(QUARTER FROM c.DateEntered)::int
ORDER BY signup_year NULLS LAST, signup_quarter NULLS LAST;
```

Step 5 — Line-by-Line Explanation:

- `EXTRACT(YEAR FROM c.DateEntered)` returns the year. `EXTRACT(QUARTER ...)` returns 1 for January to March, up to 4 for October to December.
- `::int` turns EXTRACT's numeric result into a plain integer, which reads and joins more cleanly.
- `GROUP BY` both expressions gives one row per year-quarter pair. A NULL `DateEntered` gives NULL for both, and those rows form one group, which `COUNT(*)` still counts.
- `NULLS LAST` keeps the missing-date row below the timeline. It is PostgreSQL's default for ascending sorts, so here it documents intent.

Step 6 — Sample Output:

signup_year	signup_quarter	new_customers
2022	4	118
2023	1	131
2023	2	132

2023	3	158
2023	4	249
2024	1	252
2024	2	276
2024	3	326

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 15 rows.

Step 7 — Why This Solution Works: Grouping on year and quarter together gives each calendar quarter its own row, and sorting by the same two numbers puts them in time order. Nothing is filtered, so the counts add up to the total number of accounts, and the board can see how many sign-ups cannot be dated.

Step 8 — Common Mistakes:

- Grouping by quarter alone. Every first quarter in the table is added together:

```
SELECT EXTRACT(QUARTER FROM c.DateEntered)::int AS signup_quarter,
       COUNT(*)                                 AS new_customers_all_years
FROM Customers AS c
GROUP BY EXTRACT(QUARTER FROM c.DateEntered)::int
ORDER BY signup_quarter NULLS LAST;
```

- Reading the result as a full timeline. A quarter with no sign-ups produces no row at all, so the chart jumps across the gap without showing a zero.
- Calling these new people. They are new accounts; someone who registered twice is counted twice.

Step 9 — Interview Follow-Up: The board wants each quarter's sign-ups next to the same quarter last year. How would you put the two numbers on one row?

Step 10 — Optimization or Alternative: `DATE_TRUNC('quarter', c.DateEntered)::date` gives one column (2024-04-01 for Q2 2024) that sorts and charts as a real date. To show empty quarters as 0, build the quarter list with `generate_series` and LEFT JOIN the counts to it.

Medium

Questions 26 to 60

Question 26: List-price value by category

Difficulty: Medium

Primary Topic: Three-table JOIN with GROUP BY

Combined Topics: LEFT JOIN for unmapped categories, COUNT DISTINCT

Business Scenario: The category management team splits next quarter's promotion budget across categories. For each category they want three numbers: how many orders included it, how many units moved, and what those units are worth at list price. Finance will tie the units column back to the order lines, so no line may go missing, even when its product or category reference is broken.

Problem Statement: For every category, return the number of distinct orders that contained at least one of its products, the units sold, and the list-price value (`Quantity * Sale_Price`). Lines whose product points to a `CategoryID` with no `Category` row go under `Unmapped`. Lines whose `ProductID` is not in `Products` go under `Unknown product`. Sort by list-price value, highest first.

Expected Output:

- `category_name`: `CategoryName`, `Unmapped` or `Unknown product`
- `orders`: distinct orders containing at least one line in the category
- `units_sold`: sum of `Quantity`
- `list_price_value`: sum of `Quantity * Sale_Price`

Tables Required: `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: joining a chain of three tables, LEFT JOIN along the chain, COUNT DISTINCT, CASE labels for unmatched rows, grouping on an expression.

Step 1 — Understand the Problem: One row in `OrderDetails` is one product in one order. The report's grain is the category. Units and value roll up from lines without trouble. The order count does not: an order with three snack products has three snack lines, so `COUNT(*)` counts it three times under Snacks.

`Sale_Price` is today's catalogue price, not the price the customer paid, so `Quantity * Sale_Price` is a proxy. Call it list-price value, never revenue.

Step 2 — Identify the Tables and Relationships: `OrderDetails.ProductID` → `Products.ProductID` (many lines to one product) gives the price and the `CategoryID`. `Products.CategoryID` → `Category.CategoryID` (many products to one category) gives the name. No foreign keys are enforced, so either hop can fail. A line pointing to a missing product and a product pointing to a missing category are different faults with different owners, so they get different labels.

Step 3 — Build the Query Logic:

1. Start from `OrderDetails`, since every line must be counted.
2. LEFT JOIN `Products` on the line, then LEFT JOIN `Category` on the product. Both joins are LEFT so a broken hop keeps the line.
3. Build the label with CASE. Test the product first: if there is no product row, there is no category to look up either.
4. Group by the label. Count orders with `COUNT(DISTINCT od.OrderID)` and sum units and list-price value.

Step 4 — Final SQL Solution:

```sql
SELECT CASE
           WHEN p.ProductID IS NULL    THEN 'Unknown product'
           WHEN cat.CategoryID IS NULL THEN 'Unmapped'
           ELSE cat.CategoryName
       END                             AS category_name,
       COUNT(DISTINCT od.OrderID)      AS orders,
       SUM(od.Quantity)                AS units_sold,
       SUM(od.Quantity * p.Sale_Price) AS list_price_value
FROM OrderDetails AS od
LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
GROUP BY category_name
ORDER BY list_price_value DESC NULLS LAST, category_name;
```

Step 5 — Line-by-Line Explanation:

- The CASE checks `p.ProductID IS NULL` first. After a LEFT JOIN, a NULL key on the right side means "no match", so this catches lines with a broken `ProductID`. Only then does it check the category.
- `COUNT(DISTINCT od.OrderID)` counts each order once per category, however many lines it has there.
- `SUM(od.Quantity * p.Sale_Price)` multiplies per line, then adds. `Unknown product` lines have no price, so their value is NULL: unknown, not zero.
- `LEFT JOIN Category` hangs off `p.CategoryID`. The chain goes line → product → category.

- `GROUP BY category_name` uses the output alias. PostgreSQL and MySQL allow this. SQL Server and Oracle need the full CASE expression repeated.
- `DESC NULLS LAST` keeps the NULL-valued `Unknown product` row at the bottom instead of the top.

Step 6 — Sample Output:

category_name	orders	units_sold	list_price_value
Beauty & Hygiene	25102	78508	23471683.00
Foodgrains, Oil & Masala	18741	53337	8386383.00
Gourmet & World Food	10585	25521	6769406.00
Snacks & Branded Foods	20890	59440	5271346.00
Beverages	10176	23942	4623400.00
Cleaning & Household	8511	21213	4270141.00
Kitchen, Garden & Pets	3099	6718	2907298.00
Bakery, Cakes & Dairy	8729	21706	2077639.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 13 rows.

Step 7 — Why This Solution Works: Both joins are many-to-one when read from the line side: each line finds at most one product and each product at most one category. So no line is repeated, and because both are LEFT JOINs, no line is lost. That gives you a clean check: `units_sold` must add up to `SELECT SUM(Quantity) FROM OrderDetails`. The `orders` column will not add up to the total order count, and it should not: an order with rice and tea counts once under each category. Say this before the interviewer asks.

Step 8 — Common Mistakes:

- Using INNER JOINs and `COUNT(*)`. The unmatched lines vanish, so units no longer tie to `OrderDetails`, and the order count becomes a line count:

```sql
SELECT cat.CategoryName,
       COUNT(*)                       AS orders_wrong,   -- counts lines
       SUM(od.Quantity * p.Sale_Price) AS list_price_value
FROM OrderDetails AS od
JOIN Products AS p   ON p.ProductID    = od.ProductID
JOIN Category AS cat ON cat.CategoryID = p.CategoryID
GROUP BY cat.CategoryName
ORDER BY list_price_value DESC, cat.CategoryName;
```

- Joining `Orders` and summing `Total_Order_Amount` per category. That amount belongs to the whole order, so a mixed basket hands its full value to every category in it.
- Labelling both faults `Unmapped`. A missing category is a catalogue fix; a missing product is a load problem.

Step 9 — Interview Follow-Up: Finance wants a grand-total row under the categories. Why is adding up the `orders` column wrong for that row, and how would you compute it?

Step 10 — Optimization or Alternative: Grouping by name assumes category names are unique. If they might not be, aggregate by `CategoryID` first and attach the name afterwards. Each broken `CategoryID` then gets its own row, which helps whoever fixes the data:

```sql
WITH by_category_id AS (
    SELECT p.CategoryID,
           p.ProductID IS NULL             AS unknown_product,
           COUNT(DISTINCT od.OrderID)      AS orders,
           SUM(od.Quantity)                AS units_sold,
           SUM(od.Quantity * p.Sale_Price) AS list_price_value
    FROM OrderDetails AS od
    LEFT JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY p.CategoryID, p.ProductID IS NULL
)
SELECT b.CategoryID,
       CASE WHEN b.unknown_product THEN 'Unknown product'
            ELSE COALESCE(cat.CategoryName, 'Unmapped') END AS category_name,
       b.orders,
       b.units_sold,
       b.list_price_value
FROM by_category_id AS b
LEFT JOIN Category AS cat ON cat.CategoryID = b.CategoryID
ORDER BY b.list_price_value DESC NULLS LAST, b.CategoryID;
```

The join to `Category` now runs on a handful of aggregated rows instead of every line.

Question 27: Average order value by customer state

Difficulty: Medium

Primary Topic: JOIN with GROUP BY and HAVING

Combined Topics: AOV on non-null totals, LEFT JOIN, COALESCE labels

Business Scenario: The regional marketing team wants to raise the free-delivery threshold in states where baskets are already large. They need average order value (AOV) per customer state, but a state with a handful of orders gives a number nobody should act on. They have agreed to look only at states with at least 10 orders.

Problem Statement: For each customer state, return the number of distinct ordering customers, the number of orders, the total order value and the average order value. Group orders whose customer has no `State`, or whose `CustomerID` matches no customer, under `Unknown`. Keep only states with 10 or more orders. Sort by AOV, highest first.

Expected Output:

- `state`: the customer's `State`, or `Unknown`
- `customers`: distinct customers who placed orders
- `orders`: number of orders
- `order_value`: sum of `Total_Order_Amount`
- `avg_order_value`: order value divided by the number of orders that have a total

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: LEFT JOIN to keep unmatched orders, COALESCE and NULLIF for labels, GROUP BY with HAVING, AOV built from SUM and COUNT, COUNT DISTINCT.

Step 1 — Understand the Problem: AOV is total order value divided by the number of orders. The denominator is the tricky part. An order with a NULL `Total_Order_Amount` adds nothing to the numerator; if it still counts in the denominator, AOV drops for no reason. So divide by orders that have a value.

State lives on the customer, so each order inherits the state of whoever placed it. Every order counts as placed, since cancellations and returns are not modelled.

Step 2 — Identify the Tables and Relationships: `Orders.CustomerID` → `Customers.CustomerID`, many orders to one customer. Reading from `Orders`, each order finds at most one customer, so the join never repeats an order and summing `Total_Order_Amount` after it is safe. Two things can go wrong: `CustomerID` may point to no customer, or the customer may have a blank or NULL `State`.

Step 3 — Build the Query Logic:

1. Start from `Orders` and LEFT JOIN `Customers`, so orders with an unknown customer are kept.
2. Build the state label: `TRIM` the value, turn an empty string into NULL with `NULLIF`, then replace NULL with `Unknown`.
3. Group by the label. Count customers with `COUNT(DISTINCT ...)`, count orders, sum the value.
4. Divide the sum by `COUNT(o.Total_Order_Amount)`, which skips NULL totals.
5. Filter groups with `HAVING COUNT(*) >= 10`. This is a condition on an aggregate, so it cannot go in WHERE.

Step 4 — Final SQL Solution:

```
SELECT COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')            AS state,
       COUNT(DISTINCT o.CustomerID)                               AS customers,
       COUNT(*)                                                   AS orders,
       SUM(o.Total_Order_Amount)                                  AS order_value,
       ROUND(SUM(o.Total_Order_Amount)
             / NULLIF(COUNT(o.Total_Order_Amount), 0), 2)         AS avg_order_value
FROM Orders AS o
LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
GROUP BY COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')
HAVING COUNT(*) >= 10
ORDER BY avg_order_value DESC NULLS LAST, state;
```

Step 5 — Line-by-Line Explanation:

- `COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')` handles three cases at once: a missing customer (the LEFT JOIN gives NULL), a NULL `State`, and a blank `State`.
- `COUNT(DISTINCT o.CustomerID)` counts people, not orders. A customer with 20 orders counts once.
- `COUNT(*)` counts every order, including any with a NULL total. That is the right number for "how many orders".
- `COUNT(o.Total_Order_Amount)` counts only orders that have a value. Dividing by it gives the same answer as `AVG(o.Total_Order_Amount)`, with the rule written out where a reviewer can see it.
- `NULLIF(..., 0)` stops a division by zero if a group's totals are all NULL.
- `HAVING COUNT(*) >= 10` runs after grouping, so it sees the order count per state.
- `state` in ORDER BY breaks AOV ties.

Step 6 — Sample Output:

state	customers	orders	order_value	avg_order_value
Jharkhand	32	206	304890.33	1480.05
England	38	396	578049.43	1459.72
Andhra Pradesh	57	666	948396.10	1424.02
Assam	29	277	390943.49	1411.35
Rajasthan	107	1139	1607141.74	1411.01
Uttar Pradesh	236	2229	3091539.57	1386.96

Tamil Nadu	438	4556	6210672.29	1363.19
New Jersey	31	313	425117.88	1358.20

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 26 rows.

Step 7 — Why This Solution Works: The join is many-to-one from the order side, so every order appears once and the sum is correct. The LEFT JOIN plus the `Unknown` label means no order is dropped: remove the HAVING clause and `orders` must add up to `SELECT COUNT(*) FROM Orders`. The AOV numerator and denominator cover the same orders, the ones with a value, so a missing total cannot drag the average down.

Step 8 — Common Mistakes:

- Using `COUNT(*)` as the AOV denominator. It counts orders with a NULL total, so every such order lowers the AOV of its state.
- Putting the threshold in WHERE. `WHERE COUNT(*) >= 10` is an error, because WHERE filters rows before any grouping happens.
- An INNER JOIN, which drops orders from customers not on file.
- Averaging per-customer averages. That weights every customer equally, a different question:

```
WITH per_customer AS (
    SELECT o.CustomerID, AVG(o.Total_Order_Amount) AS customer_aov
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown') AS state,
       ROUND(AVG(pc.customer_aov), 2)                 AS avg_of_customer_aov   -- not AOV
FROM per_customer AS pc
LEFT JOIN Customers AS c ON c.CustomerID = pc.CustomerID
GROUP BY COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')
ORDER BY avg_of_customer_aov DESC NULLS LAST, state;
```

Step 9 — Interview Follow-Up: The AOV for one state is pulled up by two very large orders. Which extra column would you add so the marketing team can see that, without changing the definition of AOV?

Step 10 — Optimization or Alternative: `ROUND(AVG(o.Total_Order_Amount), 2)` returns the same number with less code. Keep the SUM/COUNT form when you also want the denominator as its own column, such as `orders_with_value` next to `orders`, so a reader sees when the two differ. For a full-table report like this, a hash join over both tables is normal; an index on `Orders(CustomerID)` only pays off when you filter to a few customers.

Question 28: Payment mix: share of orders and value

Difficulty: Medium

Primary Topic: Percentage of total with a scalar subquery

Combined Topics: LEFT JOIN, COALESCE labels, GROUP BY

Business Scenario: The payments team is renegotiating gateway fees, and fees are charged per transaction for some methods and as a share of value for others. They need each payment type's share of order count and its share of order value in one table. A method can carry many small orders or a few large ones, so the two shares can tell different stories.

Problem Statement: For each payment type, return the number of orders, its percentage of all orders, the total order value and its percentage of all order value. Orders whose `PaymentID` is NULL or matches no payment row go under `Unknown`. Use scalar subqueries for the two denominators. Sort by orders, highest first.

Expected Output:

- `payment_type`: `PaymentType`, or `Unknown`
- `orders`: number of orders
- `pct_of_orders`: orders as a percentage of all orders (2 dp)
- `order_value`: sum of `Total_Order_Amount`
- `pct_of_value`: order value as a percentage of all order value (2 dp)

Tables Required: `Orders`, `Payments`

SQL Concepts Tested: scalar subquery as a denominator, percentage of total, LEFT JOIN with a fallback label, integer versus numeric division.

Step 1 — Understand the Problem: A share is a part divided by a whole. The part comes from the grouped row (one payment type). The whole comes from all orders, ignoring the grouping. In a plain GROUP BY query each row only sees its own group, so the whole has to come from somewhere else. A scalar subquery is the simplest source: a query that returns exactly one value, which PostgreSQL computes once and reuses on every row.

Each `Orders` row counts as one order. Refunds and cancellations are not modelled.

Step 2 — Identify the Tables and Relationships: `Orders.PaymentID` → `Payments.PaymentID`, many orders to one payment method. The join is many-to-one from the order side, so summing `Total_Order_Amount` after it is safe. `PaymentID` can be NULL or point to a method that does not exist. An INNER JOIN would drop those orders, and the shares would no longer add up to 100.

Step 3 — Build the Query Logic:

1. Start from `Orders`, LEFT JOIN `Payments`, label missing matches `Unknown`.
2. Group by the label. Count orders and sum the value.
3. Divide each part by `(SELECT COUNT(*) FROM Orders)` and `(SELECT SUM(Total_Order_Amount) FROM Orders)`.
4. Multiply by `100.0` first so the division is numeric, wrap the denominator in `NULLIF(..., 0)`, and round to 2 dp.

Step 4 — Final SQL Solution:

```sql
SELECT COALESCE(pay.PaymentType, 'Unknown')                          AS payment_type,
       COUNT(*)                                                       AS orders,
       ROUND(100.0 * COUNT(*)
             / NULLIF((SELECT COUNT(*) FROM Orders), 0), 2)           AS pct_of_orders,
       SUM(o.Total_Order_Amount)                                      AS order_value,
       ROUND(100.0 * SUM(o.Total_Order_Amount)
             / NULLIF((SELECT SUM(Total_Order_Amount) FROM Orders), 0), 2) AS pct_of_value
FROM Orders AS o
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
GROUP BY COALESCE(pay.PaymentType, 'Unknown')
ORDER BY orders DESC, payment_type;
```

Step 5 — Line-by-Line Explanation:

- `COALESCE(pay.PaymentType, 'Unknown')` names the group for orders with no payment match. The same expression goes in GROUP BY.
- `(SELECT COUNT(*) FROM Orders)` is uncorrelated: it does not reference `o`, so it returns the same number for every row and is evaluated once.
- `100.0 * COUNT(*)` makes the expression numeric before dividing. `COUNT(*)` returns a `bigint`, and bigint divided by bigint truncates to a whole number in PostgreSQL.
- `NULLIF(..., 0)` turns an empty `Orders` table into a NULL share instead of a division-by-zero error.
- `(SELECT SUM(Total_Order_Amount) FROM Orders)` skips NULL totals, and so does `SUM(o.Total_Order_Amount)` in each group, so the value shares sum to 100 (give or take rounding).
- `ORDER BY orders DESC, payment_type` puts the busiest method first with a name tiebreaker.

Step 6 — Sample Output:

payment_type	orders	pct_of_orders	order_value	pct_of_value
UPI	23094	49.73	30920908.50	50.46
Credit Card	8282	17.83	10537736.55	17.20
Cash on Delivery	4723	10.17	6142240.50	10.02
Debit Card	4290	9.24	5558048.55	9.07
Wallet	2743	5.91	3375942.48	5.51
Net Banking	2076	4.47	2827669.00	4.61
Pay Later	1129	2.43	1440094.95	2.35
EMI	81	0.17	431023.00	0.70

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 9 rows.

Step 7 — Why This Solution Works: The numerators and denominators cover the same rows. The LEFT JOIN keeps every order in exactly one group, and the subqueries read the same `Orders` table with no filter, so `pct_of_orders` sums to 100 and so does `pct_of_value`, before rounding. If you add a WHERE clause to the outer query (say, one year), add the same filter inside both subqueries, or the shares will be measured against the wrong whole.

Step 8 — Common Mistakes:

- Dividing two integers. Because `* 100` happens first, this does not return zeros; it returns whole numbers with the decimals cut off, and a type under 1% shows as 0. It looks plausible, which is why it slips through review:

```sql
SELECT COALESCE(pay.PaymentType, 'Unknown')                 AS payment_type,
       COUNT(*) * 100 / (SELECT COUNT(*) FROM Orders)       AS pct_wrong   -- integer division
FROM Orders AS o
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
GROUP BY COALESCE(pay.PaymentType, 'Unknown')
ORDER BY pct_wrong DESC, payment_type;
```

- Using an INNER JOIN, so unknown-payment orders disappear from the numerators but stay in the subquery denominators. The shares then sum to less than 100.
- Filtering the outer query but not the subqueries, as described in Step 7.

Step 9 — Interview Follow-Up: The team wants the same table for the latest year in the data only. How would you write the date filter once, so the outer query and both denominators cannot drift apart?

Step 10 — Optimization or Alternative: A window function over the aggregate gives the same shares without a second scan of `Orders`, and it automatically respects any WHERE clause on the outer query:

```sql
SELECT COALESCE(pay.PaymentType, 'Unknown')                                  AS payment_type,
       COUNT(*)                                                               AS orders,
       ROUND(100.0 * COUNT(*) / NULLIF(SUM(COUNT(*)) OVER (), 0), 2)         AS pct_of_orders,
       SUM(o.Total_Order_Amount)                                              AS order_value,
       ROUND(100.0 * SUM(o.Total_Order_Amount)
             / NULLIF(SUM(SUM(o.Total_Order_Amount)) OVER (), 0), 2)          AS pct_of_value
FROM Orders AS o
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
GROUP BY COALESCE(pay.PaymentType, 'Unknown')
ORDER BY orders DESC, payment_type;
```

`SUM(COUNT(*)) OVER ()` reads as: after grouping, add up the group counts across all groups. The scalar subquery is easier to explain; the window version cannot drift from the outer filter, which makes it the safer default once a WHERE clause appears.

Question 29: Order value and units by shipper without double counting

Difficulty: Medium

Primary Topic: One-to-many joins (fan-out)

Combined Topics: Pre-aggregation in a CTE, LEFT JOIN, COALESCE

Business Scenario: The logistics finance team is renegotiating courier rate cards. For each shipper they want three numbers side by side: how many orders it carried, the total order value it carried, and how many units were inside those orders. Last quarter an analyst produced this report and the order value came out higher than the finance ledger. Your job is to produce it correctly.

Problem Statement: For every shipper that carried at least one order, return the number of orders, the sum of `Total_Order_Amount` for those orders, and the total units (`Quantity`) in those orders. Orders with a missing or unrecognised `ShipperID` should appear under one row labelled `Unknown shipper`. Sort by order value, highest first.

Expected Output:

- `shipper_name`: the shipper's `CompanyName`, or `Unknown shipper`
- `orders`: number of orders carried
- `order_value`: sum of `Total_Order_Amount` for those orders, each order counted once
- `units_shipped`: sum of `Quantity` across the lines of those orders

Tables Required: `Orders`, `OrderDetails`, `Shippers`

SQL Concepts Tested: data grain, one-to-many fan-out, aggregating before joining, LEFT JOIN to keep unmatched keys, COALESCE for labels.

Step 1 — Understand the Problem: The report mixes two grains. `orders` and `order_value` live at the order grain: one row in `Orders` is one order and carries one `Total_Order_Amount`. `units_shipped` lives at the line grain: one row in `OrderDetails` is one product inside one order. The moment you put both grains in the same join, every order is repeated once per line, and any order-level number summed after that join is multiplied by the line count.

Step 2 — Identify the Tables and Relationships: `Orders` gives the order count, the order value and the `ShipperID`. `Shippers` turns `ShipperID` into a name (`Orders.ShipperID` → `Shippers.ShipperID`, many orders to one shipper). `OrderDetails` holds the quantities (`OrderDetails.OrderID` → `Orders.OrderID`, one order to many lines). That last relationship is the dangerous one.

Step 3 — Build the Query Logic:

1. Collapse `OrderDetails` to one row per order with `SUM(Quantity)`. Now it has the same grain as `Orders`.
2. Join that result to `Orders` on `OrderID`. This is a one-to-one join, so no order is repeated.
3. LEFT JOIN `Shippers` so orders with a NULL or unmatched `ShipperID` survive, and label them with COALESCE.
4. Group by shipper name and aggregate.

Step 4 — Final SQL Solution:

```sql
WITH order_units AS (
    SELECT od.OrderID,
           SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.OrderID
)
SELECT COALESCE(sh.CompanyName, 'Unknown shipper') AS shipper_name,
       COUNT(*)                                    AS orders,
       SUM(o.Total_Order_Amount)                   AS order_value,
       COALESCE(SUM(ou.units), 0)                  AS units_shipped
FROM Orders AS o
LEFT JOIN Shippers    AS sh ON sh.ShipperID = o.ShipperID
LEFT JOIN order_units AS ou ON ou.OrderID   = o.OrderID
GROUP BY COALESCE(sh.CompanyName, 'Unknown shipper')
ORDER BY order_value DESC NULLS LAST;
```

Step 5 — Line-by-Line Explanation:

- `WITH order_units AS (...)` builds a small table with one row per `OrderID` and the units in that order. Grouping by `OrderID` is what changes the grain from line to order.
- `FROM Orders AS o` makes the order the driving row, so `COUNT(*)` counts orders.

- `LEFT JOIN Shippers` keeps orders whose `ShipperID` is NULL or points to no shipper. An INNER JOIN would drop them and the totals would no longer tie to the ledger.
- `LEFT JOIN order_units` keeps orders that have no lines at all. Their `units` is NULL, which `SUM` ignores.
- `SUM(o.Total_Order_Amount)` is safe here because each order appears exactly once after the joins.
- `COALESCE(SUM(ou.units), 0)` turns a group with no lines into 0 instead of NULL.
- `ORDER BY order_value DESC NULLS LAST` puts the biggest shipper first. In PostgreSQL, DESC puts NULLs first unless told otherwise.

Step 6 — Sample Output:

shipper_name	orders	order_value	units_shipped
SwiftKart Logistics	9396	12521197.37	63812
BlueLine Express	8129	10679470.57	54606
MetroDash Hyperlocal	7738	10172210.48	52551
Ecomzip Couriers	5940	7871579.94	39999
ShadowRun Delivery	5090	6641274.73	33949
XpressHaul	4881	6398849.94	33399
Bharat Parcel Service	3066	4057663.60	20822
NorthStar Freight	2170	2884875.44	15145

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 9 rows.

Step 7 — Why This Solution Works: Every aggregate is computed at the grain where its column lives. Units are summed at the line grain inside the CTE, then the CTE is reduced to the order grain before it meets `Orders`. After that, the join is one-to-one and nothing is duplicated, so `SUM(Total_Order_Amount)` matches what you would get from `Orders` alone. A quick way to prove it: `SELECT SUM(Total_Order_Amount) FROM Orders` must equal the sum of the `order_value` column in this report.

Step 8 — Common Mistakes:

- Joining `Orders` straight to `OrderDetails` and summing `Total_Order_Amount`. An order with four lines is counted four times. This is the query that disagreed with the ledger:

```sql
SELECT sh.CompanyName,
       COUNT(*)                  AS orders_wrong,   -- counts lines, not orders
       SUM(o.Total_Order_Amount) AS order_value_wrong
FROM Orders AS o
JOIN OrderDetails AS od ON od.OrderID   = o.OrderID
JOIN Shippers     AS sh ON sh.ShipperID = o.ShipperID
GROUP BY sh.CompanyName
ORDER BY order_value_wrong DESC;
```

- Patching the inflated query with `COUNT(DISTINCT o.OrderID)` and `SUM(DISTINCT o.Total_Order_Amount)`. The count becomes right, but `SUM(DISTINCT ...)` silently drops two different orders that happen to have the same amount.
- Using INNER JOINs everywhere, which quietly removes orders with no shipper or no lines and makes the report disagree with the ledger in the other direction.

Step 9 — Interview Follow-Up: Add `avg_units_per_order` per shipper. Why is `AVG(od.Quantity)` the wrong formula for it, and what should you divide instead?

Step 10 — Optimization or Alternative: You can also aggregate each side to the shipper grain separately and join the two small results, which is handy when the line table is huge and you only need totals:

```sql
WITH by_order AS (
    SELECT o.ShipperID, COUNT(*) AS orders, SUM(o.Total_Order_Amount) AS order_value
    FROM Orders AS o
    GROUP BY o.ShipperID
),
by_line AS (
    SELECT o.ShipperID, SUM(od.Quantity) AS units_shipped
    FROM OrderDetails AS od
    JOIN Orders AS o ON o.OrderID = od.OrderID
    GROUP BY o.ShipperID
)
SELECT COALESCE(sh.CompanyName, 'Unknown shipper') AS shipper_name,
       SUM(bo.orders)                              AS orders,
       SUM(bo.order_value)                         AS order_value,
       COALESCE(SUM(bl.units_shipped), 0)          AS units_shipped
FROM by_order AS bo
LEFT JOIN by_line  AS bl ON bl.ShipperID IS NOT DISTINCT FROM bo.ShipperID
LEFT JOIN Shippers AS sh ON sh.ShipperID = bo.ShipperID
GROUP BY COALESCE(sh.CompanyName, 'Unknown shipper')
ORDER BY order_value DESC NULLS LAST;
```

`IS NOT DISTINCT FROM` lets the NULL-shipper groups match each other, which a plain `=` would not. On a large table, an index on `OrderDetails(OrderID)` helps the CTE in Step 4, because the line-to-order aggregation is grouped and joined on that column.

Question 30: Repeat versus one-time customers

Difficulty: Medium

Primary Topic: Two-level aggregation with a CTE

Combined Topics: CASE buckets, Percentage of total

Business Scenario: The growth team is deciding whether next quarter's budget goes to acquiring new buyers or to a second-order nudge campaign. The first number they ask for is how many customers ever came back. A repeat customer here means a customer with two or more orders in the whole history; a one-time customer has exactly one.

Problem Statement: Classify every customer who has placed at least one order as `One-time` (exactly one order) or `Repeat` (two or more). Return the number of customers in each group and the group's share of all ordering customers.

Expected Output:

- `customer_type`: `One-time` or `Repeat`
- `customers`: number of customers in the group
- `pct_of_customers`: share of all ordering customers (2 dp)

Tables Required: `Orders`

SQL Concepts Tested: aggregating twice (orders per customer, then customers per bucket), CTE read twice, CASE on an aggregated column, percentage of total.

Step 1 — Understand the Problem: The question has two grains. "How many orders did this customer place?" is a per-customer count. "How many customers are repeat?" is a count of those per-customer results. A single GROUP BY cannot do both, because once you group by customer the output rows are customers, and to count customers by type you have to group again. That is the whole lesson: aggregate, then aggregate the aggregate.

Cancellations and returns are not modelled, so every `Orders` row counts as a placed order. A customer who ordered once and had it refunded is still one-time here.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed. `CustomerID` identifies the buyer, and each row is one order. You do not need `Customers`, because a customer with no orders is neither one-time nor repeat; they never converted, and that is a different metric. Orders whose `CustomerID` matches no customer row still count as a buyer here: the order exists, so the buyer existed.

Step 3 — Build the Query Logic:

1. In a CTE, group `Orders` by `CustomerID` and count orders. One row per customer.
2. In the outer query, turn each count into a label with CASE.
3. Group by the label and count customers.
4. Divide each group by the total number of customers, `(SELECT COUNT(*) FROM orders_per_customer)`. A CTE can be read more than once in the same statement.

Step 4 — Final SQL Solution:

```sql
WITH orders_per_customer AS (
    SELECT o.CustomerID,
           COUNT(*) AS orders_placed
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID
)
SELECT CASE WHEN opc.orders_placed = 1 THEN 'One-time'
            ELSE 'Repeat' END                                          AS customer_type,
       COUNT(*)                                                        AS customers,
       ROUND(100.0 * COUNT(*)
             / NULLIF((SELECT COUNT(*) FROM orders_per_customer), 0), 2) AS pct_of_customers
FROM orders_per_customer AS opc
GROUP BY CASE WHEN opc.orders_placed = 1 THEN 'One-time' ELSE 'Repeat' END
ORDER BY customer_type;
```

Step 5 — Line-by-Line Explanation:

- `orders_per_customer` is the first aggregation. Each output row is one customer and the number of orders they placed.
- `WHERE o.CustomerID IS NOT NULL` keeps orders with no buyer out. GROUP BY would otherwise lump them into a single NULL "customer" and call it repeat.
- The CASE has only two branches because the CTE cannot produce a count of zero: a customer only appears if they have at least one order.
- `COUNT(*)` in the outer query counts customers, because the rows it sees are customers now.
- `(SELECT COUNT(*) FROM orders_per_customer)` reads the CTE a second time to count all ordering customers. It ignores the outer GROUP BY, so both rows get the same denominator.
- `ORDER BY customer_type` puts `One-time` before `Repeat` alphabetically, so the output order is fixed.

Step 6 — Sample Output:

customer_type	customers	pct_of_customers
One-time	702	15.86

Repeat	3725	84.14

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 2 rows returned.

Step 7 — Why This Solution Works: Each aggregation runs at its own grain. The CTE fixes the customer grain, so a customer with 30 orders contributes one row, not 30. The outer query then counts those rows. A good check: the two `customers` values must add up to `SELECT COUNT(DISTINCT CustomerID) FROM Orders`, and the two percentages to 100.

Step 8 — Common Mistakes:

- Joining the per-customer counts back to `Orders` before bucketing. Now each row is an order again, so a customer with 30 orders is counted 30 times and the repeat share is inflated:

```sql
WITH orders_per_customer AS (
    SELECT o.CustomerID, COUNT(*) AS orders_placed
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT CASE WHEN opc.orders_placed = 1 THEN 'One-time' ELSE 'Repeat' END AS customer_type,
       COUNT(*) AS customers_wrong   -- counts orders, not customers
FROM Orders AS o
JOIN orders_per_customer AS opc ON opc.CustomerID = o.CustomerID
GROUP BY CASE WHEN opc.orders_placed = 1 THEN 'One-time' ELSE 'Repeat' END
ORDER BY customer_type;
```

- Trying `CASE WHEN COUNT(*) = 1 ...` in a single query grouped by the label. The label depends on the count, and the count depends on the label; PostgreSQL rejects it because aggregates are not allowed in GROUP BY.
- Defining repeat as "ordered on two different days" in one place and "two orders" in another. Pick one rule and state it. Two orders on the same day may be a split basket, not loyalty.

Step 9 — Interview Follow-Up: Leadership now wants three buckets, `One-time`, `2-3 orders` and `4+ orders`, sorted in that order rather than alphabetically. What would you change?

Step 10 — Optimization or Alternative: A FILTER clause gives the same answer as one wide row, which is handy for a dashboard tile:

```sql
WITH orders_per_customer AS (
    SELECT o.CustomerID, COUNT(*) AS orders_placed
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID
)
SELECT COUNT(*)                                          AS ordering_customers,
       COUNT(*) FILTER (WHERE orders_placed >= 2)        AS repeat_customers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE orders_placed >= 2)
             / NULLIF(COUNT(*), 0), 2)                   AS repeat_rate_pct
FROM orders_per_customer;
```

`FILTER` is PostgreSQL syntax. On MySQL, write `SUM(CASE WHEN orders_placed >= 2 THEN 1 ELSE 0 END)`.

Question 31: Customers who spend more than the average customer

Difficulty: Medium

Primary Topic: Scalar subquery over a CTE

Combined Topics: Aggregation to customer grain, LEFT JOIN for names, Percentage difference

Business Scenario: The CRM team is building an invite list for an early-access programme. The rule is simple: invite customers whose lifetime spend is above what the average customer spends. Lifetime spend means the sum of `Total_Order_Amount` across all of a customer's orders. Refunds are not modelled, so this is gross spend.

Problem Statement: Return every customer whose total spend is greater than the average total spend per customer. Show the customer's spend, the average they were compared against, and how far above it they are in percent. Sort by spend, highest first.

Expected Output:

- `CustomerID`
- `customer_name`: first and last name, or `Unknown customer` when the ID has no `Customers` row
- `total_spend`: sum of the customer's `Total_Order_Amount`
- `avg_customer_spend`: average of `total_spend` across all ordering customers (2 dp)
- `pct_above_avg`: how far `total_spend` is above that average, in percent (2 dp)

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: aggregating to the customer grain in a CTE, scalar subquery that reads the CTE, comparing a row with a set-level value, percentage difference.

Step 1 — Understand the Problem: "The average customer" and "the average order" are different numbers. The average order is `AVG(Total_Order_Amount)` over `Orders`. The average customer's spend is: first add up spend per customer, then average those totals. A customer with ten orders pulls the per-customer average up, which is the point: we want to compare whole customers with whole customers.

So the work happens in two steps. Build one row per customer with their total, then compare each row with the average of that same column.

Step 2 — Identify the Tables and Relationships: `Orders` holds `CustomerID` and `Total_Order_Amount`; that is enough to compute spend. `Customers` is only needed for the name (`Orders.CustomerID` → `Customers.CustomerID`, many to one). Join it after aggregating, with a LEFT JOIN, so a high spender whose customer row is missing still makes the list with a placeholder name. Their money was real.

Step 3 — Build the Query Logic:

1. CTE `customer_spend`: group `Orders` by `CustomerID`, `SUM(Total_Order_Amount)`.
2. A scalar subquery `(SELECT AVG(total_spend) FROM customer_spend)` gives one number: the average customer.
3. Keep rows where `total_spend` beats that number.
4. LEFT JOIN `Customers` for the name and compute `100 * (spend - avg) / avg`.

Step 4 — Final SQL Solution:

```sql
WITH customer_spend AS (
    SELECT o.CustomerID,
           SUM(o.Total_Order_Amount) AS total_spend
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID
),
benchmark AS (
    SELECT AVG(cs.total_spend) AS avg_customer_spend
    FROM customer_spend AS cs
)
SELECT cs.CustomerID,
       CASE WHEN c.CustomerID IS NULL THEN 'Unknown customer'
            ELSE CONCAT_WS(' ', c.FirstName, c.LastName) END                AS customer_name,
       cs.total_spend,
       ROUND(b.avg_customer_spend, 2)                                       AS avg_customer_spend,
       ROUND(100.0 * (cs.total_spend - b.avg_customer_spend)
             / NULLIF(b.avg_customer_spend, 0), 2)                          AS pct_above_avg
FROM customer_spend AS cs
CROSS JOIN benchmark AS b
LEFT JOIN Customers AS c ON c.CustomerID = cs.CustomerID
WHERE cs.total_spend > (SELECT avg_customer_spend FROM benchmark)
ORDER BY cs.total_spend DESC, cs.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `customer_spend` has one row per customer. `SUM` skips an order with a NULL total, so a missing amount does not wipe out the customer's spend.
- `benchmark` averages the per-customer totals. It is one row with one column.
- `CROSS JOIN benchmark` attaches that single value to every customer row so it can be shown in the output. Crossing with a one-row table never multiplies rows.
- `WHERE cs.total_spend > (SELECT avg_customer_spend FROM benchmark)` is the scalar subquery doing the filtering. Writing `b.avg_customer_spend` here works too; the subquery form makes the "compare with one number" idea explicit.
- The CASE tests `c.CustomerID IS NULL`, which after a LEFT JOIN means "no customer row". `CONCAT_WS` skips NULL name parts.
- The percentage divides by the average, with `NULLIF` guarding against zero.

Step 6 — Sample Output:

customerid	customer_name	total_spend	avg_customer_spend	pct_above_avg
100092	Jyoti Sheikh	274363.14	13840.88	1882.27
102192	Rekha Sinha	248896.30	13840.88	1698.27
104856	Manoj Kulkarni	236573.09	13840.88	1609.23
104801	Yash Ansari	225297.92	13840.88	1527.77
102897	Aditi Khanna	223006.61	13840.88	1511.22
102422	Tarun Kapoor	202666.45	13840.88	1364.26
101294	Yash Sinha	198884.78	13840.88	1336.94
103550	Siddharth Iyengar	176654.80	13840.88	1176.33

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 1317 rows.

Step 7 — Why This Solution Works: The comparison happens at the right grain on both sides: a customer total against the average of customer totals. The average is computed once, from the same CTE the rows come from, so the population is identical. Joining `Customers` after aggregation means the name lookup cannot change the sums.

Step 8 — Common Mistakes:

- Comparing with the average order. This keeps almost every repeat customer, because any customer with a few orders beats a single average order:

```sql
SELECT o.CustomerID,
       SUM(o.Total_Order_Amount) AS total_spend
FROM Orders AS o
GROUP BY o.CustomerID
HAVING SUM(o.Total_Order_Amount) > (SELECT AVG(Total_Order_Amount) FROM Orders)  -- average ORDER
ORDER BY total_spend DESC, o.CustomerID;
```

- Writing `HAVING SUM(o.Total_Order_Amount) > AVG(o.Total_Order_Amount)` with no subquery. Inside a group, `AVG` is that customer's own average order, so this keeps every customer with two or more priced orders.
- Labelling a missing customer with `COALESCE(CONCAT_WS(' ', c.FirstName, c.LastName), 'Unknown customer')`. When both parts are NULL, `CONCAT_WS` returns an empty string, not NULL, so the fallback never fires and the name is blank. Test the join key instead, as Step 4 does.

Step 9 — Interview Follow-Up: The average is pulled up by a few very large customers, so the list is short. How would you change the benchmark to the median customer, and which function would you use?

Step 10 — Optimization or Alternative: A window average over the CTE removes the second CTE and the scalar subquery:

```sql
WITH customer_spend AS (
    SELECT o.CustomerID, SUM(o.Total_Order_Amount) AS total_spend
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID
),
compared AS (
    SELECT cs.*, AVG(cs.total_spend) OVER () AS avg_customer_spend
    FROM customer_spend AS cs
)
SELECT cmp.CustomerID,
       CASE WHEN c.CustomerID IS NULL THEN 'Unknown customer'
            ELSE CONCAT_WS(' ', c.FirstName, c.LastName) END AS customer_name,
       cmp.total_spend,
       ROUND(cmp.avg_customer_spend, 2) AS avg_customer_spend
FROM compared AS cmp
LEFT JOIN Customers AS c ON c.CustomerID = cmp.CustomerID
WHERE cmp.total_spend > cmp.avg_customer_spend
ORDER BY cmp.total_spend DESC, cmp.CustomerID;
```

The window must sit in its own CTE because WHERE runs before window functions are evaluated.

Interviewers who do not use CTEs often expect the nested form. The inner subquery builds per-customer totals as a derived table, the outer subquery averages them, and HAVING compares each customer's total against that single value:

```sql
SELECT o.CustomerID,
       SUM(o.Total_Order_Amount) AS total_spend
FROM Orders AS o
WHERE o.CustomerID IS NOT NULL
GROUP BY o.CustomerID
HAVING SUM(o.Total_Order_Amount) > (
    SELECT AVG(per_customer.total_spend)
    FROM (
        SELECT o2.CustomerID,
               SUM(o2.Total_Order_Amount) AS total_spend
        FROM Orders AS o2
        WHERE o2.CustomerID IS NOT NULL
        GROUP BY o2.CustomerID
    ) AS per_customer
)
ORDER BY total_spend DESC, o.CustomerID;
```

It returns the same customers. It reads inside-out and scans `Orders` twice, which is why the CTE version is easier to maintain; MySQL requires the alias on the derived table (`AS per_customer`), and PostgreSQL did before version 16, so always write one.

Question 32: Products priced above their category average

Difficulty: Medium

Primary Topic: Correlated subquery

Combined Topics: Window AVG with PARTITION BY as an alternative

Business Scenario: The pricing team is reviewing products that sit at the expensive end of their own category before a festive sale. A product at 600 is cheap in cookware and expensive in snacks, so a single catalogue-wide cut-off is useless. The festive sale is in Beverages first, so they start there, and they want each product compared only with the products in its own category.

Problem Statement: For the Beverages category (`CategoryID` 5009), return every product whose `Sale_Price` is strictly greater than the average `Sale_Price` of all products with the same `CategoryID`. Show the category average next to it. Sort by `CategoryID`, then price, highest first.

Expected Output:

- `ProductID`, `Product`, `CategoryID`, `Sale_Price`
- `category_avg_price`: average `Sale_Price` of products in the same `CategoryID` (2 dp)

Tables Required: `Products`

SQL Concepts Tested: correlated subquery in WHERE and SELECT, outer-row references, NULL behaviour in the correlation, window AVG as a rewrite.

Step 1 — Understand the Problem: Each product needs its own benchmark. A plain subquery such as `(SELECT AVG(Sale_Price) FROM Products)` returns one number for the whole catalogue. Here the number depends on which product we are looking at. A correlated subquery solves that: it refers to a column of the outer row, so conceptually it is re-run for each product with that product's `CategoryID`.

The average is over catalogue rows, not sales. Every listed product counts once, whether it sold or not. We are comparing list prices, so `Category` and the order tables are not involved.

Step 2 — Identify the Tables and Relationships: Only `Products` is used, twice. The outer copy (`p`) is the product being tested. The inner copy (`p2`) is the set of products in the same category. The link between them is `p2.CategoryID = p.CategoryID`. Category names are not asked for, so `Category` is not joined.

Step 3 — Build the Query Logic:

1. Outer query: read each product from `Products AS p`.
2. Inner query: `SELECT AVG(p2.Sale_Price) FROM Products AS p2 WHERE p2.CategoryID = p.CategoryID`.
3. Keep the product when `p.Sale_Price` is greater than that inner value.
4. Repeat the inner query in SELECT to show the benchmark, rounded.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       p.CategoryID,
       p.Sale_Price,
       ROUND((SELECT AVG(p2.Sale_Price)
              FROM Products AS p2
              WHERE p2.CategoryID = p.CategoryID), 2) AS category_avg_price
FROM Products AS p
WHERE p.CategoryID = 5009
  AND p.Sale_Price > (SELECT AVG(p2.Sale_Price)
                      FROM Products AS p2
                      WHERE p2.CategoryID = p.CategoryID)
ORDER BY p.CategoryID, p.Sale_Price DESC, p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `FROM Products AS p` sets up the outer row. Every reference to `p` inside the subqueries points to it.
- `WHERE p2.CategoryID = p.CategoryID` is the correlation. For a product in category 5, the inner query averages only category 5.
- The inner `AVG` includes the outer product itself. That is what "category average" means; leave it in unless the business asks for "average of the other products".
- `p.Sale_Price > (...)` is strict. A product priced exactly at the average is not above it.
- The same subquery appears in SELECT so the reader can see the benchmark. PostgreSQL may evaluate it twice per row; Step 10 removes the repetition.
- If `p.CategoryID` is NULL, `p2.CategoryID = NULL` is never true, the inner AVG returns NULL, and the comparison is not true. Such products drop out silently. If the catalogue has them, decide whether they should be compared as their own group.

Step 6 — Sample Output:

productid	product	categoryid	sale_price	category_avg_price
7429	Nutritional Powder Vanilla Delight 1kg+ Nutritional Powder Premium Chocolate 1kg	5009	2330.00	244.65
18032	Glow - Assorted Loose Leaf Tea	5009	1995.00	244.65
18036	Founders Private Reserve Trio	5009	1995.00	244.65
7419	Slim Shake - Coffee	5009	1800.00	244.65
7420	Slim Shake - Guava	5009	1800.00	244.65
7432	Chocolate Slim Shake - Weight Control Powered By Ayurveda	5009	1800.00	244.65
18029	Tea Masters Private Reserve Duo	5009	1395.00	244.65
7400	Nutritional Powder - Premium Chocolate	5009	1160.00	244.65

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 262 rows.

Step 7 — Why This Solution Works: The correlation makes the benchmark move with the row, so each product is judged only against its own category. Averages are taken from the full `Products` table, not from the filtered result, because the inner query has its own FROM and is not affected by the outer WHERE. A one-product category can never appear: its average equals its only price, and the test is strict.

Step 8 — Common Mistakes:

- Forgetting the correlation. Without `WHERE p2.CategoryID = p.CategoryID`, the subquery returns the catalogue-wide average and the result is "products above the overall average":

```sql
SELECT p.ProductID, p.Product, p.CategoryID, p.Sale_Price
FROM Products AS p
WHERE p.Sale_Price > (SELECT AVG(p2.Sale_Price) FROM Products AS p2)   -- no correlation
ORDER BY p.CategoryID, p.Sale_Price DESC, p.ProductID;
```

- Writing `WHERE p.Sale_Price > AVG(p.Sale_Price)` with a GROUP BY. Aggregates are not allowed in WHERE, and grouping by product would compare each price with itself anyway.
- Using one alias for both copies (`FROM Products` inside and out with no aliases). The inner `CategoryID = CategoryID` then binds to the inner table on both sides and is true for every row with a category, which again gives the overall average.

Step 9 — Interview Follow-Up: The team now wants products more than 25% above their category average. Which single expression changes, and how would you show the percentage gap as a column?

Step 10 — Optimization or Alternative: A window average computes every category's average in one pass and attaches it to each row, so there is no per-row subquery and no repetition:

```sql
WITH priced AS (
    SELECT p.ProductID,
           p.Product,
           p.CategoryID,
           p.Sale_Price,
           AVG(p.Sale_Price) OVER (PARTITION BY p.CategoryID) AS category_avg_price
    FROM Products AS p
)
SELECT pr.ProductID, pr.Product, pr.CategoryID, pr.Sale_Price,
       ROUND(pr.category_avg_price, 2) AS category_avg_price
FROM priced AS pr
WHERE pr.Sale_Price > pr.category_avg_price
ORDER BY pr.CategoryID, pr.Sale_Price DESC, pr.ProductID;
```

The filter sits outside the CTE because window functions are evaluated after WHERE. One behaviour differs: `PARTITION BY` puts NULL `CategoryID` products in one partition together, so they are compared with each other instead of dropping out. No product in this extract has a NULL `CategoryID`, so here both forms return the same rows. The speed difference, though, is not theoretical. Without an index, each correlated subquery re-scans all 22,088 products for every outer row: in our test run the Beverages version (709 outer rows) took about a second per subquery, and dropping the category filter to cover the whole catalogue took about 26 seconds. The window version above covers the whole catalogue in milliseconds. An index on `Products(CategoryID)` narrows each subquery to one category, but the per-row repetition remains; the window form needs a single sort on `CategoryID` instead. Drop `WHERE p.CategoryID = 5009` from Step 4 only after switching to the window form.

Question 33: Products that have never been ordered

Difficulty: Medium

Primary Topic: NOT EXISTS anti-join

Combined Topics: NOT IN NULL trap, LEFT JOIN anti-join alternative

Business Scenario: The merchandising team pays for shelf space in the warehouse and wants to delist products that have never sold. Before anything is removed from the catalogue, they want the list of products with zero order lines in the entire history. A product that appeared in even one order line, in any quantity, is not on the list.

Problem Statement: Return every product in `Products` that does not appear in any row of `OrderDetails`. Sort by `CategoryID`, then `ProductID`.

Expected Output:

- `ProductID`, `Product`, `Brand`, `CategoryID`, `Sale_Price`

Tables Required: `Products`, `OrderDetails`

SQL Concepts Tested: NOT EXISTS, correlated subquery, why NOT IN fails with a NULL in the list, LEFT JOIN with IS NULL on the join key.

Step 1 — Understand the Problem: This is an anti-join: keep rows from one table that have no partner in another. Q18 did the same thing with a LEFT JOIN for customers. Here the focus is on the three ways candidates write it, and which one breaks.

"Never ordered" is about existence, not quantity. A line with `Quantity` NULL or 0 still means the product was put in an order. Whether that counts as "sold" is a business call; the rule here is presence in `OrderDetails`.

Step 2 — Identify the Tables and Relationships: `OrderDetails.ProductID` → `Products.ProductID`, many lines to one product. We read every product and check whether at least one line points to it. Lines that point to products not in the catalogue do not matter here; they belong in an orphan check.

Step 3 — Build the Query Logic:

1. Read each product from `Products AS p`.
2. For each one, ask: is there any line in `OrderDetails` with `od.ProductID = p.ProductID`?
3. Keep the product when the answer is no.
4. `NOT EXISTS` expresses step 2 and 3 directly. It stops scanning at the first match, so it does not need to count lines.

Step 4 — Final SQL Solution:

```sql
SELECT p.ProductID,
       p.Product,
       p.Brand,
       p.CategoryID,
       p.Sale_Price
FROM Products AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM OrderDetails AS od
    WHERE od.ProductID = p.ProductID
)
ORDER BY p.CategoryID, p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `FROM Products AS p` drives the query. Every product is tested once.
- `SELECT 1` inside EXISTS is a convention. EXISTS only checks whether a row comes back; the select list is ignored, so `SELECT *` or `SELECT NULL` behave the same.
- `WHERE od.ProductID = p.ProductID` correlates the subquery with the outer product.
- `NOT EXISTS` is true when the subquery returns no rows. It is never NULL: it is either true or false. That is why it is safe when `OrderDetails.ProductID` contains NULLs.
- `ORDER BY p.CategoryID, p.ProductID` groups the list by category for the reviewers, and `ProductID` is unique, so the order is fixed.

Step 6 — Sample Output:

productid	product	brand	categoryid	sale_price
2	Surface Disinfectant Spray	Savlon	5001	257.00
4	Harpic Toilet Cleaner Liquid - Original 1 L + Lizol Floor Cleaner, Citrus - 2L	bb Combo	5001	462.00
10	Lizol Floor Cleaner, Floral - 2L + Harpic Toilet Cleaner Liquid - Original 1L	bb Combo	5001	483.00
14	Disinfectant Floor Cleaner Liquid - Floral, Kills 99.9% Germs	Lizol	5001	445.00
16	Disinfectant Surface & Floor Cleaner Liquid - [Citrus 2 L + Floral 2 L]	Lizol	5001	613.00
19	Disinfectant Surface & Floor Cleaner - [Citrus, Floral & Lavender (500 ml Each)]	Lizol	5001	253.00
23	Bathroom Cleaner Liquid - Lemon 1 L + Toilet Cleaner - Bleach, White & Shine - 500 ml	Harpic	5001	228.00
24	Disinfectant Spray Sanitizer For Germ-Protection - Original Pine	Dettol	5001	287.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 5564 rows.

Step 7 — Why This Solution Works: EXISTS uses two-valued logic. A NULL `od.ProductID` simply fails the `=` test, so that line does not count as a match, and nothing else changes. The query returns a product exactly when no line references it. PostgreSQL usually plans NOT EXISTS as a hash anti-join, which reads each table once.

Step 8 — Common Mistakes:

- `NOT IN` over a column that can hold NULL. `x NOT IN (1, 2, NULL)` means `x <> 1 AND x <> 2 AND x <> NULL`. The last part is never true, so the whole condition is never true, and the query returns nothing. The `UNION ALL SELECT NULL` below simulates one line with a missing `ProductID`:

```sql
SELECT p.ProductID, p.Product
FROM Products AS p
WHERE p.ProductID NOT IN (SELECT od.ProductID FROM OrderDetails AS od
                          UNION ALL SELECT NULL)   -- one NULL is enough
ORDER BY p.ProductID;
```

This returns zero rows, with no error and no warning. If you must use NOT IN, add `WHERE od.ProductID IS NOT NULL` inside the subquery.

- A LEFT JOIN anti-join that tests the wrong column. `WHERE od.Quantity IS NULL` also matches products that did sell but have a line with a missing quantity, so a selling product gets delisted:

```sql
SELECT DISTINCT p.ProductID, p.Product
FROM Products AS p
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
WHERE od.Quantity IS NULL          -- should test the join key
ORDER BY p.ProductID;
```

Test the column used in the ON clause, `od.ProductID IS NULL`, which can only be NULL when no line matched.

- Using `COUNT(*) = 0` with a GROUP BY after an inner join. The inner join has already removed the products you are looking for, so the HAVING clause never sees them.

Step 9 — Interview Follow-Up: The team now wants products not ordered in the last 180 days of data, including products never ordered at all. Which table do you add to the subquery, and where does the date condition go?

Step 10 — Optimization or Alternative: The LEFT JOIN form gives the same rows when written correctly:

```sql
SELECT p.ProductID, p.Product, p.Brand, p.CategoryID, p.Sale_Price
FROM Products AS p
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
WHERE od.ProductID IS NULL
ORDER BY p.CategoryID, p.ProductID;
```

Unmatched products produce exactly one row each, so no DISTINCT is needed. In PostgreSQL both forms usually produce the same anti-join plan; NOT EXISTS is easier to read and cannot be broken by a wrong IS NULL column. On a large `OrderDetails`, an index on `OrderDetails(ProductID)` lets the planner probe instead of hashing the whole table when `Products` is small.

Question 34: Each supplier's share of units supplied

Difficulty: Medium

Primary Topic: Window function over an aggregate

Combined Topics: FULL OUTER JOIN to keep both unmatched sides, Percentage contribution

Business Scenario: The supply chain team is reviewing dependence on each supplier before annual contracts are renewed. For every supplier on file they want the units it fulfilled and its share of all units. Two edge cases matter: a supplier with no lines must still appear with zero (it may be a contract to drop), and lines tagged with a `SupplierID` that is not on file must appear too, so the shares add up to 100.

Problem Statement: For every supplier, and for every unknown `SupplierID` found on order lines, return the units supplied and the share of all units. Suppliers with no lines show 0 units. Sort by units, highest first.

Expected Output:

- `SupplierID`: the supplier's ID, or the unrecognised ID from the lines
- `CompanyName`: the supplier's name, or `Unknown supplier`
- `units_supplied`: sum of `Quantity` (0 when none)
- `pct_of_units`: share of all units (2 dp)

Tables Required: `OrderDetails`, `Suppliers`

SQL Concepts Tested: window function over a grouped result (`SUM(SUM(...)) OVER ()`), FULL OUTER JOIN, COALESCE on join keys, percentage of total.

Step 1 — Understand the Problem: A share needs the part (one supplier's units) and the whole (everyone's units) on the same row. Q28 got the whole from a scalar subquery, which scans the table again. A window function reads it from the grouped rows instead: after GROUP BY gives one row per supplier, `SUM(...) OVER ()` adds a column across all of them.

The other half is keeping rows from both sides. A LEFT JOIN from `Suppliers` keeps idle suppliers but drops lines with unknown IDs; a LEFT JOIN from `OrderDetails` does the opposite. You need both.

Step 2 — Identify the Tables and Relationships: `OrderDetails.SupplierID` → `Suppliers.SupplierID`, many lines to one supplier. No foreign key is enforced, so the relationship can fail in both directions: a supplier with no lines, and lines with no supplier. A FULL OUTER JOIN keeps unmatched rows from both tables, filling the missing side with NULLs.

Step 3 — Build the Query Logic:

1. `Suppliers FULL OUTER JOIN OrderDetails` on `SupplierID`.
2. Build the output key with `COALESCE(s.SupplierID, od.SupplierID)`: the supplier's ID if on file, otherwise the ID from the line.
3. Group by that key and the name. `SUM(od.Quantity)` gives units; COALESCE turns a supplier with no lines into 0.
4. Divide by `SUM(SUM(od.Quantity)) OVER ()`, the total across all groups.

Step 4 — Final SQL Solution:

```sql
SELECT COALESCE(s.SupplierID, od.SupplierID)                        AS SupplierID,
       COALESCE(s.CompanyName, 'Unknown supplier')                  AS CompanyName,
       COALESCE(SUM(od.Quantity), 0)                                AS units_supplied,
       ROUND(100.0 * COALESCE(SUM(od.Quantity), 0)
             / NULLIF(SUM(SUM(od.Quantity)) OVER (), 0), 2)         AS pct_of_units
FROM Suppliers AS s
FULL OUTER JOIN OrderDetails AS od ON od.SupplierID = s.SupplierID
GROUP BY COALESCE(s.SupplierID, od.SupplierID), s.CompanyName
ORDER BY units_supplied DESC, SupplierID NULLS LAST;
```

Step 5 — Line-by-Line Explanation:

- `FULL OUTER JOIN` returns matched pairs, plus suppliers with no line (line columns NULL), plus lines with no supplier (supplier columns NULL).
- `COALESCE(s.SupplierID, od.SupplierID)` picks whichever side exists. For a known supplier both are equal; for an orphan line only `od.SupplierID` is set.
- Grouping by the coalesced ID gives each unknown ID its own row, which tells the data team exactly which keys to chase. Lines with a NULL `SupplierID` group together under a NULL ID.
- `SUM(od.Quantity)` skips NULL quantities. For an idle supplier every `od.Quantity` is NULL, so the SUM is NULL and `COALESCE(..., 0)` shows 0.
- `SUM(SUM(od.Quantity)) OVER ()`: the inner SUM is the group's units; the outer SUM with `OVER ()` adds those group totals across every

output row. Window functions run after GROUP BY, so they see one row per supplier.

- `ORDER BY ... SupplierID NULLS LAST` breaks ties between suppliers with equal units.

Step 6 — Sample Output:

supplierid	companyname	units_supplied	pct_of_units
4	Ganga Distributors LLP	43166	13.73
2	Deccan Fresh Foods Pvt Ltd	40106	12.75
5	Coromandel FMCG Supplies	37717	11.99
10	Indus Personal Care Pvt Ltd	35574	11.31
8	Sabarmati Packaged Foods	32001	10.18
9	Hooghly Consumer Goods	31864	10.13
6	Malabar Spice Exports	20791	6.61
1	Shree Balaji Agro Traders	19591	6.23

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 19 rows.

Step 7 — Why This Solution Works: Every order line lands in exactly one group, matched or not, so the denominator equals `SELECT SUM(Quantity) FROM OrderDetails`. Every supplier on file also gets a row, with 0 when it fulfilled nothing. The window total comes from the same grouped rows as the numerators, so `pct_of_units` adds up to 100 (give or take rounding).

Step 8 — Common Mistakes:

- `SUM(od.Quantity) OVER ()` without the inner aggregate. In a grouped query, `od.Quantity` is not a grouped column, so PostgreSQL rejects it: the window can only see grouped values and aggregates.
- An INNER JOIN. Idle suppliers disappear and unknown-ID units drop out of both the rows and the total, so the shares still add to 100 while describing less than the real volume.
- Counting lines instead of units. `COUNT(*)` measures how often a supplier appears on an order, not how much it shipped:

```sql
SELECT s.CompanyName,
       COUNT(*) AS lines_not_units
FROM OrderDetails AS od
JOIN Suppliers AS s ON s.SupplierID = od.SupplierID
GROUP BY s.CompanyName
ORDER BY lines_not_units DESC, s.CompanyName;
```

Step 9 — Interview Follow-Up: The team wants a `cumulative_pct` column so they can see how many suppliers make up 80% of units. Which window frame would you add, and why does the ORDER BY inside OVER need a tiebreaker?

Step 10 — Optimization or Alternative: Pre-aggregating the lines makes the join small, and the window then runs over plain columns:

```sql
WITH units_by_supplier AS (
    SELECT od.SupplierID, SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.SupplierID
)
SELECT COALESCE(s.SupplierID, u.SupplierID)            AS SupplierID,
       COALESCE(s.CompanyName, 'Unknown supplier')     AS CompanyName,
       COALESCE(u.units, 0)                            AS units_supplied,
       ROUND(100.0 * COALESCE(u.units, 0)
             / NULLIF(SUM(u.units) OVER (), 0), 2)     AS pct_of_units
FROM Suppliers AS s
FULL OUTER JOIN units_by_supplier AS u ON u.SupplierID = s.SupplierID
ORDER BY units_supplied DESC, SupplierID NULLS LAST;
```

MySQL has no FULL OUTER JOIN. There, take a LEFT JOIN from `Suppliers` and UNION ALL the CTE rows whose `SupplierID` is not in `Suppliers`.

Question 35: Late-delivery rate by shipper

Difficulty: Medium

Primary Topic: Conditional aggregation

Combined Topics: CASE status in a CTE, Exclusion rules for invalid dates, LEFT JOIN

Business Scenario: The logistics team reviews courier performance every quarter and moves volume away from the worst performer. They want each shipper's late-delivery rate. There is no promised delivery date in the data, so the team has agreed a working rule: an order is late when it is delivered more than 7 days after it was placed.

Problem Statement: For each shipper, count delivered orders, late orders and the late rate. Count orders with no delivery date separately, and keep orders with impossible date sequences out of the rate but visible in their own column. Orders with a missing or unknown `ShipperID` go under `Unknown shipper`. Sort by late rate, highest first.

Expected Output:

- `shipper_name`: `CompanyName`, or `Unknown shipper`
- `delivered_orders`: orders with a valid delivery date (the rate's denominator)
- `late_orders`: delivered orders with `DeliveryDate - OrderDate > 7`
- `late_rate_pct`: late as a percentage of delivered (2 dp)
- `not_delivered`: orders with no `DeliveryDate`
- `invalid_dates`: orders excluded because their dates are out of sequence

Tables Required: `Orders`, `Shippers`

SQL Concepts Tested: classifying rows with CASE, counting by class with `COUNT(*) FILTER`, building a rate from two conditional counts, stating and applying exclusion rules.

Step 1 — Understand the Problem: A rate's numerator and denominator must come from the same population: here, orders that were delivered and whose dates make sense. Two kinds of order fall outside it:

- No `DeliveryDate`. The order may still be in transit (common near the end of the data), or the date was never recorded. We cannot say whether it was late.
- Impossible sequences: delivered before ordered, shipped before ordered, or delivered before shipped. At least one date is wrong and we cannot tell which.

Assumption, stated once: late means `DeliveryDate::date - OrderDate::date > 7`, a one-week delivery promise. If a promised date is added later, only this line changes.

Step 2 — Identify the Tables and Relationships: `Orders` holds all three dates and `ShipperID`. `Shippers` turns the ID into a name (`Orders.ShipperID` → `Shippers.ShipperID`, many to one). LEFT JOIN, so orders with no shipper stay in the report.

Step 3 — Build the Query Logic:

1. In a CTE, give each order exactly one `delivery_status` with CASE: `Not delivered`, `Invalid dates`, `Late` or `On time`. The branches are checked in order, so missing dates are caught before any arithmetic.
2. Group by shipper and count each status with `COUNT(*) FILTER (WHERE ...)`.
3. Divide late by (late + on time) for the rate.

Step 4 — Final SQL Solution:

```sql
WITH classified AS (
    SELECT COALESCE(sh.CompanyName, 'Unknown shipper') AS shipper_name,
           CASE
               WHEN o.DeliveryDate IS NULL                       THEN 'Not delivered'
               WHEN o.DeliveryDate < o.OrderDate
                 OR o.ShipDate     < o.OrderDate
                 OR o.DeliveryDate < o.ShipDate                  THEN 'Invalid dates'
               WHEN o.DeliveryDate::date - o.OrderDate::date > 7 THEN 'Late'
               ELSE 'On time'
           END AS delivery_status
    FROM Orders AS o
    LEFT JOIN Shippers AS sh ON sh.ShipperID = o.ShipperID
)
SELECT shipper_name,
       COUNT(*) FILTER (WHERE delivery_status IN ('Late', 'On time')) AS delivered_orders,
       COUNT(*) FILTER (WHERE delivery_status = 'Late')               AS late_orders,
       ROUND(100.0 * COUNT(*) FILTER (WHERE delivery_status = 'Late')
             / NULLIF(COUNT(*) FILTER (WHERE delivery_status IN ('Late', 'On time')), 0), 2)
                                                                     AS late_rate_pct,
       COUNT(*) FILTER (WHERE delivery_status = 'Not delivered')     AS not_delivered,
       COUNT(*) FILTER (WHERE delivery_status = 'Invalid dates')     AS invalid_dates
FROM classified
GROUP BY shipper_name
ORDER BY late_rate_pct DESC NULLS LAST, shipper_name;
```

Step 5 — Line-by-Line Explanation:

- The CASE assigns one status per order. Because CASE stops at the first true branch, the statuses never overlap.
- `o.ShipDate < o.OrderDate` is NULL when `ShipDate` is missing. In an OR, NULL only matters if nothing else is true, and CASE treats a NULL condition as not true. So an order delivered with no ship date is still measurable on order-to-delivery days.
- `o.DeliveryDate::date - o.OrderDate::date` gives whole days whether the columns are DATE or TIMESTAMP.
- `COUNT(*) FILTER (WHERE ...)` counts only the rows that pass the condition. It is PostgreSQL syntax; in MySQL write `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`.
- `NULLIF(..., 0)` gives a NULL rate, not an error, for a shipper with no measurable deliveries. `NULLS LAST` keeps that row at the bottom.

Step 6 — Sample Output:

shipper_name	delivered_orders	late_orders	late_rate_pct	not_delivered	invalid_dates
NorthStar Freight	2103	1754	83.40	65	2
Bharat Parcel Service	2988	1627	54.45	75	3
Unknown shipper	28	5	17.86	0	0
BlueLine Express	7969	1287	16.15	157	3
XpressHaul	4792	746	15.57	87	2

Ecomzip Couriers	5830	527	9.04	108	2
ShadowRun Delivery	5005	164	3.28	84	1
SwiftKart Logistics	9253	232	2.51	138	5

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 9 rows.

Step 7 — Why This Solution Works: Every order gets exactly one status, so for each shipper `delivered_orders + not_delivered + invalid_dates` equals its total orders. That identity is the check to show an interviewer: nothing is dropped and nothing is counted twice. The rate's numerator (`Late`) is a subset of its denominator (`Late` plus `On time`), so it can never exceed 100.

Step 8 — Common Mistakes:

- Dividing late orders by all orders. Undelivered and invalid orders sit in the denominator, can never be late, and pull the rate down. A shipper with many in-transit orders looks better than it is.
- Leaving invalid sequences in. A negative day count is never above 7, so it counts as on time.
- Writing `AVG(CASE WHEN days > 7 THEN 1 ELSE 0 END)` over all orders. When days is NULL the CASE returns 0, so undelivered orders count as on time:

```
SELECT COALESCE(sh.CompanyName, 'Unknown shipper') AS shipper_name,
       ROUND(100.0 * AVG(CASE WHEN o.DeliveryDate::date - o.OrderDate::date > 7
                              THEN 1 ELSE 0 END), 2) AS late_rate_wrong
FROM Orders AS o
LEFT JOIN Shippers AS sh ON sh.ShipperID = o.ShipperID
GROUP BY COALESCE(sh.CompanyName, 'Unknown shipper')
ORDER BY late_rate_wrong DESC, shipper_name;
```

Step 9 — Interview Follow-Up: A shipper picks up orders slowly but delivers fast once it has them. How would you split those days between the warehouse and the courier, and which date column makes that possible?

Step 10 — Optimization or Alternative: You can skip the CTE and write the conditions straight into each FILTER. It runs the same, but each rule then appears two or three times, and a fix to one copy can miss the others. The single CASE keeps the rules in one place. For a quarterly review, add an `OrderDate` range anchored to `(SELECT MAX(OrderDate) FROM Orders)` so only the period is scanned.

Question 36: Payment method mix by month as columns

Difficulty: Medium

Primary Topic: Pivot with FILTER

Combined Topics: DATE_TRUNC grouping, LEFT JOIN, Catch-all column

Business Scenario: The payments team sends a monthly sheet to the finance controller showing how customers paid. The controller reads it across, one row per month with one column per method, so a shift from cards and cash on delivery to UPI is visible at a glance. A long format (one row per month and method) is not acceptable for this sheet.

Problem Statement: For each order month, return the number of orders paid by UPI, Credit Card, Debit Card, Net Banking, Cash on Delivery and Wallet (EMI and Pay Later go to the catch-all column) as separate columns. Add `other_or_unknown` for orders whose payment type is missing or not in that list, and `total_orders`. Sort by month, latest first.

Expected Output:

- `order_month`: first day of the month
- `upi`, `credit_card`, `debit_card`, `net_banking`, `cash_on_delivery`, `wallet`: order counts
- `other_or_unknown`: orders with no matching payment row or a type outside the list
- `total_orders`: all orders in the month

Tables Required: `Orders`, `Payments`

SQL Concepts Tested: turning rows into columns with conditional aggregation, `COUNT(*) FILTER`, month grouping with DATE_TRUNC, a catch-all column so the pivot reconciles.

Step 1 — Understand the Problem: A normal GROUP BY on month and payment type returns one row per pair, which is the long format. To pivot, group by month only and compute one aggregate per column, each counting only the rows for its method. That is conditional aggregation: same table, same group, a different condition per column.

The column list is written by hand. SQL needs to know the output columns before it reads any data, so a new payment method will not get a column on its own. That is why the catch-all column exists: anything not named lands there instead of disappearing.

Step 2 — Identify the Tables and Relationships: `Orders` gives the date and the `PaymentID`. `Payments` gives `PaymentType` (`Orders.PaymentID` → `Payments.PaymentID`, many to one). LEFT JOIN, so orders with a NULL or unknown `PaymentID` stay in the month's total.

Step 3 — Build the Query Logic:

1. LEFT JOIN `Payments` and compute `DATE_TRUNC('month', o.OrderDate)` as the month.
2. Group by month.
3. One `COUNT(*) FILTER (WHERE pay.PaymentType = '...')` per named method.

4. For the catch-all, count rows where the type is NULL or not in the named list. The NULL test must be explicit, because `NOT IN` returns NULL, not true, for a NULL type.
5. `COUNT(*)` for the total.

Step 4 — Final SQL Solution:

```sql
SELECT DATE_TRUNC('month', o.OrderDate)::date                              AS order_month,
       COUNT(*) FILTER (WHERE pay.PaymentType = 'UPI')                     AS upi,
       COUNT(*) FILTER (WHERE pay.PaymentType = 'Credit Card')             AS credit_card,
       COUNT(*) FILTER (WHERE pay.PaymentType = 'Debit Card')              AS debit_card,
       COUNT(*) FILTER (WHERE pay.PaymentType = 'Net Banking')             AS net_banking,
       COUNT(*) FILTER (WHERE pay.PaymentType = 'Cash on Delivery')        AS cash_on_delivery,
       COUNT(*) FILTER (WHERE pay.PaymentType = 'Wallet')                  AS wallet,
       COUNT(*) FILTER (WHERE pay.PaymentType IS NULL
                           OR pay.PaymentType NOT IN ('UPI', 'Credit Card', 'Debit Card',
                                                      'Net Banking', 'Cash on Delivery', 'Wallet'))
                                                                           AS other_or_unknown,
       COUNT(*)                                                            AS total_orders
FROM Orders AS o
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
WHERE o.OrderDate IS NOT NULL
GROUP BY DATE_TRUNC('month', o.OrderDate)
ORDER BY order_month DESC;
```

Step 5 — Line-by-Line Explanation:

- `DATE_TRUNC('month', o.OrderDate)::date` maps every date to the first of its month, so March 2024 and March 2025 stay apart. The cast drops the time part from the output.
- Each `COUNT(*) FILTER (WHERE ...)` counts the group's rows that pass its own condition. Every column sees the same month's rows.
- The literals must match the stored `PaymentType` exactly. If the table stores `'Net Banking'` in one row and `'Net banking'` in another, or `'UPI '` with a trailing space, compare on `UPPER(TRIM(pay.PaymentType))` instead.
- `pay.PaymentType IS NULL` catches orders whose `PaymentID` is NULL or matches nothing, because the LEFT JOIN fills `pay` with NULLs for them.
- `WHERE o.OrderDate IS NOT NULL` keeps undated orders out of a NULL month row. Report them separately if they exist.
- `ORDER BY order_month DESC` puts the latest month first; months are unique, so the order is fixed.

Step 6 — Sample Output:

order_month	upi	credit_card	debit_card	net_banking	cash_on_delivery	wallet	other_or_unknown	total_orders
2025-12-01	1573	551	255	129	254	159	9	2930
2025-11-01	2072	666	331	144	311	204	4	3732
2025-10-01	2023	680	336	170	292	214	10	3725
2025-09-01	1362	450	243	96	227	154	6	2538
2025-08-01	1336	432	209	115	246	155	8	2501
2025-07-01	1194	414	198	93	207	134	8	2248
2025-06-01	1041	408	181	86	156	96	90	2058
2025-05-01	989	330	160	79	206	115	77	1956

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 36 rows.

Step 7 — Why This Solution Works: Each order falls into exactly one of the seven count columns: either its type is one of the six names, or it is NULL or something else, which the catch-all takes. So for every row, the seven columns add up to `total_orders`. Check that identity once on real data; if it fails, a type has a spelling you did not expect.

Step 8 — Common Mistakes:

- A catch-all written as `NOT IN (...)` alone. For a NULL type, `NULL NOT IN (...)` is NULL, so unknown-payment orders are counted nowhere and the row stops adding up:

```sql
SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
       COUNT(*) FILTER (WHERE pay.PaymentType NOT IN ('UPI', 'Credit Card', 'Debit Card',
                                                      'Net Banking', 'Cash on Delivery', 'Wallet'))
                                             AS other_wrong,
       COUNT(*)                              AS total_orders
FROM Orders AS o
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
GROUP BY DATE_TRUNC('month', o.OrderDate)
ORDER BY order_month DESC;
```

- Grouping by `EXTRACT(MONTH FROM o.OrderDate)`, which merges the same month from different years.
- Using `SUM(CASE WHEN ... THEN 1 END)` without `ELSE 0`. A month with no Wallet orders shows NULL instead of 0. `COUNT(*) FILTER` returns 0.

Step 9 — Interview Follow-Up: The controller now wants each column as a percentage of the month's orders instead of a count. How would you change one column, and what stops a division by zero?

Step 10 — Optimization or Alternative: In MySQL, FILTER does not exist; write `SUM(CASE WHEN pay.PaymentType = 'UPI' THEN 1 ELSE 0 END)`. PostgreSQL also has `crosstab()` in the `tablefunc` extension, but it needs the extension installed and its column list is still fixed. When methods change often, return the long format (`order_month, payment_type, orders`) from SQL and let the reporting tool pivot it.

Question 37: Prove no order was paid with a method that is not allowed

Difficulty: Medium

Primary Topic: LEFT JOIN with a normalised flag check

Combined Topics: Conditional status with CASE, Pre-aggregation, Business-rule validation

Business Scenario: The risk team has switched off some payment methods, marked `Allowed = 'No'` in `Payments`. Before the quarterly audit they want proof that no order was taken through a disallowed method. "Zero rows came back" is not proof an auditor accepts, because a wrong join also returns zero rows. They want one row per payment method with its flag, its order count and a status, so the clean result is visible.

Problem Statement: For every row in `Payments`, return the raw `Allowed` value, a normalised yes/no, the number of orders and their total value, and a status: `OK` (allowed and used), `Allowed, unused`, `Not allowed, unused`, or `VIOLATION: not allowed but used`. A blank or NULL flag counts as not allowed. Sort violations first, then by `PaymentID`.

Expected Output:

- `PaymentID`, `PaymentType`
- `allowed_raw`: the value as stored
- `is_allowed`: TRUE or FALSE after normalising
- `orders`: orders paid with this method (0 if none)
- `order_value`: sum of `Total_Order_Amount` (0 if none)
- `status`: one of the four labels above

Tables Required: `Payments`, `Orders`

SQL Concepts Tested: driving the result from the lookup table so unused methods appear, aggregating orders before joining, normalising a text flag, turning a rule into an explicit status.

Step 1 — Understand the Problem: The question looks like "find bad orders", but the deliverable is a control report: every payment method, with evidence for each. That changes the grain to one row per payment method and the driving table to `Payments`. A method nobody used still needs a row, with 0 orders.

Step 2 — Identify the Tables and Relationships: `Payments` holds each method and its `Allowed` flag. `Orders.PaymentID` → `Payments.PaymentID`, one method to many orders. The data dictionary describes `Allowed` as a bit with "Yes"/"No" values, while Q9 showed what is actually stored, so the flag is normalised rather than compared to one literal.

Step 3 — Build the Query Logic:

1. Aggregate `Orders` to one row per `PaymentID`: count and value.
2. LEFT JOIN that from `Payments`, so unused methods keep a row; COALESCE the counts to 0.
3. Normalise the flag with the house expression; anything not clearly "yes" (including NULL) is FALSE.
4. Map flag and usage to a status with CASE; sort violations first.

Step 4 — Final SQL Solution:

```sql
WITH usage AS (
    SELECT o.PaymentID,
           COUNT(*)                 AS orders,
           SUM(o.Total_Order_Amount) AS order_value
    FROM Orders AS o
    GROUP BY o.PaymentID
),
checked AS (
    SELECT pay.PaymentID,
           pay.PaymentType,
           pay.Allowed                                                    AS allowed_raw,
           COALESCE(UPPER(TRIM(CAST(pay.Allowed AS VARCHAR)))
                    IN ('1', 'TRUE', 'T', 'Y', 'YES'), FALSE)             AS is_allowed,
           COALESCE(u.orders, 0)                                          AS orders,
           COALESCE(u.order_value, 0)                                     AS order_value
    FROM Payments AS pay
    LEFT JOIN usage AS u ON u.PaymentID = pay.PaymentID
)
SELECT ch.PaymentID,
       ch.PaymentType,
       ch.allowed_raw,
       ch.is_allowed,
       ch.orders,
       ch.order_value,
       CASE WHEN NOT ch.is_allowed AND ch.orders > 0 THEN 'VIOLATION: not allowed but used'
            WHEN NOT ch.is_allowed                   THEN 'Not allowed, unused'
            WHEN ch.orders = 0                       THEN 'Allowed, unused'
            ELSE 'OK'
       END AS status
FROM checked AS ch
ORDER BY (NOT ch.is_allowed AND ch.orders > 0) DESC, ch.PaymentID;
```

Step 5 — Line-by-Line Explanation:

- `usage` collapses `Orders` to one row per method, so the join below cannot multiply anything.
- `FROM Payments ... LEFT JOIN usage` makes `Payments` the driving table. Every method appears, used or not.
- `COALESCE(... IN (...), FALSE)` normalises the flag. If `Allowed` is NULL, the IN test returns NULL, and COALESCE turns that into FALSE, so a missing flag is treated as "not allowed".
- `COALESCE(u.orders, 0)` gives unused methods a real 0 instead of NULL.
- The CASE checks the violation first, so a disallowed method that was used can never be labelled anything milder.
- `ORDER BY (...) DESC` sorts on a boolean: TRUE (violations) comes first when descending.

Step 6 — Sample Output:

paymentid	paymenttype	allowed_raw	is_allowed	orders	order_value	status
8	Pay Later	No	false	1129	1440094.95	VIOLATION: not allowed but used
1	UPI	Yes	true	23094	30920908.50	OK
2	Credit Card	Yes	true	8282	10537736.55	OK
3	Debit Card	Yes	true	4290	5558048.55	OK
4	Net Banking	Yes	true	2076	2827669.00	OK
5	Cash on Delivery	Yes	true	4723	6142240.50	OK
6	Wallet	Yes	true	2743	3375942.48	OK
7	EMI	Yes	true	81	431023.00	OK

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 10 rows.

Step 7 — Why This Solution Works: Every payment method produces exactly one row, so the report shows its own completeness: an auditor can count the rows against `Payments` and see each method's usage. A clean result reads as a set of `OK` and `unused` statuses rather than an empty grid. If a disallowed method is ever used, its row moves to the top with the order count and value attached, which is the evidence the risk team needs to act.

Step 8 — Common Mistakes:

- Writing only the violation query and treating an empty result as proof. It cannot tell "no violations" apart from a join on the wrong column:

```sql
SELECT o.OrderID, pay.PaymentType, pay.Allowed
FROM Orders AS o
JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
WHERE COALESCE(UPPER(TRIM(CAST(pay.Allowed AS VARCHAR))) IN ('1', 'TRUE', 'T', 'Y', 'YES'), FALSE) = FALSE
ORDER BY o.OrderID;
```

- Comparing `pay.Allowed = 'No'`. A stored `'no'`, `'N'`, `'0'` or NULL slips through.
- Joining `Orders` first and counting with `COUNT(*)` after a LEFT JOIN from `Payments`. Unused methods show 1 order instead of 0, because COUNT(*) counts the NULL-extended row.
- Remember that `Allowed` describes the method's status today. A method switched off last month makes its historical orders look like violations; a fair audit compares each order to the flag in force on its `OrderDate`, which this schema does not store.

Step 9 — Interview Follow-Up: The risk team adds a `DisabledFrom` date to `Payments`. How would you change the query so an order only counts as a violation when it was placed after its method was disabled?

Step 10 — Optimization or Alternative: In a pipeline, keep this as a scheduled check that fails the run when any row has the violation status (`SELECT COUNT(*) ... WHERE status LIKE 'VIOLATION%'`, expected 0). A foreign key cannot enforce the rule, because the flag lives on the parent row; a trigger or an application check at checkout can, and this report stays as the independent control.

Question 38: Products from inactive categories that still sold in the latest year

Difficulty: Medium

Primary Topic: Multi-join aggregation with a business-rule filter

Combined Topics: Anchored latest year, Flag normalisation, CTE

Business Scenario: The catalogue governance team switches categories off when the business exits them. A switched-off category should stop selling, but the storefront hides products by category only on some pages. The team wants every product in an inactive category that still appeared in orders during the latest calendar year in the data, with units sold and the last order date, so they can close each gap.

Problem Statement: Find the latest calendar year in `Orders` from the data itself. For every product whose category is not active and that had order lines in that year, return the units sold in that year and the most recent order date. Sort by units, highest first.

Expected Output:

- `ProductID`, `Product`
- `CategoryName`
- `units_latest_year`: sum of `Quantity` on orders placed in the latest year
- `last_order_date`: latest `OrderDate` for the product in that year

Tables Required: `Products`, `Category`, `OrderDetails`, `Orders`

SQL Concepts Tested: four-table join, anchoring a date window to the data, normalised flag test, filtering before aggregating, half-open year range.

Step 1 — Understand the Problem: Two conditions decide whether an order line counts: its product's category is inactive, and its order falls in the latest year. Both are row-level conditions, so they go in WHERE, and the surviving lines are then grouped by product.

"Latest year" means the latest year in the data, not today's calendar year. A report on a fixed snapshot must give the same answer whenever it runs, so the year is anchored to `(SELECT MAX(OrderDate) FROM Orders)`, truncated to 1 January. `CURRENT_DATE` would return an empty report once the calendar moves on.

Step 2 — Identify the Tables and Relationships: The chain runs `Category` ← `Products` ← `OrderDetails` → `Orders`.

- `Products.CategoryID` → `Category.CategoryID` gives the flag and name.
- `OrderDetails.ProductID` → `Products.ProductID` gives units per line.
- `OrderDetails.OrderID` → `Orders.OrderID` gives the order date.

Inner joins are correct here. A product with no category row cannot be shown as inactive, and a line with no order has no date to test.

Step 3 — Build the Query Logic:

1. CTE `latest_year`: `DATE_TRUNC('year', MAX(OrderDate))` as `year_start`.
2. Join the four tables and cross join the one-row CTE.
3. WHERE: category not active (normalised, NULL counts as not active) and `OrderDate` in `[year_start, year_start + 1 year)`.
4. Group by product and name; sum units, take the max date.

Step 4 — Final SQL Solution:

```sql
WITH latest_year AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS year_start
    FROM Orders AS o
)
SELECT p.ProductID,
       p.Product,
       cat.CategoryName,
       SUM(od.Quantity)  AS units_latest_year,
       MAX(o.OrderDate)  AS last_order_date
FROM OrderDetails AS od
JOIN Orders       AS o   ON o.OrderID      = od.OrderID
JOIN Products     AS p   ON p.ProductID    = od.ProductID
JOIN Category     AS cat ON cat.CategoryID = p.CategoryID
CROSS JOIN latest_year AS ly
WHERE (UPPER(TRIM(CAST(cat.Active AS VARCHAR))) IN ('1', 'TRUE', 'T', 'Y', 'YES')) IS NOT TRUE
  AND o.OrderDate >= ly.year_start
  AND o.OrderDate <  ly.year_start + INTERVAL '1 year'
GROUP BY p.ProductID, p.Product, cat.CategoryName
ORDER BY units_latest_year DESC NULLS LAST, p.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `latest_year` returns one row: 1 January of the year that holds the latest order. This is the anchor.
- `CROSS JOIN latest_year` makes `ly.year_start` available on every row. Crossing with one row adds no duplicates.

- The flag test is the normalised one from Q9. `IS NOT TRUE` keeps categories whose flag is false or NULL; a blank flag is not a clear "active".
- The date filter is half-open: on or after 1 January, strictly before the next 1 January. It works whether `OrderDate` is a DATE or a TIMESTAMP.
- `SUM(od.Quantity)` adds units across all qualifying lines for the product. `MAX(o.OrderDate)` is the last time it sold within the year.
- `GROUP BY p.ProductID, p.Product, cat.CategoryName` keeps one row per product. PostgreSQL would let you drop `p.Product`, because `ProductID` is the primary key of `Products`, but not `cat.CategoryName`, which comes from another table. Listing all three works everywhere.

Step 6 — Sample Output:

productid	product	categoryname	units_latest_year	last_order_date
21624	Country (Desi) Eggs - Small, Antibiotic Residue-Free	Eggs, Meat & Fish	299	2025-12-28
21632	The Good Eggs Cage Free Eggs	Eggs, Meat & Fish	212	2025-12-24
21648	Eggs - Brown Shell	Eggs, Meat & Fish	167	2025-12-27
21628	Healthy Brown Eggs	Eggs, Meat & Fish	158	2025-12-27
12927	Chicken Coppata Salami	Eggs, Meat & Fish	150	2025-12-27
22068	Fresh Catch Prawns - Small	Eggs, Meat & Fish	115	2025-12-27
22078	Tuna Flakes - in Soya Bean Oil	Eggs, Meat & Fish	108	2025-12-23
12986	Fried Strips - Chicken	Eggs, Meat & Fish	106	2025-12-24

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 558 rows.

Step 7 — Why This Solution Works: Both filters are applied to individual lines before grouping, so the sums only include lines that pass both rules. The joins are all many-to-one when read from the line side, so no line is repeated and the unit totals are exact. Because the year comes from the data, the report is repeatable on any snapshot.

Step 8 — Common Mistakes:

- Using `CURRENT_DATE`. On historical data the "current year" has no orders and the report says there is no problem:

```
SELECT COUNT(*) AS lines_this_calendar_year
FROM Orders AS o
JOIN OrderDetails AS od ON od.OrderID = o.OrderID
WHERE o.OrderDate >= DATE_TRUNC('year', CURRENT_DATE);
```

- Testing `cat.Active = 'No'`. It works only while the flag is spelled exactly that way.
- Filtering with `EXTRACT(YEAR FROM o.OrderDate) = EXTRACT(YEAR FROM (SELECT MAX(OrderDate) FROM Orders))`. The answer is right, but wrapping the column in a function stops PostgreSQL from using an index on `OrderDate`. The range form can use one.
- Putting the year condition in HAVING on `MAX(o.OrderDate)`. That keeps products whose last sale was in the year but sums units from every year.

Step 9 — Interview Follow-Up: The team wants to know when each inactive category was switched off, but there is no date column for that. What would you ask the data platform team to add, and how would the query change?

Step 10 — Optimization or Alternative: On a large `Orders` table, filter orders to the year first so the join only touches one year of lines:

```
WITH latest_year AS (
    SELECT DATE_TRUNC('year', MAX(OrderDate))::date AS year_start FROM Orders
),
year_orders AS (
    SELECT o.OrderID, o.OrderDate
    FROM Orders AS o
    CROSS JOIN latest_year AS ly
    WHERE o.OrderDate >= ly.year_start
      AND o.OrderDate <  ly.year_start + INTERVAL '1 year'
)
SELECT p.ProductID, p.Product, cat.CategoryName,
       SUM(od.Quantity) AS units_latest_year, MAX(yo.OrderDate) AS last_order_date
FROM year_orders AS yo
JOIN OrderDetails AS od  ON od.OrderID     = yo.OrderID
JOIN Products     AS p   ON p.ProductID    = od.ProductID
JOIN Category     AS cat ON cat.CategoryID = p.CategoryID
WHERE (UPPER(TRIM(CAST(cat.Active AS VARCHAR))) IN ('1', 'TRUE', 'T', 'Y', 'YES')) IS NOT TRUE
GROUP BY p.ProductID, p.Product, cat.CategoryName
ORDER BY units_latest_year DESC NULLS LAST, p.ProductID;
```

PostgreSQL usually applies the filter early on its own; this form makes the intent obvious and pairs with indexes on `Orders(OrderDate)` and `OrderDetails(OrderID)`.

Question 39: Categories with no products, and categories with products but no sales

Difficulty: Medium

Primary Topic: LEFT JOIN chain for unmatched records at two levels

Combined Topics: COUNT DISTINCT, CASE status, COALESCE for zero

Business Scenario: The catalogue team runs a clean-up every half year. Some categories were created and never stocked; others have products that never sold. An empty category is deleted, while an unsold range is reviewed with the buyer. They want every category with its product count, how many of its products have sold, and a status naming the action.

Problem Statement: For every row in `Category`, return the number of products in it, how many of those products appear on at least one order line, the total units sold, and a status: `No products`, `Products but no sales`, `Some products unsold` or `All products sold`. Put the categories needing action first: empty categories, then the lowest share of products sold.

Expected Output:

- `CategoryID`, `CategoryName`
- `product_count`: products in the category
- `products_sold`: products with at least one order line
- `units_sold`: sum of `Quantity` (0 when none)
- `status`: one of the four labels above

Tables Required: `Category`, `Products`, `OrderDetails`

SQL Concepts Tested: chaining LEFT JOINs so missing rows survive at each level, COUNT DISTINCT after a fan-out, CASE on aggregates, COALESCE to turn NULL sums into 0.

Step 1 — Understand the Problem: There are two levels of "missing". A category may have no products. Or it has products, but none or only some appear in `OrderDetails`. Q15 handled the first level with one LEFT JOIN. Here a second LEFT JOIN is chained to `OrderDetails`, and both must stay LEFT so the gaps at each level survive.

The second join multiplies rows: a product with 40 lines becomes 40 rows. Summing units is fine; a plain product count is not.

Step 2 — Identify the Tables and Relationships:

- `Products.CategoryID` → `Category.CategoryID`, many products to one category.
- `OrderDetails.ProductID` → `Products.ProductID`, many lines to one product.

Start from `Category`, because every category must appear. Products whose `CategoryID` matches no category belong in an orphan check, not here.

Step 3 — Build the Query Logic:

1. `Category` LEFT JOIN `Products` LEFT JOIN `OrderDetails`.
2. Group by category.
3. `COUNT(DISTINCT p.ProductID)` for products (the lines have multiplied each product).
4. `COUNT(DISTINCT od.ProductID)` for products that sold: `od.ProductID` is only non-NULL when a line matched.
5. CASE on the two counts for the status.

Step 4 — Final SQL Solution:

```sql
SELECT cat.CategoryID,
       cat.CategoryName,
       COUNT(DISTINCT p.ProductID)    AS product_count,
       COUNT(DISTINCT od.ProductID)   AS products_sold,
       COALESCE(SUM(od.Quantity), 0)  AS units_sold,
       CASE
           WHEN COUNT(DISTINCT p.ProductID) = 0                          THEN 'No products'
           WHEN COUNT(DISTINCT od.ProductID) = 0                         THEN 'Products but no sales'
           WHEN COUNT(DISTINCT od.ProductID) < COUNT(DISTINCT p.ProductID) THEN 'Some products unsold'
           ELSE 'All products sold'
       END                            AS status
FROM Category AS cat
LEFT JOIN Products     AS p  ON p.CategoryID = cat.CategoryID
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
GROUP BY cat.CategoryID, cat.CategoryName
ORDER BY COUNT(DISTINCT od.ProductID)::numeric
         / NULLIF(COUNT(DISTINCT p.ProductID), 0) NULLS FIRST,
         cat.CategoryID;
```

Step 5 — Line-by-Line Explanation:

- `FROM Category` with two LEFT JOINs keeps every category, and within each category every product, even when nothing matches further down.
- `COUNT(DISTINCT p.ProductID)` counts each product once, no matter how many lines it brought. For an empty category, `p.ProductID` is NULL on its single row and the count is 0.
- `COUNT(DISTINCT od.ProductID)` only sees products that matched at least one line. Unsold products carry NULL in `od.ProductID` and are skipped.
- `COALESCE(SUM(od.Quantity), 0)` shows 0 instead of NULL for categories with no lines.

- The CASE tests the strongest condition first; ELSE is reached only when every product sold.
- The ORDER BY sorts on the share of products sold. Empty categories divide by NULL, and `NULLS FIRST` puts them on top. Output aliases cannot be used inside expressions, hence the repeated aggregates.

Step 6 — Sample Output:

categoryid	categoryname	product_count	products_sold	units_sold	status
5013	Electronics Accessories	0	0	0	No products
5002	Kitchen, Garden & Pets	2877	1159	6718	Some products unsold
5001	Cleaning & Household	2264	1544	21213	Some products unsold
5004	Gourmet & World Food	3967	2717	25521	Some products unsold
5008	Beauty & Hygiene	6142	4797	78508	Some products unsold
5005	Baby Care	504	404	4461	Some products unsold
5003	Foodgrains, Oil & Masala	2040	1808	53337	Some products unsold
5010	Eggs, Meat & Fish	334	306	6975	Some products unsold

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: The LEFT JOIN chain preserves each level of absence: a category with no products still produces one row, and a product with no lines still produces one row. COUNT DISTINCT undoes the fan-out for the product counts, while the unit sum is taken at the line grain where `Quantity` lives. Because both counts come from the same grouped rows, `products_sold` can never exceed `product_count`.

Step 8 — Common Mistakes:

- `COUNT(*)` or `COUNT(p.ProductID)` for the product count. After the second join a sold product appears once per line, so it is counted once per line:

```sql
SELECT cat.CategoryID,
       COUNT(p.ProductID) AS product_count_wrong   -- counts lines once sales exist
FROM Category AS cat
LEFT JOIN Products     AS p  ON p.CategoryID = cat.CategoryID
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
GROUP BY cat.CategoryID
ORDER BY cat.CategoryID;
```

- Making the second join an INNER JOIN. `A LEFT JOIN B JOIN C` keeps only rows that match all the way to C, which turns the first LEFT JOIN into an inner one and removes both kinds of gap you were looking for.
- Ignoring zero-quantity lines with `WHERE od.Quantity > 0`. For unsold products `od.Quantity` is NULL, the test is not true, and the rows vanish along with empty categories. Put conditions on the right-hand table in its ON clause.

Step 9 — Interview Follow-Up: The team wants the names of the unsold products in each category in one extra column. How would you build it, and why must the condition go inside the aggregate rather than in WHERE?

Step 10 — Optimization or Alternative: Pre-aggregate the lines per product so the join does not fan out at all. Each product then joins to at most one row, and plain `COUNT` works:

```sql
WITH product_sales AS (
    SELECT od.ProductID, SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.ProductID
)
SELECT cat.CategoryID,
       cat.CategoryName,
       COUNT(p.ProductID)            AS product_count,
       COUNT(ps.ProductID)           AS products_sold,
       COALESCE(SUM(ps.units), 0)    AS units_sold
FROM Category AS cat
LEFT JOIN Products      AS p  ON p.CategoryID = cat.CategoryID
LEFT JOIN product_sales AS ps ON ps.ProductID = p.ProductID
GROUP BY cat.CategoryID, cat.CategoryName
ORDER BY cat.CategoryID;
```

On a large `OrderDetails` this is cheaper: the lines are aggregated once, the join sees one row per product, and the per-group sorts behind COUNT DISTINCT go away.

Question 40: Days from sign-up to first order

Difficulty: Medium

Primary Topic: MIN per group with a date difference

Combined Topics: CTE then JOIN, Negative-gap data flag

Business Scenario: The onboarding team is testing a welcome-coupon email and needs to know when to send it. If most customers buy within a week of signing up, the email should go out on day one; if they take a month, it can wait. For every customer who has ordered, they want the number of days between sign-up and the first order. Rows where the first order seems to come before sign-up must be flagged, not hidden, because they point to a data problem.

Problem Statement: For each customer with at least one order, return their sign-up date, the date of their first order, the number of days between them, and a `data_flag` of `Order before sign-up` when that number is negative (NULL otherwise). Sort by days to first order, flagged rows first.

Expected Output:

- `CustomerID`
- `DateEntered`: sign-up date
- `first_order_date`: earliest `OrderDate` for the customer
- `days_to_first_order`: `first_order_date - DateEntered` in whole days
- `data_flag`: `Order before sign-up` when negative, otherwise NULL

Tables Required: `Customers`, `Orders`

SQL Concepts Tested: MIN as "first per group", aggregating before joining, date subtraction, CASE as a data-quality flag.

Step 1 — Understand the Problem: "First order" is the earliest order per customer, which is `MIN(OrderDate)` grouped by `CustomerID`. You do not need the first order's ID or amount here, so MIN is enough; Q58 covers pulling a whole row with ROW_NUMBER.

The gap is `first_order_date - DateEntered`; DATE minus DATE gives whole days. A negative gap is impossible (you cannot buy before you register), so one of the dates is wrong. A common cause is a customer re-registering: the new account gets a later `DateEntered` while old orders are moved onto it.

Step 2 — Identify the Tables and Relationships: `Orders.CustomerID` → `Customers.CustomerID`, many orders to one customer. `Customers` provides `DateEntered`; `Orders` provides the order dates. Customers with no orders have no first order and are out of scope (Q18 lists them). Orders with a `CustomerID` that is not in `Customers` have no sign-up date, so the gap cannot be measured; an INNER JOIN drops them, which is correct here.

Step 3 — Build the Query Logic:

1. CTE `first_orders`: one row per customer with `MIN(o.OrderDate)`.
2. Join it to `Customers` on `CustomerID`. Now each customer has both dates on one row.
3. Subtract, casting both sides to `date` so the result is an integer even if a column is a timestamp.
4. CASE: when the gap is below zero, set the flag.

Step 4 — Final SQL Solution:

```sql
WITH first_orders AS (
    SELECT o.CustomerID,
           MIN(o.OrderDate) AS first_order_date
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT c.CustomerID,
       c.DateEntered,
       fo.first_order_date,
       fo.first_order_date::date - c.DateEntered::date            AS days_to_first_order,
       CASE WHEN fo.first_order_date::date < c.DateEntered::date
            THEN 'Order before sign-up' END                       AS data_flag
FROM Customers AS c
JOIN first_orders AS fo ON fo.CustomerID = c.CustomerID
ORDER BY days_to_first_order NULLS LAST, c.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `first_orders` reduces `Orders` to one row per customer. Joining after this step means each customer row meets exactly one row, so nothing is duplicated.
- `MIN(o.OrderDate)` ignores NULL dates, so an undated order cannot become the "first" one.
- `::date - ::date` returns an integer. If both were timestamps, the subtraction would return an `interval` such as `14 days 03:00:00`, which is awkward to sort, average or compare.
- The CASE has no ELSE, so it returns NULL for valid rows. A NULL flag reads as "nothing wrong".
- A NULL `DateEntered` gives a NULL gap and flag, sorted last. If your data has them, add a `Missing sign-up date` flag.
- `ORDER BY days_to_first_order` puts negative gaps at the top for the reviewer.

Step 6 — Sample Output:

customerid	dateentered	first_order_date	days_to_first_order	data_flag
104446	2024-12-08	2024-09-12	-87	Order before sign-up
100742	2026-02-28	2025-12-20	-70	Order before sign-up
101861	2024-09-09	2024-07-07	-64	Order before sign-up
104279	2025-06-23	2025-04-27	-57	Order before sign-up

104965	2024-07-25	2024-06-03	-52	Order before sign-up
101343	2025-06-26	2025-05-12	-45	Order before sign-up
104829	2025-12-03	2025-10-24	-40	Order before sign-up
104907	2025-06-05	2025-04-26	-40	Order before sign-up

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 4425 rows.

Step 7 — Why This Solution Works: The first order is found at the order grain, then joined one-to-one to the customer, so each customer appears once with one gap. The flag uses the same two dates as the gap, so the columns cannot disagree. Negative rows stay in on purpose: hiding them flatters the average and loses the evidence the data team needs.

Step 8 — Common Mistakes:

- Taking `MIN(OrderID)` as the first order. IDs are assigned at load time, and a backfilled or migrated order can have a higher ID than a later one. Use the date, with the ID only as a tiebreaker.
- Using `AGE()` and extracting days. `AGE` returns years, months and days separately, so `EXTRACT(DAY FROM AGE(...))` gives only the leftover days, not the total:

```
SELECT DATE '2024-03-20' - DATE '2024-01-05'                    AS total_days,
       EXTRACT(DAY FROM AGE(DATE '2024-03-20', DATE '2024-01-05')) AS day_part_only;
```

- Filtering negative gaps out with `WHERE days_to_first_order >= 0` and reporting the average. The flagged customers vanish, the data problem stays hidden, and the WHERE clause cannot see a SELECT alias anyway.

Step 9 — Interview Follow-Up: The onboarding team wants the median days to first order per sign-up month, excluding flagged rows. Which function gives the median in PostgreSQL, and where does the exclusion go?

Step 10 — Optimization or Alternative: `DISTINCT ON` returns the first order row per customer directly, which is useful if you later need the first order's amount as well:

```
SELECT DISTINCT ON (o.CustomerID)
       o.CustomerID,
       c.DateEntered,
       o.OrderDate                          AS first_order_date,
       o.Total_Order_Amount                 AS first_order_amount,
       o.OrderDate::date - c.DateEntered::date  AS days_to_first_order
FROM Orders AS o
JOIN Customers AS c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate IS NOT NULL
ORDER BY o.CustomerID, o.OrderDate, o.OrderID;
```

`DISTINCT ON` is PostgreSQL only. It keeps the first row of each `CustomerID` according to the ORDER BY, and `o.OrderID` breaks ties between two orders on the same day. On MySQL, use ROW_NUMBER in a subquery instead.

Question 41: Most expensive product in each category

Difficulty: Medium

Primary Topic: ROW_NUMBER vs RANK for top-1 per group

Combined Topics: Ties, DISTINCT ON, LEFT JOIN for unmapped categories

Business Scenario: The merchandising team is planning a premium banner on each category page. The banner shows the category's most expensive product by current `Sale_Price`. If two products tie for the top price, both go on the banner, because picking one at random would favour a brand without a reason.

Problem Statement: For each category, return the product or products with the highest `Sale_Price`. Keep every product that ties for first. Products whose `CategoryID` has no `Category` row are grouped under `Unmapped`. Products with no price cannot be the most expensive. Sort by category name, then `ProductID`.

Expected Output:

- `CategoryName`: the category's name, or `Unmapped`
- `ProductID`, `Product`
- `Sale_Price`

Tables Required: `Products`, `Category`

SQL Concepts Tested: window functions with PARTITION BY, the difference between ROW_NUMBER and RANK on ties, NULL ordering in DESC sorts, PostgreSQL's DISTINCT ON.

Step 1 — Understand the Problem: "Top 1 per group" is a ranking problem. Number the products inside each category from most to least expensive, then keep number 1. The interesting part is what "number 1" means when two products share the top price:

- `ROW_NUMBER()` gives every row a different number, so one of the tied products gets 2 and is lost. Which one is arbitrary unless you add a tiebreaker.
- `RANK()` gives tied rows the same number, so both get 1 and both are kept.

The requirement says ties are kept, so the answer is RANK.

Step 2 — Identify the Tables and Relationships: `Products.CategoryID` → `Category.CategoryID`, many products to one category. The ranking only needs `Products`; `Category` supplies the name. LEFT JOIN so products with a broken `CategoryID` are still ranked within their own (unmapped) group. Partition by `CategoryID`, not by name, so two broken IDs are not merged with each other just because both say `Unmapped`.

Step 3 — Build the Query Logic:

1. In a CTE, rank products with `RANK() OVER (PARTITION BY p.CategoryID ORDER BY p.Sale_Price DESC NULLS LAST)`.
2. Exclude products with no price from the candidates.
3. Outer query: keep rank 1, attach the category name, sort.

Step 4 — Final SQL Solution:

```sql
WITH ranked AS (
    SELECT p.ProductID,
           p.Product,
           p.CategoryID,
           p.Sale_Price,
           RANK() OVER (PARTITION BY p.CategoryID
                        ORDER BY p.Sale_Price DESC NULLS LAST) AS price_rank
    FROM Products AS p
    WHERE p.Sale_Price IS NOT NULL
)
SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
       r.ProductID,
       r.Product,
       r.Sale_Price
FROM ranked AS r
LEFT JOIN Category AS cat ON cat.CategoryID = r.CategoryID
WHERE r.price_rank = 1
ORDER BY CategoryName, r.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `PARTITION BY p.CategoryID` restarts the ranking for each category, so every category gets its own number 1.
- `ORDER BY p.Sale_Price DESC` ranks the highest price first. In PostgreSQL, DESC puts NULLs first, so a product with no price would rank 1. `WHERE p.Sale_Price IS NOT NULL` is what enforces the rule. `NULLS LAST` alone is not enough: in a category whose products are all unpriced it would still rank one of them 1, the same trap as Q2. It stays only as a second guard.
- `RANK()` gives the same number to equal prices. Two products at the top price both get 1.
- The filter `price_rank = 1` must be in the outer query. Window functions are computed after WHERE, so you cannot filter on them at the same level.
- `COALESCE(cat.CategoryName, 'Unmapped')` labels products whose category is missing.
- `ORDER BY CategoryName, r.ProductID` lists tied products in a fixed order.

Step 6 — Sample Output:

categoryname	productid	product	sale_price
Baby Care	833	Baby Bottle & Food Warmer Set	3299.00
Bakery, Cakes & Dairy	19799	Cooking Butter - Unsalted, Organic	1108.00
Beauty & Hygiene	21909	Epilator SE9-9961 Legs-Body-Face	8184.00
Beverages	7429	Nutritional Powder Vanilla Delight 1kg+ Nutritional Powder Premium Chocolate 1kg	2330.00
Cleaning & Household	5155	Round Mist Diffuser	3600.00
Eggs, Meat & Fish	9467	Lamb Hind Shank - New Zealand	2249.00
Foodgrains, Oil & Masala	21573	Olive Pomace Oil	4099.00
Fruits & Vegetables	5198	Apple - Washington 4 pcs + Kiwi - Green 3 pcs + Pomegranate 4 pcs	527.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: RANK assigns 1 to every row whose price equals the partition's maximum, and only to those rows. So the result is exactly "all products at the top price in each category". A category with no products, or only unpriced products, produces no row, which is correct: there is nothing to put on the banner. Mention that case to the interviewer instead of letting it surprise them.

Step 8 — Common Mistakes:

- Using ROW_NUMBER when ties must be kept. This small example has two products tied at the top price; ROW_NUMBER keeps one, RANK keeps both:

```sql
WITH sample(CategoryID, ProductID, Sale_Price) AS (
    VALUES (1, 101, 899.00), (1, 102, 899.00), (1, 103, 520.00)
)
SELECT ProductID,
       Sale_Price,
       ROW_NUMBER() OVER (PARTITION BY CategoryID ORDER BY Sale_Price DESC) AS row_num,
       RANK()       OVER (PARTITION BY CategoryID ORDER BY Sale_Price DESC) AS rank_num
FROM sample
ORDER BY Sale_Price DESC, ProductID;
```

Without a tiebreaker in its ORDER BY, which tied row ROW_NUMBER calls 1 can change between runs.

- Joining to `MAX(Sale_Price)` per category on `CategoryID` and price. It also keeps ties, but if `CategoryID` is NULL the join `=` never matches, so those products vanish.
- Filtering `WHERE RANK() OVER (...) = 1` in the same SELECT. PostgreSQL rejects window functions in WHERE.

Step 9 — Interview Follow-Up: The banner now has room for the top three prices per category, and two products tied at second place should both appear without pushing the third price out. Which ranking function would you switch to?

Step 10 — Optimization or Alternative: When exactly one product per category is wanted, PostgreSQL's `DISTINCT ON` is the shortest form. It keeps the first row of each group according to the ORDER BY:

```sql
SELECT DISTINCT ON (p.CategoryID)
       COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
       p.ProductID,
       p.Product,
       p.Sale_Price
FROM Products AS p
LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
WHERE p.Sale_Price IS NOT NULL
ORDER BY p.CategoryID, p.Sale_Price DESC, p.ProductID;
```

It cannot keep ties, so it does not meet this brief, but it is the right tool for "one row per group" and is often faster than a window because it needs a single sort. The ORDER BY must start with the DISTINCT ON column. MySQL and SQL Server have no DISTINCT ON; use ROW_NUMBER there.

Question 42: RANK vs DENSE_RANK vs ROW_NUMBER on order counts

Difficulty: Medium

Primary Topic: Ranking function comparison

Combined Topics: Ties, Aggregation before ranking

Business Scenario: The CRM team runs a loyalty tier scheme based on order count. They have had three different "top 10" lists from three analysts, with different people on each, and suspect the ranking function is the cause. Order counts are small whole numbers, so many customers share the same count and ties are common. They want all three numbering schemes side by side for every customer.

Problem Statement: For each customer who has placed an order, return the number of orders placed and three rank columns computed over that count, highest first: `ROW_NUMBER`, `RANK` and `DENSE_RANK`. Sort by the row number.

Expected Output:

- `CustomerID`
- `orders_placed`: number of orders
- `row_num`: `ROW_NUMBER()`, unique per customer, ties broken by `CustomerID`
- `rank_num`: `RANK()`, ties share a rank and the next rank skips
- `dense_rank_num`: `DENSE_RANK()`, ties share a rank and the next rank does not skip

Tables Required: `Orders`

SQL Concepts Tested: the three ranking functions and how each treats ties, ranking over an aggregate, deterministic tie-breaking.

Step 1 — Understand the Problem: All three functions number rows in the order given by `OVER (ORDER BY ...)`. They differ only when two rows have equal sort values:

- `ROW_NUMBER` never repeats. Tied rows get consecutive numbers, and which tied row gets the lower number depends on the tiebreaker. Without one, it is arbitrary.
- `RANK` gives tied rows the same number, then skips. Two customers tied for 5th are both 5, and the next customer is 7. The number answers "how many customers beat me, plus one".
- `DENSE_RANK` gives tied rows the same number and does not skip. After two customers at 5, the next is 6. The number answers "how many distinct order counts are above mine, plus one".

None is "correct" in general. "Top 10" means exactly 10 people with ROW_NUMBER, 10 or more with RANK (ties at 10th), and possibly many more with DENSE_RANK. That is why the analysts disagreed.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed. One row is one order; `CustomerID` identifies the buyer. Count per customer first, then rank the counts. Every order counts as placed, since cancellations are not modelled.

Step 3 — Build the Query Logic:

1. CTE: group by `CustomerID`, `COUNT(*)` as `orders_placed`.
2. Outer query: compute all three functions over `orders_placed DESC`.
3. Give `ROW_NUMBER` a tiebreaker (`CustomerID`) so it is repeatable. Leave `RANK` and `DENSE_RANK` without one, because ties are the point.

Step 4 — Final SQL Solution:

```sql
WITH customer_orders AS (
    SELECT o.CustomerID,
           COUNT(*) AS orders_placed
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID
)
SELECT co.CustomerID,
       co.orders_placed,
       ROW_NUMBER() OVER (ORDER BY co.orders_placed DESC, co.CustomerID) AS row_num,
       RANK()       OVER (ORDER BY co.orders_placed DESC)                AS rank_num,
       DENSE_RANK() OVER (ORDER BY co.orders_placed DESC)                AS dense_rank_num
FROM customer_orders AS co
ORDER BY row_num;
```

Step 5 — Line-by-Line Explanation:

- `customer_orders` gives one row per customer. Ranking needs one row per thing ranked, so aggregate first.
- `WHERE o.CustomerID IS NOT NULL` keeps orders with no buyer from forming a NULL "customer" that might rank near the top.
- There is no `PARTITION BY`, so each function ranks across all customers at once.
- `ROW_NUMBER() OVER (ORDER BY co.orders_placed DESC, co.CustomerID)`: among customers with equal counts, the lower ID gets the lower number. Any stable column works; the point is that one exists.
- `RANK()` and `DENSE_RANK()` sort only on the count. Adding `CustomerID` to their ORDER BY would make every row unique, and both would behave exactly like ROW_NUMBER.
- `ORDER BY row_num` fixes the output order, since `row_num` is unique.

Step 6 — Sample Output:

customerid	orders_placed	row_num	rank_num	dense_rank_num
104801	192	1	1	1
102192	188	2	2	2
102422	187	3	3	3
100092	182	4	4	4
101237	151	5	5	5
101259	142	6	6	6
101294	138	7	7	7
104856	135	8	8	8

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 4427 rows.

Step 7 — Why This Solution Works: All three functions read the same ordered list of customers, so the columns differ only where counts tie. Read down any block of tied customers and you see the whole lesson: `row_num` keeps counting, `rank_num` repeats and then jumps by the size of the tie, `dense_rank_num` repeats and then moves up by one. For any row, `rank_num` is at most `row_num`, and `dense_rank_num` is at most `rank_num`.

Step 8 — Common Mistakes:

- Using ROW_NUMBER without a tiebreaker for "top N". The result is valid SQL but can change between runs, so two analysts on the same data get different lists.
- Adding a tiebreaker to RANK or DENSE_RANK. It silently removes the ties you meant to keep:

```sql
WITH customer_orders AS (
    SELECT o.CustomerID, COUNT(*) AS orders_placed
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT co.CustomerID,
       co.orders_placed,
       RANK() OVER (ORDER BY co.orders_placed DESC, co.CustomerID) AS rank_no_ties   -- same as ROW_NUMBER
FROM customer_orders AS co
ORDER BY rank_no_ties
LIMIT 10;
```

- Using `DENSE_RANK() <= 10` for "top 10 customers". It returns every customer in the top 10 order counts, which can be far more than 10 people when counts are crowded. Use RANK when ties at the cut-off should be kept.

Step 9 — Interview Follow-Up: The CRM team now wants the ranking restarted within each customer state. Which table do you add, and what changes inside the OVER clause?

Step 10 — Optimization or Alternative: All three functions share the same `ORDER BY orders_placed DESC` prefix, so PostgreSQL sorts once and computes them in a single pass; asking for all three costs little more than asking for one. If you only need the top 10 customers including ties, PostgreSQL 13+ also supports `ORDER BY orders_placed DESC FETCH FIRST 10 ROWS WITH TIES`, which returns the same rows as `WHERE rank_num <= 10` without a window function.

Question 43: Month-to-date order value for the latest three months

Difficulty: Medium

Primary Topic: Running total with PARTITION BY month

Combined Topics: Daily aggregation first, ROWS vs RANGE frames, Anchored date window

Business Scenario: The finance team tracks each month against its target with a month-to-date (MTD) line: on any day, how much order value has come in since the 1st. They want the last three months of data, one row per day that had orders, with that day's order value and the running MTD figure, so they can compare the shape of each month.

Problem Statement: Take the latest three calendar months in the data, counting the month of the latest order as the third. For each order date in that window, return the day's total order value and the month-to-date total, which restarts on the first day of each month. Sort by date.

Expected Output:

- `order_date`: the calendar date
- `daily_order_value`: sum of `Total_Order_Amount` for orders placed that day
- `month_to_date_value`: running total of `daily_order_value` from the start of that month up to and including this date

Tables Required: `Orders`

SQL Concepts Tested: running SUM as a window, PARTITION BY to restart the total, aggregating to the day before the window, the default frame and why ROWS is stated, anchoring the window to the data.

Step 1 — Understand the Problem: A running total adds each row's value to everything before it. A window function does this without a self join: `SUM(x) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. To restart every month, partition by the month.

Several orders can share a date, but the report is one row per day. So aggregate to the day first, then run the window over days. That gives one row per date and an unambiguous frame.

"Latest three months" is anchored to `(SELECT MAX(OrderDate) FROM Orders)`, so a historical snapshot gives the same answer whenever it runs. If the data ends mid-month, the last month is partial, which MTD handles naturally.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `OrderDate` and `Total_Order_Amount`. No joins, so no fan-out risk. Every order counts as placed, since cancellations and refunds are not modelled.

Step 3 — Build the Query Logic:

1. CTE `bounds`: from the latest order date, compute the first day of the month two months back (`start_month`) and the first day of the month after the latest (`end_month`).
2. CTE `daily`: orders in `[start_month, end_month)`, grouped by date, summing the value.
3. Outer query: `SUM(daily_order_value) OVER (PARTITION BY month ORDER BY order_date ROWS ...)`.

Step 4 — Final SQL Solution:

```sql
WITH bounds AS (
    SELECT (DATE_TRUNC('month', MAX(o.OrderDate)) - INTERVAL '2 months')::date AS start_month,
           (DATE_TRUNC('month', MAX(o.OrderDate)) + INTERVAL '1 month')::date  AS end_month
    FROM Orders AS o
),
daily AS (
    SELECT o.OrderDate::date             AS order_date,
           SUM(o.Total_Order_Amount)     AS daily_order_value
    FROM Orders AS o
    CROSS JOIN bounds AS b
    WHERE o.OrderDate >= b.start_month
      AND o.OrderDate <  b.end_month
    GROUP BY o.OrderDate::date
)
SELECT d.order_date,
       d.daily_order_value,
       SUM(d.daily_order_value) OVER (
           PARTITION BY DATE_TRUNC('month', d.order_date)
           ORDER BY d.order_date
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS month_to_date_value
FROM daily AS d
ORDER BY d.order_date;
```

Step 5 — Line-by-Line Explanation:

- `bounds` computes the window once. If the latest order is 18 December, it runs from 1 October up to 1 January.
- The half-open range `>= start_month AND < end_month` includes all of the last month, even if `OrderDate` were a timestamp.
- `daily` collapses orders to one row per date. `SUM` skips NULL totals.

- `PARTITION BY DATE_TRUNC('month', d.order_date)` restarts the running sum on each month's first row.
- `ORDER BY d.order_date` inside OVER sets the direction of accumulation.
- `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` adds this row and every earlier row in the partition. Dates are unique after `daily`, so the default frame would match here, but ROWS states the intent and avoids the trap in Step 8.
- Days with no orders do not appear; Step 10 adds them.

Step 6 — Sample Output:

order_date	daily_order_value	month_to_date_value
2025-10-01	139488.26	139488.26
2025-10-02	111841.02	251329.28
2025-10-03	110584.90	361914.18
2025-10-04	137638.15	499552.33
2025-10-05	167412.12	666964.45
2025-10-06	77889.55	744854.00
2025-10-07	65326.55	810180.55
2025-10-08	103538.50	913719.05

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 89 rows.

Step 7 — Why This Solution Works: After `daily`, each date appears once, so the running sum adds one value per day. The partition stops 1 November from carrying October's total. The last row of each month equals that month's order value: compare it with a plain `SUM(Total_Order_Amount)` for the month.

Step 8 — Common Mistakes:

- Running the window over orders instead of days, with the default frame. When ORDER BY is given and no frame is written, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes all rows with the same date as the current row. Every order on a busy day then shows the full day's total, and the output has several rows per date:

```sql
SELECT o.OrderDate,
       o.OrderID,
       o.Total_Order_Amount,
       SUM(o.Total_Order_Amount) OVER (
           PARTITION BY DATE_TRUNC('month', o.OrderDate)
           ORDER BY o.OrderDate
       ) AS mtd_default_range
FROM Orders AS o
WHERE o.OrderDate >= (SELECT DATE_TRUNC('month', MAX(OrderDate)) FROM Orders)
ORDER BY o.OrderDate, o.OrderID
LIMIT 8;
```

- Switching that query to ROWS without aggregating first. Each row is unique now, but the MTD for tied orders depends on their arbitrary order within the day.
- Forgetting `PARTITION BY`. The total runs across all three months, so the November figures include October.
- Using `CURRENT_DATE - INTERVAL '3 months'`. On a snapshot that ends in the past, the window is empty.

Step 9 — Interview Follow-Up: Finance wants each month's MTD compared with the same day-of-month in the previous month. Which window function would you add, and what would you partition by?

Step 10 — Optimization or Alternative: To include days with no orders, so the chart line is continuous, build a calendar with `generate_series` and LEFT JOIN the daily totals onto it:

```sql
WITH bounds AS (
    SELECT (DATE_TRUNC('month', MAX(OrderDate)) - INTERVAL '2 months')::date AS start_month,
           MAX(OrderDate)::date                                               AS last_day
    FROM Orders
),
calendar AS (
    SELECT gs::date AS order_date
    FROM bounds AS b,
         generate_series(b.start_month, b.last_day, INTERVAL '1 day') AS gs
),
daily AS (
    SELECT o.OrderDate::date AS order_date, SUM(o.Total_Order_Amount) AS daily_order_value
    FROM Orders AS o
    GROUP BY o.OrderDate::date
)
SELECT cal.order_date,
       COALESCE(d.daily_order_value, 0) AS daily_order_value,
       SUM(COALESCE(d.daily_order_value, 0)) OVER (
           PARTITION BY DATE_TRUNC('month', cal.order_date)
           ORDER BY cal.order_date
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS month_to_date_value
FROM calendar AS cal
LEFT JOIN daily AS d ON d.order_date = cal.order_date
ORDER BY cal.order_date;
```

`generate_series` with dates is PostgreSQL syntax; MySQL needs a recursive CTE or a calendar table.

Question 44: Month-over-month growth in order value

Difficulty: Medium

Primary Topic: LAG window function

Combined Topics: Monthly aggregation in a CTE, NULLIF for safe division

Business Scenario: The leadership team opens every monthly business review with one table: order value per month and how much it moved against the month before. The CFO wants it as a query on `Orders` instead of a spreadsheet. "Growth" means the change against the immediately preceding calendar month, not against whichever earlier month happens to have data.

Problem Statement: For every month that has orders, return the total order value, the previous calendar month's order value, the absolute change and the percentage change. If the previous calendar month has no orders, or there is no earlier month, the comparison columns must be NULL. Sort by month.

Expected Output:

- `order_month`: first day of the month
- `order_value`: sum of `Total_Order_Amount` for orders placed that month
- `prev_month_value`: order value of the previous calendar month, NULL if that month has no row
- `mom_change`: `order_value - prev_month_value`
- `mom_growth_pct`: change as a percentage of `prev_month_value`, 2 decimals

Tables Required: `Orders`

SQL Concepts Tested: monthly aggregation, the LAG window function, window ordering, gap checking, NULLIF for division.

Step 1 — Understand the Problem: Two things happen here. Orders are rolled up to one row per month, and then each month row needs a value that lives on another row: the month before. GROUP BY cannot do the second part, because a grouped row only sees its own group. A window function can. `LAG(x)` returns `x` from the row one position earlier in a stated order, without collapsing any rows.

Step 2 — Identify the Tables and Relationships: Only `Orders`. `OrderDate` decides the month and `Total_Order_Amount` is the value. One row is one order, and there is no join, so there is no fan-out risk. `SUM` skips NULL totals.

Step 3 — Build the Query Logic:

1. In a CTE, group by `DATE_TRUNC('month', OrderDate)` and sum the totals. The grain is now one row per month.
2. Over that result, read the previous row's month and value with `LAG`, ordered by month.
3. Keep the previous value only when the previous row's month is exactly one month earlier. A month with no orders has no row, so without this check LAG would compare against two months back.
4. Compute the change and the percentage, with `NULLIF` guarding the division.

Step 4 — Final SQL Solution:

```sql
WITH monthly AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           SUM(o.Total_Order_Amount)              AS order_value
    FROM Orders AS o
    WHERE o.OrderDate IS NOT NULL
    GROUP BY DATE_TRUNC('month', o.OrderDate)::date
),
with_prev AS (
    SELECT m.order_month,
           m.order_value,
           CASE WHEN LAG(m.order_month) OVER w = m.order_month - INTERVAL '1 month'
                THEN LAG(m.order_value) OVER w
           END AS prev_month_value
    FROM monthly AS m
    WINDOW w AS (ORDER BY m.order_month)
)
SELECT wp.order_month,
       wp.order_value,
       wp.prev_month_value,
       wp.order_value - wp.prev_month_value AS mom_change,
       ROUND(100.0 * (wp.order_value - wp.prev_month_value)
             / NULLIF(wp.prev_month_value, 0), 2) AS mom_growth_pct
FROM with_prev AS wp
ORDER BY wp.order_month;
```

Step 5 — Line-by-Line Explanation:

- `DATE_TRUNC('month', ...)::date` maps every date in a month to the 1st. In MySQL you would group by `DATE_FORMAT(OrderDate, '%Y-%m-01')`.
- `WINDOW w AS (ORDER BY m.order_month)` names the window once so both LAG calls share it. It is the same as writing `OVER (ORDER BY m.order_month)` twice.
- `LAG(m.order_value) OVER w` returns the previous row's value. On the first row there is no previous row, so it returns NULL. `LAG(x, 1, 0)` would supply a default of 0, but that would claim the month before had zero sales. NULL is the honest answer.
- The `CASE` keeps the previous value only when the previous row's month equals this month minus one month.

- `mom_change` inherits the NULL, so the first month and any month after a gap show no change instead of a misleading one.
- `NULLIF(wp.prev_month_value, 0)` turns a zero base into NULL, so the division returns NULL instead of an error.

Step 6 — Sample Output:

order_month	order_value	prev_month_value	mom_change	mom_growth_pct
2023-01-01	447044.37	NULL	NULL	NULL
2023-02-01	217410.49	447044.37	-229633.88	-51.37
2023-03-01	354300.85	217410.49	136890.36	62.96
2023-04-01	438229.46	354300.85	83928.61	23.69
2023-05-01	488672.06	438229.46	50442.60	11.51
2023-06-01	447517.14	488672.06	-41154.92	-8.42
2023-07-01	616163.08	447517.14	168645.94	37.68
2023-08-01	686448.76	616163.08	70285.68	11.41

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 36 rows.

Step 7 — Why This Solution Works: The CTE fixes the grain at one row per month before the window runs, so LAG steps from month to month, not from order to order. Window functions run after WHERE, GROUP BY and HAVING and before the final ORDER BY. That is why LAG cannot sit in a WHERE clause, and why it can read grouped values. LAG only reads; every month stays in the output. Self-check: on each compared row, `prev_month_value` equals the `order_value` one row above.

Step 8 — Common Mistakes:

- Plain LAG with no month check. When a month has no orders, the next month is compared against the last month that had orders, and a two-month movement is labelled month-over-month. It looks right whenever every month is present, which is what makes it dangerous:

```
SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
       SUM(o.Total_Order_Amount)              AS order_value,
       LAG(SUM(o.Total_Order_Amount))
           OVER (ORDER BY DATE_TRUNC('month', o.OrderDate)::date) AS prev_row_value
FROM Orders AS o
GROUP BY DATE_TRUNC('month', o.OrderDate)::date
ORDER BY order_month;
```

- Running LAG on raw `Orders` rows, which compares each order with the previous order.
- Reading the last row as a full month. If the data stops mid-month, check `MAX(OrderDate)` before presenting it.

Step 9 — Interview Follow-Up: Leadership also wants growth against the same month last year. How would you change the LAG call, and what breaks with an offset of 12 if a month in between is missing?

Step 10 — Optimization or Alternative: A self-join on the monthly result matches on the calendar month rather than row position, so gaps are handled by construction. The same idea works on engines without window functions, such as MySQL 5.7, once the CTE is written as a derived table (5.7 has no `WITH` either) and `DATE_TRUNC` as `DATE_FORMAT`:

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           SUM(o.Total_Order_Amount)              AS order_value
    FROM Orders AS o
    WHERE o.OrderDate IS NOT NULL
    GROUP BY DATE_TRUNC('month', o.OrderDate)::date
)
SELECT cur.order_month,
       cur.order_value,
       prev.order_value                    AS prev_month_value,
       cur.order_value - prev.order_value  AS mom_change,
       ROUND(100.0 * (cur.order_value - prev.order_value)
             / NULLIF(prev.order_value, 0), 2) AS mom_growth_pct
FROM monthly AS cur
LEFT JOIN monthly AS prev
       ON prev.order_month = cur.order_month - INTERVAL '1 month'
ORDER BY cur.order_month;
```

If empty months should appear as zero rows, LEFT JOIN the totals to a `generate_series` month spine. After that, plain LAG is safe.

Question 45: Typical gap between a customer's orders

Difficulty: Medium

Primary Topic: LAG partitioned by customer

Combined Topics: Date difference, Aggregating window output in a CTE, HAVING

Business Scenario: The CRM team wants to time replenishment reminders. If a customer usually reorders every 20 days, a nudge on day 18 makes sense; on day 5 it is noise. They need each regular customer's typical gap between consecutive orders and their longest gap, which shows how long they have gone quiet before. "Regular" means at least three orders, so there are at least two gaps to average.

Problem Statement: For every customer with three or more orders, return the number of orders, the average number of days between consecutive orders, and the longest such gap. Sort by average gap, shortest first.

Expected Output:

- `CustomerID`
- `orders` : number of orders the customer placed
- `avg_gap_days` : average days between one order and the customer's next order, 2 decimals
- `max_gap_days` : longest gap in days

Tables Required: `Orders`

SQL Concepts Tested: `LAG` with `PARTITION BY` , ordering inside a window, date subtraction, aggregating window results, HAVING.

Step 1 — Understand the Problem: A gap is a fact about a pair of orders: this order and the same customer's previous order. A customer with five orders has four gaps. So the work has two stages. First, put each order next to the previous order of the same customer and subtract the dates. Then collapse those gaps to one row per customer.

Step 2 — Identify the Tables and Relationships: Only `Orders` . `CustomerID` defines whose history an order belongs to and `OrderDate` gives the sequence. Customers with no orders are irrelevant here because they cannot reach three orders.

Step 3 — Build the Query Logic:

1. For each order, fetch the previous order date of the same customer with `LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate, OrderID)` .
2. Subtract to get `gap_days` . Each customer's first order has no previous order, so its gap is NULL.
3. Group by customer. `COUNT(*)` counts orders, `AVG` and `MAX` of the gap ignore the NULL first-order gap.
4. Keep customers with `COUNT(*) >= 3` .

Step 4 — Final SQL Solution:

```sql
WITH order_gaps AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate::date
             - LAG(o.OrderDate::date) OVER (PARTITION BY o.CustomerID
                                            ORDER BY o.OrderDate, o.OrderID) AS gap_days
    FROM Orders AS o
    WHERE o.OrderDate IS NOT NULL
)
SELECT og.CustomerID,
       COUNT(*)                   AS orders,
       ROUND(AVG(og.gap_days), 2) AS avg_gap_days,
       MAX(og.gap_days)           AS max_gap_days
FROM order_gaps AS og
GROUP BY og.CustomerID
HAVING COUNT(*) >= 3
ORDER BY avg_gap_days, og.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `PARTITION BY o.CustomerID` splits the rows into one window per customer. LAG never reaches across the boundary, so the first order of customer B is never compared with the last order of customer A.
- `ORDER BY o.OrderDate, o.OrderID` sets the sequence inside each customer. `OrderID` breaks ties when a customer places two orders on the same day, so the result is the same every run. Those same-day pairs produce a gap of 0, which is real behaviour and stays in.
- `o.OrderDate::date - LAG(...)` gives whole days as an integer, whether the column is DATE or TIMESTAMP.
- The CTE is needed because a window function cannot sit inside an aggregate. `AVG(OrderDate - LAG(OrderDate) OVER (...))` fails with "aggregate function calls cannot contain window function calls". Compute the gap first, aggregate second.
- `COUNT(*)` counts every order, including the first one whose gap is NULL. `AVG` and `MAX` skip that NULL, so they work over the real gaps only.
- `HAVING COUNT(*) >= 3` filters on the aggregate. It cannot go in WHERE, which runs before grouping.

Step 6 — Sample Output:

customerid	orders	avg_gap_days	max_gap_days
104606	3	1.00	1
100341	3	1.50	2
102911	3	1.50	2
102641	3	2.00	3
103010	3	2.00	3
103853	3	2.00	2
100873	5	2.25	3
100081	4	2.33	4

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 3225 rows.

Step 7 — Why This Solution Works: Partitioning by customer turns one long table into many short, ordered histories, and LAG walks each history one step at a time. Every gap is counted exactly once, attached to the later order of the pair. The aggregation then sees one gap per order after the first. There is a neat check: the average gap always equals `(last order date - first order date) / (orders - 1)`, because the gaps add up to the full span. If your query disagrees with that formula for any customer, the partition or the ordering is wrong.

Step 8 — Common Mistakes:

- Forgetting `PARTITION BY`. LAG then runs over all orders in date order, so each gap is measured against whichever customer ordered just before. Every gap comes out tiny, and it looks plausible:

```
WITH order_gaps AS (
    SELECT o.CustomerID,
           o.OrderDate::date
             - LAG(o.OrderDate::date) OVER (ORDER BY o.OrderDate, o.OrderID) AS gap_days
    FROM Orders AS o
)
SELECT og.CustomerID, ROUND(AVG(og.gap_days), 2) AS avg_gap_wrong
FROM order_gaps AS og
GROUP BY og.CustomerID
HAVING COUNT(*) >= 3
ORDER BY og.CustomerID;
```

- Dividing by `COUNT(*)` instead of the number of gaps. A customer with three orders has two gaps.
- Ordering the window by `OrderID` alone. IDs usually follow time, but back-dated or re-loaded orders break that.

Step 9 — Interview Follow-Up: The CRM team says one long holiday gap ruins the average for some customers. What would you report instead of the mean, and how would you compute it in PostgreSQL?

Step 10 — Optimization or Alternative: Because the gaps telescope, the average needs no window function at all:

```
SELECT o.CustomerID,
       COUNT(*) AS orders,
       ROUND((MAX(o.OrderDate::date) - MIN(o.OrderDate::date))::numeric
             / (COUNT(*) - 1), 2) AS avg_gap_days
FROM Orders AS o
WHERE o.OrderDate IS NOT NULL
GROUP BY o.CustomerID
HAVING COUNT(*) >= 3
ORDER BY avg_gap_days, o.CustomerID;
```

This is a single pass with no sort per customer, so it is cheaper on a large table. It cannot give the longest gap, the median gap or any per-gap filter, which is why the LAG version is the one to know. An index on `Orders(CustomerID, OrderDate)` lets PostgreSQL read each customer's orders already sorted for the window.

Question 46: Same-brand products where one costs at least twice the other

Difficulty: Medium

Primary Topic: Self join

Combined Topics: Non-equi join condition to orient pairs, Ratio with NULLIF

Business Scenario: The pricing team is reviewing price ladders. When one brand sells two products in the same sub-category and one costs at least twice the other, shoppers either see a clear good-better-best ladder or a gap with nothing in the middle. The team wants every such pair listed once, cheaper product on the left, so they can decide where a mid-priced pack is missing.

Problem Statement: Find every pair of products with the same `Brand` and the same `Sub_Category` where the pricier product's `Sale_Price` is at least 2 times the cheaper one's. Return each pair once. Products with no brand are not part of any brand ladder. Sort by price ratio, highest first.

Expected Output:

- `Brand`
- `Sub_Category`
- `cheaper_product`, `cheaper_price`
- `pricier_product`, `pricier_price`
- `price_ratio`: pricier price divided by cheaper price, 2 decimals

Tables Required: `Products`

SQL Concepts Tested: self join with table aliases, equality plus inequality join conditions, mirrored-pair elimination, NULL behaviour in join keys.

Step 1 — Understand the Problem: The question compares a product with other products in the same table. Each output row needs two products side by side, so the table has to appear twice in the FROM clause under two names. That is a self join. Nothing special happens in the database: it is an ordinary join where both inputs happen to be `Products`.

Step 2 — Identify the Tables and Relationships: Only `Products`, read twice. Alias `cheap` plays the cheaper product and `pricey` plays the pricier one. The join key is the pair (`Brand`, `Sub_Category`), which is many-to-many inside the table: a brand with three products in a sub-category can form three pairs.

Step 3 — Build the Query Logic:

1. Join `Products` to itself on equal `Brand` and equal `Sub_Category`.
2. Add `pricey.Sale_Price >= 2 * cheap.Sale_Price`. This does two jobs: it applies the business rule, and it fixes which side is cheaper, so the mirrored copy of the pair cannot pass.
3. Require `cheap.Sale_Price > 0`. With a zero price, `0 >= 2 * 0` is true in both directions and the mirror would come back.
4. Compute the ratio and sort.

Step 4 — Final SQL Solution:

```sql
SELECT cheap.Brand,
       cheap.Sub_Category,
       cheap.Product     AS cheaper_product,
       cheap.Sale_Price  AS cheaper_price,
       pricey.Product    AS pricier_product,
       pricey.Sale_Price AS pricier_price,
       ROUND(pricey.Sale_Price / NULLIF(cheap.Sale_Price, 0), 2) AS price_ratio
FROM Products AS cheap
JOIN Products AS pricey
  ON  pricey.Brand        = cheap.Brand
  AND pricey.Sub_Category = cheap.Sub_Category
  AND pricey.Sale_Price  >= 2 * cheap.Sale_Price
WHERE cheap.Sale_Price > 0
  AND NULLIF(TRIM(cheap.Brand), '') IS NOT NULL
ORDER BY price_ratio DESC, cheap.Brand, cheap.ProductID, pricey.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `FROM Products AS cheap JOIN Products AS pricey` reads the same table twice. Without the aliases every column reference would be ambiguous.
- `pricey.Brand = cheap.Brand` and `pricey.Sub_Category = cheap.Sub_Category` pair products inside the same brand and sub-category. A NULL `Brand` never equals anything, including another NULL, so NULL-brand products drop out. That matches the problem statement: an unknown brand is not the same brand. A blank brand is different: `' ' = ' '` is true, so two blank-brand products in one sub-category would pair up. `NULLIF(TRIM(cheap.Brand), '') IS NOT NULL` removes them. On this extract only one product has a blank brand, which is the only reason the result is the same without it.
- `pricey.Sale_Price >= 2 * cheap.Sale_Price` is a non-equi condition inside the ON clause. Joins are not limited to `=`. Because this test is one-directional, a product can never be paired with itself, and pair (A, B) can pass while (B, A) cannot.
- `WHERE cheap.Sale_Price > 0` closes the only hole in that logic, a zero price, and also removes NULL prices.
- `NULLIF(cheap.Sale_Price, 0)` is a second guard; the WHERE already rules out zero.
- `ORDER BY ... cheap.ProductID, pricey.ProductID` makes the order fixed when two pairs share a ratio.

Step 6 — Sample Output:

brand	sub_category	cheaper_product	cheaper_price	pricier_product	pricier_price	price_ratio
Drools	Pet Food & Accessories	Dog Food - Chicken & Egg, Adult	17.00	Dog Food - Focus Starter, Super Premium	5092.00	299.53
Drools	Pet Food & Accessories	Dog Food - Chicken & Egg, Adult	17.00	Dog Food - Super Premium, Focus, Puppy, Limited Offer Stock	4675.00	275.00
Drools	Pet Food & Accessories	Dog Food - Chicken & Egg, Adult	17.00	Focus - Super Dog Food, For Adults	3613.00	212.53
Cadbury	Chocolates & Candies	Cadbury Perk - Chocolate Bar	5.00	Silk Dairy Milk - Silk Miniatures Collection	1060.00	212.00
Drools	Pet Food & Accessories	Cat Food - Ocean Fish, Adult	26.00	Dog Food - Focus Starter, Super Premium	5092.00	195.85
Drools	Pet Food & Accessories	Cat Food - Ocean Fish, Kitten	26.00	Dog Food - Focus Starter, Super Premium	5092.00	195.85
Drools	Pet Food & Accessories	Cat Food - Ocean Fish, Adult	26.00	Dog Food - Super Premium, Focus, Puppy, Limited Offer Stock	4675.00	179.81
Drools	Pet Food & Accessories	Cat Food - Ocean Fish, Kitten	26.00	Dog Food - Super Premium, Focus, Puppy, Limited Offer Stock	4675.00	179.81

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 90560 rows.

Step 7 — Why This Solution Works: A self join forms every combination of rows that meet the ON clause. Equality on brand and sub-category limits those combinations to products in the same ladder. The price inequality then filters and orients at once: every surviving row has the cheaper product on the left by construction, so each pair appears exactly once with no extra deduplication step. If a brand has three products at 100, 210 and 450, the query returns (100, 210), (100, 450) and (210, 450), and nothing else.

Step 8 — Common Mistakes:

- Writing a symmetric condition and only excluding self-pairs with `<>`. Each pair then appears twice, once in each order, and the report has double the rows:

```sql
SELECT a.Brand, a.Product AS product_a, b.Product AS product_b
FROM Products AS a
JOIN Products AS b
  ON  b.Brand        = a.Brand
  AND b.Sub_Category = a.Sub_Category
  AND a.ProductID   <> b.ProductID
WHERE GREATEST(a.Sale_Price, b.Sale_Price) >= 2 * LEAST(a.Sale_Price, b.Sale_Price)
ORDER BY a.Brand, a.ProductID, b.ProductID;
```

When the condition is symmetric like this, use `a.ProductID < b.ProductID` instead of `<>`. That keeps one ordering of each pair.

- Joining on `Brand` only, which compares a face wash with a hair oil.
- Using `COALESCE(Brand, '')` in the join to "include" unbranded products. That invents a fake brand and pairs unrelated items.

Step 9 — Interview Follow-Up: The team wants only adjacent steps: for each product, the next more expensive product in the same brand and sub-category. How would you get that without a self join?

Step 10 — Optimization or Alternative: A self join compares every product with every other product in its group, so cost grows with the square of the group size. An index on `Products(Brand, Sub_Category, Sale_Price)` lets the planner find partners with a range scan. For the adjacent-step version in Step 9, `LEAD(Sale_Price) OVER (PARTITION BY Brand, Sub_Category ORDER BY Sale_Price, ProductID)` reads each product once and avoids the pair explosion.

Question 47: Brands with at least three products, and how much they sold

Difficulty: Medium

Primary Topic: COUNT DISTINCT with HAVING across a join

Combined Topics: LEFT JOIN to keep unsold products, List-price value

Business Scenario: The brand management team is choosing which brands get a dedicated brand page on the app. A brand qualifies only if it lists at least three products, because a page with one or two items looks empty. For each qualifying brand they also want to see how much of the range actually sells, so they can push brands to fix dead listings before the page goes live.

Problem Statement: For every brand with three or more products in `Products`, return how many products it lists, how many of those have at least one order line, total units sold and list-price value. Products with no brand are grouped under `Unbranded`. Sort by list-price value, highest first.

Expected Output:

- `Brand`: brand name, or `Unbranded`
- `products_listed`: distinct products of the brand in the catalogue
- `products_sold`: distinct products with at least one order line
- `units_sold`: total `Quantity`, 0 if nothing sold
- `list_price_value`: `SUM(Quantity * Sale_Price)`, 0 if nothing sold

Tables Required: `Products`, `OrderDetails`

SQL Concepts Tested: COUNT vs COUNT DISTINCT after a one-to-many join, HAVING on a distinct count, LEFT JOIN to keep zero-sale rows, COALESCE.

Step 1 — Understand the Problem: The filter is about the catalogue ("lists at least three products") but the measures are about sales. The trap is that once products are joined to order lines, a product appears once per line. A brand with two products that sold 40 lines between them now has 40 rows, and `COUNT(*) >= 3` waves it through. The filter has to count distinct products, not rows.

Step 2 — Identify the Tables and Relationships: `Products` gives the brand and the product list. `OrderDetails` gives the lines (`OrderDetails.ProductID` → `Products.ProductID`, one product to many lines). The value is list-price value, `od.Quantity * p.Sale_Price`. `Sale_Price` is today's catalogue price, not the price paid on the order, so treat it as a proxy for revenue.

Step 3 — Build the Query Logic:

1. Start from `Products` and LEFT JOIN `OrderDetails`, so a product with no lines still produces one row with NULL line columns.
2. Group by brand, with a NULL or blank brand labelled `Unbranded`.
3. Count listed products with `COUNT(DISTINCT p.ProductID)` and sold products with `COUNT(DISTINCT od.ProductID)`.
4. Sum units and value, turning a NULL sum into 0.
5. Filter groups with `HAVING COUNT(DISTINCT p.ProductID) >= 3`.

Step 4 — Final SQL Solution:

```sql
SELECT COALESCE(NULLIF(TRIM(p.Brand), ''), 'Unbranded')              AS Brand,
       COUNT(DISTINCT p.ProductID)                 AS products_listed,
       COUNT(DISTINCT od.ProductID)                AS products_sold,
       COALESCE(SUM(od.Quantity), 0)               AS units_sold,
       COALESCE(SUM(od.Quantity * p.Sale_Price), 0) AS list_price_value
FROM Products AS p
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
GROUP BY COALESCE(NULLIF(TRIM(p.Brand), ''), 'Unbranded')
HAVING COUNT(DISTINCT p.ProductID) >= 3
ORDER BY list_price_value DESC, Brand;
```

Step 5 — Line-by-Line Explanation:

- `FROM Products AS p LEFT JOIN OrderDetails AS od` keeps every product. An unsold product appears once, with `od.ProductID` and `od.Quantity` NULL.
- `GROUP BY COALESCE(NULLIF(TRIM(p.Brand), ''), 'Unbranded')` groups on the labelled value. Grouping on the expression, not the alias, avoids any doubt about whether `Brand` means the raw column or the output name.
- `COUNT(DISTINCT p.ProductID)` counts each product once however many lines it has. This is the number the business rule is about.
- `COUNT(DISTINCT od.ProductID)` counts only products that matched a line. For an unsold product the value is NULL, and COUNT skips NULLs, so it is not counted. One expression gives "products sold" with no CASE needed.
- `COALESCE(SUM(od.Quantity), 0)` returns 0 for a brand whose products never sold. SUM over only NULLs is NULL, not 0.
- `HAVING COUNT(DISTINCT p.ProductID) >= 3` filters after grouping. WHERE cannot see aggregates.

Step 6 — Sample Output:

brand	products_listed	products_sold	units_sold	list_price_value
Soulflower	98	82	1959	1623342.00
StBotanica	94	75	2300	1525179.00
Fresho	247	241	16501	1276508.00
bb Royal	294	252	7851	1272495.00
Aroma Magic	99	86	1068	595097.00
Amul	147	145	4140	573749.00
ORGANIC INDIA	56	39	923	564267.00
VAHDAM	59	59	1322	546231.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 1380 rows.

Step 7 — Why This Solution Works: The join changes the grain from product to line, and every measure is written with that in mind. Line-level measures (units, value) are summed, which is correct at line grain. Product-level measures (listed, sold) use DISTINCT on the product key, which undoes the repetition the join created. The LEFT JOIN means a brand's dead listings still count towards its three products, which is the rule the team set. `products_listed - products_sold` is exactly the number of listings to fix.

Step 8 — Common Mistakes:

- Filtering with `COUNT(*)` after the join. This counts lines, so brands with only one or two products pass easily:

```sql
SELECT p.Brand,
       COUNT(*)                    AS rows_after_join,
       COUNT(DISTINCT p.ProductID) AS actual_products
FROM Products AS p
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
GROUP BY p.Brand
HAVING COUNT(*) >= 3
ORDER BY actual_products, p.Brand;
```

Any row where `actual_products` is below 3 is a brand that should not be there.

- Using INNER JOIN. Unsold products disappear before counting, so a brand with three listings and one dead product shows two and fails the rule it actually meets.
- Writing `COUNT(od.ProductID)` without DISTINCT for `products_sold`. That counts lines with a product, not products.

Step 9 — Interview Follow-Up: The team now wants `products_listed` to count only products with a non-NULL `Sale_Price`. Where does that condition go so that unpriced products stop counting but the brand's sales numbers do not change?

Step 10 — Optimization or Alternative: Pre-aggregating lines to one row per product removes the need for DISTINCT, because the join becomes one-to-one:

```sql
WITH product_sales AS (
    SELECT od.ProductID, SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.ProductID
)
SELECT COALESCE(NULLIF(TRIM(p.Brand), ''), 'Unbranded')               AS Brand,
       COUNT(*)                                    AS products_listed,
       COUNT(ps.ProductID)                         AS products_sold,
       COALESCE(SUM(ps.units), 0)                  AS units_sold,
       COALESCE(SUM(ps.units * p.Sale_Price), 0)   AS list_price_value
FROM Products AS p
LEFT JOIN product_sales AS ps ON ps.ProductID = p.ProductID
GROUP BY COALESCE(NULLIF(TRIM(p.Brand), ''), 'Unbranded')
HAVING COUNT(*) >= 3
ORDER BY list_price_value DESC, Brand;
```

`COUNT(*)` is safe here because each product is one row. It is usually faster too, since `COUNT(DISTINCT ...)` sorts within each group.

Question 48: Do discounted products sell more?

Difficulty: Medium

Primary Topic: CASE banding with pre-aggregation

Combined Topics: LEFT JOIN to keep zero sellers, CTE, AVG over a product grain

Business Scenario: The pricing team wants a bigger promotions budget. Their argument is that discounted products sell more, so deeper discounts will lift volume. Before approving it, the head of category wants a simple check: group products into discount bands and compare how many units an average product in each band has sold. "Sells more" means higher average units per product, with unsold products counted as zero.

Problem Statement: Band every product by its discount percentage, `(Market_Price - Sale_Price) / Market_Price * 100`: `Full price` (0%), `Up to 10%`, `10% to 25%`, `Over 25%`, and `Unknown` when `Market_Price` is missing or zero. For each band, return the number of products, how many of them ever sold, the average units sold per product (zero sellers included) and total units. List bands from full price to deepest discount, `Unknown` last.

Expected Output:

- `discount_band`
- `products`: products in the band, sold or not
- `products_sold`: products in the band with at least one order line
- `avg_units_per_product`: total units in the band divided by products in the band, 2 decimals
- `total_units`: sum of `Quantity` for those products

Tables Required: `Products`, `OrderDetails`

SQL Concepts Tested: CASE bands with an explicit "unknown" branch, aggregating lines to the product grain first, LEFT JOIN plus COALESCE so zero sellers count, averaging at the right grain.

Step 1 — Understand the Problem: "Average units per product" has a fixed denominator: every product in the band. If a product never sold, it still belongs to the band and should pull the average down. The question also has two grains. Units live on order lines, while the band lives on the product. To average per product, each product must be one row with its own unit total before the band average is taken.

Step 2 — Identify the Tables and Relationships: `Products` supplies both prices and the full product list. `OrderDetails` supplies `Quantity` (`OrderDetails.ProductID` → `Products.ProductID`, one product to many lines). A large share of the catalogue has never been ordered, so how unsold products are treated decides the answer.

Step 3 — Build the Query Logic:

1. In a CTE, sum `Quantity` per `ProductID` from `OrderDetails`. One row per product that sold.
2. LEFT JOIN that CTE to `Products`, so unsold products stay with `units = 0` via COALESCE.
3. Compute the discount percentage once, with `NULLIF` guarding the division.
4. Assign each product a band with CASE, plus a sort number for presentation.
5. Group by band and take `COUNT(*)`, `COUNT(...)` of sellers, `AVG(units)` and `SUM(units)`.

Step 4 — Final SQL Solution:

```sql
WITH product_units AS (
    SELECT od.ProductID,
           SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.ProductID
),
priced AS (
    SELECT p.ProductID,
           100.0 * (p.Market_Price - p.Sale_Price) / NULLIF(p.Market_Price, 0) AS discount_pct,
           pu.units
    FROM Products AS p
    LEFT JOIN product_units AS pu ON pu.ProductID = p.ProductID
),
banded AS (
    SELECT pr.ProductID,
           CASE WHEN pr.discount_pct IS NULL THEN 'Unknown'
                WHEN pr.discount_pct <= 0    THEN 'Full price'
                WHEN pr.discount_pct <= 10   THEN 'Up to 10%'
                WHEN pr.discount_pct <= 25   THEN '10% to 25%'
                ELSE 'Over 25%'
           END AS discount_band,
           CASE WHEN pr.discount_pct IS NULL THEN 5
                WHEN pr.discount_pct <= 0    THEN 1
                WHEN pr.discount_pct <= 10   THEN 2
                WHEN pr.discount_pct <= 25   THEN 3
                ELSE 4
           END AS band_order,
           pr.units IS NOT NULL      AS has_sold,
           COALESCE(pr.units, 0)     AS units
    FROM priced AS pr
)
SELECT b.discount_band,
       COUNT(*)                          AS products,
       COUNT(*) FILTER (WHERE b.has_sold) AS products_sold,
       ROUND(AVG(b.units), 2)            AS avg_units_per_product,
       SUM(b.units)                      AS total_units
FROM banded AS b
GROUP BY b.discount_band, b.band_order
ORDER BY b.band_order;
```

Step 5 — Line-by-Line Explanation:

- `product_units` collapses lines to one row per product, so the join to `Products` is one-to-one.
- `NULLIF(p.Market_Price, 0)` turns a zero printed price into NULL, so its discount becomes NULL instead of a division error.
- `CASE WHEN pr.discount_pct IS NULL` comes first on purpose. Any comparison with NULL is unknown, so a NULL discount would otherwise fall through to ELSE as `Over 25%`.
- The bands are checked from the top down. Once `<= 10` matches, CASE stops, so `<= 25` in the next branch effectively means "above 10 and up to 25". `<= 0` also catches a product priced above its printed price, which belongs with full price rather than with the discounts.
- `band_order` gives each band a sort number. Sorting the text labels would put `10% to 25%` first, which is not the order a reader expects.
- `COALESCE(pr.units, 0)` turns "no lines" into zero units. This is what lets unsold products enter the average.
- `AVG(b.units)` averages one number per product: band units divided by band products.

Step 6 — Sample Output:

discount_band	products	products_sold	avg_units_per_product	total_units
Full price	10373	8085	15.20	157684
Up to 10%	3160	2402	13.72	43369
10% to 25%	4553	3412	15.34	69839
Over 25%	3992	2615	10.89	43460
Unknown	10	10	12.90	129

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 5 rows returned.

Step 7 — Why This Solution Works: Each aggregate runs at the grain it describes. Units are summed per product first, then the band average is taken over products. The LEFT JOIN with COALESCE keeps the denominator honest: a band is judged on all its products, including the ones nobody bought. `products_sold` next to `products` shows how much of each band is dead listings. The query shows association only: a product may be discounted because it was not selling, so this check cannot prove that discounts cause sales.

Step 8 — Common Mistakes:

- Joining with INNER JOIN, which silently drops unsold products. The average becomes "units per product that sold" and is biased upward:

```sql
SELECT CASE WHEN p.Market_Price = p.Sale_Price THEN 'Full price' ELSE 'Discounted' END AS price_type,
       COUNT(DISTINCT p.ProductID)                                    AS sellers_only,
       ROUND(SUM(od.Quantity)::numeric / COUNT(DISTINCT p.ProductID), 2)   AS avg_units_biased
FROM Products AS p
JOIN OrderDetails AS od ON od.ProductID = p.ProductID
GROUP BY 1
ORDER BY 1;
```

- Writing `AVG(od.Quantity)` on the joined lines. That is the average quantity per order line, which answers a different question.
- Leaving out the NULL branch, so products with no usable printed price are reported as deeply discounted.

Step 9 — Interview Follow-Up: Discounts may differ a lot between categories. How would you repeat the comparison within each category, and why could the overall pattern differ from the pattern inside every category?

Step 10 — Optimization or Alternative: A correlated subquery gives the same per-product units without a CTE: `COALESCE((SELECT SUM(od.Quantity) FROM OrderDetails AS od WHERE od.ProductID = p.ProductID), 0)`. It reads well but probes the line table once per product, so it relies on an index on `OrderDetails(ProductID)`. For equal-width bands, `WIDTH_BUCKET(discount_pct, 0, 50, 5)` assigns bucket numbers; the NULL and full-price cases still need their own branches.

Question 49: Top three products by units in each category

Difficulty: Medium

Primary Topic: DENSE_RANK top-N per group

Combined Topics: Aggregation in a CTE, Filtering on a window result, LEFT JOIN for unmapped category

Business Scenario: Category managers are choosing which products get the three "Bestseller" badges on each category page. They want the top three products per category by units sold. If two products sell the same number of units, both should get the badge rather than one being dropped on an arbitrary tie-break, even if that means a category shows four badges.

Problem Statement: For each category, rank products by total units sold across all order lines and return every product whose rank is 1, 2 or 3, keeping ties. Products whose `CategoryID` has no row in `Category` are grouped under `Unmapped`. Only products with at least one order line take part. Sort by category name, then rank, then product ID.

Expected Output:

- `CategoryName`: category name, or `Unmapped`
- `product_rank`: 1, 2 or 3; tied products share a rank
- `ProductID`, `Product`
- `units_sold`: total `Quantity` for the product

Tables Required: `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: aggregation before ranking, DENSE_RANK with PARTITION BY, why a window result needs a CTE before it can be filtered, LEFT JOIN for unmatched categories.

Step 1 — Understand the Problem: "Top three per category" is a ranking that restarts in every category. A global `ORDER BY units DESC LIMIT 3` gives three products for the whole shop. The ranking also runs on product totals, not order lines, so aggregation comes first. The tie rule picks the function: tied products share a position and the next product takes the next position, which is what DENSE_RANK does.

Step 2 — Identify the Tables and Relationships: `OrderDetails` holds the units (one row per product per order). `Products` maps each line to a product and its `CategoryID` (`OrderDetails.ProductID` → `Products.ProductID`). `Category` turns `CategoryID` into a name (`Products.CategoryID` → `Category.CategoryID`). Lines whose `ProductID` has no product cannot be placed in any category, so the inner join to `Products` drops them on purpose.

Step 3 — Build the Query Logic:

1. Join lines to products and sum `Quantity` per product. Keep `CategoryID` in the grouping.
2. In a second CTE, apply `DENSE_RANK() OVER (PARTITION BY CategoryID ORDER BY units_sold DESC)`.
3. In the outer query, keep rows with rank 3 or better and LEFT JOIN `Category` for the name.

Step 4 — Final SQL Solution:

```sql
WITH product_units AS (
    SELECT p.CategoryID,
           p.ProductID,
           p.Product,
           SUM(od.Quantity) AS units_sold
    FROM OrderDetails AS od
    JOIN Products     AS p ON p.ProductID = od.ProductID
    GROUP BY p.CategoryID, p.ProductID, p.Product
),
ranked AS (
    SELECT pu.CategoryID,
           pu.ProductID,
           pu.Product,
           pu.units_sold,
           DENSE_RANK() OVER (PARTITION BY pu.CategoryID
                              ORDER BY pu.units_sold DESC NULLS LAST) AS product_rank
    FROM product_units AS pu
)
SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
       r.product_rank,
       r.ProductID,
       r.Product,
       r.units_sold
FROM ranked AS r
LEFT JOIN Category AS cat ON cat.CategoryID = r.CategoryID
WHERE r.product_rank <= 3
ORDER BY CategoryName, r.product_rank, r.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `product_units` reduces lines to one row per product, so a product cannot take several positions.
- `PARTITION BY pu.CategoryID` restarts the ranking for every category.
- `ORDER BY pu.units_sold DESC NULLS LAST` ranks the biggest seller first. A product whose only line has a NULL quantity would have NULL units; `NULLS LAST` keeps it from being ranked 1, since PostgreSQL sorts NULLs first under DESC by default.
- `DENSE_RANK()` gives tied rows the same number and continues with the next integer. Units of 142, 122, 122, 94 get ranks 1, 2, 2, 3.
- The window sits in its own CTE because a window function cannot be used in WHERE. SQL evaluates WHERE before window functions, so `WHERE DENSE_RANK() OVER (...) <= 3` fails with "window functions are not allowed in WHERE". Computing the rank in a CTE and filtering outside it is the standard pattern.
- `LEFT JOIN Category` keeps products whose `CategoryID` points nowhere, labelled `Unmapped`, instead of dropping them.

Step 6 — Sample Output:

categoryname	product_rank	productid	product	units_sold
Baby Care	1	676	Shampoo - Gentle Baby	252
Baby Care	2	759	Fresh Feel Baby Powder	186
Baby Care	3	20534	Wonder Pants - Small, Combo	159
Bakery, Cakes & Dairy	1	19725	Ultimate Paneer	1525
Bakery, Cakes & Dairy	2	2297	Burger Buns - Safe, Preservative Free	1155
Bakery, Cakes & Dairy	3	20003	Butter Milk - Uht, Spicy	758
Beauty & Hygiene	1	5733	Naturally Long Shikakai Shampoo	2493
Beauty & Hygiene	2	13924	Vitamin C, E & Hyaluronic Acid De Pigmentation Cream	1665

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 36 rows.

Step 7 — Why This Solution Works: Aggregation fixes the grain at one row per product. The partition makes each category its own ranking, and DENSE_RANK applies the badge rule directly: every product on the top three units levels gets through, and none is cut by a coin toss. The filter runs after the rank exists, so it sees each product's final rank.

Step 8 — Common Mistakes:

- Using ROW_NUMBER. It always gives exactly three rows per category, so when two products tie for third, one of them loses its badge for no business reason.
- Using RANK when the rule says "three levels". RANK skips numbers after a tie (1, 2, 2, 4), so a tie at second place leaves only two levels under `<= 3`. Compare all three side by side:

```sql
WITH product_units AS (
    SELECT p.CategoryID, p.ProductID, SUM(od.Quantity) AS units_sold
    FROM OrderDetails AS od
    JOIN Products     AS p ON p.ProductID = od.ProductID
    GROUP BY p.CategoryID, p.ProductID
)
SELECT pu.CategoryID, pu.ProductID, pu.units_sold,
       ROW_NUMBER() OVER w AS row_num,
       RANK()       OVER w AS rank_num,
       DENSE_RANK() OVER w AS dense_rank_num
FROM product_units AS pu
WINDOW w AS (PARTITION BY pu.CategoryID ORDER BY pu.units_sold DESC NULLS LAST, pu.ProductID)
ORDER BY pu.CategoryID, row_num;
```

- Partitioning by `CategoryName` after an INNER JOIN to `Category`. Products in unmapped categories vanish before they are ranked.

Step 9 — Interview Follow-Up: The product team says a category with a five-way tie at third place should show at most four badges. How would you enforce that cap while still preferring ties over arbitrary cuts?

Step 10 — Optimization or Alternative: PostgreSQL also allows a LATERAL join: for each category, a subquery orders that category's products by units with `FETCH FIRST 3 ROWS WITH TIES`. Note that this keeps ties at the cut-off only, which is closer to RANK than DENSE_RANK. For a full report across all categories, the window version is simpler and makes one pass over the aggregated products.

Question 50: Customers who buy across at least ten categories

Difficulty: Medium

Primary Topic: COUNT DISTINCT with HAVING over joins

Combined Topics: Ordered list aggregation (STRING_AGG / ARRAY_AGG), Multi-table join

Business Scenario: The CRM team is building a "whole basket" segment for a loyalty pilot. A customer who already buys from several parts of the store treats the shop as their main grocer and is worth a premium tier. A "broad buyer" is a customer whose order lines, across all time, cover at least ten different product categories. The campaign team wants the category IDs in one readable column.

Problem Statement: For every customer whose order lines cover ten or more distinct `CategoryID` values, return the number of distinct categories, the category IDs as a comma-separated list in ascending numeric order, and the number of distinct orders that contain lines. Sort by categories bought (most first), then orders (most first), then customer ID.

Expected Output:

- `CustomerID`
- `categories_bought`: distinct `CategoryID` values across the customer's lines
- `category_ids`: those IDs as text, for example `1, 2, 5`, in numeric order
- `orders`: distinct orders with at least one line

Tables Required: `Orders`, `OrderDetails`, `Products`

SQL Concepts Tested: joining across three tables to reach the attribute you need, COUNT DISTINCT after a fan-out, HAVING on a distinct count, aggregating values into an ordered list.

Step 1 — Understand the Problem: The customer lives on `Orders`, the category lives on `Products`, and the only bridge between them is `OrderDetails`. After joining all three, one customer has one row per line they ever bought, and the same category repeats many times. "Ten categories" means ten distinct values in that repeated column, so the count must be `COUNT(DISTINCT ...)`. The same goes for orders: an order with four lines appears four times.

Step 2 — Identify the Tables and Relationships: `Orders.OrderID` → `OrderDetails.OrderID` (one order, many lines), `OrderDetails.ProductID` → `Products.ProductID` (many lines, one product). `Category` is not needed because the output asks for IDs, not names. A `CategoryID` with no row in `Category` is still a distinct code on the product and counts as its own category. Products with a NULL `CategoryID` add nothing, because COUNT DISTINCT skips NULLs.

Step 3 — Build the Query Logic:

1. Join `Orders` to `OrderDetails` to `Products` with inner joins. Lines whose order or product is missing cannot be attributed to a customer and category, so dropping them is correct here.
2. Group by `CustomerID`.
3. Count distinct categories and distinct orders.
4. Build the ID list with duplicates removed and numeric ordering.
5. Keep groups with `COUNT(DISTINCT p.CategoryID) >= 10`.

Step 4 — Final SQL Solution:

```sql
SELECT o.CustomerID,
       COUNT(DISTINCT p.CategoryID) AS categories_bought,
       ARRAY_TO_STRING(ARRAY_AGG(DISTINCT p.CategoryID ORDER BY p.CategoryID), ', ') AS category_ids,
       COUNT(DISTINCT o.OrderID)    AS orders
FROM Orders       AS o
JOIN OrderDetails AS od ON od.OrderID  = o.OrderID
JOIN Products     AS p  ON p.ProductID = od.ProductID
WHERE p.CategoryID IS NOT NULL
GROUP BY o.CustomerID
HAVING COUNT(DISTINCT p.CategoryID) >= 10
ORDER BY categories_bought DESC, orders DESC, o.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- The two joins take each line to its customer (through `Orders`) and its category (through `Products`). The result has one row per line.
- `WHERE p.CategoryID IS NOT NULL` keeps NULLs out of the list. COUNT DISTINCT would ignore them anyway, but ARRAY_AGG would not.
- `COUNT(DISTINCT p.CategoryID)` counts each category once per customer, however many lines repeat it.
- `ARRAY_AGG(DISTINCT p.CategoryID ORDER BY p.CategoryID)` collects the distinct IDs as integers in numeric order. `ARRAY_TO_STRING(..., ', ')` joins them into text.
- Why not `STRING_AGG` directly? STRING_AGG needs text input, and PostgreSQL requires that when an aggregate uses DISTINCT, its ORDER BY must use the same expression as the argument. So you are forced to write `STRING_AGG(DISTINCT p.CategoryID::text, ', ' ORDER BY p.CategoryID::text)`, which sorts as text: `1, 10, 2`. Aggregating the integers first avoids that.
- `COUNT(DISTINCT o.OrderID)` undoes the line fan-out for orders.
- `HAVING` filters on the distinct count after grouping.

Step 6 — Sample Output:

customerid	categories_bought	category_ids	orders
102192	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	188
101237	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	151
101294	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	138
104856	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	135
102897	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	124
103381	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	96
101751	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	88
104233	12	5001, 5002, 5003, 5004, 5005, 5006, 5007, 5008, 5009, 5010, 5011, 5012	84

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 547 rows.

Step 7 — Why This Solution Works: The join produces line grain, and every measure in the SELECT is written for that grain. Distinct counts collapse the repetition for categories and orders, and the HAVING clause uses the same distinct count as the output column, so the filter and the reported number can never disagree. The ID list is built from the same distinct set, so its length always equals `categories_bought`.

Step 8 — Common Mistakes:

- Counting without DISTINCT. `COUNT(p.CategoryID)` counts lines, so a customer who bought from a few categories across many orders qualifies as a ten-category buyer:

```
SELECT o.CustomerID,
       COUNT(p.CategoryID)          AS lines_not_categories,
       COUNT(DISTINCT p.CategoryID) AS real_categories
FROM Orders       AS o
JOIN OrderDetails AS od ON od.OrderID  = o.OrderID
JOIN Products     AS p  ON p.ProductID = od.ProductID
GROUP BY o.CustomerID
HAVING COUNT(p.CategoryID) >= 10
ORDER BY real_categories, o.CustomerID;
```

Every row with `real_categories` below 10 is a false positive.

- Putting the condition in WHERE. `WHERE COUNT(DISTINCT p.CategoryID) >= 10` is rejected, because aggregates do not exist yet when WHERE runs.
- Grouping by `c.CustomerID` after a LEFT JOIN to `Customers`. Orders whose customer is missing collapse into one NULL "customer" with a mixed basket. Group by `o.CustomerID`.

Step 9 — Interview Follow-Up: The CRM team wants the same segment but only counting the last 12 months of the data, anchored to the latest order date. Where does that filter go, and why does it change `orders` but not the meaning of `categories_bought`?

Step 10 — Optimization or Alternative: Deduplicating first lets plain STRING_AGG sort numerically, because there is no DISTINCT inside the aggregate any more:

```
WITH customer_category AS (
    SELECT DISTINCT o.CustomerID, p.CategoryID
    FROM Orders       AS o
    JOIN OrderDetails AS od ON od.OrderID  = o.OrderID
    JOIN Products     AS p  ON p.ProductID = od.ProductID
    WHERE p.CategoryID IS NOT NULL
)
SELECT cc.CustomerID,
       COUNT(*) AS categories_bought,
       STRING_AGG(cc.CategoryID::text, ', ' ORDER BY cc.CategoryID) AS category_ids
FROM customer_category AS cc
GROUP BY cc.CustomerID
HAVING COUNT(*) >= 10
ORDER BY categories_bought DESC, cc.CustomerID;
```

MySQL's equivalent is `GROUP_CONCAT(DISTINCT CategoryID ORDER BY CategoryID SEPARATOR ', ')`, which sorts numerically without this workaround.

Question 51: Orders fulfilled by one supplier vs several

Difficulty: Medium

Primary Topic: Two-level aggregation with bucketing

Combined Topics: COUNT DISTINCT per order, CASE buckets, Percentage of total with a window over an aggregate

Business Scenario: The fulfilment team pays a handling cost every time an order has to be consolidated from more than one supplier's stock. Before redesigning warehouse slotting, they want to know how common split orders are. "Supplier mix" means the number of different suppliers across an order's lines.

Problem Statement: For every order in `OrderDetails`, count the distinct suppliers across its lines. Bucket orders as `Single supplier`, `2 suppliers` or `3+ suppliers`, and use `No supplier recorded` when all of an order's lines have a NULL `SupplierID`. For each bucket, return the number of orders and its share of all orders. Sort from fewest suppliers to most.

Expected Output:

- `supplier_mix`: the bucket label
- `orders`: orders in the bucket
- `pct_of_orders`: share of all orders with lines, 2 decimals

Tables Required: `OrderDetails`

SQL Concepts Tested: aggregating lines to the order grain, COUNT DISTINCT, CASE over an aggregate, a second GROUP BY on buckets, `SUM(COUNT(*)) OVER ()` for the total.

Step 1 — Understand the Problem: The bucket is a property of an order, but the supplier sits on each line, and one line knows only its own supplier. So there are two levels of aggregation. Level one: one row per order with its distinct supplier count. Level two: group those order rows by bucket and count them. One GROUP BY cannot do both, because the level-two key is the result of level one.

Step 2 — Identify the Tables and Relationships: Only `OrderDetails`: one row per product line, carrying `OrderID` and `SupplierID`. The population is orders that have lines. A `SupplierID` with no row in `Suppliers` is still a distinct supplier code, so it counts.

Step 3 — Build the Query Logic:

1. In a CTE, group lines by `OrderID` and take `COUNT(DISTINCT SupplierID)`.
2. In the outer query, turn that count into a label with CASE and group by the label.
3. Count orders per bucket and divide by the total across buckets.
4. Sort buckets by the smallest supplier count they contain.

Step 4 — Final SQL Solution:

```
WITH order_suppliers AS (
    SELECT od.OrderID,
           COUNT(DISTINCT od.SupplierID) AS suppliers
    FROM OrderDetails AS od
    GROUP BY od.OrderID
)
SELECT CASE WHEN os.suppliers = 0 THEN 'No supplier recorded'
            WHEN os.suppliers = 1 THEN 'Single supplier'
            WHEN os.suppliers = 2 THEN '2 suppliers'
            ELSE '3+ suppliers'
       END                                                  AS supplier_mix,
       COUNT(*)                                             AS orders,
       ROUND(100.0 * COUNT(*) / NULLIF(SUM(COUNT(*)) OVER (), 0), 2) AS pct_of_orders
FROM order_suppliers AS os
GROUP BY 1
ORDER BY MIN(os.suppliers);
```

Step 5 — Line-by-Line Explanation:

- `COUNT(DISTINCT od.SupplierID)` per `OrderID` counts each supplier once, even if it supplied five lines in that order. It skips NULLs, so an order whose lines are all NULL gets 0, which the first CASE branch labels.
- After the CTE there is one row per order, so `COUNT(*)` in the outer query counts orders.
- The CASE covers 0, 1, 2 and everything else, so every possible count lands in exactly one bucket.
- `GROUP BY 1` groups by the first SELECT expression, the CASE, so you do not repeat it.
- `SUM(COUNT(*)) OVER ()` is a window over an aggregate. GROUP BY runs first and produces one `COUNT(*)` per bucket. The window then adds those counts across all bucket rows; `OVER ()` with nothing inside means the whole result. Each row gets the grand total next to its own count.
- `ORDER BY MIN(os.suppliers)` sorts buckets by the smallest count they hold, giving 0, 1, 2, 3+ without a separate sort column.

Step 6 — Sample Output:

supplier_mix	orders	pct_of_orders
Single supplier	9793	21.09
2 suppliers	11489	24.74
3+ suppliers	25149	54.16

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 3 rows returned.

Step 7 — Why This Solution Works: Each GROUP BY has one job. The first changes the grain from line to order, the grain the business question is about. The second groups orders into buckets. Every order lands in exactly one bucket, so bucket counts add up to the distinct orders in `OrderDetails` and the shares add up to 100, give or take rounding. Check: `SELECT COUNT(DISTINCT OrderID) FROM OrderDetails` must equal the sum of `orders`.

Step 8 — Common Mistakes:

- Using `COUNT(od.SupplierID)` without DISTINCT in the CTE. An order with three lines from one supplier is then reported as a 3-supplier order. This lists every order that version would bucket wrongly:

```
WITH order_suppliers AS (
    SELECT od.OrderID,
           COUNT(od.SupplierID)          AS supplier_lines,
           COUNT(DISTINCT od.SupplierID) AS suppliers
    FROM OrderDetails AS od
    GROUP BY od.OrderID
)
SELECT os.OrderID, os.supplier_lines, os.suppliers
FROM order_suppliers AS os
WHERE os.supplier_lines <> os.suppliers
ORDER BY os.OrderID;
```

- Dividing by the row count of `OrderDetails`. That is the number of lines, so the shares add up to far less than 100.
- Bucketing per line with `CASE WHEN SupplierID ...` and no order-level step. A line has one supplier, so every line looks like a single-supplier order.

Step 9 — Interview Follow-Up: Fulfilment now wants the split weighted by units, so a 20-unit split order matters more than a 1-unit one. What changes at each level of the query?

Step 10 — Optimization or Alternative: A scalar subquery gives the same denominator on engines without window functions: replace `SUM(COUNT(*)) OVER ()` with `(SELECT COUNT(*) FROM order_suppliers)`. If you want a numeric key to group and sort on, `LEAST(os.suppliers, 3)` maps 3 and above to one value, and the label can be added at the end.

Question 52: Where shipping time goes: dispatch vs transit by shipper

Difficulty: Medium

Primary Topic: Averages with validity filters (FILTER)

Combined Topics: Date differences, LEFT JOIN with label, CTE

Business Scenario: Customers complain that deliveries are slow, and the warehouse and the couriers blame each other. The logistics head wants each shipper's delivery time split into two parts: dispatch (order placed to handed to the courier, which the warehouse controls) and transit (handed over to delivered, which the courier controls). The two parts must add up to the total, or the meeting will argue about the arithmetic instead of the fix.

Problem Statement: For each shipper, using only orders that have both a ship date and a delivery date in a valid sequence (order ≤ ship ≤ delivery), return the number of orders measured, average days to ship, average days in transit and average total days. Also show how many of the shipper's orders were excluded. Orders with a missing or unknown `ShipperID` go under `Unknown shipper`. Sort by average total days, slowest first.

Expected Output:

- `shipper_name`: `CompanyName`, or `Unknown shipper`
- `orders_measured`: orders that pass the validity rule
- `avg_days_to_ship`: average of ship date minus order date, 2 decimals
- `avg_days_in_transit`: average of delivery date minus ship date, 2 decimals
- `avg_days_total`: average of delivery date minus order date, 2 decimals
- `orders_excluded`: the shipper's orders that fail the rule

Tables Required: `Orders`, `Shippers`

SQL Concepts Tested: date subtraction, aggregate FILTER clauses, keeping every average on the same population, labelling unmatched keys.

Step 1 — Understand the Problem: Each order has three durations, and `total = dispatch + transit` holds for every order with all three dates. That identity survives averaging only if all three averages cover the same orders. Plain `AVG` skips NULLs column by column, so an order with a ship date but no delivery date feeds the dispatch average only, and the parts stop adding up. Orders delivered before they were shipped add negative durations. Both problems need one shared rule.

Step 2 — Identify the Tables and Relationships: `Orders` holds `OrderDate`, `ShipDate`, `DeliveryDate` and `ShipperID`. `Shippers` turns `ShipperID` into a name (many orders to one shipper). There is no line table involved, so each order is counted once.

Step 3 — Build the Query Logic:

1. In a CTE, compute the three durations per order and one boolean, `is_measurable`, for the validity rule.
2. LEFT JOIN `Shippers` and group by the labelled shipper name.
3. Apply `FILTER (WHERE ot.is_measurable)` to the count and to all three averages.
4. Count the rest with `FILTER (WHERE NOT ot.is_measurable)`.

Step 4 — Final SQL Solution:

```sql
WITH order_timing AS (
    SELECT o.OrderID,
           o.ShipperID,
           o.ShipDate::date     - o.OrderDate::date AS days_to_ship,
           o.DeliveryDate::date - o.ShipDate::date  AS days_in_transit,
           o.DeliveryDate::date - o.OrderDate::date AS days_total,
           (o.OrderDate IS NOT NULL AND o.ShipDate IS NOT NULL AND o.DeliveryDate IS NOT NULL
            AND o.ShipDate >= o.OrderDate AND o.DeliveryDate >= o.ShipDate) AS is_measurable
    FROM Orders AS o
)
SELECT COALESCE(sh.CompanyName, 'Unknown shipper')                      AS shipper_name,
       COUNT(*) FILTER (WHERE ot.is_measurable)                         AS orders_measured,
       ROUND(AVG(ot.days_to_ship)    FILTER (WHERE ot.is_measurable), 2) AS avg_days_to_ship,
       ROUND(AVG(ot.days_in_transit) FILTER (WHERE ot.is_measurable), 2) AS avg_days_in_transit,
       ROUND(AVG(ot.days_total)      FILTER (WHERE ot.is_measurable), 2) AS avg_days_total,
       COUNT(*) FILTER (WHERE NOT ot.is_measurable)                     AS orders_excluded
FROM order_timing AS ot
LEFT JOIN Shippers AS sh ON sh.ShipperID = ot.ShipperID
GROUP BY COALESCE(sh.CompanyName, 'Unknown shipper')
ORDER BY avg_days_total DESC NULLS LAST, shipper_name;
```

Step 5 — Line-by-Line Explanation:

- `o.ShipDate::date - o.OrderDate::date` gives whole days as an integer. If either date is NULL, the result is NULL.
- `is_measurable` holds the validity rule in one place. The `IS NOT NULL` checks come first so the boolean is always true or false, never NULL; a NULL there would fall out of both FILTER clauses and the two counts would not add up to the shipper's orders.

- `AGG(...) FILTER (WHERE condition)` feeds the aggregate only the rows where the condition is true. Other rows stay in the group for other aggregates, which is why one query can report `orders_measured` and `orders_excluded` side by side.
- All three averages use the same filter, so they cover the same orders.
- `LEFT JOIN Shippers` keeps orders with a NULL or unknown `ShipperID`, which then share the `Unknown shipper` label.
- `DESC NULLS LAST` puts the slowest shipper first and a shipper with nothing measurable last.

Step 6 — Sample Output:

shipper_name	orders_measured	avg_days_to_ship	avg_days_in_transit	avg_days_total	orders_excluded
NorthStar Freight	2103	1.18	11.69	12.87	67
Bharat Parcel Service	2988	1.21	6.63	7.84	78
XpressHaul	4792	1.20	4.35	5.55	89
Unknown shipper	28	1.57	3.64	5.21	0
Ecomzip Couriers	5829	1.21	3.87	5.08	111
BlueLine Express	7968	1.21	3.73	4.94	161
ShadowRun Delivery	5005	1.21	2.72	3.93	85
SwiftKart Logistics	9251	1.24	2.34	3.58	145

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 9 rows.

Step 7 — Why This Solution Works: For every measured order, `days_total = days_to_ship + days_in_transit` exactly. The average of a sum equals the sum of the averages when both are taken over the same rows, so `avg_days_to_ship + avg_days_in_transit = avg_days_total` before rounding. The shared FILTER guarantees the same rows. The excluded count makes the cost of the rule visible: if a shipper has many excluded orders, its averages rest on a thin sample and its data entry needs attention.

Step 8 — Common Mistakes:

- Averaging each duration with plain AVG. Each column then uses whatever rows have the dates it needs, invalid sequences included, and the parts drift away from the total:

```sql
SELECT COALESCE(sh.CompanyName, 'Unknown shipper')              AS shipper_name,
       ROUND(AVG(o.ShipDate::date     - o.OrderDate::date), 2) AS avg_ship_all,
       ROUND(AVG(o.DeliveryDate::date - o.ShipDate::date), 2)  AS avg_transit_all,
       ROUND(AVG(o.DeliveryDate::date - o.OrderDate::date), 2) AS avg_total_all
FROM Orders AS o
LEFT JOIN Shippers AS sh ON sh.ShipperID = o.ShipperID
GROUP BY COALESCE(sh.CompanyName, 'Unknown shipper')
ORDER BY shipper_name;
```

- Putting the validity rule in WHERE. The averages are right, but `orders_excluded` is lost and a shipper whose orders all fail disappears.
- Subtracting timestamps without `::date`. The result is an interval such as `2 days 03:00:00`, and AVG returns an interval, not days.

Step 9 — Interview Follow-Up: Averages hide long tails. How would you add the 90th percentile of transit days per shipper on the same measured population?

Step 10 — Optimization or Alternative: FILTER is PostgreSQL syntax. On MySQL or SQL Server, write `AVG(CASE WHEN is_measurable THEN days_in_transit END)`: the CASE returns NULL for excluded rows and AVG ignores NULLs, so the result is the same. For a dashboard, put `is_measurable` in a reporting view so every report uses one definition of a valid delivery.

Question 53: Thirty-day sign-up to purchase conversion by country

Difficulty: Medium

Primary Topic: LEFT JOIN with a per-customer aggregate

Combined Topics: MIN per group in a CTE, Conditional COUNT with FILTER, Percentage, Date difference

Business Scenario: The growth team compares onboarding across markets. "Converted at some point" hides how long people take, and most customers here do order eventually. The number they track is speed: the share of customers whose first order came within 30 days of signing up. They want it per country, so they can see where the onboarding flow is slow.

Problem Statement: For each `Country`, return the number of customers, how many placed their first order within 30 days of `DateEntered` (day 0 to day 30 inclusive), and that share as a percentage. Customers who never ordered count in the denominator and not in the numerator. Sort by conversion rate, lowest first, then by country.

Expected Output:

- `country`
- `customers`: customers registered in that country
- `converted_30d`: customers whose first order is 0 to 30 days after sign-up
- `conversion_30d_pct`: `converted_30d` as a percentage of `customers`, 2 decimals

Tables Required: `Customers`, `Orders`

SQL Concepts Tested: reducing `Orders` to one row per customer before joining, LEFT JOIN so non-buyers stay in the denominator, date difference in days, `COUNT(*) FILTER`, percentage with `NULLIF`.

Step 1 — Understand the Problem: The denominator is customers, so the output must be built at the customer grain and then grouped by country. Each customer needs one fact from `Orders`: the date of their first order. Joining `Customers` straight to `Orders` would give one row per order and inflate the denominator for frequent buyers, so the orders are reduced first.

Step 2 — Identify the Tables and Relationships: `Customers` supplies `Country` and `DateEntered`. `Orders.CustomerID` → `Customers.CustomerID`, one customer to many orders. `DateEntered` arrives as text in the source file; the load script stores it as a DATE, so date arithmetic works directly (with a text column, cast it with `::date` first).

Step 3 — Build the Query Logic:

1. `first_orders`: `MIN(OrderDate)` per `CustomerID`.
2. LEFT JOIN it to `Customers`, so customers with no orders keep a row with a NULL first order date.
3. Compute `first_order_date - DateEntered` in days.
4. Group by country; count all customers and, with FILTER, those converted within 0 to 30 days.

Step 4 — Final SQL Solution:

```
WITH first_orders AS (
    SELECT o.CustomerID,
           MIN(o.OrderDate) AS first_order_date
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT COALESCE(NULLIF(TRIM(c.Country), ''), 'Unknown')                AS country,
       COUNT(*)                                                         AS customers,
       COUNT(*) FILTER (WHERE fo.first_order_date::date - c.DateEntered::date
                              BETWEEN 0 AND 30)                         AS converted_30d,
       ROUND(100.0 * COUNT(*) FILTER (WHERE fo.first_order_date::date - c.DateEntered::date
                                            BETWEEN 0 AND 30)
             / NULLIF(COUNT(*), 0), 2)                                  AS conversion_30d_pct
FROM Customers AS c
LEFT JOIN first_orders AS fo ON fo.CustomerID = c.CustomerID
GROUP BY COALESCE(NULLIF(TRIM(c.Country), ''), 'Unknown')
ORDER BY conversion_30d_pct, country;
```

Step 5 — Line-by-Line Explanation:

- `first_orders` collapses `Orders` to one row per customer, so the LEFT JOIN below is one-to-one and `COUNT(*)` counts customers.
- `LEFT JOIN` keeps customers who never ordered. Their `first_order_date` is NULL, so the subtraction is NULL and `BETWEEN` is not true: they count in `customers` and not in `converted_30d`, as the definition requires.
- `fo.first_order_date::date - c.DateEntered::date` is an integer number of days. The casts make it work whether the columns are DATE, TIMESTAMP or text in `YYYY/MM/DD` form.
- `BETWEEN 0 AND 30` is inclusive at both ends. Day 0 means the first order came on the sign-up day. A customer with no `DateEntered` (40 here, 37 of whom ordered) also gets a NULL gap and counts as not converted; report them separately or exclude them from both counts.
- `COALESCE(NULLIF(TRIM(c.Country), ''), 'Unknown')` groups blank and NULL countries under one label instead of two.
- `ORDER BY conversion_30d_pct` puts the slowest markets first, which is what the team acts on.

Step 6 — Sample Output:

country	customers	converted_30d	conversion_30d_pct
Singapore	47	36	76.60
United Arab Emirates	72	56	77.78
India	4832	3883	80.36
United States	77	62	80.52
United Kingdom	44	36	81.82

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 5 rows returned.

Step 7 — Why This Solution Works: Every customer contributes exactly one row before grouping, so the denominator is the customer count and the numerator is a subset of it. The 30-day definition is applied in one place, through the same expression in both the count and the percentage. A negative gap (first order before sign-up) fails `BETWEEN 0 AND 30`, so bad dates never inflate conversion; they belong in a data-quality check (Q88), not in this KPI.

Step 8 — Common Mistakes:

- Joining `Customers` to `Orders` and counting rows. Each order becomes a row, so a customer with ten orders counts ten times in the denominator:

```
SELECT c.Country,
       COUNT(*)                    AS rows_not_customers,
       COUNT(DISTINCT c.CustomerID) AS customers
FROM Customers AS c
JOIN Orders AS o ON o.CustomerID = c.CustomerID
GROUP BY c.Country
ORDER BY c.Country;
```

- Using an INNER JOIN to `first_orders`. Customers who never ordered disappear and the rate looks better than it is. On this dataset several hundred registered customers have never ordered, so the INNER version overstates conversion in every country.
- Measuring "converted ever" instead of within 30 days. Here it sits between 86% and 89% in every country, so it barely separates the markets.
- Writing `fo.first_order_date - c.DateEntered <= 30` without a lower bound, which counts negative gaps from bad dates as fast conversions.

Step 9 — Interview Follow-Up: Customers who signed up in the last 30 days of the data have not had a full 30 days yet. How would you exclude them, and why does including them bias the most recent cohort?

Step 10 — Optimization or Alternative: A correlated `EXISTS` asks the question directly, without computing first orders: `COUNT(*) FILTER (WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID AND o.OrderDate BETWEEN c.DateEntered AND c.DateEntered + 30))`. It is equivalent when no order predates sign-up, and it can stop at the first qualifying order. On a large table, an index on `Orders(CustomerID, OrderDate)` serves both versions. MySQL computes the gap with `DATEDIFF(first_order_date, DateEntered)`.

Question 54: Weighted vs simple average discount by sub-category

Difficulty: Medium

Primary Topic: Weighted average (SUM / SUM)

Combined Topics: Pre-aggregation with LEFT JOIN, Validity filter on Market_Price, Multiple CTEs

Business Scenario: The pricing team publishes an "average discount" per sub-category in its monthly deck. The merchandising head has challenged it: a deep discount on a product nobody buys counts as much as a small discount on the best seller, so the number does not describe what shoppers actually get. The team wants both views side by side: the simple average across products, and the average weighted by units sold.

Problem Statement: For each `Sub_Category`, compute each product's discount percentage, `(Market_Price - Sale_Price) / Market_Price * 100`, using only products with a valid price pair (`Market_Price > 0` and `Sale_Price` not above `Market_Price`). Return the number of products, the simple average discount, total units sold, and the unit-weighted average discount. Show the sub-categories where the two averages differ most first.

Expected Output:

- `Sub_Category`, or `Unknown`
- `products`: valid products in the sub-category, sold or not
- `simple_avg_discount_pct`: plain average of product discount percentages, 2 decimals
- `units_sold`: total units of those products
- `unit_weighted_discount_pct`: `SUM(discount_pct * units) / SUM(units)`, 2 decimals

Tables Required: `Products`, `OrderDetails`

SQL Concepts Tested: weighted average as a ratio of sums, pre-aggregating lines per product, LEFT JOIN with zero units, filtering out invalid prices before averaging.

Step 1 — Understand the Problem: A simple average gives each product one vote. A weighted average gives each product as many votes as its weight, here units sold. The formula is `SUM(value * weight) / SUM(weight)`. If every product sold the same number of units, the two averages are equal; they drift apart when big sellers have unusually small or large discounts. The gap itself is the finding.

Step 2 — Identify the Tables and Relationships: `Products` gives `Sub_Category`, `Sale_Price` and `Market_Price` (MRP). `OrderDetails` gives units (`OrderDetails.ProductID` → `Products.ProductID`, one product to many lines). Both prices are today's catalogue values, so the discount is today's discount weighted by historical units. That is a fair proxy for "what shoppers get" but not a record of past discounts.

Step 3 — Build the Query Logic:

1. Sum units per product in a CTE.
2. In a second CTE, compute `discount_pct` per product and LEFT JOIN the units, using 0 for unsold products. Apply the price validity filter here.
3. Group by sub-category and compute both averages.
4. Sort by the absolute difference between them.

Step 4 — Final SQL Solution:

```sql
WITH product_units AS (
    SELECT od.ProductID,
           SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.ProductID
),
product_discount AS (
    SELECT COALESCE(p.Sub_Category, 'Unknown')                 AS Sub_Category,
           100.0 * (p.Market_Price - p.Sale_Price) / p.Market_Price AS discount_pct,
           COALESCE(pu.units, 0)                               AS units
    FROM Products AS p
    LEFT JOIN product_units AS pu ON pu.ProductID = p.ProductID
    WHERE p.Market_Price > 0
      AND p.Sale_Price <= p.Market_Price
),
by_sub AS (
    SELECT pd.Sub_Category,
           COUNT(*)                     AS products,
           ROUND(AVG(pd.discount_pct), 2) AS simple_avg_discount_pct,
           SUM(pd.units)                AS units_sold,
           ROUND(SUM(pd.discount_pct * pd.units) / NULLIF(SUM(pd.units), 0), 2)
                                        AS unit_weighted_discount_pct
    FROM product_discount AS pd
    GROUP BY pd.Sub_Category
)
SELECT bs.Sub_Category,
       bs.products,
       bs.simple_avg_discount_pct,
       bs.units_sold,
       bs.unit_weighted_discount_pct
FROM by_sub AS bs
ORDER BY ABS(bs.unit_weighted_discount_pct - bs.simple_avg_discount_pct) DESC NULLS LAST,
         bs.Sub_Category;
```

Step 5 — Line-by-Line Explanation:

- `product_units` reduces lines to one row per product, so the join to `Products` is one-to-one and each product's discount is counted once.
- `WHERE p.Market_Price > 0` removes zero and NULL MRP, which would divide by zero or produce NULL. `p.Sale_Price <= p.Market_Price` removes products priced above MRP: a negative "discount" is a keying error, not a price cut, and it would pull both averages down. A NULL `Sale_Price` also fails this test and drops out.
- `COALESCE(pu.units, 0)` keeps unsold products. They count fully in the simple average and contribute zero weight to the weighted one, which is exactly the difference the team is asking about.
- `SUM(pd.discount_pct * pd.units) / NULLIF(SUM(pd.units), 0)` is the weighted average. If nothing in a sub-category sold, the denominator is 0, NULLIF makes it NULL, and the result is NULL instead of an error.
- `ORDER BY ABS(...) DESC NULLS LAST` lists first the sub-categories where the deck's number misleads most.

Step 6 — Sample Output:

sub_category	products	simple_avg_discount_pct	units_sold	unit_weighted_discount_pct
Cookware & Non Stick	305	23.70	739	33.27
Health Drink, Supplement	76	6.37	980	14.90
Bakeware	48	37.98	305	29.66
All Purpose Cleaners	388	7.94	4216	15.56
Sauces, Spreads & Dips	601	6.92	3201	13.09
Party & Festive Needs	43	11.38	209	17.26
Bins & Bathroom Ware	320	19.72	531	14.31
Pooja Needs	232	17.64	1938	12.57

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 90 rows.

Step 7 — Why This Solution Works: Both averages run over the same set of valid products, so the only difference between them is the weighting. Units are summed before the join, so no product is repeated and no weight is inflated. When the weighted figure is higher, the best sellers carry deeper discounts than the typical listed product; when it is lower, deep discounts sit on slow movers. Either way, the team can see which number belongs in the deck.

Step 8 — Common Mistakes:

- Averaging over the joined lines. Each product's discount appears once per order line, so this is weighted by the number of lines, not units, and unsold products vanish:

```sql
SELECT p.Sub_Category,
       ROUND(AVG(100.0 * (p.Market_Price - p.Sale_Price) / p.Market_Price), 2) AS line_weighted_pct
FROM Products AS p
JOIN OrderDetails AS od ON od.ProductID = p.ProductID
WHERE p.Market_Price > 0 AND p.Sale_Price <= p.Market_Price
GROUP BY p.Sub_Category
ORDER BY p.Sub_Category;
```

- Averaging the percentages of sub-categories to get a store-wide figure. That is a simple average of averages and ignores how big each sub-category is.
- Dividing by `Market_Price` without guarding zero. One bad MRP stops the query.

Step 9 — Interview Follow-Up: Finance prefers a value-weighted figure: total discount given in rupees divided by total MRP value of units sold. Write that ratio and explain why it can differ from the unit-weighted average.

Step 10 — Optimization or Alternative: The value-weighted rate from Step 9 is `100.0 * SUM((Market_Price - Sale_Price) * units) / NULLIF(SUM(Market_Price * units), 0)`. It weights each unit by its MRP, so expensive items dominate, which suits a finance view. Keep the validity filter in one CTE so every average shares it, and send the rows it removes to the catalogue team as a fix list.

Question 55: Category list-price value: latest year vs previous year

Difficulty: Medium

Primary Topic: Conditional aggregation by year

Combined Topics: Anchored year boundaries, Multi-table join, Growth percentage

Business Scenario: Category managers are setting next year's targets and want one table: each category's value last year, this year, and the growth between them. The table will be refreshed on whatever extract the analytics team receives, so "this year" cannot be hard-coded. It means the calendar year of the most recent order in the data, and "previous year" is the calendar year before it.

Problem Statement: For each category, return the list-price value of order lines in the previous calendar year and in the latest calendar year, plus year-over-year growth as a percentage. The latest year is the year of `MAX(OrderDate)`. Products whose `CategoryID` has no `Category` row are reported as `Unmapped`. Sort by growth, highest first.

Expected Output:

- `CategoryName`, or `Unmapped`
- `prev_year_value`: list-price value in the previous calendar year, 0 if none
- `latest_year_value`: list-price value in the latest calendar year, 0 if none
- `yoy_growth_pct`: `(latest - previous) / previous * 100`, 2 decimals, NULL when previous is 0

Tables Required: `Orders`, `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: anchoring dates to the data, half-open year ranges, conditional aggregation with FILTER to pivot two periods into columns, growth with NULLIF.

Step 1 — Understand the Problem: One row per category with one value column per year is a pivot: the same measure split by a condition into side-by-side columns. Conditional aggregation does it in one pass: group by category, and inside each SUM keep only one year's rows. The value is list-price value, `Quantity * Sale_Price`. `Sale_Price` is today's catalogue price, not the price paid, so both years are valued at the same prices. Growth here reflects volume and mix, not price rises. Say so when you present it.

Step 2 — Identify the Tables and Relationships: `Orders` supplies `OrderDate` (the year). `OrderDetails` supplies `Quantity` (one order, many lines). `Products` supplies `Sale_Price` and `CategoryID`. `Category` supplies the name, via LEFT JOIN so unmapped products are kept. No order-level column is summed, so the order-to-line fan-out is harmless here.

Step 3 — Build the Query Logic:

1. Compute the anchor once: the first day of the latest year, `DATE_TRUNC('year', MAX(OrderDate))`.
2. Join the four tables, keep only orders from the start of the previous year up to the end of the latest year, and tag each line `previous` or `latest`.
3. Group by category and sum value with a FILTER per period.
4. Compute growth against the previous year.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS latest_year_start
    FROM Orders AS o
),
line_values AS (
    SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
           o.OrderDate >= a.latest_year_start     AS is_latest_year,
           od.Quantity * p.Sale_Price             AS list_price_value
    FROM Orders AS o
    CROSS JOIN anchor AS a
    JOIN OrderDetails  AS od  ON od.OrderID    = o.OrderID
    JOIN Products      AS p   ON p.ProductID   = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    WHERE o.OrderDate >= a.latest_year_start - INTERVAL '1 year'
      AND o.OrderDate <  a.latest_year_start + INTERVAL '1 year'
),
by_category AS (
    SELECT lv.CategoryName,
           COALESCE(SUM(lv.list_price_value) FILTER (WHERE NOT lv.is_latest_year), 0) AS prev_year_value,
           COALESCE(SUM(lv.list_price_value) FILTER (WHERE lv.is_latest_year), 0)     AS latest_year_value
    FROM line_values AS lv
    GROUP BY lv.CategoryName
)
SELECT bc.CategoryName,
       bc.prev_year_value,
       bc.latest_year_value,
       ROUND(100.0 * (bc.latest_year_value - bc.prev_year_value)
             / NULLIF(bc.prev_year_value, 0), 2) AS yoy_growth_pct
FROM by_category AS bc
ORDER BY yoy_growth_pct DESC NULLS LAST, bc.CategoryName;
```

Step 5 — Line-by-Line Explanation:

- `anchor` truncates the latest order date to 1 January of that year. Anchoring on `MAX(OrderDate)` instead of `CURRENT_DATE` makes the report describe the data you have; on an extract that ends last year, `CURRENT_DATE` would give an empty "this year".
- `CROSS JOIN anchor` attaches that one date to every row, a clean way to reuse a computed constant.
- The WHERE keeps a half-open two-year window: from 1 January of the previous year up to, but not including, 1 January after the latest year. Half-open bounds work for DATE and TIMESTAMP alike.
- `is_latest_year` tags each line. Inside the window, anything not in the latest year is in the previous one.
- `SUM(...) FILTER (WHERE ...)` sums only the rows of one period. `COALESCE(..., 0)` turns "no lines in that year" into 0, because SUM over no rows returns NULL.
- `NULLIF(bc.prev_year_value, 0)` returns NULL growth for a category that had no value last year. Growth from zero has no meaningful percentage.

Step 6 — Sample Output:

categoryname	prev_year_value	latest_year_value	yoy_growth_pct
Fruits & Vegetables	162495.00	419457.00	158.14
Beverages	1182514.00	2967855.00	150.98
Bakery, Cakes & Dairy	542870.00	1318680.00	142.91
Eggs, Meat & Fish	411281.00	993172.00	141.48
Foodgrains, Oil & Masala	2240858.00	5190475.00	131.63
Gourmet & World Food	1822570.00	4186440.00	129.70
Snacks & Branded Foods	1436144.00	3267579.00	127.52
Beauty & Hygiene	6386812.00	14395264.00	125.39

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Every line is read once and lands in exactly one period, and the WHERE keeps other years out of both columns. Check: the sum of both columns across categories equals the list-price value of all lines in the two-year window.

Step 8 — Common Mistakes:

- Hard-coding years or using `CURRENT_DATE`. On next year's extract the report silently compares the wrong years or shows zeros.
- Comparing a partial latest year with a full previous year. If `MAX(OrderDate)` is in, say, June, the latest year has six months and growth looks terrible. Check how complete the latest year is first:

```sql
SELECT MAX(o.OrderDate)                                  AS latest_order_date,
       DATE_TRUNC('year', MAX(o.OrderDate))::date        AS latest_year_start,
       MAX(o.OrderDate)::date - DATE_TRUNC('year', MAX(o.OrderDate))::date + 1 AS days_covered
FROM Orders AS o;
```

If coverage is short, compare year-to-date with the same days last year.

- Filtering with `EXTRACT(YEAR FROM o.OrderDate) IN (...)`. It works, but wrapping the column in a function stops PostgreSQL from using an index on `OrderDate`.

Step 9 — Interview Follow-Up: A category that sold nothing in either year is missing from this output. How would you change the query so every row of `Category` appears with zeros?

Step 10 — Optimization or Alternative: The portable form of the pivot is `SUM(CASE WHEN is_latest_year THEN list_price_value ELSE 0 END)`, which runs on MySQL and SQL Server; FILTER is PostgreSQL syntax. With an index on `Orders(OrderDate)`, the half-open range lets the planner read only two years of orders before joining to lines, which matters once `Orders` holds many years.

Question 56: Customers who ordered last year but not this year

Difficulty: Medium

Primary Topic: EXCEPT set operation

Combined Topics: Anchored year boundaries, NOT EXISTS alternative, Join back for details

Business Scenario: The retention team is planning a win-back campaign. The target list is customers who were active in the previous calendar year and have placed nothing in the latest calendar year. "Latest year" is the calendar year of the most recent order in the data, so the list works on any extract. For prioritising calls, the team wants each customer's last order date and how many orders they placed in the previous year.

Problem Statement: Find customers with at least one order in the previous calendar year and no order in the latest calendar year. For each, return the customer name, their last order date and the number of orders they placed in the previous year. Orders whose `CustomerID` has no row in `Customers` still count; label the name `Unknown customer`. Sort by last order date, most recent first, then customer ID.

Expected Output:

- `CustomerID`
- `customer_name`: first and last name, or `Unknown customer`
- `last_order_date`: the customer's most recent order date
- `orders_prev_year`: orders placed in the previous calendar year

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: EXCEPT for set difference, set operations on a single key, anchored half-open year ranges, joining back to fetch details.

Step 1 — Understand the Problem: The question is "set A minus set B". A is the customers who ordered in the previous year, B the customers who ordered in the latest year. `EXCEPT` does exactly that: it takes the rows of the first query and removes any row that also appears in the second. Once you have the IDs, the details are an ordinary aggregation.

Step 2 — Identify the Tables and Relationships: `Orders` defines both sets through `CustomerID` and `OrderDate`. `Customers` provides the name, via LEFT JOIN so an unknown `CustomerID` is labelled rather than dropped.

Step 3 — Build the Query Logic:

1. Anchor: `DATE_TRUNC('year', MAX(OrderDate))` is 1 January of the latest year.
2. First query: `CustomerID` of orders from 1 January of the previous year up to the anchor.
3. Second query: `CustomerID` of orders on or after the anchor.
4. `EXCEPT` the second from the first. The result is a list of lapsed IDs.
5. Join those IDs back to `Orders` and `Customers`, aggregate the details.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS latest_year_start
    FROM Orders AS o
),
lapsed AS (
    SELECT o.CustomerID
    FROM Orders AS o
    CROSS JOIN anchor AS a
    WHERE o.OrderDate >= a.latest_year_start - INTERVAL '1 year'
      AND o.OrderDate <  a.latest_year_start
    EXCEPT
    SELECT o.CustomerID
    FROM Orders AS o
    CROSS JOIN anchor AS a
    WHERE o.OrderDate >= a.latest_year_start
)
SELECT l.CustomerID,
       COALESCE(NULLIF(CONCAT_WS(' ', c.FirstName, c.LastName), ''), 'Unknown customer') AS customer_name,
       MAX(o.OrderDate)                                                                  AS last_order_date,
       COUNT(*) FILTER (WHERE o.OrderDate >= a.latest_year_start - INTERVAL '1 year')    AS orders_prev_year
FROM lapsed AS l
CROSS JOIN anchor AS a
JOIN Orders         AS o ON o.CustomerID = l.CustomerID
LEFT JOIN Customers AS c ON c.CustomerID = l.CustomerID
GROUP BY l.CustomerID, c.FirstName, c.LastName
ORDER BY last_order_date DESC, l.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `anchor` sets "latest year" from the data, not `CURRENT_DATE`, so the list is right on any extract.

- The two SELECTs inside `lapsed` must return the same number of columns with compatible types, because EXCEPT compares whole rows. Each returns one column, `CustomerID`.
- `EXCEPT` also removes duplicates: a customer with five previous-year orders appears once. `EXCEPT ALL` keeps duplicates and subtracts counts, which is rarely wanted for a list of people.
- Unlike `=`, EXCEPT treats two NULLs as equal, so a NULL `CustomerID` in both years cancels out.
- The join back to `Orders` brings every order of each lapsed customer. Since none of them ordered in the latest year, `MAX(o.OrderDate)` falls in the previous year.
- `COUNT(*) FILTER (...)` counts only previous-year orders, so older orders do not inflate it.
- `CONCAT_WS` skips NULL parts and returns an empty string when both names are NULL; `NULLIF(..., '')` lets `COALESCE` label it.

Step 6 — Sample Output:

customerid	customer_name	last_order_date	orders_prev_year
100951	Anil Bose	2024-12-31	21
101270	Pallavi Chatterjee	2024-12-31	2
103373	Ananya Jain	2024-12-31	17
103662	Divya Naidu	2024-12-31	17
102566	Meera Shah	2024-12-30	2
104713	Arnav Bose	2024-12-30	5
101228	Sai Hegde	2024-12-29	13
101695	Venkat Pinto	2024-12-29	31

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 566 rows.

Step 7 — Why This Solution Works: EXCEPT states the business rule directly as a set difference on distinct values, so each customer appears at most once. The half-open ranges put every order in exactly one period. The detail query runs only for lapsed IDs, so active customers cannot leak in.

Step 8 — Common Mistakes:

- Testing both conditions on the same row. A single order cannot be in both years, so the second condition is always true and every previous-year customer comes back, including the ones still buying:

```
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS latest_year_start
    FROM Orders AS o
)
SELECT DISTINCT o.CustomerID
FROM Orders AS o
CROSS JOIN anchor AS a
WHERE o.OrderDate >= a.latest_year_start - INTERVAL '1 year'
  AND o.OrderDate <  a.latest_year_start
  AND NOT (o.OrderDate >= a.latest_year_start)
ORDER BY o.CustomerID;
```

"Did not order this year" is a fact about all of a customer's rows, so it needs EXCEPT, NOT EXISTS or an anti-join.

- Using `NOT IN (SELECT CustomerID ...)`. If the subquery returns a single NULL, `x NOT IN (..., NULL)` is never true and the result is empty.
- Putting extra columns such as `OrderDate` inside the EXCEPT queries. EXCEPT compares whole rows, so a customer would only be removed if they ordered on the same date in both years.

Step 9 — Interview Follow-Up: The team also wants customers who are new this year: ordered in the latest year and never before. Which set operation and which ranges would you use?

Step 10 — Optimization or Alternative: NOT EXISTS expresses the same rule as a filter on each previous-year order, and it is the version to use on MySQL before 8.0.31, which has no EXCEPT:

```
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS latest_year_start
    FROM Orders AS o
)
SELECT DISTINCT prev.CustomerID
FROM Orders AS prev
CROSS JOIN anchor AS a
WHERE prev.OrderDate >= a.latest_year_start - INTERVAL '1 year'
  AND prev.OrderDate <  a.latest_year_start
  AND NOT EXISTS (SELECT 1
                  FROM Orders AS cur
                  WHERE cur.CustomerID = prev.CustomerID
                    AND cur.OrderDate >= a.latest_year_start)
ORDER BY prev.CustomerID;
```

With an index on `Orders(CustomerID, OrderDate)`, PostgreSQL runs NOT EXISTS as an anti-join that stops at the first current-year order. EXCEPT builds and compares both full sets.

Question 57: One contact directory for suppliers and shippers

Difficulty: Medium

Primary Topic: UNION ALL vs UNION

Combined Topics: Literal type column, Column alignment and NULL placeholders

Business Scenario: The partner operations desk handles calls from suppliers and courier companies, and today they switch between two screens to find a phone number. They want one directory with every partner, a column saying which kind of partner it is, and the contact fields both tables can supply. Shippers have no email or country on file, and the directory should show that as blank rather than inventing values.

Problem Statement: Build one list of all suppliers and all shippers with the columns below. A company that is both a supplier and a shipper must appear twice, once per role. Sort by company name, then partner type, then ID.

Expected Output:

- `partner_type`: `Supplier` or `Shipper`
- `partner_id`: `SupplierID` or `ShipperID`
- `company_name`
- `phone`
- `email`: NULL for shippers
- `country`: NULL for shippers

Tables Required: `Suppliers`, `Shippers`

SQL Concepts Tested: stacking results with UNION ALL, how UNION differs, matching columns by position, literal columns, typed NULL placeholders, ORDER BY on a combined result.

Step 1 — Understand the Problem: Joins put tables side by side; this question stacks them. There is no key relating a supplier to a shipper, and the output wants one row per partner, not one per pair. That is a set operation. `UNION ALL` appends the rows of the second query to the rows of the first. `UNION` does the same and then removes duplicate rows. The work is choosing between them and making the two queries line up.

Step 2 — Identify the Tables and Relationships: `Suppliers` has `SupplierID`, `CompanyName`, `Phone`, `Email` and `Country`. `Shippers` has `ShipperID`, `CompanyName` and `Phone` only. The tables are unrelated, and their IDs overlap: supplier 1 and shipper 1 are different companies. That is why the type column matters.

Step 3 — Build the Query Logic:

1. Write the supplier SELECT in output order, with the literal `'Supplier'` first.
2. Write the shipper SELECT in the same order, with `'Shipper'` first and typed NULLs where shippers have no data.
3. Combine them with `UNION ALL`.
4. Add one ORDER BY at the end.

Step 4 — Final SQL Solution:

```sql
SELECT 'Supplier'    AS partner_type,
       s.SupplierID  AS partner_id,
       s.CompanyName AS company_name,
       s.Phone       AS phone,
       s.Email       AS email,
       s.Country     AS country
FROM Suppliers AS s
UNION ALL
SELECT 'Shipper',
       sh.ShipperID,
       sh.CompanyName,
       sh.Phone,
       CAST(NULL AS VARCHAR(120)),
       CAST(NULL AS VARCHAR(60))
FROM Shippers AS sh
ORDER BY company_name, partner_type, partner_id;
```

Step 5 — Line-by-Line Explanation:

- `'Supplier' AS partner_type` is a literal, the same text on every row of that branch. It records where each row came from, which would otherwise be lost once the rows are stacked.
- Columns are matched by position, not by name. Both branches need the same number of columns, and each position needs compatible types.
- Output column names come from the first branch only, so the aliases are written there and the second branch needs none.
- `CAST(NULL AS VARCHAR(120))` fills a position shippers cannot supply. PostgreSQL would accept a bare `NULL` and take the type from the other branch, but the cast states the intent and avoids type errors on engines that resolve NULL differently.
- `UNION ALL` keeps every row. The type column already makes supplier and shipper rows distinct, so there is nothing to deduplicate, and UNION ALL skips the sort or hash that UNION would run.
- The single `ORDER BY` sorts the combined result and can use only output column names or positions.

Step 6 — Sample Output:

partner_type	partner_id	company_name	phone	email	country
Shipper	6	Bharat Parcel Service	011-2338-0006	NULL	NULL
Shipper	2	BlueLine Express	+91 22 6123 0002	NULL	NULL
Shipper	9	Coastal Cargo Lines	0824-244-0009	NULL	NULL
Supplier	5	Coromandel FMCG Supplies	044-26250077	hello@coromandelfmcg.in	India
Supplier	2	Deccan Fresh Foods Pvt Ltd	040-27154400	supply@deccanfresh.in	India
Shipper	3	Ecomzip Couriers	080-4953 0003	NULL	NULL
Supplier	4	Ganga Distributors LLP	+91-120-4567810	NULL	India
Supplier	13	Gulf Gourmet Trading LLC	+971 4 340 2210	import@gulfgourmet.ae	United Arab Emirates

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 28 rows.

Step 7 — Why This Solution Works: Both branches produce rows of the same shape, so stacking them gives one valid table with a row per partner role. The literal column keeps each row traceable and overlapping IDs unambiguous. Nothing is deduplicated, so the row count always equals suppliers plus shippers, an easy check after every refresh.

Step 8 — Common Mistakes:

- Swapping two text columns in one branch. Types still match, so it runs, and shipper phone numbers appear under `email`:

```
SELECT 'Supplier' AS partner_type, s.SupplierID AS partner_id, s.CompanyName AS company_name,
       s.Phone AS phone, s.Email AS email
FROM Suppliers AS s
UNION ALL
SELECT 'Shipper', sh.ShipperID, sh.CompanyName, NULL, sh.Phone
FROM Shippers AS sh
ORDER BY partner_type, partner_id;
```

The database checks types, not meaning. Read both branches side by side.

- Using `UNION` instead of `UNION ALL`. Here it returns the same 28 rows, because the ID and contact columns keep rows apart, but it pays for a sort or hash to prove it. If someone later trims the list to name and phone, identical partner rows are silently merged and the count stops reconciling. Stacking a table on itself shows what UNION does to identical rows:

```
SELECT (SELECT COUNT(*) FROM (SELECT s.CompanyName FROM Suppliers AS s
                              UNION ALL
                              SELECT s.CompanyName FROM Suppliers AS s) AS t) AS union_all_rows,
       (SELECT COUNT(*) FROM (SELECT s.CompanyName FROM Suppliers AS s
                              UNION
                              SELECT s.CompanyName FROM Suppliers AS s) AS t) AS union_rows;
```

Step 9 — Interview Follow-Up: The desk wants to find phone numbers shared by more than one row of this directory, across both roles. How would you write that?

Step 10 — Optimization or Alternative: If the directory is used often, wrap the UNION ALL in a view so every screen reads one definition. A filter such as `WHERE partner_type = 'Shipper'` on that view can be pushed into the branches, so PostgreSQL skips the supplier table entirely. With UNION, the deduplication step sits on top and every query pays for it.

Question 58: Each customer's latest order

Difficulty: Medium

Primary Topic: ROW_NUMBER for the latest record per group

Combined Topics: Deterministic tie-break, DISTINCT ON alternative, LEFT JOIN for names

Business Scenario: The CRM team is setting up a "how was your last order?" message. For every customer who has ordered, they need exactly one row: the most recent order, its date and its amount. Customers sometimes place two orders on the same day, and the message tool will break if a customer appears twice, so the rule for that case is: the order with the higher `OrderID` counts as the later one.

Problem Statement: For every customer in `Orders`, return their latest order by `OrderDate`, breaking same-day ties by the higher `OrderID`. Return exactly one row per customer. Customers missing from `Customers` are labelled `Unknown customer`. Sort by latest order date, most recent first, then customer ID.

Expected Output:

- `CustomerID`
- `customer_name`: first and last name, or `Unknown customer`
- `latest_order_id`
- `latest_order_date`
- `latest_order_amount`: `Total_Order_Amount` of that order

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: ROW_NUMBER with PARTITION BY and ORDER BY, deterministic tie-breaking, filtering a window result, DISTINCT ON in PostgreSQL.

Step 1 — Understand the Problem: `MAX(OrderDate)` per customer gives the date but not the rest of the row. You need the whole order that holds that date. This is the "latest record per group" pattern: number each customer's orders from newest to oldest and keep number 1. The tie rule turns "newest" into a strict order, so there is always exactly one number 1.

Step 2 — Identify the Tables and Relationships: `Orders` holds everything about the order. `Customers` adds the name (`Orders.CustomerID` → `Customers.CustomerID`). The name lookup is a LEFT JOIN, done after picking the order, so a broken customer reference does not remove the order.

Step 3 — Build the Query Logic:

1. Number each customer's orders with `ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC, OrderID DESC)`.
2. In the outer query keep rows where the number is 1.
3. LEFT JOIN `Customers` for the name.

Step 4 — Final SQL Solution:

```sql
WITH numbered AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate,
           o.Total_Order_Amount,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID
                              ORDER BY o.OrderDate DESC NULLS LAST, o.OrderID DESC) AS rn
    FROM Orders AS o
)
SELECT n.CustomerID,
       COALESCE(NULLIF(CONCAT_WS(' ', c.FirstName, c.LastName), ''), 'Unknown customer') AS customer_name,
       n.OrderID            AS latest_order_id,
       n.OrderDate          AS latest_order_date,
       n.Total_Order_Amount AS latest_order_amount
FROM numbered AS n
LEFT JOIN Customers AS c ON c.CustomerID = n.CustomerID
WHERE n.rn = 1
ORDER BY n.OrderDate DESC NULLS LAST, n.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `ROW_NUMBER()` gives 1, 2, 3 … within each partition, with no repeats even when the sort values tie. That property is why it is the right tool when the business needs exactly one row.
- `PARTITION BY o.CustomerID` restarts the numbering for each customer.
- `ORDER BY o.OrderDate DESC NULLS LAST` makes the newest order number 1. Without `NULLS LAST`, PostgreSQL would put an order with a NULL date first under DESC and call it the latest.
- `o.OrderID DESC` is the tie-break. Two orders on the same date would otherwise be numbered in whatever order the database reads them, and the "latest" order could change between runs. A unique key as the last sort column makes the result deterministic.
- `WHERE n.rn = 1` filters the window result. It sits in the outer query because WHERE runs before window functions within a single SELECT.
- The LEFT JOIN to `Customers` comes after the filter, on a one-row-per-customer result, so it cannot duplicate anything.

Step 6 — Sample Output:

customerid	customer_name	latest_order_id	latest_order_date	latest_order_amount
100013	Rashmi Patel	5046264	2025-12-28	18.00
100029	Mahesh Rao	5046265	2025-12-28	988.00
100035	Aarav Rao	5046266	2025-12-28	1879.00
100092	Jyoti Sheikh	5046268	2025-12-28	998.00
100120	Arjun Mishra	5046269	2025-12-28	1447.80
100254	Isha Saxena	5046270	2025-12-28	343.00
100345	Kunal Sharma	5046271	2025-12-28	565.00
100533	Aarav D'Souza	5046272	2025-12-28	500.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 4427 rows.

Step 7 — Why This Solution Works: Partitioning gives each customer an independent numbering, and the sort with a unique tie-break defines exactly one first row. Keeping `rn = 1` therefore returns one row per customer, never zero and never two. Check: the row count must equal `SELECT COUNT(DISTINCT CustomerID) FROM Orders`.

Step 8 — Common Mistakes:

- Joining back on `MAX(OrderDate)`. It looks natural and returns two rows for any customer with two orders on their last date, which is exactly what breaks the message tool. This shows which customers the naive join would duplicate:

```sql
WITH last_dates AS (
    SELECT o.CustomerID, MAX(o.OrderDate) AS last_date
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT o.CustomerID,
       COUNT(*) AS rows_from_naive_join
FROM Orders AS o
JOIN last_dates AS ld
  ON ld.CustomerID = o.CustomerID
 AND ld.last_date  = o.OrderDate
GROUP BY o.CustomerID
HAVING COUNT(*) > 1
ORDER BY o.CustomerID;
```

- Using RANK or DENSE_RANK. Both give tied rows the same number, so `= 1` returns both same-day orders.
- Leaving out the tie-break. The query runs and usually looks fine, but the chosen order can change between runs, which is very hard to debug later.

Step 9 — Interview Follow-Up: The CRM team now wants each customer's latest order and the one before it on the same row. How would you change this query?

Step 10 — Optimization or Alternative: PostgreSQL has a shortcut for this exact pattern, `DISTINCT ON`. It keeps the first row of each group according to the ORDER BY:

```sql
SELECT DISTINCT ON (o.CustomerID)
       o.CustomerID,
       o.OrderID            AS latest_order_id,
       o.OrderDate          AS latest_order_date,
       o.Total_Order_Amount AS latest_order_amount
FROM Orders AS o
ORDER BY o.CustomerID, o.OrderDate DESC NULLS LAST, o.OrderID DESC;
```

The ORDER BY must start with the DISTINCT ON columns, so the final sort is by customer; wrap it in a subquery to re-sort by date. DISTINCT ON is PostgreSQL-only; the ROW_NUMBER version runs on MySQL 8, SQL Server and every warehouse. An index on `Orders(CustomerID, OrderDate DESC NULLS LAST, OrderID DESC)` matches the sort, so either version can read each customer's newest order first.

Question 59: Basket size by month

Difficulty: Medium

Primary Topic: Two-level aggregation (per order, then per month)

Combined Topics: Pre-aggregation CTE, Orders without lines, DATE_TRUNC

Business Scenario: Merchandising launched "add-on" suggestions at checkout and wants to know whether baskets got bigger. They track two numbers per month: the average number of units in an order and the average number of different products in an order. An order with no lines on record tells nothing about basket size and should be left out, but the team wants to see how many orders were measured each month.

Problem Statement: For each month, return the number of orders that have at least one line, the average units per order and the average distinct products per order. Months are taken from `OrderDate`. Sort by month.

Expected Output:

- `order_month`: first day of the month
- `orders_with_lines`: orders that month with at least one line in `OrderDetails`
- `avg_units_per_order`: average of each order's total `Quantity`, 2 decimals
- `avg_distinct_products_per_order`: average of each order's distinct `ProductID` count, 2 decimals

Tables Required: `Orders`, `OrderDetails`

SQL Concepts Tested: averaging at the right grain, aggregating lines to orders before averaging, COUNT DISTINCT per order, the difference between per-line and per-order averages.

Step 1 — Understand the Problem: "Average units per order" is an average over orders, so the thing being averaged must be one number per order: that order's total units. The lines table holds one row per product per order, so the per-order totals do not exist until you build them. The tempting shortcut, `AVG(Quantity)` on the lines, averages a different thing: units per line. An order with three lines of 2 units has 6 units in the basket, but its lines average 2.

Step 2 — Identify the Tables and Relationships: `Orders` gives the month (`OrderDate`). `OrderDetails` gives `Quantity` and `ProductID` (`OrderDetails.OrderID` → `Orders.OrderID`, one order to many lines). Orders with no lines cannot be measured. Lines whose `OrderID` has no order cannot be placed in a month. Inner joins drop both, which is what this question wants.

Step 3 — Build the Query Logic:

1. In a CTE, group `OrderDetails` by `OrderID`: total units and distinct products per order. This is level one.
2. Join that to `Orders` to get each order's month. The join is one-to-one now.
3. Group by month and average the per-order numbers. This is level two.

Step 4 — Final SQL Solution:

```sql
WITH order_baskets AS (
    SELECT od.OrderID,
           SUM(od.Quantity)            AS units,
           COUNT(DISTINCT od.ProductID) AS distinct_products
    FROM OrderDetails AS od
    GROUP BY od.OrderID
)
SELECT DATE_TRUNC('month', o.OrderDate)::date      AS order_month,
       COUNT(*)                                    AS orders_with_lines,
       ROUND(AVG(ob.units), 2)                     AS avg_units_per_order,
       ROUND(AVG(ob.distinct_products), 2)         AS avg_distinct_products_per_order
FROM Orders AS o
JOIN order_baskets AS ob ON ob.OrderID = o.OrderID
WHERE o.OrderDate IS NOT NULL
GROUP BY DATE_TRUNC('month', o.OrderDate)::date
ORDER BY order_month;
```

Step 5 — Line-by-Line Explanation:

- `order_baskets` turns lines into one row per order. `SUM(od.Quantity)` is the basket's units. `COUNT(DISTINCT od.ProductID)` is the number of different products; DISTINCT matters because a re-loaded duplicate line would otherwise count one product twice.
- `JOIN order_baskets` is an inner join, so only orders with at least one line reach the average. `COUNT(*)` after it counts exactly those orders, which is `orders_with_lines`.
- `AVG(ob.units)` averages one number per order. This is the per-order average the team asked for.
- If a line has a NULL `Quantity`, `SUM` ignores it, so that order's units are slightly understated rather than NULL. If every line of an order had a NULL quantity, `units` would be NULL and AVG would skip that order for units only.
- `DATE_TRUNC('month', ...)::date` labels each month by its first day and keeps years apart.

Step 6 — Sample Output:

order_month	orders_with_lines	avg_units_per_order	avg_distinct_products_per_order
2023-01-01	288	6.57	3.35
2023-02-01	185	6.29	3.61
2023-03-01	270	7.54	3.80
2023-04-01	310	6.43	3.53
2023-05-01	365	6.98	3.58
2023-06-01	371	6.49	3.52
2023-07-01	414	6.76	3.74
2023-08-01	464	7.36	3.69

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 36 rows.

Step 7 — Why This Solution Works: Each level of aggregation runs at one grain. Level one sums lines into orders, so every order contributes a single basket figure. Level two averages those figures within each month, so a large order counts once, not once per line. The two averages measure different things, units and variety, and both come from the same set of orders, so they can be read side by side.

Step 8 — Common Mistakes:

- Averaging `Quantity` straight from the lines. This gives units per line, which is much smaller than units per basket and moves for different reasons:

```sql
SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
       ROUND(AVG(od.Quantity), 2)             AS avg_units_per_line
FROM Orders AS o
JOIN OrderDetails AS od ON od.OrderID = o.OrderID
GROUP BY DATE_TRUNC('month', o.OrderDate)::date
ORDER BY order_month;
```

If baskets got more varied but each line stayed small, this number would fall while baskets grew.

- Computing `SUM(od.Quantity) / COUNT(*)` on the joined rows. `COUNT(*)` counts lines, not orders; it is the same mistake in a different form. `SUM(od.Quantity) / COUNT(DISTINCT o.OrderID)` is correct for units, but there is no equivalent shortcut for the distinct-products average.
- LEFT JOINing orders without lines and COALESCE-ing them to 0. That reports empty baskets that never existed and drags the average down.

Step 9 — Interview Follow-Up: The team also wants the median basket size per month, because a few bulk orders inflate the average. How would you add it?

Step 10 — Optimization or Alternative: The units average alone can be computed in one pass, `SUM(od.Quantity)::numeric / COUNT(DISTINCT o.OrderID)`, because a total divided by the number of orders is the average of the order totals. The distinct-products average has no such shortcut, so the CTE version is the one to keep. To report orders without lines as a data-quality count, add `COUNT(*) FILTER (WHERE ob.OrderID IS NULL)` on a LEFT JOIN version and keep it out of the averages.

Question 60: Sub-category share of its category

Difficulty: Medium

Primary Topic: Window SUM with PARTITION BY over aggregates

Combined Topics: GROUP BY at sub-category grain, LEFT JOIN for unmapped category, Percentage

Business Scenario: Each category manager splits shelf space and promotion slots among the sub-categories in their area. To do that fairly they want to know what share of the category's value each sub-category brings in. A sub-category's share is measured against its own category, not against the whole store, so every category's shares add up to 100.

Problem Statement: For every category and sub-category, return the list-price value of order lines and the sub-category's percentage of its category's list-price value. Products whose `CategoryID` has no row in `Category` are grouped under `Unmapped`. Sort by category name, then share, highest first.

Expected Output:

- `CategoryName`, or `Unmapped`
- `Sub_Category`, or `Unknown`
- `list_price_value`: `SUM(Quantity * Sale_Price)` for the sub-category
- `pct_of_category`: share of the category's list-price value, 2 decimals

Tables Required: `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: grouping at a fine grain, then computing a coarser total with a window function over the grouped rows, PARTITION BY for per-group totals, contribution percentages.

Step 1 — Understand the Problem: Each output row is a sub-category, but its denominator is a category total. Those two numbers live at different grains. One option is to compute category totals separately and join them back. A window function does it without a second pass: once the sub-category rows exist, `SUM(value) OVER (PARTITION BY category)` adds up the rows of the same category and writes that total next to each of them, without collapsing anything.

Step 2 — Identify the Tables and Relationships: `OrderDetails` gives `Quantity` (one row per product line). `Products` gives `Sale_Price`, `Sub_Category` and `CategoryID` (`OrderDetails.ProductID` → `Products.ProductID`). `Category` gives the name, via LEFT JOIN so unmapped products stay in. The value is list-price value: `Sale_Price` is today's catalogue price, not the price paid, so this is a proxy for revenue. Lines whose product is missing have no price or sub-category and drop out at the inner join.

Step 3 — Build the Query Logic:

1. Join the three tables and group by category name and sub-category. Sum `Quantity * Sale_Price`.
2. On those grouped rows, compute `SUM(SUM(...)) OVER (PARTITION BY category name)`, the category total.
3. Divide the sub-category value by that total.

Step 4 — Final SQL Solution:

```sql
SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
       COALESCE(p.Sub_Category, 'Unknown')    AS Sub_Category,
       SUM(od.Quantity * p.Sale_Price)        AS list_price_value,
       ROUND(100.0 * SUM(od.Quantity * p.Sale_Price)
             / NULLIF(SUM(SUM(od.Quantity * p.Sale_Price))
                      OVER (PARTITION BY COALESCE(cat.CategoryName, 'Unmapped')), 0), 2)
                                              AS pct_of_category
FROM OrderDetails AS od
JOIN Products      AS p   ON p.ProductID    = od.ProductID
LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
GROUP BY COALESCE(cat.CategoryName, 'Unmapped'), COALESCE(p.Sub_Category, 'Unknown')
ORDER BY CategoryName, pct_of_category DESC NULLS LAST, Sub_Category;
```

Step 5 — Line-by-Line Explanation:

- `GROUP BY` category name and sub-category produces one row per sub-category, each with its own `SUM(od.Quantity * p.Sale_Price)`.
- `SUM(SUM(...)) OVER (PARTITION BY ...)` reads from the inside out. The inner `SUM` is the ordinary aggregate, one value per grouped row. The outer `SUM ... OVER` is a window function that runs after grouping, adds up the inner sums of all rows in the same category, and attaches the result to each of those rows. Nested like this it looks odd, but it is standard SQL and the clearest way to get a group total next to its members.
- `PARTITION BY COALESCE(cat.CategoryName, 'Unmapped')` must use the same expression as the GROUP BY, so all `Unmapped` rows share one total.
- `100.0 * ... / NULLIF(..., 0)` gives a decimal percentage and returns NULL instead of an error if a category's total is 0.
- Grouping by category name, not `CategoryID`, is safe only if names are unique; if two IDs could share a name, group and partition by the ID as well.

Step 6 — Sample Output:

categoryname	sub_category	list_price_value	pct_of_category
Baby Care	Diapers & Wipes	936905.00	51.16
Baby Care	Baby Bath & Hygiene	503896.00	27.52

Baby Care	Feeding & Nursing	188087.00	10.27
Baby Care	Baby Food & Formula	131599.00	7.19
Baby Care	Baby Accessories	43257.00	2.36
Baby Care	Mothers & Maternity	27599.00	1.51
Bakery, Cakes & Dairy	Dairy	1239169.00	59.64
Bakery, Cakes & Dairy	Breads & Buns	289559.00	13.94

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 107 rows.

Step 7 — Why This Solution Works: The query runs in a fixed order: join, group, then window. Grouping fixes the grain at sub-category. The window then works over those grouped rows only, so its sum is the category total built from the same numbers shown in `list_price_value`. Every sub-category belongs to exactly one partition, which makes each category's shares add up to 100 apart from rounding. Check: `SUM(pct_of_category)` grouped by category should be 100 within a cent or two.

Step 8 — Common Mistakes:

- Dividing by the store total. `OVER ()` with no partition sums every row, so the shares add up to 100 across the whole store, not within each category:

```
SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
       p.Sub_Category,
       ROUND(100.0 * SUM(od.Quantity * p.Sale_Price)
             / SUM(SUM(od.Quantity * p.Sale_Price)) OVER (), 2) AS pct_of_store
FROM OrderDetails AS od
JOIN Products      AS p   ON p.ProductID    = od.ProductID
LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
GROUP BY COALESCE(cat.CategoryName, 'Unmapped'), p.Sub_Category
ORDER BY CategoryName, p.Sub_Category;
```

- Writing the window over a raw column, `SUM(od.Quantity * p.Sale_Price) OVER (PARTITION BY ...)` without the inner SUM, in a grouped query. PostgreSQL rejects it, because after GROUP BY the raw line columns no longer exist.
- Using an INNER JOIN to `Category`. Unmapped products disappear, and the sub-categories that remain look complete when they are not.

Step 9 — Interview Follow-Up: The category managers also want each sub-category's rank within its category and a running share from largest to smallest. Which two window functions would you add, and what ORDER BY goes inside each?

Step 10 — Optimization or Alternative: The same result with a CTE and a join reads more plainly to people new to window functions:

```
WITH sub AS (
    SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
           COALESCE(p.Sub_Category, 'Unknown')    AS Sub_Category,
           SUM(od.Quantity * p.Sale_Price)        AS list_price_value
    FROM OrderDetails AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    GROUP BY 1, 2
),
cat_total AS (
    SELECT s.CategoryName, SUM(s.list_price_value) AS category_value
    FROM sub AS s
    GROUP BY s.CategoryName
)
SELECT s.CategoryName, s.Sub_Category, s.list_price_value,
       ROUND(100.0 * s.list_price_value / NULLIF(ct.category_value, 0), 2) AS pct_of_category
FROM sub AS s
JOIN cat_total AS ct ON ct.CategoryName = s.CategoryName
ORDER BY s.CategoryName, pct_of_category DESC NULLS LAST, s.Sub_Category;
```

It reads the grouped result twice. The window version does it in one pass and is usually what an interviewer hopes to see.

Hard

Questions 61 to 85

Question 61: Monthly cohort retention

Difficulty: Hard

Primary Topic: Cohort analysis

Combined Topics: Multiple CTEs, Conditional aggregation with FILTER, Anchored partial-period handling

Business Scenario: The growth team changed its first-order coupon last year and wants to know whether the customers it brought in actually came back. They group customers by the month of their first order (the cohort) and ask a simple question per cohort: what share of these people placed another order one, two, three and six months later? The table goes into the monthly growth review, so a cohort that has not yet reached month six must show a blank, not a misleading zero.

Problem Statement: For every cohort month, return the cohort size and the percentage of the cohort who placed at least one order in calendar month 1, 2, 3 and 6 after the cohort month. Month 0 is the cohort month itself. If the data does not fully cover a cohort's month N yet, return NULL for that column. Anchor "how far the data goes" to `MAX(OrderDate)`.

Expected Output:

- `cohort_month`: first day of the month of the customer's first order
- `cohort_size`: customers whose first order fell in that month
- `m1_pct`: percentage of the cohort who ordered in the month after the cohort month
- `m2_pct`: same, two months after
- `m3_pct`: same, three months after
- `m6_pct`: same, six months after

Tables Required: `Orders`

SQL Concepts Tested: customer-month deduplication, MIN per customer for the cohort, month offset arithmetic, conditional aggregation with `FILTER`, anchoring to the data's last date, NULL for unobserved periods.

Step 1 — Understand the Problem: Two definitions drive everything. A customer belongs to exactly one cohort: the month of their first order. A customer is "retained in month N" if they placed any order in the calendar month that is N months after the cohort month. Retention here is not cumulative: a customer can be absent in month 2 and present in month 3. The unit being counted is a customer, never an order, so a customer with four orders in month 1 still counts once. Cancellations and returns are not modelled, so every `Orders` row counts as an order.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` and `OrderDate`. We do not join `Customers`, so an order whose `CustomerID` has no customer record still forms its own one-person cohort. That is usually what growth wants (they bought, so they count). If the business only wants registered customers, add an inner join to `Customers` and say so in the report.

Step 3 — Build the Query Logic:

1. Reduce `Orders` to distinct (customer, month) pairs. This removes repeat orders within a month before anything is counted.
2. Take each customer's earliest month as the cohort month.
3. Join the pairs back to the cohort and compute `month_number` as the whole-month gap between the activity month and the cohort month.
4. Count customers per cohort with `FILTER (WHERE month_number = N)`. Because step 1 already made the rows distinct, `COUNT(*)` counts customers.
5. Work out the first month the data does not fully cover, from `MAX(OrderDate)`. Show a percentage only if month N lies before it.

Step 4 — Final SQL Solution:

```sql
WITH customer_month AS (
    SELECT DISTINCT o.CustomerID,
           DATE_TRUNC('month', o.OrderDate)::date AS activity_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
cohort AS (
    SELECT cm.CustomerID,
           MIN(cm.activity_month) AS cohort_month
    FROM customer_month AS cm
    GROUP BY cm.CustomerID
),
activity AS (
    SELECT co.cohort_month,
           cm.CustomerID,
           (EXTRACT(YEAR  FROM cm.activity_month) - EXTRACT(YEAR  FROM co.cohort_month)) * 12
         + (EXTRACT(MONTH FROM cm.activity_month) - EXTRACT(MONTH FROM co.cohort_month)) AS month_number
    FROM customer_month AS cm
    JOIN cohort         AS co ON co.CustomerID = cm.CustomerID
),
cohort_counts AS (
    SELECT a.cohort_month,
           COUNT(*) FILTER (WHERE a.month_number = 0) AS cohort_size,
           COUNT(*) FILTER (WHERE a.month_number = 1) AS m1_customers,
           COUNT(*) FILTER (WHERE a.month_number = 2) AS m2_customers,
           COUNT(*) FILTER (WHERE a.month_number = 3) AS m3_customers,
           COUNT(*) FILTER (WHERE a.month_number = 6) AS m6_customers
    FROM activity AS a
    GROUP BY a.cohort_month
),
anchor AS (
    -- first month the data does not fully cover
    SELECT DATE_TRUNC('month', MAX(o.OrderDate)::date + 1)::date AS first_open_month
    FROM Orders AS o
)
SELECT cc.cohort_month,
       cc.cohort_size,
       CASE WHEN cc.cohort_month + INTERVAL '1 month'  < an.first_open_month
            THEN ROUND(100.0 * cc.m1_customers / NULLIF(cc.cohort_size, 0), 2) END AS m1_pct,
       CASE WHEN cc.cohort_month + INTERVAL '2 months' < an.first_open_month
            THEN ROUND(100.0 * cc.m2_customers / NULLIF(cc.cohort_size, 0), 2) END AS m2_pct,
       CASE WHEN cc.cohort_month + INTERVAL '3 months' < an.first_open_month
            THEN ROUND(100.0 * cc.m3_customers / NULLIF(cc.cohort_size, 0), 2) END AS m3_pct,
       CASE WHEN cc.cohort_month + INTERVAL '6 months' < an.first_open_month
            THEN ROUND(100.0 * cc.m6_customers / NULLIF(cc.cohort_size, 0), 2) END AS m6_pct
FROM cohort_counts AS cc
CROSS JOIN anchor  AS an
ORDER BY cc.cohort_month;
```

Step 5 — Line-by-Line Explanation:

- `customer_month` uses `SELECT DISTINCT` on (customer, truncated month). After this CTE, one row means "this customer was active in this month", which is exactly the fact retention needs.
- `cohort` takes `MIN(activity_month)`. The first active month and the month of the first order are the same thing, so there is no need to go back to raw dates.
- `month_number` uses year and month parts instead of subtracting dates. `(activity - cohort) / 30` drifts across 31-day months and February, and it puts 31 January and 1 February 0 months apart.
- `COUNT(*) FILTER (WHERE month_number = 0)` is the cohort size. Every customer has exactly one month-0 row, so this equals the number of customers in the cohort.
- `anchor` takes the day after the latest order and truncates to its month. A latest order on 31 December makes December complete; one on 18 December leaves December open. Anchoring to the data instead of `CURRENT_DATE` gives the same answer on an old snapshot.
- Each `CASE` returns NULL when month N has not been fully observed. Without it, a young cohort shows 0% and reads as "nobody came back".

Step 6 — Sample Output:

cohort_month	cohort_size	m1_pct	m2_pct	m3_pct	m6_pct
2023-01-01	133	52.63	55.64	54.14	50.38
2023-02-01	29	58.62	55.17	48.28	34.48
2023-03-01	41	56.10	58.54	56.10	31.71
2023-04-01	45	57.78	55.56	55.56	44.44
2023-05-01	41	58.54	53.66	48.78	43.90
2023-06-01	40	62.50	52.50	57.50	42.50
2023-07-01	47	63.83	53.19	53.19	44.68
2023-08-01	46	60.87	67.39	67.39	41.30

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 36 rows.

Step 7 — Why This Solution Works: Each count is taken at the customer-month grain, so repeat orders inside one month cannot inflate a percentage. The cohort size is computed from the same rows as the retention counts, so numerator and denominator always describe the same people. The NULL rule separates "measured and zero" from "not measured yet", which is the distinction the growth review cares about. Small cohorts swing hard (one customer out of two is 50%), so put `cohort_size` next to every percentage and do not average cohort percentages without weighting by size.

Edge cases: one-order customers count in the cohort size and in no retention column; NULL `CustomerID` or `OrderDate` rows drop out at the first step; a month with no first orders has no row; a partial latest month is never treated as complete. Not handled: a person who re-registers under a second `CustomerID` lands in a new cohort. Deduplicate accounts first (Q71) if that matters.

Step 8 — Common Mistakes:

- Counting orders instead of customers. This version reports a cohort as more than 100% retained whenever retained customers order several times in a month:

```
WITH first_order AS (
    SELECT o.CustomerID, DATE_TRUNC('month', MIN(o.OrderDate))::date AS cohort_month
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT fo.cohort_month,
       COUNT(DISTINCT fo.CustomerID) AS cohort_size,
       COUNT(*) FILTER (WHERE DATE_TRUNC('month', o.OrderDate)::date
                              = fo.cohort_month + INTERVAL '1 month') AS m1_orders_not_customers
FROM first_order AS fo
JOIN Orders AS o ON o.CustomerID = fo.CustomerID
GROUP BY fo.cohort_month
ORDER BY fo.cohort_month;
```

- Using `DateEntered` as the cohort. That measures sign-up cohorts, a different question, and customers who never ordered drag every number down.
- Returning 0 for months the data has not reached, so the latest cohorts look like a retention collapse.
- Computing the month gap as `(date2 - date1) / 30`.

Step 9 — Interview Follow-Up: The team now wants "rolling" retention: the share of the cohort who ordered in month N or any later month. Which line of the solution changes, and how does the NULL rule change?

Step 10 — Optimization or Alternative: Dashboards usually want the long format, one row per cohort and month number, so a heatmap can show every offset without hard-coded columns. `generate_series` builds the offsets and a LEFT JOIN keeps zeros as zeros:

```
WITH customer_month AS (
    SELECT DISTINCT o.CustomerID, DATE_TRUNC('month', o.OrderDate)::date AS activity_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL AND o.OrderDate IS NOT NULL
),
cohort AS (
    SELECT cm.CustomerID, MIN(cm.activity_month) AS cohort_month
    FROM customer_month AS cm
    GROUP BY cm.CustomerID
),
cohort_size AS (
    SELECT co.cohort_month, COUNT(*) AS customers
    FROM cohort AS co
    GROUP BY co.cohort_month
),
anchor AS (
    SELECT DATE_TRUNC('month', MAX(o.OrderDate)::date + 1)::date AS first_open_month
    FROM Orders AS o
),
grid AS (
    SELECT cs.cohort_month, cs.customers, n.month_number
    FROM cohort_size AS cs
    CROSS JOIN anchor AS an
    CROSS JOIN generate_series(0, 12) AS n(month_number)
    WHERE cs.cohort_month + n.month_number * INTERVAL '1 month' < an.first_open_month
),
retained AS (
    SELECT co.cohort_month, cm.activity_month, COUNT(*) AS customers
    FROM customer_month AS cm
    JOIN cohort AS co ON co.CustomerID = cm.CustomerID
    GROUP BY co.cohort_month, cm.activity_month
)
SELECT g.cohort_month,
       g.month_number,
       g.customers                                  AS cohort_size,
       COALESCE(r.customers, 0)                     AS active_customers,
       ROUND(100.0 * COALESCE(r.customers, 0) / NULLIF(g.customers, 0), 2) AS retention_pct
FROM grid AS g
LEFT JOIN retained AS r
       ON r.cohort_month   = g.cohort_month
      AND r.activity_month = g.cohort_month + g.month_number * INTERVAL '1 month'
ORDER BY g.cohort_month, g.month_number;
```

The `WHERE` inside `grid` drops unobserved cells instead of nulling them: the same rule in long form. On a large `Orders` table, an index on `(CustomerID, OrderDate)` helps the customer-month step, where most of the work happens.

Question 62: Which products make up 80% of list-price value (Pareto)

Difficulty: Hard

Primary Topic: Pareto analysis

Combined Topics: Running total window with an explicit ROWS frame, Contribution percentage, CTE aggregation

Business Scenario: The merchandising team is deciding which products get guaranteed shelf space and stock buffers in the new warehouse. Their rule: protect the smallest set of products that together account for 80% of sales value, and review the rest as the long tail. Sales value here means list-price value, since order lines carry no price of their own.

Problem Statement: Rank products by list-price value (`Quantity * Sale_Price`, summed over all order lines), highest first. For each product show its share of the total and the running share. Label a product `Top 80%` if it is needed to reach 80% of the total, including the product that crosses the line, and `Long tail` otherwise.

Expected Output:

- `ProductID`, `Product`
- `list_price_value`: SUM of `Quantity * Sale_Price` for the product
- `pct_of_total`: the product's share of all list-price value, 2 dp
- `cumulative_pct`: running share from the top product down to this one, 2 dp
- `pareto_group`: `Top 80%` or `Long tail`

Tables Required: `OrderDetails`, `Products`

SQL Concepts Tested: aggregation in a CTE, `SUM() OVER ()` for a grand total, running SUM with an explicit `ROWS` frame, deterministic ordering for ties, contribution percentages, CASE on a running value.

Step 1 — Understand the Problem: This is a running-total problem with one twist. The running share must increase product by product, even when two products have the same value. Sale_Price is today's catalogue price, not the price paid, so list-price value is a proxy for revenue. It is fine for ranking products against each other, but it will not tie to finance's revenue figure.

Step 2 — Identify the Tables and Relationships: `OrderDetails` holds `ProductID` and `Quantity` per line. `Products` supplies `Product` and `Sale_Price` (`OrderDetails.ProductID` → `Products.ProductID`, many lines to one product). An inner join is right here: a line whose `ProductID` has no product row has no price, so it cannot contribute value. That means those orphan lines are left out of the total, and you should report how many there are separately.

Step 3 — Build the Query Logic:

1. Aggregate lines to one row per product: `SUM(od.Quantity * p.Sale_Price)`.
2. Drop products whose value is NULL or zero (for example, every line has a NULL quantity). They cannot be ranked meaningfully.
3. Compute the grand total with `SUM(...) OVER ()`.
4. Compute the running value with `SUM(...) OVER (ORDER BY value DESC, ProductID ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.
5. A product is in the top group if the running value before it (running minus its own value) is still below 80% of the total.

Step 4 — Final SQL Solution:

```sql
WITH product_value AS (
    SELECT p.ProductID,
           p.Product,
           SUM(od.Quantity * p.Sale_Price) AS list_price_value
    FROM OrderDetails AS od
    JOIN Products     AS p ON p.ProductID = od.ProductID
    GROUP BY p.ProductID, p.Product
    HAVING SUM(od.Quantity * p.Sale_Price) > 0
),
running AS (
    SELECT pv.ProductID,
           pv.Product,
           pv.list_price_value,
           SUM(pv.list_price_value) OVER () AS total_value,
           SUM(pv.list_price_value) OVER (
               ORDER BY pv.list_price_value DESC, pv.ProductID
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
           )                                AS running_value
    FROM product_value AS pv
)
SELECT r.ProductID,
       r.Product,
       r.list_price_value,
       ROUND(100.0 * r.list_price_value / NULLIF(r.total_value, 0), 2) AS pct_of_total,
       ROUND(100.0 * r.running_value    / NULLIF(r.total_value, 0), 2) AS cumulative_pct,
       CASE WHEN r.running_value - r.list_price_value < 0.8 * r.total_value
            THEN 'Top 80%'
            ELSE 'Long tail'
       END                                                              AS pareto_group
FROM running AS r
ORDER BY r.list_price_value DESC, r.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `product_value` changes the grain from order line to product. A NULL `Quantity` makes that line's value NULL, which `SUM` skips, so the product keeps its other lines.

- `HAVING SUM(...) > 0` removes products with no measurable value. A NULL value would otherwise sort first under `DESC` in PostgreSQL and sit at the top of a Pareto chart.
- `SUM(...) OVER ()` with an empty window puts the grand total on every row, so no second pass over the table is needed.
- The running `SUM` orders by value and then by `ProductID`. The tiebreaker makes the order unique, and the explicit `ROWS` frame makes the sum add exactly one row at a time.
- `running_value - list_price_value` is the value accumulated before this product. If that is still under 80%, this product is needed to reach the line, so it belongs in the top group. Testing `running_value <= 0.8 * total` instead would exclude the product that crosses 80% and leave the protected set short of its target.
- The final `ORDER BY` repeats the window order so the report reads top-down.

Step 6 — Sample Output:

productid	product	list_price_value	pct_of_total	cumulative_pct	pareto_group
1710	Lemongrass Hexagon Bath Personal Care Gift Set	1246350.00	2.01	2.01	Top 80%
13924	Vitamin C, E & Hyaluronic Acid De Pigmentation Cream	1113885.00	1.80	3.81	Top 80%
14210	Women's Body Oil - Revitalising, Rose Geranium	480180.00	0.78	4.59	Top 80%
21584	GIR Cow Ghee/Tuppa - A2 Milk (Vedic Bilona Method)	470250.00	0.76	5.35	Top 80%
6220	Natural Nourishing Hair Conditioner	399415.00	0.65	6.00	Top 80%
5575	Health Drink - Morning Miracle	390925.00	0.63	6.63	Top 80%
13015	Intensive Care Deep Moisture Body Lotion - With Pure Oat Extract	361204.00	0.58	7.21	Top 80%
21577	Buffalo Ghee/Tuppa - Special Grade with Milk Fat	356902.00	0.58	7.79	Top 80%

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 16520 rows.

Step 7 — Why This Solution Works: The running share comes from a frame that grows by one row at a time over a unique ordering, so it rises monotonically and the boundary falls on exactly one product. The 80% test uses the value before each product, which gives the smallest set that reaches 80%. The last `cumulative_pct` is 100.00, which is a quick self-check. Never-ordered products have no lines, so they do not appear; they are long tail by definition, and Q33 lists them.

Step 8 — Common Mistakes:

- Leaving out the frame. With `ORDER BY` and no frame clause, PostgreSQL uses `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which treats rows with equal sort values as peers and gives them all the same running total. Two products tied on value jump together, so the running share skips a step and the 80% boundary can land in the wrong place. This small example shows the difference:

```
SELECT t.id,
       t.v,
       SUM(t.v) OVER (ORDER BY t.v DESC)                                        AS default_range_frame,
       SUM(t.v) OVER (ORDER BY t.v DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame
FROM (VALUES (1, 500), (2, 300), (3, 300), (4, 100)) AS t(id, v)
ORDER BY t.v DESC, t.id;
```

The two 300 rows both get 1100 under the default frame. Adding `ProductID` to the window `ORDER BY` also removes the peers, but the explicit `ROWS` frame states the intent and survives someone deleting the tiebreaker later.

- Ranking by units instead of value. A cheap fast-mover can top a units chart and still be a small share of value.
- Dividing by a total computed after a `WHERE` that removed products, so the shares no longer sum to 100.

Step 9 — Interview Follow-Up: Merchandising now wants the Pareto split within each category, with the 80% line drawn per category. What changes in the two window definitions?

Step 10 — Optimization or Alternative: The other way to read Pareto is "what share of value do the top 20% of products produce?". `NTILE(5)` splits the ranked products into five equal-count buckets, and the first bucket is the top 20%:

```
WITH product_value AS (
    SELECT p.ProductID,
           SUM(od.Quantity * p.Sale_Price) AS list_price_value
    FROM OrderDetails AS od
    JOIN Products     AS p ON p.ProductID = od.ProductID
    GROUP BY p.ProductID
    HAVING SUM(od.Quantity * p.Sale_Price) > 0
),
bucketed AS (
    SELECT pv.ProductID,
           pv.list_price_value,
           NTILE(5) OVER (ORDER BY pv.list_price_value DESC, pv.ProductID) AS quintile
    FROM product_value AS pv
)
SELECT b.quintile,
       COUNT(*)                                                    AS products,
       SUM(b.list_price_value)                                     AS list_price_value,
       ROUND(100.0 * SUM(b.list_price_value)
             / NULLIF(SUM(SUM(b.list_price_value)) OVER (), 0), 2) AS pct_of_total
FROM bucketed AS b
GROUP BY b.quintile
ORDER BY b.quintile;
```

`SUM(SUM(...)) OVER ()` is a window over the grouped result: the inner `SUM` runs per quintile, the outer one adds the quintiles up. When the product count is not divisible by five, `NTILE` makes the early buckets one product larger, so "top 20%" is approximate on small catalogues. On MySQL 8 the same window syntax works; on MySQL 5.7 there are no window functions and you would need a self-join or variables.

Question 63: RFM segmentation

Difficulty: Hard

Primary Topic: Customer segmentation

Combined Topics: NTILE scoring, Multiple CTEs, CASE segment rules, Anchored recency

Business Scenario: The CRM team runs a different email track for each customer segment: early access for the best customers, a nudge for those slipping away, and a win-back coupon for the dormant ones. They want every customer scored 1 to 5 on recency (how recently they ordered), frequency (how many orders) and monetary value (how much they spent), then placed in one named segment by fixed rules. Only registered customers can be emailed.

Problem Statement: For each registered customer who has ordered, compute recency in days since their last order, measured from the latest order date in the data, the number of orders, and the total order value. Score each metric into quintiles (5 = best). Customers with identical values must get identical scores. Assign a segment:

- `Champions`: R ≥ 4, F ≥ 4 and M ≥ 4
- `Loyal`: R ≥ 3 and F ≥ 4
- `New or promising`: R ≥ 4 and F ≤ 2
- `At risk`: R ≤ 2 and F ≥ 3
- `Hibernating`: R ≤ 2 and F ≤ 2
- `Needs attention`: everyone else

Rules apply in that order; the first match wins.

Expected Output:

- `CustomerID`
- `recency_days`: days between the customer's last order and `MAX(OrderDate)`
- `frequency`: number of orders
- `monetary`: sum of `Total_Order_Amount`
- `r_score`, `f_score`, `m_score`: 1 to 5, 5 is best
- `segment`: one of the six labels above

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: anchored date arithmetic, customer-grain aggregation, `NTILE`, making `NTILE` tie-consistent with a second window, ordered `CASE` rules, multiple CTEs.

Step 1 — Understand the Problem: RFM has three steps: measure, score, label. Measuring is plain aggregation at the customer grain. Scoring is relative: a score of 5 means "in the top fifth of this customer base", not a fixed threshold. Labelling is a business rulebook, so it belongs in one `CASE` that anyone can read and change. Recency is measured from the last date in the data, `MAX(OrderDate)`, not from today. On a snapshot that is a few months old, `CURRENT_DATE` would push every customer towards "dormant".

Step 2 — Identify the Tables and Relationships: `Orders` gives `CustomerID`, `OrderDate` and `Total_Order_Amount`. `Customers` is joined with an inner join (`Orders.CustomerID` → `Customers.CustomerID`) because the output feeds an email tool: an order whose customer record is missing cannot be contacted, so it is left out. Note that the anchor date is still taken from all orders, since it describes how fresh the data is, not who is in scope.

Step 3 — Build the Query Logic:

1. `anchor`: the latest order date.
2. `customer_stats`: one row per registered customer with recency, frequency and monetary.
3. `raw_scores`: `NTILE(5)` on each metric. Recency sorts descending so the most recent land in bucket 5.
4. `scores`: `NTILE` splits by row count, so two customers with the same frequency can land in different buckets. Fix that by giving every customer the lowest bucket any customer with the same value received: `MIN(f_raw) OVER (PARTITION BY frequency)`.
5. Apply the segment `CASE` in priority order.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS as_of_date
    FROM Orders AS o
),
customer_stats AS (
    SELECT c.CustomerID,
           a.as_of_date - MAX(o.OrderDate)::date  AS recency_days,
           COUNT(*)                               AS frequency,
           COALESCE(SUM(o.Total_Order_Amount), 0) AS monetary
    FROM Orders    AS o
    JOIN Customers AS c ON c.CustomerID = o.CustomerID
    CROSS JOIN anchor AS a
    WHERE o.OrderDate IS NOT NULL
    GROUP BY c.CustomerID, a.as_of_date
),
raw_scores AS (
    SELECT cs.CustomerID,
           cs.recency_days,
           cs.frequency,
           cs.monetary,
           NTILE(5) OVER (ORDER BY cs.recency_days DESC, cs.CustomerID) AS r_raw,
           NTILE(5) OVER (ORDER BY cs.frequency,         cs.CustomerID) AS f_raw,
           NTILE(5) OVER (ORDER BY cs.monetary,          cs.CustomerID) AS m_raw
    FROM customer_stats AS cs
),
scores AS (
    SELECT rs.CustomerID,
           rs.recency_days,
           rs.frequency,
           rs.monetary,
           MIN(rs.r_raw) OVER (PARTITION BY rs.recency_days) AS r_score,
           MIN(rs.f_raw) OVER (PARTITION BY rs.frequency)    AS f_score,
           MIN(rs.m_raw) OVER (PARTITION BY rs.monetary)     AS m_score
    FROM raw_scores AS rs
)
SELECT s.CustomerID,
       s.recency_days,
       s.frequency,
       s.monetary,
       s.r_score,
       s.f_score,
       s.m_score,
       CASE
           WHEN s.r_score >= 4 AND s.f_score >= 4 AND s.m_score >= 4 THEN 'Champions'
           WHEN s.r_score >= 3 AND s.f_score >= 4                    THEN 'Loyal'
           WHEN s.r_score >= 4 AND s.f_score <= 2                    THEN 'New or promising'
           WHEN s.r_score <= 2 AND s.f_score >= 3                    THEN 'At risk'
           WHEN s.r_score <= 2 AND s.f_score <= 2                    THEN 'Hibernating'
           ELSE 'Needs attention'
       END AS segment
FROM scores AS s
ORDER BY s.r_score + s.f_score + s.m_score DESC, s.monetary DESC, s.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `anchor` is a one-row CTE. `CROSS JOIN` puts it on every order row without changing the row count.
- `as_of_date - MAX(o.OrderDate)::date` is an integer day count. The customer who placed the latest order gets 0.
- `COALESCE(SUM(...), 0)` stops a customer whose only order has a NULL total from getting a NULL monetary value. PostgreSQL sorts NULLs last in ascending order, so that customer would land in the top monetary bucket.
- `NTILE(5) OVER (ORDER BY recency_days DESC, ...)` puts the oldest last order in bucket 1 and the most recent in bucket 5. Frequency and monetary sort ascending for the same reason.
- `MIN(r_raw) OVER (PARTITION BY recency_days)` is the tie fix. Frequency is the metric where it matters most, because many customers share a value of 1 or 2 orders. Without it, two one-order customers could get F = 1 and F = 2 purely because of their IDs.
- The `CASE` is ordered from most to least specific. `Champions` must come before `Loyal`, because a champion also meets the loyal rule.

Step 6 — Sample Output:

customerid	recency_days	frequency	monetary	r_score	f_score	m_score	segment
100092	0	182	274363.14	5	5	5	Champions
102192	4	188	248896.30	5	5	5	Champions
104801	7	192	225297.92	5	5	5	Champions
102422	1	187	202666.45	5	5	5	Champions
101294	4	138	198884.78	5	5	5	Champions
103550	0	116	176654.80	5	5	5	Champions
102931	8	102	175152.52	5	5	5	Champions
103278	1	105	163762.64	5	5	5	Champions

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 4425 rows.

Step 7 — Why This Solution Works: Measurement happens once at the customer grain, so no order is counted twice. Scores are relative and tie-consistent, so the same behaviour always gets the same score. The segment rules cover every combination: anything not matched falls into `Needs attention`, so no customer ends up unlabelled.

Edge cases to mention in an interview. Customers with one order cluster at F = 1, and after the tie fix the bottom frequency bucket can hold far more than a fifth of customers. That is correct: those customers really are identical on frequency. Registered customers with no orders are not in the output; they belong to an activation campaign, not RFM. Duplicate accounts split one person's orders across two IDs and understate both F and M (Q71 deduplicates them). With fewer than five customers, `NTILE(5)` leaves some scores unused.

Step 8 — Common Mistakes:

- Measuring recency from `CURRENT_DATE`. Every number then depends on the day the query is run.
- Skipping the tie fix. This shows how many frequency values end up split across two scores:

```sql
WITH f AS (
    SELECT o.CustomerID, COUNT(*) AS frequency
    FROM Orders AS o
    GROUP BY o.CustomerID
),
raw AS (
    SELECT f.CustomerID, f.frequency,
           NTILE(5) OVER (ORDER BY f.frequency, f.CustomerID) AS f_raw
    FROM f
)
SELECT raw.frequency,
       COUNT(*)                   AS customers,
       COUNT(DISTINCT raw.f_raw)  AS different_scores_for_same_value
FROM raw
GROUP BY raw.frequency
HAVING COUNT(DISTINCT raw.f_raw) > 1
ORDER BY raw.frequency;
```

- Using `SUM(Total_Order_Amount)` after joining to `OrderDetails`. Monetary is an order-grain number; a line join multiplies it.
- Putting `Loyal` above `Champions` in the `CASE`. Champions disappear.

Step 9 — Interview Follow-Up: Marketing says quintiles move every month, so a customer's segment changes even when their behaviour does not. How would you switch to fixed thresholds (for example, F = 5 means 20+ orders), and what do you lose?

Step 10 — Optimization or Alternative: `PERCENT_RANK` gives tie-consistent scores in a single pass, because rows with equal values always get the same rank. Map the 0-1 rank to 1-5 with `FLOOR`:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS as_of_date
    FROM Orders AS o
),
customer_stats AS (
    SELECT c.CustomerID,
           a.as_of_date - MAX(o.OrderDate)::date  AS recency_days,
           COUNT(*)                               AS frequency,
           COALESCE(SUM(o.Total_Order_Amount), 0) AS monetary
    FROM Orders    AS o
    JOIN Customers AS c ON c.CustomerID = o.CustomerID
    CROSS JOIN anchor AS a
    WHERE o.OrderDate IS NOT NULL
    GROUP BY c.CustomerID, a.as_of_date
)
SELECT cs.CustomerID,
       cs.recency_days,
       cs.frequency,
       cs.monetary,
       LEAST(5, 1 + FLOOR(5 * PERCENT_RANK() OVER (ORDER BY cs.recency_days DESC)))::int AS r_score,
       LEAST(5, 1 + FLOOR(5 * PERCENT_RANK() OVER (ORDER BY cs.frequency)))::int         AS f_score,
       LEAST(5, 1 + FLOOR(5 * PERCENT_RANK() OVER (ORDER BY cs.monetary)))::int          AS m_score
FROM customer_stats AS cs
ORDER BY cs.CustomerID;
```

`LEAST(5, ...)` handles the top row, whose `PERCENT_RANK` is exactly 1. The bucket sizes differ from `NTILE`'s, since `PERCENT_RANK` cuts on rank position, not row count. Either is defensible; say which one you use and keep it stable month to month so segment movement reflects behaviour.

Question 64: Three-month moving average of order value, with no missing months

Difficulty: Hard

Primary Topic: Moving averages

Combined Topics: generate_series month spine, LEFT JOIN to fill gaps, Explicit ROWS frame, Multiple CTEs

Business Scenario: Finance presents monthly order value to the board with a three-month moving average beside it, so one festive month or one slow month does not swing the story. Last quarter a month with no orders was simply missing from the chart, and the "three-month" average quietly averaged across four calendar months. They want a version that cannot do that.

Problem Statement: For every calendar month from the first order month to the latest order month in the data, return the total order value (0 for months with no orders) and the average of that month and the two before it. Return NULL for the moving average until three months are available.

Expected Output:

- `order_month`: first day of the month, every month in the range, no gaps
- `order_value`: sum of `Total_Order_Amount` in that month, 0 when there were no orders
- `moving_avg_3m`: average of `order_value` over this month and the previous two, 2 dp; NULL for the first two months

Tables Required: `Orders`

SQL Concepts Tested: calendar spine with `generate_series`, LEFT JOIN to keep empty periods, `COALESCE` to zero, `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`, a named `WINDOW`, suppressing incomplete windows with `COUNT(*) OVER`.

Step 1 — Understand the Problem: A window frame such as `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` counts rows, not months. If a month has no orders, `GROUP BY` produces no row for it, and "two rows back" reaches three calendar months back. The fix is to make sure every month has a row before the window runs. The months come from a generated calendar (a spine), not from the orders themselves.

The range is anchored to the data: first month = month of `MIN(OrderDate)`, last month = month of `MAX(OrderDate)`. The last month may be partial (the data may stop mid-month), which pulls its average down. The output keeps it, since finance asked for every month, but say so when you present it.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `OrderDate` and `Total_Order_Amount`. No join to lines is involved, so there is no fan-out risk. Cancellations are not modelled, so every row counts as an order.

Step 3 — Build the Query Logic:

1. `bounds`: first and last month in the data.
2. `month_spine`: `generate_series(first_month, last_month, INTERVAL '1 month')`, one row per month.
3. `monthly`: order value per month from `Orders`.
4. `filled`: spine LEFT JOIN monthly, with `COALESCE(order_value, 0)`.
5. Average over a frame of the current row and two preceding rows. Because the spine has no gaps, two rows back is exactly two months back.
6. `COUNT(*) OVER` the same frame tells you whether three months are present; return NULL when it is less than three.

Step 4 — Final SQL Solution:

```sql
WITH bounds AS (
    SELECT DATE_TRUNC('month', MIN(o.OrderDate))::date AS first_month,
           DATE_TRUNC('month', MAX(o.OrderDate))::date AS last_month
    FROM Orders AS o
),
month_spine AS (
    SELECT gs::date AS order_month
    FROM bounds AS b
    CROSS JOIN generate_series(b.first_month, b.last_month, INTERVAL '1 month') AS gs
),
monthly AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           SUM(o.Total_Order_Amount)               AS order_value
    FROM Orders AS o
    WHERE o.OrderDate IS NOT NULL
    GROUP BY DATE_TRUNC('month', o.OrderDate)::date
),
filled AS (
    SELECT ms.order_month,
           COALESCE(m.order_value, 0) AS order_value
    FROM month_spine AS ms
    LEFT JOIN monthly AS m ON m.order_month = ms.order_month
)
SELECT f.order_month,
       f.order_value,
       CASE WHEN COUNT(*) OVER w = 3
            THEN ROUND(AVG(f.order_value) OVER w, 2)
       END AS moving_avg_3m
FROM filled AS f
WINDOW w AS (ORDER BY f.order_month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY f.order_month;
```

Step 5 — Line-by-Line Explanation:

- `bounds` reads the first and last order month once. `MIN` and `MAX` ignore NULL dates.
- `generate_series(start, stop, INTERVAL '1 month')` returns timestamps; `::date` turns them back into dates so the join to `monthly` compares like with like.
- `monthly` groups by the truncated month. The `WHERE` excludes NULL dates, which would otherwise form a NULL month that matches nothing.
- `filled` is where the gap disappears. A month with no orders gets a spine row and `order_value = 0`.
- `WINDOW w AS (...)` defines the frame once and both `COUNT(*)` and `AVG` use it, so they cannot drift apart.
- `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` is written out on purpose. The default frame with an `ORDER BY` is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which is a running average from the first month, not a three-month one.
- `COUNT(*) OVER w = 3` is false for the first two months, because their frame holds fewer rows. Without this check, the first month's "three-month average" is just itself.

Step 6 — Sample Output:

order_month	order_value	moving_avg_3m
2023-01-01	447044.37	NULL
2023-02-01	217410.49	NULL
2023-03-01	354300.85	339585.24
2023-04-01	438229.46	336646.93
2023-05-01	488672.06	427067.46
2023-06-01	447517.14	458139.55
2023-07-01	616163.08	517450.76
2023-08-01	686448.76	583376.33

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 36 rows.

Step 7 — Why This Solution Works: The spine guarantees one row per calendar month, so a row-based frame of three rows is always three calendar months. Zero-order months contribute a real zero, which is what finance means by "nothing sold that month". Incomplete frames are shown as NULL instead of an average of one or two months.

Edge cases. A month with orders whose totals are all NULL gets a NULL from `SUM`, which `COALESCE` turns into 0; if that is a data error, flag it separately rather than averaging it as zero. The partial latest month is included and labelled in the presentation. If there are fewer than three months of data, every moving average is NULL, which is the honest answer.

Step 8 — Common Mistakes:

- Skipping the spine. This demo keeps the first five months of data and deletes the third to simulate a month with no orders. With `ROWS`, the fourth month's "three-month" average reaches back to month one. With `RANGE ... INTERVAL '2 months' PRECEDING`, it averages only two months, because a missing month is absent, not zero:

```
WITH first_month AS (
    SELECT DATE_TRUNC('month', MIN(o.OrderDate)) AS m0
    FROM Orders AS o
),
monthly AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           SUM(o.Total_Order_Amount)               AS order_value
    FROM Orders AS o
    CROSS JOIN first_month AS fm
    WHERE o.OrderDate < fm.m0 + INTERVAL '5 months'
      AND DATE_TRUNC('month', o.OrderDate) <> fm.m0 + INTERVAL '2 months'
    GROUP BY DATE_TRUNC('month', o.OrderDate)::date
)
SELECT m.order_month,
       m.order_value,
       ROUND(AVG(m.order_value) OVER (ORDER BY m.order_month
             ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2)                    AS rows_avg_wrong,
       ROUND(AVG(m.order_value) OVER (ORDER BY m.order_month
             RANGE BETWEEN INTERVAL '2 months' PRECEDING AND CURRENT ROW), 2) AS range_avg_no_zero
FROM monthly AS m
ORDER BY m.order_month;
```

- Leaving out the frame, so `AVG(...) OVER (ORDER BY month)` becomes a cumulative average.
- Using `CURRENT_DATE` for the end of the spine, which adds trailing zero months the data never covered.
- Averaging daily rows with a 90-row frame and calling it three months. Months have 28 to 31 days.

Step 9 — Interview Follow-Up: Finance also wants the same three months a year earlier next to each row. How would you add a `same_period_last_year` column without a self join?

Step 10 — Optimization or Alternative: On engines without `generate_series` (MySQL 8, SQL Server), a recursive CTE builds the same spine. PostgreSQL needs the `RECURSIVE` keyword:

```sql
WITH RECURSIVE bounds AS (
    SELECT DATE_TRUNC('month', MIN(o.OrderDate))::date AS first_month,
           DATE_TRUNC('month', MAX(o.OrderDate))::date AS last_month
    FROM Orders AS o
),
month_spine (order_month) AS (
    SELECT b.first_month
    FROM bounds AS b
    UNION ALL
    SELECT (ms.order_month + INTERVAL '1 month')::date
    FROM month_spine AS ms
    CROSS JOIN bounds AS b
    WHERE ms.order_month < b.last_month
),
monthly AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           SUM(o.Total_Order_Amount)              AS order_value
    FROM Orders AS o
    WHERE o.OrderDate IS NOT NULL
    GROUP BY DATE_TRUNC('month', o.OrderDate)::date
)
SELECT ms.order_month,
       COALESCE(m.order_value, 0) AS order_value,
       CASE WHEN COUNT(*) OVER w = 3
            THEN ROUND(AVG(COALESCE(m.order_value, 0)) OVER w, 2)
       END AS moving_avg_3m
FROM month_spine AS ms
LEFT JOIN monthly AS m ON m.order_month = ms.order_month
WINDOW w AS (ORDER BY ms.order_month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY ms.order_month;
```

The `::date` cast in the recursive member matters: date plus interval is a timestamp, and the recursive member must return the same type as the anchor member. In production, a permanent calendar table replaces both versions and also carries fiscal periods and holidays.

Question 65: Longest run of consecutive months with an order

Difficulty: Hard

Primary Topic: Gaps and islands

Combined Topics: DISTINCT customer-months, ROW_NUMBER difference, Top N with ties

Business Scenario: The retention team is launching a "streak" badge in the app that rewards customers for ordering every month. Before deciding the badge tiers, they want to see the longest unbroken monthly streak each customer has ever had, and who the top streakers are. A streak is broken by any calendar month with no order.

Problem Statement: For each customer, find the longest run of consecutive calendar months in which they placed at least one order. If a customer has two runs of the same length, report the more recent one. Return the ten customers with the longest streaks, keeping everyone tied with tenth place.

Expected Output:

- `CustomerID`
- `streak_start_month`: first month of the customer's longest streak
- `streak_end_month`: last month of that streak
- `streak_months`: number of months in it

Tables Required: `Orders`

SQL Concepts Tested: reducing to one row per customer-month, the row-number-difference trick for islands, grouping by an island key, picking one island per customer, `RANK` for top N with ties.

Step 1 — Understand the Problem: This is a gaps-and-islands problem. Each customer's active months form islands of consecutive months separated by gaps. For example, a customer active in Jan, Feb, Mar, May, Jun has two islands: Jan to Mar (3 months) and May to Jun (2 months). Several orders in one month still make one active month, so the first job is to deduplicate to customer-month. Cancellations are not modelled, so every order counts as activity.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` and `OrderDate`. No join to `Customers` is made, so an order with an unknown `CustomerID` still forms its own streak. Rows with a NULL customer or date cannot belong to a streak and are excluded.

Step 3 — Build the Query Logic:

1. Reduce `Orders` to distinct (customer, month) pairs.
2. Convert each month to a single integer, `year * 12 + month`, so consecutive months differ by exactly 1 even across a year boundary.
3. Number each customer's months 1, 2, 3... in order with `ROW_NUMBER`.
4. Subtract: `month_index - rn`. Inside an island both values rise by 1 together, so the difference stays constant. After a gap, the month index jumps more than the row number, and the difference changes. That difference is the island key.
5. Group by customer and island key to get each streak's start, end and length.
6. Keep each customer's longest streak (most recent on a tie), then rank customers and keep rank ≤ 10.

Step 4 — Final SQL Solution:

```sql
WITH customer_months AS (
    SELECT DISTINCT o.CustomerID,
           DATE_TRUNC('month', o.OrderDate)::date AS order_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
numbered AS (
    SELECT cm.CustomerID,
           cm.order_month,
           (EXTRACT(YEAR FROM cm.order_month) * 12 + EXTRACT(MONTH FROM cm.order_month))::int AS month_index,
           ROW_NUMBER() OVER (PARTITION BY cm.CustomerID ORDER BY cm.order_month)             AS rn
    FROM customer_months AS cm
),
streaks AS (
    SELECT n.CustomerID,
           MIN(n.order_month) AS streak_start_month,
           MAX(n.order_month) AS streak_end_month,
           COUNT(*)           AS streak_months
    FROM numbered AS n
    GROUP BY n.CustomerID, n.month_index - n.rn
),
longest AS (
    SELECT s.CustomerID,
           s.streak_start_month,
           s.streak_end_month,
           s.streak_months,
           ROW_NUMBER() OVER (PARTITION BY s.CustomerID
                              ORDER BY s.streak_months DESC, s.streak_start_month DESC) AS pick
    FROM streaks AS s
),
ranked AS (
    SELECT l.CustomerID,
           l.streak_start_month,
           l.streak_end_month,
           l.streak_months,
           RANK() OVER (ORDER BY l.streak_months DESC) AS streak_rank
    FROM longest AS l
    WHERE l.pick = 1
)
SELECT r.CustomerID,
       r.streak_start_month,
       r.streak_end_month,
       r.streak_months
FROM ranked AS r
WHERE r.streak_rank <= 10
ORDER BY r.streak_months DESC, r.streak_end_month DESC, r.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `SELECT DISTINCT ... DATE_TRUNC('month', ...)` turns orders into active months. Without it, two orders in March would give March two row numbers, and the island key would shift in the middle of a streak.
- `month_index` turns 2023-12 into 24288 and 2024-01 into 24289. Subtracting dates would give 31 or 30 or 28 days, not 1.
- `ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY order_month)` numbers active months per customer. Months are already distinct per customer, so there are no ties to break.
- `GROUP BY n.CustomerID, n.month_index - n.rn` collapses each island into one row. The key itself is not meaningful, so it is not selected.
- `longest` picks one island per customer with `ROW_NUMBER`, ordering by length and then by start month descending, which implements "most recent on a tie" and makes the pick deterministic.
- `RANK() OVER (ORDER BY streak_months DESC)` gives tied customers the same rank, so `streak_rank <= 10` keeps everyone tied at tenth place. `LIMIT 10` would cut the tie arbitrarily.

Step 6 — Sample Output:

customerid	streak_start_month	streak_end_month	streak_months
102192	2023-01-01	2025-12-01	36
103441	2023-01-01	2025-12-01	36
104856	2023-01-01	2025-12-01	36
101259	2023-03-01	2025-12-01	34
102422	2023-04-01	2025-12-01	33
103550	2023-04-01	2025-12-01	33
103381	2023-03-01	2025-09-01	31
101237	2023-07-01	2025-12-01	30

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 10 rows.

Step 7 — Why This Solution Works: The only property the trick needs is that both sequences step by 1 inside an island. Deduplicating to customer-months guarantees that for the row number; the integer month index guarantees it for the calendar. Each island therefore maps to exactly one key, and grouping by it returns every streak.

Edge cases. A customer with a single order has one island of length 1. A streak whose `streak_end_month` equals the data's last month may still be running: the badge team should treat it as "at least N", and the last month may itself be partial. The output can hold more than ten rows when there is a tie at tenth place, which is intended; say so to whoever consumes it. A customer with two accounts (see Q71) has the streak split between them.

Step 8 — Common Mistakes:

- Skipping the customer-month deduplication. The row number then rises faster than the month index inside an active month, and real streaks break apart. This shows the wrong lengths:

```
WITH numbered AS (
    SELECT o.CustomerID,
           DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           (EXTRACT(YEAR FROM o.OrderDate) * 12 + EXTRACT(MONTH FROM o.OrderDate))::int
             - ROW_NUMBER() OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID) AS island_key
    FROM Orders AS o
)
SELECT n.CustomerID,
       COUNT(DISTINCT n.order_month) AS streak_months_wrong
FROM numbered AS n
GROUP BY n.CustomerID, n.island_key
ORDER BY streak_months_wrong DESC, n.CustomerID
LIMIT 10;
```

- Using `OrderDate - ROW_NUMBER()` days on monthly data. That finds runs of consecutive days, not months.
- Using `DENSE_RANK` in place of `ROW_NUMBER` inside `numbered`. On distinct months it gives the same numbers, but on undeduplicated rows it hides the problem instead of fixing it.
- `LIMIT 10` without a tie rule.

Step 9 — Interview Follow-Up: The badge team wants each customer's current streak: the run that ends in the latest month of the data, or 0 if they did not order that month. What do you change?

Step 10 — Optimization or Alternative: The other standard pattern flags the start of each island with `LAG` and turns the flags into island IDs with a running `SUM`. It is longer, but the logic reads directly as "a new streak starts when last month was not active", and it adapts to rules such as "a one-month gap is allowed":

```
WITH customer_months AS (
    SELECT DISTINCT o.CustomerID,
           DATE_TRUNC('month', o.OrderDate)::date AS order_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
flagged AS (
    SELECT cm.CustomerID,
           cm.order_month,
           CASE WHEN LAG(cm.order_month) OVER (PARTITION BY cm.CustomerID ORDER BY cm.order_month)
                     = cm.order_month - INTERVAL '1 month'
                THEN 0 ELSE 1
           END AS starts_new_streak
    FROM customer_months AS cm
),
labelled AS (
    SELECT f.CustomerID,
           f.order_month,
           SUM(f.starts_new_streak) OVER (PARTITION BY f.CustomerID ORDER BY f.order_month
                                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS streak_id
    FROM flagged AS f
)
SELECT l.CustomerID,
       l.streak_id,
       MIN(l.order_month) AS streak_start_month,
       MAX(l.order_month) AS streak_end_month,
       COUNT(*)           AS streak_months
FROM labelled AS l
GROUP BY l.CustomerID, l.streak_id
ORDER BY streak_months DESC, l.CustomerID, l.streak_id
LIMIT 10;
```

The first month of every customer has a NULL `LAG`, so the `CASE` falls to 1 and opens streak 1. To allow one skipped month, compare with `order_month - INTERVAL '2 months'` as well. Both patterns sort each customer's months once; an index on `(CustomerID, OrderDate)` helps the distinct step on a large table.

Question 66: How fast do customers come back for a second order?

Difficulty: Hard

Primary Topic: Repeat purchase behaviour

Combined Topics: LEAD and ROW_NUMBER, PERCENTILE_CONT, Multiple CTEs

Business Scenario: The retention team is deciding when to send the "come back" reminder after a customer's first purchase. Send it too early and it annoys people who were going to return anyway; too late and the customer has gone elsewhere. They want, for each year of first purchase, how many customers ever placed a second order and how long the typical customer took to do it, using the median and the 75th percentile because a few very slow returners would drag a mean.

Problem Statement: Group customers by the calendar year of their first order. For each year, return the number of customers, how many placed a second order on a later day, the percentage that did, and the median and 75th-percentile number of days from the first order to the second. A second order placed on the same day as the first is treated as part of the first visit.

Expected Output:

- `first_order_year`: year of the customer's first order
- `customers`: customers whose first order fell in that year
- `customers_with_second_order`: those with an order on a later day
- `pct_repeating`: percentage of `customers`, 2 dp
- `median_days_to_second`: median days from first to second order, among those who came back
- `p75_days_to_second`: 75th percentile of the same

Tables Required: `Orders`

SQL Concepts Tested: `ROW_NUMBER` to find the first order, `LEAD` to fetch the next one, `COUNT(column)` skipping NULLs, `PERCENTILE_CONT ... WITHIN GROUP`, same-day deduplication.

Step 1 — Understand the Problem: For each customer we need two dates: the first order and the next order after it. `LEAD` reads the next row in a window, so with each customer's orders sorted by date, `LEAD(order_date)` on the first row is the second order date. Customers who never came back get NULL there, and NULLs are what separate "repeating" from "one-time".

Two definitions matter. First, "second order" means the next order on a later calendar day. Two orders on the first day are often one shopping session split in two (or a duplicate submission), and counting them would give a large group of 0-day returners that pulls the median down. Second, the percentiles are computed only over customers who returned; the one-time customers are reported in `pct_repeating`, not hidden as zeros or infinities.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` and `OrderDate`. No join is made, so orders with a `CustomerID` missing from `Customers` still count as a customer. Cancellations are not modelled, so every row is a placed order.

Step 3 — Build the Query Logic:

1. Reduce `Orders` to distinct (customer, order day) pairs. This folds same-day orders together.
2. For each customer, number the days with `ROW_NUMBER` and fetch the next day with `LEAD`.
3. Keep only the first day per customer. `days_to_second` is next day minus first day, NULL if there is none.
4. Group by the year of the first order. `COUNT(*)` counts customers; `COUNT(second_order_date)` counts only those with a second order.
5. `PERCENTILE_CONT` over `days_to_second` gives the median and p75. Ordered-set aggregates ignore NULL inputs, so one-time customers are left out automatically.

Step 4 — Final SQL Solution:

```sql
WITH order_days AS (
    SELECT DISTINCT o.CustomerID,
           o.OrderDate::date AS order_date
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
sequenced AS (
    SELECT od.CustomerID,
           od.order_date,
           ROW_NUMBER() OVER (PARTITION BY od.CustomerID ORDER BY od.order_date)        AS day_seq,
           LEAD(od.order_date) OVER (PARTITION BY od.CustomerID ORDER BY od.order_date) AS next_order_date
    FROM order_days AS od
),
first_orders AS (
    SELECT s.CustomerID,
           s.order_date                    AS first_order_date,
           s.next_order_date               AS second_order_date,
           s.next_order_date - s.order_date AS days_to_second
    FROM sequenced AS s
    WHERE s.day_seq = 1
)
SELECT EXTRACT(YEAR FROM fo.first_order_date)::int                      AS first_order_year,
       COUNT(*)                                                         AS customers,
       COUNT(fo.second_order_date)                                      AS customers_with_second_order,
       ROUND(100.0 * COUNT(fo.second_order_date) / NULLIF(COUNT(*), 0), 2)  AS pct_repeating,
       ROUND((PERCENTILE_CONT(0.5)  WITHIN GROUP (ORDER BY fo.days_to_second))::numeric, 1) AS median_days_to_second,
       ROUND((PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY fo.days_to_second))::numeric, 1) AS p75_days_to_second
FROM first_orders AS fo
GROUP BY EXTRACT(YEAR FROM fo.first_order_date)
ORDER BY first_order_year;
```

Step 5 — Line-by-Line Explanation:

- `SELECT DISTINCT CustomerID, OrderDate::date` is the same-day rule. After it, every row is a distinct shopping day.
- `ROW_NUMBER()` and `LEAD()` share the same partition and order. Day values are distinct per customer, so there are no ties to break.
- `WHERE s.day_seq = 1` keeps one row per customer. Filtering must happen after the window runs, which is why it sits in the next CTE and not in `sequenced`.
- `next_order_date - order_date` is an integer number of days for `DATE` values. It is NULL when `LEAD` found nothing.
- `COUNT(fo.second_order_date)` counts non-NULL values only, so it counts returners. `COUNT(*)` counts everybody.
- `PERCENTILE_CONT(0.5)` interpolates between the two middle values when the count is even, so the median can be a half day. It returns

`double precision` in PostgreSQL; the cast to `numeric` is needed before `ROUND(x, 1)`. `PERCENTILE_DISC` would return an actual observed value instead.

Step 6 — Sample Output:

first_order_year	customers	customers_with_second_order	pct_repeating	median_days_to_second	p75_days_to_second
2023	673	616	91.53	17.0	40.0
2024	1194	1101	92.21	19.0	43.0
2025	2560	2006	78.36	15.0	34.0

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 3 rows returned.

Step 7 — Why This Solution Works: Each customer contributes exactly one row to the final aggregation, so the counts are customer counts. One-time customers are kept in the denominator for `pct_repeating` and excluded from the percentiles, so neither number is distorted by the other.

The edge case to raise unprompted is right-censoring. A customer whose first order was in the last few weeks of the data has not had time to return, so the latest year's `pct_repeating` is biased low and its median is biased short (only the fast returners have shown up yet). Do not compare the latest year against older years without fixing the observation window; Step 10 does that. Duplicate accounts (Q71) can also make a returning customer look like two one-time customers.

Step 8 — Common Mistakes:

- Taking `LEAD` over raw orders without folding same-day orders. Customers with two orders on their first day show a 0-day return, which lowers the median and inflates `pct_repeating`:

```
WITH sequenced AS (
    SELECT o.CustomerID,
           o.OrderDate::date AS order_date,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID)        AS order_seq,
           LEAD(o.OrderDate::date) OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID) AS next_order_date
    FROM Orders AS o
)
SELECT EXTRACT(YEAR FROM s.order_date)::int                                    AS first_order_year,
       COUNT(s.next_order_date)                                                AS returners,
       COUNT(*) FILTER (WHERE s.next_order_date = s.order_date)                AS zero_day_returns,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY s.next_order_date - s.order_date) AS median_days
FROM sequenced AS s
WHERE s.order_seq = 1
GROUP BY EXTRACT(YEAR FROM s.order_date)
ORDER BY first_order_year;
```

- Using `AVG(days_to_second)`. One customer who came back after two years can double the mean.
- Putting `WHERE day_seq = 1` in the same `SELECT` as the window function. Window functions are evaluated after `WHERE`, so PostgreSQL rejects it.
- Treating NULL `days_to_second` as 0 with `COALESCE`. That turns every one-time customer into an instant returner.

Step 9 — Interview Follow-Up: The team wants the same table but split by the payment type used on the first order. Which CTE needs the extra column, and what happens to customers whose first-day orders used two different payment types?

Step 10 — Optimization or Alternative: To compare years fairly, fix the window: "returned within 90 days", counting only customers whose first order is at least 90 days before the last date in the data. A bounded `LEFT JOIN` plus `MIN` replaces the window functions:

```
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS as_of_date
    FROM Orders AS o
),
firsts AS (
    SELECT o.CustomerID,
           MIN(o.OrderDate)::date AS first_order_date
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
    GROUP BY o.CustomerID
),
returns AS (
    SELECT f.CustomerID,
           f.first_order_date,
           MIN(o.OrderDate)::date AS second_order_date
    FROM firsts AS f
    LEFT JOIN Orders AS o
           ON o.CustomerID = f.CustomerID
          AND o.OrderDate::date >  f.first_order_date
          AND o.OrderDate::date <= f.first_order_date + 90
    GROUP BY f.CustomerID, f.first_order_date
)
SELECT EXTRACT(YEAR FROM r.first_order_date)::int                          AS first_order_year,
       COUNT(*)                                                            AS customers_observed_90d,
       COUNT(r.second_order_date)                                          AS returned_within_90d,
       ROUND(100.0 * COUNT(r.second_order_date) / NULLIF(COUNT(*), 0), 2)  AS pct_returned_90d
FROM returns AS r
CROSS JOIN anchor AS a
WHERE r.first_order_date + 90 <= a.as_of_date
GROUP BY EXTRACT(YEAR FROM r.first_order_date)
ORDER BY first_order_year;
```

The join condition `> first_order_date` applies the same-day rule, and the outer `WHERE` drops customers who have not been observed for 90 days. MySQL has no `PERCENTILE_CONT`; there the median needs `ROW_NUMBER` and a count per group.

Question 67: Median vs mean order value by payment type

Difficulty: Hard

Primary Topic: Median and percentiles

Combined Topics: PERCENTILE_CONT WITHIN GROUP, LEFT JOIN with Unknown label, Skew ratio

Business Scenario: The payments team is negotiating gateway fees, and some fee slabs depend on "typical order size". The average order value per payment method is easy to pull, but a handful of very large orders can drag an average far above what a normal order looks like. They want the mean and the median side by side for every payment type, plus a ratio that shows how skewed each method is.

Problem Statement: For each payment type, return the number of orders with a known order total, the mean order value, the median order value and the ratio of mean to median. Orders whose `PaymentID` is NULL or does not exist in `Payments` go under `Unknown`. Sort by the ratio, most skewed first.

Expected Output:

- `payment_type`: `PaymentType`, or `Unknown`
- `orders`: orders with a non-NULL `Total_Order_Amount`
- `mean_order_value`: `AVG(Total_Order_Amount)`, 2 dp
- `median_order_value`: 50th percentile of `Total_Order_Amount`, 2 dp
- `mean_to_median_ratio`: mean divided by median, 2 dp

Tables Required: `Orders`, `Payments`

SQL Concepts Tested: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ...)`, type of its result, NULL handling inside aggregates, LEFT JOIN plus `COALESCE` label, `NULLIF` in a ratio, a CTE to avoid repeating the median expression.

Step 1 — Understand the Problem: The mean is the total divided by the count, so one large order moves it. The median is the middle value when orders are sorted, so one large order barely moves it. When the mean is well above the median, the distribution has a long right tail: most orders are small and a few are big. A ratio near 1 means the two agree; a ratio of 1.5 means the "average" order is 50% larger than the typical one.

Every row in `Orders` is one order and cancellations are not modelled. The mean and median should be computed over the same set of orders, so the `orders` column counts non-NULL totals only, which is the set both aggregates actually use.

Step 2 — Identify the Tables and Relationships: `Orders` gives `PaymentID` and `Total_Order_Amount`. `Payments` gives `PaymentType` (`Orders.PaymentID` → `Payments.PaymentID`, many orders to one payment method). Use a LEFT JOIN so orders with a NULL or unmatched `PaymentID` are not dropped; they are grouped as `Unknown`. The `Allowed` flag is irrelevant here.

Step 3 — Build the Query Logic:

1. LEFT JOIN `Orders` to `Payments` and label missing types `Unknown`.
2. Group by payment type. Compute `COUNT(Total_Order_Amount)`, `AVG(...)` and `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Total_Order_Amount)`.
3. Do this in a CTE so the ratio in the outer query can use the unrounded mean and median by name.
4. Divide mean by median with `NULLIF(median, 0)`, round at the end, and sort.

Step 4 — Final SQL Solution:

```sql
WITH payment_stats AS (
    SELECT COALESCE(pay.PaymentType, 'Unknown')                          AS payment_type,
           COUNT(o.Total_Order_Amount)                                    AS orders,
           AVG(o.Total_Order_Amount)                                      AS mean_value,
           (PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY o.Total_Order_Amount))::numeric AS median_value
    FROM Orders AS o
    LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
    GROUP BY COALESCE(pay.PaymentType, 'Unknown')
)
SELECT ps.payment_type,
       ps.orders,
       ROUND(ps.mean_value, 2)                                  AS mean_order_value,
       ROUND(ps.median_value, 2)                                AS median_order_value,
       ROUND(ps.mean_value / NULLIF(ps.median_value, 0), 2)     AS mean_to_median_ratio
FROM payment_stats AS ps
WHERE ps.orders > 0
ORDER BY mean_to_median_ratio DESC NULLS LAST, ps.payment_type;
```

Step 5 — Line-by-Line Explanation:

- `COALESCE(pay.PaymentType, 'Unknown')` covers two cases at once: `PaymentID` is NULL, or it points to no row in `Payments`. In both cases the LEFT JOIN returns NULL for `PaymentType`.
- `COUNT(o.Total_Order_Amount)` skips NULL totals. `AVG` and `PERCENTILE_CONT` skip them too, so all three describe the same orders. `COUNT(*)` would include orders that neither statistic used.
- `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ...)` is an ordered-set aggregate. It sorts the group's values and interpolates: with an

even count it returns the average of the two middle values.

- In PostgreSQL `PERCENTILE_CONT` returns `double precision`, even for a `numeric` input. The `::numeric` cast is needed because `ROUND(double precision, 2)` does not exist.
- The CTE keeps the mean and median unrounded, so the ratio is not computed from already-rounded values.
- `NULLIF(ps.median_value, 0)` protects against a group whose median is zero.
- `WHERE ps.orders > 0` drops a payment type whose orders all have NULL totals; it would otherwise show NULLs everywhere.
- `DESC NULLS LAST` keeps any NULL ratio at the bottom.

Step 6 — Sample Output:

payment_type	orders	mean_order_value	median_order_value	mean_to_median_ratio
Unknown	20	1995.53	1001.80	1.99
Net Banking	2075	1362.73	797.00	1.71
Wallet	2743	1230.75	739.00	1.67
UPI	23092	1339.03	815.00	1.64
Credit Card	8282	1272.37	780.95	1.63
Pay Later	1128	1276.68	797.50	1.60
Debit Card	4290	1295.58	814.00	1.59
Cash on Delivery	4722	1300.77	831.50	1.56

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 9 rows.

Step 7 — Why This Solution Works: Both statistics are computed in the same group over the same non-NULL values, so the ratio compares like with like. The `Unknown` row keeps the order count reconciling to `Orders`.

Edge cases. Orders keyed with a total of 0 are real values to `AVG` and `PERCENTILE_CONT`: they pull both down and can make a small group's median 0, which is why the ratio uses `NULLIF`. If 0 means "not captured" in your data, filter it out and say so. Small groups give unstable medians (with three orders, the median is one order), so show `orders` next to every ratio and do not rank on a ratio from a handful of orders. A payment type with no orders at all does not appear, since the query starts from `Orders`.

Step 8 — Common Mistakes:

- Calling `ROUND` directly on `PERCENTILE_CONT`. PostgreSQL raises "function round(double precision, integer) does not exist". This version fails on purpose, so it is shown as text:

```
SELECT ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY o.Total_Order_Amount), 2)
FROM Orders AS o;
```

- Using `PERCENTILE_CONT` as a window function. PostgreSQL does not support ordered-set aggregates with `OVER`, so a per-row median needs a grouped CTE joined back.
- Joining to `OrderDetails` "for more context" before taking the median. Each order then appears once per line, and multi-line orders count several times in the median. Compare the order-grain median with the fanned-out one:

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY o.Total_Order_Amount) AS median_order_grain,
       (SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY o2.Total_Order_Amount)
        FROM Orders AS o2
        JOIN OrderDetails AS od ON od.OrderID = o2.OrderID)              AS median_after_line_join
FROM Orders AS o;
```

- Using `PERCENTILE_DISC` and calling it the median. It returns an actual value from the data (the lower middle value for an even count), which is fine, but say which one you used.

Step 9 — Interview Follow-Up: The team now asks for the 90th percentile too, and whether the large orders are concentrated in one method. How do you get p50 and p90 in one pass, and what would you add to show where the large orders sit?

Step 10 — Optimization or Alternative: On MySQL there is no `PERCENTILE_CONT`. The portable median numbers the sorted values per group and averages the middle one or two rows:

```
WITH valued AS (
    SELECT COALESCE(pay.PaymentType, 'Unknown') AS payment_type,
           o.Total_Order_Amount,
           ROW_NUMBER() OVER (PARTITION BY COALESCE(pay.PaymentType, 'Unknown')
                              ORDER BY o.Total_Order_Amount, o.OrderID)            AS rn,
           COUNT(*)     OVER (PARTITION BY COALESCE(pay.PaymentType, 'Unknown'))  AS n
    FROM Orders AS o
    LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
    WHERE o.Total_Order_Amount IS NOT NULL
)
SELECT v.payment_type,
       MAX(v.n)                            AS orders,
       ROUND(AVG(v.Total_Order_Amount), 2) AS median_order_value
FROM valued AS v
WHERE v.rn IN (FLOOR((v.n + 1) / 2), FLOOR((v.n + 2) / 2))
GROUP BY v.payment_type
ORDER BY v.payment_type;
```

`FLOOR` makes the `IN` list pick one row for an odd count and two for an even count, which matches `PERCENTILE_CONT(0.5)`. It is required on MySQL, where `/` always returns a decimal (`5 / 2` is 2.5), so an even count would match only the upper middle row. Filtering NULL totals before numbering is essential; otherwise `n` counts rows the median cannot use. To get several percentiles in PostgreSQL in one pass, pass an array: `PERCENTILE_CONT(ARRAY[0.5, 0.9]) WITHIN GROUP (ORDER BY ...)` returns both.

Question 68: Second-highest spender in each state, counting delivered orders only

Difficulty: Hard

Primary Topic: Conditional ranking

Combined Topics: DENSE_RANK over an aggregate, SUM with FILTER, LEFT JOIN with Unknown and Unmapped labels

Business Scenario: The CRM team runs a state-level loyalty programme. The top spender in each state already gets a personal account manager, so this quarter they want to invite the runner-up in every state to a priority-delivery pilot. Spend must count only orders that actually reached the customer, because an order that never arrived says nothing about loyalty. If two customers tie for second place, both are invited.

Problem Statement: For each customer state, rank customers by the total `Total_Order_Amount` of their delivered orders and return everyone at the second-highest spend level. An order counts as delivered when `DeliveryDate` is present and not before `OrderDate`. Customers with a NULL or blank `State` go under `Unknown`; orders whose `CustomerID` has no customer record go under `Unmapped customer`.

Expected Output:

- `State`: the customer's state, `Unknown` or `Unmapped customer`
- `CustomerID`
- `customer_name`: first and last name, or `Unknown customer`
- `delivered_spend`: sum of `Total_Order_Amount` over delivered orders
- `spend_rank`: always 2 in the output (ties share it)

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: `SUM(...) FILTER (WHERE ...)`, a window function ordered by an aggregate, `DENSE_RANK` vs `RANK` vs `ROW_NUMBER` for "second", `PARTITION BY` on a derived label, filtering on a window result in an outer query.

Step 1 — Understand the Problem: "Second-highest" needs a precise meaning. Here it is the second-highest spend value in the state: if two customers share the top value, the next value down is second. That is what `DENSE_RANK` returns. `RANK` would number the next customer 3 after a tie at the top, leaving nobody at 2, and `ROW_NUMBER` would pick one of two tied customers arbitrarily.

"Delivered" is an assumption, because the schema has no status column: a delivery date exists and is not earlier than the order date. Orders with a NULL delivery date are in transit or lost, and a delivery before the order is a data error, so neither counts. Cancellations and returns are not modelled.

Step 2 — Identify the Tables and Relationships: `Orders` gives `CustomerID`, `Total_Order_Amount`, `OrderDate` and `DeliveryDate`. `Customers` gives `State` and the name (`Orders.CustomerID` → `Customers.CustomerID`, many orders to one customer). A LEFT JOIN keeps orders whose customer is missing. Those orders are labelled `Unmapped customer`, which is kept separate from customers who exist but have no state, because they are different problems with different owners.

Step 3 — Build the Query Logic:

1. `order_rows`: one row per order with the state label, the name and a boolean `is_delivered`.
2. `ranked`: group by state and customer. Sum the delivered totals with `FILTER (WHERE is_delivered)` and rank inside the same `SELECT` with `DENSE_RANK() OVER (PARTITION BY state ORDER BY <that sum> DESC)`.
3. `HAVING ... > 0` removes customers with no delivered spend, so a customer with nothing delivered cannot be someone's runner-up.
4. The outer query keeps `spend_rank = 2`. Window results cannot be filtered in the `WHERE` of the query that computes them.

Step 4 — Final SQL Solution:

```sql
WITH order_rows AS (
    SELECT o.OrderID,
           o.CustomerID,
           CASE WHEN c.CustomerID IS NULL THEN 'Unmapped customer'
                ELSE COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')
           END                                                    AS state,
           COALESCE(NULLIF(CONCAT_WS(' ', c.FirstName, c.LastName), ''), 'Unknown customer') AS customer_name,
           o.Total_Order_Amount,
           (o.DeliveryDate IS NOT NULL
            AND o.DeliveryDate::date >= o.OrderDate::date)            AS is_delivered
    FROM Orders AS o
    LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
),
ranked AS (
    SELECT r.state,
           r.CustomerID,
           r.customer_name,
           SUM(r.Total_Order_Amount) FILTER (WHERE r.is_delivered)     AS delivered_spend,
           DENSE_RANK() OVER (
               PARTITION BY r.state
               ORDER BY SUM(r.Total_Order_Amount) FILTER (WHERE r.is_delivered) DESC
           )                                                          AS spend_rank
    FROM order_rows AS r
    GROUP BY r.state, r.CustomerID, r.customer_name
    HAVING SUM(r.Total_Order_Amount) FILTER (WHERE r.is_delivered) > 0
)
SELECT rk.state AS State,
       rk.CustomerID,
       rk.customer_name,
       rk.delivered_spend,
       rk.spend_rank
FROM ranked AS rk
WHERE rk.spend_rank = 2
ORDER BY rk.state, rk.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- The `CASE` checks `c.CustomerID IS NULL` first. After a LEFT JOIN that means "no matching customer", which is different from a customer whose `State` is NULL.
- `NULLIF(TRIM(c.State), '')` turns blank or space-only states into NULL so they join the `Unknown` group instead of forming a group of their own.
- `CONCAT_WS` skips NULL parts, and returns an empty string when both names are NULL, so `NULLIF(..., '')` is needed before the `COALESCE`.
- `is_delivered` is computed once and reused, so the rule cannot drift between the sum, the rank and the `HAVING`.
- `SUM(...) FILTER (WHERE r.is_delivered)` adds only delivered orders. In the same group you could also report `SUM(r.Total_Order_Amount)` for all orders, which a `WHERE` on delivery would make impossible.
- `DENSE_RANK() OVER (... ORDER BY SUM(...) ...)` runs after `GROUP BY` and `HAVING`, so it ranks one row per customer.
- `WHERE rk.spend_rank = 2` sits in the outer query because `WHERE` is evaluated before window functions.

Step 6 — Sample Output:

state	customerid	customer_name	delivered_spend	spend_rank
Abu Dhabi	103764	Ritu Sharma	27694.10	2
Andhra Pradesh	100067	Suresh Choudhary	69609.23	2
Assam	101679	Karthik Banerjee	68845.72	2
Bihar	104375	Megha Iyer	56484.20	2
California	103426	Sunita Nair	76207.80	2
Chandigarh	103248	Arjun Pillai	80609.35	2
Delhi	100152	Vihaan Khanna	134727.12	2
Dubai	102915	Aditya Malhotra	60179.24	2

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 27 rows.

Step 7 — Why This Solution Works: The grain is fixed at state-customer before ranking, so each customer is ranked once. Delivered spend is defined in one place. `DENSE_RANK` returns every customer at the second-highest value, and the outer filter returns all of them.

Edge cases. A state with only one delivering customer has no rank 2 and does not appear; if the CRM team needs a row per state anyway, LEFT JOIN a distinct state list to this result. Ties at second place produce several rows for one state, which is intended. A customer who moved state has one `State` value today, so all of their history counts there. Duplicate accounts (Q71) split one person's spend and can push them below a rival. An order with a NULL total is skipped by `SUM`, so it neither helps nor hurts.

Step 8 — Common Mistakes:

- Using `RANK` or `ROW_NUMBER`. With a tie at the top, `RANK` skips 2, so the state disappears from the output; `ROW_NUMBER` labels one of the tied leaders 2, chosen only by the tiebreaker. This small example shows all three:

```sql
SELECT t.customer,
       t.spend,
       ROW_NUMBER() OVER (ORDER BY t.spend DESC, t.customer) AS row_num,
       RANK()       OVER (ORDER BY t.spend DESC)             AS rank_num,
       DENSE_RANK() OVER (ORDER BY t.spend DESC)             AS dense_rank_num
FROM (VALUES ('A', 900), ('B', 900), ('C', 700), ('D', 500)) AS t(customer, spend)
ORDER BY t.spend DESC, t.customer;
```

- Ranking by total spend and then filtering to delivered orders in an outer query. The rank was already decided by undelivered orders.
- `WHERE DENSE_RANK() OVER (...) = 2` in one `SELECT`. PostgreSQL rejects window functions in `WHERE`.
- Grouping by `c.State` without trimming or labelling, so `NULL`, `''` and unmatched customers become separate, unlabelled groups.

Step 9 — Interview Follow-Up: The CRM team now wants the runner-up and the gap to the leader in each state, as both an amount and a percentage. Which window function gives you the leader's spend on the runner-up's row?

Step 10 — Optimization or Alternative: Without window functions (MySQL 5.7, some legacy warehouses), "second-highest value per group" is the highest value below the group's maximum. This returns the same rows as `DENSE_RANK() = 2`:

```sql
WITH spend AS (
    SELECT CASE WHEN c.CustomerID IS NULL THEN 'Unmapped customer'
                ELSE COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')
           END                                  AS state,
           o.CustomerID,
           SUM(o.Total_Order_Amount)            AS delivered_spend
    FROM Orders AS o
    LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
    WHERE o.DeliveryDate IS NOT NULL
      AND o.DeliveryDate::date >= o.OrderDate::date
    GROUP BY 1, o.CustomerID
    HAVING SUM(o.Total_Order_Amount) > 0
),
second_value AS (
    SELECT s.state,
           MAX(s.delivered_spend) AS second_spend
    FROM spend AS s
    WHERE s.delivered_spend < (SELECT MAX(s2.delivered_spend)
                               FROM spend AS s2
                               WHERE s2.state = s.state)
    GROUP BY s.state
)
SELECT s.state, s.CustomerID, s.delivered_spend
FROM spend AS s
JOIN second_value AS sv
  ON sv.state = s.state
 AND sv.second_spend = s.delivered_spend
ORDER BY s.state, s.CustomerID;
```

Here a `WHERE` on delivery is fine, because nothing else in the query needs undelivered orders. The window version reads the grouped data once; the correlated `MAX` reads it again per state, which is acceptable at state-level cardinality.

Question 69: Customers whose spend grew every active quarter of the latest year

Difficulty: Hard

Primary Topic: Comparing current and previous periods

Combined Topics: LAG over quarterly aggregates, BOOL_AND with FILTER, FIRST_VALUE and LAST_VALUE with a full frame, Anchored year

Business Scenario: Account managers want a short list of "rising" customers to call before annual planning: people whose spending went up every time they came back during the latest year. A customer who grew strongly in one quarter and fell in another is not on the list. To keep it meaningful, the customer must have been active in at least three quarters of that year.

Problem Statement: Take the calendar year of the latest order date in the data. For each customer, sum `Total_Order_Amount` per quarter in which they ordered (an active quarter). Return customers with at least three active quarters whose spend in each active quarter was strictly higher than in their previous active quarter, with their first and last quarter spend.

Expected Output:

- `CustomerID`
- `active_quarters`: quarters of the latest year with at least one order (3 or 4)
- `first_quarter_spend`: spend in the customer's first active quarter of the year
- `last_quarter_spend`: spend in their last active quarter of the year

Tables Required: `Orders`

SQL Concepts Tested: anchoring the year to `MAX(OrderDate)`, aggregation to customer-quarter, `LAG` over aggregated rows, `BOOL_AND` as an "every row passes" test, NULL-safe comparisons, `FIRST_VALUE` / `LAST_VALUE` with an explicit full frame, a named `WINDOW`.

Step 1 — Understand the Problem: "Grew every active quarter" is a check across rows: compare each quarter with the one before it and require every comparison to pass. `LAG` puts the previous quarter on the current row; `BOOL_AND` returns true only if a condition is true on every row of the group. The first active quarter has nothing to compare against, so it is excluded from the check.

"Previous" means the previous active quarter, not the previous calendar quarter. A customer active in Q1, Q2 and Q4 is compared Q2 against Q1 and Q4 against Q2. The latest year is anchored to the data: the year containing `MAX(OrderDate)`. If the data ends mid-year, the last quarter is partial, and a partial quarter rarely beats a full one, so read the result with that in mind. Cancellations are not modelled.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID`, `OrderDate`, `Total_Order_Amount`. Nothing is joined, so there is no fan-out, and an order with an unknown `CustomerID` is still analysed under that ID.

Step 3 — Build the Query Logic:

1. `anchor`: start of the year of the latest order.
2. `quarterly`: spend per customer per quarter, inside that year (half-open range).
3. `compared`: per customer, ordered by quarter, add the quarter's position, the previous quarter's spend (`LAG`), and the first and last quarter's spend (`FIRST_VALUE`, `LAST_VALUE` over the whole partition).
4. Group by customer. Keep those with `COUNT(*) >= 3` where `BOOL_AND(spend > previous)` holds for every quarter after the first.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS year_start
    FROM Orders AS o
),
quarterly AS (
    SELECT o.CustomerID,
           DATE_TRUNC('quarter', o.OrderDate)::date AS order_quarter,
           SUM(o.Total_Order_Amount)                AS quarter_spend
    FROM Orders AS o
    CROSS JOIN anchor AS a
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate >= a.year_start
      AND o.OrderDate <  a.year_start + INTERVAL '1 year'
    GROUP BY o.CustomerID, DATE_TRUNC('quarter', o.OrderDate)::date
),
compared AS (
    SELECT q.CustomerID,
           q.order_quarter,
           q.quarter_spend,
           ROW_NUMBER() OVER w_order                AS quarter_seq,
           LAG(q.quarter_spend) OVER w_order        AS prev_spend,
           FIRST_VALUE(q.quarter_spend) OVER w_all  AS first_quarter_spend,
           LAST_VALUE(q.quarter_spend)  OVER w_all  AS last_quarter_spend
    FROM quarterly AS q
    WINDOW w_order AS (PARTITION BY q.CustomerID ORDER BY q.order_quarter),
           w_all   AS (PARTITION BY q.CustomerID ORDER BY q.order_quarter
                       ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
)
SELECT c.CustomerID,
       COUNT(*)                   AS active_quarters,
       MAX(c.first_quarter_spend) AS first_quarter_spend,
       MAX(c.last_quarter_spend)  AS last_quarter_spend
FROM compared AS c
GROUP BY c.CustomerID
HAVING COUNT(*) >= 3
   AND BOOL_AND(COALESCE(c.quarter_spend > c.prev_spend, FALSE)) FILTER (WHERE c.quarter_seq > 1)
ORDER BY last_quarter_spend DESC, c.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `DATE_TRUNC('year', MAX(o.OrderDate))` gives 1 January of the latest year in the data. `+ INTERVAL '1 year'` closes the half-open range, which still works if `OrderDate` is ever a timestamp.
- `quarterly` has one row per customer per active quarter. Quarters with no orders produce no row, which is exactly the "active quarter" definition.
- `WINDOW w_order` and `w_all` name the two window definitions so each is written once.
- `LAG(...) OVER w_order` returns NULL on the first quarter. `quarter_seq > 1` removes that row from the check, so the first quarter cannot fail it.
- `FIRST_VALUE` and `LAST_VALUE` use `w_all`, whose frame is the entire partition. With the default frame, `LAST_VALUE` stops at the current row and simply returns the current quarter's spend.
- Every row of a customer carries the same first and last values, so `MAX(...)` just picks that value after grouping.
- `COALESCE(c.quarter_spend > c.prev_spend, FALSE)` handles a quarter whose spend is NULL (every order total missing). The comparison would be NULL, and `BOOL_AND` ignores NULLs, so without the `COALESCE` an unknown quarter would silently pass.

Step 6 — Sample Output:

customerid	active_quarters	first_quarter_spend	last_quarter_spend
101751	4	14899.97	47649.47
102278	3	10507.09	42430.30
102649	4	27365.00	40641.70
102575	3	3464.00	32883.99
102307	3	11556.00	29185.00
100924	3	7756.00	28160.00

102388	3	2302.00	27501.30
103216	3	1758.00	27269.90

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 166 rows.

Step 7 — Why This Solution Works: Each customer-quarter is one row, so `LAG` compares quarter totals, not individual orders. `BOOL_AND` over the filtered rows is a direct translation of "every comparison is true". The three-quarter minimum sits in `HAVING`, applied after the comparison has been evaluated per customer.

Edge cases. Equal spend in two consecutive quarters is not growth (`>` not `>=`). A customer with only two active quarters cannot qualify even if they grew. A strict rule can return very few customers, and that is a finding, not a bug; if the list is too short, relax the rule (for example, "grew in at least two of three comparisons") and state it. Duplicate accounts (Q71) split one person's quarters.

Step 8 — Common Mistakes:

- Comparing only the last quarter with the first. A customer who dipped in the middle passes, and the list is much longer than it should be:

```sql
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS year_start
    FROM Orders AS o
),
quarterly AS (
    SELECT o.CustomerID,
           DATE_TRUNC('quarter', o.OrderDate)::date AS order_quarter,
           SUM(o.Total_Order_Amount)                AS quarter_spend
    FROM Orders AS o
    CROSS JOIN anchor AS a
    WHERE o.OrderDate >= a.year_start
      AND o.OrderDate <  a.year_start + INTERVAL '1 year'
    GROUP BY o.CustomerID, DATE_TRUNC('quarter', o.OrderDate)::date
),
ends AS (
    SELECT q.CustomerID,
           COUNT(*) AS active_quarters,
           (ARRAY_AGG(q.quarter_spend ORDER BY q.order_quarter))[1]      AS first_quarter_spend,
           (ARRAY_AGG(q.quarter_spend ORDER BY q.order_quarter DESC))[1] AS last_quarter_spend
    FROM quarterly AS q
    GROUP BY q.CustomerID
)
SELECT e.CustomerID, e.active_quarters, e.first_quarter_spend, e.last_quarter_spend
FROM ends AS e
WHERE e.active_quarters >= 3
  AND e.last_quarter_spend > e.first_quarter_spend
ORDER BY e.last_quarter_spend DESC, e.CustomerID;
```

- `LAST_VALUE` without a full frame, which returns the current row's value.
- Running `LAG` over raw orders instead of quarter totals. It then compares one order with the previous order.
- `BOOL_AND(spend > prev_spend)` without the filter or `COALESCE`: the first quarter's NULL is ignored, which happens to work, but a NULL spend later in the year also passes.

Step 9 — Interview Follow-Up: The account managers now want "grew every calendar quarter", so a skipped quarter counts as a drop to zero. How would you change the query so that a customer active in Q1, Q2 and Q4 fails?

Step 10 — Optimization or Alternative: Instead of `BOOL_AND`, count the failures and require zero. This works on engines without `BOOL_AND` (MySQL, SQL Server) and makes it easy to list how many times each customer dipped:

```sql
WITH anchor AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date AS year_start
    FROM Orders AS o
),
quarterly AS (
    SELECT o.CustomerID,
           DATE_TRUNC('quarter', o.OrderDate)::date AS order_quarter,
           SUM(o.Total_Order_Amount)                AS quarter_spend
    FROM Orders AS o
    CROSS JOIN anchor AS a
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate >= a.year_start
      AND o.OrderDate <  a.year_start + INTERVAL '1 year'
    GROUP BY o.CustomerID, DATE_TRUNC('quarter', o.OrderDate)::date
),
compared AS (
    SELECT q.CustomerID,
           q.quarter_spend,
           LAG(q.quarter_spend) OVER (PARTITION BY q.CustomerID ORDER BY q.order_quarter) AS prev_spend,
           ROW_NUMBER()         OVER (PARTITION BY q.CustomerID ORDER BY q.order_quarter) AS quarter_seq
    FROM quarterly AS q
)
SELECT c.CustomerID,
       COUNT(*)                                                    AS active_quarters,
       SUM(CASE WHEN c.quarter_seq > 1
                 AND NOT COALESCE(c.quarter_spend > c.prev_spend, FALSE)
                THEN 1 ELSE 0 END)                                 AS quarters_without_growth
FROM compared AS c
GROUP BY c.CustomerID
HAVING COUNT(*) >= 3
ORDER BY quarters_without_growth, c.CustomerID;
```

Customers with `quarters_without_growth = 0` are the answer; those with 1 are the natural "nearly rising" list if the strict one is too short.

Question 70: Sub-categories most often bought together

Difficulty: Hard

Primary Topic: Self join for pair counting

Combined Topics: DISTINCT before pairing, Inequality join to drop mirrored pairs, RANK for top N with ties, Choosing a grain dense enough to count

Business Scenario: The merchandising team is planning cross-category placements: which aisles to put next to each other in the app and which sub-categories to suggest at checkout. They want the ten pairs of sub-categories that appear in the same order most often. A pair counts once per order, whatever the quantities or the number of products from each side, and (A, B) is the same pair as (B, A). Placement is decided per aisle, so they count pairs of sub-categories rather than individual products; a single aisle holds hundreds of products, and a product-level ranking would split that demand across them.

Problem Statement: Find every pair of different `Sub_Category` values that appear together in at least one order, count the distinct orders containing both, and return the top ten pairs by that count. Keep any pairs tied with tenth place.

Expected Output:

- `sub_category_a`: the alphabetically first sub-category of the pair
- `sub_category_b`: the second
- `orders_together`: number of distinct orders containing both

Tables Required: `OrderDetails`, `Products`

SQL Concepts Tested: self join on `OrderID`, `b.x > a.x` to drop mirrored pairs and self-pairs, deduplicating to the grain of the question before pairing, `RANK` for top N with ties.

Step 1 — Understand the Problem: A self join of order lines on `OrderID` produces every combination of two lines in the same order. That raw output has three problems. It pairs a line with itself. It produces each pair twice, as (A, B) and (B, A). And if an order has three skin-care products, every pairing with skin care is counted three times for that one order. The query has to remove all three before counting, and the last one is solved by moving to the grain of the question first: one row per order and sub-category.

Grain choice is part of the answer. A pair count is only useful if pairs repeat often enough to separate the leaders. Run the product-level version (Step 8) and even the strongest product pair appears in only a few dozen orders, while the top aisle pairs appear in more than a thousand. Ninety sub-categories match the decision and give every pair enough orders to rank reliably.

Step 2 — Identify the Tables and Relationships: `OrderDetails` holds `OrderID` and `ProductID`. `Products` maps each product to its `Sub_Category` (`OrderDetails.ProductID` → `Products.ProductID`). The result of that join is then joined to itself on `OrderID`. `Orders` is not needed: a pair is defined by lines sharing an `OrderID`.

Step 3 — Build the Query Logic:

1. `order_subcats`: join lines to products and `SELECT DISTINCT OrderID, Sub_Category`. Now each sub-category appears at most once per order.
2. `pairs`: join `order_subcats` to itself on `OrderID` with `b.Sub_Category > a.Sub_Category`. The strict inequality removes self-pairs and keeps exactly one orientation of each pair.
3. Group by the two sub-categories and `COUNT(*)`. Because step 1 made (order, sub-category) unique, each row in a group is a different order. Rank the pairs with `RANK()`.
4. Keep rank ≤ 10 and sort.

Step 4 — Final SQL Solution:

```sql
WITH order_subcats AS (
    SELECT DISTINCT od.OrderID,
           p.Sub_Category
    FROM OrderDetails AS od
    JOIN Products AS p ON p.ProductID = od.ProductID
    WHERE p.Sub_Category IS NOT NULL
),
pairs AS (
    SELECT a.Sub_Category                       AS sub_category_a,
           b.Sub_Category                       AS sub_category_b,
           COUNT(*)                             AS orders_together,
           RANK() OVER (ORDER BY COUNT(*) DESC) AS pair_rank
    FROM order_subcats AS a
    JOIN order_subcats AS b
      ON b.OrderID      = a.OrderID
     AND b.Sub_Category > a.Sub_Category
    GROUP BY a.Sub_Category, b.Sub_Category
)
SELECT pr.sub_category_a,
       pr.sub_category_b,
       pr.orders_together
FROM pairs AS pr
WHERE pr.pair_rank <= 10
ORDER BY pr.orders_together DESC, pr.sub_category_a, pr.sub_category_b;
```

Step 5 — Line-by-Line Explanation:

- `SELECT DISTINCT od.OrderID, p.Sub_Category` collapses an order's lines to the sub-categories it touched. Quantity and the number of

products are irrelevant to "bought together", so they are dropped here.

- `b.Sub_Category > a.Sub_Category` does two jobs in one condition: a sub-category is never paired with itself, and only the alphabetical (first, second) orientation survives. The inequality works on text as well as on IDs.
- `COUNT(*)` counts orders. `COUNT(DISTINCT a.OrderID)` gives the same number here only because of step 1; the dedup step is the real fix.
- `RANK() OVER (ORDER BY COUNT(*) DESC)` runs after the `GROUP BY`, so it ranks pairs. Tied pairs share a rank, and `pair_rank <= 10` keeps every pair whose count is in the top ten positions, which can return more than ten rows.
- The final `ORDER BY` adds both names after the count, so tied pairs always print in the same order.

Step 6 — Sample Output:

sub_category_a	sub_category_b	orders_together
Bath & Hand Wash	Hair Care	2408
Bath & Hand Wash	Skin Care	1670
Masalas & Spices	Skin Care	1442
Biscuits & Cookies	Tea	1415
Ready To Cook & Eat	Skin Care	1350
Hair Care	Skin Care	1348
Bath & Hand Wash	Masalas & Spices	1205
Biscuits & Cookies	Skin Care	1168

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 10 rows.

Step 7 — Why This Solution Works: After deduplication, each (order, sub-category) appears once, so the self join produces each unordered pair at most once per order. Grouping by the ordered pair then counts orders, which is the definition asked for.

Edge cases. Orders that touch a single sub-category produce no pairs, which is correct. Large sub-categories pair with everything simply because they are in many baskets, so a high count does not by itself mean two aisles are related (Step 10). The self join grows with the square of basket breadth: an order touching 10 sub-categories produces 45 pairs, which is why deduplicating first matters for cost as well as correctness.

Step 8 — Common Mistakes:

- Pairing raw lines instead of distinct sub-categories. An order with three skin-care products and one hair-care product counts the pair three times:

```
SELECT pa.Sub_Category              AS sub_category_a,
       pb.Sub_Category              AS sub_category_b,
       COUNT(*)                     AS orders_together_wrong,
       COUNT(DISTINCT a.OrderID)    AS orders_together_right
FROM OrderDetails AS a
JOIN Products     AS pa ON pa.ProductID = a.ProductID
JOIN OrderDetails AS b  ON b.OrderID    = a.OrderID
JOIN Products     AS pb ON pb.ProductID = b.ProductID
WHERE pb.Sub_Category > pa.Sub_Category
GROUP BY pa.Sub_Category, pb.Sub_Category
ORDER BY orders_together_wrong DESC, sub_category_a, sub_category_b
LIMIT 10;
```

- Staying at product level because the question said "bought together". The strongest product pair on this catalogue appears in a few dozen orders, so most of the top ten would be separated by a handful of baskets. Check density before ranking:

```
WITH op AS (SELECT DISTINCT od.OrderID, od.ProductID FROM OrderDetails AS od)
SELECT MAX(n) AS best_product_pair_orders
FROM (
    SELECT a.ProductID, b.ProductID, COUNT(*) AS n
    FROM op AS a
    JOIN op AS b ON b.OrderID = a.OrderID AND b.ProductID > a.ProductID
    GROUP BY a.ProductID, b.ProductID
) AS x;
```

- Joining on `b.Sub_Category <> a.Sub_Category`. Every pair appears twice, and the top ten becomes the top five, each listed in both orientations.
- `ORDER BY orders_together DESC, sub_category_a, sub_category_b FETCH FIRST 10 ROWS WITH TIES`. `WITH TIES` compares the whole sort key, and the names make every key unique, so exactly ten rows come back and a tie at tenth place is cut. Ranking first, as in Step 4, keeps the ties and a stable order.

Step 9 — Interview Follow-Up: The team wants the pairs as a share: of all orders containing sub-category A, what percentage also contain B? Which extra count do you need, and does the percentage change when you swap A and B?

Step 10 — Optimization or Alternative: Raw counts favour big sub-categories. Lift compares how often a pair occurs with how often it would occur by chance: lift = (orders with both × total orders) / (orders with A × orders with B), where a value above 1 means the pair occurs more often than chance. Q98 builds support, confidence and lift at category level; the same CTEs work here by swapping `CategoryID` for `Sub_Category`. On a large line table, an index on `OrderDetails(OrderID, ProductID)` serves the `DISTINCT` and the self join, and pairing distinct sub-categories rather than lines keeps the intermediate row count down.

Question 71: Deduplicate customer accounts by email

Difficulty: Hard

Primary Topic: Deduplication with window functions

Combined Topics: Normalised match key, Deterministic tie-break, Self join to the survivor row

Business Scenario: The data platform team is building a customer master table. Some people have registered more than once with the same email, typed with different capitalisation or a stray space. Before the merge job runs, the team wants a mapping table they can review: for every duplicate account, which account survives and which one gets merged into it. The rule is that the earliest registration survives.

Problem Statement: Treat two accounts as the same person when their emails match after trimming spaces and lower-casing. Within each group of matching accounts, the survivor is the one with the earliest `DateEntered`; if two accounts registered on the same day, the lower `CustomerID` survives. Return one row per non-surviving account. Accounts with a NULL or blank email are never matched.

Expected Output:

- `normalized_email`: `LOWER(TRIM(Email))`
- `survivor_customer_id`: the account that is kept
- `duplicate_customer_id`: an account to merge into the survivor
- `survivor_date_entered`, `duplicate_date_entered`: their registration dates

Tables Required: `Customers`

SQL Concepts Tested: normalising a match key, `ROW_NUMBER() OVER (PARTITION BY key ORDER BY ...)`, deterministic tie-breaking with `NULLS LAST`, joining a ranked set to itself to pair each duplicate with its survivor.

Step 1 — Understand the Problem: Deduplication has three decisions, and each needs to be stated. The match key: what makes two rows the same entity (here, the normalised email). The survivor rule: which row wins (earliest `DateEntered`, then lowest `CustomerID`). The exclusions: which rows must never be matched (NULL or blank emails, since every customer without an email would otherwise merge into one person).

The output is a mapping, not a deletion. Merging accounts is a business decision with consequences for orders, loyalty points and consent records, so the SQL proposes and a person approves.

Step 2 — Identify the Tables and Relationships: Only `Customers` is needed: `CustomerID`, `Email` and `DateEntered`. Orders are not touched here; repointing `Orders.CustomerID` from the duplicate to the survivor is the next step of the merge job, and this mapping is its input.

Step 3 — Build the Query Logic:

1. `keyed`: compute `normalized_email = LOWER(TRIM(Email))` and number the accounts inside each email group with `ROW_NUMBER()`, ordered by `DateEntered NULLS LAST, CustomerID`.
2. Rows with `account_rank = 1` are survivors; rows with `account_rank > 1` are duplicates.
3. Join duplicates to the survivor of the same email to put both IDs and dates on one row.

Step 4 — Final SQL Solution:

```sql
WITH keyed AS (
    SELECT c.CustomerID,
           LOWER(TRIM(c.Email)) AS normalized_email,
           c.DateEntered,
           ROW_NUMBER() OVER (
               PARTITION BY LOWER(TRIM(c.Email))
               ORDER BY c.DateEntered NULLS LAST, c.CustomerID
           )                    AS account_rank
    FROM Customers AS c
    WHERE NULLIF(TRIM(c.Email), '') IS NOT NULL
)
SELECT d.normalized_email,
       s.CustomerID  AS survivor_customer_id,
       d.CustomerID  AS duplicate_customer_id,
       s.DateEntered AS survivor_date_entered,
       d.DateEntered AS duplicate_date_entered
FROM keyed AS d
JOIN keyed AS s
  ON s.normalized_email = d.normalized_email
 AND s.account_rank     = 1
WHERE d.account_rank > 1
ORDER BY d.normalized_email, d.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `LOWER(TRIM(c.Email))` is the match key. `TRIM` removes leading and trailing spaces, `LOWER` removes case differences. PostgreSQL compares text case-sensitively, so `'A@x.com' = 'a@x.com'` is false without it.
- `WHERE NULLIF(TRIM(c.Email), '') IS NOT NULL` drops NULL, empty and space-only emails in one test. `ROW_NUMBER` would otherwise put all NULL emails in a single partition and rank them as one person.
- `PARTITION BY LOWER(TRIM(c.Email))` repeats the expression instead of using the alias, because aliases from the same `SELECT` cannot be used inside a window definition.
- `ORDER BY c.DateEntered NULLS LAST, c.CustomerID` is the survivor rule. `NULLS LAST` matters: ascending order already puts NULLs last in PostgreSQL, but writing it makes the rule explicit and keeps it correct if the query is ported. `CustomerID` makes the order unique, so the

same survivor is chosen on every run.

- The self join pairs each duplicate (`d`) with the rank-1 row (`s`) of its email. A group of three accounts produces two rows, both pointing at the same survivor.
- The final sort groups each email's rows together.

Step 6 — Sample Output:

normalized_email	survivor_customer_id	duplicate_customer_id	survivor_date_entered	duplicate_date_entered
aarav.bose@icloud.com	102195	105017	2024-02-24	2025-01-24
aarav.dsouza@gmail.com	100533	105054	2025-10-08	2025-12-20
abhishek.chowdary@gmail.com	100976	105043	2025-02-05	2025-07-12
ajay.khan3084@gmail.com	104189	105025	2023-05-16	2023-09-19
ananya.rawat@hotmail.com	102593	105046	2025-07-27	2025-12-20
anjali.banerjee@gmail.com	100250	105033	2024-06-02	2024-10-21
arjun.khanna@gmail.com	101797	105005	2023-07-04	2024-07-16
arnav.iyer8706@gmail.com	103737	105038	2024-06-03	2025-02-26

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 72 rows.

Step 7 — Why This Solution Works: `ROW_NUMBER` gives exactly one rank-1 row per email, so each duplicate maps to one survivor and the join cannot fan out. The ordering is total (date, then ID), so the mapping is stable between runs, which matters when the merge job is re-run after a failure.

Edge cases. NULL or blank emails are never matched. A NULL `DateEntered` cannot win while a dated account exists. Two accounts registered on the same day are separated by `CustomerID`. Groups of three or more produce one row per extra account. The rule can be wrong in a way SQL cannot see: two family members sharing one email are two people, and a person who changed email is not caught. This is why the output goes to review first. Also note that "earliest registration" is not always the account with the most orders; check order counts before merging (Step 10).

Step 8 — Common Mistakes:

- Matching on the raw `Email`. Variants with a capital letter or a leading space survive as separate customers. Compare how many duplicates each key finds:

```sql
SELECT 'Raw Email'            AS match_key,
       COUNT(*) - COUNT(DISTINCT c.Email) AS duplicate_accounts
FROM Customers AS c
WHERE NULLIF(TRIM(c.Email), '') IS NOT NULL
UNION ALL
SELECT 'LOWER(TRIM(Email))',
       COUNT(*) - COUNT(DISTINCT LOWER(TRIM(c.Email)))
FROM Customers AS c
WHERE NULLIF(TRIM(c.Email), '') IS NOT NULL;
```

- Ordering by `DateEntered` alone. Two accounts on the same day get an arbitrary order, and the survivor can change between runs.
- Using `RANK` instead of `ROW_NUMBER`. Two accounts tied on date both get rank 1, so the group has two survivors and every duplicate maps to both.
- Using `GROUP BY normalized_email` with `MIN(CustomerID)` as the survivor. That picks the lowest ID, not the earliest registration, and the two are not the same.
- Deleting duplicates directly with `DELETE ... WHERE account_rank > 1`. Their orders become orphans.

Step 9 — Interview Follow-Up: The team decides the survivor should be the account with the most orders, falling back to the earliest registration on a tie. What changes in the query, and why must the order counts be computed before the window runs?

Step 10 — Optimization or Alternative: The merge job needs a full mapping (every account to its survivor, including survivors mapped to themselves), with the order count each survivor ends up holding. `FIRST_VALUE` with a full frame gives the survivor on every row, and orders are pre-aggregated per account so the join cannot fan out:

```sql
WITH survivor_map AS (
    SELECT c.CustomerID,
           FIRST_VALUE(c.CustomerID) OVER (
               PARTITION BY LOWER(TRIM(c.Email))
               ORDER BY c.DateEntered NULLS LAST, c.CustomerID
               ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
           ) AS survivor_customer_id
    FROM Customers AS c
    WHERE NULLIF(TRIM(c.Email), '') IS NOT NULL
),
orders_per_account AS (
    SELECT o.CustomerID, COUNT(*) AS orders
    FROM Orders AS o
    GROUP BY o.CustomerID
)
SELECT sm.survivor_customer_id,
       COUNT(*)                                                AS accounts_merged,
       STRING_AGG(sm.CustomerID::text, ', ' ORDER BY sm.CustomerID) AS account_ids,
       COALESCE(SUM(opa.orders), 0)                            AS orders_after_merge
FROM survivor_map AS sm
LEFT JOIN orders_per_account AS opa ON opa.CustomerID = sm.CustomerID
GROUP BY sm.survivor_customer_id
HAVING COUNT(*) > 1
ORDER BY sm.survivor_customer_id;
```

In production, store `LOWER(TRIM(Email))` as a generated column with an index, or add a unique index on that expression once the data is clean, so new duplicates are rejected at sign-up instead of found later.

Question 72: Same product on several lines of one order: split or duplicate?

Difficulty: Hard

Primary Topic: Window functions over a grouped key (ROW_NUMBER, COUNT, SUM OVER)

Combined Topics: GROUP BY HAVING to find repeated keys, COUNT DISTINCT for classification, CASE verdicts, Deduplication design

Business Scenario: The data platform team is adding a uniqueness rule to `OrderDetails`: one line per product per order. Before switching it on, they need to know which existing orders break it and why. A product can appear twice in one order for a legitimate reason (the units were sourced from two different suppliers) or because the line was entered twice and should be consolidated. The team wants every offending line, the total quantity per product, and a verdict per group, so the clean-up script only merges what should be merged.

Problem Statement: Find every `(OrderID, ProductID)` that appears on more than one line. For each such line return its details, its position within the group (by `OrderDetailID`), the group's line count and total quantity, and a verdict: `Split across suppliers` when the group's lines come from more than one `SupplierID`, otherwise `Same supplier: consolidate`. Add an action per line: for a split group every line is `keep`; for a same-supplier group the first line is `keep, set quantity to total` and the others are `remove`. Sort by `OrderID`, `ProductID`, line position.

Expected Output:

- `OrderDetailID`, `OrderID`, `ProductID`, `Quantity`, `SupplierID`
- `line_number`: 1, 2, ... within the group, by `OrderDetailID`
- `lines_in_group`, `group_quantity`: count of lines and sum of `Quantity` for that order-product
- `verdict`, `action`

Tables Required: `OrderDetails`

SQL Concepts Tested: GROUP BY HAVING to find keys that repeat, COUNT(DISTINCT) to classify a group, window functions to attach group totals and positions to each row without collapsing it, deterministic ordering for a delete/keep decision.

Step 1 — Understand the Problem: The rule is about a key, `(OrderID, ProductID)`, not about whole rows. There are two levels of duplication to separate. An exact duplicate (same order, product, quantity and supplier) is almost always a reload error. A repeated product with different suppliers is a business fact the schema represents as two lines. The verdict has to be decided at the group level (how many suppliers does this order-product have?), while the action is decided per line (which line survives?). That is why the query mixes a GROUP BY for the verdict with window functions for the per-line detail.

Step 2 — Identify the Tables and Relationships: Only `OrderDetails`. One row is one line; `OrderDetailID` is unique and gives a stable order for choosing the surviving line. No join to `Orders` or `Products` is needed to find the problem, though a clean-up script would join `Products` to show names to a reviewer.

Step 3 — Build the Query Logic:

1. `repeated`: group lines by `(OrderID, ProductID)`, keep groups with `COUNT(*) > 1`, and count distinct suppliers in each.
2. `numbered`: join every line back to its group, then add `ROW_NUMBER()`, `COUNT(*) OVER` and `SUM(Quantity) OVER` partitioned by the same key.
3. Final SELECT: verdict from the supplier count, action from verdict plus line position.

Step 4 — Final SQL Solution:

```sql
WITH repeated AS (
    SELECT od.OrderID,
           od.ProductID,
           COUNT(DISTINCT od.SupplierID) AS suppliers_in_group
    FROM OrderDetails AS od
    GROUP BY od.OrderID, od.ProductID
    HAVING COUNT(*) > 1
),
numbered AS (
    SELECT od.OrderDetailID,
           od.OrderID,
           od.ProductID,
           od.Quantity,
           od.SupplierID,
           r.suppliers_in_group,
           ROW_NUMBER() OVER (PARTITION BY od.OrderID, od.ProductID ORDER BY od.OrderDetailID) AS line_number,
           COUNT(*)     OVER (PARTITION BY od.OrderID, od.ProductID)                           AS lines_in_group,
           SUM(od.Quantity) OVER (PARTITION BY od.OrderID, od.ProductID)                       AS group_quantity
    FROM OrderDetails AS od
    JOIN repeated AS r
      ON r.OrderID   = od.OrderID
     AND r.ProductID = od.ProductID
)
SELECT n.OrderDetailID,
       n.OrderID,
       n.ProductID,
       n.Quantity,
       n.SupplierID,
       n.line_number,
       n.lines_in_group,
       n.group_quantity,
       CASE WHEN n.suppliers_in_group > 1 THEN 'Split across suppliers'
            ELSE 'Same supplier: consolidate'
       END AS verdict,
       CASE WHEN n.suppliers_in_group > 1 THEN 'keep'
            WHEN n.line_number = 1        THEN 'keep, set quantity to total'
            ELSE 'remove'
       END AS action
FROM numbered AS n
ORDER BY n.OrderID, n.ProductID, n.line_number;
```

Step 5 — Line-by-Line Explanation:

- `GROUP BY od.OrderID, od.ProductID HAVING COUNT(*) > 1` finds the keys that break the rule. It cannot show the lines themselves, because grouping collapses them.
- `COUNT(DISTINCT od.SupplierID)` decides the verdict for the whole group. COUNT(DISTINCT) ignores NULLs; if `SupplierID` could be NULL you would add `COUNT(*) FILTER (WHERE od.SupplierID IS NULL)` and treat unknown suppliers as their own case.
- Joining back to `OrderDetails` brings the individual lines back, restricted to offending groups.
- `ROW_NUMBER() OVER (... ORDER BY od.OrderDetailID)` numbers lines inside each group. The lowest ID is the survivor, and because IDs are unique the numbering is the same on every run.
- `COUNT(*) OVER` and `SUM(od.Quantity) OVER` attach group totals to every line without collapsing rows. The survivor needs `group_quantity` so consolidating does not lose units.

Step 6 — Sample Output:

orderdetailid	orderid	productid	quantity	supplierid	line_number	lines_in_group	group_quantity	verdict	action
2580	5000726	21431	1	7	1	2	2	Same supplier: consolidate	keep, set quantity to total
168513	5000726	21431	1	7	2	2	2	Same supplier: consolidate	remove
2704	5000754	1392	1	5	1	2	2	Same supplier: consolidate	keep, set quantity to total
168529	5000754	1392	1	5	2	2	2	Same supplier: consolidate	remove
7075	5001985	20113	1	7	1	2	2	Same supplier: consolidate	keep, set quantity to total
168523	5001985	20113	1	7	2	2	2	Same supplier: consolidate	remove
9056	5002509	6525	1	9	1	2	2	Same supplier: consolidate	keep, set quantity to total
168539	5002509	6525	1	9	2	2	2	Same supplier: consolidate	remove

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 180 rows.

Step 7 — Why This Solution Works: The group-level decision and the line-level decision are kept apart. The verdict comes from one aggregate per key, so all lines in a group get the same verdict. The action comes from the line's position within the same key, so exactly one line per consolidate-group is kept and it carries the full quantity. Split groups are left untouched, which is the important protection: a blind

"keep the first line" rule would delete real units sourced from a second supplier.

Step 8 — Common Mistakes:

- Deduplicating on the whole row. It catches exact reloads but misses a product repeated with a different quantity or supplier, so the uniqueness rule can still fail after the clean-up:

```sql
SELECT COUNT(*) AS exact_duplicate_groups
FROM (
    SELECT od.OrderID, od.ProductID, od.Quantity, od.SupplierID
    FROM OrderDetails AS od
    GROUP BY od.OrderID, od.ProductID, od.Quantity, od.SupplierID
    HAVING COUNT(*) > 1
) AS g;
```

- Keeping the first line and deleting the rest without adding their quantities. Units disappear from every report.
- Ordering `ROW_NUMBER()` by `Quantity` or by nothing. Ties or an unspecified order make the survivor change between runs, so a re-run deletes a different line.
- Treating the split groups as errors. Merging two suppliers into one line loses which supplier shipped what, which breaks supplier-level reporting.

Step 9 — Interview Follow-Up: After clean-up the team wants the database to enforce the rule going forward while still allowing split sourcing. What unique constraint would you add, and does the split case change the table design?

Step 10 — Optimization or Alternative: The clean-up for consolidate groups is two statements in one transaction: update the survivor's `Quantity` to `group_quantity`, then delete the `remove` lines by `OrderDetailID`. For the rule itself, a unique index on `(OrderID, ProductID, SupplierID)` matches the business meaning (one line per product per supplier per order) and still allows legitimate splits. Keep the exact-duplicate query from Step 8 in the nightly checks: it should return 0 once the clean-up has run.

Question 73: Category share of list-price value: latest vs previous year

Difficulty: Hard

Primary Topic: Contribution percentages

Combined Topics: Multiple CTEs, Window SUM partitioned by year, Percentage-point change, Anchored years

Business Scenario: The category management team sets next year's shelf space and marketing budget per category. Almost every category grew last year because the whole business grew (Baby Care is the one exception), so growth figures do not tell them much. They want mix shift instead: what share of total sales value each category had in the previous year and in the latest year, and how many percentage points that share moved. A category that grew slower than the business lost share, even if its own sales went up.

Problem Statement: For the latest calendar year in the data and the year before it, compute each category's share of total list-price value (`Quantity * Sale_Price`) within that year. Return both shares and the change in percentage points. Products whose `CategoryID` has no row in `Category` go under `Unmapped`. Sort by the change, biggest gain first.

Expected Output:

- `CategoryName`: category name, or `Unmapped`
- `prev_share_pct`: share of the previous year's list-price value, 2 dp (0 if no sales that year)
- `latest_share_pct`: share of the latest year's list-price value, 2 dp (0 if no sales that year)
- `share_change_pp`: latest share minus previous share, in percentage points, 2 dp

Tables Required: `Orders`, `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: anchoring two years to `MAX(OrderDate)`, aggregating a line-level measure to category-year, `SUM() OVER (PARTITION BY year)` for within-year shares, pivoting two years into columns with `FILTER`, percentage points vs percentage change.

Step 1 — Understand the Problem: A share is a category's value divided by the total for the same year. Each year's shares add up to 100, so one category gains share only if others lose it. The change is in percentage points (pp): 12% to 15% is +3 pp. Calling that "+25%" is a relative change, a different number; mixing the two is the classic error in this report.

Sale_Price is today's catalogue price, not the price paid, so list-price value is a proxy. Using one price for both years also strips out price changes, so this is closer to a volume mix. Say so when presenting it.

Step 2 — Identify the Tables and Relationships: `Orders` supplies `OrderDate`. `OrderDetails` supplies `Quantity` (`OrderDetails.OrderID` → `Orders.OrderID`). `Products` supplies `Sale_Price` and `CategoryID` (`OrderDetails.ProductID` → `Products.ProductID`). `Category` supplies the name (`Products.CategoryID` → `Category.CategoryID`), joined with LEFT JOIN so products with an unknown category are labelled `Unmapped` instead of disappearing.

The joins to `Orders` and `Products` are inner on purpose: a line with no order has no date, and a line with no product has no price. Both drop out of numerator and denominator; count them with an anti-join if you need to report the gap.

Step 3 — Build the Query Logic:

1. `anchor`: the latest year in the data.
2. `line_value`: one row per order line in the two years, tagged `prev` or `latest`, with its category and list-price value.
3. `category_period`: sum to category-period.

4. `shares`: divide by `SUM(value) OVER (PARTITION BY period)`, the total for that period only.
5. Pivot: one row per category with `MAX(share) FILTER (WHERE period = ...)`, `COALESCE` to 0 for a year with no sales, and subtract.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT EXTRACT(YEAR FROM MAX(o.OrderDate))::int AS latest_year
    FROM Orders AS o
),
line_value AS (
    SELECT CASE WHEN EXTRACT(YEAR FROM o.OrderDate) = a.latest_year
                THEN 'latest' ELSE 'prev' END        AS period,
           COALESCE(cat.CategoryName, 'Unmapped')    AS category_name,
           od.Quantity * p.Sale_Price                AS list_price_value
    FROM OrderDetails  AS od
    JOIN Orders        AS o   ON o.OrderID      = od.OrderID
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    CROSS JOIN anchor  AS a
    WHERE o.OrderDate >= MAKE_DATE(a.latest_year - 1, 1, 1)
      AND o.OrderDate <  MAKE_DATE(a.latest_year + 1, 1, 1)
),
category_period AS (
    SELECT lv.period,
           lv.category_name,
           SUM(lv.list_price_value) AS list_price_value
    FROM line_value AS lv
    GROUP BY lv.period, lv.category_name
),
shares AS (
    SELECT cp.period,
           cp.category_name,
           100.0 * cp.list_price_value
                 / NULLIF(SUM(cp.list_price_value) OVER (PARTITION BY cp.period), 0) AS share_pct
    FROM category_period AS cp
),
pivoted AS (
    SELECT s.category_name,
           COALESCE(MAX(s.share_pct) FILTER (WHERE s.period = 'prev'),   0) AS prev_share,
           COALESCE(MAX(s.share_pct) FILTER (WHERE s.period = 'latest'), 0) AS latest_share
    FROM shares AS s
    GROUP BY s.category_name
)
SELECT pv.category_name                              AS CategoryName,
       ROUND(pv.prev_share, 2)                       AS prev_share_pct,
       ROUND(pv.latest_share, 2)                     AS latest_share_pct,
       ROUND(pv.latest_share - pv.prev_share, 2)     AS share_change_pp
FROM pivoted AS pv
ORDER BY share_change_pp DESC, pv.category_name;
```

Step 5 — Line-by-Line Explanation:

- `anchor` finds the latest year once, and both years derive from it.
- `MAKE_DATE(latest_year - 1, 1, 1)` to `MAKE_DATE(latest_year + 1, 1, 1)` is a half-open range covering exactly the two years, safe if `OrderDate` becomes a timestamp.
- The `CASE` tags each line with its period. Inside the range, anything not in the latest year is in the previous one.
- A NULL `Quantity` makes the line's value NULL; `SUM` skips it.
- `category_period` changes the grain from line to category-period, one row per category per year.
- `SUM(...) OVER (PARTITION BY cp.period)` is the year total on every row of that year. Dividing by it gives shares that sum to 100 within each year.
- `MAX(...) FILTER (WHERE period = ...)` pulls each year's share into its own column; with one row per category and period, `MAX` just picks it.
- `COALESCE(..., 0)` handles a category with sales in one year only: a new category's previous share is 0, not NULL, so its gain shows.
- The change uses unrounded shares and is rounded once, so it can differ by 0.01 from subtracting the rounded columns.

Step 6 — Sample Output:

categoryname	prev_share_pct	latest_share_pct	share_change_pp
Beverages	6.98	7.84	0.86
Foodgrains, Oil & Masala	13.22	13.71	0.49
Beauty & Hygiene	37.68	38.02	0.34
Gourmet & World Food	10.75	11.06	0.30
Bakery, Cakes & Dairy	3.20	3.48	0.28
Eggs, Meat & Fish	2.43	2.62	0.20
Snacks & Branded Foods	8.47	8.63	0.16
Fruits & Vegetables	0.96	1.11	0.15

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Each share uses the total of its own year as the denominator, so both columns sum to 100 and the

changes sum to roughly 0 (up to rounding). That is the self-check: if `share_change_pp` does not net to about zero, a denominator is wrong.

Edge cases. If the latest year is partial, shares stay comparable in size but seasonal categories are skewed (a festive-season category looks weak in a year that ends in September); state the date range on the report. Categories with no sales in either year do not appear; start from `Category` with a LEFT JOIN if the list must be complete. Cancellations and returns are not modelled.

Step 8 — Common Mistakes:

- Dividing by the two-year grand total. Shares then sum to 100 across both years together, and most categories appear to gain share only because the latest year was bigger:

```
WITH anchor AS (
    SELECT EXTRACT(YEAR FROM MAX(o.OrderDate))::int AS latest_year
    FROM Orders AS o
),
category_year AS (
    SELECT EXTRACT(YEAR FROM o.OrderDate)::int        AS order_year,
           COALESCE(cat.CategoryName, 'Unmapped')     AS category_name,
           SUM(od.Quantity * p.Sale_Price)            AS list_price_value
    FROM OrderDetails  AS od
    JOIN Orders        AS o   ON o.OrderID      = od.OrderID
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    CROSS JOIN anchor  AS a
    WHERE o.OrderDate >= MAKE_DATE(a.latest_year - 1, 1, 1)
      AND o.OrderDate <  MAKE_DATE(a.latest_year + 1, 1, 1)
    GROUP BY 1, 2
)
SELECT cy.category_name,
       cy.order_year,
       ROUND(100.0 * cy.list_price_value / SUM(cy.list_price_value) OVER (), 2) AS share_of_both_years_wrong
FROM category_year AS cy
ORDER BY cy.category_name, cy.order_year;
```

- Reporting `(latest - prev) / prev` as "share growth". A move from 2% to 3% is +1 pp but +50%, which reads as a far bigger shift than it is.
- Summing `Total_Order_Amount` after joining to lines. Category value needs line-level value; an order-level total joined to lines is multiplied by the line count.
- Inner-joining `Category`, which drops unmapped products from the numerator and the denominator and quietly changes every share.

Step 9 — Interview Follow-Up: A category head says their category "grew 30% last year", yet your table shows its share fell. How can both be true, and what one extra column per year would let the report show it?

Step 10 — Optimization or Alternative: A wide version computes both years' values in one pass with conditional aggregation and then takes shares with `SUM(...) OVER ()` per column. It also keeps the absolute values, which answers the follow-up directly:

```
WITH anchor AS (
    SELECT EXTRACT(YEAR FROM MAX(o.OrderDate))::int AS latest_year
    FROM Orders AS o
),
category_values AS (
    SELECT COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           COALESCE(SUM(od.Quantity * p.Sale_Price)
                    FILTER (WHERE EXTRACT(YEAR FROM o.OrderDate) = a.latest_year - 1), 0) AS prev_value,
           COALESCE(SUM(od.Quantity * p.Sale_Price)
                    FILTER (WHERE EXTRACT(YEAR FROM o.OrderDate) = a.latest_year), 0)     AS latest_value
    FROM OrderDetails  AS od
    JOIN Orders        AS o   ON o.OrderID      = od.OrderID
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    CROSS JOIN anchor  AS a
    WHERE o.OrderDate >= MAKE_DATE(a.latest_year - 1, 1, 1)
      AND o.OrderDate <  MAKE_DATE(a.latest_year + 1, 1, 1)
    GROUP BY COALESCE(cat.CategoryName, 'Unmapped')
)
SELECT cv.category_name,
       cv.prev_value,
       cv.latest_value,
       ROUND(100.0 * (cv.latest_value - cv.prev_value) / NULLIF(cv.prev_value, 0), 2)  AS value_growth_pct,
       ROUND(100.0 * cv.prev_value   / NULLIF(SUM(cv.prev_value)   OVER (), 0), 2)     AS prev_share_pct,
       ROUND(100.0 * cv.latest_value / NULLIF(SUM(cv.latest_value) OVER (), 0), 2)     AS latest_share_pct
FROM category_values AS cv
ORDER BY cv.latest_value DESC, cv.category_name;
```

`SUM(...) OVER ()` here needs no partition, because each column already holds one year. A category's share rises only when its growth beats the business-wide growth, which this layout makes easy to see.

Question 74: Customers whose latest order is below their own average

Difficulty: Hard

Primary Topic: Window frames (ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)

Combined Topics: ROW_NUMBER for the latest row, Named WINDOW clause, Anchored date filter, Percentage with NULLIF

Business Scenario: The CRM team runs a "basket shrink" nudge: a voucher for customers who are still buying but whose most recent basket was smaller than usual for them. "Smaller than usual" means the latest order's `Total_Order_Amount` is below the mean of that customer's earlier orders. A customer needs at least three earlier orders before the team trusts that mean. Customers whose latest order is more than 90 days old belong to the lapsed-customer campaign, not this one.

Problem Statement: For each customer, compare the latest order with the average of all orders placed before it. Return customers who have at least three earlier orders with an amount, whose latest order falls in the last 90 days of the data, and whose latest amount is below their prior average. Sort by the size of the drop, largest first.

Expected Output:

- `CustomerID`
- `latest_order_date`: date of the customer's most recent order
- `latest_amount`: `Total_Order_Amount` of that order
- `prior_orders`: number of earlier orders that have an amount (at least 3)
- `avg_prior_amount`: mean amount of those earlier orders, 2 decimals
- `pct_below_avg`: how far the latest amount sits below that mean, as a percentage of the mean

Tables Required: `Orders`

SQL Concepts Tested: explicit window frame ending at `1 PRECEDING`, named `WINDOW` clause, descending `ROW_NUMBER` to pick the latest row, deterministic tie-breaks, anchored date filter, percentage with `NULLIF`.

Step 1 — Understand the Problem: Each customer needs two numbers: the latest order amount, and the average of everything before it. The benchmark must exclude the latest order, or a small basket drags its own benchmark down and the drop looks smaller than it is. A window frame that stops one row before the current row gives "the orders before this one" for every row; we then keep only the latest row per customer.

"Latest" needs a tie rule, because same-day orders happen. We sort by `OrderDate`, then `OrderID`. The ranking window and the average window must use the same sort, or they can disagree about which row is the latest.

The 90-day limit is measured back from `MAX(OrderDate)`, not `CURRENT_DATE`. On a dataset that stopped loading last month, `CURRENT_DATE` would return nothing, and the answer would change every day you ran it.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: one row per order with `CustomerID`, `OrderDate` and `Total_Order_Amount`. No join means no fan-out. If the team wants names later, LEFT JOIN `Customers` so orphan customer IDs survive.

Step 3 — Build the Query Logic:

1. `anchor`: the last order date in the data.
2. `order_history`: for every order, three window values over that customer's orders. A descending `ROW_NUMBER` marks the latest order as 1. `COUNT` and `AVG` of `Total_Order_Amount` run over the frame `ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING`.
3. Keep rank 1 rows with `prior_orders >= 3`, a latest date inside the 90-day window, and a latest amount below the prior average.
4. Express the gap as a percentage of the prior average.

Step 4 — Final SQL Solution:

```
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS last_day
    FROM Orders AS o
),
order_history AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate,
           o.Total_Order_Amount,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID
                              ORDER BY o.OrderDate DESC, o.OrderID DESC) AS recency_rank,
           COUNT(o.Total_Order_Amount) OVER earlier_orders               AS prior_orders,
           AVG(o.Total_Order_Amount)   OVER earlier_orders               AS avg_prior
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
    WINDOW earlier_orders AS (
        PARTITION BY o.CustomerID
        ORDER BY o.OrderDate, o.OrderID
        ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    )
)
SELECT oh.CustomerID,
       oh.OrderDate                 AS latest_order_date,
       oh.Total_Order_Amount        AS latest_amount,
       oh.prior_orders,
       ROUND(oh.avg_prior, 2)       AS avg_prior_amount,
       ROUND(100.0 * (oh.avg_prior - oh.Total_Order_Amount)
             / NULLIF(oh.avg_prior, 0), 2) AS pct_below_avg
FROM order_history AS oh
CROSS JOIN anchor AS a
WHERE oh.recency_rank = 1
  AND oh.prior_orders >= 3
  AND oh.OrderDate >= a.last_day - 89
  AND oh.Total_Order_Amount < oh.avg_prior
ORDER BY pct_below_avg DESC, oh.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `anchor` holds one date. `a.last_day - 89` is the first day of a 90-day window that ends on the anchor day.
- `WHERE o.CustomerID IS NOT NULL` matters because `PARTITION BY` would put every NULL-customer order into one shared partition and treat them as a single shopper.
- `ROW_NUMBER() ... ORDER BY o.OrderDate DESC, o.OrderID DESC` gives 1 to the latest order. The `OrderID` tie-break makes the choice repeatable.
- `WINDOW earlier_orders AS (...)` names the frame once so `COUNT` and `AVG` cannot drift apart. `ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING` means every row from the customer's first order up to, but not including, the current one.
- `COUNT(o.Total_Order_Amount)` counts only orders that have an amount, which is what the average is built from. `COUNT(*)` would claim more evidence than the average has.
- `oh.Total_Order_Amount < oh.avg_prior` keeps only drops. A NULL latest amount makes this comparison unknown, so the row is excluded.
- `NULLIF(oh.avg_prior, 0)` protects the division if every earlier order was keyed as 0.

Step 6 — Sample Output:

customerid	latest_order_date	latest_amount	prior_orders	avg_prior_amount	pct_below_avg
103063	2025-11-14	0.00	7	1114.24	100.00
102812	2025-12-28	9.00	5	3613.20	99.75
101518	2025-12-27	20.00	11	3272.70	99.39
100953	2025-12-27	9.50	19	1404.95	99.32
102097	2025-11-09	10.00	11	1382.18	99.28
103352	2025-11-13	12.00	25	1452.18	99.17
101244	2025-11-15	15.00	12	1518.75	99.01
102262	2025-12-08	10.00	5	829.00	98.79

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 1077 rows.

Step 7 — Why This Solution Works: The frame is evaluated per row, so for the latest order it covers exactly the orders before it in the same sort used for ranking. Filtering to `recency_rank = 1` afterwards picks that one row per customer, and the numbers attached to it already describe the history before it.

Edge cases handled on purpose:

- One-order customers: the frame is empty, `COUNT` returns 0 and `AVG` returns NULL, so `prior_orders >= 3` removes them.
- NULL amounts among earlier orders: `AVG` skips them and `prior_orders` shows how many values the mean rests on.
- Zero latest amount: a header keyed as 0 shows as 100% below. Check it against the order lines before a voucher goes out.
- Orphan `CustomerID`: the row still appears, but CRM cannot contact that customer. LEFT JOIN `Customers` when you hand over the list.

Step 8 — Common Mistakes:

- Averaging over the whole partition, which includes the latest order in its own benchmark:

```sql
WITH h AS (
    SELECT o.CustomerID, o.OrderDate, o.Total_Order_Amount,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate DESC, o.OrderID DESC) AS rn,
           AVG(o.Total_Order_Amount) OVER (PARTITION BY o.CustomerID) AS avg_all_orders  -- includes the latest
    FROM Orders AS o
)
SELECT h.CustomerID, h.Total_Order_Amount, ROUND(h.avg_all_orders, 2) AS avg_all_orders
FROM h
WHERE h.rn = 1 AND h.Total_Order_Amount < h.avg_all_orders
ORDER BY h.CustomerID;
```

- Relying on the default frame. `AVG(...) OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate)` means `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes the current row and every other order on the same date. That is a running average, not a prior average.
- Writing `WHERE ROW_NUMBER() OVER (...) = 1`. PostgreSQL rejects it because windows are computed after `WHERE`. Rank in a CTE, then filter.

Step 9 — Interview Follow-Up: The team wants to compare against the last five earlier orders instead of the full history. What changes in the frame, and what does the query do for a customer who has only three earlier orders?

Step 10 — Optimization or Alternative: The same answer can come from `DISTINCT ON` for the latest order plus a `LATERAL` subquery for the history. The row comparison `(p.OrderDate, p.OrderID) < (l.OrderDate, l.OrderID)` expresses "earlier in the same sort" in one line and also works in MySQL 8. `DISTINCT ON` is PostgreSQL-only; on MySQL use the `ROW_NUMBER` version from Step 4.

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS last_day
    FROM Orders AS o
),
latest AS (
    SELECT DISTINCT ON (o.CustomerID)
           o.CustomerID, o.OrderID, o.OrderDate, o.Total_Order_Amount
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
    ORDER BY o.CustomerID, o.OrderDate DESC, o.OrderID DESC
)
SELECT l.CustomerID,
       l.OrderDate                    AS latest_order_date,
       l.Total_Order_Amount           AS latest_amount,
       h.prior_orders,
       ROUND(h.avg_prior, 2)          AS avg_prior_amount
FROM latest AS l
CROSS JOIN anchor AS a
CROSS JOIN LATERAL (
    SELECT COUNT(p.Total_Order_Amount) AS prior_orders,
           AVG(p.Total_Order_Amount)   AS avg_prior
    FROM Orders AS p
    WHERE p.CustomerID = l.CustomerID
      AND (p.OrderDate, p.OrderID) < (l.OrderDate, l.OrderID)
) AS h
WHERE h.prior_orders >= 3
  AND l.OrderDate >= a.last_day - 89
  AND l.Total_Order_Amount < h.avg_prior
ORDER BY l.CustomerID;
```

The window version reads and sorts `Orders` once. The `LATERAL` version wins when you need only a few customers and `Orders(CustomerID, OrderDate, OrderID)` is indexed, so each history lookup is a short range scan.

Question 75: Rolling 28-day active customers, week by week

Difficulty: Hard

Primary Topic: Rolling distinct counts with a range join

Combined Topics: generate_series week spine, COUNT(DISTINCT) with FILTER, Anchored dates, LEFT JOIN

Business Scenario: The growth team reviews two health numbers every Monday. Weekly active customers are distinct customers who placed at least one order in the 7 days ending on the report date. 28-day active customers are distinct customers who ordered at least once in the 28 days ending on that date. They want the last 12 weeks side by side to see whether a dip in one week is noise or the start of a slide.

Problem Statement: Build 12 weekly report dates, each 7 days apart, with the most recent one on the last order date in the data. For each date, count distinct ordering customers in the trailing 7 days and in the trailing 28 days, both windows ending on and including that date. Weeks with no orders must still appear with 0. Drop any report date whose 28-day window starts before the first order in the data.

Expected Output:

- `week_end`: the report date (last day of the window)
- `weekly_active_customers`: distinct customers with an order in `week_end - 6` to `week_end`
- `rolling_28d_active_customers`: distinct customers with an order in `week_end - 27` to `week_end`

Tables Required: `Orders`

SQL Concepts Tested: date spine with `generate_series`, range join on a date window, `COUNT(DISTINCT)` with `FILTER`, why `COUNT(DISTINCT)` cannot be a window function, anchored dates, partial-window handling.

Step 1 — Understand the Problem: A rolling sum can be built from weekly sums. A rolling distinct count cannot: a customer who ordered in three of the last four weeks is one active customer, not three.

The tempting shortcut is `COUNT(DISTINCT o.CustomerID) OVER (...)`. PostgreSQL rejects it with "DISTINCT is not implemented for window functions". So each report date is joined to the raw orders inside its window, and the distinct count happens in an ordinary `GROUP BY`.

Report dates are anchored to `MAX(OrderDate)`, so the latest window ends on the last day with data. Calendar weeks from `DATE_TRUNC('week', ...)` would leave a half-empty final week whenever the data stops mid-week, and it would look like a collapse.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` and `OrderDate`. The second input is generated: a spine of 12 report dates. The join is on a date range, not a key, so one order joins to every report date whose window contains it (up to four). That repetition is intended, and the `DISTINCT` inside each group removes it.

Step 3 — Build the Query Logic:

1. `bounds`: first and last order dates in the data.
2. `week_ends`: `last_day - 7 * n` for n = 0 to 11, keeping only dates whose 28-day window starts on or after `first_day`.
3. LEFT JOIN `Orders` to each report date on `OrderDate` inside the 28-day window, written half-open.
4. Group by report date. `COUNT(DISTINCT CustomerID)` gives the 28-day number; the same count with `FILTER` on the last 7 days gives the weekly number.

Step 4 — Final SQL Solution:

```sql
WITH bounds AS (
    SELECT MIN(o.OrderDate)::date AS first_day,
           MAX(o.OrderDate)::date AS last_day
    FROM Orders AS o
),
week_ends AS (
    SELECT b.last_day - 7 * g.n AS week_end
    FROM bounds AS b
    CROSS JOIN generate_series(0, 11) AS g(n)
    WHERE b.last_day - 7 * g.n - 27 >= b.first_day
)
SELECT we.week_end,
       COUNT(DISTINCT o.CustomerID)
           FILTER (WHERE o.OrderDate >= we.week_end - 6) AS weekly_active_customers,
       COUNT(DISTINCT o.CustomerID)                      AS rolling_28d_active_customers
FROM week_ends AS we
LEFT JOIN Orders AS o
       ON o.OrderDate >= we.week_end - 27
      AND o.OrderDate <  we.week_end + 1
GROUP BY we.week_end
ORDER BY we.week_end;
```

Step 5 — Line-by-Line Explanation:

- `bounds` reads `Orders` once for both ends of the data. Casting to `date` keeps the arithmetic in whole days even if the column is a timestamp.
- `generate_series(0, 11)` produces 12 offsets. `b.last_day - 7 * g.n` steps back a week at a time, so every report date falls on the same weekday as the anchor.
- `WHERE b.last_day - 7 * g.n - 27 >= b.first_day` drops report dates whose 28-day window would start before the data does. Those windows would undercount and look like growth when later weeks catch up.
- `LEFT JOIN Orders ... ON o.OrderDate >= we.week_end - 27 AND o.OrderDate < we.week_end + 1` keeps all 28 days including `week_end`. The `< week_end + 1` form still works if `OrderDate` is a timestamp with a time part.
- `COUNT(DISTINCT o.CustomerID)` counts each customer once per report date. NULL customer IDs are ignored, and a week with no orders returns 0 because of the LEFT JOIN.
- `FILTER (WHERE o.OrderDate >= we.week_end - 6)` narrows the same joined rows to the last 7 days, so both metrics come from one pass.

Step 6 — Sample Output:

week_end	weekly_active_customers	rolling_28d_active_customers
2025-10-12	592	1321
2025-10-19	710	1408
2025-10-26	731	1516
2025-11-02	753	1612
2025-11-09	803	1714
2025-11-16	788	1779
2025-11-23	548	1749
2025-11-30	572	1705

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Each report date gets its own set of order rows, and `DISTINCT` is applied inside that set, which is exactly the definition of "active in the window". Nothing is added across weeks, so a repeat buyer is counted once per window. The weekly count is always less than or equal to the 28-day count, because its rows are a subset; that is a quick sanity check on any output.

Edge cases:

- Empty weeks: the LEFT JOIN keeps the report date and `COUNT` returns 0.
- Partial windows at the start of the data: excluded by the `first_day` test.
- Orphan `CustomerID` values: still counted, since an order did happen. If only registered customers count, add an `EXISTS` check against `Customers`.
- Duplicate order submissions: two orders from one customer on one day are still one active customer.

Step 8 — Common Mistakes:

- Adding up four weekly counts to get a 28-day count. It double counts every customer who ordered in more than one of those weeks:

```sql
WITH bounds AS (
    SELECT MAX(o.OrderDate)::date AS last_day FROM Orders AS o
),
weekly AS (
    SELECT b.last_day - 7 * g.n          AS week_end,
           COUNT(DISTINCT o.CustomerID)  AS weekly_active
    FROM bounds AS b
    CROSS JOIN generate_series(0, 14) AS g(n)
    LEFT JOIN Orders AS o
           ON o.OrderDate >= b.last_day - 7 * g.n - 6
          AND o.OrderDate <  b.last_day - 7 * g.n + 1
    GROUP BY b.last_day - 7 * g.n
)
SELECT w.week_end,
       w.weekly_active,
       SUM(w.weekly_active) OVER (ORDER BY w.week_end
                                  ROWS BETWEEN 3 PRECEDING AND CURRENT ROW) AS rolling_28d_wrong
FROM weekly AS w
ORDER BY w.week_end;
```

- Writing `COUNT(DISTINCT o.CustomerID) OVER (...)`. PostgreSQL raises an error; SQL Server and MySQL do not support it either.
- Using `INNER JOIN` from the spine, which deletes empty weeks and hides the dip the team is looking for.
- Using `BETWEEN we.week_end - 27 AND we.week_end` on a timestamp column, which drops every order after midnight on the last day.

Step 9 — Interview Follow-Up: Add a "stickiness" column: weekly active customers as a percentage of 28-day active customers. What does a falling stickiness with a flat 28-day count tell the growth team?

Step 10 — Optimization or Alternative: The range join touches every order up to four times. On a large table, shrink the input first: bucket each order into "weeks back from the anchor" and keep one row per customer per bucket. A 28-day window is then exactly four buckets, and the join runs on small integers instead of dates.

```sql
WITH bounds AS (
    SELECT MAX(o.OrderDate)::date AS last_day FROM Orders AS o
),
customer_weeks AS (
    SELECT DISTINCT o.CustomerID,
           (b.last_day - o.OrderDate::date) / 7 AS weeks_back
    FROM Orders AS o
    CROSS JOIN bounds AS b
    WHERE o.CustomerID IS NOT NULL
)
SELECT b.last_day - 7 * g.n                                          AS week_end,
       COUNT(DISTINCT cw.CustomerID) FILTER (WHERE cw.weeks_back = g.n) AS weekly_active_customers,
       COUNT(DISTINCT cw.CustomerID)                                 AS rolling_28d_active_customers
FROM bounds AS b
CROSS JOIN generate_series(0, 11) AS g(n)
LEFT JOIN customer_weeks AS cw
       ON cw.weeks_back BETWEEN g.n AND g.n + 3
GROUP BY b.last_day - 7 * g.n
ORDER BY week_end;
```

Integer division puts days 0 to 6 back in bucket 0, days 7 to 13 in bucket 1, and so on, so the buckets line up exactly with the Step 4 windows. The partial-window filter is left out for brevity. On MySQL, replace `generate_series` with a recursive CTE or a numbers table.

Question 76: Order value from new vs returning customers by month

Difficulty: Hard

Primary Topic: Window MIN to tag each customer's first order month

Combined Topics: Conditional aggregation with FILTER, Multiple CTEs, Anchored 12-month window, Contribution percentage

Business Scenario: The growth team is splitting next year's budget between acquisition ads and retention campaigns. Before that meeting they want to know how much of each month's order value came from customers who were new that month and how much came from customers who had ordered in an earlier month. A "new customer" in a month is one whose first ever order falls in that calendar month. Every other ordering customer that month is "returning".

Problem Statement: For each of the last 12 calendar months in the data, return the number of orders and the order value from new customers and from returning customers, and the share of order value that came from returning customers. A customer's first order month must be computed from their full history, not just from the 12 months shown.

Expected Output:

- `order_month`: first day of the month
- `new_customer_orders`: orders placed in the customer's first order month
- `returning_orders`: orders from customers whose first order month is earlier
- `new_customer_value`: `SUM(Total_Order_Amount)` of new-customer orders
- `returning_value`: `SUM(Total_Order_Amount)` of returning orders
- `returning_value_pct`: returning value as a percentage of the month's total value

Tables Required: `Orders`

SQL Concepts Tested: `MIN() OVER (PARTITION BY ...)` without collapsing rows, conditional aggregation with `FILTER`, order of evaluation (WHERE before window functions), anchored month range, contribution percentage.

Step 1 — Understand the Problem: Every order needs a label, and the label depends on something outside the order: the customer's first order month. A window `MIN(OrderDate) OVER (PARTITION BY CustomerID)` attaches that value to every order row without grouping the rows away, so we can compare each order's month with it.

The definition here is month-level. If a customer's first order is on 3 March and they order again on 20 March, both March orders count as new-customer orders. From April onward, their orders are returning. That matches how acquisition spend is judged: what did the customers we won this month spend this month?

The 12-month window is anchored to the month of `MAX(OrderDate)`, not `CURRENT_DATE`. The last month in the data may be partial, so compare its share, not its absolute value, with earlier months.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` defines the partition, `OrderDate` drives both the first-month calculation and the monthly grouping, and `Total_Order_Amount` is the value. Staying at the order grain means no join and no fan-out.

Step 3 — Build the Query Logic:

1. `anchor`: the first day of the last month in the data.
2. `tagged`: every order with its own month and its customer's first order month, computed over all history.
3. `monthly`: keep the last 12 months, then count and sum with `FILTER` for the two labels.
4. Final select: returning share of value with `NULLIF`.

The order of steps 2 and 3 is the whole question. Window functions run after `WHERE` in the same query level, so the date filter must sit in a later CTE than the window.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT DATE_TRUNC('month', MAX(o.OrderDate))::date AS last_month
    FROM Orders AS o
),
tagged AS (
    SELECT o.OrderID,
           o.Total_Order_Amount,
           DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           DATE_TRUNC('month', MIN(o.OrderDate) OVER (PARTITION BY o.CustomerID))::date
                                                  AS first_order_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
monthly AS (
    SELECT t.order_month,
           COUNT(*) FILTER (WHERE t.order_month = t.first_order_month) AS new_customer_orders,
           COUNT(*) FILTER (WHERE t.order_month > t.first_order_month) AS returning_orders,
           COALESCE(SUM(t.Total_Order_Amount)
                    FILTER (WHERE t.order_month = t.first_order_month), 0) AS new_customer_value,
           COALESCE(SUM(t.Total_Order_Amount)
                    FILTER (WHERE t.order_month > t.first_order_month), 0) AS returning_value
    FROM tagged AS t
    CROSS JOIN anchor AS a
    WHERE t.order_month >= (a.last_month - INTERVAL '11 months')::date
    GROUP BY t.order_month
)
SELECT m.order_month,
       m.new_customer_orders,
       m.returning_orders,
       m.new_customer_value,
       m.returning_value,
       ROUND(100.0 * m.returning_value
             / NULLIF(m.new_customer_value + m.returning_value, 0), 2) AS returning_value_pct
FROM monthly AS m
ORDER BY m.order_month;
```

Step 5 — Line-by-Line Explanation:

- `anchor` turns the last order date into the first day of its month. `DATE_TRUNC` returns a timestamp, so `::date` brings it back to a date.
- `MIN(o.OrderDate) OVER (PARTITION BY o.CustomerID)` has no `ORDER BY`, so its frame is the whole partition: the customer's first order date on every row. Truncating it gives the first order month.
- `WHERE o.CustomerID IS NOT NULL` keeps NULL-customer orders out. They would otherwise form one fake customer partition. Report them separately if they exist.
- `WHERE t.order_month >= ... - INTERVAL '11 months'` keeps 12 months including the anchor month. It sits in `monthly`, after the window has seen the full history.
- `COUNT(*) FILTER (...)` and `SUM(...) FILTER (...)` split one pass over the rows into two labelled buckets.
- `COALESCE(..., 0)` turns a month with no new customers into 0 instead of NULL, so the percentage stays computable.
- `NULLIF(... , 0)` guards a month whose entire value is 0 or NULL.

Step 6 — Sample Output:

order_month	new_customer_orders	returning_orders	new_customer_value	returning_value	returning_value_pct
2025-01-01	231	1493	324897.33	2128981.67	86.76

2025-02-01	217	1304	273076.79	1781831.58	86.71
2025-03-01	310	1624	397080.89	2137392.21	84.33
2025-04-01	228	1563	333249.80	2054578.87	86.04
2025-05-01	291	1665	422168.40	2089957.93	83.19
2025-06-01	241	1817	298481.28	2341850.85	88.70
2025-07-01	290	1958	370416.29	2522916.55	87.20
2025-08-01	318	2183	469155.38	2838710.79	85.82

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: The label for each order is decided from the customer's complete history, and only then is the report narrowed to 12 months. The two labels are mutually exclusive and cover every order, because an order's month can only be equal to or later than the first order month. So `new_customer_orders + returning_orders` equals the month's order count, which is a reconciliation you can run against a plain `GROUP BY` on `Orders`.

Edge cases:

- One-order customers appear only as new-customer orders, in their single month.
- Same-day first orders: both count as new, because the comparison is by month, not by the first row.
- NULL `Total_Order_Amount` is counted as an order but adds nothing to value, so value-per-order would be understated. A header keyed as 0 behaves the same way.
- Orphan `CustomerID` (no row in `Customers`) still has an order history, so it is labelled normally.
- Duplicate accounts for one person make a returning customer look new. Deduplicating customers comes before this report.

Step 8 — Common Mistakes:

- Filtering to the last 12 months in the same level as the window. `WHERE` runs first, so every customer whose real first order is older than the window looks brand new in the first month shown:

```
WITH anchor AS (
    SELECT DATE_TRUNC('month', MAX(o.OrderDate))::date AS last_month FROM Orders AS o
),
tagged_wrong AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           DATE_TRUNC('month', MIN(o.OrderDate) OVER (PARTITION BY o.CustomerID))::date AS first_order_month
    FROM Orders AS o
    CROSS JOIN anchor AS a
    WHERE o.OrderDate >= (a.last_month - INTERVAL '11 months')::date
)
SELECT t.order_month,
       COUNT(*) FILTER (WHERE t.order_month = t.first_order_month) AS new_customer_orders_wrong
FROM tagged_wrong AS t
GROUP BY t.order_month
ORDER BY t.order_month;
```

- Comparing `OrderDate = first_order_date` and calling everything else returning. A second order in the same month becomes "returning" within days of acquisition, which moves value from the acquisition bucket to the retention bucket.
- Grouping by `EXTRACT(MONTH FROM OrderDate)`, which merges the same month across years.

Step 9 — Interview Follow-Up: The team now wants a third bucket, "reactivated": returning customers whose previous order was more than 90 days before this one. Which window function do you add, and which bucket loses those orders?

Step 10 — Optimization or Alternative: If the business defines "new" strictly as the first order, not the first month, use `ROW_NUMBER` with a tie-break so exactly one order per customer is new:

```
WITH anchor AS (
    SELECT DATE_TRUNC('month', MAX(o.OrderDate))::date AS last_month
    FROM Orders AS o
),
sequenced AS (
    SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
           o.Total_Order_Amount,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID
                              ORDER BY o.OrderDate, o.OrderID) AS order_seq
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
)
SELECT s.order_month,
       COUNT(*) FILTER (WHERE s.order_seq = 1)                              AS first_orders,
       COUNT(*) FILTER (WHERE s.order_seq > 1)                              AS repeat_orders,
       COALESCE(SUM(s.Total_Order_Amount) FILTER (WHERE s.order_seq = 1), 0) AS first_order_value,
       COALESCE(SUM(s.Total_Order_Amount) FILTER (WHERE s.order_seq > 1), 0) AS repeat_order_value
FROM sequenced AS s
CROSS JOIN anchor AS a
WHERE s.order_month >= (a.last_month - INTERVAL '11 months')::date
GROUP BY s.order_month
ORDER BY s.order_month;
```

Both versions are valid; they answer different questions, so write the definition into the report header. On a large table, an index on `Orders(CustomerID, OrderDate)` lets the window read rows already in partition order. `FILTER` is PostgreSQL syntax; on MySQL write `SUM(CASE WHEN ... THEN Total_Order_Amount END)`.

Question 77: Single-source products and the value at risk

Difficulty: Hard

Primary Topic: COUNT(DISTINCT) with a single-value filter

Combined Topics: Window SUM for share of total, Multiple CTEs, LEFT JOIN for orphan keys, List-price value

Business Scenario: The supply chain risk team wants to know which products depend on one supplier only. If that supplier has a strike, a flood or a customs hold, the product goes out of stock. For each such product they need the supplier, its country (cross-border suppliers carry extra risk), and how much of the business sits on that product, so they can decide where to qualify a second supplier first.

Problem Statement: Find every product whose order lines, across all history, name exactly one supplier and never leave the supplier blank. For each, show the supplier, the supplier's country, the product's list-price value, and that value as a percentage of the list-price value of all products. Sort by value, highest first.

Expected Output:

- `ProductID`
- `Product`: product name, or `Unknown product` when the line's `ProductID` has no catalogue row
- `sole_supplier`: `CompanyName`, or `Unknown supplier` when the `SupplierID` has no supplier row
- `supplier_country`: `Country`, or `Unknown`
- `list_price_value`: `SUM(Quantity * Sale_Price)` for the product
- `pct_of_total_value`: share of list-price value across all products, single-source or not

Tables Required: `OrderDetails`, `Products`, `Suppliers`

SQL Concepts Tested: `COUNT(DISTINCT)` at the product grain, NULL behaviour in `COUNT(DISTINCT)`, window `SUM() OVER ()` for a grand total, filter after the window, LEFT JOIN for orphan keys, list-price value.

Step 1 — Understand the Problem: There is no product-to-supplier master table here. The only evidence of who supplies a product is `OrderDetails.SupplierID`, so "single-source" means "every recorded line for this product came from the same supplier". That is a history-based definition. A product sold twice can look single-source only because it was rarely sold. Say this when you present the list.

Value uses list-price value, `Quantity * Sale_Price`. `Sale_Price` is today's catalogue price, not what was paid, so this is a proxy for business exposure. It is the right grain, because `Total_Order_Amount` is per order and cannot be split across products honestly.

The denominator is every product's value, computed before we filter to single-source products. If you filter first, the shares add up to 100% among the risky products and the risk looks bigger than it is.

Step 2 — Identify the Tables and Relationships: `OrderDetails` is the driving table: one row per product line, carrying `ProductID`, `Quantity` and `SupplierID`. `Products` supplies `Product` and `Sale_Price` (`OrderDetails.ProductID` → `Products.ProductID`). `Suppliers` supplies `CompanyName` and `Country` (`OrderDetails.SupplierID` → `Suppliers.SupplierID`). Without enforced keys, either lookup can miss, so both are LEFT JOINs.

Step 3 — Build the Query Logic:

1. `product_supply`: group lines by `ProductID`. Count distinct suppliers, count lines with a NULL supplier, take the one supplier ID, sum list-price value.
2. `with_share`: add `SUM(list_price_value) OVER ()`, the total across all products.
3. Final select: keep products with exactly one supplier and no blank supplier lines, attach names, compute the share.

Step 4 — Final SQL Solution:

```sql
WITH product_supply AS (
    SELECT od.ProductID,
           COUNT(DISTINCT od.SupplierID)                 AS suppliers_used,
           COUNT(*) FILTER (WHERE od.SupplierID IS NULL) AS lines_without_supplier,
           MIN(od.SupplierID)                            AS only_supplier_id,
           SUM(od.Quantity * p.Sale_Price)               AS list_price_value
    FROM OrderDetails AS od
    LEFT JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY od.ProductID
),
with_share AS (
    SELECT ps.*,
           SUM(ps.list_price_value) OVER () AS total_value
    FROM product_supply AS ps
)
SELECT ws.ProductID,
       COALESCE(p.Product, 'Unknown product')      AS Product,
       COALESCE(s.CompanyName, 'Unknown supplier') AS sole_supplier,
       COALESCE(s.Country, 'Unknown')              AS supplier_country,
       ws.list_price_value,
       ROUND(100.0 * ws.list_price_value / NULLIF(ws.total_value, 0), 2) AS pct_of_total_value
FROM with_share AS ws
LEFT JOIN Products  AS p ON p.ProductID  = ws.ProductID
LEFT JOIN Suppliers AS s ON s.SupplierID = ws.only_supplier_id
WHERE ws.suppliers_used = 1
  AND ws.lines_without_supplier = 0
ORDER BY ws.list_price_value DESC NULLS LAST, ws.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `COUNT(DISTINCT od.SupplierID)` counts distinct non-NULL supplier IDs. A product with lines from supplier 3 and lines with no supplier still returns 1.
- `lines_without_supplier` catches that case. A blank supplier is unknown, not "the same supplier", so such products are excluded from a single-source claim.
- `MIN(od.SupplierID)` returns the only supplier ID once we know there is exactly one. Any aggregate that returns that value would do.
- `LEFT JOIN Products` inside the CTE keeps lines for product IDs missing from the catalogue. Their `Sale_Price` is NULL, so their value is NULL.
- `SUM(ps.list_price_value) OVER ()` puts the grand total on every product row. It runs in its own CTE, before the single-source filter, so the denominator covers all products.
- `LEFT JOIN Suppliers` keeps a product whose only supplier ID points to no supplier row, labelled `Unknown supplier`. That is still a single-source risk, and a worse one.
- `ORDER BY ... DESC NULLS LAST, ws.ProductID` puts unvalued orphan products at the bottom with a stable order.

Step 6 — Sample Output:

productid	product	sole_supplier	supplier_country	list_price_value	pct_of_total_value
13924	Vitamin C, E & Hyaluronic Acid De Pigmentation Cream	Hooghly Consumer Goods	'india '	1113885.00	1.80
14210	Women's Body Oil - Revitalising, Rose Geranium	Coromandel FMCG Supplies	India	480180.00	0.78
6220	Natural Nourishing Hair Conditioner	Hooghly Consumer Goods	'india '	399415.00	0.65
5575	Health Drink - Morning Miracle	Ganga Distributors LLP	India	390925.00	0.63
13015	Intensive Care Deep Moisture Body Lotion - With Pure Oat Extract	Indus Personal Care Pvt Ltd	India	361204.00	0.58
20335	Organic Biodegradable Baby Laundry Clothing Detergent	Indus Personal Care Pvt Ltd	India	329035.00	0.53
14841	Facial Kit - Derma Chill	Indus Personal Care Pvt Ltd	India	277200.00	0.45
1660	Tea Tree & Acitivated Charcoal Soap	Coromandel FMCG Supplies	India	253731.00	0.41

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 13626 rows.

Step 7 — Why This Solution Works: Supplier count and value are both measured at the product grain from the same lines, and the share is computed against a total that has not been filtered. Every non-orphan product contributes once to that total, so the shares of all products sum to 100% and the single-source ones show their true slice.

Edge cases:

- NULL `SupplierID` on some lines: excluded by `lines_without_supplier = 0`, so the list never claims certainty it does not have.
- Orphan `ProductID`: shown as `Unknown product` with NULL value. It is still a real line that a supplier shipped, so the catalogue team should fix the mapping.
- Orphan `SupplierID` as the only supplier: kept and labelled `Unknown supplier`.
- NULL `Quantity`: the product of NULL and a price is NULL, so that line adds nothing to value but still counts towards the supplier test.
- Products never ordered: absent, because there is no evidence of any supplier. They carry no current value at risk.

Step 8 — Common Mistakes:

- Filtering to single-source products before computing the total, which inflates every share:

```sql
SELECT od.ProductID,
       SUM(od.Quantity * p.Sale_Price) AS list_price_value,
       ROUND(100.0 * SUM(od.Quantity * p.Sale_Price)
             / SUM(SUM(od.Quantity * p.Sale_Price)) OVER (), 2) AS pct_of_single_source_only
FROM OrderDetails AS od
JOIN Products AS p ON p.ProductID = od.ProductID
GROUP BY od.ProductID
HAVING COUNT(DISTINCT od.SupplierID) = 1
ORDER BY list_price_value DESC, od.ProductID;
```

`HAVING` runs before the window, so `SUM(...) OVER ()` only sees single-source products.

- Using `COUNT(od.SupplierID) = 1`. That counts lines, so it finds products sold once, not products with one supplier.
- Joining `Suppliers` with an INNER JOIN, which silently drops products whose only supplier is missing from the master. Those are the riskiest rows.

Step 9 — Interview Follow-Up: The team wants to see each supplier's total single-source exposure, so they can tell which supplier's failure would hurt most. How do you roll this result up, and does the share still add correctly?

Step 10 — Optimization or Alternative: `MIN(od.SupplierID) = MAX(od.SupplierID)` is an equivalent single-supplier test that avoids the distinct sort, and it also works on engines without `FILTER`:

```sql
WITH product_supply AS (
    SELECT od.ProductID,
           MIN(od.SupplierID)              AS only_supplier_id,
           SUM(od.Quantity * p.Sale_Price) AS list_price_value,
           CASE WHEN MIN(od.SupplierID) = MAX(od.SupplierID)
                 AND COUNT(od.SupplierID) = COUNT(*)
                THEN 1 ELSE 0 END          AS is_single_source
    FROM OrderDetails AS od
    LEFT JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY od.ProductID
)
SELECT ps.ProductID,
       ps.only_supplier_id,
       ps.list_price_value,
       ROUND(100.0 * ps.list_price_value
             / NULLIF((SELECT SUM(x.list_price_value) FROM product_supply AS x), 0), 2) AS pct_of_total_value
FROM product_supply AS ps
WHERE ps.is_single_source = 1
ORDER BY ps.list_price_value DESC NULLS LAST, ps.ProductID;
```

`COUNT(od.SupplierID) = COUNT(*)` does the NULL-supplier check, and the scalar subquery is the denominator in place of the window. On a large table, an index on `OrderDetails(ProductID, SupplierID)` lets PostgreSQL compute the supplier test from the index alone.

Question 78: Expensive within their sub-category but selling below its median

Difficulty: Hard

Primary Topic: PERCENT_RANK and PERCENTILE_CONT per group

Combined Topics: Multiple CTEs, Window vs ordered-set aggregates, LEFT JOIN to keep zero sellers, Minimum group size

Business Scenario: The pricing team suspects some premium products are priced above what customers will pay. Their test: a product sits in the top quarter of prices within its own sub-category, yet it has sold fewer units than the sub-category's median product. Those products are candidates for a price review. Comparing within the sub-category matters, because a 600-rupee face cream and a 600-rupee pressure cooker are not in the same price tier.

Problem Statement: For each product, compute its price percentile rank within its `Sub_Category`, its total units sold (0 if it never sold), and the median units of that sub-category, counting every product including zero sellers. Return products with a price percentile rank of at least 0.75 whose units are strictly below the sub-category median. Only judge sub-categories with at least 20 products. Sort by sub-category, then price rank (highest first), then `ProductID`.

Expected Output:

- `Sub_Category`
- `ProductID`, `Product`, `Sale_Price`
- `price_pct_rank`: `PERCENT_RANK` of `Sale_Price` within the sub-category, 2 decimals (0 = cheapest; exactly 1 only for a unique most expensive product)
- `units_sold`: total `Quantity` across all order lines, 0 for unsold products
- `subcat_median_units`: median of `units_sold` across all products in the sub-category

Tables Required: `Products`, `OrderDetails`

SQL Concepts Tested: `PERCENT_RANK()` as a window function, `PERCENTILE_CONT` as an ordered-set aggregate (not usable with `OVER` in PostgreSQL), aggregating lines to the product grain before joining, keeping zero sellers, a minimum group size, joining two group-level results.

Step 1 — Understand the Problem: Three measures live at three grains. Price rank is a property of a product relative to its sub-category: a window function. Units sold is a sum over order lines: an aggregate that must be collapsed to one row per product first. The median is one number per sub-category: an ordered-set aggregate. The final filter compares the product-level numbers against the sub-category number, so all three have to land on the same product row.

Two judgment calls are stated up front. Unsold products count as 0 units in the median, because a sub-category full of dead listings really does have a low typical seller. And small sub-categories are skipped: in a group of three products, "top quarter" is one product and the median is one product, which is noise rather than evidence.

Step 2 — Identify the Tables and Relationships: `Products` gives `Sub_Category` and `Sale_Price`. `OrderDetails` gives `Quantity` (`OrderDetails.ProductID` → `Products.ProductID`, one product to many lines). Units must be summed per product before they meet `Products`, or each product row is repeated once per line and the percentile rank is computed over the wrong rows.

Step 3 — Build the Query Logic:

1. `product_units`: sum `Quantity` per `ProductID`.
2. `scored`: LEFT JOIN products to their units (COALESCE to 0), compute `PERCENT_RANK()` and the group size with window functions partitioned by `Sub_Category`.
3. `subcat_median`: `PERCENTILE_CONT(0.5)` of the zero-filled units per sub-category. It has to be computed from `scored` (zero sellers included), not from `product_units`.
4. Join the two, keep groups of 20 or more, top-quartile price, units below median.

Step 4 — Final SQL Solution:

```sql
WITH product_units AS (
    SELECT od.ProductID,
           SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.ProductID
),
scored AS (
    SELECT p.Sub_Category,
           p.ProductID,
           p.Product,
           p.Sale_Price,
           COALESCE(pu.units, 0)                                                    AS units_sold,
           PERCENT_RANK() OVER (PARTITION BY p.Sub_Category ORDER BY p.Sale_Price) AS price_pct_rank,
           COUNT(*)       OVER (PARTITION BY p.Sub_Category)                       AS subcat_products
    FROM Products AS p
    LEFT JOIN product_units AS pu ON pu.ProductID = p.ProductID
    WHERE p.Sale_Price   IS NOT NULL
      AND p.Sub_Category IS NOT NULL
),
subcat_median AS (
    SELECT s.Sub_Category,
           PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY s.units_sold) AS subcat_median_units
    FROM scored AS s
    GROUP BY s.Sub_Category
)
SELECT s.Sub_Category,
       s.ProductID,
       s.Product,
       s.Sale_Price,
       ROUND(s.price_pct_rank::numeric, 2) AS price_pct_rank,
       s.units_sold,
       sm.subcat_median_units
FROM scored AS s
JOIN subcat_median AS sm ON sm.Sub_Category = s.Sub_Category
WHERE s.subcat_products >= 20
  AND s.price_pct_rank  >= 0.75
  AND s.units_sold       < sm.subcat_median_units
ORDER BY s.Sub_Category, s.price_pct_rank DESC, s.ProductID;
```

Step 5 — Line-by-Line Explanation:

- `product_units` collapses lines to one row per product, so the LEFT JOIN in `scored` is one-to-one.
- `COALESCE(pu.units, 0)` gives unsold products an explicit 0. Without it their units would be NULL, `PERCENTILE_CONT` would ignore them, and the median would describe sellers only.
- `PERCENT_RANK() OVER (PARTITION BY p.Sub_Category ORDER BY p.Sale_Price)` returns `(rank - 1) / (rows - 1)`. The cheapest product scores 0. Tied prices share a rank and so a score: two products tied at the top price both score (n - 2) / (n - 1), just under 1 (0.9974 for the two ₹1,699 products in All Purpose Cleaners). Only a unique top price scores exactly 1.
- `COUNT(*) OVER (PARTITION BY ...)` puts the group size on every row without collapsing them, so the size filter can be applied in the final WHERE.
- `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ...)` is the median. It interpolates, so the median of an even-sized group can be a value like 7.5 that no product has. PostgreSQL does not allow it as a window function, which is why it sits in its own grouped CTE and is joined back.
- `s.units_sold < sm.subcat_median_units` is strict: a product exactly at the median is typical, not underperforming.

Step 6 — Sample Output:

sub_category	productid	product	sale_price	price_pct_rank	units_sold	subcat_median_units
All Purpose Cleaners	183	Dishwasher Tablets Powerball Classic	1699.00	1.00	2	3

All Purpose Cleaners	260	Original All-In-One Regular Dishwasher Tablets	1699.00	1.00	0	3
All Purpose Cleaners	131	IMPORTED - 3Ply Kitchen Towel Super Absorbent Paper Tissue Roll	1340.00	0.99	0	3
All Purpose Cleaners	116	Classic Detergent - Lemon, Imported	1299.00	0.99	1	3
All Purpose Cleaners	375	Anti-Bactarial Laundry Cleaner - Lavender	1149.00	0.99	0	3
All Purpose Cleaners	387	Wall Stickable Pack	1000.00	0.98	0	3
All Purpose Cleaners	273	Max Gel Sink & Pipe - Unblocker	799.00	0.98	1	3
All Purpose Cleaners	372	Cleaner & Polish - Stainless Steel	775.00	0.97	0	3

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 2617 rows.

Step 7 — Why This Solution Works: Each measure is computed at its own grain and only then brought together on the product row. The window function ranks every priced product in the sub-category, the grouped CTE produces one median per sub-category from the same zero-filled population, and the join attaches that median to each product. Because both measures come from `scored`, the rank and the median describe exactly the same set of products. The size threshold keeps the result to sub-categories big enough for "top quarter" and "median" to mean something.

Step 8 — Common Mistakes:

- Computing the median from `product_units` instead of `scored`. Unsold products vanish, the median rises to "the median seller", and far more products look like underperformers.
- Trying `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY units) OVER (PARTITION BY Sub_Category)`. PostgreSQL rejects it: ordered-set aggregates cannot be used as window functions.
- Joining `OrderDetails` before the window function. Every product row is repeated once per order line, so `PERCENT_RANK` ranks lines, not products, and best-sellers dominate the top of the price ranking.
- Using `NTILE(4) = 4` for "top quarter". NTILE cuts by row count, so two products with the same price can land in different quartiles. `PERCENT_RANK` treats ties identically.

Step 9 — Interview Follow-Up: Units sold depends on how long a product has been listed, and the schema has no listing date. What proxy could you use from the data you have, and how would you adjust the comparison?

Step 10 — Optimization or Alternative: A MySQL 8 version has `PERCENT_RANK()` but no `PERCENTILE_CONT`. Compute the median with `ROW_NUMBER()` and `COUNT(*)` per sub-category, keeping the middle one or two rows and averaging them:

```sql
WITH product_units AS (
    SELECT od.ProductID, SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    GROUP BY od.ProductID
),
ranked AS (
    SELECT p.Sub_Category,
           COALESCE(pu.units, 0) AS units_sold,
           ROW_NUMBER() OVER (PARTITION BY p.Sub_Category ORDER BY COALESCE(pu.units, 0), p.ProductID) AS rn,
           COUNT(*)     OVER (PARTITION BY p.Sub_Category) AS n
    FROM Products AS p
    LEFT JOIN product_units AS pu ON pu.ProductID = p.ProductID
)
SELECT r.Sub_Category,
       AVG(r.units_sold) AS subcat_median_units
FROM ranked AS r
WHERE r.rn IN (FLOOR((r.n + 1) / 2), FLOOR((r.n + 2) / 2))
GROUP BY r.Sub_Category
ORDER BY r.Sub_Category;
```

`FLOOR` is needed because MySQL's `/` returns a decimal; it makes both positions the same row for odd `n` and the two middle rows for even `n`. On the PostgreSQL side, the window functions in `scored` share one partition and sort, so the planner sorts the product list once.

Question 79: At which order does a customer cross 10,000 in spend?

Difficulty: Hard

Primary Topic: Running SUM per customer with an explicit ROWS frame

Combined Topics: First crossing with DISTINCT ON, Named WINDOW clause, Date difference from first order, NULL amounts

Business Scenario: The CRM team is designing a "Gold" tier that unlocks once a customer's lifetime order value reaches 10,000. Before they promise anything, they want to know how quickly customers get there: on which order number the threshold is crossed, on what date, and how many days after the first order. If most customers cross within a few months, the tier is a retention tool; if it takes years, it is a loyalty reward for a handful.

Problem Statement: For each customer whose cumulative `Total_Order_Amount` reaches at least 10,000, return the first order at which the running total reaches 10,000: its position in the customer's order sequence, its date, the days since the customer's first order, and the running total at that point. Customers who never reach 10,000 are left out. Sort by days to cross, fastest first.

Expected Output:

- `CustomerID`
- `crossing_order_number`: position of the crossing order (1 = first order)
- `crossing_order_date`: `OrderDate` of that order
- `days_from_first_order`: days between the first order and the crossing order
- `spend_at_crossing`: running total including the crossing order

Tables Required: `Orders`

SQL Concepts Tested: running `SUM` with an explicit `ROWS` frame, `ROWS` vs the default `RANGE` frame on tied dates, named `WINDOW`, picking the first qualifying row per group, date difference, NULL amounts in a running total.

Step 1 — Understand the Problem: A running total per customer gives, for every order, "lifetime spend up to and including this order". The crossing order is the first row where that value is at least 10,000.

Order sequence needs a tie-break, because same-day orders are common, so the sequence is `OrderDate`, then `OrderID`. The running total must use the same sequence and a `ROWS` frame. The default frame for `SUM(...) OVER (... ORDER BY OrderDate)` is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which adds every order on the same date at once. Two same-day orders then show the same running total, and the crossing can land one order too early.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` for the partition, `OrderDate` and `OrderID` for the sequence, `Total_Order_Amount` for spend. Lifetime spend is an order-grain number, so no lines are involved and no fan-out is possible.

Step 3 — Build the Query Logic:

1. `running`: for every order, its sequence number, the running spend with an explicit `ROWS` frame, and the customer's first order date.
2. `first_crossing`: rows with running spend of at least 10,000, reduced to the earliest one per customer.
3. Final select: the order number, date, days from first order and the running spend.

Step 4 — Final SQL Solution:

```sql
WITH running AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate,
           ROW_NUMBER() OVER customer_orders AS order_number,
           SUM(COALESCE(o.Total_Order_Amount, 0)) OVER (customer_orders
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_spend,
           MIN(o.OrderDate) OVER (PARTITION BY o.CustomerID)   AS first_order_date
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
    WINDOW customer_orders AS (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID)
),
first_crossing AS (
    SELECT DISTINCT ON (r.CustomerID)
           r.CustomerID,
           r.order_number,
           r.OrderDate,
           r.first_order_date,
           r.running_spend
    FROM running AS r
    WHERE r.running_spend >= 10000
    ORDER BY r.CustomerID, r.order_number
)
SELECT fc.CustomerID,
       fc.order_number                   AS crossing_order_number,
       fc.OrderDate                      AS crossing_order_date,
       fc.OrderDate - fc.first_order_date AS days_from_first_order,
       fc.running_spend                  AS spend_at_crossing
FROM first_crossing AS fc
ORDER BY days_from_first_order, fc.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `WINDOW customer_orders AS (...)` defines partition and sort once. `ROW_NUMBER` uses it as is.
- `OVER (customer_orders ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` reuses the named window and adds a frame. PostgreSQL allows adding a frame to a named window that has none.
- `SUM(COALESCE(o.Total_Order_Amount, 0))` treats a missing amount as no spend and makes that choice visible to a reviewer.
- `MIN(o.OrderDate) OVER (PARTITION BY o.CustomerID)` has no `ORDER BY`, so it sees the full partition: the first order date on every row.
- `WHERE r.running_spend >= 10000` keeps only rows at or over the threshold. Filtering on a window result has to happen in the next CTE, never in the level where the window is computed.
- `DISTINCT ON (r.CustomerID) ... ORDER BY r.CustomerID, r.order_number` keeps the earliest qualifying row per customer.
- `fc.OrderDate - fc.first_order_date` is an integer number of days, because both are dates.

Step 6 — Sample Output:

customerid	crossing_order_number	crossing_order_date	days_from_first_order	spend_at_crossing
100202	1	2025-10-09	0	12948.00
100588	1	2024-11-09	0	11453.00
100595	1	2023-08-16	0	10057.00
100699	1	2025-01-19	0	10583.00

100707	1	2024-05-16	0	17124.00
101040	1	2024-11-04	0	13500.00
101266	1	2025-11-12	0	13326.00
102049	1	2025-06-05	0	10072.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 1710 rows.

Step 7 — Why This Solution Works: The running total and the sequence number come from the same ordering, and the `ROWS` frame adds exactly one order per row. So the running total on row n is the sum of the first n orders, and the first row over the threshold is the order that pushed the customer across. Customers who never reach 10,000 have no qualifying rows and drop out naturally.

Edge cases:

- Crossing on the first order: order number 1 and 0 days. That is a real outcome, usually one large order.
- Same-day orders: the `OrderID` tie-break orders them, and the `ROWS` frame counts them one at a time.
- One-order customers: included only if that order alone is at least 10,000.
- NULL or zero headers: they add nothing and still take a sequence number. If a header is wrong but its lines are fine, the customer crosses later than they should. Reconcile headers first if this report drives rewards.
- Orphan `CustomerID`: still appears. LEFT JOIN `Customers` before handing the list to CRM so missing profiles are visible.

Step 8 — Common Mistakes:

- Using the default frame. This query lists every order where the default `RANGE` running total disagrees with the `ROWS` running total; each one is a same-date tie where the crossing could be reported an order early:

```
WITH both_frames AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate,
           SUM(o.Total_Order_Amount) OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate) AS running_range_default,
           SUM(o.Total_Order_Amount) OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID
                                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_rows
    FROM Orders AS o
)
SELECT bf.CustomerID, bf.OrderID, bf.OrderDate, bf.running_range_default, bf.running_rows
FROM both_frames AS bf
WHERE bf.running_range_default <> bf.running_rows
ORDER BY bf.CustomerID, bf.OrderDate, bf.OrderID;
```

- Taking `MIN(OrderDate)` over all rows with running spend of at least 10,000 and calling it the crossing order. If two orders share that date, you still do not know which one crossed, and the order number is lost.
- Computing the running total over all customers at once by forgetting `PARTITION BY`. The threshold is then crossed by the business, not by a customer.

Step 9 — Interview Follow-Up: The tier rule changes to "10,000 within any rolling 365 days". Which frame clause do you need, and why must it be `RANGE` with an interval instead of `ROWS`?

Step 10 — Optimization or Alternative: Instead of `DISTINCT ON` (PostgreSQL-only), also compute the running total before the current order. The crossing order is the single row where the previous total is under 10,000 and the current total is at or over it, so no second ranking step is needed:

```
WITH running AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate,
           ROW_NUMBER() OVER w                                          AS order_number,
           SUM(COALESCE(o.Total_Order_Amount, 0)) OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS spend_after,
           COALESCE(SUM(COALESCE(o.Total_Order_Amount, 0)) OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING), 0) AS
spend_before,
           MIN(o.OrderDate) OVER (PARTITION BY o.CustomerID)     AS first_order_date
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
    WINDOW w AS (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID)
)
SELECT r.CustomerID,
       r.order_number                  AS crossing_order_number,
       r.OrderDate                     AS crossing_order_date,
       r.OrderDate - r.first_order_date AS days_from_first_order,
       r.spend_after                   AS spend_at_crossing
FROM running AS r
WHERE r.spend_before < 10000
  AND r.spend_after >= 10000
ORDER BY days_from_first_order, r.CustomerID;
```

The outer `COALESCE` matters: for the first order the frame ending at `1 PRECEDING` is empty and `SUM` returns NULL. This version depends on amounts never being negative; if refunds were stored as negative orders, a customer could cross twice and the `DISTINCT ON` version would be the safe one.

Question 80: Sub-category repurchase rate within 90 days

Difficulty: Hard

Primary Topic: Correlated EXISTS with a later-date condition

Combined Topics: Multiple CTEs, DISTINCT at the customer-sub-category-day grain, Observation window with anchored cut-off, HAVING minimum sample, Semi-join vs per-row subplan

Business Scenario: The merchandising team is choosing where to launch "subscribe and save". A sub-category is a good fit when people who buy from it come back for it soon: skin care or spices, rather than crockery. They define the repurchase rate as: of the customers who bought from the sub-category, the share who bought from it again on a later day within 90 days of their first purchase there. They work at sub-category level because a subscription is sold per need (tea, detergent, shampoo), not per SKU, and because most of the 22,000 products have too few buyers to judge on their own. Sub-categories with fewer than 30 eligible buyers are too small to judge.

Problem Statement: For each `Sub_Category`, count its eligible buyers and how many of them bought from it again on a later day within 90 days of their first purchase there. A buyer is eligible only if that first purchase is at least 90 days before the latest `OrderDate` in the data, so every buyer has had a full 90 days to come back. Return sub-categories with at least 30 eligible buyers, sorted by repurchase rate, highest first.

Expected Output:

- `Sub_Category`
- `buyers`: eligible customers who bought from the sub-category
- `repeat_buyers`: of those, customers who bought from it again on a later day within 90 days
- `repurchase_rate_pct`: `repeat_buyers` as a percentage of `buyers`, 2 decimals

Tables Required: `Orders`, `OrderDetails`, `Products`

SQL Concepts Tested: correlated `EXISTS` with a range condition on the outer row, de-duplicating to the grain of the question, an anchored observation window, minimum sample size with HAVING, and why the position of `EXISTS` decides whether PostgreSQL can turn it into a semi-join.

Step 1 — Understand the Problem: The unit is a customer and sub-category pair. For each pair we need one fact: did a later purchase happen within 90 days? "Later" means a later day. Two lines from the same sub-category in one order, or two orders on the same day, are one shopping trip, not a repurchase.

The 90-day window and the eligibility cut-off deal with partial periods. A customer who first bought from a sub-category last week has had no chance to come back, and counting them as a non-repeater drags the rate down for every sub-category that grew recently. So everyone gets the same 90 days, and anyone who has not had them yet is left out. The cut-off is `MAX(OrderDate) - 90`, not `CURRENT_DATE - 90`.

Step 2 — Identify the Tables and Relationships: `OrderDetails` says which product was in which order. `Orders` says who bought and when (`OrderDetails.OrderID` → `Orders.OrderID`, many lines to one order). `Products` maps each product to its `Sub_Category` (`OrderDetails.ProductID` → `Products.ProductID`). Inner joins are right here: a line with no order has no customer or date, and a line with no product has no sub-category, so neither can be judged.

Step 3 — Build the Query Logic:

1. `anchor`: the latest order date.
2. `purchase_days`: distinct (customer, sub-category, day) rows. This removes same-trip repeats.
3. `first_buys`: each customer's first purchase day per sub-category.
4. `eligible`: first buys at least 90 days before the anchor.
5. `repeaters`: eligible pairs for which a correlated `EXISTS` finds a later day within 90 days. The `EXISTS` sits in a WHERE clause on purpose (Step 10).
6. Final select: LEFT JOIN eligible pairs to repeaters, count both per sub-category, keep groups with at least 30 buyers.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate) AS last_day
    FROM Orders AS o
),
purchase_days AS (
    SELECT DISTINCT o.CustomerID, p.Sub_Category, o.OrderDate AS purchase_date
    FROM OrderDetails AS od
    JOIN Orders   AS o ON o.OrderID   = od.OrderID
    JOIN Products AS p ON p.ProductID = od.ProductID
),
first_buys AS (
    SELECT pd.Sub_Category, pd.CustomerID, MIN(pd.purchase_date) AS first_purchase_date
    FROM purchase_days AS pd
    GROUP BY pd.Sub_Category, pd.CustomerID
),
eligible AS (
    SELECT fb.*
    FROM first_buys AS fb
    CROSS JOIN anchor AS a
    WHERE fb.first_purchase_date <= a.last_day - 90
),
repeaters AS (
    SELECT e.Sub_Category, e.CustomerID
    FROM eligible AS e
    WHERE EXISTS (
        SELECT 1
        FROM purchase_days AS later
        WHERE later.CustomerID     = e.CustomerID
          AND later.Sub_Category   = e.Sub_Category
          AND later.purchase_date  > e.first_purchase_date
          AND later.purchase_date <= e.first_purchase_date + 90
    )
)
SELECT e.Sub_Category,
       COUNT(*)                          AS buyers,
       COUNT(r.CustomerID)               AS repeat_buyers,
       ROUND(100.0 * COUNT(r.CustomerID) / NULLIF(COUNT(*), 0), 2) AS repurchase_rate_pct
FROM eligible AS e
LEFT JOIN repeaters AS r
       ON r.Sub_Category = e.Sub_Category
      AND r.CustomerID   = e.CustomerID
GROUP BY e.Sub_Category
HAVING COUNT(*) >= 30
ORDER BY repurchase_rate_pct DESC, buyers DESC, e.Sub_Category;
```

Step 5 — Line-by-Line Explanation:

- `purchase_days` uses `SELECT DISTINCT` on customer, sub-category and day. Buying two shampoos in one order, or in two orders on the same day, becomes one row, so a trip can never count as its own repurchase.
- `first_buys` takes `MIN(purchase_date)` per customer and sub-category. The 90-day clock starts here.
- `WHERE fb.first_purchase_date <= a.last_day - 90` is the eligibility rule. Subtracting an integer from a DATE moves it back that many days.
- The `EXISTS` subquery is correlated: it refers to `e.CustomerID`, `e.Sub_Category` and `e.first_purchase_date` from the outer row. It stops at the first match, so a customer with five later trips still counts once.
- `later.purchase_date > e.first_purchase_date` is strict, which is what makes "later day" mean a different day.
- `LEFT JOIN repeaters` keeps every eligible pair. `COUNT(*)` counts buyers and `COUNT(r.CustomerID)` counts only pairs that found a match, because the unmatched side is NULL.
- `HAVING COUNT(*) >= 30` applies the minimum sample after grouping.

Step 6 — Sample Output:

sub_category	buyers	repeat_buyers	repurchase_rate_pct
Skin Care	2113	1041	49.27
Bath & Hand Wash	1970	865	43.91
Ready To Cook & Eat	1629	643	39.47
Hair Care	1730	682	39.42
Masalas & Spices	1735	666	38.39
Snacks & Namkeen	1430	507	35.45
Dairy	1501	509	33.91
Biscuits & Cookies	1485	482	32.46

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 87 rows.

Step 7 — Why This Solution Works: The work is done at the customer-sub-category grain, where the question lives. De-duplicating to days first means the `EXISTS` test answers exactly "was there another trip", and the eligibility cut-off gives every counted buyer the same 90 days, so a long-established sub-category and a fast-growing one are judged on equal terms.

Edge cases:

- Customers with one order: they appear as buyers and can never be repeat buyers, which is correct.

- Sub-categories at the bottom of the list with 0%: their buyers never came back within 90 days. With 30 or more eligible buyers that is a real signal, not noise.
- Duplicate customer accounts: one person on two IDs looks like two one-time buyers. Merge accounts first if you know them (Q71, Q92).

Step 8 — Common Mistakes:

- Counting "bought in more than one order" with no date condition. Same-day trips count as repeats, and there is no observation window:

```sql
SELECT pc.Sub_Category,
       COUNT(*)                                AS buyers,
       COUNT(*) FILTER (WHERE pc.orders >= 2) AS repeat_buyers_wrong
FROM (
    SELECT p.Sub_Category, o.CustomerID, COUNT(DISTINCT o.OrderID) AS orders
    FROM OrderDetails AS od
    JOIN Orders   AS o ON o.OrderID   = od.OrderID
    JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY p.Sub_Category, o.CustomerID
) AS pc
GROUP BY pc.Sub_Category
ORDER BY pc.Sub_Category;
```

Here 337 (customer, sub-category, day) combinations hold two or more orders, and this version counts each as a repeat whether or not the customer ever came back.

- Joining the purchase table to itself without `DISTINCT` or `EXISTS`. A customer with five later trips appears five times, and `COUNT(*)` inflates repeat buyers.
- Writing the cut-off as `CURRENT_DATE - 90`, which makes the result change daily and breaks on historical extracts.
- Running the same logic per product. Outside the best sellers most products have only a few buyers, so their rates swing between 0% and 100% and the minimum-sample rule removes most of the catalogue.

Step 9 — Interview Follow-Up: The team wants the median days between a customer's first and second purchase in each sub-category, to set the subscription interval. Which window function gets you the second purchase date, and how do you keep the same eligibility rule?

Step 10 — Optimization or Alternative: Where `EXISTS` sits changes the plan. Written as a column (`EXISTS (...) AS bought_again` in the SELECT list), PostgreSQL runs it as a subplan once per outer row; against a CTE with no index that is a scan of tens of thousands of rows for each of tens of thousands of pairs, which on this dataset ran past our 30-second test limit. Written in a WHERE clause, as in Step 4, the planner can turn it into a hash semi-join and the whole query finishes in well under a second here. `LEAD` gives the same answer in one sorted pass, with no lookups at all:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate) AS last_day FROM Orders AS o
),
purchase_days AS (
    SELECT DISTINCT o.CustomerID, p.Sub_Category, o.OrderDate AS purchase_date
    FROM OrderDetails AS od
    JOIN Orders   AS o ON o.OrderID   = od.OrderID
    JOIN Products AS p ON p.ProductID = od.ProductID
),
sequenced AS (
    SELECT pd.Sub_Category,
           pd.CustomerID,
           pd.purchase_date,
           ROW_NUMBER() OVER (PARTITION BY pd.CustomerID, pd.Sub_Category ORDER BY pd.purchase_date) AS purchase_seq,
           LEAD(pd.purchase_date) OVER (PARTITION BY pd.CustomerID, pd.Sub_Category ORDER BY pd.purchase_date) AS
next_purchase_date
    FROM purchase_days AS pd
)
SELECT s.Sub_Category,
       COUNT(*)                                                                AS buyers,
       COUNT(*) FILTER (WHERE s.next_purchase_date <= s.purchase_date + 90) AS repeat_buyers
FROM sequenced AS s
CROSS JOIN anchor AS a
WHERE s.purchase_seq = 1
  AND s.purchase_date <= a.last_day - 90
GROUP BY s.Sub_Category
HAVING COUNT(*) >= 30
ORDER BY s.Sub_Category;
```

Days are already distinct, so `next_purchase_date` is always a later day; a NULL next date fails the `FILTER` and counts as not repeating. For lookup-style access on a large table, indexes on `Orders(CustomerID, OrderDate)` and `OrderDetails(OrderID)` keep the `EXISTS` probes cheap.

Question 81: Year-to-date vs same period last year

Difficulty: Hard

Primary Topic: Anchored date ranges with conditional aggregation

Combined Topics: Multiple CTEs, Interval arithmetic and 29 February, FILTER for two periods in one pass, Growth percentage

Business Scenario: The finance team sends the board a one-line trading update: orders and order value so far this year, against the same stretch of days last year. Comparing a partial current year with the whole of last year makes every update before December look like a decline. So "same period last year" means from 1 January of the prior year to the same calendar day as the latest order date, one year earlier.

Problem Statement: Using the last order date in the data as the end of the current year-to-date (YTD) period, return one row with the YTD start and end, the matching prior-year start and end, orders and order value in each period, and the growth in order value as a percentage. When the YTD end is 29 February, the prior period must end on 28 February. Count every order in `Orders` as placed; cancellations are not modelled.

Expected Output:

- `ytd_start`: 1 January of the anchor year
- `ytd_end`: the last order date in the data
- `prior_ytd_start`, `prior_ytd_end`: the same dates one year earlier
- `ytd_orders`, `ytd_value`: order count and `SUM(Total_Order_Amount)` in the current period
- `prior_ytd_orders`, `prior_ytd_value`: the same for the prior period
- `value_growth_pct`: percentage change in order value

Tables Required: `Orders`

SQL Concepts Tested: anchoring to `MAX(OrderDate)`, `DATE_TRUNC('year', ...)`, `INTERVAL '1 year'` arithmetic and leap days, half-open date ranges, conditional aggregation with `FILTER`, growth percentage with `NULLIF`.

Step 1 — Understand the Problem: The output is one row, but there are two date ranges to get right before any counting starts. The current range is from 1 January of the anchor year to the anchor date. The prior range shifts both ends back one year.

"Back one year" is where candidates slip. Subtracting 365 days is wrong in any range that crosses 29 February: from 1 March 2024 it lands on 2 March 2023. `INTERVAL '1 year'` moves the calendar date, and when the target date does not exist (29 February 2023) PostgreSQL clamps it to the last valid day of that month, 28 February. That clamping is exactly the rule the finance team asked for.

The anchor is the last order date, not today. If the extract stopped on some earlier date, `CURRENT_DATE` would compare a period with missing data against a full one.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `OrderDate` places each order in a period, `OrderID` is counted, and `Total_Order_Amount` is summed. Everything stays at the order grain, so there is no fan-out risk.

Step 3 — Build the Query Logic:

1. `anchor`: the last order date.
2. `periods`: four dates computed once from the anchor.
3. `totals`: LEFT JOIN `Orders` to the periods on the full span from prior start to current end, then use `FILTER` to count and sum each period separately.
4. Final select: growth as a percentage of the prior value.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS ytd_end
    FROM Orders AS o
),
periods AS (
    SELECT DATE_TRUNC('year', a.ytd_end)::date                      AS ytd_start,
           a.ytd_end,
           (DATE_TRUNC('year', a.ytd_end) - INTERVAL '1 year')::date AS prior_ytd_start,
           (a.ytd_end - INTERVAL '1 year')::date                     AS prior_ytd_end
    FROM anchor AS a
),
totals AS (
    SELECT pr.ytd_start,
           pr.ytd_end,
           pr.prior_ytd_start,
           pr.prior_ytd_end,
           COUNT(o.OrderID) FILTER (WHERE o.OrderDate >= pr.ytd_start
                                    AND o.OrderDate <  pr.ytd_end + 1)       AS ytd_orders,
           COALESCE(SUM(o.Total_Order_Amount) FILTER (WHERE o.OrderDate >= pr.ytd_start
                                                     AND o.OrderDate <  pr.ytd_end + 1), 0) AS ytd_value,
           COUNT(o.OrderID) FILTER (WHERE o.OrderDate >= pr.prior_ytd_start
                                    AND o.OrderDate <  pr.prior_ytd_end + 1) AS prior_ytd_orders,
           COALESCE(SUM(o.Total_Order_Amount) FILTER (WHERE o.OrderDate >= pr.prior_ytd_start
                                                     AND o.OrderDate <  pr.prior_ytd_end + 1), 0) AS prior_ytd_value
    FROM periods AS pr
    LEFT JOIN Orders AS o
           ON o.OrderDate >= pr.prior_ytd_start
          AND o.OrderDate <  pr.ytd_end + 1
    GROUP BY pr.ytd_start, pr.ytd_end, pr.prior_ytd_start, pr.prior_ytd_end
)
SELECT t.ytd_start,
       t.ytd_end,
       t.prior_ytd_start,
       t.prior_ytd_end,
       t.ytd_orders,
       t.ytd_value,
       t.prior_ytd_orders,
       t.prior_ytd_value,
       ROUND(100.0 * (t.ytd_value - t.prior_ytd_value) / NULLIF(t.prior_ytd_value, 0), 2) AS value_growth_pct
FROM totals AS t
ORDER BY t.ytd_start;
```

Step 5 — Line-by-Line Explanation:

- `MAX(o.OrderDate)::date` gives the anchor as a date, even if the column were a timestamp.

- `DATE_TRUNC('year', a.ytd_end)::date` is 1 January of the anchor year. `DATE_TRUNC` returns a timestamp, hence the cast.
- `(a.ytd_end - INTERVAL '1 year')::date` shifts the calendar date back one year, clamping 29 February to 28 February.
- The LEFT JOIN brings in every order from the prior start to the current end. Orders that fall between the two periods are joined but matched by neither `FILTER`, so they are ignored.
- `o.OrderDate < pr.ytd_end + 1` is the half-open form of "on or before `ytd_end`". It keeps working if `OrderDate` becomes a timestamp.
- `COUNT(o.OrderID)` counts real orders only. Because `periods` drives the LEFT JOIN, a span with no orders still returns one row with zeros instead of no row at all.
- `NULLIF(t.prior_ytd_value, 0)` returns NULL growth when there was nothing last year, which is honest: growth from zero is undefined.

Step 6 — Sample Output:

ytd_start	ytd_end	prior_ytd_start	prior_ytd_end	ytd_orders	ytd_value	prior_ytd_orders	prior_ytd_value	value_growth_pct
2025-01-01	2025-12-28	2024-01-01	2024-12-28	28658	37506098.66	12524	16585539.56	126.14

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 1 row returned.

Step 7 — Why This Solution Works: Both periods are derived from one anchor with calendar arithmetic, so they always cover the same calendar days in their own years. Conditional aggregation computes both periods from one scan of `Orders`, which guarantees the two halves of the comparison use identical rules (same NULL handling, same order definition).

Edge cases:

- 29 February as the YTD end: the prior period ends on 28 February. The current period has one more day than the prior one, which is unavoidable and worth one line in the board note.
- 28 February in a year after a leap year: the prior end is 28 February, so the leap day is left out and both periods have 59 days.
- A partial final day: if the extract was taken mid-day, the anchor day is incomplete in the current period but complete in the prior one. Ending at the previous full day is safer for daily reporting.
- NULL `Total_Order_Amount`: counted as an order, adds nothing to value.
- No prior-year orders at all: `prior_ytd_value` is 0 and growth is NULL.

Step 8 — Common Mistakes:

- Shifting the end date by 365 days. Compare the two methods across the leap day:

```
SELECT v.ytd_end,
       (v.ytd_end - INTERVAL '1 year')::date AS prior_end_interval,
       v.ytd_end - 365                       AS prior_end_minus_365
FROM (VALUES (DATE '2024-02-28'), (DATE '2024-02-29'), (DATE '2024-03-01'),
             (DATE '2025-02-28'), (DATE '2025-03-01')) AS v(ytd_end)
ORDER BY v.ytd_end;
```

From 1 March 2024, minus 365 lands on 2 March 2023, so the prior period silently gains a day.

- Building the date with `MAKE_DATE(year - 1, month, day)`. It raises "date field value out of range" when the anchor is 29 February.
- Comparing `EXTRACT(YEAR FROM o.OrderDate) = 2024` with `= 2023`. That is a partial year against a full year.
- Using `BETWEEN pr.ytd_start AND pr.ytd_end` on a timestamp column, which drops the last day after midnight.

Step 9 — Interview Follow-Up: Finance also wants month-to-date and quarter-to-date against the same periods last year, in the same result. How would you generate the periods so the counting query does not change?

Step 10 — Optimization or Alternative: Put the periods in rows instead of columns. One `periods` CTE with a label, start and end per period, joined once to `Orders` and grouped by label, scales to MTD, QTD and YTD without adding more `FILTER` clauses:

```
WITH anchor AS (
    SELECT MAX(o.OrderDate)::date AS ytd_end
    FROM Orders AS o
),
periods AS (
    SELECT 'Current YTD' AS period,
           DATE_TRUNC('year', a.ytd_end)::date AS period_start,
           a.ytd_end                           AS period_end
    FROM anchor AS a
    UNION ALL
    SELECT 'Prior YTD',
           (DATE_TRUNC('year', a.ytd_end) - INTERVAL '1 year')::date,
           (a.ytd_end - INTERVAL '1 year')::date
    FROM anchor AS a
)
SELECT pr.period,
       pr.period_start,
       pr.period_end,
       COUNT(o.OrderID)                       AS orders,
       COALESCE(SUM(o.Total_Order_Amount), 0) AS order_value
FROM periods AS pr
LEFT JOIN Orders AS o
       ON o.OrderDate >= pr.period_start
      AND o.OrderDate <  pr.period_end + 1
GROUP BY pr.period, pr.period_start, pr.period_end
ORDER BY pr.period_start DESC;
```

Either form uses a plain range predicate on `OrderDate`, so an index on `Orders(OrderDate)` serves it. On MySQL, `DATE_SUB(d, INTERVAL 1 YEAR)` clamps 29 February the same way.

Question 82: Each customer's favourite category

Difficulty: Hard

Primary Topic: ROW_NUMBER over aggregates with a deterministic tie-break

Combined Topics: Multiple CTEs, Window SUM for share of customer units, LEFT JOIN for unmapped products, Multi-table join

Business Scenario: The personalisation team is building the home-page banner. Each customer sees one category, their favourite, and the banner system accepts exactly one category per customer. "Favourite" means the category in which the customer bought the most units across all their orders. The team also wants to know how dominant that category is, because a favourite that holds 25% of a customer's units is a weaker signal than one that holds 80%.

Problem Statement: For every customer who has order lines, return exactly one favourite category: the one with the most units. Break ties by the number of the customer's orders that contained the category, then by the most recent order date in that category, then alphabetically by category name. Show the units in that category and their share of the customer's total units. Lines whose product or category is missing go into a category labelled `Unmapped`.

Expected Output:

- `CustomerID`
- `favourite_category`: `CategoryName`, or `Unmapped`
- `units_in_category`: `SUM(Quantity)` in that category
- `pct_of_customer_units`: that as a percentage of the customer's total units

Tables Required: `Orders`, `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: aggregating to customer-category grain before ranking, `ROW_NUMBER` with a business tie-break, window `SUM` over the same partition for a share, LEFT JOINs to keep unmapped products, NULL quantities in ranking.

Step 1 — Understand the Problem: This is top-1 per group, but the thing being ranked does not exist as a row yet. First build one row per customer per category with the units. Then rank categories within each customer and keep rank 1.

The banner needs exactly one row per customer, so the function is `ROW_NUMBER`, not `RANK`. `RANK` would return two favourites for a customer with a tie, and the banner system would pick one at random. Ties are common for light buyers: a customer with two orders of one unit each in different categories is a tie on units. The tie-break is a business choice, so it is stated in the problem: more separate orders containing the category is a stronger habit, and a more recent purchase is a fresher signal. The category name only makes the result repeatable.

Step 2 — Identify the Tables and Relationships: `Orders` gives `CustomerID`. `OrderDetails` gives `ProductID` and `Quantity` (`OrderDetails.OrderID` → `Orders.OrderID`, one order to many lines). `Products` maps a product to `CategoryID`, and `Category` gives `CategoryName`. Products missing from the catalogue, or pointing at a category that does not exist, must stay in the totals, so both lookups are LEFT JOINs. `Orders` to `OrderDetails` is an inner join on purpose: an order with no lines has no category to contribute.

Step 3 — Build the Query Logic:

1. `customer_category`: group lines by customer and category. Compute units, orders containing the category, and last purchase date.
2. `ranked`: within each customer, `ROW_NUMBER` by units, then orders, then recency, then name; also `SUM(units) OVER (PARTITION BY CustomerID)` for the customer total.
3. Keep rank 1 and compute the share.

Step 4 — Final SQL Solution:

```sql
WITH customer_category AS (
    SELECT o.CustomerID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           SUM(od.Quantity)                       AS units,
           COUNT(DISTINCT o.OrderID)              AS orders_with_category,
           MAX(o.OrderDate)                       AS last_bought
    FROM Orders AS o
    JOIN OrderDetails  AS od  ON od.OrderID     = o.OrderID
    LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID, COALESCE(cat.CategoryName, 'Unmapped')
),
ranked AS (
    SELECT cc.CustomerID,
           cc.category_name,
           cc.units,
           SUM(cc.units) OVER (PARTITION BY cc.CustomerID) AS customer_units,
           ROW_NUMBER() OVER (PARTITION BY cc.CustomerID
                              ORDER BY cc.units DESC NULLS LAST,
                                       cc.orders_with_category DESC,
                                       cc.last_bought DESC,
                                       cc.category_name) AS category_rank
    FROM customer_category AS cc
)
SELECT r.CustomerID,
       r.category_name AS favourite_category,
       r.units         AS units_in_category,
       ROUND(100.0 * r.units / NULLIF(r.customer_units, 0), 2) AS pct_of_customer_units
FROM ranked AS r
WHERE r.category_rank = 1
ORDER BY r.CustomerID;
```

Step 5 — Line-by-Line Explanation:

- `COALESCE(cat.CategoryName, 'Unmapped')` appears in both `SELECT` and `GROUP BY`, so all unmapped lines for a customer form one group instead of disappearing.
- `SUM(od.Quantity)` is safe here: we are summing a line-level column at the line grain, with no order-level column involved.
- `COUNT(DISTINCT o.OrderID)` counts orders, not lines. A customer who bought three products from one category in a single order has one order with that category.
- `SUM(cc.units) OVER (PARTITION BY cc.CustomerID)` puts the customer's total units on every category row. It is computed before the rank filter, so the denominator covers all categories.
- `ORDER BY cc.units DESC NULLS LAST` keeps a category whose units are all NULL from ranking first. In PostgreSQL, `DESC` puts NULLs first by default.
- The last three sort keys are the tie-break chain. `cc.category_name` guarantees no two rows are equal, so the result is the same on every run.
- `NULLIF(r.customer_units, 0)` guards a customer whose quantities are all NULL or 0.

Step 6 — Sample Output:

customerid	favourite_category	units_in_category	pct_of_customer_units
99001	Beauty & Hygiene	17	45.95
99003	Foodgrains, Oil & Masala	4	33.33
100001	Foodgrains, Oil & Masala	16	40.00
100002	Snacks & Branded Foods	19	22.09
100003	Beauty & Hygiene	6	85.71
100004	Beauty & Hygiene	60	38.71
100005	Foodgrains, Oil & Masala	21	33.33
100006	Snacks & Branded Foods	1	100.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 4427 rows.

Step 7 — Why This Solution Works: Ranking happens on rows that are already at the grain of the decision (customer and category), so each candidate category appears exactly once per customer. The share uses a window over the same partition before filtering, so it compares the winner with everything the customer bought. With a fully determined sort, `ROW_NUMBER` returns exactly one row per customer.

Edge cases:

- Customers with one order and one category: the favourite is that category at 100%. That is a weak signal; the share column shows it.
- Ties on units: resolved by the stated chain, not by luck.
- Orphan products and products in a category with no `Category` row: grouped as `Unmapped`. If `Unmapped` wins for a customer, the catalogue mapping needs fixing before the banner goes live.
- Duplicate order lines inflate units for that category. Deduplicate `OrderDetails` first if you know they exist.
- Orphan `CustomerID` values still get a favourite. The banner cannot target them; join `Customers` with a LEFT JOIN to see them.

Step 8 — Common Mistakes:

- Joining back on `MAX(units)`. Every tied category survives, so customers get two or three favourites:

```
WITH customer_category AS (
    SELECT o.CustomerID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           SUM(od.Quantity)                       AS units
    FROM Orders AS o
    JOIN OrderDetails  AS od  ON od.OrderID     = o.OrderID
    LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    GROUP BY o.CustomerID, COALESCE(cat.CategoryName, 'Unmapped')
),
top_units AS (
    SELECT cc.CustomerID, MAX(cc.units) AS max_units
    FROM customer_category AS cc
    GROUP BY cc.CustomerID
)
SELECT cc.CustomerID, cc.category_name AS favourite_category, cc.units
FROM customer_category AS cc
JOIN top_units AS tu ON tu.CustomerID = cc.CustomerID AND tu.max_units = cc.units
ORDER BY cc.CustomerID, cc.category_name;
```

- Ranking with `ORDER BY units DESC` only. Ties are broken by whatever order the rows arrive in, so a customer's banner can change between runs with no new data.
- Using `COUNT(*)` of lines as "orders". A basket of three snack products would count as three snack orders.
- INNER JOIN to `Category`, which drops unmapped products from the customer's total and inflates the winner's share.

Step 9 — Interview Follow-Up: The team wants the favourite based on the last 180 days only, but falling back to the all-time favourite for customers with no orders in that window. How would you structure the CTEs so each customer still gets exactly one row?

Step 10 — Optimization or Alternative: PostgreSQL's `DISTINCT ON` does top-1 per group with the same tie-break and no explicit rank column. The share then comes from a window computed in the same level, since `DISTINCT ON` is applied after window functions:

```sql
WITH customer_category AS (
    SELECT o.CustomerID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           SUM(od.Quantity)                       AS units,
           COUNT(DISTINCT o.OrderID)              AS orders_with_category,
           MAX(o.OrderDate)                       AS last_bought
    FROM Orders AS o
    JOIN OrderDetails  AS od  ON od.OrderID     = o.OrderID
    LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID, COALESCE(cat.CategoryName, 'Unmapped')
)
SELECT DISTINCT ON (cc.CustomerID)
       cc.CustomerID,
       cc.category_name AS favourite_category,
       cc.units         AS units_in_category,
       ROUND(100.0 * cc.units / NULLIF(SUM(cc.units) OVER (PARTITION BY cc.CustomerID), 0), 2) AS pct_of_customer_units
FROM customer_category AS cc
ORDER BY cc.CustomerID, cc.units DESC NULLS LAST, cc.orders_with_category DESC,
         cc.last_bought DESC, cc.category_name;
```

`DISTINCT ON` is PostgreSQL-only; on MySQL or SQL Server use the `ROW_NUMBER` version. Both need the same complete `ORDER BY`; without it, `DISTINCT ON` picks an arbitrary row just as a bare `ROW_NUMBER` would.

Question 83: Underperforming sub-categories in the last six months

Difficulty: Hard

Primary Topic: PERCENTILE_CONT per group as a relative threshold

Combined Topics: Anchored date window, LEFT JOIN to keep zero sellers, Normalising by group size, Multiple CTEs, Flag normalisation

Business Scenario: Category managers review their ranges twice a year and decide where to cut listings or push promotions. A fixed cut-off like "fewer than 500 units" does not work: 500 units is a lot for water and nothing for skin care. Raw units are also unfair to small sub-categories, which simply list fewer products. So each category gets its own bar, measured per listed product: a sub-category underperforms if its units sold per listed product in the last six months fall below the 25th percentile of that figure across the sub-categories of the same category. Only active categories are reviewed.

Problem Statement: For every `Sub_Category` in an active category, compute units sold in the six months ending on the latest `OrderDate` in the data (0 if none), the number of products listed, and units per listed product. Compute each category's 25th percentile of units per listed product. Return sub-categories strictly below their category's threshold. Sort by category, then units per product, lowest first.

Expected Output:

- `CategoryName`, `Sub_Category`
- `listed_products`: products in the sub-category
- `units_last_6m`: `SUM(Quantity)` in the window, 0 when nothing sold
- `units_per_product`: `units_last_6m / listed_products`, 2 decimals
- `category_p25`: the category's 25th percentile of `units_per_product`, 2 decimals

Tables Required: `Orders`, `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: anchoring a rolling window to the data, aggregating lines to the product grain and then to the sub-category, keeping zero sellers with a LEFT JOIN, `PERCENTILE_CONT` as a per-group threshold, joining a threshold back to the rows it judges.

Step 1 — Understand the Problem: The threshold is relative to the category, so it must be computed from the same population it judges: every sub-category of an active category, including any that sold nothing in the window. The unit of judgment matters as much as the threshold. At product level this catalogue is too sparse to rank: 46% of products sold nothing in the last six months, so in 8 of the 12 categories the product-level 25th percentile is 0 and "below the 25th percentile" can never be true, while in the others a product is judged on one or two units. One level up, each sub-category has enough sales to compare, and dividing by listed products stops big sub-categories winning on size alone.

Step 2 — Identify the Tables and Relationships: `Orders` supplies `OrderDate` for the window. `OrderDetails` supplies `Quantity` (`OrderDetails.OrderID` → `Orders.OrderID`). `Products` maps each line to `Sub_Category` and `CategoryID` (`OrderDetails.ProductID` → `Products.ProductID`). `Category` supplies the name and the `Active` flag (`Products.CategoryID` → `Category.CategoryID`).

Step 3 — Build the Query Logic:

1. `bounds`: the anchor, `MAX(OrderDate)`, and the window start six months earlier.
2. `recent_units`: units per `ProductID` for orders inside the window.
3. `subcats`: start from `Products` joined to active `Category` rows, LEFT JOIN `recent_units`, and aggregate to one row per category and sub-category: listed products, units, units per product.
4. `thresholds`: `PERCENTILE_CONT(0.25)` of units per product within each category.
5. Join and filter strictly below the threshold.

Step 4 — Final SQL Solution:

```sql
WITH bounds AS (
    SELECT MAX(o.OrderDate)                                  AS last_day,
           (MAX(o.OrderDate) - INTERVAL '6 months')::date    AS window_start
    FROM Orders AS o
),
recent_units AS (
    SELECT od.ProductID,
           SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    JOIN Orders AS o   ON o.OrderID = od.OrderID
    CROSS JOIN bounds AS b
    WHERE o.OrderDate >  b.window_start
      AND o.OrderDate <= b.last_day
    GROUP BY od.ProductID
),
subcats AS (
    SELECT cat.CategoryName,
           p.Sub_Category,
           COUNT(*)                                          AS listed_products,
           COALESCE(SUM(ru.units), 0)                        AS units_last_6m,
           COALESCE(SUM(ru.units), 0)::numeric / COUNT(*)    AS units_per_product
    FROM Products AS p
    JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    LEFT JOIN recent_units AS ru ON ru.ProductID = p.ProductID
    WHERE COALESCE(UPPER(TRIM(CAST(cat.Active AS VARCHAR))) IN ('1', 'TRUE', 'T', 'Y', 'YES'), FALSE)
    GROUP BY cat.CategoryName, p.Sub_Category
),
thresholds AS (
    SELECT s.CategoryName,
           PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY s.units_per_product) AS category_p25
    FROM subcats AS s
    GROUP BY s.CategoryName
)
SELECT s.CategoryName,
       s.Sub_Category,
       s.listed_products,
       s.units_last_6m,
       ROUND(s.units_per_product, 2)       AS units_per_product,
       ROUND(t.category_p25::numeric, 2)   AS category_p25
FROM subcats AS s
JOIN thresholds AS t ON t.CategoryName = s.CategoryName
WHERE s.units_per_product < t.category_p25
ORDER BY s.CategoryName, s.units_per_product, s.Sub_Category;
```

Step 5 — Line-by-Line Explanation:

- `bounds` anchors the window to the data. `INTERVAL '6 months'` moves the calendar month, so the window is six calendar months whatever their lengths.
- `o.OrderDate > b.window_start AND o.OrderDate <= b.last_day` is a half-open window ending on the anchor day.
- `recent_units` sums lines per product inside the window, so the later join to `Products` is one-to-one.
- `subcats` starts from `Products`, so every listed product counts in `listed_products`, sold or not. The LEFT JOIN keeps unsold products, and `COALESCE(SUM(ru.units), 0)` turns a sub-category with no recent sales into 0 instead of NULL.
- `::numeric / COUNT(*)` avoids integer division, which would truncate 0.8 units per product to 0.
- The flag test uses the house normalisation wrapped in COALESCE, so a NULL `Active` counts as inactive rather than disappearing silently.
- `PERCENTILE_CONT(0.25)` interpolates between neighbouring values, so the threshold can fall between two sub-categories' figures.

Step 6 — Sample Output:

categoryname	sub_category	listed_products	units_last_6m	units_per_product	category_p25
Bakery, Cakes & Dairy	Cakes & Pastries	112	567	5.06	6.33
Bakery, Cakes & Dairy	Cookies, Rusk & Khari	129	660	5.12	6.33
Beauty & Hygiene	Fragrances & Deos	723	1265	1.75	2.06
Beauty & Hygiene	Health & Medicine	970	1732	1.79	2.06
Beverages	Health Drink, Supplement	76	410	5.39	7.74
Beverages	Energy & Soft Drinks	69	405	5.87	7.74
Cleaning & Household	Stationery	352	152	0.43	1.76
Cleaning & Household	Bins & Bathroom Ware	320	206	0.64	1.76

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 22 rows.

Step 7 — Why This Solution Works: Every sub-category of an active category appears once in `subcats` with a real number, zero included, so the percentile describes the whole range. The threshold and the rows it judges come from the same CTE, so they cannot disagree about who is in the population. Normalising by listed products means a small sub-category that sells well per item is not flagged just for being small, and a large one coasting on its size is not protected.

Step 8 — Common Mistakes:

- Running the same rule at product level. In 8 of 12 categories the threshold is 0, so nothing can be flagged; in the rest it is 1 to 8 units, so single products are judged on a unit or two:

```sql
WITH b AS (
    SELECT MAX(OrderDate) AS last_day,
           (MAX(OrderDate) - INTERVAL '6 months')::date AS window_start
    FROM Orders
),
recent_units AS (
    SELECT od.ProductID, SUM(od.Quantity) AS units
    FROM OrderDetails AS od
    JOIN Orders AS o ON o.OrderID = od.OrderID
    CROSS JOIN b
    WHERE o.OrderDate > b.window_start
      AND o.OrderDate <= b.last_day
    GROUP BY od.ProductID
),
pu AS (
    SELECT p.CategoryID, COALESCE(ru.units, 0) AS units
    FROM Products AS p
    LEFT JOIN recent_units AS ru ON ru.ProductID = p.ProductID
)
SELECT pu.CategoryID,
       PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY pu.units) AS product_level_p25,
       ROUND(100.0 * COUNT(*) FILTER (WHERE pu.units = 0) / COUNT(*), 1) AS pct_unsold
FROM pu
GROUP BY pu.CategoryID
ORDER BY pu.CategoryID;
```

The date filter sits in `recent_units`, before units are summed. Putting it on a LEFT JOIN to `Orders` after the lines (`... AND o.OrderDate > ...`) keeps every line's `Quantity`, so "six-month" units silently become all-time units.

- Starting from sales with an INNER JOIN. Sub-categories with no recent sales vanish and can never be flagged, though they are the clearest candidates.
- Using `CURRENT_DATE` for the window. On a historical extract the window is empty and every sub-category scores 0.
- Comparing raw units instead of units per product, which mostly flags the sub-categories with the fewest listings.

Step 9 — Interview Follow-Up: The category managers want to see whether each flagged sub-category is declining or was always weak. How would you add its units per product in the previous six-month window, and what would you show for a sub-category with no sales in either window?

Step 10 — Optimization or Alternative: `PERCENT_RANK() OVER (PARTITION BY CategoryName ORDER BY units_per_product)` gives each sub-category its position in the category directly, so the filter becomes `pct_rank < 0.25` without a separate threshold CTE. It differs at the edges: PERCENT_RANK uses positions, PERCENTILE_CONT interpolates values, so a sub-category sitting exactly on an interpolated threshold can fall on different sides. For large catalogues, an index on `Orders(OrderDate)` lets `recent_units` read only the window, and `OrderDetails(OrderID)` serves the join.

Question 84: Delivery time by state and shipper with subtotals

Difficulty: Hard

Primary Topic: GROUP BY ROLLUP with GROUPING()

Combined Topics: Small-cell suppression with CASE, Date validity filters, LEFT JOIN with labelled unknowns, Date arithmetic

Business Scenario: The logistics team is deciding which courier to prefer in each state. They want one table: average days from order to delivery for every state and shipper pair, a subtotal for each state across all shippers, and a grand total. Averages from a handful of orders swing wildly and have caused bad decisions before, so any cell with fewer than five delivered orders must show its count but not its average.

Problem Statement: For delivered orders with a valid date sequence, report the count and average delivery days by state and shipper, with state subtotals and a grand total in the same result. Label subtotal and total rows clearly, and keep them distinct from customers whose state is missing. Show `avg_delivery_days` only when the row has at least five delivered orders.

Expected Output:

- `state`: customer state, `Unknown` when missing or unmatched, `All states` on the grand total row
- `shipper_name`: shipper, `Unknown shipper` when missing or unmatched, `All shippers` on subtotal and total rows
- `row_level`: `State x shipper`, `State subtotal` or `Grand total`
- `delivered_orders`: orders counted in the row
- `avg_delivery_days`: average of `DeliveryDate - OrderDate`, 2 decimals, NULL when `delivered_orders < 5`

Tables Required: `Orders`, `Customers`, `Shippers`

SQL Concepts Tested: `GROUP BY ROLLUP`, `GROUPING()` to tell subtotal rows from NULL data, small-cell suppression with `CASE`, validity filters on dates, LEFT JOINs with labelled unknowns, sorting subtotals under their detail rows.

Step 1 — Understand the Problem: `ROLLUP (state, shipper_name)` produces three grouping levels in one pass: (state, shipper), (state) and (). On the subtotal and total rows, the rolled-up columns come back as NULL. That creates a trap: customers with no state also produce NULL in the state column. If you label NULL as "All states", a real group of unknown-state customers looks like a second grand total.

`GROUPING(col)` returns 1 only when the column was rolled up, so it is the reliable way to label those rows.

Delivery days are `DeliveryDate::date - OrderDate::date`. Undelivered orders have no value yet. Orders delivered or shipped before they were placed, or delivered before they shipped, are data errors that would pull averages towards zero, so they are excluded. An order with a delivery date but no ship date is kept, because the measure does not use the ship date.

Step 2 — Identify the Tables and Relationships: `Orders` holds the dates, `CustomerID` and `ShipperID`. `Customers` gives `State` (`Orders.CustomerID` → `Customers.CustomerID`). `Shippers` gives `CompanyName` (`Orders.ShipperID` → `Shippers.ShipperID`). Both are many-to-one from `Orders`, so joining them cannot duplicate orders. Both are LEFT JOINs so that orders with a missing or unknown customer or shipper are counted under a label.

Step 3 — Build the Query Logic:

1. `delivered`: one row per valid delivered order with its state label, shipper label and delivery days.
2. `GROUP BY ROLLUP (state, shipper_name)`.
3. Label with `GROUPING()`, suppress the average when the count is below 5.
4. Sort states by delivered volume, with each state's subtotal under its detail rows and the grand total last.

Step 4 — Final SQL Solution:

```sql
WITH delivered AS (
    SELECT COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown') AS state,
           COALESCE(sh.CompanyName, 'Unknown shipper')     AS shipper_name,
           o.DeliveryDate::date - o.OrderDate::date        AS delivery_days
    FROM Orders AS o
    LEFT JOIN Customers AS c  ON c.CustomerID = o.CustomerID
    LEFT JOIN Shippers  AS sh ON sh.ShipperID = o.ShipperID
    WHERE o.DeliveryDate IS NOT NULL
      AND o.OrderDate    IS NOT NULL
      AND o.DeliveryDate >= o.OrderDate
      AND (o.ShipDate IS NULL
           OR (o.ShipDate >= o.OrderDate AND o.DeliveryDate >= o.ShipDate))
)
SELECT CASE WHEN GROUPING(d.state) = 1 THEN 'All states' ELSE d.state END               AS state,
       CASE WHEN GROUPING(d.shipper_name) = 1 THEN 'All shippers' ELSE d.shipper_name END AS shipper_name,
       CASE GROUPING(d.state, d.shipper_name)
            WHEN 0 THEN 'State x shipper'
            WHEN 1 THEN 'State subtotal'
            ELSE 'Grand total'
       END                                                                              AS row_level,
       COUNT(*)                                                                         AS delivered_orders,
       CASE WHEN COUNT(*) >= 5 THEN ROUND(AVG(d.delivery_days), 2) END                  AS avg_delivery_days
FROM delivered AS d
GROUP BY ROLLUP (d.state, d.shipper_name)
ORDER BY GROUPING(d.state),
         MAX(COUNT(*)) OVER (PARTITION BY d.state) DESC,
         d.state,
         GROUPING(d.shipper_name),
         d.shipper_name;
```

Step 5 — Line-by-Line Explanation:

- `COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown')` maps NULL state, blank state and orders whose customer does not exist to one label before grouping, so they never collide with the rollup NULLs.
- The `WHERE` clause is the validity rule. `o.ShipDate IS NULL OR (...)` keeps orders with no ship date but checks the sequence when a ship date exists.
- `GROUP BY ROLLUP (d.state, d.shipper_name)` returns detail rows, one subtotal per state and one grand total.
- `GROUPING(d.state)` is 1 only on the grand total. `GROUPING(d.shipper_name)` is 1 on subtotals and the grand total.
- `GROUPING(d.state, d.shipper_name)` returns a bitmask: 0 for detail, 1 for a state subtotal (shipper rolled up), 3 for the grand total.
- `CASE WHEN COUNT(*) >= 5 THEN ... END` returns NULL for small cells while still showing the count.
- In `ORDER BY`, `MAX(COUNT(*)) OVER (PARTITION BY d.state)` gives every row of a state the size of its subtotal, so states sort by volume. Window functions run after grouping, so they can reference aggregates. Within a state, `GROUPING(d.shipper_name)` puts the subtotal after its shippers.

Step 6 — Sample Output:

state	shipper_name	row_level	delivered_orders	avg_delivery_days
Maharashtra	Bharat Parcel Service	State x shipper	389	7.58
Maharashtra	BlueLine Express	State x shipper	1295	3.31
Maharashtra	Ecomzip Couriers	State x shipper	1013	4.71
Maharashtra	MetroDash Hyperlocal	State x shipper	1969	1.66
Maharashtra	NorthStar Freight	State x shipper	206	9.55
Maharashtra	ShadowRun Delivery	State x shipper	1016	3.64
Maharashtra	SwiftKart Logistics	State x shipper	1750	3.29
Maharashtra	Unknown shipper	State x shipper	6	3.83

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 206 rows.

Step 7 — Why This Solution Works: All three levels are aggregated from the same filtered rows in one statement, so subtotals always add up to the detail counts. Separate queries glued with `UNION ALL` lose that guarantee the first time someone edits one of them. `GROUPING()` labels rows by how they were produced, not by their data, so an `Unknown` state and the grand total cannot be confused. The subtotal average comes from the underlying orders, not from the shipper averages, so a busy shipper weighs more than a quiet one.

Edge cases:

- Orders not yet delivered: excluded, not counted as 0 days.
- Impossible date sequences: excluded from count and average. Report them separately (Q88 style).
- Orphan `CustomerID` or `ShipperID`: counted under `Unknown` and `Unknown shipper`.
- A state subtotal can show an average while all its shipper cells are suppressed, because together they pass the threshold. That is intended.
- Inconsistent spelling of a state ("Tamilnadu" vs "Tamil Nadu") splits a state; clean the dimension first.

Step 8 — Common Mistakes:

- Labelling rollup rows with `COALESCE`. A real NULL state and the grand total both become "All states":

```
WITH delivered AS (
    SELECT c.State AS state,
           o.DeliveryDate::date - o.OrderDate::date AS delivery_days
    FROM Orders AS o
    LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
    WHERE o.DeliveryDate IS NOT NULL
      AND o.DeliveryDate >= o.OrderDate
)
SELECT COALESCE(d.state, 'All states') AS state_label,
       COUNT(*)                       AS delivered_orders
FROM delivered AS d
GROUP BY ROLLUP (d.state)
ORDER BY state_label, delivered_orders;
```

- Building the state subtotal as `AVG` of the shipper averages. It gives every shipper equal weight regardless of volume.
- Dropping small cells with `HAVING COUNT(*) >= 5`. The subtotals still include them, so the detail rows no longer add up to the subtotal and readers think orders are missing.
- Leaving delivered-before-ordered rows in: negative days pull every average down.

Step 9 — Interview Follow-Up: The team also wants a subtotal per shipper across all states. What changes if you replace `ROLLUP` with `CUBE`, and what new value of `GROUPING(state, shipper_name)` appears?

Step 10 — Optimization or Alternative: `GROUPING SETS` lists the levels explicitly. It returns the same rows as `ROLLUP` here, and it makes it easy to add or drop a level, such as a per-shipper subtotal, without changing anything else:

```
WITH delivered AS (
    SELECT COALESCE(NULLIF(TRIM(c.State), ''), 'Unknown') AS state,
           COALESCE(sh.CompanyName, 'Unknown shipper')     AS shipper_name,
           o.DeliveryDate::date - o.OrderDate::date        AS delivery_days
    FROM Orders AS o
    LEFT JOIN Customers AS c  ON c.CustomerID = o.CustomerID
    LEFT JOIN Shippers  AS sh ON sh.ShipperID = o.ShipperID
    WHERE o.DeliveryDate IS NOT NULL
      AND o.OrderDate    IS NOT NULL
      AND o.DeliveryDate >= o.OrderDate
      AND (o.ShipDate IS NULL
           OR (o.ShipDate >= o.OrderDate AND o.DeliveryDate >= o.ShipDate))
)
SELECT CASE WHEN GROUPING(d.state) = 1 THEN 'All states' ELSE d.state END               AS state,
       CASE WHEN GROUPING(d.shipper_name) = 1 THEN 'All shippers' ELSE d.shipper_name END AS shipper_name,
       COUNT(*)                                                                        AS delivered_orders,
       CASE WHEN COUNT(*) >= 5 THEN ROUND(AVG(d.delivery_days), 2) END                 AS avg_delivery_days
FROM delivered AS d
GROUP BY GROUPING SETS ((d.state, d.shipper_name), (d.state), ())
ORDER BY GROUPING(d.state), d.state, GROUPING(d.shipper_name), d.shipper_name;
```

MySQL 8 supports `WITH ROLLUP` and `GROUPING()`, but not `GROUPING SETS` or `CUBE`. Most orders pass the delivery filter, so a sequential scan of `Orders` is the right plan here; an index would not help.

Question 85: Win-back customers returning after 180+ days

Difficulty: Hard

Primary Topic: LAG gap per customer

Combined Topics: Filter after the window, Anchored 12-month reporting window, Date difference, Deterministic tie-break

Business Scenario: The retention team ran reactivation campaigns over the past year and wants to see who actually came back. A win-back is an order placed at least 180 days after the same customer's previous order. They need each win-back event in the last 12 months, with the gap and the size of the return order, to compare against the campaign calendar and to decide whether returning customers spend like new ones or like loyal ones.

Problem Statement: For every order, find the same customer's previous order date. Return orders whose gap from the previous order is at least 180 days and whose date falls in the 12 months ending on the last order date in the data. The previous order may be older than those 12 months. A customer can appear more than once if they lapsed and returned more than once. Sort by return date.

Expected Output:

- `CustomerID`
- `previous_order_date`: the customer's order immediately before the return order
- `return_order_date`: `OrderDate` of the return order
- `gap_days`: days between the two
- `return_order_amount`: `Total_Order_Amount` of the return order

Tables Required: `Orders`

SQL Concepts Tested: `LAG` within a customer partition, deterministic ordering with an ID tie-break, filtering on a window result in an outer query, reporting window applied after the window function, anchored dates, date difference.

Step 1 — Understand the Problem: A win-back is a property of a pair of consecutive orders, not of a customer. `LAG(OrderDate)` puts the previous order's date on each order row, and the gap is a simple subtraction. Each qualifying row is one event, so a customer who lapsed twice gives two rows.

The two filters must run in order. The gap needs full history: a customer who returned in March after two years away has their previous order far outside the reporting window. Restrict rows to the last 12 months before `LAG`, and the return order becomes the customer's first order in the window, its `LAG` is NULL, and the win-back disappears. So compute `LAG` over all orders, then keep return orders inside the window. The 12 months end on `MAX(OrderDate)`, not today, so a stale extract still shows its win-backs.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `CustomerID` for the partition, `OrderDate` and `OrderID` for the sequence, `Total_Order_Amount` for the return order's value. Comparing a customer's orders with each other is done by the window function, not by a self join.

Step 3 — Build the Query Logic:

1. `bounds`: last order date.
2. `sequenced`: every order with `LAG(OrderDate)` over the customer's orders, sorted by date then ID.
3. Outer query: keep rows with a gap of at least 180 days and a return date in the last 12 months.

Step 4 — Final SQL Solution:

```sql
WITH bounds AS (
    SELECT MAX(o.OrderDate)::date AS last_day
    FROM Orders AS o
),
sequenced AS (
    SELECT o.CustomerID,
           o.OrderID,
           o.OrderDate,
           o.Total_Order_Amount,
           LAG(o.OrderDate) OVER (PARTITION BY o.CustomerID
                                  ORDER BY o.OrderDate, o.OrderID) AS previous_order_date
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
)
SELECT s.CustomerID,
       s.previous_order_date,
       s.OrderDate                          AS return_order_date,
       s.OrderDate - s.previous_order_date AS gap_days,
       s.Total_Order_Amount                 AS return_order_amount
FROM sequenced AS s
CROSS JOIN bounds AS b
WHERE s.OrderDate - s.previous_order_date >= 180
  AND s.OrderDate >= (b.last_day - INTERVAL '1 year')::date + 1
ORDER BY s.OrderDate, s.CustomerID, s.OrderID;
```

Step 5 — Line-by-Line Explanation:

- `bounds` gives the anchor. `(b.last_day - INTERVAL '1 year')::date + 1` is the first day of the 12 months that end on it.
- `WHERE o.CustomerID IS NOT NULL` inside `sequenced` stops NULL-customer orders from forming one shared partition, where `LAG` would compare unrelated orders.
- `LAG(o.OrderDate) OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID)` returns the previous order's date in the customer's sequence. The `OrderID` tie-break fixes the order of same-day orders.
- `sequenced` has no date-window filter, so `LAG` sees the full history. The reporting-window filter sits in the outer query, after `LAG` has run.
- `s.OrderDate - s.previous_order_date >= 180` is an integer comparison because both are dates. For a customer's first order, `previous_order_date` is NULL, the subtraction is NULL, and the row is dropped.

Step 6 — Sample Output:

customerid	previous_order_date	return_order_date	gap_days	return_order_amount
102328	2024-01-30	2025-01-05	341	337.00
100486	2023-11-22	2025-01-06	411	539.00
104299	2024-07-04	2025-01-12	192	560.00

100001	2024-01-26	2025-01-14	354	288.00
102087	2024-04-18	2025-01-17	274	1102.00
102381	2024-06-19	2025-01-17	212	281.00
102675	2024-06-15	2025-01-20	219	464.00
104406	2024-03-27	2025-01-23	302	1895.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 156 rows.

Step 7 — Why This Solution Works: `LAG` with a full sort key gives exactly one previous order per order, so each gap is between true neighbours and each win-back is reported once. Computing the gap over all history and filtering only the return date to the reporting window means a return in the window is found no matter how old the previous order is.

Edge cases:

- One-order customers: their only row has a NULL previous date and never qualifies.
- Same-day orders: the second has a gap of 0 days and does not qualify; the first carries the real gap.
- Customers who lapse and return twice: two rows, each with its own gap.
- Exactly 180 days: included, because the rule is "at least 180".
- NULL `Total_Order_Amount` on the return order: the event is still listed, with a NULL amount.
- Duplicate customer accounts: a person who returned under a new account looks new, not won back. Merge accounts first if possible.
- Orphan `CustomerID`: still analysed; LEFT JOIN `Customers` before handing the list over.

Step 8 — Common Mistakes:

- Filtering to the last 12 months before computing `LAG`. Every customer whose previous order is older than the window loses it, and most win-backs vanish:

```
WITH bounds AS (
    SELECT MAX(o.OrderDate)::date AS last_day FROM Orders AS o
),
sequenced_wrong AS (
    SELECT o.CustomerID,
           o.OrderDate,
           LAG(o.OrderDate) OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID) AS previous_order_date
    FROM Orders AS o
    CROSS JOIN bounds AS b
    WHERE o.OrderDate >= (b.last_day - INTERVAL '1 year')::date + 1
)
SELECT s.CustomerID, s.previous_order_date, s.OrderDate AS return_order_date
FROM sequenced_wrong AS s
WHERE s.OrderDate - s.previous_order_date >= 180
ORDER BY s.OrderDate, s.CustomerID;
```

- Using a self join with `later.OrderDate - earlier.OrderDate >= 180`. It pairs every order with every order at least 180 days older, not just the one immediately before, so a loyal customer who ordered every week is flagged hundreds of times.
- Ordering `LAG` by `OrderDate` only. With two orders on one day, which is "previous" depends on physical row order, and results can change between runs.
- Measuring the gap from the customer's first order instead of the previous one. That flags every order a long-tenure customer places.

Step 9 — Interview Follow-Up: The team wants the win-back rate: of customers who went 180 days without ordering at some point in the past year, what share came back? How do you find customers who lapsed but have not returned, given they have no "return order" row?

Step 10 — Optimization or Alternative: A `LATERAL` subquery can fetch each order's previous order date directly, which shows exactly what `LAG` computes. On an engine without window functions, the same lookup works as a correlated scalar subquery in the `SELECT` list:

```
-- The lookup below needs this index. Without it, every order rescans the whole table.
CREATE INDEX ix_orders_cust_date ON Orders (CustomerID, OrderDate, OrderID);

WITH bounds AS (
    SELECT MAX(o.OrderDate)::date AS last_day
    FROM Orders AS o
)
SELECT o.CustomerID,
       prev.previous_order_date,
       o.OrderDate                          AS return_order_date,
       o.OrderDate - prev.previous_order_date AS gap_days,
       o.Total_Order_Amount                 AS return_order_amount
FROM Orders AS o
CROSS JOIN bounds AS b
CROSS JOIN LATERAL (
    SELECT MAX(p.OrderDate) AS previous_order_date
    FROM Orders AS p
    WHERE p.CustomerID = o.CustomerID
      AND (p.OrderDate, p.OrderID) < (o.OrderDate, o.OrderID)
) AS prev
WHERE o.CustomerID IS NOT NULL
  AND o.OrderDate - prev.previous_order_date >= 180
  AND o.OrderDate >= (b.last_day - INTERVAL '1 year')::date + 1
ORDER BY o.OrderDate, o.CustomerID, o.OrderID;
```

The row comparison `(p.OrderDate, p.OrderID) < (o.OrderDate, o.OrderID)` is the same "earlier in sequence" rule as the `LAG` sort. This version only has to look at orders in the reporting window, but it runs one lookup per order, so it needs the index created at the top of the block. On this dataset's 46,438 orders the difference is stark: without the index the query ran past our 30-second test limit; with it, it finished in under a tenth of a second. The `LAG` version reads `Orders` once and is usually faster when the whole table is involved.

Expert

Questions 86 to 100

Question 86: Reconcile order headers with order lines

Difficulty: Expert

Primary Topic: Reconciliation with FULL OUTER JOIN of pre-aggregates

Combined Topics: Multiple CTEs, Rule-based status with CASE, Fixed status list with LEFT JOIN so zeros show, Discovering a hidden business rule from the data, STRING_AGG DISTINCT as evidence

Business Scenario: Finance books revenue from order headers (`Orders.Total_Order_Amount`). The category and supplier dashboards are built from order lines (`Quantity * Sale_Price`). The two never quite agree, and every month someone explains the gap as "catalogue price changes". Before month-end close, the finance controller wants one table that shows exactly how header value relates to line value, order by order, with every gap explained or flagged. The data platform team will run it nightly after the order load.

Problem Statement: Pair every order header with the list-price value of its lines and give each order exactly one status. Rules apply top to bottom; first match wins:

1. `Lines with NULL OrderID`: line rows that belong to no order.
2. `Lines without header`: lines whose `OrderID` has no row in `Orders`.
3. `Header without lines`: an order with no rows in `OrderDetails`.
4. `Header zero or NULL`: `Total_Order_Amount` is NULL, zero or negative.
5. `Exact match`: header equals line value to the cent.
6. `Discount 5%`, `Discount 10%`, `Discount 12%`, `Discount 15%`: header equals line value times 0.95, 0.90, 0.88 or 0.85, to the cent.
7. `Other shortfall`: header below line value by any other amount.
8. `Header above lines`: header above line value.

Return one row per status in this order, including statuses with zero orders, with the order count, both values, the total absolute difference and the payment types seen.

Expected Output:

- `recon_status`: one of the eleven statuses above
- `orders`: order keys in the status (0 for the NULL-key status)
- `header_value`: sum of `Total_Order_Amount`
- `line_list_value`: sum of `Quantity * Sale_Price` over the lines
- `abs_difference`: sum of per-order |header − line value|, a missing side counted as 0
- `payment_types`: distinct `PaymentType` values of the orders in the status

Tables Required: `Orders`, `OrderDetails`, `Products`, `Payments`

SQL Concepts Tested: collapsing each side to the order grain before joining, FULL OUTER JOIN to see both kinds of orphan, telling a NULL key from a broken key, rule order in CASE, a fixed status list so empty statuses still print, using `STRING_AGG(DISTINCT ...)` as evidence for a hypothesis.

Step 1 — Understand the Problem: Assumptions, stated before any SQL:

- The header is the money of record; the lines are the detail. A difference is either a known business rule or an error.
- `Sale_Price` is today's catalogue price. If prices had changed since checkout, honest orders would drift by small, irregular amounts, and only a tolerance band could classify them.
- The schema has no discount, coupon or fee column, so any systematic gap is a rule the schema does not model.

The right first move is to measure the gap, not to pick a tolerance. On this dataset the implied discount (Step 10) is not spread out the way price drift would be. Almost every order sits on one of five exact values: 0%, 5%, 10%, 12% or 15%. The 5% and 10% tiers belong to Credit Card and Debit Card orders, the 12% and 15% tiers to Wallet orders. That is a payment-linked discount, not drift. The few tier orders tagged Pay Later are themselves a finding: their payment method was changed after checkout, which Q37 picks up. So the statuses encode the tiers exactly, and anything that does not fit a tier is a real exception.

Both sides live at different grains and either can exist without the other, which rules out a plain join from `Orders`.

Step 2 — Identify the Tables and Relationships: `Orders` holds the header amount and `PaymentID`. `OrderDetails` holds the lines (`OrderDetails.OrderID` → `Orders.OrderID`, one to many). `Products` prices each line (`OrderDetails.ProductID` → `Products.ProductID`). `Payments` names the payment method (`Orders.PaymentID` → `Payments.PaymentID`), which is the evidence for the discount rule.

Step 3 — Build the Query Logic:

1. `headers`: one row per order, with its amount, payment type and a `TRUE` marker.
2. `line_totals`: lines collapsed to one row per `OrderID`, with a `TRUE` marker and the list-price value.

3. `classified`: FULL OUTER JOIN the two on `OrderID`, so headers without lines and lines without headers both survive, then apply the rules in order.
4. `statuses`: a fixed list of every status with a sort number.
5. LEFT JOIN statuses to classified orders and aggregate, so a status with no orders still prints as 0.

Step 4 — Final SQL Solution:

```sql
WITH headers AS (
    SELECT o.OrderID,
           o.Total_Order_Amount,
           pay.PaymentType,
           TRUE AS has_header
    FROM Orders AS o
    LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
),
line_totals AS (
    SELECT od.OrderID,
           TRUE                                      AS has_lines,
           COUNT(*)                                  AS line_rows,
           COALESCE(SUM(od.Quantity * p.Sale_Price), 0) AS line_list_value
    FROM OrderDetails AS od
    LEFT JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY od.OrderID
),
classified AS (
    SELECT COALESCE(h.OrderID, lt.OrderID) AS order_key,
           h.Total_Order_Amount            AS header_value,
           h.PaymentType,
           lt.line_rows,
           lt.line_list_value,
           CASE
               WHEN h.has_header IS NULL AND lt.OrderID IS NULL THEN 'Lines with NULL OrderID'
               WHEN h.has_header IS NULL                        THEN 'Lines without header'
               WHEN lt.has_lines IS NULL                        THEN 'Header without lines'
               WHEN h.Total_Order_Amount IS NULL
                 OR h.Total_Order_Amount <= 0                   THEN 'Header zero or NULL'
               WHEN ABS(h.Total_Order_Amount - lt.line_list_value)              <= 0.01 THEN 'Exact match'
               WHEN ABS(h.Total_Order_Amount - ROUND(lt.line_list_value * 0.95, 2)) <= 0.01 THEN 'Discount 5%'
               WHEN ABS(h.Total_Order_Amount - ROUND(lt.line_list_value * 0.90, 2)) <= 0.01 THEN 'Discount 10%'
               WHEN ABS(h.Total_Order_Amount - ROUND(lt.line_list_value * 0.88, 2)) <= 0.01 THEN 'Discount 12%'
               WHEN ABS(h.Total_Order_Amount - ROUND(lt.line_list_value * 0.85, 2)) <= 0.01 THEN 'Discount 15%'
               WHEN h.Total_Order_Amount < lt.line_list_value  THEN 'Other shortfall'
               ELSE 'Header above lines'
           END AS recon_status
    FROM headers AS h
    FULL OUTER JOIN line_totals AS lt ON lt.OrderID = h.OrderID
),
statuses (sort_order, recon_status) AS (
    VALUES (1, 'Exact match'),
           (2, 'Discount 5%'),
           (3, 'Discount 10%'),
           (4, 'Discount 12%'),
           (5, 'Discount 15%'),
           (6, 'Other shortfall'),
           (7, 'Header above lines'),
           (8, 'Header zero or NULL'),
           (9, 'Header without lines'),
           (10, 'Lines without header'),
           (11, 'Lines with NULL OrderID')
)
SELECT st.recon_status,
       COUNT(cl.order_key)                                      AS orders,
       COALESCE(SUM(cl.header_value), 0)                        AS header_value,
       COALESCE(SUM(cl.line_list_value), 0)                     AS line_list_value,
       COALESCE(SUM(ABS(COALESCE(cl.header_value, 0)
                      - COALESCE(cl.line_list_value, 0))), 0)   AS abs_difference,
       STRING_AGG(DISTINCT cl.PaymentType, ', ')                AS payment_types
FROM statuses AS st
LEFT JOIN classified AS cl ON cl.recon_status = st.recon_status
GROUP BY st.sort_order, st.recon_status
ORDER BY st.sort_order;
```

Step 5 — Line-by-Line Explanation:

- `headers` and `line_totals` are each one row per order, so the FULL OUTER JOIN is one-to-one and nothing is double counted. Joining raw lines to headers would repeat each header once per line.
- The `TRUE` markers tell the two kinds of missing apart. After a FULL OUTER JOIN, `h.has_header IS NULL` means no header matched; `lt.OrderID IS NULL` as well means the line's own key was NULL.
- `COALESCE(h.OrderID, lt.OrderID)` gives every row a key, whichever side it came from.
- Rules 1 to 4 come first because the arithmetic tests below them need both sides and a positive header.
- Each tier test compares the header with the line value multiplied by the tier and rounded to cents. Rounding first matters: the header was rounded once at checkout, so comparing with the unrounded product would miss by a fraction of a cent.
- `statuses` is a VALUES list; `LEFT JOIN classified` keeps every status, and `COUNT(cl.order_key)` is 0 where nothing matched.
- `STRING_AGG(DISTINCT cl.PaymentType, ', ')` lists the payment methods behind each status. It is the one column that turns the table from a count into an explanation.

Step 6 — Sample Output:

recon_status	orders	header_value	line_list_value	abs_difference	payment_types

Exact match	41386	55034581.00	55034581.00	0.00	Cash on Delivery, Credit Card, Debit Card, EMI, Net Banking, Pay Later, UPI, Wallet
Discount 5%	2227	2729387.05	2873039.00	143651.95	Credit Card, Debit Card, Pay Later
Discount 10%	1417	1712687.40	1902986.00	190298.60	Credit Card, Debit Card
Discount 12%	841	1030861.92	1171434.00	140572.08	Pay Later, Wallet
Discount 15%	416	458124.50	538970.00	80845.50	Wallet
Other shortfall	95	196049.91	249018.00	52968.09	Cash on Delivery, Credit Card, Debit Card, Net Banking, Pay Later, UPI, Wallet
Header above lines	37	101131.50	84590.00	16541.50	Cash on Delivery, Credit Card, Debit Card, Net Banking, Pay Later, UPI
Header zero or NULL	9	0.00	10716.00	10716.00	Cash on Delivery, Credit Card, Net Banking, Pay Later, UPI

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 11 rows.

Step 7 — Why This Solution Works: Every order lands in exactly one status, because the CASE is exhaustive and first-match-wins. Every status prints, so "zero exceptions" is visible rather than inferred from an empty result. Two control totals prove nothing was dropped or double counted: the `orders` column must sum to `COUNT(*)` from `Orders` plus any orphan line groups, and `header_value` must sum to `SUM(Total_Order_Amount)` from `Orders`.

On this dataset the tiers explain almost all of the gap. Line-based dashboards show gross value, headers show net of a payment-linked discount, and for more than 99% of orders the difference is that discount, to the cent. What is left is a short, specific work list rather than noise: orders whose lines carry duplicated or extra rows (`Other shortfall`), headers keyed too high or priced from lines the catalogue can no longer price (`Header above lines`), zero or missing headers, headers without lines and lines without headers. Each status points to a different owner, which is what makes the table useful at month-end close.

Nightly, store this summary with the run date. Alert when an exception status becomes non-zero, or when the share of discounted orders moves sharply, which usually means the discount rule changed at checkout without anyone telling analytics.

Step 8 — Common Mistakes:

- Starting with tolerance bands. They turn an exact business rule into vague "drift" and a large "mismatch" bucket that someone then explains away every month:

```
WITH line_totals AS (
    SELECT od.OrderID, SUM(od.Quantity * p.Sale_Price) AS line_list_value
    FROM OrderDetails AS od
    JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY od.OrderID
)
SELECT CASE WHEN ABS(o.Total_Order_Amount - lt.line_list_value) <= 0.01 * o.Total_Order_Amount THEN 'Matched'
            WHEN ABS(o.Total_Order_Amount - lt.line_list_value) <= 0.10 * o.Total_Order_Amount THEN 'Price drift'
            ELSE 'Mismatch'
       END       AS naive_status,
       COUNT(*)  AS orders
FROM Orders AS o
JOIN line_totals AS lt ON lt.OrderID = o.OrderID
GROUP BY 1
ORDER BY 1;
```

- Joining `Orders` to `OrderDetails` and summing `Total_Order_Amount`. Each header is repeated once per line and the header side is inflated several times over.
- Driving from `Orders` with a LEFT JOIN. Lines whose order is missing never appear, so the check cannot see one of the two failure modes it exists for.
- Comparing to the unrounded tier value (`line_list_value * 0.95`) with `=`. Orders that are exactly on the tier fail by fractions of a cent.
- Writing `WHERE status <> 'Exact match'` and treating no rows as success. A broken join also returns no rows.

Step 9 — Interview Follow-Up: Finance wants net revenue by category, but the discount is only known at order level. How would you allocate each order's discount to its lines, and how would you make sure the allocated line values add back to the header exactly despite rounding?

Step 10 — Optimization or Alternative: The tiers in Step 4 were found from the data, and this is the query that finds them. It computes each order's implied discount and groups by payment type, so a rule shows up as a handful of exact values per method:

```
WITH line_totals AS (
    SELECT od.OrderID, SUM(od.Quantity * p.Sale_Price) AS line_list_value
    FROM OrderDetails AS od
    JOIN Products AS p ON p.ProductID = od.ProductID
    GROUP BY od.OrderID
)
SELECT pay.PaymentType,
       ROUND(100.0 * (lt.line_list_value - o.Total_Order_Amount)
             / NULLIF(lt.line_list_value, 0), 1) AS implied_discount_pct,
       COUNT(*)                                  AS orders
FROM Orders AS o
JOIN line_totals AS lt ON lt.OrderID = o.OrderID
LEFT JOIN Payments AS pay ON pay.PaymentID = o.PaymentID
GROUP BY pay.PaymentType, 2
ORDER BY pay.PaymentType, implied_discount_pct;
```

In production, keep the tiers in a small reference table (`tier_pct`, `valid_from`) and join to it instead of hard-coding them, so a new tier is a row, not a code change. Performance: both CTEs are single grouped scans, and the FULL OUTER JOIN is a hash join on `OrderID`. On a large table, restrict both sides to orders loaded since the last run (by `OrderDate` with a lookback, as in Q89) and keep the full-history version weekly. An index on `OrderDetails(OrderID)` helps the incremental version; the full scan does not need one. MySQL has no FULL OUTER JOIN: emulate it with a LEFT JOIN UNION ALL the right-only rows.

Question 87: Referential integrity audit across all seven relationships

Difficulty: Expert

Primary Topic: Referential integrity validation

Combined Topics: UNION ALL of anti-joins, NOT EXISTS, NULL FK vs broken FK, Conditional aggregation with FILTER, Relationship spine with VALUES

Business Scenario: The shop's tables arrive as nightly CSV loads with no foreign key constraints. The data platform team wants to add foreign keys to the warehouse copy so bad rows are rejected at load time. Before they switch that on, they need one audit that says, for every relationship, how many child rows would be rejected and which parent keys are missing, so they can send a list to each source owner.

Problem Statement: Audit all seven documented relationships in one query. For each one, count the child rows, the child rows whose foreign key is NULL, and the child rows whose foreign key is not NULL but has no matching parent (orphans). Show up to five distinct orphan key values as a sample. Return all seven relationships every night, including clean ones, in a fixed order.

Expected Output:

- `relationship`: written as `Child.Column -> Parent`
- `child_rows`: rows in the child table
- `null_fk_rows`: child rows where the foreign key is NULL
- `orphan_rows`: child rows where the foreign key is not NULL and no parent row has that key
- `sample_orphan_keys`: up to five distinct orphan keys, ascending, comma separated (NULL when there are none)

Tables Required: `Products`, `Category`, `Orders`, `Customers`, `Payments`, `Shippers`, `OrderDetails`, `Suppliers`

SQL Concepts Tested: anti-join with NOT EXISTS, why NOT IN fails with NULLs, separating missing keys from broken keys, stacking many checks with UNION ALL, FILTER on aggregates, array slicing for samples.

Step 1 — Understand the Problem: Assumptions:

- The seven relationships are the ones in the schema: `Products.CategoryID`, `Orders.CustomerID`, `Orders.PaymentID`, `Orders.ShipperID`, `OrderDetails.OrderID`, `OrderDetails.ProductID`, `OrderDetails.SupplierID`.
- Parent keys are the parent tables' primary keys, so they are unique.
- All keys are integers. If a load ever stores them as text, `'12 '` and `12` will not match and you will see false orphans; cast and trim before the check.

The core distinction: a NULL foreign key means "no value was recorded". That can be legitimate (an order not yet assigned a shipper) or a load defect, but it is a completeness problem. An orphan means "a value was recorded and it points at nothing". That is always a defect, usually a parent row that was deleted or never loaded. They have different owners and different fixes, so they get separate columns. A foreign key constraint will not catch the first kind: under SQL's default `MATCH SIMPLE`, a NULL child key passes the constraint. Only `NOT NULL` stops it.

Step 2 — Identify the Tables and Relationships: Three child tables (`Products`, `Orders`, `OrderDetails`) and six parent tables. `Orders` is both: it is a child of `Customers`, `Payments` and `Shippers`, and the parent of `OrderDetails`. Every relationship is many-to-one from child to parent.

Step 3 — Build the Query Logic:

1. For each relationship, select every child row with two things: the foreign key value, and a boolean saying whether a parent with that key exists.
2. Compute the boolean with `NOT EXISTS`. It checks existence without joining rows, so a parent table with duplicate keys could not inflate the child count.
3. Tag each block with a relationship number and stack the seven blocks with UNION ALL into one long table.
4. Drive the final query from a fixed list of seven relationships, LEFT JOIN the stacked rows, and aggregate with FILTER.
5. Collect distinct orphan keys into a sorted array, keep the first five, and turn them into a string.

Step 4 — Final SQL Solution:

```sql
WITH relationships (rel_id, relationship) AS (
    VALUES (1, 'Products.CategoryID -> Category'),
           (2, 'Orders.CustomerID -> Customers'),
           (3, 'Orders.PaymentID -> Payments'),
           (4, 'Orders.ShipperID -> Shippers'),
           (5, 'OrderDetails.OrderID -> Orders'),
           (6, 'OrderDetails.ProductID -> Products'),
           (7, 'OrderDetails.SupplierID -> Suppliers')
),
child_keys AS (
    SELECT 1 AS rel_id, p.CategoryID AS fk_value,
           NOT EXISTS (SELECT 1 FROM Category AS cat WHERE cat.CategoryID = p.CategoryID) AS no_parent
    FROM Products AS p
    UNION ALL
    SELECT 2, o.CustomerID,
           NOT EXISTS (SELECT 1 FROM Customers AS c WHERE c.CustomerID = o.CustomerID)
    FROM Orders AS o
    UNION ALL
    SELECT 3, o.PaymentID,
           NOT EXISTS (SELECT 1 FROM Payments AS pay WHERE pay.PaymentID = o.PaymentID)
    FROM Orders AS o
    UNION ALL
    SELECT 4, o.ShipperID,
           NOT EXISTS (SELECT 1 FROM Shippers AS sh WHERE sh.ShipperID = o.ShipperID)
    FROM Orders AS o
    UNION ALL
    SELECT 5, od.OrderID,
           NOT EXISTS (SELECT 1 FROM Orders AS o WHERE o.OrderID = od.OrderID)
    FROM OrderDetails AS od
    UNION ALL
    SELECT 6, od.ProductID,
           NOT EXISTS (SELECT 1 FROM Products AS p WHERE p.ProductID = od.ProductID)
    FROM OrderDetails AS od
    UNION ALL
    SELECT 7, od.SupplierID,
           NOT EXISTS (SELECT 1 FROM Suppliers AS s WHERE s.SupplierID = od.SupplierID)
    FROM OrderDetails AS od
)
SELECT r.relationship,
       COUNT(ck.rel_id)                                                    AS child_rows,
       COUNT(ck.rel_id) FILTER (WHERE ck.fk_value IS NULL)                 AS null_fk_rows,
       COUNT(ck.rel_id) FILTER (WHERE ck.fk_value IS NOT NULL AND ck.no_parent) AS orphan_rows,
       ARRAY_TO_STRING(
           (ARRAY_AGG(DISTINCT ck.fk_value ORDER BY ck.fk_value)
                FILTER (WHERE ck.fk_value IS NOT NULL AND ck.no_parent))[1:5],
           ', ')                                                           AS sample_orphan_keys
FROM relationships AS r
LEFT JOIN child_keys AS ck ON ck.rel_id = r.rel_id
GROUP BY r.rel_id, r.relationship
ORDER BY r.rel_id;
```

Step 5 — Line-by-Line Explanation:

- `relationships` is the spine. It fixes the row order and keeps a relationship in the output even if its child table is empty.
- Each `child_keys` block emits one row per child row. Column names come from the first block; later blocks only need matching types and positions.
- `NOT EXISTS (... WHERE parent.key = child.key)`: when the child key is NULL, the `=` is never true, so `no_parent` is TRUE. That is why the orphan count also requires `fk_value IS NOT NULL`. Without it, every NULL key would be counted twice, once as NULL and once as an orphan.
- `COUNT(ck.rel_id)` rather than `COUNT(*)`: a relationship with zero child rows still gets one row from the LEFT JOIN, and `COUNT(*)` would report 1.
- `ARRAY_AGG(DISTINCT ... ORDER BY ...) FILTER (...)` collects each orphan key once, sorted. With `DISTINCT`, the `ORDER BY` must use the aggregated expression, which it does.
- `[1:5]` slices the array to five keys. `ARRAY_TO_STRING` turns it into `99001, 99002`. If there are no orphans, the FILTER leaves the aggregate NULL and the column stays NULL.

Step 6 — Sample Output:

relationship	child_rows	null_fk_rows	orphan_rows	sample_orphan_keys
Products.CategoryID -> Category	22088	0	40	5012
Orders.CustomerID -> Customers	46438	0	8	99001, 99003
Orders.PaymentID -> Payments	46438	20	0	NULL
Orders.ShipperID -> Shippers	46438	25	3	11
OrderDetails.OrderID -> Orders	168567	0	12	5999001, 5999002, 5999003
OrderDetails.ProductID -> Products	168567	0	15	30001, 30002, 30003, 30004
OrderDetails.SupplierID -> Suppliers	168567	0	5	21

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 7 rows returned.

Step 7 — Why This Solution Works: Every child row is counted exactly once in exactly one relationship block, and the three counts are defined so they cannot overlap: a row is NULL, an orphan, or valid. A quick check: for any relationship, `child_rows` must equal the row count of the child table. `NOT EXISTS` is safe whatever the parent looks like, because it answers yes or no per child row and never multiplies rows.

How to run it as a nightly control: store each run's rows with a run date. Alert on any `orphan_rows > 0` for `OrderDetails.OrderID` and `OrderDetails.ProductID`, because those break revenue and units reporting directly. For `Orders.PaymentID` and `Orders.ShipperID`, alert on change: a NULL shipper on today's orders may only mean "not dispatched yet", but a jump from 3 to 300 means the source stopped sending the column. The sample keys tell the source owner where to look. A run of consecutive keys usually means a parent batch was never loaded; a single high key like `999` usually means a test or placeholder value.

Order matters when you fix things: load or repair parents first, then rerun the audit before touching children.

Step 8 — Common Mistakes:

- Using `NOT IN` against a parent column that can hold NULL. One NULL parent key makes `x NOT IN (...)` never true, and the check reports zero orphans. Child rows with a NULL key also drop out silently.

```
SELECT COUNT(*) AS orphan_lines_not_in
FROM OrderDetails AS od
WHERE od.OrderID NOT IN (SELECT o.OrderID FROM Orders AS o);
```

- Counting orphans with `LEFT JOIN ... WHERE parent.key IS NULL` and forgetting the child key itself can be NULL. NULL child keys then appear as orphans, which mixes a completeness problem into a break count and sends it to the wrong owner.
- Running seven separate queries with seven shapes. A relationship that was never checked looks the same as one that passed.
- Checking only `Orders` → `OrderDetails` in one direction. Orders with no lines are not a referential integrity break (a parent can have zero children), so they belong in the reconciliation check of Q86, not here.

Step 9 — Interview Follow-Up: The audit shows 40,000 orphan lines on `OrderDetails.OrderID`, all from yesterday's load. What would you check first to tell "headers not loaded yet" apart from "headers deleted upstream", and how would you change the nightly job so this cannot reach the dashboards again?

Step 10 — Optimization or Alternative: The LEFT JOIN anti-join is the other common form. It is fine here because each parent key is unique, but it would fan out if a parent table had duplicate keys:

```
SELECT 'OrderDetails.OrderID -> Orders'                              AS relationship,
       COUNT(*)                                                      AS child_rows,
       COUNT(*) FILTER (WHERE od.OrderID IS NULL)                    AS null_fk_rows,
       COUNT(*) FILTER (WHERE od.OrderID IS NOT NULL
                         AND o.OrderID IS NULL)                      AS orphan_rows
FROM OrderDetails AS od
LEFT JOIN Orders AS o ON o.OrderID = od.OrderID;
```

Performance: because each `NOT EXISTS` sits in the select list, PostgreSQL runs it as a hashed subplan (check `EXPLAIN VERBOSE`): it reads the parent keys once into memory and probes them for every child row. If a parent's keys do not fit in `work_mem`, the planner falls back to one primary-key index probe per child row, which is still correct but slower. On a large line table, do not rescan history every night: audit only child rows loaded since the last run (by `OrderDetailID` watermark), and rerun the full audit weekly to catch parents deleted after their children loaded.

Once the counts reach zero, move the rule into the database and keep this query as the monitor. `NOT VALID` adds the constraint for new rows without checking old ones, and `VALIDATE CONSTRAINT` later checks history without blocking writes:

```
ALTER TABLE OrderDetails
  ADD CONSTRAINT fk_orderdetails_orders
  FOREIGN KEY (OrderID) REFERENCES Orders (OrderID) NOT VALID;
ALTER TABLE OrderDetails VALIDATE CONSTRAINT fk_orderdetails_orders;  -- fails while orphans exist
ALTER TABLE OrderDetails ALTER COLUMN OrderID SET NOT NULL;           -- the FK alone allows NULL
```

On MySQL, swap the array expression for `GROUP_CONCAT(DISTINCT ... ORDER BY ... SEPARATOR ', ')` and trim to five keys with `SUBSTRING_INDEX`.

Question 88: Date validity audit

Difficulty: Expert

Primary Topic: Invalid date detection

Combined Topics: Rule table with CROSS JOIN LATERAL (VALUES ...), UNION ALL across tables, Anchored date logic, LEFT JOIN

Business Scenario: The logistics team's delivery-time dashboard shows a handful of negative delivery times and a few orders that seem to take forever. The data quality lead wants two things from one rule set: a daily scorecard with a count per rule, zeros included, and an exception list the support team can work through, one row per broken rule per record. Error rows will be excluded from delivery KPIs until fixed; warnings stay in but get reviewed.

Problem Statement: Check every order and every customer against the date rules below. Return one row per rule: its number, name and severity, how many records were checked, how many break it, the violation rate, and up to five example IDs. A rule with no violations must still appear, with 0. The row-level exception list goes in Step 10. Rules (all day differences in whole days):

#	Rule	Severity	Condition
1	Order date missing	error	`OrderDate` is NULL
2	Shipped before ordered	error	`ShipDate < OrderDate`
3	Delivered before shipped	error	`DeliveryDate < ShipDate`

4	Delivered before ordered	error	`DeliveryDate < OrderDate`
5	Delivered with no ship date	error	`DeliveryDate` present, `ShipDate` NULL
6	Ordered before customer sign-up	warning	`OrderDate < DateEntered`
7	Delivered after the 7-day promise	warning	`DeliveryDate - OrderDate > 7`
8	Not shipped 14+ days after order	warning	no ship or delivery date, and the latest order date in the data is 14+ days after `OrderDate`
9	Customer sign-up date missing	warning	`DateEntered` is NULL

Expected Output:

- `rule_id`, `rule`, `severity`: from the table above
- `records_checked`: orders for rules 1-8, customers for rule 9
- `violations`: records that break the rule (0 when none)
- `violation_pct`: violations as a percentage of records checked, 2 decimals
- `example_ids`: up to five `OrderID`s (or `CustomerID`s for rule 9) that break it, lowest first; NULL when there are none

Tables Required: `Orders`, `Customers`

SQL Concepts Tested: expressing rules as data with LATERAL VALUES, three-valued logic in comparisons, UNION ALL with aligned columns and typed NULLs, anchoring "today" to the data, LEFT JOIN to keep orders with no customer.

Step 1 — Understand the Problem: Assumptions:

- Dates are stored as DATE, so rules compare days. If they arrive as timestamps from different systems, cast with `::date` first, or a few minutes of clock skew between the order server and the warehouse scanner shows up as "shipped before ordered".
- "Today" for rule 8 is the latest `OrderDate` in the data, not `CURRENT_DATE`, so the report gives the same answer when rerun on a historical snapshot.
- Rule 7 uses the house one-week delivery promise (7 days); rule 8's 14 days is a starting threshold for review. No promised delivery date exists in the schema.
- A missing ship or delivery date alone is not a violation: the order may not have shipped yet. Rule 8 catches the ones waiting too long.

The design question is how to check nine rules without nine queries. Each rule is a boolean condition on one row. Lay the rules out as rows of a small table, attach it to every order, and each order is checked against every rule in one pass. Counting the rows where the condition is true, per rule, gives the scorecard; listing them gives the exception report.

Step 2 — Identify the Tables and Relationships: `Orders` holds three of the four dates. `Customers.DateEntered` is needed for rule 6, through `Orders.CustomerID` → `Customers.CustomerID` (many to one). Rule 9 is about the customer alone, so it reads `Customers` directly.

Step 3 — Build the Query Logic:

1. Compute the anchor date once: `MAX(OrderDate)`.
2. For orders, LEFT JOIN `Customers` (orders with a broken `CustomerID` must still be checked), then `CROSS JOIN LATERAL (VALUES ...)` with one row per rule. Each row carries the rule number, name, severity and the boolean result of its condition for that order. Keep every row, violated or not: the non-violations are what make zero counts possible.
3. Build the customer-level rule the same way, one row per customer.
4. UNION ALL the two, group by rule, and count with `FILTER (WHERE violated)`.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate) AS max_order_date
    FROM Orders AS o
),
order_checks AS (
    SELECT r.rule_id, r.rule, r.severity, o.OrderID AS record_id, r.violated
    FROM Orders AS o
    LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
    CROSS JOIN anchor AS a
    CROSS JOIN LATERAL (
        VALUES
            (1, 'Order date missing',               'error',   o.OrderDate IS NULL),
            (2, 'Shipped before ordered',           'error',   o.ShipDate < o.OrderDate),
            (3, 'Delivered before shipped',         'error',   o.DeliveryDate < o.ShipDate),
            (4, 'Delivered before ordered',         'error',   o.DeliveryDate < o.OrderDate),
            (5, 'Delivered with no ship date',      'error',   o.DeliveryDate IS NOT NULL AND o.ShipDate IS NULL),
            (6, 'Ordered before customer sign-up',  'warning', o.OrderDate < c.DateEntered),
            (7, 'Delivered after 7-day promise',    'warning', o.DeliveryDate::date - o.OrderDate::date > 7),
            (8, 'Not shipped 14+ days after order', 'warning', o.ShipDate IS NULL AND o.DeliveryDate IS NULL
                                                               AND a.max_order_date::date - o.OrderDate::date >= 14)
    ) AS r(rule_id, rule, severity, violated)
),
customer_checks AS (
    SELECT 9                              AS rule_id,
           'Customer sign-up date missing' AS rule,
           'warning'                      AS severity,
           c.CustomerID                   AS record_id,
           c.DateEntered IS NULL          AS violated
    FROM Customers AS c
),
all_checks AS (
    SELECT * FROM order_checks
    UNION ALL
    SELECT * FROM customer_checks
)
SELECT ac.rule_id,
       ac.rule,
       ac.severity,
       COUNT(*)                                                                    AS records_checked,
       COUNT(*) FILTER (WHERE ac.violated)                                         AS violations,
       ROUND(100.0 * COUNT(*) FILTER (WHERE ac.violated) / NULLIF(COUNT(*), 0), 2) AS violation_pct,
       (ARRAY_AGG(ac.record_id ORDER BY ac.record_id) FILTER (WHERE ac.violated))[1:5] AS example_ids
FROM all_checks AS ac
GROUP BY ac.rule_id, ac.rule, ac.severity
ORDER BY ac.rule_id;
```

Step 5 — Line-by-Line Explanation:

- `anchor` is a one-row CTE. `CROSS JOIN anchor` puts `max_order_date` on every order row without a subquery per row.
- `LEFT JOIN Customers`: with an INNER JOIN, an order whose customer is missing would skip all eight rules, including the ones that have nothing to do with the customer.
- `CROSS JOIN LATERAL (VALUES ...)`: LATERAL lets the VALUES list read columns of the current order row. Each order becomes eight rows, one per rule, so every rule has a row to count even when nothing breaks it.
- `violated` can be TRUE, FALSE or NULL. `o.ShipDate < o.OrderDate` is NULL when either date is missing, and `FILTER (WHERE ac.violated)` counts only TRUE. So a comparison rule never fires on missing data; missing data has its own rules (1, 5, 8, 9).
- Rule 6 on an order with no customer row compares against a NULL `DateEntered`, so it does not fire. Broken customer keys belong to the integrity audit in Q87.
- `customer_checks` has the same five columns as `order_checks`, so UNION ALL lines them up.
- `ARRAY_AGG(... ORDER BY ...) FILTER (WHERE ...)` collects the violating IDs in order; `[1:5]` slices the first five. It is NULL when nothing qualifies.

Step 6 — Sample Output:

rule_id	rule	severity	records_checked	violations	violation_pct	example_ids
1	Order date missing	error	46438	0	0.00	NULL
2	Shipped before ordered	error	46438	12	0.03	[5001735, 5002010, 5007823, 5011889, 5014403]
3	Delivered before shipped	error	46438	9	0.02	[5004120, 5005142, 5006412, 5019765, 5020419]
4	Delivered before ordered	error	46438	4	0.01	[5004120, 5020419, 5020728, 5023506]
5	Delivered with no ship date	error	46438	6	0.01	[5018853, 5024291, 5035121, 5035401, 5037129]
6	Ordered before customer sign-up	warning	46438	29	0.06	[5002922, 5008219, 5008989, 5009964, 5010462]
7	Delivered after 7-day promise	warning	46438	6350	13.67	[5000014, 5000016, 5000018, 5000022, 5000024]
8	Not shipped 14+ days after order	warning	46438	320	0.69	[5000121, 5000239, 5000297, 5000397, 5000589]

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 9 rows.

Step 7 — Why This Solution Works: Each rule is evaluated once per order, in a single scan of `Orders`, and every rule produces a row whether it fires or not. A silent rule shows as 0 rather than disappearing, which is the difference between "checked and clean" and "not checked". On this dataset every error rule except a missing order date fires a handful of times, which is what a real feed looks like: few

enough to fix by hand, and enough to break a delivery KPI if left in. Because the rules are rows, adding a tenth is one line in the VALUES list rather than a new query, and the rule number gives every violation a stable identity across nightly runs.

The severity split is what makes it usable downstream. Rules 2 to 5 describe sequences that cannot happen, so any delivery-time KPI should exclude those orders (this is the "exclude impossible sequences" rule the delivery questions use). Rule 6 often has a process cause: orders re-pointed to a newer duplicate account during a merge, or `DateEntered` overwritten when a profile is updated. That is why it is a warning and not an error. Rules 7 and 8 are operational: they find stuck shipments, not bad data.

Nightly, store this summary with the run date and alert when error counts rise above zero or warnings jump. A spike in rule 3 or 5 usually points at one carrier's feed, so add `ShipperID` when you investigate.

Step 8 — Common Mistakes:

- Replacing missing dates with a sentinel before comparing. Every unshipped order now "ships before it was ordered", and the error count is mostly noise:

```
SELECT COUNT(*) AS shipped_before_ordered_wrong
FROM Orders AS o
WHERE COALESCE(o.ShipDate, DATE '1900-01-01') < o.OrderDate;
```

- Using `CURRENT_DATE` for rule 8. On a historical snapshot every open order looks months overdue, and the same query gives a different answer each day.
- Comparing timestamps without casting. `DeliveryDate - OrderDate` on timestamps is an interval, so `> 7` is a type error, and same-day events compare by clock time.
- Building the customer block of the Step 10 listing with plain `NULL`. Inside a CTE, an untyped NULL column resolves to text, and the union then fails with "UNION types date and text cannot be matched":

```
WITH customer_rows AS (SELECT c.CustomerID, NULL AS OrderDate FROM Customers AS c)
SELECT o.CustomerID, o.OrderDate FROM Orders AS o
UNION ALL
SELECT cr.CustomerID, cr.OrderDate FROM customer_rows AS cr;
```

Step 9 — Interview Follow-Up: The business now wants rule 7 to use a different promise per shipper (express carriers 14 days, ocean freight 28). Where would you store the thresholds, and how does the LATERAL rule table change?

Step 10 — Optimization or Alternative: The support team needs the rows behind each count. The same rule table produces the exception list: keep only violated rows, and give the customer-level block typed NULLs for the order columns:

```
WITH anchor AS (
    SELECT MAX(o.OrderDate) AS max_order_date
    FROM Orders AS o
),
order_violations AS (
    SELECT r.rule_id, r.rule, r.severity,
           o.OrderID, o.CustomerID, o.OrderDate, o.ShipDate, o.DeliveryDate,
           c.DateEntered
    FROM Orders AS o
    LEFT JOIN Customers AS c ON c.CustomerID = o.CustomerID
    CROSS JOIN anchor AS a
    CROSS JOIN LATERAL (
        VALUES
            (1, 'Order date missing',               'error',   o.OrderDate IS NULL),
            (2, 'Shipped before ordered',           'error',   o.ShipDate < o.OrderDate),
            (3, 'Delivered before shipped',         'error',   o.DeliveryDate < o.ShipDate),
            (4, 'Delivered before ordered',         'error',   o.DeliveryDate < o.OrderDate),
            (5, 'Delivered with no ship date',      'error',   o.DeliveryDate IS NOT NULL AND o.ShipDate IS NULL),
            (6, 'Ordered before customer sign-up',  'warning', o.OrderDate < c.DateEntered),
            (7, 'Delivered after 7-day promise',    'warning', o.DeliveryDate::date - o.OrderDate::date > 7),
            (8, 'Not shipped 14+ days after order', 'warning', o.ShipDate IS NULL AND o.DeliveryDate IS NULL
                                                               AND a.max_order_date::date - o.OrderDate::date >= 14)
    ) AS r(rule_id, rule, severity, violated)
    WHERE r.violated
),
customer_violations AS (
    SELECT 9 AS rule_id, 'Customer sign-up date missing' AS rule, 'warning' AS severity,
           NULL::int AS OrderID, c.CustomerID,
           NULL::date AS OrderDate, NULL::date AS ShipDate, NULL::date AS DeliveryDate,
           c.DateEntered
    FROM Customers AS c
    WHERE c.DateEntered IS NULL
)
SELECT v.rule, v.severity, v.OrderID, v.CustomerID,
       v.OrderDate, v.ShipDate, v.DeliveryDate, v.DateEntered
FROM (
    SELECT * FROM order_violations
    UNION ALL
    SELECT * FROM customer_violations
) AS v
ORDER BY v.rule_id, v.OrderID NULLS LAST, v.CustomerID;
```

Performance: the LATERAL form scans `Orders` once and multiplies rows in memory by the number of rules, which is cheaper than eight separate scans glued with UNION ALL. The cost is CPU per rule per row, so on hundreds of millions of orders, run the detail report only for orders placed or changed recently, and keep the full-history summary weekly. No index helps a full audit; an index on `Orders(OrderDate)` helps the recent-window version. MySQL 8.0.14+ supports `LATERAL`; the usual port replaces the VALUES list with one-row `SELECT`s glued by UNION ALL inside the lateral subquery.

Question 89: Incremental load with a watermark and lookback

Difficulty: Expert

Primary Topic: Incremental extraction by date

Combined Topics: Watermark CTE, INSERT ... ON CONFLICT upsert, Data-modifying CTE with RETURNING, IS DISTINCT FROM change detection

Business Scenario: The analytics warehouse keeps a copy of `Orders` that is rebuilt from scratch every night. The full reload now takes too long and blocks the morning dashboards. The data engineering team wants an incremental job: pull only recent orders, insert the new ones, update the ones that changed, and leave everything else alone. The job must be safe to rerun after a failure, and it must report what it did so the scheduler can log it.

Problem Statement: Write one nightly load from `Orders` into a target table `orders_target` with the same columns and primary key plus an `etl_updated_at` timestamp. The batch is every source order with `OrderDate` on or after the target's latest `OrderDate` minus a 21-day lookback, plus any source order with a NULL `OrderDate`. Insert orders the target does not have. Update orders whose values differ. Do not touch identical rows. Return one row per action with its row count. For the demo, create the target as a TEMP table and seed it as the previous night's run would have left it.

Expected Output:

- `action`: `inserted`, `updated` or `unchanged`, always all three, in that order
- `rows`: number of batch rows that got that action

Tables Required: `Orders` (the TEMP target is created inside the solution)

SQL Concepts Tested: high-water marks, lookback windows for late changes, idempotent upserts, `ON CONFLICT DO UPDATE ... WHERE`, NULL-safe row comparison, data-modifying CTEs and their shared snapshot.

Step 1 — Understand the Problem: Assumptions and the limitation that shapes everything:

- `Orders` has no `updated_at` column and no change log. The only time-like column is `OrderDate`, which records when the order was placed, not when the row last changed.
- `ShipDate`, `DeliveryDate` and occasionally `Total_Order_Amount` are filled in or corrected days after the order is placed.
- Deletes in the source are not handled by this job.

With an `updated_at`, the watermark would be "rows changed since the last run" and the job would be exact. Without it, the best available proxy is "rows placed recently". The lookback re-reads the last 21 days of orders every night, which catches late ship and delivery dates as long as they arrive within 21 days of the order. Anything that changes later is missed. That is a real gap, not a rounding error, and Step 10 shows how to size the lookback and close the gap.

Step 2 — Identify the Tables and Relationships: One source, `Orders`, keyed by `OrderID`. One target, `orders_target`, created with `LIKE Orders INCLUDING ALL`, which copies columns, defaults and the primary key. The primary key is what `ON CONFLICT (OrderID)` needs: an upsert requires a unique index to detect the conflict.

Step 3 — Build the Query Logic:

1. Demo only: create the target and seed it as a run 14 days before the latest order would have left it.
2. Read the watermark from the target: `MAX(OrderDate)`, or a far-past date if the target is empty, so the first run is a full load.
3. Select the batch: `OrderDate >= watermark - 21 days`, plus NULL order dates, which no date filter reaches. Mark whether the target already has each row.
4. Upsert with `INSERT ... ON CONFLICT (OrderID) DO UPDATE`, only when a column differs.
5. Use `RETURNING` to get the keys actually written, and classify each batch row.

Step 4 — Final SQL Solution:

```sql
-- Target table: same columns and primary key as Orders, plus a load timestamp.
CREATE TEMP TABLE orders_target (
    LIKE Orders INCLUDING ALL,
    etl_updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Seed it as the previous run left it: that run happened 14 days before the
-- latest order, so it held no later orders and no later ship/delivery dates.
INSERT INTO orders_target
       (OrderID, CustomerID, PaymentID, OrderDate, ShipperID,
        Total_Order_Amount, ShipDate, DeliveryDate)
SELECT o.OrderID, o.CustomerID, o.PaymentID, o.OrderDate, o.ShipperID,
       o.Total_Order_Amount,
       CASE WHEN o.ShipDate     <= prev.run_date THEN o.ShipDate     END,
       CASE WHEN o.DeliveryDate <= prev.run_date THEN o.DeliveryDate END
FROM Orders AS o
CROSS JOIN (SELECT MAX(src.OrderDate) - 14 AS run_date FROM Orders AS src) AS prev
WHERE o.OrderDate <= prev.run_date;

-- Tonight's incremental load.
WITH params AS (
    SELECT 21 AS lookback_days
),
watermark AS (
    SELECT COALESCE(MAX(t.OrderDate), DATE '1900-01-01') AS high_water_date
    FROM orders_target AS t
),
batch AS (
    SELECT o.*,
           EXISTS (SELECT 1 FROM orders_target AS t
                   WHERE t.OrderID = o.OrderID) AS already_loaded
    FROM Orders AS o
    CROSS JOIN watermark AS w
    CROSS JOIN params    AS p
    WHERE o.OrderDate >= w.high_water_date - p.lookback_days
       OR o.OrderDate IS NULL
),
upserted AS (
    INSERT INTO orders_target AS t
           (OrderID, CustomerID, PaymentID, OrderDate, ShipperID,
            Total_Order_Amount, ShipDate, DeliveryDate)
    SELECT b.OrderID, b.CustomerID, b.PaymentID, b.OrderDate, b.ShipperID,
           b.Total_Order_Amount, b.ShipDate, b.DeliveryDate
    FROM batch AS b
    ON CONFLICT (OrderID) DO UPDATE
       SET CustomerID         = EXCLUDED.CustomerID,
           PaymentID          = EXCLUDED.PaymentID,
           OrderDate          = EXCLUDED.OrderDate,
           ShipperID          = EXCLUDED.ShipperID,
           Total_Order_Amount = EXCLUDED.Total_Order_Amount,
           ShipDate           = EXCLUDED.ShipDate,
           DeliveryDate       = EXCLUDED.DeliveryDate,
           etl_updated_at     = now()
     WHERE (t.CustomerID, t.PaymentID, t.OrderDate, t.ShipperID,
            t.Total_Order_Amount, t.ShipDate, t.DeliveryDate)
           IS DISTINCT FROM
           (EXCLUDED.CustomerID, EXCLUDED.PaymentID, EXCLUDED.OrderDate, EXCLUDED.ShipperID,
            EXCLUDED.Total_Order_Amount, EXCLUDED.ShipDate, EXCLUDED.DeliveryDate)
    RETURNING t.OrderID
),
outcome AS (
    SELECT CASE WHEN u.OrderID IS NULL THEN 'unchanged'
                WHEN b.already_loaded  THEN 'updated'
                ELSE 'inserted'
           END AS action
    FROM batch AS b
    LEFT JOIN upserted AS u ON u.OrderID = b.OrderID
),
actions (sort_order, action) AS (
    VALUES (1, 'inserted'), (2, 'updated'), (3, 'unchanged')
)
SELECT a.action,
       COUNT(oc.action) AS rows
FROM actions AS a
LEFT JOIN outcome AS oc ON oc.action = a.action
GROUP BY a.sort_order, a.action
ORDER BY a.sort_order;
```

Step 5 — Line-by-Line Explanation:

- The seed's `CASE WHEN o.ShipDate <= prev.run_date` blanks dates that had not happened by the previous run. Those orders come back tonight as updates.
- `watermark` reads the target, not a stored parameter. If last night's run failed halfway, tonight's run starts from whatever actually landed.
- `>=`, not `>`: orders placed on the watermark date after the last run started would be skipped by `>`. Re-reading the boundary day is harmless because the upsert is idempotent.
- `OR o.OrderDate IS NULL`: a NULL date fails every comparison, so these orders would never load without it.
- `already_loaded` is computed inside the same statement as the insert. All CTEs in one statement see the table as it was before the statement began, so this reflects the target before tonight's writes.
- `DO UPDATE ... WHERE (...) IS DISTINCT FROM (...)` skips identical rows. Row-wise `IS DISTINCT FROM` treats two NULLs as equal and NULL vs a value as different. Plain `<>` returns NULL when `DeliveryDate` goes from NULL to a date, the `WHERE` fails, and the newly arrived delivery date is never written.

- `RETURNING t.OrderID` returns only rows that were inserted or actually updated. A conflict whose `WHERE` was false returns nothing.
- `outcome` joins every batch row to what was written. No returned key means unchanged; a returned key on a row the target already had means updated.

Step 6 — Sample Output:

action	rows
inserted	1423
updated	494
unchanged	1817

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 3 rows returned.

Step 7 — Why This Solution Works: The job is idempotent: run it twice and the second run reports zero inserted, zero updated and the whole batch unchanged, because nothing differs. That property is what makes retries safe. It is also a control total: `inserted + updated + unchanged` must equal the batch size, and `SELECT COUNT(*) FROM orders_target` must grow by exactly `inserted`.

What should page someone: zero inserted on a normal business day (the source stopped sending), updated far above its usual level (a source backfill or a bulk correction you were not told about), or a target row count that falls behind the source by more than the lookback can explain. Log the three counts plus the watermark for every run.

Skipping identical rows matters beyond speed. Every rewrite creates a dead tuple and bumps `etl_updated_at`, so downstream jobs that read changes would believe 21 days of data changed every night.

Step 8 — Common Mistakes:

- Using only `OrderDate > watermark` with no lookback. New orders load, but a delivery date that arrives tomorrow never reaches the warehouse, and delivery KPIs drift further from the source every day.
- Upserting from a staging table with duplicate `OrderID`s. PostgreSQL stops with "ON CONFLICT DO UPDATE command cannot affect row a second time". Deduplicate the batch first.
- Detecting inserts with `RETURNING (xmax = 0)`. It works today but relies on an internal system column, not a documented contract.
- Writing `DO UPDATE SET ...` with no `WHERE`. The counts become meaningless (everything in the lookback is "updated") and the table bloats.
- Taking the watermark from `CURRENT_DATE - 21`. If the job is down for a month, it comes back and silently skips the gap. A target-derived watermark cannot skip.

Step 9 — Interview Follow-Up: Customer support occasionally corrects `Total_Order_Amount` on orders that are three months old. Without adding an `updated_at` column to the source, how would you make sure those corrections reach the warehouse, and what would it cost?

Step 10 — Optimization or Alternative: Size the lookback from the data rather than guessing. Measure how long after the order date the last field usually changes; delivery is the latest event in this schema:

```
SELECT PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY o.DeliveryDate - o.OrderDate) AS p99_days_to_delivery,
       MAX(o.DeliveryDate - o.OrderDate)                                          AS max_days_to_delivery
FROM Orders AS o
WHERE o.DeliveryDate >= o.OrderDate;
```

Set the lookback comfortably above the p99 and review it quarterly. For changes the lookback misses, run a weekly full comparison: hash the same business columns on both sides (`md5(ROW(o.CustomerID, o.ShipDate, ...)::text)`) and upsert the differences. The real fix is upstream: an `updated_at` maintained by a trigger, or change data capture from the database log, turns the watermark into "changed since". Source deletes need CDC or a periodic anti-join.

PostgreSQL 15+ also offers `MERGE`, which reads closer to the business rule and is the portable form for Snowflake, SQL Server and Databricks:

```
MERGE INTO orders_target AS t
USING batch AS b ON t.OrderID = b.OrderID
WHEN MATCHED AND (t.ShipDate, t.DeliveryDate, ...) IS DISTINCT FROM (b.ShipDate, b.DeliveryDate, ...)
     THEN UPDATE SET ShipDate = b.ShipDate, DeliveryDate = b.DeliveryDate, ...
WHEN NOT MATCHED THEN INSERT (...) VALUES (...);
```

`MERGE` has no `RETURNING` before PostgreSQL 17, so the counts are harder to get. MySQL uses `INSERT ... ON DUPLICATE KEY UPDATE` with the NULL-safe `<=>` per column.

Performance: the batch filter needs an index on `Orders(OrderDate)` in the source, or the job scans the whole table to read three weeks of rows. The target's primary key serves both the conflict check and `already_loaded`. The watermark and the batch are read in one statement, so they share one snapshot; an order committed a second later is picked up the next night, because the lookback re-reads its date.

Question 90: Column-by-column completeness profile of Orders

Difficulty: Expert

Primary Topic: NULL and missing-value analysis

Combined Topics: Unpivot with CROSS JOIN LATERAL (VALUES ...), Conditional aggregation with FILTER, COUNT DISTINCT, Anchored recent window

Business Scenario: The data platform team is writing a data contract for `Orders` with the checkout team that owns the source. The contract will say, column by column, which fields may be NULL and how many distinct values are normal. Before negotiating, they need a profile of what the table looks like today, in a shape that can be stored nightly and compared run to run.

Problem Statement: Profile every column of `Orders` in one query. Return one row per column, in table order, with the total row count, the number and percentage of NULLs, the NULL percentage among orders from the last 30 days of data, and the number of distinct non-NULL values.

Expected Output:

- `column_name`: the column, in table order
- `total_rows`: rows in `Orders` (same on every row, a built-in check)
- `null_rows`: rows where the column is NULL
- `null_pct`: `null_rows` as a percentage of all rows, 2 dp
- `null_pct_last_30d`: the same percentage for orders with `OrderDate` in the 30 days ending at the latest order date
- `distinct_values`: distinct non-NULL values

Tables Required: `Orders`

SQL Concepts Tested: unpivoting columns into rows with LATERAL VALUES, casting mixed types to a common type, `COUNT(*)` vs `COUNT(col)`, FILTER, COUNT DISTINCT ignoring NULLs, anchoring a recent window to the data.

Step 1 — Understand the Problem: Assumptions:

- The profile covers the eight documented columns. A column added later must be added to the list; Step 10 shows a version that picks columns up automatically.
- "Last 30 days" is anchored to `MAX(OrderDate)`, not `CURRENT_DATE`, so the profile is reproducible on a snapshot.
- Orders with a NULL `OrderDate` fall outside the recent window, because the window test on them is NULL. They still count in `null_pct`.

The output grain is one row per column, but SQL aggregates run down columns, not across them. Eight columns of different types also cannot sit in one result column unless they share a type. So the work is: turn each order into eight `(column_name, value)` rows with the value cast to text, then group by `column_name`.

The recent-window percentage earns its place. `ShipDate` and `DeliveryDate` are legitimately NULL for orders that have not shipped or arrived yet, so their NULL rate is naturally higher in the last few weeks. A contract that says "DeliveryDate NULL rate must be under 2%" would fire every night on recent orders; the right rule is "under 2% for orders older than N days". Seeing both percentages side by side shows which NULLs are timing and which are defects.

Step 2 — Identify the Tables and Relationships: One table, `Orders`. No joins, so no fan-out risk. Every one of the eight unpivoted rows comes from exactly one order row, which is what makes `total_rows` a check.

Step 3 — Build the Query Logic:

1. Compute the anchor date once.
2. For every order, `CROSS JOIN LATERAL (VALUES ...)` with one row per column: position, name, and the value cast to text. Carry an `is_recent` flag computed from `OrderDate`.
3. Group by column position and name.
4. Count all rows, NULL rows and recent NULL rows with FILTER, and count distinct non-NULL values.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate) AS max_order_date
    FROM Orders AS o
),
unpivoted AS (
    SELECT v.column_position,
           v.column_name,
           v.column_value,
           o.OrderDate > a.max_order_date - 30 AS is_recent
    FROM Orders AS o
    CROSS JOIN anchor AS a
    CROSS JOIN LATERAL (
        VALUES (1, 'OrderID',            o.OrderID::text),
               (2, 'CustomerID',         o.CustomerID::text),
               (3, 'PaymentID',          o.PaymentID::text),
               (4, 'OrderDate',          o.OrderDate::text),
               (5, 'ShipperID',          o.ShipperID::text),
               (6, 'Total_Order_Amount', o.Total_Order_Amount::text),
               (7, 'ShipDate',           o.ShipDate::text),
               (8, 'DeliveryDate',       o.DeliveryDate::text)
    ) AS v(column_position, column_name, column_value)
)
SELECT u.column_name,
       COUNT(*)                                                    AS total_rows,
       COUNT(*) FILTER (WHERE u.column_value IS NULL)              AS null_rows,
       ROUND(100.0 * COUNT(*) FILTER (WHERE u.column_value IS NULL)
             / NULLIF(COUNT(*), 0), 2)                             AS null_pct,
       ROUND(100.0 * COUNT(*) FILTER (WHERE u.column_value IS NULL AND u.is_recent)
             / NULLIF(COUNT(*) FILTER (WHERE u.is_recent), 0), 2) AS null_pct_last_30d,
       COUNT(DISTINCT u.column_value)                              AS distinct_values
FROM unpivoted AS u
GROUP BY u.column_position, u.column_name
ORDER BY u.column_position;
```

Step 5 — Line-by-Line Explanation:

- `CROSS JOIN anchor` attaches the one-row anchor to every order.
- `CROSS JOIN LATERAL (VALUES ...)`: LATERAL lets each VALUES row read the current order's columns. Eight rows come out per order.
- `::text` on every value: all rows of a VALUES list must share one type. Casting keeps NULL as NULL, so NULL checks still work after the cast.
- `o.OrderDate > a.max_order_date - 30`: DATE minus integer is a DATE, so this is the 30 days ending at the anchor, inclusive of the anchor day.
- `COUNT(*)` counts every unpivoted row, which equals the number of orders for each column.
- `COUNT(*) FILTER (WHERE u.column_value IS NULL)` is the NULL count. It is the same as `COUNT(*) - COUNT(u.column_value)`, but the FILTER form reads as the rule.
- `null_pct_last_30d` divides recent NULLs by recent rows, not by all rows. `NULLIF(..., 0)` returns NULL instead of an error if the window is empty.
- `COUNT(DISTINCT u.column_value)` ignores NULLs, so `distinct_values` is the count of real values. For `OrderID` it must equal `total_rows`; if it does not, the primary key is not unique.
- `GROUP BY u.column_position, u.column_name` and `ORDER BY u.column_position` keep table order rather than alphabetical order.

Step 6 — Sample Output:

column_name	total_rows	null_rows	null_pct	null_pct_last_30d	distinct_values
OrderID	46438	0	0.00	0.00	46438
CustomerID	46438	0	0.00	0.00	4427
PaymentID	46438	20	0.04	0.00	8
OrderDate	46438	0	0.00	0.00	1093
ShipperID	46438	25	0.05	0.00	9
Total_Order_Amount	46438	5	0.01	0.00	8870
ShipDate	46438	455	0.98	4.11	1093
DeliveryDate	46438	796	1.71	15.17	1093

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 8 rows returned.

Step 7 — Why This Solution Works: One scan of `Orders` produces every metric for every column, and the numbers are comparable because they share a denominator. Reading the output is where the value is:

- `OrderID` with `distinct_values = total_rows` and zero NULLs: the key is intact.
- Foreign keys (`CustomerID`, `PaymentID`, `ShipperID`) with a few NULLs: a completeness gap. Whether those NULL keys also point at nothing is a separate question for the integrity audit in Q87.
- `PaymentID` and `ShipperID` distinct counts should not exceed the rows in `Payments` and `Shippers`. A count at or near the limit can still hide an orphan code standing in for an unused valid one, so confirm with Q87.
- `ShipDate` and `DeliveryDate` with a much higher recent NULL rate than overall: timing, not a defect. A high overall NULL rate on old orders is the defect.
- `distinct_values` for `OrderDate` far below the number of days in the range: missing days, or a load that stamped every row with the load

date.

For the nightly job, store each run's rows with the run date. Alert on change, not on level: a NULL rate on `Total_Order_Amount` going from 0.2% to 5% overnight means the source changed, even though 5% might be tolerable on its own.

Step 8 — Common Mistakes:

- Using `COUNT(column)` as the NULL count. It counts non-NULL values, so the "NULL" number is really the filled number.
- Unpivoting without casting. VALUES fails because integer, date and numeric cannot share one column:

```
CROSS JOIN LATERAL (VALUES ('OrderID', o.OrderID), ('OrderDate', o.OrderDate)) AS v(column_name, column_value)
-- ERROR: VALUES types integer and date cannot be matched
```

- Writing eight `SELECT ... UNION ALL` branches, each with its own `FROM Orders`. It gives the same answer with eight scans, and it is easy to copy one branch and forget to change the column name.
- Trusting text casts for distinct counts on unconstrained NUMERIC. `1.5` and `1.50` are equal numbers but different strings. `Total_Order_Amount` is `NUMERIC(12,2)` here, so every value has the same scale, but check before reusing the pattern.
- Treating empty strings as complete. On text columns, `''` and `' '` are not NULL. Add `OR TRIM(u.column_value) = ''` to the missing rule when profiling text-heavy tables like `Customers`.

Step 9 — Interview Follow-Up: The contract team wants a rule that `DeliveryDate` may be NULL only for orders less than 14 days old. Turn that into a single check that returns the number of violating orders, and say where the 14 days should come from.

Step 10 — Optimization or Alternative: A version that adapts when columns are added: convert each row to JSON and expand its keys. It includes every column automatically, at the cost of losing the table order and the original types:

```
SELECT kv.key                                                  AS column_name,
       COUNT(*)                                                AS total_rows,
       COUNT(*) FILTER (WHERE kv.value IS NULL)                AS null_rows,
       ROUND(100.0 * COUNT(*) FILTER (WHERE kv.value IS NULL)
             / NULLIF(COUNT(*), 0), 2)                         AS null_pct,
       COUNT(DISTINCT kv.value)                                AS distinct_values
FROM Orders AS o
CROSS JOIN LATERAL jsonb_each_text(to_jsonb(o)) AS kv(key, value)
GROUP BY kv.key
ORDER BY kv.key;
```

`jsonb_each_text` returns SQL NULL for JSON nulls, so the NULL counts match. Keys come back in lower case because that is how PostgreSQL stores unquoted identifiers.

Performance: both versions scan `Orders` once, then sort or hash eight times as many rows as the table has. The expensive part is `COUNT(DISTINCT)`, which sorts every column's values. On a billion-row table, profile a sample with `TABLESAMPLE SYSTEM (1)`, or read PostgreSQL's planner statistics, which `ANALYZE` already maintains:

```
SELECT ps.attname    AS column_name,
       ps.null_frac  AS approx_null_fraction,
       ps.n_distinct AS approx_distinct
FROM pg_stats AS ps
WHERE ps.schemaname = 'public'
  AND ps.tablename  = 'orders'
ORDER BY ps.attname;
```

These are estimates from a sample. A negative `n_distinct` means a fraction of the row count (-1 means every value is distinct). Good for a cheap daily trend; use the exact query for the contract. On MySQL, the usual port of the unpivot is a lateral derived table built from `SELECT ... UNION ALL` rows.

Question 91: Cumulative value per customer by cohort (LTV curve)

Difficulty: Expert

Primary Topic: Cohort analysis

Combined Topics: Cohort spine with generate_series, Running SUM window, Pivot with FILTER, Maturity masking with an anchored date, Multiple CTEs

Business Scenario: The growth team pays for new customers through performance marketing. Finance will only approve a higher acquisition budget if they can see how much order value a typical new customer brings in over their first year, and whether recent cohorts are better or worse than older ones. They want an LTV curve: cumulative order value per acquired customer at fixed points after the first order, one row per acquisition quarter.

Problem Statement: Group customers into cohorts by the calendar quarter of their first order. For each cohort, report the number of customers and the cumulative order value per customer through month 0, 3, 6 and 12 after each customer's first order month. A value must be NULL, not a partial number, when the cohort has not yet been observed for that long.

Expected Output:

- `cohort_quarter`: first day of the quarter of the customer's first order
- `cohort_customers`: customers whose first order fell in that quarter
- `value_per_customer_m0`: cumulative `Total_Order_Amount` in the first order month, divided by `cohort_customers`

- `value_per_customer_m3`, `value_per_customer_m6`, `value_per_customer_m12`: the same, cumulative through month offset 3, 6 and 12

Tables Required: `Orders`

SQL Concepts Tested: cohort assignment with MIN, calendar month offsets, a dense spine of cohort × month, running SUM with an explicit ROWS frame, pivoting with FILTER, masking immature cells.

Step 1 — Understand the Problem: Assumptions:

- A customer is a `CustomerID` in `Orders`. Duplicate accounts (see Q92) split one person into two cohorts; merge them first if that matters.
- Value is `Total_Order_Amount` as booked. Returns and refunds are not modelled, so this is gross value, an LTV proxy rather than margin.
- Month offset is a calendar-month difference: an order in the same calendar month as the first order is offset 0, the next calendar month is offset 1, and so on. Offsets are per customer, measured from that customer's first order month.
- The data ends at `MAX(OrderDate)`. The last complete month is the month before it, unless the anchor is the month's final day.

Two traps decide whether the numbers are honest. First, the denominator is every customer in the cohort, including those who never ordered again. LTV per acquired customer, not per active customer. Second, immaturity. A cohort acquired three months before the data ends has no month-6 data yet; reporting its partial sum next to older cohorts makes it look like the worst cohort ever. A cell is shown only when the youngest possible customer in the cohort (one acquired in the cohort's last month) has a complete month at that offset.

Step 2 — Identify the Tables and Relationships: Only `Orders`. The relationship that matters is within it: many orders per `CustomerID`, and one first order month per customer.

Step 3 — Build the Query Logic:

1. Anchor: the last complete month in the data.
2. First order month per customer with `MIN(OrderDate)`, truncated to month.
3. Cohort size per quarter.
4. Every order gets its cohort quarter and month offset. Sum value per cohort and offset, keeping offsets 0 to 12.
5. Build a spine: every cohort × every offset 0 to 12 with `generate_series`. LEFT JOIN the sums so offsets with no orders become 0.
6. Running SUM down the offsets per cohort gives cumulative value.
7. Flag each cell as complete or not, then pivot offsets 0, 3, 6, 12 into columns and divide by cohort size.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT (DATE_TRUNC('month', MAX(o.OrderDate) + 1) - INTERVAL '1 month')::date AS last_full_month
    FROM Orders AS o
),
first_orders AS (
    SELECT o.CustomerID,
           DATE_TRUNC('month', MIN(o.OrderDate))::date AS first_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate IS NOT NULL
    GROUP BY o.CustomerID
),
cohorts AS (
    SELECT DATE_TRUNC('quarter', fo.first_month)::date AS cohort_quarter,
           COUNT(*)                                    AS cohort_customers
    FROM first_orders AS fo
    GROUP BY DATE_TRUNC('quarter', fo.first_month)::date
),
order_offsets AS (
    SELECT DATE_TRUNC('quarter', fo.first_month)::date AS cohort_quarter,
           (EXTRACT(YEAR  FROM o.OrderDate) - EXTRACT(YEAR  FROM fo.first_month)) * 12
         + (EXTRACT(MONTH FROM o.OrderDate) - EXTRACT(MONTH FROM fo.first_month)) AS month_offset,
           o.Total_Order_Amount
    FROM Orders AS o
    JOIN first_orders AS fo ON fo.CustomerID = o.CustomerID
),
offset_value AS (
    SELECT oo.cohort_quarter,
           oo.month_offset::int              AS month_offset,
           SUM(oo.Total_Order_Amount)        AS order_value
    FROM order_offsets AS oo
    WHERE oo.month_offset <= 12
    GROUP BY oo.cohort_quarter, oo.month_offset
),
spine AS (
    SELECT c.cohort_quarter,
           c.cohort_customers,
           m.month_offset
    FROM cohorts AS c
    CROSS JOIN generate_series(0, 12) AS m(month_offset)
),
curve AS (
    SELECT s.cohort_quarter,
           s.cohort_customers,
           s.month_offset,
           SUM(COALESCE(ov.order_value, 0)) OVER (
               PARTITION BY s.cohort_quarter
               ORDER BY s.month_offset
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
           ) AS cumulative_value,
           -- the youngest customer in the cohort joined in its third month
           (s.cohort_quarter + INTERVAL '2 months'
                             + s.month_offset * INTERVAL '1 month')::date
               <= a.last_full_month AS is_complete
    FROM spine AS s
    CROSS JOIN anchor AS a
    LEFT JOIN offset_value AS ov
           ON ov.cohort_quarter = s.cohort_quarter
          AND ov.month_offset   = s.month_offset
)
SELECT cv.cohort_quarter,
       cv.cohort_customers,
       ROUND(MAX(cv.cumulative_value) FILTER (WHERE cv.month_offset = 0  AND cv.is_complete)
             / cv.cohort_customers, 2) AS value_per_customer_m0,
       ROUND(MAX(cv.cumulative_value) FILTER (WHERE cv.month_offset = 3  AND cv.is_complete)
             / cv.cohort_customers, 2) AS value_per_customer_m3,
       ROUND(MAX(cv.cumulative_value) FILTER (WHERE cv.month_offset = 6  AND cv.is_complete)
             / cv.cohort_customers, 2) AS value_per_customer_m6,
       ROUND(MAX(cv.cumulative_value) FILTER (WHERE cv.month_offset = 12 AND cv.is_complete)
             / cv.cohort_customers, 2) AS value_per_customer_m12
FROM curve AS cv
GROUP BY cv.cohort_quarter, cv.cohort_customers
ORDER BY cv.cohort_quarter;
```

Step 5 — Line-by-Line Explanation:

- `anchor`: `MAX(OrderDate) + 1` truncated to month, minus one month, is the last fully observed month. If the data ends on 31 December, December counts; if it ends on 15 December, November is the last full month.
- `first_orders`: one row per customer. Truncating to month makes offsets calendar-based.
- `order_offsets`: years × 12 plus months gives the calendar-month distance between an order and the customer's first month. Every order is offset 0 or more by construction.
- `offset_value` keeps offsets up to 12; later orders do not affect any reported column.
- `spine` has exactly 13 rows per cohort. Without it, a cohort with no orders at offset 3 would have no row at offset 3, and `month_offset = 3` in the pivot would find nothing.
- `COALESCE(ov.order_value, 0)` turns empty offsets into 0 so the running SUM carries forward.
- `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` is the cumulative frame. Offsets are unique per cohort, so RANGE would give the same result here, but ROWS states the intent.
- `is_complete` places the cohort's youngest customer (joined in the quarter's third month) at that offset and checks the month is fully observed.

- `MAX(...) FILTER (WHERE month_offset = 3 AND is_complete)` picks one cell per cohort. MAX is only there because an aggregate is needed; the filter leaves at most one value. An immature cell leaves the FILTER empty, so the column is NULL.
- Dividing by `cohort_customers` gives value per acquired customer.

Step 6 — Sample Output:

cohort_quarter	cohort_customers	value_per_customer_m0	value_per_customer_m3	value_per_customer_m6	value_per_customer_m12
2023-01-01	203	2895.27	7424.71	11414.38	19553.06
2023-04-01	126	2110.10	6849.02	10981.25	16357.10
2023-07-01	138	1936.66	7087.96	10611.94	15786.41
2023-10-01	206	1835.81	5513.52	8064.22	12722.01
2024-01-01	220	2369.60	6719.33	10720.40	17021.09
2024-04-01	228	2368.85	6388.72	10073.38	15306.44
2024-07-01	301	1725.03	6657.23	10204.10	15939.99
2024-10-01	445	2164.06	6971.86	10227.62	NULL

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Each customer's orders are measured on the customer's own clock, so a customer acquired in March and one acquired in January are compared at the same age. The spine guarantees every cohort has a value at every offset, the running SUM makes values cumulative, and the mask removes cells that would be biased low. The diagonal of NULLs in the bottom-right of the output is correct: that is where the data runs out.

Two checks to run when the numbers matter. `SUM(cohort_customers)` over all cohorts must equal `COUNT(DISTINCT CustomerID)` in `Orders`. And for a cohort old enough to have its m12 cell, `value_per_customer_m12 × cohort_customers` must equal the sum of `Total_Order_Amount` for those customers' orders up to offset 12, which you can compute independently from `order_offsets`.

Reading the curve: a steep rise from m0 to m3 means early repeat purchase drives value. Compare cohorts at the same column, never along a row against another cohort's row. Small cohorts swing a lot; show `cohort_customers` next to every value and do not read a trend from cohorts of a handful of customers.

Step 8 — Common Mistakes:

- Skipping the spine and filtering to one offset. A cohort with no orders in exactly offset 3 drops out of the m3 report, even though its customers had spent money by then:

```
WITH first_orders AS (
    SELECT o.CustomerID, DATE_TRUNC('month', MIN(o.OrderDate))::date AS first_month
    FROM Orders AS o
    GROUP BY o.CustomerID
),
by_offset AS (
    SELECT DATE_TRUNC('quarter', fo.first_month)::date AS cohort_quarter,
           ((EXTRACT(YEAR FROM o.OrderDate) - EXTRACT(YEAR FROM fo.first_month)) * 12
          + (EXTRACT(MONTH FROM o.OrderDate) - EXTRACT(MONTH FROM fo.first_month)))::int AS month_offset,
           SUM(o.Total_Order_Amount) AS order_value
    FROM Orders AS o
    JOIN first_orders AS fo ON fo.CustomerID = o.CustomerID
    GROUP BY 1, 2
)
SELECT bo.cohort_quarter, bo.month_offset,
       SUM(bo.order_value) OVER (PARTITION BY bo.cohort_quarter ORDER BY bo.month_offset) AS cumulative_value
FROM by_offset AS bo
WHERE bo.month_offset = 3
ORDER BY bo.cohort_quarter;
```

Worse, the `WHERE` runs before the window, so the running SUM sees only offset 3 and is not cumulative at all.

- Dividing by customers who ordered in that month. The curve then shows value per survivor, which rises over time as low-value customers drop out, and looks far better than it is.
- Leaving immature cells in. The newest cohorts always look weakest, and the team cuts the channel that acquired them.
- Measuring offsets from the cohort quarter start instead of each customer's first month. Customers acquired in the quarter's last month get two fewer months at every column.

Step 9 — Interview Follow-Up: Finance wants the same curve, but net of the acquisition cost of ₹400 per customer, and the month at which each cohort pays back. How would you add a `payback_month` column?

Step 10 — Optimization or Alternative: If you only need the four fixed points and not the full curve, a conditional sum replaces the spine and the window. Each column sums orders up to that offset directly, which also avoids the missing-offset trap:

```sql
WITH anchor AS (
    SELECT (DATE_TRUNC('month', MAX(o.OrderDate) + 1) - INTERVAL '1 month')::date AS last_full_month
    FROM Orders AS o
),
first_orders AS (
    SELECT o.CustomerID, DATE_TRUNC('month', MIN(o.OrderDate))::date AS first_month
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
    GROUP BY o.CustomerID
),
offsets AS (
    SELECT DATE_TRUNC('quarter', fo.first_month)::date AS cohort_quarter,
           fo.CustomerID,
           (EXTRACT(YEAR FROM o.OrderDate) - EXTRACT(YEAR FROM fo.first_month)) * 12
         + (EXTRACT(MONTH FROM o.OrderDate) - EXTRACT(MONTH FROM fo.first_month)) AS month_offset,
           o.Total_Order_Amount
    FROM first_orders AS fo
    JOIN Orders AS o ON o.CustomerID = fo.CustomerID
)
SELECT ofs.cohort_quarter,
       COUNT(DISTINCT ofs.CustomerID) AS cohort_customers,
       CASE WHEN (ofs.cohort_quarter + INTERVAL '5 months')::date <= MAX(a.last_full_month)
            THEN ROUND(COALESCE(SUM(ofs.Total_Order_Amount) FILTER (WHERE ofs.month_offset <= 3), 0)
                       / COUNT(DISTINCT ofs.CustomerID), 2)
       END AS value_per_customer_m3
FROM offsets AS ofs
CROSS JOIN anchor AS a
GROUP BY ofs.cohort_quarter
ORDER BY ofs.cohort_quarter;
```

The spine version is still the one to keep when the product team wants the whole curve as a chart, because it produces every month.

Performance: the query scans `Orders` twice (first orders, then all orders) and works on tiny intermediate tables after that. An index on `Orders(CustomerID, OrderDate)` makes the `MIN` per customer cheap. At scale, materialise `first_orders` as a customer dimension updated incrementally, since a customer's first order month never changes once set. A cohort's m12 cell freezes 15 months after the cohort starts, so the nightly job only needs to recompute cohorts younger than that. `DATE_TRUNC('quarter', ...)` and `generate_series` are PostgreSQL; on MySQL, build the spine with a recursive CTE.

Question 92: How duplicate accounts distort the repeat-customer rate

Difficulty: Expert

Primary Topic: Deduplication impact on metrics

Combined Topics: Survivor mapping with a window MIN, Normalised match key, Metric recomputed before and after, UNION ALL, Conditional aggregation

Business Scenario: The growth team reports the repeat-customer rate to leadership every month. Support has noticed that some people sign up again with the same email in a different case, or with a stray space, because they forgot they had an account. Leadership wants to know whether the reported rate is wrong, by how much, and in which direction, before the data platform team spends a quarter building a proper customer merge.

Problem Statement: Compute the repeat-customer rate twice: once with each `CustomerID` treated as a customer, and once with accounts merged when their normalised email (`LOWER(TRIM(Email))`) matches. A repeat customer is one with two or more orders. Only customers with at least one order count. Return one row per method, plus the total orders in each method as a control.

Expected Output:

- `method`: `Raw CustomerID` or `Merged by email`
- `customers`: customers (or merged people) with at least one order
- `repeat_customers`: those with two or more orders
- `repeat_rate_pct`: `repeat_customers / customers × 100`, 2 dp
- `orders`: total orders attributed, which must be identical in both rows

Tables Required: `Customers`, `Orders`

SQL Concepts Tested: building a match key, mapping every account to a survivor with a window MIN, LEFT JOIN so orders with unknown customers are not lost, computing one metric two ways with UNION ALL, control totals.

Step 1 — Understand the Problem: Assumptions and rules:

- Match key: `LOWER(TRIM(Email))`. Two accounts with the same key are the same person. Accounts with a NULL or blank email never merge with anyone; each forms its own group.
- Survivor: the lowest `CustomerID` in the group. Any deterministic rule works (earliest `DateEntered` with an ID tiebreak is another), as long as it is stable across runs.
- Orders whose `CustomerID` has no `Customers` row keep their raw ID as their own person; they cannot be matched on email, but dropping them would change the order total. Orders with a NULL `CustomerID` cannot be attributed to anyone and are excluded from both methods.
- An order is an `Orders` row. Duplicate order submissions (Q93) are a separate problem and are not removed here.

The direction of the error is not obvious, and interviewers like to ask it. With `r` repeaters out of `n` customers, merging two accounts does one of three things:

- Two one-order accounts become one repeat person: `(r + 1) / (n - 1)`. The rate goes up.
- A one-order account joins a repeat account: `r / (n - 1)`. The rate goes up.
- Two repeat accounts become one: `(r - 1) / (n - 1)`. Below 100%, the rate goes down.

Which case dominates depends on the data, so you have to compute both.

Step 2 — Identify the Tables and Relationships: `Customers` supplies the email. `Orders.CustomerID` → `Customers.CustomerID` is many-to-one and not enforced, so some orders may point to no customer. The merge happens on `Customers`, then flows to orders through that key.

Step 3 — Build the Query Logic:

1. For every customer, compute the match key and the survivor ID: `MIN(CustomerID) OVER (PARTITION BY match_key)`.
2. Attach the survivor to every order with a LEFT JOIN. If there is no customer row, the order's own `CustomerID` is its person.
3. Count orders per raw `CustomerID` and per survivor, and stack the two with UNION ALL, labelled by method.
4. For each method, count units, units with two or more orders, the rate, and total orders.

Step 4 — Final SQL Solution:

```sql
WITH customer_map AS (
    SELECT c.CustomerID,
           MIN(c.CustomerID) OVER (
               PARTITION BY COALESCE(NULLIF(LOWER(TRIM(c.Email)), ''),
                                     'no-email:' || c.CustomerID::text)
           ) AS person_id
    FROM Customers AS c
),
attributed_orders AS (
    SELECT o.OrderID,
           o.CustomerID,
           COALESCE(cm.person_id, o.CustomerID) AS person_id
    FROM Orders AS o
    LEFT JOIN customer_map AS cm ON cm.CustomerID = o.CustomerID
    WHERE o.CustomerID IS NOT NULL
),
per_unit AS (
    SELECT 1 AS sort_order, 'Raw CustomerID' AS method,
           ao.CustomerID AS unit_id, COUNT(*) AS orders
    FROM attributed_orders AS ao
    GROUP BY ao.CustomerID
    UNION ALL
    SELECT 2, 'Merged by email',
           ao.person_id, COUNT(*)
    FROM attributed_orders AS ao
    GROUP BY ao.person_id
)
SELECT pu.method,
       COUNT(*)                                          AS customers,
       COUNT(*) FILTER (WHERE pu.orders >= 2)            AS repeat_customers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE pu.orders >= 2)
             / NULLIF(COUNT(*), 0), 2)                   AS repeat_rate_pct,
       SUM(pu.orders)                                    AS orders
FROM per_unit AS pu
GROUP BY pu.sort_order, pu.method
ORDER BY pu.sort_order;
```

Step 5 — Line-by-Line Explanation:

- `NULLIF(LOWER(TRIM(c.Email)), '')` turns a blank email into NULL, then `COALESCE(..., 'no-email:' || CustomerID)` gives each email-less account a unique key. Without this, every customer with no email would share the NULL partition and merge into one giant person.
- `MIN(c.CustomerID) OVER (PARTITION BY ...)` keeps one row per customer and adds the survivor beside it. That is a mapping table: every account points to the account that represents its person. A GROUP BY could not keep the account-level rows.
- `LEFT JOIN customer_map`: an order with an unknown `CustomerID` still survives, and `COALESCE(cm.person_id, o.CustomerID)` makes it its own person.
- `WHERE o.CustomerID IS NOT NULL` applies to both methods, so the comparison is like for like.
- The two `per_unit` branches use the same `attributed_orders`, so they count the same orders. Only the grouping key differs.
- `COUNT(*) FILTER (WHERE pu.orders >= 2)` is the repeat count. `NULLIF(COUNT(*), 0)` guards the division.
- `SUM(pu.orders)` is the control column. If the two rows disagree, the merge lost or duplicated orders.

Step 6 — Sample Output:

method	customers	repeat_customers	repeat_rate_pct	orders
Raw CustomerID	4427	3725	84.14	46438
Merged by email	4382	3698	84.39	46438

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 2 rows returned.

Step 7 — Why This Solution Works: Both methods read one set of orders and differ only in who those orders belong to, so any difference in the rate is caused by the merge and nothing else. The `orders` column proves it. A second check: `customers` in the merged row can never exceed the raw row, and the gap can never exceed the number of accounts that share their email key with another account.

What to tell leadership is the size and sign of the gap. A gap of a fraction of a point means the dashboard is fine and the merge project can

wait. A gap of several points, especially one that has been widening, means the reported rate has been wrong in a known direction, and acquisition numbers (new customers per month) are inflated too, because a returning person who re-registers counts as a new customer.

Run nightly, the merged rate should be the official number and the raw rate a diagnostic. Alert when the number of merged groups jumps: that usually means a sign-up flow stopped checking for existing emails, or a source started sending a default email such as `noreply@...` for guest checkouts.

Step 8 — Common Mistakes:

- Grouping by raw `Email` without `LOWER` and `TRIM`. `Pari.Singh@gmail.com` and `pari.singh@gmail.com` stay separate, and you conclude there are no duplicates.
- Letting NULL emails merge. `PARTITION BY LOWER(TRIM(Email))` puts every NULL-email account into one partition, which merges strangers into one heavy repeat customer.
- Joining orders straight to customers and grouping by email. Orders whose customer row is missing disappear, the order total drops, and the "after" number is computed on fewer orders than the "before":

```sql
SELECT COUNT(*)                                  AS people,
       COUNT(*) FILTER (WHERE x.orders >= 2)     AS repeat_people,
       SUM(x.orders)                             AS orders_counted
FROM (
    SELECT LOWER(TRIM(c.Email)) AS email_key, COUNT(*) AS orders
    FROM Orders AS o
    JOIN Customers AS c ON c.CustomerID = o.CustomerID
    GROUP BY LOWER(TRIM(c.Email))
) AS x;
```

- Merging on email alone without checking the groups. Families share an email, and recycled or placeholder addresses join strangers. An over-merge inflates the repeat rate just as an under-merge deflates it. Review the groups before trusting the merged number (Step 10).

Step 9 — Interview Follow-Up: Some duplicates use a different email but the same phone number. If you add "same phone" as a second match rule, why is one window MIN no longer enough, and what kind of query is needed?

Step 10 — Optimization or Alternative: Before the merged number goes to leadership, check the groups. Accounts that share an email but disagree on name or phone are candidates for a shared or recycled address, not a duplicate person:

```sql
SELECT LOWER(TRIM(c.Email))                                    AS email_key,
       COUNT(*)                                                AS accounts,
       COUNT(DISTINCT LOWER(TRIM(c.FirstName)) || ' '
                    || LOWER(TRIM(c.LastName)))                AS distinct_names,
       COUNT(DISTINCT TRIM(c.City))                            AS distinct_cities,
       STRING_AGG(c.CustomerID::text, ', ' ORDER BY c.CustomerID) AS customer_ids
FROM Customers AS c
WHERE NULLIF(TRIM(c.Email), '') IS NOT NULL
GROUP BY LOWER(TRIM(c.Email))
HAVING COUNT(*) > 1
ORDER BY accounts DESC, email_key;
```

Groups with `distinct_names > 1` go to a manual review list or get a stricter key (email plus normalised last name). Different cities alone are weaker evidence; people move.

Performance: the window partitions `Customers` by an expression, which is a sort of the customer table. An expression index on `LOWER(TRIM(Email))` speeds up lookups for a single email at sign-up time, which is where duplicates should be stopped. Do not recompute the map on every dashboard query: materialise `customer_map` nightly as a table of `(CustomerID, person_id)` and join reporting queries to it. Keep the survivor rule stable, or a person's history will jump between IDs from one night to the next.

Question 93: Possible duplicate order submissions

Difficulty: Expert

Primary Topic: Duplicate detection

Combined Topics: Line signature with ordered STRING_AGG, Self join on orders, NOT EXISTS to pick the earliest original, Two-level aggregation, IS NOT DISTINCT FROM

Business Scenario: Customer support keeps getting complaints from people charged twice: they tapped "Place order" again when the app hung, and two orders went through. The risk team wants a daily control that says whether any double submissions happened, plus the list support needs to cancel the copies before they ship. The data platform team will use the same list to keep duplicates out of order counts and revenue.

Problem Statement: Build the nightly control for double submissions. Pair every two orders placed by the same customer on the same `OrderDate`, and report in one row: how many such pairs exist, how many have exactly the same basket (same products in the same quantities), how many have the same amount, how many are exact copies (same basket and same header: amount, payment method, shipper), and a verdict. The row-level list of suspected duplicates, each pointing at its original, goes in Step 10.

Expected Output:

- `same_day_pairs`: pairs of orders from one customer on one day
- `same_basket_pairs`: of those, pairs with identical baskets
- `same_amount_pairs`: pairs with identical `Total_Order_Amount`

- `exact_copies`: same basket and same header
- `verdict`: `No duplicate submissions` or `Review: send same-basket pairs to support`

Tables Required: `Orders`, `OrderDetails`

SQL Concepts Tested: collapsing many rows into one comparable value with ordered STRING_AGG, a self join with an inequality to avoid mirrored pairs, NOT EXISTS to anchor every copy on the first order, NULL-safe row comparison.

Step 1 — Understand the Problem: Assumptions:

- `OrderDate` is a date with no time, so "same submission" can only mean "same day". A customer who genuinely buys the same basket twice in one day will be flagged. That is a false positive support can dismiss; with an order timestamp you would tighten this to a few minutes.
- `OrderID` is assigned in increasing order, so the lowest ID in a group is the first submission.
- The basket is products and quantities. `SupplierID` is left out on purpose: it records how the order was fulfilled, and a resubmitted basket can be routed to a different supplier.
- An order with no lines has no basket and cannot be matched here. Header-only duplicates need a header-level check (see Step 10).
- An order with a line whose `ProductID` or `Quantity` is NULL has an unknown basket. It is excluded rather than matched on a guess.
- Exact duplicate line rows inside one order (a reload, see Q72) double that product's quantity here. Deduplicate lines first, or a real double submission can fail to match.

A basket lives in many rows, and SQL compares rows one at a time. So first turn each order's lines into one value: a sorted string of `ProductID x Quantity`. Two orders have the same basket exactly when their strings are equal.

Step 2 — Identify the Tables and Relationships: `OrderDetails` holds the lines; `OrderDetails.OrderID` → `Orders.OrderID` is many-to-one. `Orders` holds the customer, date and header values. The match is `Orders` to itself, once each order has a signature.

Step 3 — Build the Query Logic:

1. Collapse lines to one row per order and product, summing quantity. Mark the row incomplete if any line has a NULL quantity or product.
2. Collapse those to one row per order: an ordered `STRING_AGG` signature plus a flag saying the whole basket is known.
3. Attach the signature to `Orders`, keeping orders with a known basket, a customer and a date.
4. Self join on customer and date with `b.OrderID > a.OrderID`, so each same-day pair appears once.
5. For each pair, compare basket, amount and header; count each comparison with `FILTER`.

Step 4 — Final SQL Solution:

```sql
WITH product_lines AS (
    SELECT od.OrderID,
           od.ProductID,
           SUM(od.Quantity)                                          AS quantity,
           od.ProductID IS NOT NULL AND COUNT(od.Quantity) = COUNT(*) AS is_known
    FROM OrderDetails AS od
    WHERE od.OrderID IS NOT NULL
    GROUP BY od.OrderID, od.ProductID
),
signatures AS (
    SELECT pl.OrderID,
           STRING_AGG(pl.ProductID::text || 'x' || pl.quantity::text, '|'
                      ORDER BY pl.ProductID)                         AS line_signature,
           BOOL_AND(pl.is_known)                                     AS basket_known
    FROM product_lines AS pl
    GROUP BY pl.OrderID
),
order_sig AS (
    SELECT o.OrderID, o.CustomerID, o.OrderDate, o.PaymentID, o.ShipperID,
           o.Total_Order_Amount, sg.line_signature
    FROM Orders AS o
    JOIN signatures AS sg ON sg.OrderID = o.OrderID
    WHERE sg.basket_known
      AND o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
same_day_pairs AS (
    SELECT a.OrderID                                          AS first_order_id,
           b.OrderID                                          AS second_order_id,
           a.line_signature = b.line_signature                AS same_basket,
           a.Total_Order_Amount = b.Total_Order_Amount        AS same_amount,
           (a.Total_Order_Amount, a.PaymentID, a.ShipperID)
               IS NOT DISTINCT FROM
           (b.Total_Order_Amount, b.PaymentID, b.ShipperID)   AS same_header
    FROM order_sig AS a
    JOIN order_sig AS b
      ON b.CustomerID = a.CustomerID
     AND b.OrderDate  = a.OrderDate
     AND b.OrderID    > a.OrderID
)
SELECT COUNT(*)                                                 AS same_day_pairs,
       COUNT(*) FILTER (WHERE sp.same_basket)                   AS same_basket_pairs,
       COUNT(*) FILTER (WHERE sp.same_amount)                   AS same_amount_pairs,
       COUNT(*) FILTER (WHERE sp.same_basket AND sp.same_header) AS exact_copies,
       CASE WHEN COUNT(*) FILTER (WHERE sp.same_basket) = 0
            THEN 'No duplicate submissions'
            ELSE 'Review: send same-basket pairs to support'
       END                                                      AS verdict
FROM same_day_pairs AS sp;
```

Step 5 — Line-by-Line Explanation:

- `product_lines` groups by order and product, so the same product split over two lines (2 + 1) and a single line of 3 give the same basket.
- `is_known` is false when the product is NULL or any quantity in the group is NULL (`COUNT(od.Quantity)` skips NULLs, `COUNT(*)` does not).
- `STRING_AGG(... ORDER BY pl.ProductID)` sorts before joining, so the signature does not depend on insertion order.
- The `x` and `|` separators keep the parts apart: without them, product 1 quantity 12 and product 11 quantity 2 would both read `112`.
- `BOOL_AND(pl.is_known)` is true only if every product in the order is known. `WHERE sg.basket_known` drops the rest.
- `b.OrderID > a.OrderID` returns each pair once. `<>` would return every pair twice, mirrored, and double every count.
- The join is on customer and date only, so the control also counts same-day pairs with different baskets. That denominator is what makes a zero meaningful: the checker found pairs and compared them.
- Row-wise `IS NOT DISTINCT FROM` compares three header fields at once and treats NULL = NULL as equal, so a copy with a NULL `PaymentID` on both sides still counts as an exact copy.
- `COUNT(*) FILTER (WHERE ...)` counts the pairs that pass each test in one scan of the pairs.

Step 6 — Sample Output:

same_day_pairs	same_basket_pairs	same_amount_pairs	exact_copies	verdict
322	45	45	39	Review: send same-basket pairs to support

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 1 row returned.

Step 7 — Why This Solution Works: Two orders get the same signature only if they contain the same products in the same total quantities, so comparing signatures compares whole baskets, not single lines. Counting every same-day pair, not just the matches, shows the check had material to work on: on this dataset a few hundred same-day pairs exist, most of them genuine second purchases, and only the pairs with an identical basket are flagged. When the verdict flips, the Step 10 list gives support the rows. In that list, an exact header copy is almost certainly a double tap. A differing header (another payment method, a different amount) can be a customer who retried after a failed payment, which still needs a refund check, or a genuine second purchase.

Checks on the Step 10 list: every `duplicate_order_id` appears once, no `original_order_id` also appears as a duplicate, and the row count equals `same_basket_pairs` minus the extra pairs a triplicate creates.

Nightly, run it on the last few days of orders and send new rows to the support queue. Alert if the daily count jumps: that usually means a checkout release broke the "disable button after tap" guard, or the payment callback is retrying.

Step 8 — Common Mistakes:

- Building the signature without `ORDER BY`. The string follows whatever order the rows are read in, and identical baskets can fail to match:

```
SELECT od.OrderID,
       STRING_AGG(od.ProductID::text || 'x' || od.Quantity::text, '|') AS unstable_signature
FROM OrderDetails AS od
GROUP BY od.OrderID
ORDER BY od.OrderID;
```

- Joining on `a.OrderID <> b.OrderID`. Every pair shows twice, and support cancels both orders.
- Using `>` without the earliest-original rule. A triplicate produces three rows, and one of them names a duplicate as the "original".
- Matching on header fields only. Two different baskets at the same price are flagged, and a duplicate whose amount was corrected is missed.
- Treating NULL quantities as 0. Two incomplete baskets then match on missing data, which is a guess, not evidence.

Step 9 — Interview Follow-Up: If `Orders` had an order timestamp, how would you change the match so that two identical orders 40 seconds apart are flagged but the same basket bought again at 9 pm is not? What threshold would you choose, and how would you check it?

Step 10 — Optimization or Alternative: The support queue needs the rows. Keep the same signature CTEs and replace the summary with a self join on customer, date and signature. `NOT EXISTS` keeps only pairs whose `orig` is the earliest order in its group, so a triplicate A, B, C yields A–B and A–C, not B–C:

```sql
WITH product_lines AS (
    SELECT od.OrderID,
           od.ProductID,
           SUM(od.Quantity)                                          AS quantity,
           od.ProductID IS NOT NULL AND COUNT(od.Quantity) = COUNT(*) AS is_known
    FROM OrderDetails AS od
    WHERE od.OrderID IS NOT NULL
    GROUP BY od.OrderID, od.ProductID
),
signatures AS (
    SELECT pl.OrderID,
           STRING_AGG(pl.ProductID::text || 'x' || pl.quantity::text, '|'
                      ORDER BY pl.ProductID)                         AS line_signature,
           BOOL_AND(pl.is_known)                                     AS basket_known
    FROM product_lines AS pl
    GROUP BY pl.OrderID
),
order_sig AS (
    SELECT o.OrderID, o.CustomerID, o.OrderDate, o.PaymentID, o.ShipperID,
           o.Total_Order_Amount, sg.line_signature
    FROM Orders AS o
    JOIN signatures AS sg ON sg.OrderID = o.OrderID
    WHERE sg.basket_known
      AND o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
)
SELECT dup.CustomerID,
       dup.OrderDate,
       orig.OrderID              AS original_order_id,
       dup.OrderID               AS duplicate_order_id,
       dup.Total_Order_Amount,
       dup.line_signature,
       CASE WHEN (orig.Total_Order_Amount, orig.PaymentID, orig.ShipperID)
                 IS NOT DISTINCT FROM
                 (dup.Total_Order_Amount, dup.PaymentID, dup.ShipperID)
            THEN 'Exact header copy'
            ELSE 'Same basket, header differs'
       END                       AS evidence
FROM order_sig AS orig
JOIN order_sig AS dup
  ON dup.CustomerID     = orig.CustomerID
 AND dup.OrderDate      = orig.OrderDate
 AND dup.line_signature = orig.line_signature
 AND dup.OrderID        > orig.OrderID
WHERE NOT EXISTS (
    SELECT 1
    FROM order_sig AS earlier
    WHERE earlier.CustomerID     = orig.CustomerID
      AND earlier.OrderDate      = orig.OrderDate
      AND earlier.line_signature = orig.line_signature
      AND earlier.OrderID        < orig.OrderID
)
ORDER BY dup.CustomerID, dup.OrderDate, dup.OrderID;
```

On this dataset it returns one row per suspected copy, the same count as `same_basket_pairs` in the summary.

Performance: the expensive part is building a signature for every order, which sorts every line. In the nightly job, restrict `OrderDetails` to orders placed in the last few days (join to `Orders` filtered on `OrderDate`, with an index on `Orders(OrderDate)`), since a duplicate submission always has the same date as its original. Long baskets make long strings; hash them with `md5(line_signature)` before the join and keep the full string only for display. An index on `Orders(CustomerID, OrderDate)` supports the self join. On MySQL, `GROUP_CONCAT(... ORDER BY ... SEPARATOR '|')` replaces `STRING_AGG`, and its default length limit of 1024 characters silently truncates long signatures unless `group_concat_max_len` is raised.

Question 94: Supplier concentration per category (HHI)

Difficulty: Expert

Primary Topic: Share-squared sums (HHI)

Combined Topics: Window SUM for shares, ROW_NUMBER for the top supplier, CASE bands, LEFT JOIN for unknown keys

Business Scenario: The supply chain risk team is building the festive-season buying plan. Before demand peaks they want to know which categories lean too hard on one or two suppliers, so they can qualify a backup vendor in time. "Concentration" here means how unevenly a category's list-price value is spread across the suppliers who fulfilled its order lines, measured with the Herfindahl-Hirschman Index (HHI).

Problem Statement: For every category with sales, work out each supplier's percentage share of the category's list-price value across all order lines. From those shares return the number of suppliers used, the top supplier and its share, the HHI (sum of squared percentage shares, 0 to 10,000) and a concentration band. Sort by HHI, highest first.

Expected Output:

- `CategoryName`: category name, or `Unmapped` when the product's `CategoryID` has no `Category` row
- `suppliers_used`: distinct `SupplierID` values on the category's lines (an ID with no `Suppliers` row still counts)
- `top_supplier`: `CompanyName` of the largest supplier, or `Unknown supplier <id>`
- `top_supplier_share_pct`: the top supplier's share of category value, 2 dp
- `hhi`: sum of squared percentage shares, rounded to a whole number

- `concentration_band`: `Single source`, `Highly concentrated` (above 2,500), `Moderately concentrated` (1,500 to 2,500) or `Unconcentrated` (below 1,500)

Tables Required: `OrderDetails`, `Products`, `Category`, `Suppliers`

SQL Concepts Tested: line-grain aggregation, window SUM for shares, sums of squares, ROW_NUMBER with a tie-break, CASE banding, labelling unmatched keys.

Step 1 — Understand the Problem: State the assumptions before writing SQL.

1. Value is list-price value, `od.Quantity * p.Sale_Price`. `Sale_Price` is today's catalogue price, not the price paid, so this is a proxy. A supplier that ships many cheap units will look smaller here than in a unit count.
2. The supplier that matters is the one on the line (`OrderDetails.SupplierID`). The same product can be fulfilled by different suppliers on different orders.
3. The whole order history is used. A risk view that needs recent exposure only would add an anchored date filter (see the follow-up).
4. Lines with a NULL `SupplierID`, `Quantity` or price cannot be attributed or valued, so they are left out. A `SupplierID` with no `Suppliers` row stays in as its own supplier. That volume came from somewhere, and dropping it would push every remaining share up.

HHI with shares written as percentages: if suppliers hold s1, s2, ... sN percent of a category, HHI = $s1^2 + s2^2 + \ldots + sN^2$. One supplier gives $100^2 = 10{,}000$. N equal suppliers give 10,000 / N. The top share alone ignores everyone else: a 60/40 split and a 60/10/10/10/10 split have the same top share, but HHI is 5,200 for the first and 4,000 for the second, and the second is easier to recover from if the big supplier fails. The band thresholds come from the 2010 US horizontal merger guidelines. Treat them as a convention the procurement team can change.

Step 2 — Identify the Tables and Relationships: `OrderDetails` is the driving table: one row is one product line in one order, and it carries both `ProductID` and `SupplierID`. `OrderDetails.ProductID` → `Products.ProductID` (many lines to one product) gives the price and `CategoryID`. `Products.CategoryID` → `Category.CategoryID` gives the name. `OrderDetails.SupplierID` → `Suppliers.SupplierID` gives the supplier name. `Orders` is not needed, because nothing here is order-level and there is no date filter.

Step 3 — Build the Query Logic:

1. Aggregate lines to one row per (category, supplier) with the summed list-price value. Group on `cat.CategoryID` from the LEFT JOIN, so every product whose category is missing lands in a single NULL group that will be labelled `Unmapped`.
2. Divide each row by the category total, computed with `SUM(...) OVER (PARTITION BY category)`, to get the percentage share.
3. Over the same partition, count suppliers, sum the squared shares and number the suppliers by value with `ROW_NUMBER`, breaking ties on `SupplierID`.
4. Keep rank 1, attach the supplier name with a LEFT JOIN, and band the HHI with CASE.

Window functions let the top supplier's row carry its category's totals, so there is no second GROUP BY and no join back.

Step 4 — Final SQL Solution:

```sql
WITH supplier_value AS (
    SELECT cat.CategoryID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           od.SupplierID,
           SUM(od.Quantity * p.Sale_Price)        AS list_price_value
    FROM OrderDetails  AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    WHERE od.SupplierID IS NOT NULL
      AND od.Quantity   IS NOT NULL
      AND p.Sale_Price  IS NOT NULL
    GROUP BY cat.CategoryID, COALESCE(cat.CategoryName, 'Unmapped'), od.SupplierID
),
supplier_share AS (
    SELECT sv.CategoryID,
           sv.category_name,
           sv.SupplierID,
           sv.list_price_value,
           100.0 * sv.list_price_value
                 / NULLIF(SUM(sv.list_price_value) OVER (PARTITION BY sv.CategoryID), 0) AS share_pct
    FROM supplier_value AS sv
),
category_stats AS (
    SELECT ss.CategoryID,
           ss.category_name,
           ss.SupplierID,
           ss.share_pct,
           COUNT(*)                       OVER (PARTITION BY ss.CategoryID) AS suppliers_used,
           SUM(ss.share_pct * ss.share_pct) OVER (PARTITION BY ss.CategoryID) AS hhi,
           ROW_NUMBER() OVER (PARTITION BY ss.CategoryID
                              ORDER BY ss.list_price_value DESC, ss.SupplierID) AS supplier_rank
    FROM supplier_share AS ss
)
SELECT cs.category_name                                              AS CategoryName,
       cs.suppliers_used,
       COALESCE(s.CompanyName, CONCAT('Unknown supplier ', cs.SupplierID))  AS top_supplier,
       ROUND(cs.share_pct, 2)                                        AS top_supplier_share_pct,
       ROUND(cs.hhi, 0)                                              AS hhi,
       CASE
           WHEN cs.suppliers_used = 1 THEN 'Single source'
           WHEN cs.hhi > 2500         THEN 'Highly concentrated'
           WHEN cs.hhi >= 1500        THEN 'Moderately concentrated'
           ELSE                            'Unconcentrated'
       END                                                           AS concentration_band
FROM category_stats AS cs
LEFT JOIN Suppliers AS s ON s.SupplierID = cs.SupplierID
WHERE cs.supplier_rank = 1
ORDER BY cs.hhi DESC, cs.category_name;
```

Step 5 — Line-by-Line Explanation:

- `supplier_value` changes the grain from line to (category, supplier). The inner JOIN to `Products` drops lines whose product does not exist, since they have no price and no category. The LEFT JOIN to `Category` keeps products with an unknown `CategoryID`.
- Grouping on `cat.CategoryID`, not `p.CategoryID`, matters: two different broken category IDs both become NULL and fall into one `Unmapped` bucket instead of two rows with the same label.
- `share_pct` divides by a window total. `NULLIF(..., 0)` guards a category whose value sums to zero.
- `COUNT(*) OVER` equals distinct suppliers because `supplier_value` already has one row per supplier.
- `SUM(share_pct * share_pct) OVER` is the HHI. Shares stay in percent, so the HHI runs from 0 to 10,000 and lines up with the bands.
- `ROW_NUMBER` picks exactly one top supplier. `SupplierID` breaks ties so reruns give the same answer.
- `COALESCE(s.CompanyName, CONCAT('Unknown supplier ', cs.SupplierID))` labels an ID with no `Suppliers` row instead of printing NULL.
- `Single source` is tested first because one supplier always gives 10,000 and would otherwise read as "Highly concentrated", which undersells the risk.

Step 6 — Sample Output:

categoryname	suppliers_used	top_supplier	top_supplier_share_pct	hhi	concentration_band
Unmapped	2	Coromandel FMCG Supplies	99.62	9923	Highly concentrated
Kitchen, Garden & Pets	3	Kaveri Home Essentials	56.56	5071	Highly concentrated
Baby Care	3	Coromandel FMCG Supplies	54.22	5032	Highly concentrated
Eggs, Meat & Fish	3	Punjab Dairy & Bakers Co-op	52.93	5014	Highly concentrated
Bakery, Cakes & Dairy	4	Punjab Dairy & Bakers Co-op	41.77	3483	Highly concentrated
Cleaning & Household	3	Indus Personal Care Pvt Ltd	40.10	3406	Highly concentrated
Snacks & Branded Foods	4	Ganga Distributors LLP	39.36	3395	Highly concentrated
Fruits & Vegetables	3	Ganga Distributors LLP	36.18	3380	Highly concentrated

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 12 rows.

Step 7 — Why This Solution Works: Every number comes from one consistent base: the (category, supplier) value table. Shares are computed against the same partition they are summed over, so they must add to 100 per category, and the HHI must sit between 10,000 / N and 10,000 for a category with N suppliers. Both properties are cheap to test, and a failure means the grain broke somewhere. Run this

before handing the report over:

```
WITH supplier_value AS (
    SELECT cat.CategoryID,
           od.SupplierID,
           SUM(od.Quantity * p.Sale_Price) AS list_price_value
    FROM OrderDetails  AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    WHERE od.SupplierID IS NOT NULL
      AND od.Quantity   IS NOT NULL
      AND p.Sale_Price  IS NOT NULL
    GROUP BY cat.CategoryID, od.SupplierID
),
shares AS (
    SELECT sv.CategoryID,
           100.0 * sv.list_price_value
                 / NULLIF(SUM(sv.list_price_value) OVER (PARTITION BY sv.CategoryID), 0) AS share_pct
    FROM supplier_value AS sv
),
per_category AS (
    SELECT sh.CategoryID,
           COUNT(*)                          AS suppliers_used,
           SUM(sh.share_pct)                 AS share_total,
           SUM(sh.share_pct * sh.share_pct) AS hhi
    FROM shares AS sh
    GROUP BY sh.CategoryID
)
SELECT COUNT(*)                                                    AS categories_checked,
       COUNT(*) FILTER (WHERE ABS(pc.share_total - 100) > 0.0001)  AS shares_not_summing_to_100,
       COUNT(*) FILTER (WHERE pc.hhi < 10000.0 / pc.suppliers_used - 0.0001
                           OR pc.hhi > 10000.0001)                 AS hhi_out_of_bounds,
       (SELECT COUNT(*)
        FROM OrderDetails AS od
        WHERE od.SupplierID IS NULL
           OR NOT EXISTS (SELECT 1 FROM Products AS p
                          WHERE p.ProductID = od.ProductID))       AS lines_left_out
FROM per_category AS pc;
```

The first two checks should be zero. `lines_left_out` is not an error, but it belongs in the footnote of the report, because those lines are value the HHI cannot see.

Step 8 — Common Mistakes:

- Computing shares as fractions (0 to 1) and then comparing the HHI with 1,500 and 2,500. Fractional HHI runs from 0 to 1, so every category lands in `Unconcentrated`. Pick one scale and keep the bands on it.
- INNER JOIN to `Suppliers` before computing shares. Lines with an unknown supplier disappear, the remaining suppliers' shares grow, and the HHI moves for a reason nobody can see in the output. Join for the name only after the shares exist.
- Using `RANK()` for the top supplier. Two suppliers tied on value both get rank 1 and the category appears twice. Use `ROW_NUMBER` with a tie-break, or keep the tie and show both names deliberately with `STRING_AGG`.
- Computing shares from units when the question is about value. A supplier of cheap staples can dominate units and be minor in value.
- Averaging HHIs. A category's yearly HHI is not the average of its monthly HHIs. Re-sum the values for the period, then recompute shares and squares.

Step 9 — Interview Follow-Up: Procurement wants to see which categories became more concentrated in the latest year than in the year before. How would you compute an HHI per category per anchored calendar year and report the change, and what would you do with a category that had one supplier last year and none this year?

Step 10 — Optimization or Alternative: Business readers often find the "effective number of suppliers" easier than a 0 to 10,000 score. It is 10,000 / HHI: a category with an HHI of 5,000 behaves like two equal suppliers, whatever its raw supplier count says.

```
WITH supplier_value AS (
    SELECT cat.CategoryID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           od.SupplierID,
           SUM(od.Quantity * p.Sale_Price)        AS list_price_value
    FROM OrderDetails  AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    WHERE od.SupplierID IS NOT NULL
      AND od.Quantity   IS NOT NULL
      AND p.Sale_Price  IS NOT NULL
    GROUP BY cat.CategoryID, COALESCE(cat.CategoryName, 'Unmapped'), od.SupplierID
),
shares AS (
    SELECT sv.CategoryID,
           sv.category_name,
           100.0 * sv.list_price_value
                 / NULLIF(SUM(sv.list_price_value) OVER (PARTITION BY sv.CategoryID), 0) AS share_pct
    FROM supplier_value AS sv
)
SELECT sh.category_name,
       COUNT(*)                                              AS suppliers_used,
       ROUND(SUM(sh.share_pct * sh.share_pct), 0)            AS hhi,
       ROUND(10000.0 / NULLIF(SUM(sh.share_pct * sh.share_pct), 0), 2) AS effective_suppliers
FROM shares AS sh
GROUP BY sh.CategoryID, sh.category_name
ORDER BY effective_suppliers, sh.category_name;
```

Performance: the cost sits in the first CTE, a full scan of `OrderDetails` hash-joined to a small `Products` table. Indexes do little for a whole-table aggregate. If the report runs daily on a large line table, keep a summary table of value per (day, category, supplier) loaded incrementally, then sum it for any period and recompute shares and squares at report time. Store values, never shares or HHIs, because neither one adds up across periods. On MySQL 8 the same window functions work; the `FILTER` clause in the validation query becomes `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`, and the label must use `CONCAT`, because in MySQL `||` means OR.

Question 95: Debug a revenue-by-category query that gives wrong numbers

Difficulty: Expert

Primary Topic: Debugging an incorrect query

Combined Topics: One-to-many fan-out, WHERE clause on a LEFT JOIN, Anchored calendar-year filter, FULL JOIN to keep unmatched rows

Business Scenario: The category management team reviews every category once a year, including the ones that sold nothing, because a zero is a decision too. An analyst wrote the query that feeds the review. Finance says the category numbers add up to far more than the company sold, and the category manager for a newly added category cannot find it in the report. You have been asked to find out why and ship a corrected version.

Problem Statement: The analyst's query is reproduced in Step 3. It runs without error. Find every fault, show how a row-count or total check exposes each one, and write the corrected report: for the latest calendar year in the data, one row per category with the number of orders that contained it and its list-price value. Categories with no sales must appear with zeros, and sales of products with no matching category must appear as `Unmapped`. Sort from the lowest value up, so the review opens with the categories that need a decision.

Expected Output:

- `CategoryName`: category name, or `Unmapped`
- `orders`: distinct orders in the latest year with at least one line in the category
- `list_price_value`: `SUM(Quantity * Sale_Price)` over those lines

Tables Required: `Orders`, `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: grain and fan-out, how a WHERE filter on the right-hand table of a LEFT JOIN turns it into an inner join, anchored half-open year boundaries, FULL JOIN for unmatched rows on both sides, reconciliation checks.

Step 1 — Understand the Problem: Assumptions first.

1. "Latest year" is the calendar year that contains `MAX(OrderDate)`. Anchoring to the data, not `CURRENT_DATE`, keeps the report reproducible on a historical extract. If the data stops mid-year, label the year as partial in the report title.
2. The measure is list-price value, `od.Quantity * p.Sale_Price`. `Sale_Price` is today's catalogue price, not the price paid, so this is a proxy. `Total_Order_Amount` cannot be used here because it belongs to the whole order, and an order can hold several categories.
3. An order counts once per category it touches, so the `orders` column adds up to more than the year's order count. That is expected.
4. Every `Orders` row is a placed order. Cancellations and returns are not modelled.

Step 2 — Identify the Tables and Relationships: `Category` 1 → many `Products` (on `CategoryID`), `Products` 1 → many `OrderDetails` (on `ProductID`), `Orders` 1 → many `OrderDetails` (on `OrderID`). The order date lives on `Orders`, quantities on `OrderDetails`, prices on `Products`, names on `Category`. No foreign keys are enforced, so a product can point at a `CategoryID` that does not exist, and a line can point at a product that does not exist.

Step 3 — Build the Query Logic: Here is the query as the analyst wrote it:

```sql
SELECT cat.CategoryName,
       COUNT(o.OrderID)          AS orders,
       SUM(o.Total_Order_Amount) AS revenue
FROM Category AS cat
LEFT JOIN Products     AS p  ON p.CategoryID = cat.CategoryID
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
LEFT JOIN Orders       AS o  ON o.OrderID    = od.OrderID
WHERE o.OrderDate >= (SELECT MAX(OrderDate) FROM Orders) - INTERVAL '1 year'
GROUP BY cat.CategoryName
ORDER BY revenue DESC;
```

Do not start by rewriting it. Start by proving what is wrong, one claim per check. Each row below compares what the report should contain with what the broken query actually feeds into its aggregates:

```sql
WITH broken_rows AS (
    SELECT cat.CategoryName, o.OrderID, o.OrderDate, o.Total_Order_Amount
    FROM Category AS cat
    LEFT JOIN Products     AS p  ON p.CategoryID = cat.CategoryID
    LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
    LEFT JOIN Orders       AS o  ON o.OrderID    = od.OrderID
    WHERE o.OrderDate >= (SELECT MAX(OrderDate) FROM Orders) - INTERVAL '1 year'
),
broken_orders AS (
    SELECT DISTINCT br.OrderID, br.Total_Order_Amount
    FROM broken_rows AS br
)
SELECT 'A. rows summed vs distinct orders' AS check_name,
       (SELECT COUNT(*) FROM broken_orders)::text AS expected,
       (SELECT COUNT(*) FROM broken_rows)::text   AS actual
UNION ALL
SELECT 'B. revenue summed vs ledger for the same orders',
       (SELECT SUM(bo.Total_Order_Amount) FROM broken_orders AS bo)::text,
       (SELECT SUM(br.Total_Order_Amount) FROM broken_rows AS br)::text
UNION ALL
SELECT 'C. categories returned vs Category rows',
       (SELECT COUNT(*) FROM Category)::text,
       (SELECT COUNT(DISTINCT br.CategoryName) FROM broken_rows AS br)::text
UNION ALL
SELECT 'D. calendar years inside the date filter',
       '1',
       (SELECT COUNT(DISTINCT EXTRACT(YEAR FROM br.OrderDate)) FROM broken_rows AS br)::text
UNION ALL
SELECT 'E. lines whose product has no Category row',
       '0',
       (SELECT COUNT(*)
        FROM OrderDetails AS od
        JOIN Products     AS p ON p.ProductID = od.ProductID
        WHERE NOT EXISTS (SELECT 1 FROM Category AS cat
                          WHERE cat.CategoryID = p.CategoryID))::text;
```

What each check proves:

- **Fault 1, fan-out (checks A and B).** After joining to lines, each order appears once per line. `COUNT(o.OrderID)` counts lines, and `SUM(o.Total_Order_Amount)` adds the full order total once per line. An order with lines in three categories also hands its whole total to all three. If `actual` is above `expected` in A, the sum in B is inflated. Fix: measure at line grain (list-price value) and count orders with `COUNT(DISTINCT ...)`.
- **Fault 2, WHERE kills the LEFT JOIN (check C).** A category with no sales reaches the WHERE clause with `o.OrderDate` NULL. `NULL >= anything` is not true, so the row is removed and the outer join behaves like an inner join. Fix: apply the date filter where it cannot remove the category row, by aggregating the year's lines first and joining categories to the result.
- **Fault 3, wrong period (check D).** `MAX(OrderDate) - INTERVAL '1 year'` is a rolling window ending at the anchor, not a calendar year, and `>=` makes it 366 days. Any value above 1 in D means two calendar years are mixed. Fix: `DATE_TRUNC('year', MAX(OrderDate))` as a half-open start, and the next 1 January as the exclusive end.
- **Fault 4, unmapped products vanish (check E).** Driving from `Category` can never show lines whose product has an unknown `CategoryID`. A non-zero count is value missing from the report. Fix: drive from the lines and FULL JOIN to `Category`.

Step 4 — Final SQL Solution:

```sql
WITH latest_year AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date                     AS year_start,
           (DATE_TRUNC('year', MAX(o.OrderDate)) + INTERVAL '1 year')::date AS next_year_start
    FROM Orders AS o
),
year_lines AS (
    SELECT od.OrderID,
           cat.CategoryID,
           od.Quantity * p.Sale_Price AS list_price_value
    FROM Orders        AS o
    JOIN latest_year   AS ly  ON o.OrderDate >= ly.year_start
                             AND o.OrderDate <  ly.next_year_start
    JOIN OrderDetails  AS od  ON od.OrderID     = o.OrderID
    LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
),
category_sales AS (
    SELECT yl.CategoryID,
           COUNT(DISTINCT yl.OrderID) AS orders,
           SUM(yl.list_price_value)   AS list_price_value
    FROM year_lines AS yl
    GROUP BY yl.CategoryID
)
SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
       COALESCE(cs.orders, 0)                 AS orders,
       COALESCE(cs.list_price_value, 0)       AS list_price_value
FROM Category AS cat
FULL JOIN category_sales AS cs ON cs.CategoryID = cat.CategoryID
ORDER BY list_price_value, CategoryName;
```

Step 5 — Line-by-Line Explanation:

- `latest_year` turns the anchor into two dates: 1 January of the anchor's year and 1 January of the next. The filter is half-open, so it works whether `OrderDate` is a DATE or a TIMESTAMP.
- `year_lines` filters `Orders` first, on the driving table, so the date test never sees a NULL from an outer join. One row per line from that year.

- `LEFT JOIN Products` and `LEFT JOIN Category` keep lines with an unknown product or an unknown category. Their `cat.CategoryID` is NULL, so they form one group. A line with an unknown product has no price, so it adds to `orders` but not to value.
- `category_sales` aggregates at the category grain. `COUNT(DISTINCT yl.OrderID)` counts an order once even when it has several lines in the category.
- `FULL JOIN` keeps both kinds of unmatched row: categories with no sales (left side only) and the NULL-category sales group (right side only, since NULL never equals NULL).
- `COALESCE` turns missing sales into 0 and the NULL category into `Unmapped`. The ascending sort puts zero-sales categories at the top, where the review starts.

Step 6 — Sample Output:

categoryname	orders	list_price_value
Electronics Accessories	0	0
Unmapped	138	33524.00
Fruits & Vegetables	3030	419457.00
Baby Care	833	708133.00
Eggs, Meat & Fish	2047	993172.00
Bakery, Cakes & Dairy	5461	1318680.00
Kitchen, Garden & Pets	1864	1768147.00
Cleaning & Household	5234	2609875.00

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 13 rows.

Step 7 — Why This Solution Works: Each fault is removed at its source: value is measured at line grain, the date filter sits on the driving table, the period is a calendar year, and both unmatched sides survive the final join. Then prove it. The fixed report should have one row per `Category` row plus an `Unmapped` row when unmapped sales exist, and its value must equal the year's line value computed with no categories involved:

```
WITH latest_year AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date                    AS year_start,
           (DATE_TRUNC('year', MAX(o.OrderDate)) + INTERVAL '1 year')::date AS next_year_start
    FROM Orders AS o
),
year_lines AS (
    SELECT od.OrderID, cat.CategoryID, od.Quantity * p.Sale_Price AS list_price_value
    FROM Orders        AS o
    JOIN latest_year   AS ly  ON o.OrderDate >= ly.year_start AND o.OrderDate < ly.next_year_start
    JOIN OrderDetails  AS od  ON od.OrderID     = o.OrderID
    LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
),
report AS (
    SELECT COALESCE(cat.CategoryName, 'Unmapped') AS CategoryName,
           COALESCE(cs.list_price_value, 0)       AS list_price_value
    FROM Category AS cat
    FULL JOIN (SELECT yl.CategoryID, SUM(yl.list_price_value) AS list_price_value
               FROM year_lines AS yl
               GROUP BY yl.CategoryID) AS cs ON cs.CategoryID = cat.CategoryID
)
SELECT (SELECT COUNT(*) FROM report)                    AS report_rows,
       (SELECT COUNT(*) FROM Category)                  AS category_rows,
       (SELECT SUM(r.list_price_value) FROM report AS r) AS report_value,
       (SELECT SUM(od.Quantity * p.Sale_Price)
        FROM Orders       AS o
        JOIN latest_year  AS ly ON o.OrderDate >= ly.year_start AND o.OrderDate < ly.next_year_start
        JOIN OrderDetails AS od ON od.OrderID  = o.OrderID
        JOIN Products     AS p  ON p.ProductID = od.ProductID) AS line_value_for_year;
```

`report_value` must equal `line_value_for_year`. Keep this check next to the report and run it on every refresh.

Step 8 — Common Mistakes:

- Fixing fault 2 by moving the date test into the last `ON` clause. Categories come back, but the lines are still joined for every year: `orders` is right while `list_price_value` silently covers all of history.

```
SELECT cat.CategoryName,
       COUNT(DISTINCT o.OrderID)                    AS orders,
       COALESCE(SUM(od.Quantity * p.Sale_Price), 0) AS list_price_value_all_years
FROM Category AS cat
LEFT JOIN Products     AS p  ON p.CategoryID = cat.CategoryID
LEFT JOIN OrderDetails AS od ON od.ProductID = p.ProductID
LEFT JOIN Orders       AS o  ON o.OrderID    = od.OrderID
                            AND o.OrderDate >= (SELECT DATE_TRUNC('year', MAX(OrderDate))::date FROM Orders)
                            AND o.OrderDate <  (SELECT (DATE_TRUNC('year', MAX(OrderDate)) + INTERVAL '1 year')::date FROM
Orders)
GROUP BY cat.CategoryName
ORDER BY list_price_value_all_years DESC, cat.CategoryName;
```

- Patching fault 1 with `SUM(DISTINCT o.Total_Order_Amount)`. It drops two different orders that share an amount and still gives each category the whole order total.

- Filtering with `EXTRACT(YEAR FROM o.OrderDate) = 2024`. The year is hard-coded, and the function on the column stops an index on `OrderDate` from being used.
- Treating the sum of `orders` across categories as the year's order count. Multi-category orders are counted in each category they touch.

Step 9 — Interview Follow-Up: The business now wants "the latest 12 complete months" instead of the latest calendar year. Write the boundaries so the window holds exactly 12 whole months ending with the last complete month before the anchor, and explain how that differs from the analyst's original filter.

Step 10 — Optimization or Alternative: If finance insists on order money rather than list price, allocate each order's `Total_Order_Amount` across its categories in proportion to their list-price value. The allocated amounts then add back to the order totals instead of multiplying them:

```
WITH latest_year AS (
    SELECT DATE_TRUNC('year', MAX(o.OrderDate))::date                    AS year_start,
           (DATE_TRUNC('year', MAX(o.OrderDate)) + INTERVAL '1 year')::date AS next_year_start
    FROM Orders AS o
),
order_category_value AS (
    SELECT o.OrderID,
           o.Total_Order_Amount,
           cat.CategoryID,
           SUM(od.Quantity * p.Sale_Price) AS list_price_value
    FROM Orders        AS o
    JOIN latest_year   AS ly  ON o.OrderDate >= ly.year_start
                             AND o.OrderDate <  ly.next_year_start
    JOIN OrderDetails  AS od  ON od.OrderID     = o.OrderID
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    GROUP BY o.OrderID, o.Total_Order_Amount, cat.CategoryID
),
allocated AS (
    SELECT ocv.CategoryID,
           ocv.OrderID,
           ocv.Total_Order_Amount * ocv.list_price_value
               / NULLIF(SUM(ocv.list_price_value) OVER (PARTITION BY ocv.OrderID), 0) AS allocated_order_value
    FROM order_category_value AS ocv
)
SELECT COALESCE(cat.CategoryName, 'Unmapped')              AS CategoryName,
       COUNT(DISTINCT a.OrderID)                           AS orders,
       ROUND(COALESCE(SUM(a.allocated_order_value), 0), 2) AS allocated_order_value
FROM Category AS cat
FULL JOIN allocated AS a ON a.CategoryID = cat.CategoryID
GROUP BY cat.CategoryID, COALESCE(cat.CategoryName, 'Unmapped')
ORDER BY allocated_order_value DESC, CategoryName;
```

Orders with a NULL or zero header total allocate nothing, and orders with no priced lines cannot be allocated at all. Report both counts beside the result.

Performance: the fixed query filters `Orders` by a plain range on `OrderDate`, which an index on `Orders(OrderDate)` can serve, then probes `OrderDetails` by `OrderID`, which wants an index on `OrderDetails(OrderID)`. The broken query started from `Category`, so it had to read every line of every year before the WHERE clause threw most of them away. On MySQL, which has no `FULL JOIN`, write the final step as `Category LEFT JOIN category_sales` plus `UNION ALL` of the NULL-category row.

Question 96: Make a slow order report sargable and index it

Difficulty: Expert

Primary Topic: Query performance and indexing

Combined Topics: Sargable date ranges, Correlated subquery to join rewrite, Covering indexes, EXPLAIN

Business Scenario: The risk operations team manually reviews every order of 2,000 or more placed in the current month. A scheduled query builds their list each morning, with the line count and units so a reviewer can spot an odd basket at a glance. It was fast when the business was small. Now that `Orders` holds years of history it is one of the slowest jobs on the warehouse, and the platform team has asked you to fix it without changing a single output row.

Problem Statement: The current query is in Step 3. Rewrite it so the database can use indexes on the date and the order key, prove the new version returns exactly the same rows, and propose the indexes with the `EXPLAIN` evidence you would attach to the change request.

Expected Output:

- `OrderID`, `OrderDate`, `CustomerID`, `Total_Order_Amount`: the order header
- `line_items`: number of `OrderDetails` rows for the order (0 if none)
- `units`: `SUM(Quantity)` for the order (0 if none)
- Sorted by `Total_Order_Amount` descending, then `OrderID`

Tables Required: `Orders`, `OrderDetails`

SQL Concepts Tested: sargability, functions on indexed columns, NULL semantics of `COALESCE` in a predicate, correlated subqueries vs a single join, covering indexes, reading `EXPLAIN`, proving two queries equivalent.

Step 1 — Understand the Problem: Assumptions:

1. "Current month" is the calendar month containing `MAX(OrderDate)`. On a live system that month is usually still in progress, and that is fine

here: the list is month to date by design. Anchoring to the data instead of `CURRENT_DATE` keeps the result reproducible on an extract.
2. The threshold is 2,000, inclusive. Orders with a NULL `Total_Order_Amount` are not on the list in either version.
3. Output must match the old query row for row. A performance fix that changes results needs its own sign-off.

A predicate is sargable (Search ARGument ABLE) when the indexed column stands alone on one side of the comparison, so the database can turn it into a start and stop point in an index. `OrderDate >= x AND OrderDate < y` is sargable. `TO_CHAR(OrderDate, 'YYYY-MM') = '2024-12'` is not: the database must compute `TO_CHAR` for every row before it can compare anything.

Step 2 — Identify the Tables and Relationships: `Orders` holds one row per order with the date, customer and amount. `OrderDetails` holds one row per line; `OrderDetails.OrderID` → `Orders.OrderID` is many to one. The only index that exists by default is each table's primary key, so nothing supports a search on `Orders.OrderDate` or on `OrderDetails.OrderID`.

Step 3 — Build the Query Logic: The query in production today. It is correct; it is just slow:

```
SELECT o.OrderID,
       o.OrderDate,
       o.CustomerID,
       o.Total_Order_Amount,
       (SELECT COUNT(*) FROM OrderDetails AS od
        WHERE od.OrderID = o.OrderID)                        AS line_items,
       COALESCE((SELECT SUM(od.Quantity) FROM OrderDetails AS od
                 WHERE od.OrderID = o.OrderID), 0)           AS units
FROM Orders AS o
WHERE TO_CHAR(o.OrderDate, 'YYYY-MM') = (SELECT TO_CHAR(MAX(OrderDate), 'YYYY-MM') FROM Orders)
  AND COALESCE(o.Total_Order_Amount, 0) >= 2000
ORDER BY o.Total_Order_Amount DESC, o.OrderID;
```

Three problems, each with its fix:

1. **`TO_CHAR` on the date column.** Every row of `Orders` is formatted and compared, whatever indexes exist. You cannot even rescue it with an expression index: PostgreSQL refuses, because converting a DATE to text goes through the session time zone and the function is not immutable. Fix: compute the month's first day and the next month's first day once, from the anchor, and compare the bare column with a half-open range.
2. **`COALESCE` on the amount.** `COALESCE(NULL, 0) >= 2000` is false, and so is `NULL >= 2000`, so the wrapper changes no result. It only hides the column from any index. Fix: `o.Total_Order_Amount >= 2000`.
3. **Two correlated subqueries per output row.** PostgreSQL runs each one once per qualifying order. With no index on `OrderDetails.OrderID`, each run reads the whole line table. Fix: one `LEFT JOIN` to the lines and a single `GROUP BY`, so the lines for all selected orders are fetched in one pass.

Step 4 — Final SQL Solution:

```
WITH month_bounds AS (
    SELECT DATE_TRUNC('month', MAX(o.OrderDate))::date                       AS month_start,
           (DATE_TRUNC('month', MAX(o.OrderDate)) + INTERVAL '1 month')::date AS next_month_start
    FROM Orders AS o
)
SELECT o.OrderID,
       o.OrderDate,
       o.CustomerID,
       o.Total_Order_Amount,
       COUNT(od.OrderID)             AS line_items,
       COALESCE(SUM(od.Quantity), 0) AS units
FROM Orders AS o
CROSS JOIN month_bounds AS mb
LEFT JOIN OrderDetails AS od ON od.OrderID = o.OrderID
WHERE o.OrderDate >= mb.month_start
  AND o.OrderDate <  mb.next_month_start
  AND o.Total_Order_Amount >= 2000
GROUP BY o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount
ORDER BY o.Total_Order_Amount DESC, o.OrderID;
```

Step 5 — Line-by-Line Explanation:

- `month_bounds` returns one row with two dates. With an index on `OrderDate`, `MAX(OrderDate)` reads one entry from the end of the index.
- `CROSS JOIN month_bounds` attaches those dates to every candidate order. One row times many is safe.
- `o.OrderDate >= mb.month_start AND o.OrderDate < mb.next_month_start` leaves the column bare, so an index on `OrderDate` can jump straight to the month. The half-open form also stays correct if the column ever becomes a TIMESTAMP.
- `o.Total_Order_Amount >= 2000` has the same meaning as before, without the wrapper.
- `LEFT JOIN OrderDetails` keeps a qualifying order that has no lines. `COUNT(od.OrderID)` counts only matched lines, so that order gets 0, exactly as `COUNT(*)` in the old subquery did. `COUNT(*)` here would wrongly return 1.
- `GROUP BY` lists the four header columns. Each order is one group, and `Total_Order_Amount` is grouped on, never summed, so the one-to-many join cannot inflate it.
- The `ORDER BY` is unchanged, including the `OrderID` tie-break.

Step 6 — Sample Output:

orderid	orderdate	customerid	total_order_amount	line_items	units
5044466	2025-12-10	102697	37640.00	2	11
5045859	2025-12-24	100042	37586.00	6	31
5044853	2025-12-13	104750	27701.00	4	32

5044431	2025-12-10	100897	27224.00	4	28
5044476	2025-12-10	103178	21441.00	6	28
5045932	2025-12-24	103671	21355.00	5	13
5044977	2025-12-15	100384	20888.00	5	33
5044965	2025-12-14	104743	17365.00	5	30

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 563 rows.

Step 7 — Why This Solution Works: The filter now describes a contiguous slice of the date index instead of a computed value, and the lines are fetched by order key in a single join. Logically nothing changed, and you should prove that rather than assert it. `EXCEPT ALL` compares two result sets as multisets, so duplicated rows count too. Both directions must return 0:

```
WITH old_report AS (
    SELECT o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount,
           (SELECT COUNT(*) FROM OrderDetails AS od WHERE od.OrderID = o.OrderID) AS line_items,
           COALESCE((SELECT SUM(od.Quantity) FROM OrderDetails AS od
                     WHERE od.OrderID = o.OrderID), 0)                          AS units
    FROM Orders AS o
    WHERE TO_CHAR(o.OrderDate, 'YYYY-MM') = (SELECT TO_CHAR(MAX(OrderDate), 'YYYY-MM') FROM Orders)
      AND COALESCE(o.Total_Order_Amount, 0) >= 2000
),
new_report AS (
    SELECT o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount,
           COUNT(od.OrderID) AS line_items, COALESCE(SUM(od.Quantity), 0) AS units
    FROM Orders AS o
    LEFT JOIN OrderDetails AS od ON od.OrderID = o.OrderID
    WHERE o.OrderDate >= (SELECT DATE_TRUNC('month', MAX(OrderDate))::date FROM Orders)
      AND o.OrderDate <  (SELECT (DATE_TRUNC('month', MAX(OrderDate)) + INTERVAL '1 month')::date FROM Orders)
      AND o.Total_Order_Amount >= 2000
    GROUP BY o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount
)
SELECT (SELECT COUNT(*) FROM (SELECT * FROM old_report EXCEPT ALL SELECT * FROM new_report) AS x) AS only_in_old,
       (SELECT COUNT(*) FROM (SELECT * FROM new_report EXCEPT ALL SELECT * FROM old_report) AS y) AS only_in_new,
       (SELECT COUNT(*) FROM new_report)                                                        AS rows_compared;
```

Run it on a full copy of production data; orders without lines and NULL amounts hide in the long tail.

Step 8 — Common Mistakes:

- Trying `CREATE INDEX ... ON Orders (TO_CHAR(OrderDate, 'YYYY-MM'))` to keep the old filter. It fails with "functions in index expression must be marked IMMUTABLE". Even where an expression index is allowed, such as `EXTRACT(YEAR FROM OrderDate)`, you end up with one index per way of writing the filter.
- Swapping to `BETWEEN '2024-12-01' AND '2024-12-31'`. It is sargable, but the date is hard-coded, and on a TIMESTAMP column it drops everything after midnight on the last day.
- Keeping `COALESCE` "for safety". NULL already fails the comparison.
- Rewriting the subqueries as a join but writing `COUNT(*)`, which turns 0 lines into 1.
- Indexing every column in the WHERE clause separately and hoping the planner combines them. Design one index for the access path the query actually takes.

Step 9 — Interview Follow-Up: The 2,000 threshold becomes a parameter that the risk team changes most weeks, and the list must now be split by `ShipperID`. Would you change the `Orders` index, and when would a partial index such as `WHERE Total_Order_Amount >= 2000` stop being usable?

Step 10 — Optimization or Alternative: The indexes, tried inside a transaction so nothing persists:

```sql
BEGIN;

CREATE INDEX ix_orders_orderdate
    ON Orders (OrderDate) INCLUDE (OrderID, CustomerID, Total_Order_Amount);
CREATE INDEX ix_orderdetails_orderid
    ON OrderDetails (OrderID) INCLUDE (Quantity);

-- Before: look for the date test on a Filter line
EXPLAIN
SELECT o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount,
       (SELECT COUNT(*) FROM OrderDetails AS od WHERE od.OrderID = o.OrderID) AS line_items,
       COALESCE((SELECT SUM(od.Quantity) FROM OrderDetails AS od
                 WHERE od.OrderID = o.OrderID), 0)                          AS units
FROM Orders AS o
WHERE TO_CHAR(o.OrderDate, 'YYYY-MM') = (SELECT TO_CHAR(MAX(OrderDate), 'YYYY-MM') FROM Orders)
  AND COALESCE(o.Total_Order_Amount, 0) >= 2000
ORDER BY o.Total_Order_Amount DESC, o.OrderID;

-- After: the date range is eligible for an Index Cond line
EXPLAIN
WITH month_bounds AS (
    SELECT DATE_TRUNC('month', MAX(o.OrderDate))::date                      AS month_start,
           (DATE_TRUNC('month', MAX(o.OrderDate)) + INTERVAL '1 month')::date AS next_month_start
    FROM Orders AS o
)
SELECT o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount,
       COUNT(od.OrderID) AS line_items, COALESCE(SUM(od.Quantity), 0) AS units
FROM Orders AS o
CROSS JOIN month_bounds AS mb
LEFT JOIN OrderDetails AS od ON od.OrderID = o.OrderID
WHERE o.OrderDate >= mb.month_start
  AND o.OrderDate <  mb.next_month_start
  AND o.Total_Order_Amount >= 2000
GROUP BY o.OrderID, o.OrderDate, o.CustomerID, o.Total_Order_Amount
ORDER BY o.Total_Order_Amount DESC, o.OrderID;

ROLLBACK;
```

Why these indexes. `Orders(OrderDate)` is keyed on the range column because one month out of several years is the selective part of the filter; the amount is checked within that slice. The `INCLUDE` columns are everything the report reads from `Orders`, so PostgreSQL can answer from the index alone (an index-only scan) once the table has been vacuumed. `OrderDetails(OrderID)` serves the join, and almost every order-to-lines query needs it too.

Reading the plans. In the old query, the `TO_CHAR` test can only appear on a `Filter:` line under a scan of all of `Orders`, with or without the index. In the new one, the range can appear as `Index Cond:` on `ix_orders_orderdate`. What this dataset shows: with the two indexes in place on the 46,438 orders, the old query still plans a `Seq Scan on orders` with the `to_char(...)` test on its `Filter:` line, because a function on the column cannot use an index. The rewrite plans an `Index Only Scan using ix_orders_orderdate` with the month range on the `Index Cond:` line, and reaches the lines through `ix_orderdetails_orderid`. Do not read too much into the timings at this size: both versions still finish in milliseconds. The structural difference is what scales. At production size one month is a small fraction of the table, the range scan touches only that slice, and the gap grows every month, while the old query keeps reading everything. `SET LOCAL enable_seqscan = off` is a diagnostic for checking whether an index can be used at all, never a fix. When timing the two, use `EXPLAIN (ANALYZE, BUFFERS)` and compare buffers read, not only time, because a warm cache flatters any plan.

Shipping it: on a live table use `CREATE INDEX CONCURRENTLY`, which does not block writes but cannot run inside a transaction block. Every index slows inserts, so check `pg_stat_user_indexes` later and drop what is never scanned. A partial index `ON Orders (OrderDate) WHERE Total_Order_Amount >= 2000` would be much smaller, but the planner can use it only when the query's threshold is at least that literal. MySQL 8 has no `INCLUDE`, so put the extra columns in the key instead, as in `(OrderDate, Total_Order_Amount, CustomerID)`.

Question 97: What do customers buy on their second order?

Difficulty: Expert

Primary Topic: Sequence analysis with ROW_NUMBER

Combined Topics: Dominant category per order, Conditional aggregation pivot, Window SUM over aggregates for row percentages, LEFT JOIN to keep customers without a second order

Business Scenario: The growth team sends a "come back" voucher a few days after a customer's first order. Today every voucher shows the same banner. The merchandising lead wants the banner to depend on what the customer bought first, but only if first-order category actually predicts the second order. If people who start with staples come back for staples, the banner should show staples; if they move to snacks, it should show snacks.

Problem Statement: For each customer, find the dominant category of their first order and of their second order. Count customers for every (first category, second category) pair, including customers who never placed a second order, and show each pair as a percentage of all customers who started in that first category.

Expected Output:

- `first_order_category`: dominant category of the customer's first order
- `second_order_category`: dominant category of the second order, or `No second order`
- `customers`: customers on that path

- `pct_of_first_category`: `customers` as a percentage of everyone with the same `first_order_category`, 2 dp

Tables Required: `Orders`, `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: sequencing events per customer with ROW_NUMBER, collapsing a multi-category order to one label, pivoting two events onto one row, keeping customers with no second event, row percentages with a window over an aggregate.

Step 1 — Understand the Problem: Two definitions drive everything, so state them first.

1. **Order sequence.** A customer's orders are numbered by `OrderDate`, then `OrderID`. Same-day orders still get a stable order. Orders with a NULL `CustomerID` or `OrderDate` cannot be placed in a sequence and are left out. A `CustomerID` with no `Customers` row is still a buyer and stays in.
2. **Dominant category.** An order can hold several categories, and the report needs exactly one label per order. The dominant category is the one with the highest list-price value in that order, `SUM(od.Quantity * p.Sale_Price)`. `Sale_Price` is today's catalogue price, not the price paid, so this is a proxy. Ties go to more units, then to the category name, so the label is deterministic. A line whose product has no `Category` row is labelled `Unmapped`. An order with no lines at all is labelled `No line items` rather than dropped.

Other assumptions: every `Orders` row is a real placed order (cancellations are not modelled), and the whole history is used. A customer whose first order was recent has had little time to return, so their `No second order` is not final. Step 10 deals with that.

Step 2 — Identify the Tables and Relationships: `Orders` gives the sequence (`CustomerID`, `OrderDate`, `OrderID`). `OrderDetails.OrderID` → `Orders.OrderID` gives the lines in each order. `OrderDetails.ProductID` → `Products.ProductID` gives price and `CategoryID`. `Products.CategoryID` → `Category.CategoryID` gives the name. `Customers` is not needed: the customer key on `Orders` is enough to group.

Step 3 — Build the Query Logic:

1. Number each customer's orders with `ROW_NUMBER()` and keep orders 1 and 2. Doing this before touching the lines keeps the expensive joins small.
2. Aggregate those orders' lines to (order, category) with list-price value and units.
3. Rank categories inside each order and keep rank 1. That is the order's dominant category.
4. Pivot to one row per customer: `MAX(...) FILTER (WHERE order_number = 1)` for the first category and the same for the second. The LEFT JOIN from the order list to the dominant categories keeps orders without lines.
5. Count customers per (first, second) pair. Compute the row percentage with `SUM(COUNT(*)) OVER (PARTITION BY first_order_category)`, which adds up the counts of every pair that shares the first category.

Step 4 — Final SQL Solution:

```sql
WITH order_seq AS (
    SELECT o.OrderID,
           o.CustomerID,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID
                              ORDER BY o.OrderDate, o.OrderID) AS order_number
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
first_two AS (
    SELECT os.OrderID, os.CustomerID, os.order_number
    FROM order_seq AS os
    WHERE os.order_number <= 2
),
order_category_value AS (
    SELECT ft.OrderID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name,
           SUM(od.Quantity * p.Sale_Price)        AS list_price_value,
           SUM(od.Quantity)                       AS units
    FROM first_two AS ft
    JOIN OrderDetails  AS od  ON od.OrderID     = ft.OrderID
    LEFT JOIN Products AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
    GROUP BY ft.OrderID, COALESCE(cat.CategoryName, 'Unmapped')
),
dominant AS (
    SELECT ocv.OrderID,
           ocv.category_name,
           ROW_NUMBER() OVER (PARTITION BY ocv.OrderID
                              ORDER BY ocv.list_price_value DESC NULLS LAST,
                                       ocv.units DESC NULLS LAST,
                                       ocv.category_name) AS category_rank
    FROM order_category_value AS ocv
),
customer_path AS (
    SELECT ft.CustomerID,
           MAX(COALESCE(d.category_name, 'No line items')) FILTER (WHERE ft.order_number = 1) AS first_order_category,
           MAX(COALESCE(d.category_name, 'No line items')) FILTER (WHERE ft.order_number = 2) AS second_order_category
    FROM first_two AS ft
    LEFT JOIN dominant AS d
           ON d.OrderID       = ft.OrderID
          AND d.category_rank = 1
    GROUP BY ft.CustomerID
)
SELECT cp.first_order_category,
       COALESCE(cp.second_order_category, 'No second order') AS second_order_category,
       COUNT(*)                                              AS customers,
       ROUND(100.0 * COUNT(*)
             / SUM(COUNT(*)) OVER (PARTITION BY cp.first_order_category), 2) AS pct_of_first_category
FROM customer_path AS cp
GROUP BY cp.first_order_category,
         COALESCE(cp.second_order_category, 'No second order')
ORDER BY SUM(COUNT(*)) OVER (PARTITION BY cp.first_order_category) DESC,
         cp.first_order_category,
         customers DESC,
         second_order_category;
```

Step 5 — Line-by-Line Explanation:

- `order_seq` numbers each customer's orders. `OrderID` breaks same-day ties, so a rerun never swaps first and second.
- `first_two` keeps only orders 1 and 2, so the line joins below touch a small slice of the history.
- `order_category_value` joins lines to products and categories and sums value and units per (order, category). The LEFT JOINs keep lines with an unknown product or category under `Unmapped` instead of losing them.
- `dominant` ranks categories within each order. `NULLS LAST` matters: an `Unmapped` group built from a missing product has NULL value and must not outrank a real category.
- `customer_path` is the pivot. The LEFT JOIN keeps an order with no lines (its category comes back NULL and is relabelled `No line items`). `MAX(...) FILTER` picks the one value per customer for each order number. A customer with only one order gets NULL in the second column.
- In the final SELECT, `COUNT(*)` counts customers on each path. `SUM(COUNT(*)) OVER (PARTITION BY first_order_category)` runs after the GROUP BY, so it adds the path counts back up to the size of the first-category group.
- The ORDER BY puts the largest first-category groups first and keeps every row in a fixed position.

Step 6 — Sample Output:

first_order_category	second_order_category	customers	pct_of_first_category
Beauty & Hygiene	Beauty & Hygiene	518	34.86
Beauty & Hygiene	No second order	226	15.21
Beauty & Hygiene	Foodgrains, Oil & Masala	164	11.04
Beauty & Hygiene	Gourmet & World Food	134	9.02
Beauty & Hygiene	Snacks & Branded Foods	125	8.41
Beauty & Hygiene	Beverages	85	5.72
Beauty & Hygiene	Cleaning & Household	79	5.32

Beauty & Hygiene	Bakery, Cakes & Dairy	63	4.24

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 137 rows.

Step 7 — Why This Solution Works: Each customer becomes exactly one row in `customer_path`, so a customer can only be counted on one path. That gives three checks to run before anyone reads the percentages. The `customers` column summed over all rows must equal the number of distinct non-NULL `CustomerID` values on `Orders`. The rows that are not `No second order` must add up to the number of customers who have a second order. And within every `first_order_category`, `pct_of_first_category` must add up to 100, give or take rounding:

```
WITH order_seq AS (
    SELECT o.OrderID, o.CustomerID,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID ORDER BY o.OrderDate, o.OrderID) AS order_number
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL AND o.OrderDate IS NOT NULL
)
SELECT COUNT(DISTINCT os.CustomerID)                AS customers_expected,
       COUNT(*) FILTER (WHERE os.order_number = 2)  AS customers_with_second_order,
       COUNT(*) FILTER (WHERE os.order_number <= 2
                        AND NOT EXISTS (SELECT 1 FROM OrderDetails AS od
                                        WHERE od.OrderID = os.OrderID)) AS first_two_orders_without_lines
FROM order_seq AS os;
```

Step 8 — Common Mistakes:

- Joining all lines of the first two orders and counting customers per (category of any line, category of any line). A customer whose first order had three categories is then counted on three paths, and percentages add up to far more than 100.
- Picking the "first category" with `MIN(CategoryName)`. That is alphabetical, not dominant, and has nothing to do with what the customer spent on.
- Using an inner join from orders to lines. Customers whose first or second order has no lines disappear, and the denominators shrink with no trace in the output.
- Taking order 2 as a real return visit without checking for duplicate submissions. If the same order was submitted twice on the same day, the "second order" is a copy of the first and inflates same-category repeats. Deduplicate first, or require the second order to fall on a later date.
- Computing the percentage over all customers instead of within the first category. The merchandising question is conditional: given what they bought first, what did they buy next?

Step 9 — Interview Follow-Up: The team wants a lift-style comparison: for each first category, is the share of customers whose second order is in the same category higher than that category's share of all second orders? Write the query and explain what a value above 1 would mean for the banner.

Step 10 — Optimization or Alternative: Right-censoring. A customer whose first order was last week sits in `No second order` only because there has not been time to come back. Restrict the cohort to customers whose first order is at least 90 days before the anchor (`MAX(OrderDate)`), and the "came back" rate becomes comparable across categories. The version below also picks the dominant category with `DISTINCT ON`, a PostgreSQL shortcut for "first row per group in this order" (on MySQL use `ROW_NUMBER` as in Step 4):

```sql
WITH params AS (
    SELECT MAX(o.OrderDate) - 90 AS mature_before
    FROM Orders AS o
),
order_seq AS (
    SELECT o.OrderID, o.CustomerID, o.OrderDate,
           ROW_NUMBER() OVER (PARTITION BY o.CustomerID
                              ORDER BY o.OrderDate, o.OrderID) AS order_number
    FROM Orders AS o
    WHERE o.CustomerID IS NOT NULL
      AND o.OrderDate  IS NOT NULL
),
mature_customers AS (
    SELECT os.CustomerID
    FROM order_seq AS os
    CROSS JOIN params AS pr
    WHERE os.order_number = 1
      AND os.OrderDate <= pr.mature_before
),
dominant AS (
    SELECT DISTINCT ON (os.OrderID)
           os.OrderID, os.CustomerID, os.order_number,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name
    FROM order_seq AS os
    JOIN mature_customers AS mc  ON mc.CustomerID  = os.CustomerID
    JOIN OrderDetails     AS od  ON od.OrderID     = os.OrderID
    LEFT JOIN Products    AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category    AS cat ON cat.CategoryID = p.CategoryID
    WHERE os.order_number <= 2
    GROUP BY os.OrderID, os.CustomerID, os.order_number, COALESCE(cat.CategoryName, 'Unmapped')
    ORDER BY os.OrderID,
             SUM(od.Quantity * p.Sale_Price) DESC NULLS LAST,
             SUM(od.Quantity) DESC NULLS LAST,
             COALESCE(cat.CategoryName, 'Unmapped')
)
SELECT f.category_name                                          AS first_order_category,
       COUNT(*)                                                 AS mature_customers,
       COUNT(s.OrderID)                                         AS came_back,
       COUNT(*) FILTER (WHERE s.category_name = f.category_name) AS stayed_in_category,
       ROUND(100.0 * COUNT(*) FILTER (WHERE s.category_name = f.category_name)
             / NULLIF(COUNT(s.OrderID), 0), 2)                  AS same_category_pct_of_returners
FROM dominant AS f
LEFT JOIN dominant AS s
       ON s.CustomerID   = f.CustomerID
      AND s.order_number = 2
WHERE f.order_number = 1
GROUP BY f.category_name
ORDER BY mature_customers DESC, first_order_category;
```

This shorter version drops orders without lines, which Step 4 keeps; say so if you present it. Performance: the window in `order_seq` sorts all of `Orders` by (`CustomerID`, `OrderDate`, `OrderID`), and an index on exactly those columns lets PostgreSQL read them in order. Filtering to two orders per customer before joining `OrderDetails` is the main saving; with an index on `OrderDetails(OrderID)`, the line lookup touches only those orders. Small first-category groups produce noisy percentages, so show `customers` beside every percentage and agree a minimum group size with the team before a banner decision is made from it.

Question 98: Category association rules: support, confidence, lift

Difficulty: Expert

Primary Topic: Market basket analysis

Combined Topics: Distinct order-category baskets, Self join on OrderID, Multiple CTEs, Ratio metrics with a minimum-support filter

Business Scenario: The merchandising team is redesigning the cart page. When a shopper adds something from one category, a strip below suggests a second category. The team wants the suggestion to be a category that the first one genuinely pulls along, not just whichever category is most popular overall. They have asked for association rules between categories, with enough volume behind each rule to trust it.

Problem Statement: Treat each order as a basket of the distinct categories in it. For every ordered pair of categories (A, B) bought together in at least 2% of baskets, return the number of baskets containing both, the support, the confidence of the rule "A leads to B" and the lift. Sort by lift, highest first.

Expected Output:

- `category_a`: the category already in the basket (the rule's left side)
- `category_b`: the category being suggested (the right side)
- `orders_with_both`: baskets containing both A and B
- `support_pct`: `orders_with_both` as a percentage of all baskets, 2 dp
- `confidence_a_to_b_pct`: `orders_with_both` as a percentage of baskets containing A, 2 dp
- `lift`: confidence of A to B divided by the share of baskets that contain B, 3 dp

Tables Required: `OrderDetails`, `Products`, `Category`

SQL Concepts Tested: deduplicating to a set per order, self join on a shared key, pair counting, ratios built from three different denominators, thresholds as parameters.

Step 1 — Understand the Problem: Definitions first, because the three metrics are easy to mix up. Let N be the number of baskets, n(A) the baskets containing A, and n(A,B) the baskets containing both.

- **Support** = n(A,B) / N. How common the pair is. It is the same for A to B and B to A.
- **Confidence (A to B)** = n(A,B) / n(A). Of the baskets that have A, the share that also have B. It is directional: A to B and B to A have different denominators.
- **Lift** = n(A,B) × N / (n(A) × n(B)). Confidence divided by B's overall basket share. Lift of 1 means A tells you nothing about B. Above 1, B appears with A more often than chance. Below 1, less often. Lift is symmetric.

Why lift matters here: a category that sits in most baskets will have high confidence from almost every A, because it is everywhere. Suggesting it adds nothing. Lift removes that popularity effect.

Assumptions:

1. A basket is one `OrderID` in `OrderDetails`. Category presence is what counts, not quantity or value: an order with ten lines of snacks has snacks once.
2. Lines whose product has no `Products` row have no category and are left out. Lines whose product points to a missing category count as `Unmapped`, so N matches the number of orders that have at least one known product.
3. Orders with a single category are part of N. They are real baskets in which nothing was bought together, and leaving them out would inflate every support figure.
4. The minimum support is 2% of baskets. It is a parameter: on a large catalogue you might use 0.5%; on a small sample, higher.

Step 2 — Identify the Tables and Relationships: `OrderDetails` gives `OrderID` and `ProductID`. `OrderDetails.ProductID` → `Products.ProductID` gives `CategoryID`. `Products.CategoryID` → `Category.CategoryID` gives the name. `Orders` is not needed: the basket key already sits on the lines, and nothing here depends on the order header or date.

Step 3 — Build the Query Logic:

1. Build `baskets`: `SELECT DISTINCT OrderID, category_name`. This is the step that makes everything else correct. Without it, an order with three snack lines would count snacks three times.
2. Count N, the distinct orders in `baskets`.
3. Count n(category) per category.
4. Self join `baskets` to itself on `OrderID` with `a.category_name <> b.category_name`. Each order produces every ordered pair of its distinct categories once, so `COUNT(*)` per (A, B) is n(A,B).
5. Join n(A) and n(B), compute the three ratios, and filter on support.

Step 4 — Final SQL Solution:

```sql
WITH params AS (
    SELECT 2.0 AS min_support_pct
),
baskets AS (
    SELECT DISTINCT
           od.OrderID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name
    FROM OrderDetails  AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
),
basket_total AS (
    SELECT COUNT(DISTINCT b.OrderID) AS total_orders
    FROM baskets AS b
),
category_orders AS (
    SELECT b.category_name,
           COUNT(*) AS orders_with_category
    FROM baskets AS b
    GROUP BY b.category_name
),
pair_orders AS (
    SELECT a.category_name AS category_a,
           b.category_name AS category_b,
           COUNT(*)        AS orders_with_both
    FROM baskets AS a
    JOIN baskets AS b
      ON b.OrderID        = a.OrderID
     AND b.category_name <> a.category_name
    GROUP BY a.category_name, b.category_name
)
SELECT po.category_a,
       po.category_b,
       po.orders_with_both,
       ROUND(100.0 * po.orders_with_both / bt.total_orders, 2)          AS support_pct,
       ROUND(100.0 * po.orders_with_both / ca.orders_with_category, 2)  AS confidence_a_to_b_pct,
       ROUND(1.0 * po.orders_with_both * bt.total_orders
             / (ca.orders_with_category * cb.orders_with_category), 3)  AS lift
FROM pair_orders AS po
JOIN category_orders AS ca ON ca.category_name = po.category_a
JOIN category_orders AS cb ON cb.category_name = po.category_b
CROSS JOIN basket_total AS bt
CROSS JOIN params       AS pr
WHERE 100.0 * po.orders_with_both / bt.total_orders >= pr.min_support_pct
ORDER BY lift DESC, confidence_a_to_b_pct DESC, po.category_a, po.category_b;
```

Step 5 — Line-by-Line Explanation:

- `params` holds the threshold in one place, so changing it never means hunting through the query.
- `baskets` is the set of (order, category). `DISTINCT` turns line grain into presence grain. The inner JOIN to `Products` drops lines with an unknown product; the LEFT JOIN to `Category` keeps an unknown category as `Unmapped`.
- `basket_total` is N. `COUNT(DISTINCT OrderID)` is needed because `baskets` has one row per category per order.
- `category_orders` is n(A) for every category. After the `DISTINCT`, `COUNT(*)` per category is a count of orders.
- `pair_orders` is the self join. `<>` (rather than `<`) keeps both directions, (A, B) and (B, A), because confidence differs by direction. Both rows carry the same `orders_with_both`.
- In the final SELECT, `100.0 *` and `1.0 *` force numeric division. Integer division in PostgreSQL would truncate every ratio to 0. No `NULLIF` is needed in the denominators: a category that appears in a pair has at least one basket, so n(A), n(B) and N are never zero.
- `WHERE` applies minimum support to the pair, which is the same for both directions, so a rule and its reverse pass or fail together.

Step 6 — Sample Output:

category_a	category_b	orders_with_both	support_pct	confidence_a_to_b_pct	lift
Eggs, Meat & Fish	Bakery, Cakes & Dairy	932	2.01	28.41	1.511
Bakery, Cakes & Dairy	Eggs, Meat & Fish	932	2.01	10.68	1.511
Beverages	Snacks & Branded Foods	5001	10.77	49.15	1.092
Snacks & Branded Foods	Beverages	5001	10.77	23.94	1.092
Kitchen, Garden & Pets	Foodgrains, Oil & Masala	1312	2.83	42.34	1.049
Foodgrains, Oil & Masala	Kitchen, Garden & Pets	1312	2.83	7.00	1.049
Fruits & Vegetables	Snacks & Branded Foods	2254	4.85	46.28	1.029
Snacks & Branded Foods	Fruits & Vegetables	2254	4.85	10.79	1.029

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 68 rows.

Step 7 — Why This Solution Works: Every count is taken from the same deduplicated basket table, so numerators and denominators share a grain. Two properties make good tests. First, `orders_with_both` for (A, B) must equal the value for (B, A). Second, a slow, obviously correct brute-force count with `EXISTS` must agree with the self-join count for every pair. Both mismatch columns should be 0:

```
WITH line_category AS (
    SELECT od.OrderID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name
    FROM OrderDetails  AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
),
baskets AS (
    SELECT DISTINCT lc.OrderID, lc.category_name FROM line_category AS lc
),
pair_orders AS (
    SELECT a.category_name AS category_a, b.category_name AS category_b, COUNT(*) AS orders_with_both
    FROM baskets AS a
    JOIN baskets AS b ON b.OrderID = a.OrderID AND b.category_name <> a.category_name
    GROUP BY a.category_name, b.category_name
),
brute_force AS (
    SELECT po.category_a, po.category_b,
           (SELECT COUNT(DISTINCT x.OrderID)
            FROM line_category AS x
            WHERE x.category_name = po.category_a
              AND EXISTS (SELECT 1 FROM line_category AS y
                          WHERE y.OrderID = x.OrderID
                            AND y.category_name = po.category_b)) AS orders_with_both_check
    FROM pair_orders AS po
)
SELECT COUNT(*)                                                          AS directed_pairs,
       COUNT(*) FILTER (WHERE bf.orders_with_both_check <> po.orders_with_both) AS count_mismatches,
       COUNT(*) FILTER (WHERE po.orders_with_both <> rev.orders_with_both)      AS asymmetric_pairs
FROM pair_orders AS po
JOIN brute_force AS bf  ON bf.category_a  = po.category_a AND bf.category_b  = po.category_b
JOIN pair_orders AS rev ON rev.category_a = po.category_b AND rev.category_b = po.category_a;
```

Step 8 — Common Mistakes:

- Self joining `OrderDetails` lines instead of distinct baskets. An order with two snack lines and three tea lines yields six snack-tea pairs instead of one, and support goes above what is possible.
- Using `a.category_name < b.category_name` and then reporting confidence. That keeps only one direction, so half the rules disappear, and the surviving direction is decided by alphabetical order rather than by the business question.
- Taking N as the number of orders in `Orders`. Orders without lines are not baskets, so they would dilute support. Count N from the same table the pairs come from.
- Ranking by confidence alone. A very common category has high confidence from almost any A, and the cart strip would show it every time. Use lift to rank and support as a floor.
- Reading a high lift on tiny counts as a finding. With five baskets, one coincidence moves lift a lot. That is what the minimum-support filter is for; show `orders_with_both` next to every rule.

Step 9 — Interview Follow-Up: The team now wants rules at product level instead of category level, on a catalogue of 50,000 products. What breaks in this query, and how would you cut the pair explosion before the self join?

Step 10 — Optimization or Alternative: The cost is the self join. An order with k categories yields k × (k − 1) directed pairs, so a few very large baskets dominate the work. Two changes help. First, apply the Apriori rule: a pair can only reach the minimum support if each of its categories does, so remove infrequent categories before the self join. Second, count each unordered pair once with `<`, then emit both directions with a two-row `VALUES` list:

```
WITH params AS (
    SELECT 2.0 AS min_support_pct
),
baskets AS (
    SELECT DISTINCT od.OrderID,
           COALESCE(cat.CategoryName, 'Unmapped') AS category_name
    FROM OrderDetails  AS od
    JOIN Products      AS p   ON p.ProductID    = od.ProductID
    LEFT JOIN Category AS cat ON cat.CategoryID = p.CategoryID
),
basket_total AS (
    SELECT COUNT(DISTINCT b.OrderID) AS total_orders FROM baskets AS b
),
frequent_categories AS (
    SELECT b.category_name, COUNT(*) AS orders_with_category
    FROM baskets AS b
    CROSS JOIN basket_total AS bt
    CROSS JOIN params       AS pr
    GROUP BY b.category_name, bt.total_orders, pr.min_support_pct
    HAVING 100.0 * COUNT(*) / bt.total_orders >= pr.min_support_pct
),
frequent_baskets AS (
    SELECT b.OrderID, b.category_name
    FROM baskets AS b
    JOIN frequent_categories AS fc ON fc.category_name = b.category_name
),
unordered_pairs AS (
    SELECT x.category_name AS cat_1, y.category_name AS cat_2, COUNT(*) AS orders_with_both
    FROM frequent_baskets AS x
    JOIN frequent_baskets AS y
      ON y.OrderID       = x.OrderID
     AND y.category_name > x.category_name
    GROUP BY x.category_name, y.category_name
)
SELECT r.category_a,
       r.category_b,
       up.orders_with_both,
       ROUND(100.0 * up.orders_with_both / bt.total_orders, 2)          AS support_pct,
       ROUND(100.0 * up.orders_with_both / fa.orders_with_category, 2)  AS confidence_a_to_b_pct,
       ROUND(1.0 * up.orders_with_both * bt.total_orders
             / (fa.orders_with_category * fb.orders_with_category), 3)  AS lift
FROM unordered_pairs AS up
CROSS JOIN LATERAL (VALUES (up.cat_1, up.cat_2), (up.cat_2, up.cat_1)) AS r(category_a, category_b)
JOIN frequent_categories AS fa ON fa.category_name = r.category_a
JOIN frequent_categories AS fb ON fb.category_name = r.category_b
CROSS JOIN basket_total  AS bt
CROSS JOIN params        AS pr
WHERE 100.0 * up.orders_with_both / bt.total_orders >= pr.min_support_pct
ORDER BY lift DESC, confidence_a_to_b_pct DESC, r.category_a, r.category_b;
```

This version returns the same rows as Step 4 and does half the pair work. On real volumes, build `baskets` once as a table with an index on (`category_name`, `OrderID`), restrict it to a recent anchored window so rules reflect current behaviour, and refresh it on a schedule rather than recomputing from raw lines for every dashboard view. `CROSS JOIN LATERAL (VALUES ...)` is PostgreSQL; on MySQL use `UNION ALL` of the two directions instead.

Question 99: Flag unusual weeks in order volume

Difficulty: Expert

Primary Topic: Anomaly detection with trailing window statistics

Combined Topics: generate_series week spine, ROWS frame that excludes the current row, STDDEV_SAMP and AVG as window functions, Z-score with NULLIF

Business Scenario: The data platform team gets a Monday alert when last week's order count looks wrong. The alert has two jobs: catch a broken load (a week that is far too low because a feed failed) and catch a real event (a sale that tripled volume). The current rule, "alert if orders drop 30% week on week", fires every holiday and misses slow drifts. They want a rule that adapts to each period's normal level and spread.

Problem Statement: Build a weekly order-count series with no missing weeks. For each week, compute the mean and sample standard deviation of the previous eight weeks (not including the week itself), a z-score for the week, and a flag when the absolute z-score is 3 or more. Return only complete weeks. Sort flagged weeks first, then newest first.

Expected Output:

- `week_start`: Monday of the ISO week
- `orders`: orders placed that week (0 for a week with none)
- `trailing_mean`: mean weekly orders over the 8 previous weeks, 2 dp

- `trailing_stddev`: sample standard deviation over the same 8 weeks, 2 dp
- `z_score`: `(orders − trailing_mean) / trailing_stddev`, 2 dp; NULL until 8 prior weeks exist or when the spread is 0
- `is_anomaly`: `true` when `|z_score| >= 3`, NULL when there is no score

Tables Required: `Orders`

SQL Concepts Tested: building a complete date spine, aggregating to the spine with a LEFT JOIN, window frames that exclude the current row, sample vs population standard deviation, warm-up periods, threshold flags with NULL semantics.

Step 1 — Understand the Problem: Assumptions and definitions:

1. **Week.** ISO weeks, Monday to Sunday, from `DATE_TRUNC('week', ...)`. A business that reports Sunday to Saturday would shift the boundary; the logic does not change.
2. **Complete weeks only.** The first and last week in the data are usually partial: the extract starts and ends on whatever day it starts and ends. A partial last week always looks like a collapse, and that is the week the alert cares about most. So the series runs from the first Monday on or after `MIN(OrderDate)` to the last Monday whose Sunday is on or before `MAX(OrderDate)`. Both bounds come from the data, never `CURRENT_DATE`.
3. **Baseline.** The previous 8 complete weeks, excluding the current week. If the current week were inside its own baseline, a spike would inflate the mean and the spread it is being compared with, and the alert would mute itself.
4. **Spread.** Sample standard deviation (`STDDEV_SAMP`), because 8 weeks are a sample of the process, not the whole population.
5. **Threshold.** $|z| \geq 3$. With 8 points this is a rule of thumb, not a probability statement: weekly counts are not normal, and seasonality breaks the "same process" assumption. Tune it on known incidents.
6. **Warm-up.** Weeks with fewer than 8 prior weeks get no score. A z-score from 2 prior points is noise.
7. Every `Orders` row counts as a placed order. Orders with a NULL `OrderDate` cannot be placed in a week; they are reported separately in Step 7.

Step 2 — Identify the Tables and Relationships: Only `Orders` is needed: `OrderID` to count and `OrderDate` to bucket. There are no joins to other tables. The one join in the query is between `Orders` and a generated calendar, and it has to be a LEFT JOIN from the calendar so that weeks with zero orders exist as rows.

Step 3 — Build the Query Logic:

1. `bounds`: first and last complete week. `DATE_TRUNC('week', MIN(OrderDate) + 6)` rounds the first date up to a Monday (unless it already is one). `DATE_TRUNC('week', MAX(OrderDate) - 6)` gives the last Monday whose full week fits.
2. `week_spine`: one row per Monday between the bounds.
3. `week_counts`: orders per week from `Orders`, then a LEFT JOIN from the spine and `COALESCE(..., 0)`. Without the spine, a week with no orders would be missing, the window would reach back past it, and a dead feed would be invisible.
4. `baseline`: `AVG`, `STDDEV_SAMP` and `COUNT` over `ROWS BETWEEN 8 PRECEDING AND 1 PRECEDING`. `ROWS` counts physical rows, and the spine guarantees one row per week, so 8 rows means 8 weeks.
5. `scored`: z-score only when the baseline has all 8 weeks. `NULLIF(stddev, 0)` avoids dividing by zero when the 8 weeks were identical.

Step 4 — Final SQL Solution:

```sql
WITH bounds AS (
    SELECT DATE_TRUNC('week', MIN(o.OrderDate) + 6)::date AS first_full_week,
           DATE_TRUNC('week', MAX(o.OrderDate) - 6)::date AS last_full_week
    FROM Orders AS o
),
week_spine AS (
    SELECT gs::date AS week_start
    FROM bounds AS b
    CROSS JOIN generate_series(b.first_full_week, b.last_full_week, INTERVAL '7 days') AS gs
),
week_counts AS (
    SELECT DATE_TRUNC('week', o.OrderDate)::date AS week_start,
           COUNT(*)                              AS orders
    FROM Orders AS o
    WHERE o.OrderDate IS NOT NULL
    GROUP BY DATE_TRUNC('week', o.OrderDate)::date
),
weekly AS (
    SELECT ws.week_start,
           COALESCE(wc.orders, 0) AS orders
    FROM week_spine AS ws
    LEFT JOIN week_counts AS wc ON wc.week_start = ws.week_start
),
baseline AS (
    SELECT w.week_start,
           w.orders,
           COUNT(*)              OVER prior_8w AS baseline_weeks,
           AVG(w.orders)         OVER prior_8w AS trailing_mean,
           STDDEV_SAMP(w.orders) OVER prior_8w AS trailing_stddev
    FROM weekly AS w
    WINDOW prior_8w AS (ORDER BY w.week_start
                        ROWS BETWEEN 8 PRECEDING AND 1 PRECEDING)
),
scored AS (
    SELECT b.week_start,
           b.orders,
           b.trailing_mean,
           b.trailing_stddev,
           CASE WHEN b.baseline_weeks = 8
                THEN (b.orders - b.trailing_mean) / NULLIF(b.trailing_stddev, 0)
           END AS z_score
    FROM baseline AS b
)
SELECT s.week_start,
       s.orders,
       ROUND(s.trailing_mean, 2)   AS trailing_mean,
       ROUND(s.trailing_stddev, 2) AS trailing_stddev,
       ROUND(s.z_score, 2)         AS z_score,
       ABS(s.z_score) >= 3         AS is_anomaly
FROM scored AS s
ORDER BY is_anomaly DESC NULLS LAST, s.week_start DESC;
```

Step 5 — Line-by-Line Explanation:

- `+ 6` and `- 6` in `bounds` are the partial-week guards. Without them the first and last rows of the series are partial weeks.
- `generate_series(..., INTERVAL '7 days')` returns timestamps; `::date` brings them back to dates so they match `week_counts`.
- `week_counts` groups on the same `DATE_TRUNC('week', ...)` expression the spine uses, so the join keys line up exactly.
- `COALESCE(wc.orders, 0)` turns a missing week into a real zero. A zero week is data, and the most important data this alert sees.
- `WINDOW prior_8w AS (...)` names the frame once and reuses it for three functions. The frame ends at `1 PRECEDING`, which is what excludes the current week.
- `COUNT(*) OVER prior_8w` is how the query knows whether the window is full. Early weeks have fewer rows and get a NULL score.
- `AVG` and `STDDEV_SAMP` of an integer column return numeric in PostgreSQL, so `ROUND(x, 2)` works directly.
- `ABS(z) >= 3` returns NULL when `z` is NULL, so warm-up weeks are neither flagged nor cleared. `NULLS LAST` keeps them at the bottom of the sort.

Step 6 — Sample Output:

week_start	orders	trailing_mean	trailing_stddev	z_score	is_anomaly
2025-10-13	938	593.50	64.03	5.38	true
2025-10-06	714	581.50	44.12	3.00	true
2025-08-04	595	504.50	25.73	3.52	true
2025-07-28	555	493.88	18.40	3.32	true
2025-01-20	468	340.25	19.78	6.46	true
2024-10-14	435	255.25	34.14	5.26	true
2024-07-01	235	191.25	12.65	3.46	true
2024-05-27	202	179.38	7.37	3.07	true

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). Showing the first 8 of 156 rows.

Step 7 — Why This Solution Works: The statistic for each week is built only from earlier, complete, gap-free weeks, so a z-score means the same thing everywhere in the series. Before trusting the alert, prove the spine. It must hold one row per week between the bounds, and its total must equal a direct count over the same date range. Report orders with no date beside it, because they are outside every week:

```
WITH bounds AS (
    SELECT DATE_TRUNC('week', MIN(o.OrderDate) + 6)::date AS first_full_week,
           DATE_TRUNC('week', MAX(o.OrderDate) - 6)::date AS last_full_week
    FROM Orders AS o
),
week_spine AS (
    SELECT gs::date AS week_start
    FROM bounds AS b
    CROSS JOIN generate_series(b.first_full_week, b.last_full_week, INTERVAL '7 days') AS gs
),
weekly AS (
    SELECT ws.week_start, COUNT(o.OrderID) AS orders
    FROM week_spine AS ws
    LEFT JOIN Orders AS o
           ON o.OrderDate >= ws.week_start
          AND o.OrderDate <  ws.week_start + 7
    GROUP BY ws.week_start
)
SELECT (SELECT COUNT(*) FROM weekly)                                          AS weeks_in_spine,
       (SELECT (b.last_full_week - b.first_full_week) / 7 + 1 FROM bounds AS b) AS weeks_expected,
       (SELECT COUNT(*) FROM weekly AS w WHERE w.orders = 0)                   AS zero_order_weeks,
       (SELECT SUM(w.orders) FROM weekly AS w)                                 AS orders_in_spine,
       (SELECT COUNT(*) FROM Orders AS o CROSS JOIN bounds AS b
        WHERE o.OrderDate >= b.first_full_week
          AND o.OrderDate <  b.last_full_week + 7)                             AS orders_in_range,
       (SELECT COUNT(*) FROM Orders AS o WHERE o.OrderDate IS NULL)            AS orders_without_date;
```

Step 8 — Common Mistakes:

- Including the current week in its own window. With n points, the largest possible $|z|$ of one of them against their own mean and sample standard deviation is $(n - 1) / \sqrt{n}$. For 8 weeks that is about 2.47, so a threshold of 3 can never fire. This version shows the ceiling directly:

```
WITH weekly AS (
    SELECT gs::date AS week_start,
           (SELECT COUNT(*) FROM Orders AS o
            WHERE o.OrderDate >= gs::date AND o.OrderDate < gs::date + 7) AS orders
    FROM generate_series((SELECT DATE_TRUNC('week', MIN(OrderDate) + 6) FROM Orders),
                         (SELECT DATE_TRUNC('week', MAX(OrderDate) - 6) FROM Orders),
                         INTERVAL '7 days') AS gs
),
inclusive AS (
    SELECT w.orders,
           COUNT(*) OVER win AS window_weeks,
           (w.orders - AVG(w.orders) OVER win)
               / NULLIF(STDDEV_SAMP(w.orders) OVER win, 0) AS z_score_wrong
    FROM weekly AS w
    WINDOW win AS (ORDER BY w.week_start ROWS BETWEEN 7 PRECEDING AND CURRENT ROW)
)
SELECT ROUND(MAX(ABS(i.z_score_wrong)), 2)               AS largest_abs_z,
       ROUND(7 / SQRT(8.0), 2)                           AS ceiling_for_8_weeks,
       COUNT(*) FILTER (WHERE ABS(i.z_score_wrong) >= 3) AS weeks_flagged
FROM inclusive AS i
WHERE i.window_weeks = 8;
```

- Grouping `Orders` by week with no spine. Zero weeks vanish, `8 PRECEDING` silently spans more than 8 calendar weeks, and a complete outage produces no row to alert on.
- Using `RANGE` or the default frame. With `ORDER BY` and no frame, the window runs from the first row to the current one, so the baseline keeps growing and includes the current week.
- Scoring the partial last week. It is always low, so it alerts every Monday.
- Using population standard deviation (`STDDEV_POP`) on 8 points. It understates spread and raises the false-alarm rate.

Step 9 — Interview Follow-Up: Diwali week is high every year, and this rule flags it every year. How would you change the baseline so that a festive week is compared with what is normal for that time of year, and what is the minimum history you would need?

Step 10 — Optimization or Alternative: Mean and standard deviation are pulled around by the outliers they are meant to catch: one spike in the baseline raises the spread and hides the next spike for 8 weeks. An outlier-resistant alternative uses the trailing median and the median absolute deviation (MAD). `1.4826 × MAD` estimates the standard deviation when the data are roughly normal, so the same threshold of 3 applies. PostgreSQL does not allow `PERCENTILE_CONT` as a window function, so the trailing median is computed with `LATERAL` subqueries:

```sql
WITH bounds AS (
    SELECT DATE_TRUNC('week', MIN(o.OrderDate) + 6)::date AS first_full_week,
           DATE_TRUNC('week', MAX(o.OrderDate) - 6)::date AS last_full_week
    FROM Orders AS o
),
weekly AS (
    SELECT ws.week_start, COUNT(o.OrderID) AS orders
    FROM (SELECT gs::date AS week_start
          FROM bounds AS b
          CROSS JOIN generate_series(b.first_full_week, b.last_full_week, INTERVAL '7 days') AS gs) AS ws
    LEFT JOIN Orders AS o
           ON o.OrderDate >= ws.week_start
          AND o.OrderDate <  ws.week_start + 7
    GROUP BY ws.week_start
),
baseline AS (
    SELECT w.week_start, w.orders, med.median_orders, mad.mad_orders
    FROM weekly AS w
    CROSS JOIN LATERAL (
        SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY p.orders) AS median_orders,
               COUNT(*)                                              AS baseline_weeks
        FROM weekly AS p
        WHERE p.week_start >= w.week_start - 56
          AND p.week_start <  w.week_start
    ) AS med
    CROSS JOIN LATERAL (
        SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ABS(p.orders - med.median_orders)) AS mad_orders
        FROM weekly AS p
        WHERE p.week_start >= w.week_start - 56
          AND p.week_start <  w.week_start
    ) AS mad
    WHERE med.baseline_weeks = 8
)
SELECT b.week_start,
       b.orders,
       b.median_orders,
       b.mad_orders,
       ROUND(((b.orders - b.median_orders) / NULLIF(1.4826 * b.mad_orders, 0))::numeric, 2) AS median_based_z
FROM baseline AS b
WHERE b.mad_orders > 0
  AND ABS(b.orders - b.median_orders) >= 3 * 1.4826 * b.mad_orders
ORDER BY b.week_start DESC;
```

On low counts the MAD can be 0 or tiny, which makes this score jumpy, so also require a minimum absolute change in orders. Performance: the weekly series is tiny, so the cost is one scan of `Orders` to bucket dates. In production, keep a daily counts table, append to it each load, and score only recent months. On MySQL 8 the window functions and the named `WINDOW` carry over; `generate_series` becomes a recursive CTE calendar, `DATE_TRUNC('week', d)` becomes `DATE_SUB(d, INTERVAL WEEKDAY(d) DAY)`, `::date` becomes `CAST(... AS DATE)`, `INTERVAL '7 days'` becomes `INTERVAL 7 DAY`, and `NULLS LAST` needs an extra `is_anomaly IS NULL` sort key.

Question 100: Monthly KPI scorecard with month-over-month change

Difficulty: Expert

Primary Topic: Multi-stage KPI calculation (capstone)

Combined Topics: Multiple CTEs, Partial-period handling with like-for-like windows, Conditional aggregation with FILTER, Unpivot with CROSS JOIN LATERAL (VALUES ...), Fan-out control, Percentage-point vs percent change

Business Scenario: Every month the leadership team gets one table: eight KPIs for the latest month next to the month before, with the change. The data extract rarely ends on a month end. This one ends on 28 December, so December is a partial month, and comparing 28 days of December with 30 days of November would show a fall that is only the calendar. The analytics lead wants the comparison to be fair and the choice visible on the page.

Problem Statement: Anchor on the latest `OrderDate`. Compare the latest month from day 1 to the anchor day against the same day range of the previous month (capped at that month's last day). For each period compute: orders, order value (`Total_Order_Amount`), average order value, active customers, new customers, repeat-customer order share, on-time delivery rate, and units. Return one row per KPI with both values, the absolute change, the percent change (counts and money only), and the two date windows. Keep the KPI order fixed.

Expected Output:

- `kpi`, `unit` (`count`, `money` or `%`)
- `prior_value`, `latest_value`
- `change`: `latest_value - prior_value` (percentage points for `%` rows)
- `change_pct`: percent change for count and money rows; NULL for `%` rows
- `prior_window`, `latest_window`: the exact dates compared

KPI definitions:

- **Orders:** rows in `Orders` dated in the window. Cancellations and returns are not modelled.
- **Order value:** `SUM(Total_Order_Amount)`, the net amount payable after the payment-linked discount.
- **Average order value:** `AVG(Total_Order_Amount)` over the window's orders.
- **Active customers:** distinct customers with an order in the window.

- **New customers:** active customers whose first-ever order falls in the window.
- **Repeat-customer order share:** share of the window's orders placed by customers whose first order was before the window started.
- **On-time delivery:** share of delivered orders with `DeliveryDate - OrderDate <= 7` (the house one-week promise). A delivery dated before its order is a data error (Q88 rule 4) and is left out of both counts.
- **Units:** `SUM(Quantity)` over the window's order lines.

Tables Required: `Orders`, `OrderDetails`

SQL Concepts Tested: anchoring to the data, building comparison windows in a CTE, first-order dates with MIN per group, KPI definitions as conditional aggregates, avoiding fan-out when one KPI needs line-level data, unpivoting a wide row into a long table, the difference between percentage points and percent change.

Step 1 — Understand the Problem: Assumptions, stated up front:

- The anchor is `MAX(OrderDate)`, not `CURRENT_DATE`, so the scorecard is reproducible on any extract.
- The partial month is handled with like-for-like windows: day 1 to the anchor day in both months. If the anchor is the 31st and the prior month has 30 days, the prior window stops at its own month end. The alternative, reporting only the last complete month, is simpler but always a month stale; showing the windows in the output makes either choice auditable.
- "New" is defined by first-ever order date, not by `DateEntered`, because a sign-up without a purchase is not a customer for this table.
- On-time is measured on orders that have a delivery date. The most recent orders in the extract are still in transit, so the latest window's on-time rate covers the orders delivered so far and can still move as late deliveries land. Read it as provisional.

The structure follows the grains. Seven KPIs live at the order grain; units live at the line grain. Computing units in the same aggregation as order counts would multiply every order by its line count, so units get their own CTE.

Step 2 — Identify the Tables and Relationships: `Orders` supplies dates, customers, amounts and delivery dates. `OrderDetails.OrderID` → `Orders.OrderID` (one to many) supplies `Quantity`. No dimension table is needed: every KPI is defined on the fact tables.

Step 3 — Build the Query Logic:

1. `anchor`: the latest order date.
2. `windows`: two rows, `latest` and `prior`, each with a start and end day.
3. `first_orders`: each customer's first-ever order date (over all history, not just the windows).
4. `period_orders`: orders falling inside each window, tagged with the window and the customer's first order date.
5. `period_units`: units per window, aggregated from lines separately.
6. `kpis`: one row per window with all eight measures.
7. `labels`: the two windows as text.
8. Final: put the prior and latest rows side by side with a self join, then unpivot with `CROSS JOIN LATERAL (VALUES ...)` into one row per KPI.

Step 4 — Final SQL Solution:

```sql
WITH anchor AS (
    SELECT MAX(o.OrderDate) AS last_day
    FROM Orders AS o
),
windows AS (
    SELECT 'latest'                              AS period,
           DATE_TRUNC('month', a.last_day)::date  AS start_day,
           a.last_day                             AS end_day
    FROM anchor AS a
    UNION ALL
    SELECT 'prior',
           (DATE_TRUNC('month', a.last_day) - INTERVAL '1 month')::date,
           LEAST((DATE_TRUNC('month', a.last_day) - INTERVAL '1 month')::date
                 + (EXTRACT(DAY FROM a.last_day)::int - 1),
                 (DATE_TRUNC('month', a.last_day) - INTERVAL '1 day')::date)
    FROM anchor AS a
),
first_orders AS (
    SELECT o.CustomerID, MIN(o.OrderDate) AS first_order_date
    FROM Orders AS o
    GROUP BY o.CustomerID
),
period_orders AS (
    SELECT w.period, w.start_day, o.OrderID, o.CustomerID, o.OrderDate,
           o.DeliveryDate, o.Total_Order_Amount, fo.first_order_date
    FROM windows AS w
    JOIN Orders AS o        ON o.OrderDate BETWEEN w.start_day AND w.end_day
    LEFT JOIN first_orders AS fo ON fo.CustomerID = o.CustomerID
),
period_units AS (
    SELECT po.period, SUM(od.Quantity) AS units
    FROM period_orders AS po
    JOIN OrderDetails AS od ON od.OrderID = po.OrderID
    GROUP BY po.period
),
kpis AS (
    SELECT po.period,
           COUNT(*)::numeric                                                          AS orders,
           SUM(po.Total_Order_Amount)                                                 AS order_value,
           ROUND(AVG(po.Total_Order_Amount), 2)                                       AS aov,
           COUNT(DISTINCT po.CustomerID)::numeric                                     AS active_customers,
           COUNT(DISTINCT po.CustomerID) FILTER (WHERE po.first_order_date >= po.start_day)::numeric AS new_customers,
           ROUND(100.0 * COUNT(*) FILTER (WHERE po.first_order_date < po.start_day)
                 / NULLIF(COUNT(*), 0), 2)                                            AS repeat_order_share_pct,
           ROUND(100.0 * COUNT(*) FILTER (WHERE po.DeliveryDate >= po.OrderDate
                                          AND po.DeliveryDate - po.OrderDate <= 7)
                 / NULLIF(COUNT(*) FILTER (WHERE po.DeliveryDate >= po.OrderDate), 0), 2) AS on_time_pct,
           MAX(pu.units)::numeric                                                     AS units
    FROM period_orders AS po
    JOIN period_units AS pu ON pu.period = po.period
    GROUP BY po.period
),
labels AS (
    SELECT MAX(w.start_day || ' to ' || w.end_day) FILTER (WHERE w.period = 'prior')  AS prior_window,
           MAX(w.start_day || ' to ' || w.end_day) FILTER (WHERE w.period = 'latest') AS latest_window
    FROM windows AS w
)
SELECT v.kpi,
       v.unit,
       v.prior_value,
       v.latest_value,
       v.latest_value - v.prior_value                                                 AS change,
       CASE WHEN v.unit = '%' THEN NULL
            ELSE ROUND(100.0 * (v.latest_value - v.prior_value) / NULLIF(v.prior_value, 0), 2)
       END                                                                            AS change_pct,
       lb.prior_window,
       lb.latest_window
FROM kpis AS p
JOIN kpis AS l ON p.period = 'prior' AND l.period = 'latest'
CROSS JOIN labels AS lb
CROSS JOIN LATERAL (
    VALUES (1, 'Orders',                   'count', p.orders,                 l.orders),
           (2, 'Order value',              'money', p.order_value,            l.order_value),
           (3, 'Average order value',      'money', p.aov,                    l.aov),
           (4, 'Active customers',         'count', p.active_customers,       l.active_customers),
           (5, 'New customers',            'count', p.new_customers,          l.new_customers),
           (6, 'Repeat-customer order share','%',   p.repeat_order_share_pct, l.repeat_order_share_pct),
           (7, 'On-time delivery (<= 7 days)', '%', p.on_time_pct,            l.on_time_pct),
           (8, 'Units',                    'count', p.units,                  l.units)
) AS v(sort_order, kpi, unit, prior_value, latest_value)
ORDER BY v.sort_order;
```

Step 5 — Line-by-Line Explanation:

- `windows` builds the comparison in data, not in WHERE clauses scattered through the query. `DATE_TRUNC('month', ...)` finds the month start; adding `EXTRACT(DAY FROM last_day) - 1` days to the prior month start gives the same day number, and `LEAST(..., prior month end)` caps it for short months.
- `first_orders` is computed over all orders. Restricting it to the windows would make every customer "new" in the first window they appear in.
- `period_orders` joins orders to windows with `BETWEEN start_day AND end_day`. The windows do not overlap, so each order lands in at most one period.
- `period_units` aggregates `OrderDetails` to one row per period before it meets `kpis`. The join in `kpis` is then one row per period on each side, and `MAX(pu.units)` simply carries that single value through the GROUP BY.

- `COUNT(DISTINCT po.CustomerID) FILTER (WHERE first_order_date >= start_day)` counts new customers; the same FILTER inverted on `COUNT(*)` gives orders from established customers.
- The on-time denominator, `COUNT(*) FILTER (WHERE po.DeliveryDate >= po.OrderDate)`, counts only delivered orders with a possible date: undelivered orders fail on NULL, and a delivery dated before its order would otherwise count as on time. `period_orders` LEFT JOINs `first_orders`, so an order with a NULL `CustomerID` still counts in Orders and Order value.
- `JOIN kpis AS l ON p.period = 'prior' AND l.period = 'latest'` turns two rows into one wide row with both periods.
- `CROSS JOIN LATERAL (VALUES ...)` unpivots: each tuple names a KPI and picks its prior and latest value from that wide row. `sort_order` fixes the presentation order.
- The final CASE leaves `change_pct` NULL for `%` rows. A move from 80% to 76% is a change of 4 percentage points; calling it "-5%" invites misreading.

Step 6 — Sample Output:

kpi	unit	prior_value	latest_value	change	change_pct	prior_window	latest_window
Orders	count	3524	2930	-594	-16.86	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
Order value	money	4541404.63	3871916.06	-669488.57	-14.74	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
Average order value	money	1288.71	1321.47	32.76	2.54	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
Active customers	count	1733	1615	-118	-6.81	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
New customers	count	376	260	-116	-30.85	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
Repeat-customer order share	%	83.12	86.86	3.74	NULL	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
On-time delivery (<= 7 days)	%	85.09	90.67	5.58	NULL	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28
Units	count	24222	19318	-4904	-20.25	2025-11-01 to 2025-11-28	2025-12-01 to 2025-12-28

Actual output from running the query above on the PrepNPlaced Commerce Dataset v2 (loaded with load_dataset_v2.sql). 8 rows returned.

Step 7 — Why This Solution Works: Each KPI is defined once, in one aggregate, over one clearly bounded set of orders, and the windows are part of the result, so a reader can see exactly what was compared. Like-for-like day ranges remove the calendar effect of a partial month: with 28 days on each side, a change in orders is a change in demand, not in days. Units cannot be inflated by the line join because they are summed before they meet the order-level aggregates.

Validation before publishing:

- Orders and order value for each window must match a direct `COUNT(*)` and `SUM(Total_Order_Amount)` over `Orders` with the same date range.
- New customers must be at most active customers, and repeat-customer order share must be between 0 and 100.
- Units must match `SUM(Quantity)` over lines joined to orders in the window.

Read the results with the data in mind. A festive month brings a wave of first-time buyers, and new customers often place several orders in their first weeks. A drop in repeat-customer share in such a month does not have to mean established customers bought less. Look at active customers and new-customer orders together before writing the commentary.

Step 8 — Common Mistakes:

- Comparing the partial latest month with the full previous month. The latest month is short by however many days the extract misses, so its growth is understated, or a fall appears that is only the calendar. Count the days on each side before comparing:

```sql
SELECT DATE_TRUNC('month', o.OrderDate)::date AS order_month,
       COUNT(*)                               AS orders,
       COUNT(DISTINCT o.OrderDate)            AS days_with_orders
FROM Orders AS o
WHERE o.OrderDate >= (SELECT DATE_TRUNC('month', MAX(OrderDate)) - INTERVAL '1 month' FROM Orders)
GROUP BY 1
ORDER BY 1;
```

- Computing units in the same query as orders by joining `OrderDetails` into `period_orders`. `COUNT(*)` then counts lines, `SUM(Total_Order_Amount)` repeats each order once per line, and AOV becomes meaningless.
- Defining "new" from first order inside the window only. Every returning customer looks new in their first window of the report.
- Reporting percent change for percentage KPIs. Use percentage points.
- Using `CURRENT_DATE` for the anchor, which produces two empty periods on any historical extract.

Step 9 — Interview Follow-Up: Leadership wants the same scorecard for the latest complete month instead (November against October here). Which single CTE changes, and how would you let a dashboard switch between the two modes without duplicating the query?

Step 10 — Optimization or Alternative: For a scheduled report, materialise a daily fact table, one row per date with orders, value, active customers and units, and compute any window from it. Daily distinct customers cannot be summed across days, so keep a customer-date table (or a HyperLogLog sketch on very large data) for the distinct counts. On the raw tables, an index on `Orders(OrderDate)` lets `period_orders` read only the two windows instead of the whole history, and `OrderDetails(OrderID)` serves `period_units`. `first_orders` scans all history every run; in production keep it as an incrementally maintained table keyed by `CustomerID`, updated only when a customer places their first order. MySQL 8 has no `FILTER` and no `LATERAL (VALUES ...)`: use `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` and an unpivot built from `UNION ALL` of one SELECT per KPI.

Deliverable D: Topic coverage matrix

Each topic from the brief, with the questions that exercise it. 83 of 84 topics are covered; 85 of 100 questions combine two or more SQL concepts.

A. SQL fundamentals

Topic	Questions	Count
SELECT, DISTINCT	Q1, Q98	2
WHERE and filtering	Q1, Q3, Q4, Q5, Q6, Q23, Q24, Q32, Q46	9
AND, OR, NOT	Q4, Q23, Q24	3
IN, BETWEEN, LIKE	Q3, Q4, Q5, Q23	4
ORDER BY	Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q8, Q12, Q13, Q14, Q16, Q17, Q18, Q19, Q20, Q21, Q23, Q24, Q25, Q32, Q42, Q43, Q46, Q57, Q58	26
LIMIT and equivalents	Q2, Q6, Q21	3
NULL handling	Q1, Q2, Q4, Q6, Q7, Q8, Q9, Q10, Q11, Q12, Q13, Q16, Q17, Q18, Q20, Q21, Q24, Q25, Q27, Q28, Q29, Q32, Q33, Q35, Q36, Q37, Q38, Q39, Q41, Q46, Q47, Q48, Q52, Q53, Q54, Q57, Q61, Q63, Q64, Q67, Q68, Q69, Q71, Q74, Q78, Q79, Q83, Q85, Q87, Q88, Q89, Q93, Q96, Q99	54
CASE WHEN	Q4, Q8, Q24, Q30, Q35, Q37, Q39, Q40, Q48, Q51, Q62, Q63, Q72, Q84, Q86, Q89, Q93, Q94	18
Type conversion	Q6, Q9, Q20, Q25, Q38, Q50, Q57, Q67, Q90	9
Date filtering	Q5, Q14, Q16, Q17, Q18, Q20, Q36, Q38, Q43, Q55, Q56, Q75, Q81, Q83, Q85, Q89, Q95, Q96	18

B. Aggregations

Topic	Questions	Count
COUNT, SUM, AVG, MIN, MAX	Q9, Q10, Q11, Q12, Q13, Q14, Q15, Q19, Q21, Q22, Q25, Q27, Q29, Q31, Q32, Q37, Q40, Q44, Q45, Q48, Q52, Q54, Q59, Q64	24
GROUP BY	Q9, Q12, Q13, Q14, Q15, Q19, Q21, Q22, Q25, Q26, Q27, Q28, Q29, Q30, Q31, Q34, Q36, Q37, Q38, Q39, Q40, Q42, Q43, Q44, Q45, Q47, Q48, Q49, Q50, Q51, Q53, Q54, Q56, Q59, Q60, Q67, Q70, Q83, Q84, Q96	40
HAVING	Q13, Q22, Q27, Q45, Q47, Q50, Q69, Q72, Q80	9
Conditional aggregation	Q12, Q35, Q36, Q52, Q53, Q55, Q56, Q61, Q68, Q73, Q75, Q76, Q81, Q86, Q87, Q90, Q91, Q92, Q97, Q100	20
Distinct customer and order counts	Q10, Q11, Q19, Q22, Q26, Q27, Q39, Q47, Q50, Q51, Q59, Q61, Q72, Q75, Q77, Q90, Q95, Q98, Q100	19
Revenue and quantity calculations	Q19, Q22, Q26, Q29, Q34, Q47, Q48, Q54, Q59, Q60	10
Percentage calculations	Q7, Q11, Q28, Q30, Q31, Q34, Q35, Q44, Q51, Q53, Q54, Q55, Q60, Q61, Q62, Q66, Q73, Q74, Q80, Q81, Q90, Q92, Q97, Q98, Q100	25

C. Joins

Topic	Questions	Count
INNER JOIN	Q16, Q19, Q38, Q40, Q50, Q62, Q63, Q70	8
LEFT JOIN	Q11, Q15, Q17, Q18, Q21, Q26, Q27, Q28, Q29, Q31, Q33, Q35, Q36, Q37, Q39, Q41, Q47, Q48, Q49, Q52, Q53, Q54, Q55, Q56, Q58, Q59, Q60, Q64, Q67, Q68, Q73, Q75, Q77, Q78, Q80, Q81, Q82, Q83, Q84, Q86, Q88, Q89, Q91, Q95, Q96, Q99	46
Self join	Q46, Q70, Q71, Q93, Q98	5
Multi-table joins	Q17, Q26, Q29, Q38, Q39, Q49, Q50, Q55, Q60, Q73, Q77, Q80, Q82, Q83, Q84, Q94, Q98	17
Finding unmatched records	Q11, Q15, Q16, Q17, Q18, Q26, Q33, Q34, Q39, Q86, Q87, Q95	12
One-to-many relationship handling	Q11, Q15, Q19, Q22, Q29, Q47, Q48, Q50, Q51, Q59, Q86, Q93, Q95, Q96, Q100	15
Avoiding duplicate counts and inflated sums	Q29, Q59, Q86, Q95, Q100	5

D. Subqueries and CTEs

Topic	Questions	Count
Scalar subquery	Q17, Q18, Q28, Q31	4
Correlated subquery	Q32, Q33, Q80, Q96	4
Nested subquery	Q31	1
Common Table Expression	Q29, Q30, Q31, Q35, Q37, Q38, Q40, Q41, Q42, Q44, Q45, Q48, Q49, Q51, Q52, Q53, Q56, Q58, Q59, Q62, Q70, Q84, Q90	23
Multiple CTEs	Q43, Q54, Q55, Q61, Q63, Q64, Q65, Q66, Q68, Q69, Q72, Q73, Q74, Q75, Q76, Q77, Q78, Q79, Q80, Q81, Q82, Q83, Q85, Q86, Q88, Q89, Q91, Q93, Q94, Q95, Q97, Q98, Q99, Q100	34

Recursive CTE	Q64	1

E. Window functions

Topic	Questions	Count
ROW_NUMBER	Q41, Q42, Q58, Q65, Q66, Q67, Q71, Q72, Q74, Q79, Q82, Q94, Q97	13
RANK and DENSE_RANK	Q41, Q42, Q49, Q65, Q68, Q70	6
LAG and LEAD	Q44, Q45, Q66, Q69, Q85	5
Running totals	Q43, Q62, Q79, Q91	4
Moving averages	Q64, Q99	2
PARTITION BY	Q32, Q41, Q43, Q45, Q49, Q58, Q60, Q63, Q65, Q68, Q69, Q71, Q72, Q73, Q74, Q76, Q78, Q79, Q82, Q85, Q91, Q92, Q94, Q97	24
Top N per group	Q41, Q49, Q65, Q68, Q70, Q82	6
First and latest record per customer	Q40, Q53, Q58, Q66, Q74, Q79, Q97, Q100	8
Ranking products within categories	Q41, Q49, Q78	3
Comparing current and previous periods	Q44, Q55, Q69, Q73, Q81, Q100	6

F. Advanced SQL

Topic	Questions	Count
EXISTS and NOT EXISTS	Q33, Q56, Q80, Q87, Q89, Q93	6
Set operations	Q9, Q56, Q57, Q87, Q88, Q92, Q96	7
Conditional ranking	Q68	1
Deduplication	Q58, Q70, Q71, Q72, Q92	5
Gaps and islands	Q65	1
Date-based analytics	Q14, Q20, Q25, Q36, Q38, Q40, Q43, Q44, Q45, Q52, Q53, Q55, Q56, Q59, Q61, Q63, Q64, Q65, Q69, Q73, Q75, Q76, Q79, Q81, Q83, Q85, Q88, Q91, Q99, Q100	30
Rolling calculations	Q64, Q74, Q75, Q99	4
Cohort analysis	Q61, Q91	2
Customer retention	Q18, Q56, Q61, Q65, Q66, Q75, Q76, Q80, Q85	9
Repeat purchase behaviour	Q30, Q45, Q66, Q76, Q80, Q85, Q97, Q100	8
Month-over-month growth	Q44, Q100	2
Pareto analysis	Q62	1
Contribution percentages	Q28, Q34, Q51, Q60, Q62, Q73, Q76, Q77, Q82, Q94, Q98	11
Median and percentiles	Q66, Q67, Q78, Q83	4

G. Business and product analytics

Topic	Questions	Count
Revenue by product, category, supplier, customer	Q26, Q27, Q47, Q55, Q60, Q62	6
Average order value	Q10, Q27, Q67, Q74, Q100	5
Best-selling products	Q19, Q49, Q70	3
Underperforming products	Q33, Q78, Q83	3
Customer segmentation	Q30, Q31, Q42, Q50, Q63, Q82	6
Repeat versus one-time customers	Q30, Q92	2
Customer lifetime value proxies	Q31, Q63, Q79, Q91	4
Discount analysis (Market_Price vs Sale_Price)	Q6, Q7, Q12, Q48, Q54	5
Payment method adoption	Q28, Q36, Q37, Q67, Q86	5
Delivery performance	Q20, Q29, Q35, Q52, Q68, Q84, Q100	7
Shipping delays	Q20, Q35, Q52, Q84	4
Supplier contribution	Q34, Q51, Q77, Q94	4
Product ratings	Not covered (see audit)	0
Inactive categories	Q9, Q38, Q39	3
Customer acquisition trends	Q25, Q40, Q53, Q76, Q100	5

Sales growth	Q44, Q55, Q69, Q81, Q100	5
Revenue concentration	Q62, Q77, Q94	3
Product and category performance	Q15, Q26, Q39, Q55, Q60, Q73, Q78, Q83, Q95, Q97, Q98	11

H. Data engineering SQL

Topic	Questions	Count
Data quality checks	Q3, Q4, Q7, Q9, Q24, Q37, Q54, Q71, Q72, Q86, Q87, Q88, Q90, Q93, Q99	15
Duplicate detection	Q22, Q71, Q72, Q92, Q93	5
NULL and missing-value analysis	Q90	1
Referential integrity validation	Q87	1
Order total vs order-detail reconciliation	Q86	1
Orphan records	Q34, Q77, Q86, Q87	4
Invalid order or delivery dates	Q35, Q40, Q52, Q84, Q88	5
Incremental extraction by date	Q89	1
Deduplication with window functions	Q71, Q72, Q92	3
ETL validation queries	Q37, Q86, Q88, Q89, Q90, Q99	6

X. Interview skills

Topic	Questions	Count
Query performance and indexing	Q80, Q96	2
Debugging an incorrect query	Q95	1

Questions that combine two or more SQL concepts

85 questions: Q4, Q6, Q9, Q11, Q12, Q15, Q17, Q18, Q19, Q21, Q26, Q27, Q28, Q29, Q30, Q31, Q32, Q33, Q34, Q35, Q36, Q37, Q38, Q39, Q40, Q41, Q42, Q43, Q44, Q45, Q46, Q47, Q48, Q49, Q50, Q51, Q52, Q53, Q54, Q55, Q56, Q57, Q58, Q59, Q60, Q61, Q62, Q63, Q64, Q65, Q66, Q67, Q68, Q69, Q70, Q71, Q72, Q73, Q74, Q75, Q76, Q77, Q78, Q79, Q80, Q81, Q82, Q83, Q84, Q85, Q86, Q87, Q88, Q89, Q90, Q91, Q92, Q93, Q94, Q95, Q96, Q97, Q98, Q99, Q100.

Deliverable E: Final quality audit

Each requirement from the brief, checked against the finished files.

Requirement	Result	Evidence
Exactly 100 main questions	Pass	Question IDs run 1 to 100 with no gaps. Follow-ups in Step 9 are not counted.
Difficulty distribution 25 / 35 / 25 / 15	Pass	Easy 25 (Q1-25), Medium 35 (Q26-60), Hard 25 (Q61-85), Expert 15 (Q86-100).
Required SQL topics covered where the schema supports them	Pass, with 1 documented exception	83 of 84 topics covered (Deliverable D). Not covered: Product ratings. `Products.Rating` exists in v2 but no question is built around it; the README suggests rating exercises. Recursive CTEs appear only as a calendar alternative (Q64) because no table is hierarchical.
At least 40 questions combine two or more concepts	Pass	85 questions are marked combined and list their secondary concepts.
No nonexistent tables or columns	Pass	All 266 SQL blocks in the bank (final solutions, mistakes and alternatives) were executed on the loaded dataset without error. A column the data lacks fails execution.
SQL matches the chosen dialect	Pass	PostgreSQL 16.15. Dialect-specific syntax is listed with MySQL equivalents in Deliverable A.
Queries correct for the stated output grain	Pass, checked by execution and review	Every Step 6 output was read for grain, duplicates and sensible values. Control totals tie: shipper order value sums to the ledger total (Q29); catalogue counts reconcile (Q11); reconciliation statuses sum to all orders (Q86). No separate independent reviewer was used.
No fabricated query results	Pass	Every Step 6 table is generated by the runner from live execution on the PrepNPlaced Commerce Dataset v2 and labelled as actual output. Prose states a dataset fact only where a query verified it.
No two questions test the same primary business objective	Pass	Objectives were fixed in a master spec before writing; all 100 titles are distinct. Where two questions share a technique (for example ranking in Q41, Q42, Q49), the business question differs.
Every question has an explanation and a defined expected output	Pass	The validator confirms all 18 required labels appear once, in order, in every question, including Expected Output and Steps 1 to 10.
Explanation depth proportional to difficulty	Pass	Words outside code: Easy 586-985, Medium 837-1171, Hard 1099-1368, Expert 1368-1805.

Known limits of this audit. Correctness was checked by running every query on the v2 dataset and reading every result, not by a second reviewer. The bank was first written for the original Customer Dataset and then moved to v2: years, the late-delivery rule (now more than 7 days) and every sentence that quotes a dataset fact were re-checked against the new results. The v2 transactions are generated, so results describe this teaching dataset, not a real company; Deliverable A says exactly what is real and what is generated. After the first release, a reader spotted that Q2 relied on `NULLS LAST` where its rule needed a `WHERE` filter. That prompted a second full review against the data: 26 questions were corrected, 6 of them in the final query (Q1, Q2, Q6, Q46, Q47, Q100), the rest in explanations, mistake examples and MySQL notes.